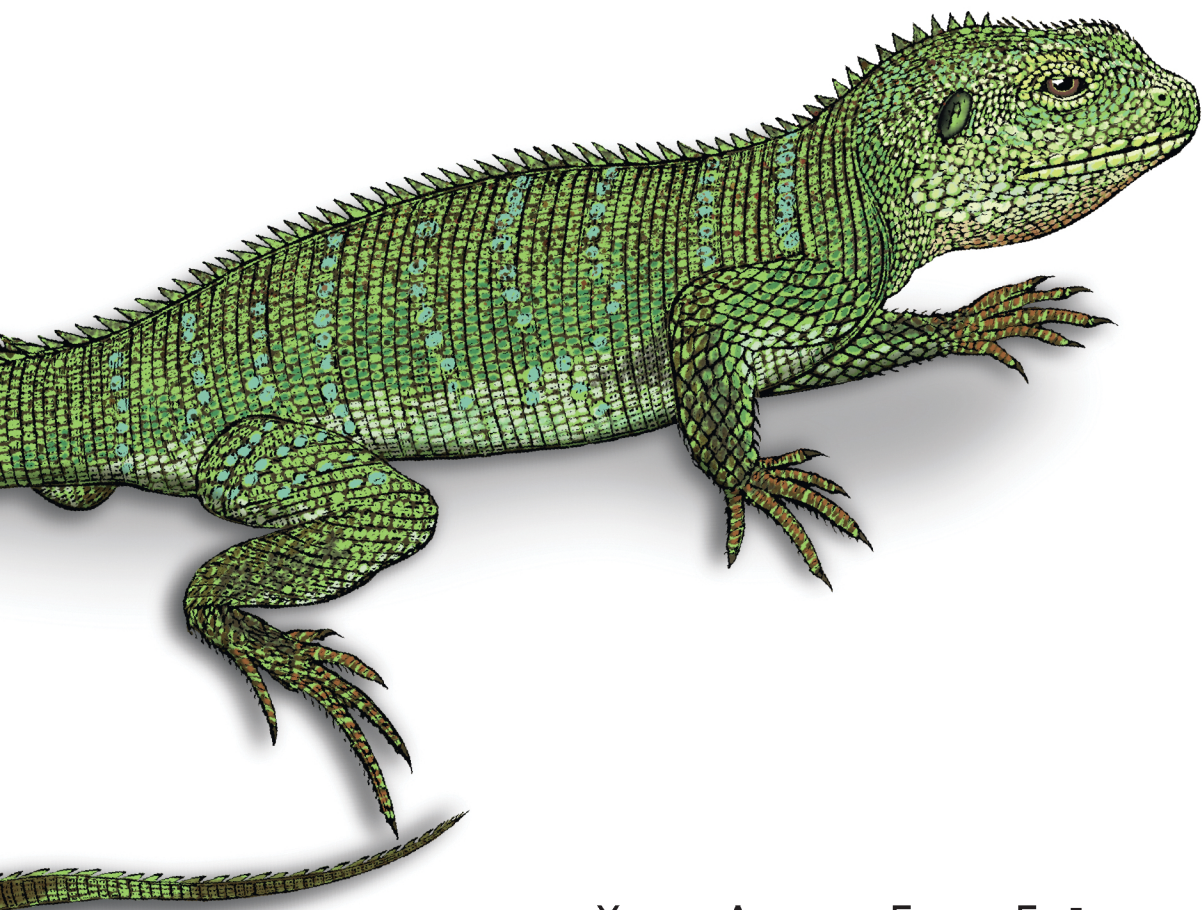


O'REILLY®

Безопасные и надежные системы

Лучшие практики проектирования,
внедрения и обслуживания как в Google



Хизер Адкинс, Бетси Бейер,
Пол Бланкиншип, Петр Левандовски,
Ана Опреа, Адам Стабблфилд



Building Secure and Reliable Systems

*Best Practices for Designing, Implementing,
and Maintaining Systems*

*Heather Adkins, Betsy Beyer, Paul Blankinship,
Piotr Lewandowski, Ana Oprea, and Adam Stubblefield*

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Безопасные и надежные СИСТЕМЫ

*Лучшие практики проектирования, внедрения
и обслуживания как в Google*

*Хизер Адкинс, Бетси Бейер, Пол Бланкиншип,
Петр Левандовски, Ана Опреа, Адам Стабблфилд*



Санкт-Петербург • Москва • Минск

2025

Выпущено
при поддержке

КРОК

ББК 32.988.02-018
УДК 004.738.5+004.775
Б40

**Хизер Адкинс, Бетси Бейер, Пол Бланкиншип,
Петр Левандовски, Ана Опра, Адам Стабблфилд**

Б40 Безопасные и надежные системы: Лучшие практики проектирования, внедрения и обслуживания как в Google. — СПб.: Питер, 2025. — 592 с.: ил. — (Серия «Библиотека специалиста»).

ISBN 978-5-4461-1770-3

Может ли быть надежной небезопасная система? Можно ли считать безопасной ненадежную систему? Безопасность имеет решающее значение для разработки и эксплуатации масштабируемых систем в реальных условиях, поскольку вносит важный вклад в качество, производительность и доступность продукта. В этой книге эксперты Google делятся передовым опытом, позволяющим разрабатывать масштабируемые и надежные системы, которые действительно будут безопасны.

Вам уже знакомы два бестселлера, написанные разработчиками из Google — «Site Reliability Engineering. Надежность и безотказность как в Google» и «Site Reliability Workbook: практическое применение», значит, вы понимаете, что только неуклонное следование жизненному циклу обслуживания позволяет успешно создавать, развертывать и поддерживать программные системы. Сейчас же мы предлагаем взглянуть на проектирование, реализацию и обслуживание систем с точки зрения практиков, специализирующихся на безопасности и надежности.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018
УДК 004.738.5+004.775

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги.

В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1492083122 англ.

Authorized Russian translation of the English edition of Building Secure and Reliable Systems ISBN 9781492083122 © 2020 Google LLC.
This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

ISBN 978-5-4461-1770-3

© Перевод на русский язык ООО «Прогресс книга», 2024

© Издание на русском языке, оформление ООО «Прогресс книга», 2024

© Серия «Библиотека специалиста», 2024

Краткое содержание

Отзывы о книге.....	19
Предисловие Рояла Хансена.....	22
Предисловие Майкла Вайлдпанера	25
Введение	27

ЧАСТЬ I. ВВОДНЫЕ МАТЕРИАЛЫ

Глава 1. Пересечение безопасности и надежности	38
Глава 2. Виды злоумышленников.....	51

ЧАСТЬ II. ПРОЕКТИРОВАНИЕ СИСТЕМ

Глава 3. Пример из практики: безопасные прокси.....	77
Глава 4. Компромиссные решения при проектировании	83
Глава 5. Принцип наименьших привилегий при проектировании.....	102
Глава 6. Проектирование для понятности.....	136
Глава 7. Проектирование для меняющегося ландшафта.....	172
Глава 8. Проектирование для устойчивости	197
Глава 9. Проектирование для восстановления	242
Глава 10. Защита от DoS-атак	281

ЧАСТЬ III. ВНЕДРЕНИЕ СИСТЕМ

Глава 11. Пример: проектирование, разработка и поддержка публично доверенного центра сертификации	296
Глава 12. Написание кода	306
Глава 13. Тестирование	335
Глава 14. Развертывание.....	370
Глава 15. Исследование систем.....	402

ЧАСТЬ IV. ОБСЛУЖИВАНИЕ СИСТЕМ

Глава 16. Планирование аварийных ситуаций.....	435
Глава 17. Антикризисное управление	461
Глава 18. Восстановление и последствия	494

ЧАСТЬ V. ОРГАНИЗАЦИЯ И КУЛЬТУРА

Глава 19. Пример из практики: команда безопасности Chrome	524
Глава 20. Понимание ролей и обязанностей.....	535
Глава 21. Формирование культуры безопасности и надежности	554
Заключение	583
Приложение. Матрица оценки риска бедствий.....	585
Об авторах	588
Иллюстрация на обложке.....	590

Оглавление

Отзывы о книге.....	19
Предисловие Рояла Хансена.....	22
Предисловие Майкла Вайлдпанера	25
Введение	27
Почему мы написали эту книгу	27
Для кого предназначена эта книга.....	28
Примечание о культуре.....	29
Как читать эту книгу	29
Условные обозначения, используемые в книге	30
Благодарности	31
От издательства.....	36
О научном редакторе русскоязычного издания	36

ЧАСТЬ I. ВВОДНЫЕ МАТЕРИАЛЫ

Глава 1. Пересечение безопасности и надежности	38
О паролях и перфораторах.....	38
Надежность и безопасность: соображения проектирования	40
Конфиденциальность, целостность, доступность	41
Конфиденциальность	41
Целостность	41
Доступность	42
Надежность и безопасность: общие черты	43
Незаметность.....	43
Оценка	44
Простота	44
Эволюция	44
Устойчивость	45
От проектирования до использования.....	47
Исследование систем и ведение журналов	47

Антикризисное реагирование	48
Восстановление.....	49
Резюме	50
Глава 2. Виды злоумышленников.....	51
Мотивация атакующего	52
Профили злоумышленников.....	54
Любители	54
Исследователи уязвимостей	55
Правительства и правоохранительные органы	55
Преступники	60
Автоматизация и искусственный интеллект	62
Инсайдеры	62
Методы атакующего	69
Анализ угроз	69
Cyber Kill Chains™	70
Тактика, методы и процедуры	71
Оценка риска.....	72
Резюме	73

ЧАСТЬ II. ПРОЕКТИРОВАНИЕ СИСТЕМ

Глава 3. Пример из практики: безопасные прокси.....	77
Безопасные прокси в производственных средах	77
Tool Proxy в Google.....	80
Резюме	82
Глава 4. Компромиссные решения при проектировании	83
Цели и требования при проектировании.....	84
Функциональные требования	84
Нефункциональные требования	85
Характеристики в сравнении с эмерджентными свойствами.....	86
Пример: документ о дизайне в Google	88
Балансировка требований	89
Пример: обработка платежей	90
Управление противоречиями и согласование целей	95
Пример: микросервисы и фреймворк веб-приложений Google.....	95
Согласование требований к эмерджентным свойствам	97

Начальная скорость по сравнению с установившейся скоростью.....	98
Резюме.....	101
Глава 5. Принцип наименьших привилегий при проектировании.....	102
Концепции и терминология	103
Наименьшая привилегия.....	103
Сеть с нулевым доверием.....	103
Zero Touch.....	104
Классификация доступа на основе риска	104
Лучшие практики.....	106
Небольшие функциональные API	106
Механизм аварийного доступа.....	109
Аудит.....	110
Тестирование и минимальные привилегии	113
Диагностика отказа в доступе	116
Механизмы постепенного отказа и аварийного доступа	117
Пример из практики: распределение конфигурации	118
POSIX API через OpenSSH.....	119
API обновления ПО	120
Пользовательский параметр ForceCommand для OpenSSH.....	120
Пользовательский HTTP-получатель (расширенный)	121
Пользовательский HTTP-получатель (внутрипроцессный)	121
Компромиссы	121
Структура политик для решений аутентификации и авторизации.....	122
Использование расширенных средств контроля авторизации	124
Вложение в широко используемый фреймворк авторизации	125
Избегание потенциальных ловушек.....	125
Расширенные средства контроля.....	126
Многосторонняя авторизация (MPA)	126
Трехфакторная авторизация (3FA).....	127
Бизнес-обоснования	130
Временный доступ	130
Прокси	131
Компромиссы и противоречия	132
Повышенная сложность безопасности	132
Влияние на сотрудничество и корпоративную культуру.....	132
Качественные данные и системы, влияющие на безопасность	132

Влияние на производительность пользователей	133
Влияние на сложность для разработчиков	133
Резюме	134
Глава 6. Проектирование для понятности.....	136
Почему понятность важна?	137
Инварианты системы	138
Анализ инвариантов	139
Ментальные модели	140
Проектирование понятных систем	142
Сложность и понятность	142
Разрушение сложности	143
Централизованная ответственность за требования безопасности и надежности	144
Архитектура системы	145
Понятные спецификации интерфейса	146
Понятные идентификационные данные, аутентификация и контроль доступа	149
Границы безопасности	155
Проектирование программного обеспечения	162
Использование фреймворков приложений для общих требований к сервису	162
Понимание сложных потоков данных	163
Рассматриваем удобство использования API	167
Резюме	170
Глава 7. Проектирование для меняющегося ландшафта.....	172
Типы изменений безопасности	173
Проектирование ваших изменений	173
Архитектурные решения для простых изменений	175
Постоянно обновляйте зависимости и регулярно пересобирайте код	175
Частые выпуски с использованием автоматизированного тестирования	175
Использование контейнеров	176
Использование микросервисов	177
Разные изменения: разные скорости, разные сроки	180
Краткосрочное изменение: уязвимость нулевого дня	181
Среднесрочные изменения: улучшение состояния безопасности	185
Долгосрочные изменения: внешнее требование	189

Сложности: когда планы меняются	193
Пример: растущая область действия — Heartbleed	194
Резюме	196
Глава 8. Проектирование для устойчивости	197
Принципы проектирования для устойчивости	198
Защита в глубину	199
Троянский конь	199
Анализ Google App Engine	201
Управление деградацией	205
Разделение затрат на сбои	207
Развертывание механизмов реагирования	209
Автоматизируйте ответственно	213
Управление радиусом взлома	215
Разделение по ролям	218
Разделение по местоположению	218
Разделение по времени	223
Области отказов и избыточность	223
Области отказов	224
Типы компонентов	227
Контроль избыточности	230
Непрерывная проверка	232
Основные направления проверки	234
Проверка на практике	235
Практические советы: с чего начать	239
Резюме	241
Глава 9. Проектирование для восстановления	242
От чего мы восстанавливаемся?	243
Случайные ошибки	243
Непреднамеренные ошибки	244
Ошибки программного обеспечения	245
Вредоносные действия	245
Принципы проектирования для восстановления	246
Проектирование должно быть максимально быстрым (в соответствии с политикой)	246
Ограничение зависимости от внешних представлений о времени	251
Откаты — компромисс между безопасностью и надежностью	253
Минимально допустимые номера версий безопасности	257
Использование явного механизма отката	263

Знайте свое предполагаемое состояние, вплоть до байтов.....	267
Проектирование для тестирования и непрерывной проверки.....	274
Экстренный доступ.....	275
Контроль доступа.....	276
Связь.....	277
Привычки респондентов.....	278
Неожиданные преимущества.....	279
Резюме.....	280
Глава 10. Защита от DoS-атак.....	281
Стратегии атаки и защиты.....	282
Стратегия атакующего.....	282
Стратегия защитника.....	283
Проектирование для защиты.....	284
Защищаемая архитектура.....	284
Защищаемые сервисы.....	286
Предотвращение атак.....	287
Мониторинг и оповещение.....	287
Постепенная деградация.....	288
Система защиты от DoS.....	288
Стратегический ответ.....	290
Борьба с внутренними атаками.....	291
Поведение пользователя.....	291
Поведение клиента при повторных попытках.....	293
Резюме.....	293

ЧАСТЬ III. ВНЕДРЕНИЕ СИСТЕМ

Глава 11. Пример: проектирование, разработка и поддержка публично доверенного центра сертификации.....	296
Общие сведения о публично доверенных центрах сертификации.....	296
Зачем нам нужен доверенный ЦС.....	298
Создать или купить?.....	298
Вопросы проектирования, реализации и обслуживания.....	299
Выбор языка программирования.....	300
Сложность в сравнении с понятностью.....	301
Защита сторонних компонентов и компонентов с открытым исходным кодом.....	302

Тестирование.....	303
Устойчивость набора ключей ЦС	304
Проверка данных	304
Резюме.....	305
Глава 12. Написание кода.....	306
Основа обеспечения безопасности и надежности.....	307
Преимущества использования фреймворков	308
Пример: фреймворк для серверов RPC	310
Распространенные уязвимости безопасности	314
Уязвимости SQL-внедрений: TrustedSqlString.....	316
Предотвращение XSS: SafeHtml	318
Уроки оценки и создания фреймворков	320
Простые, безопасные, надежные библиотеки для общих задач	321
Стратегия внедрения.....	322
Простота — залог безопасного и надежного кода.....	324
Избегайте многоуровневого вложения	324
Устранение «запахов» YAGNI.....	325
Погашение технического долга	326
Рефакторинг.....	327
Безопасность и надежность по умолчанию	328
Выберите правильные инструменты.....	328
Использование строгих типов	330
Очистка кода.....	333
Резюме.....	334
Глава 13. Тестирование	335
Модульное тестирование.....	336
Написание эффективных модульных тестов	337
Когда писать модульные тесты	337
Как модульное тестирование влияет на код	339
Интеграционное тестирование	340
Написание эффективных интеграционных тестов.....	341
Динамический анализ программ.....	342
Нечеткое тестирование.....	345
Как работают механизмы фаззинга	347
Написание эффективных драйверов фаззинга	351
Пример фаззера.....	352
Непрерывное нечеткое тестирование	356

Статический анализ программ.....	357
Инструменты автоматической проверки кода	359
Интеграция статического анализа в рабочий процесс.....	364
Абстрактная интерпретация	366
Формальные методы.....	368
Резюме.....	369
Глава 14. Развертывание.....	370
Концепции и терминология	371
Модель угроз.....	373
Лучшие практики.....	375
Сделайте рецензирование обязательным.....	375
Автоматизируйте	376
Проверяйте не только людей, но и артефакты	377
Относитесь к настройкам как к коду	378
Защита в соответствии с моделью угроз	380
Расширенные стратегии смягчения угроз.....	382
Бинарное происхождение	383
Что указывать в бинарном происхождении	383
Политики развертывания на основе происхождения	386
Верифицируемые сборки	388
Пункты контроля развертывания	394
Проверки после развертывания.....	396
Практические советы	397
Двигайтесь небольшими шагами	397
Предоставьте информативные сообщения об ошибках	397
Убедитесь в однозначности происхождения	398
Создавайте однозначные политики.....	399
Добавьте механизм аварийного доступа при развертывании.....	399
Пересмотр защиты в соответствии с моделью угроз	400
Резюме.....	400
Глава 15. Исследование систем.....	402
От отладки до исследования.....	403
Пример: временные файлы	403
Методы отладки	405
Что делать, если вы зашли в тупик	414
Совместная отладка: способ обучения	419
Чем отличаются исследования безопасности от отладки	420

Сбор подходящих и полезных записей журналов	422
Сделайте журналы неизменяемыми	422
Примите во внимание конфиденциальность	423
Определите, какие журналы безопасности нужно сохранить	424
Бюджет на ведение журнала	429
Надежный и безопасный доступ при отладке	430
Надежность	430
Безопасность	431
Резюме	432

ЧАСТЬ IV. ОБСЛУЖИВАНИЕ СИСТЕМ

Глава 16. Планирование аварийных ситуаций	435
Определение «бедствия»	436
Стратегии динамического реагирования на бедствия	436
Анализ риска катастроф	438
Создание команды реагирования на происшествия	439
Определение членов команды и их ролей	439
Создание устава команды	441
Создание моделей серьезности и приоритетов	442
Определите рабочие параметры для привлечения IR-команды	443
Разработка планов реагирования	444
Создание подробных инструкций	446
Убедитесь в наличии механизмов доступа и обновления	446
Предварительная подготовка людей и систем	447
Настройка систем	447
Обучение	448
Процессы и процедуры	450
Тестирование систем и планов реагирования	450
Аудит автоматизированных систем	451
Проведение ненавязчивых настольных упражнений	452
Тестирование реагирования в производственных средах	454
Тестирование красных команд	456
Оценка ответов	457
Примеры из опыта Google	458
Тест с глобальным воздействием	458
Тестирование DiRT при экстренном доступе	459
Отраслевые уязвимости	459
Резюме	460

Глава 17. Антикризисное управление	461
Это кризис или нет?	462
Сортировка инцидента	463
Компрометации и ошибки.....	465
Управление инцидентом.....	466
Первый шаг: не паникуйте!	466
Начинаем реагировать	467
Создаем команды по реагированию на инцидент.....	468
Операционная безопасность.....	470
Жертвуем хорошим OpSec для большего блага.....	472
Процесс расследования	473
Сохранение контроля над инцидентом	477
Распараллеливание действий	477
Передача обязанностей	479
Командный дух	482
Коммуникация	483
Недопонимание	483
Уклонение от ответа	484
Встречи.....	484
Информирование нужных людей с правильными уровнями детализации	486
Объединяем все вместе	488
Сортировка инцидента	488
Объявление о происшествии	488
Коммуникация и операционная безопасность.....	488
Начало инцидента.....	489
Передача обязанностей	490
Обратная передача обязанностей	490
Информирование пользователей и исправление	491
Завершение реагирования	492
Резюме.....	493
Глава 18. Восстановление и последствия	494
Логистика восстановления	496
Сроки восстановления	498
Планирование восстановления.....	499
Определение объема восстановления	499

Соображения по восстановлению.....	500
Чек-листы восстановления	506
Запуск восстановления.....	507
Изоляция активов (карантин)	508
Повторная сборка системы и обновление программного обеспечения	509
Очистка данных.....	510
Данные для восстановления	511
Учетные данные и замена секретов.....	512
Действия после восстановления	515
Постмортемы	516
Примеры.....	517
Скомпрометированные облачные экземпляры	517
Крупномасштабная фишинговая атака.....	519
Целенаправленная атака, требующая сложного восстановления	520
Резюме.....	522

ЧАСТЬ V. ОРГАНИЗАЦИЯ И КУЛЬТУРА

Глава 19. Пример из практики: команда безопасности Chrome	524
Возникновение и развитие команды	524
Безопасность — командная ответственность	528
Помогаем пользователям безопасно перемещаться в Интернете	530
Скорость имеет значение.....	531
Проектирование для защиты в глубину	531
Будьте открыты и вовлекайте сообщество	532
Резюме.....	533
Глава 20. Понимание ролей и обязанностей.....	535
Кто отвечает за безопасность и надежность	536
Роли специалистов	537
Что такое экспертиза в безопасности	539
Сертификаты и образование.....	540
Интеграция безопасности в организацию	541
Внедрение специалистов и команд по безопасности.....	544
Пример: внедрение безопасности в Google.....	545
Специальные команды: синие и красные	548
Внешние исследователи.....	551
Резюме.....	553

Глава 21. Формирование культуры безопасности и надежности	554
Определение здоровой культуры безопасности и надежности	556
Культура безопасности и надежности по умолчанию	556
Культура рецензирования	557
Культура осведомленности	559
Культура согласия	563
Культура неизбежности	564
Культура рационального использования	566
Изменение культуры с помощью хорошей практики	568
Согласуйте цели проекта и поощрения участников	569
Уменьшайте опасения с помощью механизмов снижения риска	569
Обеспечьте подстраховку	571
Повышайте производительность и удобство использования	572
Поощряйте общение и прозрачность	574
Развивайте эмпатию	575
Убедите руководство	576
Поймите, как принимаются решения	577
Обоснуйте причину для изменения	578
Расставляйте приоритеты	580
Эскалация и решение проблем	581
Резюме	581
Заключение	583
Приложение. Матрица оценки риска бедствий	585
Об авторах	588
Иллюстрация на обложке	590

Отзывы о книге

Сложно получить практические советы о том, как построить и в дальнейшем эксплуатировать надежную инфраструктуру для миллиардов пользователей. Эта книга — первая, в которой действительно собраны знания лучших в мире команд в области безопасности и надежности. И хотя немногие компании работают в масштабе Google, многие инженеры и операторы могут извлечь пользу из нелегко полученного опыта по обеспечению безопасности обширных распределенных систем. Книга богата полезными идеями, и все примеры и случаи из практики наполнены мудростью, приходящей с экспериментами, неудачами и измерениями масштабных реальных результатов. Она просто необходима всем, кто хочет правильно строить собственные системы с самого первого дня.

*Алекс Стэймос, директор Стэнфордской
интернет-обсерватории и бывший главный
сотрудник службы информационной
безопасности Facebook и Yahoo*

Эта книга — редкая находка для ветеранов отрасли и новичков. Авторы предлагают тщательно продуманные и богато иллюстрированные советы по созданию и сопровождению программного обеспечения, которые действительно выдержали испытание временем. В книге они показали прекрасный пример того, что надежность, удобство использования и безопасность идут рука об руку как полностью неразделимые основы хорошего проектирования систем.

*Михал Залевски, вице-президент
по безопасности в Snap, Inc., автор книг
The Tangled Web и Silence on the Wire*

Это «реальный мир» из статей исследователей.

*Дж. Ф. Аумассон, генеральный директор Taserakt,
главный сотрудник службы информационной
безопасности и сооснователь Taurus Group и автор
книжки Serious Cryptography*

Google сталкивается со сложнейшими проблемами безопасности, возникающими в любой компании, и раскрывает здесь собственные основные принципы безопасности. Если вы работаете в сфере надежности или безопасности систем и вам интересно, как одна из компаний-гигантов внедряет безопасность в свои системы от проектирования до эксплуатации, обратите внимание на эту книгу.

*Келли Шортридж, вице-президент по стратегии
развития программных продуктов в Capsule8*

Если вы отвечаете за работу или безопасность интернет-сервиса, будьте осторожны! Благодаря Google и другим компаниям это кажется весьма простым делом. Но все совсем не так. Я имел честь работать с этими авторами в течение многих лет и постоянно удивлялся тому, что они обнаруживали и какие меры по защите пользовательских данных предпринимали. Если у вас те же обязанности или вы просто пытаетесь понять, что нужно для масштабной защиты сервисов в современном мире, изучите эту книгу. Ничто в ней не рассматривается подробно — для этого есть и другие ресурсы — но я не знаю других источников, где можно найти столько практических советов и честного обсуждения компромиссов.

*Эрик Гросс, бывший вице-президент
по безопасности в Google*

*Сюзанне, чьи любовь к стратегическому
управлению проектами и страсть к надежности
и безопасности помогли нам не сбиться с пути!*

Предисловие Рояла Хансена

Долгие годы я мечтал, чтобы кто-нибудь написал такую книгу. Я часто восхищался книгами Google по обеспечению надежности веб-сервисов (Site Reliability Engineering, SRE) и рекомендовал их. Поэтому я был рад узнать, что к тому моменту, когда я пришел в Google, работа над книгой, посвященной безопасности и надежности, уже началась. Мне было очень приятно внести даже небольшой вклад в процесс. С момента начала моей работы в индустрии высоких технологий в компаниях разных масштабов я встречал людей, задававшихся одним и тем же вопросом: как должна быть организована безопасность? Должна ли она быть централизованной или федеративной? Независимой или встроенной? Оперативной или консультативной? Технической или управляющей? Этот список можно продолжать очень долго...

Когда модель SRE (<https://landing.google.com/sre/sre-book/chapters/introduction/>) и SRE-подобные версии DevOps стали популярными, я заметил, что пространство задач, в котором рассматривается SRE, показывает динамику, аналогичную задачам безопасности. Некоторые организации объединили эти две дисциплины в подход под названием «DevSecOps».

И SRE, и безопасность сильно зависят от классических команд разработчиков программного обеспечения. Тем не менее их специалисты принципиально отличаются от классических разработчиков программного обеспечения по нескольким причинам.

- Инженеры по обеспечению надежности веб-сервисов (SR-инженеры) и инженеры по безопасности склонны ломать и чинить, так же как и создавать.
- Помимо разработки, они занимаются эксплуатацией.
- SR-инженеры и инженеры по безопасности скорее техники, чем разработчики классического ПО.
- В основном они изолированы от команды по разработке продуктов, а не внедрены в нее.

В сфере SRE созданы необычные роли и обязанности для набора навыков, аналогичных навыкам инженера по безопасности, введена модель реализации, которая объединяет команды. Скорее всего, это можно считать следующим шагом, который нужно предпринять сообществу безопасности. Многие годы

мы с коллегами утверждали, что безопасность должна быть основной характеристикой ПО. Я считаю, что использование подхода, основанного на SRE, — логический шаг в этом направлении.

С начала работы в Google я узнал больше о том, как здесь была создана модель SRE, как она реализует философию DevOps и как развивались SRE и DevOps. В то же время я перенес свой опыт в области ИТ-безопасности в сфере финансовых услуг на технические и программные возможности безопасности в Google. Эти две области не связаны между собой, но каждая имеет свою историю, которую стоит узнать. В то же время предприятия находятся в критической точке, когда облачные вычисления, разные формы машинного обучения и сложный ландшафт кибербезопасности совместно определяют, куда движется цифровой мир, как быстро он туда попадет и какие риски связаны с этим.

По мере того как мое понимание взаимосвязи между безопасностью и SRE углублялось, я еще больше убеждался в том, что важно тщательнее интегрировать методы обеспечения безопасности в полный жизненный цикл ПО и сервисов данных. Большая часть современного гибридного облака основана на программных фреймворках с открытым исходным кодом, предлагающих взаимосвязанные данные и микросервисы. Это делает еще более важными возможности обеспечения безопасности и устойчивости.

Последние 20 лет эксплуатационные и организационные подходы к обеспечению безопасности на крупных предприятиях значительно различались. Самые известные из них подразумевают полностью централизованных руководителей по информационной безопасности и ядро эксплуатационной инфраструктуры, представленное брандмауэрами, службами каталогов, прокси-серверами, то есть командами, в которых работают сотни или тысячи сотрудников. На другом конце спектра у объединенных команд по защите деловой информации есть деловая либо техническая экспертиза, которая нужна для управления определенным списком функций или бизнес-операций. Где-то посередине — комиссии, метрики и нормативные требования, которые регулируют политики безопасности, а внедренные «чемпионы безопасности»¹ могут либо управлять взаимоотношениями, либо отслеживать проблемы для конкретного организационного подразделения. Я даже встречал варианты, когда команды пересматривали модель SRE, превращая специалистов из этой сферы в своеобразных инженеров по безопасности сайта или назначая им отдельную Agile- и Scrum-роль в командах безопасности.

¹ Security Champion — ответственный по вопросам безопасности, разработчик, который хорошо знает код и смотрит на него глазами специалиста по безопасности. — *Примеч. ред.*

По понятным причинам команды безопасности предприятия в основном сосредоточены на конфиденциальности. Но организации нередко признают такую же важность целостности и доступности данных и рассматривают эти характеристики с помощью разных команд и разных средств управления. Наличие SRE считается топовым подходом к надежности. SR-инженеры способны выявлять технические проблемы в режиме реального времени и реагировать на них, в том числе на связанные с безопасностью атаки на привилегированный доступ или конфиденциальные данные. Хотя команды часто организационно разделены в соответствии со специальными навыками, у них есть общая цель: обеспечение качества и безопасности системы или приложения.

В мире, который с каждым годом становится все более зависимым от технологий, книга о подходах к безопасности и надежности, основанных на опыте Google и всей отрасли, может стать важным вкладом в развитие разработки ПО, управления системами и защиты данных. По мере развития ландшафта угроз в настоящее время нужен динамичный и комплексный подход к защите. Надеюсь, что эта книга будет полезна для разных команд внутри и за пределами организаций в сфере безопасности. Она укрепила мою уверенность в том, что темы, которые в ней охватываются, заслуживают обсуждения и продвижения в отрасли, особенно когда все больше организаций используют архитектуры DevOps, DevSecOps, SRE и гибридных облаков вместе со связанными с ними операционными моделями. Как минимум, выход этой книги является еще одним шагом в эволюции системы безопасности и защиты данных в нашем все более цифровом мире.

*Роял Хансен, вице-президент
по безопасности в Google*

Предисловие Майкла Вайлдпанера

По своей сути как обеспечение надежности, так и обеспечение безопасности занимаются поддержанием работоспособности системы. Такие проблемы, как неработающие релизы, нехватка мощностей и неправильная конфигурация, могут сделать систему непригодной для использования (по крайней мере, временно). Инциденты с безопасностью или конфиденциальностью, которые снижают доверие пользователей, тоже делают систему менее полезной. Поэтому безопасность системы является для SRE главным приоритетом.

На этапе проектирования безопасность стала очень динамичным свойством распределенных систем. Мы прошли долгий путь от учетных записей без паролей на первых телефонных коммутаторах на Unix (ни у кого не было модема для подключения к ним или, по крайней мере, так считалось), статических комбинаций имени пользователя и пароля и статических правил брандмауэра. Сегодня вместо этого мы используем ограниченные по времени токены доступа и многомерную оценку рисков с миллионами запросов в секунду. Детальная криптография данных на лету и в состоянии покоя вместе с частой сменой ключей делает управление этими ключами дополнительной зависимостью любых сетей и систем обработки или хранения данных, которые работают с конфиденциальной информацией. Создание и использование таких программных систем безопасности требует тесного сотрудничества между первоначальными разработчиками, инженерами по безопасности и SRE.

Безопасность распределенных систем имеет для меня еще одно, личное значение. Со времени обучения в университете и до прихода в Google я строил карьеру в области безопасности, уделяя особое внимание тестированию на проникновение в сеть. Я многое узнал о хрупкости распределенных программных систем и различиях между разработчиками систем и операторами по сравнению со злоумышленниками: первым нужно продумать защиту от всех возможных атак, а злоумышленнику нужно найти только одну уязвимость, которую можно использовать.

В идеале SR-инженеры должны участвовать как в важных обсуждениях проекта, так и в реальных изменениях системы. Как один из первых технических руководителей по SRE в Gmail, я начал рассматривать SRE в качестве одной из лучших линий защиты (и, в случае системных изменений, буквально как последнюю линию защиты) в предотвращении влияния плохого дизайна или плохих реализаций на безопасность наших систем.

В двух книгах о SRE от Google — *Site Reliability Engineering*¹ (<https://www.oreilly.com/library/view/site-reliability-engineering/9781491929117/>) и *The Site Reliability Workbook*² (<https://www.oreilly.com/library/view/the-site-reliability/9781492029496/>) — рассказывается о принципах и передовых методах SRE, но не дается подробностей о взаимодействии надежности и безопасности. Эта книга восполняет данный пробел и дает возможность глубже погрузиться в темы, связанные с безопасностью.

В течение многих лет в Google мы отвлекали инженеров и рассказывали им о том, как следует обращаться с безопасностью наших систем. Но уже давно созрел более формальный подход к проектированию и эксплуатации безопасных распределенных систем. Так мы сможем лучше расширить это ранее неформальное сотрудничество.

Безопасность находится на переднем плане поиска новых классов атак и защиты наших систем от разных угроз в современных сетевых средах, тогда как SRE важно для предотвращения и устранения таких проблем. Просто нет альтернативы стремлению к надежности и безопасности как к неотъемлемым частям жизненного цикла разработки программного обеспечения.

*Майкл Вайлдпанер,
старший управляющий, SRE*

¹ Бейер Б., Джоунс К., Петофф Д., Мерфи Р. *Site Reliability Engineering*. Надежность и безотказность как в Google. — СПб.: Питер, 2019.

² Бейер Б., Рензин Д., Кавахара К., Торн С. *Site Reliability Workbook*: практическое применение. — СПб.: Питер, 2021.

Введение

Может ли система считаться действительно надежной, если по сути она не является безопасной? И наоборот, можно ли считать ее безопасной, если она ненадежна?

Для успешного проектирования, внедрения и обслуживания систем требуется стремление к обеспечению полного жизненного цикла системы. Это возможно лишь тогда, когда безопасность и надежность являются центральными элементами в архитектуре систем. Но оба эти явления часто считаются второстепенными и учитываются только после появления инцидента, что приводит к дорогостоящим и иногда трудным нововведениям.

Конструктивная информационная безопасность (Security by Design) (https://wiki.owasp.org/index.php/Security_by_Design_Principles) все более важна в мире, где многие продукты имеют выход в Интернет и облачные технологии становятся все более распространенными. Чем больше мы полагаемся на эти системы, тем надежнее они должны быть. Чем больше мы доверяем их безопасности, тем безопаснее они должны быть.

СИСТЕМЫ

В этой книге мы в основном говорим о *системах*, которые подразумевают концептуальный подход к группам компонентов, взаимодействующих между собой для выполнения некоторой функции. В нашем контексте системного проектирования эти компоненты обычно включают в себя части ПО, работающие на процессорах различных компьютеров. Они могут также включать само оборудование и процессы, с помощью которых люди проектируют, внедряют и обслуживают системы. Рассуждать о поведении систем может быть трудно, так как нередко оно бывает сложным и непредсказуемым.

Почему мы написали эту книгу

Мы хотели написать книгу, в которой основное внимание уделяется интеграции безопасности и надежности в жизненный цикл ПО и системы, с тем чтобы рассказать о технологиях и методах, которые защищают системы и обеспечивают их надежность, а также проиллюстрировать, как эти практики взаимодействуют друг с другом. Цель нашей книги — предоставить информацию о разработке,

внедрении и обслуживании систем от профессионалов-практиков, которые специализируются на безопасности и надежности.

Надо сразу признать, что отдельные стратегии, рекомендованные в книге, требуют поддержки инфраструктуры, которой просто может не быть там, где вы сейчас работаете. По возможности мы рекомендуем подходы, подходящие для организаций любого масштаба. Но здесь мы хотели поделиться мыслью о том, как можно развивать и совершенствовать существующие методы обеспечения безопасности и надежности, ведь все члены нашего растущего и квалифицированного сообщества могут многому научиться друг у друга. Надеемся, что другие организации тоже захотят поделиться с сообществом своими успехами и историями. Чем шире наши представления о безопасности и надежности, тем полезнее это будет для профессиональной среды. Инженерия безопасности и надежности все еще быстро развивается. Мы постоянно сталкиваемся с условиями и случаями, которые заставляют нас пересматривать (или в некоторых случаях заменять) ранее беспрекословно принимаемые убеждения.

Для кого предназначена эта книга

Безопасность и надежность — это ответственность каждого, поэтому мы ориентируемся на широкую аудиторию: людей, которые проектируют, внедряют и поддерживают системы. Мы не будем проводить границы между традиционными профессиональными ролями разработчиков, архитекторов, инженеров по обеспечению надежности сайтов (SRE) (<https://landing.google.com/sre/sre-book/chapters/introduction/>), системных администраторов и инженеров по проектированию безопасности. Хотя некоторые раскрываемые нами темы могут быть более полезны для опытных инженеров, мы приглашаем вас — читателей — примерить на себя разные роли, которых у вас (в настоящее время) нет, и представить, как можно было бы улучшить свои системы.

Мы уверены, что всем стоит задумываться об основах надежности и безопасности с самого начала процесса разработки и интегрировать эти принципы на ранних этапах жизненного цикла системы. Это важнейшая концепция, вокруг которой построен весь материал книги. В отрасли ведется множество дискуссий о том, что специалисты по защите информации все больше похожи на разработчиков ПО, а SR-инженеры и разработчики ПО больше напоминают специалистов по защите информации¹. Мы приглашаем вас присоединиться к обсуждению.

¹ См., например, статью: *Zovi D. D. Every Security Team is a Software Team Now* (www.blackhat.com/us-19/briefings/schedule/index.html#every-security-team-is-a-software-team-now-17280) на Black Hat USA 2019, DevSecOps (https://oreil.ly/_PAzE) на саммите Open Security и аналитическую статью в SANS: *Shackleford D. A DevSecOps Playbook* (<https://oreil.ly/Wmcx>).

Когда мы говорим «вы» в книге, мы подразумеваем читателя, независимого от конкретной работы или уровня опыта. Эта книга бросает вызов традиционным представлениям о ролях специалистов. Она направлена на то, чтобы вы могли нести ответственность за безопасность и надежность на протяжении всего жизненного цикла продукта. Не нужно стараться использовать все методы, описанные здесь, в ваших конкретных обстоятельствах. Вместо этого мы советуем возвращаться к этой книге на разных этапах карьеры или по мере развития вашей организации. Помните, что идеи, которые на первый взгляд не казались ценными, могут получить новое значение.

Примечание о культуре

Для применения широко распространенных передовых практик, которые мы рекомендуем в книге, нужен определенный уровень культуры, способствующей таким изменениям. Мы считаем, что, выбирая технологию, которой вы будете пользоваться, важно учитывать культуру вашей организации. Тогда вы сможете сосредоточиться как на безопасности, так и на надежности, чтобы любые внесенные вами изменения были постоянными и жизнеспособными. По нашему мнению, организации, которые не осознают важность безопасности и надежности, должны измениться, а перестройка культуры самой организации часто требует предварительных инвестиций.

Мы включили в книгу лучшие технические практики и подкрепили их данными. Но невозможно включить лучшие культурные практики, основанные на данных. Хотя в этой книге предлагаются подходы, которые, как мы думаем, другие могут адаптировать или обобщить, каждая организация имеет особую и уникальную культуру. Мы рассматриваем работу Google в рамках своей культуры, но это может не иметь ничего общего с культурой вашей организации. Вместо этого мы рекомендуем извлечь подходящие вам практики из рекомендаций, которые мы включили в эту книгу.

Как читать эту книгу

Хоть книга и содержит множество примеров, это не сборник рецептов. Мы описываем истории Google и индустрии в целом, а также делимся тем, что узнали за эти годы. Все примеры инфраструктуры разные, поэтому некоторые предлагаемые нами решения вам нужно будет адаптировать. Часть из них вообще могут не подходить вашей организации. Мы стараемся предоставить вам высокоуровневые принципы и практические решения, которые вы сможете применить в соответствии с вашей уникальной средой.

Начните с глав 1 и 2, а затем можете читать главы, которые вас больше всего интересуют. Многие главы начинаются с предисловия или краткого обзора, в котором изложено следующее.

- Формулировка проблемы.
- На каком этапе жизненного цикла разработки ПО стоит применять описанные принципы и практики.
- Пересечения и/или компромиссы между надежностью и безопасностью, которые необходимо учитывать.

В каждой главе темы, как правило, упорядочены от фундаментальных до самых сложных. Углубленный анализ и специализированные темы мы обозначаем значком с изображением аллигатора.

В этой книге много инструментов или методов, которые считаются в профессиональной среде хорошими практиками. Не каждая идея подойдет для вашего конкретного случая использования, поэтому сначала оцените требования своего проекта и проектные решения, адаптированные к вашему конкретному ландшафту рисков.

Несмотря на то что это книга для самостоятельной работы, вы найдете ссылки на издания Site Reliability Engineering (<https://landing.google.com/sre/sre-book/toc/index.html>) и The Site Reliability Workbook (<https://landing.google.com/sre/workbook/toc/>), где эксперты из Google описывают, как именно надежность влияет на проектирование сервисов. Чтение этих книг может дать более глубокое понимание некоторых концептов, но читать их необязательно.

Мы надеемся, что вам понравится наша книга и что некоторая информация на этих страницах поможет вам повысить надежность и безопасность ваших систем.

Условные обозначения, используемые в книге

В этой книге используются следующие типографские условные обозначения.

Курсив

Курсивом выделены новые термины.

Рубленый шрифт

Используется, чтобы отмечать URL-адреса, адреса электронной почты, элементы интерфейса.

Моноширинный шрифт

Используется для листингов программ, а также внутри текста для выделения элементов программ, таких как имена переменных, функций, баз данных, типов данных, переменных окружения, инструкций и ключевых слов. Им также выделены имена и расширения файлов.

Полужирный моноширинный шрифт

Выделяет команды или другой текст, который пользователь должен ввести самостоятельно.

Курсивный моноширинный шрифт

Выделяет текст, который должен быть заменен значениями, введенными пользователем, или значениями, определяемыми контекстом.



Этот элемент обозначает общее примечание.



Этот значок указывает на углубленный анализ.

Благодарности

Эта книга — результат энтузиазма и щедрого вклада около 150 человек, включая авторов, технических писателей, руководителей отделов и рецензентов из инженерных, юридических и маркетинговых отделов, которые охватывают 18 часовых поясов в Северной и Южной Америке, Европе и Азиатско-Тихоокеанском регионе. Мы хотели бы поблагодарить всех, кто еще не внесен в список для каждой главы.

В качестве лидеров безопасности в Google и SRE Гордон Чаффи, Роял Хансен, Бен Латч, Сунил Потти, Дэйв Ренсин, Бенджамин Трейнор Слосс и Майкл Вайлдпанер были кураторами проекта в Google. Их вера в проект, который фокусируется на интеграции безопасности и надежности непосредственно в жизненный цикл программного обеспечения и системы, была важна для выпуска книги.

Эта книга никогда не вышла бы без стремления и преданности Аны Опра. Она распознала ценность этой идеи, инициировала ее в Google, поделилась ею с лидерами SRE и безопасности и организовала огромный объем работы, необходимой для ее реализации.

Мы хотели бы выразить признание людям, которые внесли свой вклад, предоставив данные с их обсуждением и рассмотрением.

- Глава 1: Фелипе Кабрера, Перри Циник и Аманда Уокер.
- Глава 2: Джон Асанте, Шейн Хантли и Майк Койвунен.
- Глава 3: Амайя Букер, Михал Чапинский, Скотт Дайер и Райнер Волафка.
- Глава 4: Фелипе Кабрера, Дуглас Колиш, Питер Дафф, Кори Хардман, Ана Опра и Сергей Симаков.
- Глава 5: Поль Гуглиельмино и Мэттью Сакс.
- Глава 6: Дуглас Колиш, Поль Гуглиельмино, Кори Хардман, Сергей Симаков и Питер Вальчев.
- Глава 7: Адам Бахус, Брэндон Бейкер, Аманда Бурридж, Грег Касл, Петр Левановски, Марк Лодато, Дэн Лоренк, Дамиан Меншер, Анкур Рати, Даниэль Реболledo Сампер, Миче Смит, Сампат Шринивас, Кевин Стадмейер и Аманда Уокер.
- Глава 8: Пьер Бурдон, Перри Циник, Джим Хиггинс, Август Хубер, Петр Левановски, Ана Опра, Адам Стаблфилд, Сет Варго и Тоби Вайнгартнер.
- Глава 9: Ана Опра и Дж. К. ван Винкель.
- Глава 10: Золтан Егид, Петр Левановски и Ана Опра.
- Глава 11: Хизер Адкинс, Бетси Бейер, Ана Опра и Райан Сливи.
- Глава 12: Дуглас Колиш, Феликс Грёберт, Кристоф Керн, Макс Люббе, Сергей Симаков и Питер Вальчев.
- Глава 13: Дуглас Колиш, Даниэль Фабиан, Адриен Куныш, Сергей Симаков и Дж. К. ван Винкель.
- Глава 14: Брэндон Бейкер, Макс Люббе и Федерико Скринци.
- Глава 15: Оливер Барретт, Пьер Бурдон и Сандра Райчевич.
- Глава 16: Хизер Адкинс, Джон Асанте, Тим Крейг и Макс Люббе.
- Глава 17: Хизер Адкинс, Йохан Бергтрен, Джон Ланни, Джеймс Неттесхайм, Аарон Петерсон и Сара Смоллетт.
- Глава 18: Йохан Бергтрен, Мэтт Линтон, Майкл Синно и Сара Смоллетт.
- Глава 19: Абхишек Арья, Уилл Харрис, Крис Палмер, Карлос Пизано, Эдриенн Портер Фелт и Джастин Шух.
- Глава 20: Ангус Кэмерон, Даниэль Фабиан, Вера Хаас, Роял Хансен, Джим Хиггинс, Август Хубер, Артур Янк, Майкл Яноско, Майк Койвунен, Макс

Люббе, Ана Опра, Эндрю Поллок, Лора Посей, Сара Смоллетт, Питер Вальчев и Эдуардо Вела Нава.

- Глава 21: Дэвид Чаллонер, Артур Янк, Кристоф Керн, Майк Койвунен, Костя Серебряный и Дейв Вайнштейн.

Мы также хотели выразить особую благодарность Андрею Силину за его руководство на протяжении всего периода работы над книгой.

Следующие рецензенты предоставили ценную информацию и отзывы, которые весьма помогли нам: Хизер Адкинс, Кристин Бердан, Шоди Данаи Армстронг, Мишель Даффи, Джим Хиггинс, Роб Манн, Роберт Морлино, Ли-Анн Малхолланд, Дейв О'Коннор, Чарльз Проктор, Оливия Пуэрта, Джон Риз, Панкадж Рохатги, Бриттани Стагнаро, Адам Стабблфилд, Тодд Андервуд и Миа Ву. Особая благодарность Дж. К. ван Винклю за проверку целостности книги.

Мы также благодарны следующим авторам, которые поделились опытом и ресурсами: Аве Катунка, Кенту Кавахара, Кевину Молду, Дженнифер Петофф, Тому Саплу, Салиму Вирджи и Мерри Йен.

Внешний обзор от Эрика Гросса помог нам найти хороший баланс между новизной и практическими советами. Мы очень ценим его помощь, а также содержательные отзывы, которые мы получили от рецензентов: Блейка Биссета, Дэвида Н. Бланк-Эдельмана, Дженнифер Дэвис и Келли Шортридж. Детальные обзоры следующих людей сделали каждую главу лучше ориентированной на целевую аудиторию: Курт Андерсен, Андреа Барберию, Ахил Бехл, Алекс Блевитт, Крис Блоу, Джош Бранхам, Анджело Файлла, Тони Годфри, Марко Гуэрри, Эндрю Хоффман, Стив Хафф, Дженнифер Янеско, Эндрю Калат, Томас А. Лимончелли, Аллан Лиска, Джон Луни, Найл Ричард Мерфи, Лукаш Сиудут, Дженнифер Стивенс, Марк ван Хольштейн и Витсе Венема.

Особенно мы хотим поблагодарить Шилайе Нукала и Пола Бланкиншипа, которые помогли командам технических писателей, поделившись своими знаниями в SRE и безопасности.

Наконец, мы хотели бы поблагодарить следующих авторов, которые работали над содержанием, не вошедшим в эту книгу: Хизер Адкинс, Амайю Букер, Пьера Бурдон, Алекса Брэмли, Ангуса Кэмерона, Дэвида Чаллонера, Дугласа Колиша, Скотта Дира, Фануэля Гриаба, Феликса Грёберта, Рояла Хансена, Джима Хиггинса, Августа Хубера, Криса Ханта, Артура Янка, Майкла Яноска, Хантера Кинга, Майка Койвунена, Сюзанну Ландерс, Роксану Лоза (Roxana Loza), Макса Люббе, Томаса Мауфера, Шилайю Нукала, Ану Опра, Массимилиано Полетто, Эндрю Поллока, Лауру Посей, Сандру Райчевич, Фатиму Ривера, Стивена

Роддиса, Джули Сарацино, Дэвида Сейдмана, Фермина Серна, Сергея Симакова, Сару Смоллетт, Йохана Страмфер, Питера Вальчев, Сайруса Везуна, Джанет Вонг, Якуба Вармуза, Энди Уорнера и Дж. К. ван Винкля.

Спасибо команде O'Reilly Media — Вирджинии Уилсон, Кристен Браун, Джону Девинсу, Коллин Лобнер и Никки Макдональд — за их помощь и поддержку в реализации этой книги. Спасибо Рэйчел Хэд за фантастический опыт редактирования!

Наконец, основная команда книги хотела бы лично поблагодарить следующих людей.

От Хизер Адкинс

Меня часто спрашивают, как Google всегда остается безопасным. Если отвечать вкратце, то разнообразные качества его людей — это основа способности Google обеспечивать свою безопасность. Я уверена, что за всю свою жизнь не найду большего числа работающих в одной команде защитников Интернета, чем люди из Google. Хочу выразить особенную благодарность Уиллу, моему замечательному мужу, моей маме (Либби), папе (Майку) и брату (Патрику), а также Аполлону и Ориону. Спасибо моей команде и коллегам в Google за то, что они терпели мои пропуски во время написания этой книги, и за их стойкость перед лицом устрашающих злоумышленников; Эрику Гроссу, Биллу Кофрану, Урсу Хельцле, Роялу Хансену, Виталию Гуданцу и Сергею Брину за их руководство, обратную связь и периодическое поднятие брови за последние 17 с лишним лет, моим дорогим друзьям и коллегам (Мерри, Макс, Сэму, Ли, Сиобану, Пенни, Марку, Джесс, Бену, Рене, Джеку, Ричу, Джеймсу, Алексу, Лиаму, Джейн, Томиславу и Натали), особенно r00t++, за их поддержку. Спасибо, доктор Джон Бернхардт, за то, что вы многому меня научили; простите, что я так и не получила степень!

От Бетси Бейер

Бабушке, Эллиотту, тете Е и Джоан, которые вдохновляют меня каждый день. Вы все мои герои! Кроме того, Даззи, Хаммеру, Кики, Мини и Салиму, которые помогли мне стать такой фанатичной писательницей и человеком, каким я являюсь.

От Пола Бланкиншипа

Прежде всего хочу поблагодарить Эрин и Миллера, на чью поддержку я полагаюсь, и Мэтта и Ноа, которые никогда не перестают меня смешить. Я хочу выразить свою благодарность моим друзьям и коллегам в Google — особенно техническим писателям, которые борются с концепциями и языком и которые должны одновременно быть экспертами и защитниками наивных

пользователей. Огромная благодарность остальным авторам этой книги — я восхищаюсь вами и уважаю каждого из вас. То, что мое имя ассоциируется с вашими, — большая честь для меня.

От Сюзанны Ландерс

Всем авторам этой книги: я не могу выразить, насколько важно для меня быть частью этого путешествия! Я не была бы там, где я нахожусь сегодня, без нескольких особенных людей: без Тома — он помог мне в самый нужный момент; Кирилла — он научил меня всему, что я знаю сегодня; Ханнеса, Майкла и Петра — они пригласили меня присоединиться к самой удивительной команде в истории (*Semper Tuti* — всегда в безопасности!). Всем, кто зовет меня на кофе (вы знаете, кого я имею в виду!). Жизнь была бы невероятно скучной без вас. Вербене, которая, вероятно, повлияла на формирование меня самой больше, чем кто-либо другой, и, самое главное, любви всей моей жизни — за твою безоговорочную поддержку и наших самых удивительных и замечательных детей. Я не знаю, чем я заслужила всех вас, но я сделаю для вас все возможное.

От Петра Левандовски

Всем, кто делает мир лучше, чем он был до их прихода. Моей семье за безусловную любовь. Моей партнерше за то, что поделила свою жизнь с моей, к лучшему или к худшему. Другьям за радость, которую они приносят в мою жизнь. Коллегам за то, что они были определенно лучшей частью моей работы. Моим наставникам за их постоянное доверие. Я не был бы соавтором этой книги без их поддержки.

От Аны Опреа

Малышу, который родится, когда книга выйдет из типографии. Спасибо моему мужу Фабиану, который поддержал меня и позволил поработать над этой и многими другими вещами. Я благодарна, что мои родители, Ика и Ион, понимают, что я нахожусь далеко. Этот проект — доказательство того, что не может быть никакого прогресса без открытой, конструктивной обратной связи. Я смогла руководить работой над созданием книги только благодаря опыту, накопленному в последние годы, благодаря моему менеджеру Яну и всей команде разработчиков. И последнее, но не менее важное: хочу выразить свою благодарность сообществу поддержки BSides: Мюнхен и MUC:SEC, которые были созидающим местом и откуда я постоянно черпаю знания.

От Адама Стабблфилда

Спасибо моей жене, моей семье и всем моим коллегам и наставникам за эти годы.

От издательства

Мы выражаем огромную благодарность компании «КРОК» за помощь в работе над русскоязычным изданием книги и их вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

О научном редакторе русскоязычного издания

Анна Белых — старший инженер-разработчик в компании КРОК. Участвовала в проектировании и разработке высоконагруженных информационных систем разных масштабов и с применением различных архитектурных решений. Занимается вопросами производительности серверной (бэкенд) части информационных систем.

ЧАСТЬ I

Вводные материалы

Если вы попросите своих клиентов назвать их любимые характеристики ваших продуктов, вряд ли их списки начнутся с безопасности и надежности. Обе характеристики для них часто скрыты: если с ними все в порядке, ваши клиенты их не замечают.

Мы считаем, что безопасность и надежность *должны* быть в приоритете для любой организации. В конечном счете, мало кто хочет использовать продукт, который не является безопасным и надежным. Поэтому такие аспекты обеспечивают отличительную ценность для бизнеса.

В части I описываются основные для безопасных и надежных систем практики и компромиссы, которые вам, возможно, придется сделать. Далее мы предоставляем руководство для ваших потенциальных противников: как они могут действовать и как их действия могут повлиять на жизненный цикл ваших систем.

ГЛАВА 1

Пересечение безопасности и надежности

*Адам Стабблфилд, Массимилиано Полетто
и Петр Левандовски с Дэвидом Хуска
и Бетси Бейер*

0 паролях и перфораторах

Двадцать седьмого сентября 2012 года невинное объявление на весь Google вызвало серию каскадных сбоев во внутреннем сервисе. В итоге для восстановления после этих сбоев потребовалось использование перфоратора.

В Google есть внутренний менеджер паролей, который позволяет сотрудникам хранить секреты и обмениваться ими для сторонних сервисов, которые не поддерживают более совершенные механизмы аутентификации. Один из таких секретов — пароль гостевой системы Wi-Fi на большом парке автобусов, которые соединяют кампусы Google в области залива Сан-Франциско.

В тот день в сентябре корпоративная команда по транспорту отправила по электронной почте объявление тысячам сотрудников об изменении пароля Wi-Fi. Результирующий всплеск трафика был намного больше того, с которым могла бы справиться система управления паролями, разработанная несколькими годами ранее для небольшой аудитории ИТ-администраторов.

Нагрузка привела к тому, что основная реплика менеджера паролей перестала отвечать на запросы, поэтому балансировщик нагрузки перенаправил трафик на дополнительную реплику, которая тоже сразу же перестала работать. В этот момент система направила запрос дежурному инженеру. У него не было опыта реагирования на сбой сервиса: менеджер паролей поддерживался с максимальными усилиями, и за пять лет его существования не было ни одного перебоя.

Дежурный инженер попытался перезапустить сервис, но не знал, что для этого нужна смарт-карта модуля аппаратного обеспечения безопасности (HSM).

Такие смарт-карты хранились в нескольких сейфах в разных офисах Google по всему миру, но не в Нью-Йорке, где находился дежурный инженер. Когда сервис не удалось перезапустить, инженер связался с коллегой в Австралии, чтобы получить смарт-карту. К их великому сожалению, инженер в Австралии не смог открыть сейф, потому что комбинация для открытия была сохранена в теперь отключенном менеджере паролей. К счастью, другой коллега из Калифорнии запомнил комбинацию для открытия сейфа и смог извлечь смарт-карту. Однако даже после того, как инженер из Калифорнии вставил карту в считыватель, сервис не смог перезапуститься, выдав загадочную ошибку: «Пароль не может загрузить ни одну из карт, защищающих этот ключ».

В этот момент инженеры в Австралии решили, что подход к проблеме безопасности с применением грубой силы оправдан, и использовали для этой цели перфоратор. Час спустя сейф был открыт, но даже использование недавно извлеченных карт приводило к одному и тому же сообщению об ошибке.

Команде потребовался дополнительный час, чтобы понять, что зеленый индикатор на устройстве чтения смарт-карт не означает, что карта правильно вставлена. Когда инженеры перевернули карту, сервис перезапустился, и работа продолжилась.

Надежность и безопасность — важные компоненты системы, действительно заслуживающей доверия. Но создать надежную и безопасную систему непросто. Хотя требования к надежности и безопасности имеют много общего, для них нужны разные конструктивные решения. Легко пропустить тонкое взаимодействие между надежностью и безопасностью, что может привести к неожиданным результатам. Отказ менеджера паролей был вызван проблемой надежности — плохой стратегией балансировки нагрузки и сброса нагрузки — и его восстановление позже было осложнено большим количеством мер для повышения безопасности системы.

ПЕРЕСЕЧЕНИЕ БЕЗОПАСНОСТИ И КОНФИДЕНЦИАЛЬНОСТИ

Безопасность и конфиденциальность тесно взаимосвязаны. Чтобы система соблюдала конфиденциальность пользователей, она должна быть основательно защищенной и в присутствии злоумышленника вести себя так, как задумано. И наоборот — совершенно безопасная система не удовлетворяет потребности многих пользователей, если она не соблюдает их конфиденциальность. Хотя эта книга посвящена безопасности, вы можете применять описанные здесь общие подходы для достижения целей конфиденциальности.

Надежность и безопасность: соображения проектирования

Чтобы обеспечить безопасность и надежность во время проектирования, нужно учитывать различные риски. Основные риски для надежности не вредоносны по своей природе, например плохое обновление ПО или сбой физического устройства. Угрозу безопасности представляют злоумышленники, которые активно пытаются использовать уязвимости системы. При проектировании в целях надежности вы предполагаете, что в какой-то момент что-то может пойти не так. При проектировании в целях безопасности вы должны исходить из того, что в любой момент злоумышленник может попытаться что-то сделать не так.

В результате разные системы спроектированы так, что реагируют на сбои совершенно по-разному. При отсутствии злоумышленника системы часто *отказоустойчивы* (или *открыты*). Например, электронный замок должен оставаться открытым в случае сбоя питания, чтобы обеспечить безопасный выход через дверь. Отказоустойчивое/открытое поведение может привести к очевидным уязвимостям безопасности. Чтобы защититься от злоумышленника, который может использовать сбой питания, можно спроектировать дверь так, чтобы она была *защищенной* от взлома и оставалась закрытой, даже если не подключена к электросети.

КОМПРОМИСС НАДЕЖНОСТИ И БЕЗОПАСНОСТИ: ИЗБЫТОЧНОСТЬ

При проектировании для надежности нам часто приходится добавлять в системы избыточность. Например, многие электронные замки отказоустойчивы, но принимают физический ключ при сбоях питания. Похожим образом пожарные выходы обеспечивают дополнительный выходной путь в чрезвычайных ситуациях. Хотя избыточность повышает надежность, она также увеличивает возможность атаки. Злоумышленнику достаточно найти уязвимость только в одном месте, чтобы успешно атаковать.

КОМПРОМИСС НАДЕЖНОСТИ И БЕЗОПАСНОСТИ: УПРАВЛЕНИЕ ИНЦИДЕНТАМИ

Присутствие злоумышленника может также повлиять на методы сотрудничества и информацию, доступную респондентам во время инцидента. Инциденты надежности выигрывают от наличия респондентов с множеством точек зрения, которые могут помочь быстро найти и устранить первопричину. В то же время часто нужно обрабатывать инциденты безопасности с наименьшим количеством людей, которые могут эффективно решить проблему, поэтому злоумышленник не будет предупрежден об усилиях по восстановлению. В случае безопасности вы будете делиться информацией *по мере необходимости*. Точно так же наличие объемных системных журналов позволяет информировать об ответе на инцидент и сокращать время на восстановление, но — в зависимости от того, что записано — эти журналы могут быть ценной мишенью и для злоумышленника.

Конфиденциальность, целостность, доступность

Безопасность и надежность связаны с конфиденциальностью, целостностью и доступностью систем, но они видят эти свойства по-разному. Основное различие между этими двумя точками зрения — наличие или отсутствие целенаправленного злоумышленника. Надежная система не должна случайно нарушать конфиденциальность, как, например, неисправная система чата, которая может неправильно доставлять, искажать или терять сообщения. Также защищенная система не должна позволить злоумышленнику открыть доступ, подделать или уничтожить конфиденциальные данные. Рассмотрим несколько примеров, демонстрирующих, как проблема надежности может привести к проблеме безопасности.



Конфиденциальность (confidentiality), целостность (integrity) и доступность (availability) традиционно считаются базовыми атрибутами защищенных систем и называются *триадой CIA*. Хотя многие другие модели расширяют набор атрибутов безопасности за пределы трех вышеперечисленных, триада CIA остается популярной.

Конфиденциальность

В авиационной отрасли есть примечательная проблема с конфиденциальностью: микрофон типа «нажми и говори» зависает на этапе передачи (<https://oreil.ly/QXg1F>). В нескольких задокументированных случаях зависший микрофон транслировал личные разговоры между пилотами в кабине, что представляет собой нарушение конфиденциальности. В этом случае злоумышленник не участвует: недостаток надежности оборудования приводит к тому, что устройство передает данные, когда пилот не намерен этого делать.

Целостность

Компрометация целостности данных точно так же не нуждается в наличии активного злоумышленника. В 2015 году инженеры по обеспечению надежности сайтов Google (SRE) заметили, что сквозные проверки криптографической целостности нескольких блоков данных не выполнялись. Поскольку у некоторых машин, обрабатывающих данные, позже обнаружили наличие неисправленных ошибок с памятью, SR решили написать ПО, которое проводило исчерпывающую проверку целостности для каждой версии данных с однобитовым переворотом (0 меняется на 1 и наоборот). Так они могли видеть, соответствует ли один из результатов исходной проверки целостности. Все ошибки действительно оказались связаны с однобитовым переворотом, и SR-инженеры восстановили все данные. Интересно, что это был пример метода обеспечения безопасности, который пришел на помощь во время инцидента надежности (системы хранения Google также

используют некриптографические сквозные проверки целостности, но другие проблемы не позволяли SR-инженерам обнаруживать перевороты битов).

Доступность

Доступность — это проблема и надежности, и безопасности. Злоумышленник может использовать слабое место системы, чтобы остановить ее или нарушить ее работу для авторизованных пользователей. Можно также контролировать большое количество устройств, разбросанных по всему миру, для выполнения классической распределенной атаки типа «отказ в обслуживании» (DDoS), инструктирующей множество устройств наводнить жертву трафиком.

Атаки типа «отказ в обслуживании» (DoS) довольно любопытны, так как они затрагивают области надежности и безопасности. С точки зрения жертвы, злонамеренная атака может быть похожа на допустимый всплеск трафика. Например, обновление ПО в 2018 году (redd.it/9iivc5) привело к тому, что некоторые устройства Google Home и Chromecast генерировали большие синхронизированные пики в сетевом трафике при настройке часов на устройствах. Это приводило к непредвиденной загрузке центральной службы времени Google. Точно так же важная новость или другое событие, побуждающее миллионы людей выдавать практически идентичные запросы, может очень сильно походить на традиционную DDoS-атаку на уровне приложений. Как показано на рис. 1.1, когда в полночь в октябре 2019 года в области залива Сан-Франциско произошло землетрясение силой 4,5 балла, в инфраструктуре Google, обслуживающей эту область, был отмечен поток запросов.

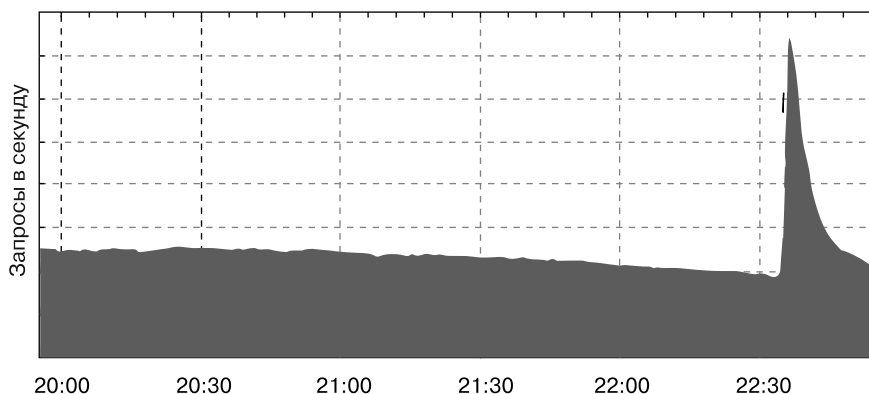


Рис. 1.1. Веб-трафик, измеренный в HTTP-запросах в секунду, пришедший в инфраструктуру Google, обслуживающую пользователей в области залива Сан-Франциско, когда 14 октября 2019 года в регионе произошло землетрясение силой 4,5 балла

Надежность и безопасность: общие черты

В отличие от многих других характеристик системы, надежность и безопасность — эмерджентные¹ свойства системы. И ту и другую трудно настроить в последнюю очередь, поэтому в идеале нужно принимать во внимание обе характеристики на самых ранних этапах проектирования. Обе требуют постоянного внимания и тестирования в течение всего жизненного цикла системы. Так происходит потому, что изменения системы могут непреднамеренно повлиять на надежность и безопасность. В сложной системе эти свойства часто определяются взаимодействием многих компонентов. Кажущееся безвредным обновление одного компонента способно в конечном счете повлиять на надежность или безопасность всей системы так, что это может быть неочевидным до тех пор, пока не обнаружится инцидент. Давайте подробнее рассмотрим эти и другие общие черты.

Незаметность

Надежность и безопасность в большинстве случаев незаметны, пока все идет хорошо. Но одна из целей команд надежности и безопасности — накопить и сохранить доверие клиентов и партнеров. Налаживание коммуникации — не только в трудные времена, но и когда дела идут хорошо — является прочной основой для такого доверия. Важно, чтобы информация была честной и конкретной, свободной от банальностей и жаргона.

К сожалению, из-за своей незаметности надежность и безопасность при отсутствии аварийных ситуаций часто рассматриваются как затраты, которые можно сократить или отложить без немедленных последствий. Но затраты на устранение ошибок надежности и безопасности могут быть серьезными. Согласно сообщениям СМИ, утечка данных (<https://oreil.ly/QJuBm>), возможно, привела к снижению цены, которую Verizon заплатила за приобретение интернет-бизнеса Yahoo! в 2017 году, на 350 млн долларов США. В том же году из-за сбоя питания (<https://oreil.ly/vyXAE>) отключились основные компьютерные системы в Delta Airlines, что привело к почти 700 отменам рейсов и тысячам задержек, уменьшив пропускную способность Delta в течение дня примерно на 60 %.

¹ Возникающие из совокупности частей системы и их взаимодействия, не присущие ни одному элементу системы по отдельности.

Оценка

Достичь идеальной надежности или безопасности на практике невозможно. Поэтому вы можете использовать риск-ориентированные подходы для оценки стоимости последствий негативных событий, а также первоначальных и возможных затрат на предотвращение этих событий. Тем не менее для надежности и безопасности нужно по-разному измерять вероятность негативных событий. Вы можете рассуждать о надежности композиции систем и хотя бы частично планировать разработку в соответствии с желаемыми бюджетами ошибок¹, потому что можно предполагать независимость отказов для отдельных компонентов. Безопасность такой композиции оценить сложнее. Анализ конструкции и реализации системы может обеспечить определенный уровень уверенности. Состязательное тестирование — смоделированные атаки, обычно выполняемые с точки зрения определенного злоумышленника, — тоже можно использовать для оценки устойчивости системы к определенным видам атак, эффективности механизмов обнаружения и потенциальных последствий атак.

Простота

Поддержание максимально простой конструкции системы — один из лучших способов улучшить вашу способность оценивать как надежность, так и безопасность системы. Более простая конструкция уменьшает поверхность атаки, вероятность непредвиденных взаимодействий внутри системы и облегчает людям понимание системы и рассуждения о ней. Ясность важна в чрезвычайных ситуациях, когда нужно быстро устранить симптомы и сократить среднее время восстановления (mean time to repair, MTTR). В главе 6 подробно рассматривается эта тема и обсуждаются такие стратегии, как минимизация поверхностей атаки и изоляция ответственности инвариантов безопасности в небольших, простых подсистемах, которые могут рассматриваться независимо.

Эволюция

Независимо от того, насколько простым и элегантным является первоначальный дизайн, со временем системы все равно приходится изменять. Сложность вносят новые требования к функциям, изменения в масштабе и развитие базовой инфраструктуры. Что касается безопасности, необходимость идти в ногу с развивающимися атаками и новыми злоумышленниками тоже может стать причиной усложнения системы. Кроме того, давление для удовлетворения потребностей рынка может привести к тому, что разработчики и мейнтейнеры

¹ Для получения дополнительной информации о бюджетах ошибок см. главу 3 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/embracing-risk/>).

будут срезать углы и накапливать технический долг. В главе 7 рассматриваются некоторые из этих проблем.

Сложность часто накапливается непреднамеренно, но это может привести к критическим ситуациям. Например, когда небольшие и явно безобидные изменения имеют серьезные последствия для надежности или безопасности системы. Ошибка, появившаяся в 2006 году и обнаруженная почти два года спустя в версии библиотеки OpenSSL в Debian GNU/Linux, — печально известный пример (<https://oreil.ly/OIX0b>) серьезного сбоя, вызванного небольшим изменением. Разработчик открытого исходного кода заметил, что Valgrind, стандартный инструмент для отладки проблем с памятью, сообщает о предупреждениях об использовании памяти перед инициализацией. Для устранения предупреждений разработчик удалил две строки кода. К сожалению, это привело к тому, что генератор псевдослучайных чисел в OpenSSL выдавал только идентификатор процесса, который в Debian в то время имел значение от 1 до 32 768. Можно было легко взломать криптографические ключи простым перебором значений.

Компания Google не была застрахована от сбоев, вызванных на первый взгляд безобидными изменениями. Например, в октябре 2018 года YouTube более часа не работал (<https://oreil.ly/CpxXL>) в глобальном масштабе из-за небольшого изменения в стандартной библиотеке журналов. Изменение кода для улучшения детализации ведения журнала событий показалось невинным как его автору, так и назначенному проверяющему код и прошло все тесты. Но разработчики не до конца осознали влияние изменений в масштабе YouTube: в условиях производственной нагрузки это изменение быстро привело к нехватке памяти на серверах YouTube. По мере того, как сбои переносили пользовательский трафик на другие, все еще работоспособные серверы, каскадные сбои останавливали весь сервис.

Устойчивость

Конечно, проблема использования памяти не должна была вызывать глобальный перерыв в обслуживании. Системы нужно проектировать так, чтобы они были устойчивыми при неблагоприятных или неожиданных обстоятельствах. С точки зрения надежности такие обстоятельства часто бывают вызваны неожиданно высокой нагрузкой или отказами компонентов. Нагрузка зависит от объема и средней стоимости запросов в системе. Поэтому можно добиться устойчивости, уменьшая часть входящей нагрузки (обрабатывая меньше) или снижая стоимость обработки для каждого запроса (обрабатывая дешевле). Чтобы можно было устранить сбои компонентов, проект системы должен допускать избыточность и включать отдельные области сбоев для ограничения влияния сбоев путем перенаправления запросов. В главе 8 обсуждаются эти темы, а в главе 10 подробно рассматриваются меры по снижению уровня DoS.

Какими бы гибкими ни были отдельные компоненты системы, когда она станет достаточно сложной, вы не сможете легко продемонстрировать, что вся она неуязвима для компрометации. Можно решить эту проблему частично, используя защиту в глубину и четкие области сбоев. *Защита в глубину* (Defence in Depth, (DiD)) — это применение нескольких защитных механизмов, иногда избыточных. *Четкие области сбоев* ограничивают «радиус взрыва» сбоя, а также повышают надежность. Хорошая конструкция системы ограничивает способность злоумышленника использовать скомпрометированный хост или украденные учетные данные для перемещения в сторону или повышения привилегий и воздействия на другие части системы.

Можно реализовать четкие области сбоев, разделяя разрешения или ограничивая область действия учетных данных. Например, внутренняя инфраструктура Google поддерживает учетные данные, ограниченные географическим регионом. Эти типы функций не дают нарушителю, подвергающему сервер риску в одном регионе, перемещаться в сторону на серверы в других.

Еще один распространенный механизм защиты в глубину — использование независимых уровней шифрования для чувствительных данных. Даже если диски обеспечивают шифрование на уровне устройства, неплохо будет шифровать данные и на прикладном уровне. Таким образом даже некорректной реализации алгоритма шифрования в контроллере диска будет недостаточно для нарушения конфиденциальности защищенных данных, если у злоумышленника будет физический доступ к устройству хранения.

Эти примеры касаются внешних злоумышленников, но стоит учитывать и возможные угрозы со стороны внутренних нарушителей. Хотя такой нарушитель может знать больше, чем внешний злоумышленник, впервые ворующий учетные данные сотрудника, на практике эти случаи очень похожи. *Принцип наименьших привилегий* позволяет смягчать внутренние угрозы. Это означает, что пользователь должен иметь минимальный набор привилегий, которые нужны для выполнения его работы в данный момент времени. Например, такие механизмы, как **sudo** в Unix, поддерживают детализированные политики, указывающие, какие пользователи могут запускать какие команды и под какой ролью.

В Google мы используем многостороннюю авторизацию для гарантии того, что конфиденциальные операции проверены и одобрены определенными группами сотрудников. Этот многосторонний механизм защищает от злоумышленников и снижает риск ненамеренных человеческих ошибок, что является частой причиной сбоев надежности. Минимальная привилегия и многосторонняя авторизация не новы — они использовались во многих некомпьютерных сценариях, от ядерных пусковых шахт до банковских хранилищ. Глава 5 раскрывает эти концепции более подробно.

От проектирования до использования

Всегда учитывайте соображения безопасности и надежности. Даже при переводе надежного проекта в полностью развернутую систему на стадии эксплуатации. При разработке кода можно выявлять потенциальные проблемы безопасности и надежности, проанализировав его, и даже предотвращать целые классы проблем с помощью общих сред и библиотек. Некоторые из этих методов обсуждаются в главе 12.

Перед развертыванием системы можно протестировать ее, чтобы убедиться в ее правильной работе в обычных сценариях и в крайних случаях, в которых обычно проявляются недостатки надежности и безопасности. Если вы хотите быть уверены в том, что созданная система соответствует вашему замыслу, то без тестирования не обойтись. Используете ли вы нагрузочное тестирование, чтобы понять поведение системы при большом количестве запросов, анализ предельных значений, чтобы исследовать поведение на потенциально неожиданных входных данных, или специальные тесты, чтобы убедиться, что криптографические библиотеки не пропускают информацию. Глава 13 подробно описывает эти подходы.

Некоторые подходы к непосредственному развертыванию (см. главу 14) понижают риск нарушения безопасности и надежности. Например, канареечное тестирование и медленное развертывание могут предотвратить взлом системы для всех пользователей одновременно. Точно так же система развертывания, которая принимает только код, который был должным образом проверен, может помочь снизить риск того, что внутренний нарушитель отправит вредоносный бинарный файл в эксплуатацию.

Исследование систем и ведение журналов

До сих пор, чтобы предотвратить сбои в области надежности и безопасности, мы базировались на принципах проектирования и подходах к реализации. Но это нецелесообразно и слишком дорого для достижения идеальной надежности или безопасности. Стоит предположить, что предупреждающие механизмы не работают, и составить план по обнаружению и устранению неисправностей.

Как мы обсудим в главе 15, правильное ведение журналов — основа для обнаружения сбоев и готовности к ним. Чем более полны и подробны журналы, тем лучше, но в этом руководстве есть предостережения. Чем больше объем журнала, тем выше будет его стоимость, а его анализ может стать менее эффективным. Пример с YouTube, приведенный ранее в главе, показывает, что ведение журналов тоже

может создавать проблемы с надежностью. Журналы безопасности — отдельная забота. Они не должны содержать конфиденциальную информацию вроде учетных данных для аутентификации или личную информацию (personally identifiable information, PII), чтобы не привлекать злоумышленников.

Антикризисное реагирование

В чрезвычайной ситуации команды должны работать быстро и слаженно, потому что проблемы могут повлечь немедленные последствия. В худшем случае происшествие может разрушить бизнес за считанные минуты. Например, в 2014 году злоумышленник вывел из строя сервис Code Spaces за несколько часов, войдя в административные инструменты сервиса и удалив все его данные, включая резервные копии (<https://oreil.ly/dYXkG>). Правильно налаженное сотрудничество и хорошее управление инцидентами имеют решающее значение для своевременного реагирования в этих ситуациях.

Организовать антикризисное реагирование сложно, поэтому лучше иметь план до того, как возникнет чрезвычайная ситуация. К моменту, когда вы обнаружите инцидент, время уже может быть на исходе. В любом случае респонденты работают в условиях стресса и нехватки времени и (по крайней мере, поначалу) с ограниченной осведомленностью о ситуации. Если организация большая и для инцидента нужно реагировать круглосуточно или сотрудничать в разных часовых поясах, необходимость поддерживать состояние в командах и передавать управление инцидентами в рамках рабочих смен еще больше усложняет работу. Инциденты безопасности обычно создают внутреннее напряжение, вызванное поиском баланса между стремлением вовлечь все основные стороны и необходимостью — часто обусловленную юридически или нормативно — ограничивать обмен информацией исходя из принципа действительной потребности. Более того, первоначальный инцидент безопасности может оказаться лишь верхушкой айсберга. Расследование может выходить за пределы компании или протекать с привлечением правоохранительных органов.

Во время кризиса важно иметь четкую цепочку командования и надежный набор чек-листов, сборников и протоколов. Как обсуждается в главах 16 и 17, в Google кодифицировали антикризисное реагирование (<https://oreil.ly/NVSXJ>) в программу под названием «Управление инцидентами в Google» (Incident Management at Google, IMAG). Она устанавливает стандартный, последовательный способ обработки всех типов инцидентов — от сбоев системы до стихийных бедствий — и организации эффективного реагирования. IMAG была смоделирована на основе системы управления инцидентами (Incident Command System, ICS) правительства США (<https://oreil.ly/uSpFn>), стандартизированного подхода к командованию, контролю и координации

действий в чрезвычайных ситуациях среди респондентов из нескольких правительственных учреждений.

Когда респонденты не сталкиваются с давлением продолжающегося инцидента, они обычно договариваются о длительных интервалах работы с небольшой активностью. За это время командам нужно сохранять навыки и мотивацию отдельных лиц, а также улучшать процессы и инфраструктуру при подготовке к следующей чрезвычайной ситуации. С помощью программы тестирования восстановления после катастроф в Google (Disaster Recovery Testing, DiRT) (<https://oreil.ly/hoBK3>) регулярно моделируются различные внутренние сбои системы, чтобы команды научились справляться с ними. Частые учения по безопасности позволяют проверить защиту и помогают выявить новые уязвимости. Google использует IMAG даже для небольших инцидентов, что побуждает нас регулярно задействовать аварийные инструменты и процессы.

Восстановление

Для восстановления после сбоя системе безопасности нужны исправления, чтобы устранить уязвимости. Понятно, что желательно, чтобы этот процесс происходил как можно быстрее, с применением регулярно используемых и потому достаточно надежных механизмов. Но возможность быстрого внесения изменений может быть обоюдоострым мечом. Хотя она позволяет быстро устранить уязвимости, она может и приводить к ошибкам или проблемам с производительностью, которые наносят большой ущерб. Стремление к быстрому добавлению исправлений становится еще большим, если уязвимость широко известна или серьезна. При медленном внесении исправлений возникает большая уверенность в том, что непреднамеренных побочных эффектов нет, но есть риск, что уязвимость будет использована. В конечном счете, выбирая между медленным внесением исправлений и быстрым реагированием, мы приходим к оценке риска и анализу бизнес-решения. Например, может быть допустимо потерять часть производительности или увеличить использование ресурсов, чтобы устранить серьезную уязвимость.

Такие решения подчеркивают необходимость в надежных механизмах восстановления, которые позволяют быстро развертывать нужные изменения и обновления без ущерба для безопасности, а также дают возможность находить потенциальные проблемы до того, как они вызовут повсеместный сбой. Например, надежная система восстановления флота должна иметь устойчивое представление о текущем и желаемом состоянии каждой машины, а также предоставлять резервные копии для гарантии того, что состояние никогда не снизится до устаревшей или небезопасной версии. Глава 9 охватывает этот и многие другие подходы, а в главе 18 обсуждается восстановление системы после происшествия.

Резюме

У безопасности и надежности много общего — обе являются неотъемлемыми свойствами всех информационных систем. Цель этой книги — помочь вам решить неизбежные проблемы, связанные с безопасностью и надежностью, на ранних этапах эволюции и развития ваших систем. Наряду с усилиями по разработке каждая организация должна понимать роли и обязанности (см. главу 20), способствующие формированию культуры безопасности и надежности (глава 21), для сохранения проверенных методов работы. Мы делимся своим опытом и извлеченными уроками для того, чтобы вы смогли избежать более серьезных последствий в будущем, приняв на вооружение некоторые из описанных здесь принципов на ранних этапах жизненного цикла системы.

Мы написали эту книгу для широкой аудитории и надеемся, что она будет полезна для вас независимо от стадии или масштаба проекта. Читая, помните о профиле риска вашего проекта — управление фондовой биржей имеет кардинально иной профиль риска, чем управление сайтом приюта для животных.

В следующей главе подробно рассматриваются классы злоумышленников и их возможные мотивации.

Виды злоумышленников

*Хизер Адкинс и Дэвид Хуска
с Джен Барнасон*

В августе 1986 года системный администратор Ливерморской лаборатории имени Лоуренса Клиффорд Столл наткнулся на безобидную ошибку в бухгалтерском учете, которая привела к десятимесячному поиску вора государственных секретов Соединенных Штатов¹. Столл возглавил это расследование. Оно считается первым публичным примером подобных действий, в котором были раскрыты конкретные тактики, методы и процедуры (tactics, techniques and procedures, TTP), используемые противником для достижения своих целей. После тщательного изучения команда по расследованию составила представление о том, как злоумышленник находил и извлекал данные из защищенных систем. Многие системные разработчики учли уроки, извлеченные из оригинальной статьи Столла, описывающей усилия команды.

В марте 2012 года Google отреагировал на необычное отключение электроэнергии в одном из своих бельгийских центров обработки данных. В конечном счете это привело к повреждению локальных данных. Расследование показало, что кошка повредила ближайший внешний источник питания, что и вызвало серию каскадных сбоев в энергосистемах здания. Google изучил, как подобные сложные системы выходят из строя, и внедрил гибкие методы проектирования при подвешивании, закапывании и погружении в воду кабелей по всему миру.

Очень важно понимать противников системы безопасности, чтобы обеспечить устойчивость и выживаемость в случае широкого спектра катастроф. В контексте надежности противники обычно действуют с добрыми намерениями

¹ Столл задокументировал атаку в статье «Выслеживание коварного хакера» (Stalking the Willy Hacker) (<http://pdf.textfiles.com/academics/wilyhacker.pdf>), напечатанной в Communications of the ACM, и в книге «Яйцо кукушки, или Преследуя шпиона в компьютерном лабиринте» (The Cuckoo's Egg: Tracking a Spy Through the Maze of Computer Espionage). Обе публикации будут полезны тем, кто разрабатывает безопасные и надежные системы, так как их данные все еще актуальны.

и принимают абстрактную форму. Они проявляются как обычные аппаратные сбои или случаи чрезмерного интереса пользователя (так называемые катастрофы успеха). Это могут быть изменения конфигурации, которые приводят к тому, что системы ведут себя неожиданно, или рыболовные суда, которые случайно разрывают подводные волоконно-оптические кабели. Злоумышленники же в контексте безопасности — это люди, оказывающие нежелательное воздействие на целевую систему. Несмотря на эти противоречивые намерения и методы, изучение противников в части надежности и безопасности важно для понимания того, как проектировать и внедрять отказоустойчивые системы. Без этого знания предвидеть действия коварного хакера или любопытного кота было бы довольно сложно.

В этой главе мы тщательно изучим противников в контексте безопасности, что поможет специалистам из разных областей развить понимание «вражеского» мышления. Очень заманчиво представлять их через призму популярных стереотипов: злоумышленники в темных подвалах с хитроумными прозвищами и подозрительным поведением. Такие персонажи, конечно, существуют, но любой, у кого есть время, знания или деньги, может подорвать безопасность системы. За небольшую плату каждый может приобрести ПО, позволяющее завладеть информацией компьютера или мобильного телефона, к которым есть физический доступ. Правительства регулярно покупают или создают программное обеспечение для компрометации систем в своих целях. Исследователи часто изучают механизмы безопасности систем, чтобы понять, как они работают. Поэтому, думая о тех, кто может атаковать систему, сохраняйте объективность.

Нет двух одинаковых атак или атакующих. Мы рекомендуем заглянуть в главу 21, чтобы обсудить культурные аспекты обращения с противниками. Прогнозирование будущих катастроф безопасности — это часто игра в догадки, даже для опытных экспертов. В следующих разделах мы рассмотрим три структуры для понимания противников. Уже многие годы мы считаем их полезными, исследуем возможные мотивы нападения людей на системы, некоторые общие профили противников и то, как стоит думать о методах злоумышленников. Мы также предоставим иллюстративные (и, надеюсь, занимательные) примеры из трех фреймворков.

Мотивация атакующего

Противники безопасности — это прежде всего люди (по крайней мере, пока). Поэтому мы можем взглянуть на цель атак глазами людей, их проводящих. Это может помочь нам лучше понять, как нужно реагировать проактивно (при проектировании системы) и реактивно (при инцидентах). Рассмотрим следующие мотивы атаки.

Развлечение

Подорвать безопасность системы ради удовольствия от осознания того, что это возможно.

Слава

Получить известность, похваставшись техническими навыками.

Активизм

Высказать свое мнение или широко транслировать сообщение — как правило, политическую точку зрения.

Финансовая выгода

Заработать деньги.

Принуждение

Принудить жертву сознательно сделать то, чего она не желает.

Манипуляции

Добиться ожидаемого результата или изменения поведения — например, путем публикации ложных данных (дезинформация).

Шпионаж

Получить информацию, которая может быть ценной (слежка, экономический шпионаж). Такие атаки часто выполняются спецслужбами.

Уничтожение

Разрушить систему, просто отключить ее или уничтожить данные.

Злоумышленник может быть одновременно финансово мотивированным багхантером, агентом государственной разведки и преступным элементом! Например, в июне 2018 года Министерство юстиции США обвинило Пак Джин Хёка, гражданина Северной Кореи, в участии во многих мероприятиях (<https://oreil.ly/mVluG>) от имени своего правительства. В том числе в создании пресловутого WannaCry Ransomware в 2017 году (используемого для получения финансовой выгоды), компрометации Sony Pictures в 2014 году (чтобы заставить Sony не выпускать скандальный фильм и нанести ущерб инфраструктуре компании) и компрометации электроэнергетических компаний (предположительно для шпионажа или разрушения). Исследователи также наблюдали, как правительственные злоумышленники использовали те же вредоносные программы, которые применялись в атаках для кражи электронных денег (<https://oreil.ly/u4cJV>) в видеоиграх для получения личной выгоды.

При проектировании систем важно помнить об этих разнообразных мотивах. Рассмотрим организацию, которая обрабатывает денежные переводы от имени своих клиентов. Понимание цели злоумышленника поможет нам в разработке более безопасной системы. Хороший пример возможных мотивов можно увидеть в действиях группы злоумышленников правительства Северной Кореи (включая Пака), которые якобы пытались украсть миллионы долларов (<https://oreil.ly/u4cJV>), взломав банковские системы и используя транзакционную систему SWIFT для перевода денег с учетных записей клиентов.

Профили злоумышленников

Понять мотивы злоумышленников будет проще, рассматривая самих людей: кто они, выполняют они атаки для себя или кого-то, каковы их общие интересы. Здесь мы расскажем о некоторых *профилях* злоумышленников, указав, как они связаны с разработчиком системы, и дадим несколько советов по защите систем от нарушителей такого типа. Для краткости мы допускаем некоторые вольности в обобщениях, но помните: не существует двух одинаковых атак или атакующих. Эта информация — иллюстративная, а не окончательная.

ПРОИСХОЖДЕНИЕ ХАКИНГА

МТИ (Массачусетский технологический институт) — место рождения термина «*хакерство*», восходящего к 1950-м годам, когда эта деятельность возникла из невинных розыгрышей. Такое «благородное» происхождение привело к тому, что некоторые стали разделять «хакерство» и «нападение» на отдельные понятия неосознанного и осознанного поведения. Такую традицию мы продолжаем и в нашей книге. Сообщество хакеров МТИ сегодня действует, ориентируясь на нестрогие нормы этики, описанные в руководстве МТИ (<https://handbook.mit.edu/hacking>).

Любители

Первыми компьютерными хакерами были *любители* — любопытствующие технические специалисты, которые хотели понять, как работают системы. В процессе разборки компьютеров или отладки программ эти «хакеры» обнаруживали недостатки, не замеченные разработчиками оригинальной системы. Проще говоря, любители мотивированы своей жадой знаний. Они взламывают системы ради развлечения и могут быть союзниками разработчиков, желающих повысить устойчивость системы. В основном любители придерживаются личной

этики о том, чтобы не наносить вред системам. Они не пересекают границы преступного поведения. Зная то, как эти хакеры думают о проблемах, вы можете сделать собственные системы более безопасными.

Исследователи уязвимостей

Исследователи уязвимостей профессионально используют свои знания. Они находят недостатки в безопасности как штатные сотрудники, фрилансеры, работающие неполный день, или даже случайно, как обычные пользователи, которые столкнулись с ошибками. Многие исследователи участвуют в программах вознаграждения за найденные в ПО ошибки, также известных как *отлов ошибок*, или *bug bounties* (см. главу 20).

Исследователи уязвимостей обычно заинтересованы в улучшении систем и могут быть важными союзниками организаций, стремящихся обезопасить свои системы. Они работают в рамках предсказуемых норм раскрытия, устанавливающих ожидания между владельцами систем и исследователями в отношении того, как уязвимости обнаруживаются, фиксируются, исправляются и обсуждаются. Исследователи, действующие в соответствии с этими нормами, избегают неправомерного доступа к данным, причинения вреда или нарушения закона. Обычно работа за пределами этих норм сводит на нет возможность получения вознаграждения и может расцениваться как преступное поведение.

Кроме того, *красные команды* и тестировщики на проникновение атакуют цели с разрешения владельца системы и могут быть наняты явно для этих практик. Как и исследователи, они ищут способы обойти безопасность системы с акцентом на ее повышение и действуют в соответствии с набором этических принципов. Дополнительную информацию о красных командах вы найдете в главе 20.

Правительства и правоохранительные органы

Правительственные организации (например, правоохранительные органы и спецслужбы) могут нанимать экспертов по безопасности для сбора разведанных, полицейского надзора за внутренними преступлениями, экономического шпионажа или дополнения военных операций. На данный момент большинство национальных правительств повышают свой уровень безопасности в этих целях. В некоторых случаях правительства могут обратиться к талантливым студентам, недавно окончившим обучение, к исправившимся преступникам, вышедшим из

тюрьмы, или к известным специалистам в сфере безопасности. Мы не можем подробно охватить эти типы злоумышленников, но приведем несколько примеров их наиболее распространенных действий.

Сбор разведывательных данных

Сбор разведанных — наиболее публично обсуждаемая правительственная деятельность. Ею занимаются люди, которые знают, как взламывать системы. В последние несколько десятилетий привычные технологии шпионажа, включая сигнальную разведку (signals intelligence, SIGINT) и человеческую разведку (human intelligence, HUMINT), модернизировались. В одном известном примере 2011 года охранная компания RSA была скомпрометирована (<https://oreil.ly/Es3PA>) злоумышленником, которого многие эксперты связывают с разведывательным аппаратом Китая (<https://oreil.ly/DuLZV>). Атакующие скомпрометировали RSA, чтобы украсть криптографические начальные значения для их популярных токенов двухфакторной аутентификации. После того, как атакующие получили эти значения, им больше не были нужны физические токены для генерации одно-разовых аутентификационных учетных данных для входа в системы Lockheed Martin, оборонного подрядчика, разрабатывающего технологии для военных США. Когда-то давно взломом такой компании, как Lockheed, могли бы заниматься сотрудники-оперативники, например, через подкуп служащих или с помощью нанятого фирмой шпиона. Однако вторжение в системы сделало возможным для атакующих использовать новые пути получения секретов с помощью более сложных электронных методов.

Военные цели

Правительства могут взламывать системы в военных целях — это то, что специалисты часто называют *кибервойнами* или *информационными войнами*. Представьте, что правительство хочет вторгнуться в другую страну. Может ли оно каким-то образом атаковать системы ПВО, принадлежащие цели, и обмануть их, чтобы те не узнали входящие воздушные силы? Может ли оно отключить электроэнергию, водоснабжение или банковские системы?¹ В качестве альтернативы представьте, что правительство хочет помешать другой стране создать или получить оружие. Может ли оно дистанционно и незаметно нарушить прогресс? Похожая ситуация была в Иране в конце 2000-х годов, когда атакующие

¹ Пример того, насколько сложной может быть эта область, — не все злоумышленники в таких конфликтах являются частью организованной армии. Например, голландские хакеры, по сообщениям, скомпрометировали вооруженные силы США во время войны в Персидском заливе (1991 г.) и предложили украденную информацию правительству Ирака (<https://oreil.ly/7eNxW>).

незаконно внедрили модульную часть ПО в системы управления центрифугами, используемыми для обогащения урана. По сообщениям, эта операция, названная исследователями Stuxnet (<https://oreil.ly/WNu9A>), была предназначена для уничтожения центрифуг и прекращения ядерной программы Ирана.

Контроль за внутренней деятельностью

Правительства могут взломать системы контроля за внутренней деятельностью. В недавнем примере NSO Group, подрядчик по кибербезопасности, продавал различным правительствам ПО, позволяющее вести частное наблюдение за связями между людьми без их ведома (путем удаленного мониторинга звонков по мобильному телефону). По сообщениям, это ПО было предназначено для наблюдения за террористами и преступниками — относительно допустимыми целями. К сожалению, некоторые правительственные клиенты NSO Group также использовали программное обеспечение для прослушивания журналистов и активистов. В некоторых случаях это приводило к преследованиям, арестам и даже, возможно, смертям¹. Этика правительств, использующих эти возможности против своего народа, является предметом горячих споров, особенно в странах без сильной правовой базы и надлежащего надзора.

Защита ваших систем от государственных исполнителей

Разработчики систем должны тщательно рассчитать, могут ли те быть целью государственного исполнителя. Для этого нужно понять действия, выполняемые вашей организацией, которые могут быть привлекательны для этих исполнителей. Рассмотрим технологическую компанию, которая разрабатывает и продает микропроцессорные технологии военной ветви государства. Возможно, другие государства тоже будут заинтересованы в получении этих чипов и могут прибегнуть к краже разработок с помощью электронных средств.

Ваш сервис может иметь данные, которые государство хочет получить, но другими способами получить их сложно. Проще говоря, спецслужбы и правоохранительные органы ценят личные сообщения, данные о местонахождении и подобные типы конфиденциальной информации. В январе 2010 года компания Google объявила (<https://oreil.ly/d3C-z>), что стала свидетелем изоэренной целевой атаки со стороны Китая (названной исследователями «Операция «Аврора»») на корпоративную инфраструктуру компании. Как теперь хорошо известно, она

¹ Деятельность NSO Group была рассмотрена и задокументирована CitizenLab, исследовательской и политической лабораторией, находящейся в Школе глобальных отношений и государственной политики Мунка в университете Торонто. Например, см. https://oreil.ly/IqDN_.

была направлена на долгосрочный доступ к учетным записям Gmail. Хранение личной информации клиентов, особенно личных сообщений, может повысить риск того, что спецслужбы или правоохранительные органы будут заинтересованы в ваших системах.

Иногда вы можете стать целью неосознанно. Операция «Аврора» не ограничивалась крупными технологическими компаниями — она затронула по меньшей мере 20 жертв в различных секторах финансов, технологий, СМИ и химической промышленности. Эти организации были разных размеров, и многие из них не думали, что подвержены риску нападения со стороны государства.

Рассмотрим приложение для предоставления спортсменам аналитики отслеживания данных, в том числе о том, где они бегают или ездят на велосипеде. Будут ли эти данные привлекательной целью для спецслужб? Аналитики, рассматривающие публичную таблицу активности, созданную компанией по отслеживанию тренировок Strava, ответили на этот вопрос в 2018 году. Они заметили, что местонахождение секретных военных баз в Сирии было обнаружено (https://oreil.ly/N1g_X) из-за того, что американские войска пользовались сервисом для анализа своих тренировок.

Проектировщики систем должны знать, что правительства могут использовать значительные ресурсы для получения доступа к интересующим их данным. Чтобы защититься от заинтересованного в данных правительства, может потребоваться гораздо больше ресурсов, чем ваша компания может выделить на внедрение решений по безопасности. Мы рекомендуем организациям внедрять защитные меры в долгосрочной перспективе. Например, вкладывая средства в защиту наиболее уязвимых активов на ранних этапах и создавая постоянно обновляемую программу, к которой позже можно будет применять новые уровни защиты. Идеальный результат — заставить злоумышленника потратить на атаку большое количество своих ресурсов — это увеличит его риск быть пойманным и его действия могут стать явными для других возможных жертв и государственных органов.

Активисты

Хактивизм — это использование технологий для призыва к социальным изменениям. Этот термин применяется к широкому кругу политических действий в Сети, от подрывной деятельности против государственной слежки до злонамеренного нарушения работы систем¹. Последний случай мы рассмотрим здесь как пример для размышлений о проектировании систем.

¹ Ведутся споры о том, кто придумал этот термин и что он означает, но широкое распространение он получил после 1996 года, когда был принят группой Hacktivism (<https://oreil.ly/oWzO8>), связанной с Культурой Мертвой Коровы (Cult of the Dead Cow, cDc).

Известно, что хактивисты *портят* сайты, то есть заменяют обычный контент политическим посланием. В одном из примеров 2015 года (<https://oreil.ly/fZAD->) Сирийская электронная армия — группа хакеров, действующих в поддержку режима Башара аль-Асада (Bashar al-Assad), — захватила сеть распространения контента (content distribution network, CDN), обслуживающую веб-трафик для www.army.mil. После этого нарушители добавили на сайт сообщение в поддержку Асада, которое увидели пользователи. Этот вид атаки очень досаден для владельцев сайтов и может подорвать доверие к сайту.

Другие хактивистские атаки могут быть гораздо более разрушительными. Например, в ноябре 2012 года децентрализованная международная хактивистская группа Anonymous¹ вывела сетевыми атаками из строя множество израильских сайтов (<https://oreil.ly/Btovx>). В результате любой посетитель затронутых сайтов сталкивался с медленной загрузкой или ошибкой. Распределенные сетевые атаки отправляют жертве поток трафика с тысяч скомпрометированных машин, разбросанных по всему миру. Брокеры этих так называемых бот-сетей часто предоставляют возможность купить их в Интернете. Это делает атаки обыденными и легкими для проведения. В более серьезных случаях злоумышленники могут даже угрожать полностью уничтожить или вывести из строя системы. Поэтому некоторые исследователи называют их кибертеррористами.

В отличие от других типов злоумышленников, хактивисты открыто говорят о своей деятельности и часто публично берут на себя ответственность за атаки. Это может проявляться по-разному, включая размещение в социальных сетях или разрушение систем. Активисты, участвующие в таких атаках, могут даже не быть технически подкованными. Это усложняет прогнозирование или защиту от хактивизма.

Защита ваших систем от хактивистов

Подумайте о том, вовлечен ли ваш бизнес или проект в спорные темы, которые могут привлечь внимание активистов. Например, позволяет ли ваш сайт пользователям размещать собственный контент — блоги или видео? Поднимает ли ваш проект глобальный вопрос, например права животных? Используют ли активисты какие-либо из ваших продуктов, например сервис обмена сообщениями? Если ответ на любой из этих вопросов «да», вам придется задуматься о надежных, многоуровневых мерах безопасности, обеспечивающих защиту систем от уязвимостей и устойчивость к сетевым атакам. Возможность быстрого восстановления системы и ее данных из резервных копий тоже понадобится.

¹ Anonymous — это прозвище, которое многие люди используют для хактивистской (и другой) деятельности. Оно может (или не может) относиться к одному человеку или коллективу связанных лиц, в зависимости от ситуации.

Преступники

Методы сетевых атак используются для совершения преступлений, очень похожих на их нецифровых родичей, например, на мошенничество с использованием личных данных, кражу денег и шантаж. *Преступники* обладают широким спектром технических способностей. Одни могут изощриться и написать собственные инструменты. Другие могут приобретать или одалживать инструменты, которые создают третьи, полагаясь на их простые — «нажми и атакуй» — интерфейсы. На самом деле *социальная инженерия* — обман жертвы, помогающий в атаке, — очень эффективна, несмотря на то что она находится на самом низком уровне сложности. Единственный порог вхождения для совершения большинства преступлений — немного времени, денег и наличие компьютера.

Невозможно рассмотреть полный перечень видов преступной деятельности в цифровой сфере, но мы приведем здесь несколько иллюстративных примеров. Представьте, что вы хотите спрогнозировать действия по слиянию и поглощению, чтобы можно было рассчитывать время определенных сделок с акциями. Трое преступников в Китае воспользовались такой идеей в 2014–2015 годах и заработали несколько миллионов долларов (<https://oreil.ly/F8p9a>), похитив конфиденциальную информацию ничего не подозревающих юридических фирм.

За последние десять лет злоумышленники поняли, что жертвы готовы отдавать деньги, если их конфиденциальные данные находятся под угрозой. *Ransomware* (программа-вымогатель) — это ПО, удерживающее систему или ее информацию в заложниках (обычно путем ее шифрования) до тех пор, пока жертва не заплатит преступнику. Обычно хакеры заражают компьютеры-жертвы этим программным обеспечением (которое часто упаковывается и продается как инструментарий), используя уязвимости, распространяя вымогательское ПО в связке с законным программным обеспечением или обманывая пользователя, заставляя его устанавливать подобное ПО самостоятельно.

Преступная деятельность — это не всегда явные попытки кражи денег. *Stalkerware* (программа-шпион, сталкерское ПО) — программное обеспечение, которое продается всего за 20 долларов, — нацелено на сбор информации о человеке без его ведома. Вредоносное ПО загружается на компьютер или мобильный телефон жертвы либо путем обмана жертвы, либо с помощью прямой установки злоумышленником, имеющим доступ к устройству. После установки программа может записывать видео и аудио. Так как сталкерское ПО часто используется

людьми, близкими к жертве (<https://oreil.ly/xvpFp>), например супругами, такой вид злоупотребления доверием может быть очень эффективным.

Не все преступники работают на себя. Компании, юридические фирмы, политические институты, картели, банды и другие организации нанимают злоумышленников для своих целей. Например, колумбийский хакер (<https://oreil.ly/5sOcj>) заявил, что он был нанят для оказания помощи кандидату в президентской гонке 2012 года в Мексике и на других выборах в Латинской Америке путем кражи оппозиционной информации и распространения дезинформации. В ошеломляющем случае из Либерии сотрудник поставщика услуг мобильной связи Cellcom, по сообщениям, нанял хакера (<https://oreil.ly/aSCRX>) для вывода из строя сети их конкурента — Lonestar. Эти атаки сильно навредили Lonestar, в результате чего компания понесла значительные убытки.

Защита систем от преступников

Проектируя системы, устойчивые к преступной деятельности, имейте в виду, что хакеры ищут самый простой способ достижения своих целей с наименьшими затратами и усилиями. Если ваша система будет достаточно устойчивой, они могут переключиться на другую жертву. Поэтому подумайте, на какие системы они могут ориентироваться и как сделать атаки на ваше ПО затратными. Эволюция систем полностью автоматизированного тестирования Тьюринга (САРТСНА) — хороший пример того, как со временем можно повысить стоимость атак. САРТСНА используются, чтобы определить, человек или автоматизированный бот взаимодействует с сайтом, например, во время входа в систему. Боты — это частый признак вредоносной активности, поэтому важно определить, является ли пользователь человеком. Ранние системы САРТСНА запрашивали проверку с помощью слегка искаженных букв или цифр, которые ботам было трудно распознать. По мере того, как боты становились более изощренными, разработчики САРТСНА начали использовать искаженные изображения и распознавание объектов. Эта тактика направлена на то, чтобы со временем значительно увеличить стоимость атаки САРТСНА¹.

¹ Гонка за повышение эффективности методов САРТСНА продолжается с новыми достижениями, использующими поведенческий анализ пользователей при их взаимодействии с САРТСНА. reCAPTCHA (<https://oreil.ly/C8BXL>) — это бесплатный сервис, который вы можете использовать на своем сайте. Чтобы ознакомиться с обзором литературы об исследованиях, см. книгу Чоу Ян-Вея (Chow Yang-Wei), Вилли Сусило (Willy Susilo) и Пайрата Торнчароэнсри (Pairat Thorncharoensri) *Advances in Cyber Security: Principles, Techniques and Applications*, 2019 г.

Автоматизация и искусственный интеллект

В 2015 году Управление перспективных исследований и разработок Министерства обороны США (Defense Advanced Research Projects Agency, DARPA) объявило конкурс Cyber Grand Challenge (<https://oreil.ly/3ySRv>) на разработку системы искусственного интеллекта, которая могла бы самообучаться и работать без вмешательства человека. Она нужна, чтобы находить недостатки в ПО, разрабатывать способы использования этих недостатков, а затем исправлять их. Семь команд приняли участие в финальном мероприятии в прямом эфире и, не выходя из большого актового зала, наблюдали, как их полностью независимые системы искусственного интеллекта атакуют друг друга. Команде, занявшей первое место, удалось разработать такую самообучающуюся систему!

Успех Cyber Grand Challenge позволяет предположить, что некоторые атаки в будущем могут быть выполнены без участия и контроля людей. Ведутся споры о том, могут ли разумные машины получить достаточно способностей, чтобы научиться атаковать друг друга. Независимые платформы атак заставляют задуматься о более автоматизированной защите, которая, как мы прогнозируем, станет важной областью исследований для будущих разработчиков систем.

Защита систем от автоматических атак

Для противостояния натиску автоматических атак разработчики должны по умолчанию учитывать конструкцию отказоустойчивых систем и иметь возможность автоматически проверять состояние безопасности своих систем. В книге мы рассмотрим такие темы, как автоматическое распределение конфигурации и обоснование доступа в главе 5; автоматическую сборку, тестирование и развертывание кода в главе 14 и обработку сетевых атак в главе 8.

Инсайдеры

В каждой организации существуют *инсайдеры*: нынешние или бывшие сотрудники, которым доверяют внутренний доступ к системам или коммерческую информацию. *Инсайдерский* (или *внутренний*) *риск* — это угроза со стороны таких лиц. Человек становится *внутренней угрозой*, когда может совершать действия вредоносного, небрежного или случайного характера, которые могут навредить организации. Внутренний риск — это обширная тема, которой хватит на несколько книг. Чтобы помочь разработчикам систем, мы кратко рассмотрим эту область, приведя три основные категории (табл. 2.1).

Таблица 2.1. Основные категории инсайдеров и примеры

Первостепенные инсайдеры	Сторонние инсайдеры	Связанные инсайдеры
Сотрудники.	Сторонние разработчики приложений.	Друзья.
Стажеры.	Соавторы открытого исходного кода.	Члены семьи.
Руководители.	Соавторы доверенного контента.	Соседи по комнате
Советы директоров	Коммерческие партнеры.	
	Подрядчики.	
	Поставщики.	
	Аудиторы	

ПЕРЕСЕЧЕНИЕ НАДЕЖНОСТИ И БЕЗОПАСНОСТИ: ВЛИЯНИЕ ИНСАЙДЕРОВ

Когда речь идет о защите от злоумышленников, надежность и безопасность в большей степени пересекаются, если вы проектируете системы так, чтобы они были устойчивыми против инсайдеров. Это пересечение в значительной степени связано с привилегированным доступом инсайдеров к вашим системам.

Большинство инцидентов, связанных с надежностью, происходят от действий инсайдеров, которые не осознают свое влияние на систему — например, предлагая некачественный код или ошибочное изменение конфигурации. Что касается безопасности, если злоумышленник может получить доступ к учетной записи сотрудника, то он может нанести вред вашей системе, как если бы он был этим инсайдером. Любые разрешения или привилегии, которые вы назначаете инсайдерам, становятся доступными злоумышленнику.

Проектируя надежные и безопасные системы, лучше учитывать наличие как благонамеренных инсайдеров, которые могут совершать ошибки, так и нарушителей, которые сознательно могут захватить учетную запись сотрудника. Например, если у вас есть БД с конфиденциальной информацией о клиентах, имеющая решающее значение для вашего бизнеса, вы захотите предотвратить ее случайное удаление сотрудниками во время выполнения работ по техобслуживанию. Также нужно будет защитить информацию от злоумышленника, который может получить доступ к учетной записи сотрудника. Методы наименьшей привилегии, описанные в главе 5, защищают от рисков как надежности, так и безопасности.

Первостепенные инсайдеры

Первостепенные инсайдеры — это люди, собранные вместе по определенной задумке — обычно для непосредственного участия в достижении деловых целей. В эту категорию входят сотрудники, работающие в организации, руководители и члены совета директоров, которые принимают важные для компании

решения. Вы можете представить и других людей, которые тоже попадают в эту категорию. Инсайдеры, имеющие доступ к конфиденциальным данным и системам от первого лица, становятся героями большинства новостных сюжетов о рисках, связанных с инсайдерской деятельностью. Возьмите случай с инженером, работающим на General Electric (<https://oreil.ly/hiKqf>), которому в апреле 2019 года было предъявлено обвинение в краже коммерческих файлов, встраивании их в фотографии с использованием стеганографического ПО (чтобы скрыть их кражу) и отправке их на личный почтовый адрес. Обвинители утверждают, что его целью было получить возможность вместе с деловым партнером в Китае производить недорогие версии турбомашин GE и продавать их китайскому правительству. Подобные истории распространены среди высокотехнологичных фирм, которые производят устройства следующего поколения.

Доступ к персональным данным может быть заманчивым для инсайдеров с извращенными наклонностями, людей, которые хотят почувствовать собственную важность при получении привилегированного доступа, и для людей, которые хотят продавать собранную информацию. В печально известном случае 2008 года (<https://oreil.ly/Em5wF>) несколько медицинских работников были уволены из Медицинского центра Калифорнийского университета в Лос-Анджелесе за неуместный просмотр файлов пациентов, в том числе знаменитостей высокого уровня. Учитывая, что все больше потребителей регистрируются в социальных сетях, сервисах обмена сообщениями и банковских сервисах, защита их данных от несанкционированного доступа со стороны сотрудников становится как никогда важной.

Некоторые из наиболее впечатляющих историй о внутреннем риске связаны с недовольными инсайдерами. В январе 2019 года человек, уволенный за плохую работу, был осужден за удаление 23 виртуальных серверов своего бывшего работодателя (<https://oreil.ly/7avdj>). В результате компания потеряла ключевые контракты и понесла значительные убытки. Почти любая компания, какое-то время находящаяся на рынке, может похвастаться подобными историями. Из-за динамики трудовых отношений такой риск неизбежен.

Предыдущие примеры охватывают сценарии, в которых злоумышленник покушается на безопасность систем и информации. Но сторонние инсайдеры могут влиять и на надежность систем. Например, в предыдущей главе обсуждался ряд неудачных внутренних действий при проектировании, использовании и обслуживании системы хранения паролей, которые препятствовали доступу SR-инженеров к учетным данным в чрезвычайной ситуации. Как мы увидим, умение предусмотреть ошибки, которые могут совершить инсайдеры, жизненно важно для обеспечения целостности системы.

Сторонние инсайдеры

С появлением ПО с открытым исходным кодом и открытых платформ становится ясным, что внутренней угрозой может быть тот, кого в вашей организации видели немногие (или вообще никто). Рассмотрим следующий сценарий: ваша компания разработала новую библиотеку для обработки изображений. Вы решите сделать открытым исходный код библиотеки и принять от общественности предложения по его изменению. Теперь инсайдеры для вас — это не только сотрудники компании, но и соавторы открытого исходного кода. В конце концов, если программист, внесший вклад в открытый исходный код, находится в другой точке мира и вы с ним никогда не встречались, его изменения в коде все равно могут нанести вред людям, использующим вашу библиотеку.

Разработчики программ с открытым исходным кодом редко могут тестировать свой код во всех средах, где он может быть развернут. Дополнения кодовой базы могут привести к непредсказуемым проблемам с надежностью, таким как непредвиденное снижение производительности или проблемы совместимости оборудования. В этом случае стоит внедрить средства контроля, обеспечивающие тщательный анализ и проверку всего представленного кода. Подробнее о лучших решениях в этой области см. в главах 13 и 14.

Вам также нужно тщательно продумать, как расширить функциональность продукта с помощью интерфейсов прикладного программирования (application programming interfaces, API). Предположим, ваша организация разрабатывает платформу управления персоналом с API стороннего разработчика, чтобы компании могли легко расширить функциональность вашего ПО. Если сторонний разработчик имеет привилегированный или специальный доступ к данным, он теперь может стать внутренней угрозой. Тщательно продумайте доступ, который вы открываете через API, и то, что может сделать третья сторона, получив этот доступ. Можно ли ограничить влияние, оказываемое инсайдерами на надежность и безопасность системы?

Связанные инсайдеры

Обычно мы доверяем людям, с которыми живем, но разработчики при работе над защищенными системами часто игнорируют такие отношения¹. Рассмотрим ситуацию, в которой сотрудник забирает свой ноутбук домой в выходные дни. У кого есть доступ к этому устройству, когда оно разблокировано на кухонном столе,

¹ Для примера изучения того, как на функции безопасности и конфиденциальности влияет домашнее насилие со стороны партнера, см. статью Мэтью Тара (Matthews Tara) и др. *Stories from Survivors: Privacy & Security Practices When Coping with Intimate Partner Abuse* (2017). <https://ai.google/research/pubs/pub46080>.

и какое влияние эти люди могут оказать, умышленно или непреднамеренно? Дис-танционная работа, работа на дому и ночное дежурство у мониторов все популярнее в среде технических специалистов. Рассматривая модель внутренней угрозы, обязательно используйте широкое определение «рабочее место», которое включает и дом. Человек за клавиатурой не всегда может быть «типичным» инсайдером.

ОПРЕДЕЛЕНИЕ НАМЕРЕНИЙ ИНСАЙДЕРА

Если система выходит из строя из-за действий инсайдера и он утверждает, что его действия были случайностью, вы ему поверите? Ответить на этот вопрос может быть трудно, а в крайних случаях халатности может быть невозможно окончательно подтвердить или исключить ненамеренность действий. В таких ситуациях часто не обойтись без экспертов вроде представителей юридического отдела вашей организации, отдела кадров и, возможно, даже правоохранительных органов. Прежде всего, при проектировании, запуске и обслуживании систем планируйте как злонамеренные, так и непреднамеренные действия и предполагайте, что разница между ними не всегда известна.

Моделирование внутренних угроз

Есть множество платформ для моделирования внутреннего риска, от простых до тематически специфичных, сложных и подробных. Если вашей организации для начала нужна простая модель, мы можем успешно использовать структуру из табл. 2.2. Эта модель может быть адаптирована для быстрого мозгового штурма или веселой карточной игры.

Таблица 2.2. Основы для моделирования внутренних рисков

Исполнитель/роль	Мотивация	Действия	Цель
Инженеры	Случайность	Доступ к данным	Пользовательские данные
Операторы	Халатность	Утечка (кража) данных	Исходный код
Специалисты по продажам	Компрометация	Удаление	Документы
Юристы	Финансовая выгода	Изменение	Журналы
Маркетологи	Идеология	Внедрение	Инфраструктура
Руководство	Мсть	Передача СМИ	Службы
	Тщеславие		Финансовая отчетность

Сначала определите список *исполнителей/ролей*, которые есть в вашей организации. Попробуйте продумать все *действия*, которые могут причинить вред (включая несчастные случаи), и потенциальные *цели* (данные, системы и т. д.). Вы можете сочетать сущности из каждой категории, чтобы создать много сценариев. Вот несколько примеров, с которых можно начать.

- *Инженеру*, имеющему доступ к *исходному коду*, не нравится отзыв о производительности. Он принимает ответные меры, внедряя вредоносную ловушку в производственный код, которая крадет *пользовательские данные*.
- К *специалисту SRE* с доступом к *ключам шифрования SSL* сайта обращается незнакомец и *настоятельно рекомендует* ему (например, путем угроз в адрес его семьи) передать конфиденциальные материалы.
- *Финансовый аналитик*, подготавливающий *финансовые показатели компании*, работает сверхурочно и случайно изменяет *итоговые показатели годовой выручки* с коэффициентом 1000 %.
- *Ребенок специалиста SRE* использует ноутбук своих родителей дома и загружает игру, в которой установлено *вредоносное ПО*, блокирующее компьютер и не позволяющее *SRE реагировать* на серьезный сбой.

ОШИБКИ ПРИ МОДЕЛИРОВАНИИ УГРОЗЫ

Иногда люди просто делают что-то неверно — человеку свойственно ошибаться. В течение примерно 40 минут 31 января 2009 г. поисковая система Google отображала злоещее предупреждение — «Этот сайт может нанести вред вашему компьютеру» — каждому пользователю, при каждом запросе! Это предупреждение обычно зарезервировано для результатов поиска, ссылающихся на сайт, который либо скомпрометирован, либо содержит вредоносное ПО. Основная причина (https://oreil.ly/Jua6_) этой проблемы оказалась очень простой: символ «/» был неявно (и случайно!) добавлен в системный список сайтов, которые могут установить вредоносное программное обеспечение в фоновом режиме — что соответствует всем сайтам на планете.

Достаточно долго поработав с системами, каждый может столкнуться с такой ситуацией. Эти ошибки могут быть вызваны тем, что вы работаете по ночам, не высыпаясь как следует, и делаете опечатки, или просто непредвиденными действиями в системе. При разработке безопасных и надежных систем помните, что люди ошибаются, и подумайте, как это предотвратить. Автоматическая проверка того, был ли символ «/» добавлен в конфигурацию, предотвратила бы вышеупомянутый сбой.

Проектирование с учетом инсайдерских рисков

В этой книге много стратегий разработки для обеспечения безопасности, которые применимы для защиты от внутреннего риска и злоумышленников «извне». При проектировании систем вы должны учитывать, что человек, имеющий доступ к системе или ее данным, может относиться к любому типу атакующего, описанному в этой главе. Поэтому стратегии выявления и снижения риска обоих типов схожи.

Мы обнаружили, что некоторые концепции особенно эффективны при рассмотрении внутренних рисков.

Наименьшая привилегия

Предоставление наименьших привилегий, нужных для выполнения рабочих обязанностей с точки зрения объема и продолжительности доступа. См. главу 5.

Нулевое доверие

Разработка автоматических или прокси-механизмов для управления системами, чтобы у инсайдеров не было широкого доступа, позволяющего им причинять вред. См. главу 3.

Многосторонняя авторизация

Использование технических средств управления с требованием, чтобы конфиденциальные действия были авторизованы несколькими людьми. См. главу 5.

Обоснование бизнеса

Требование официального документирования причины доступа сотрудников к конфиденциальным данным или системам. См. главу 5.

Аудит и обнаружение

Просмотр всех журналов доступа и обоснований, чтобы убедиться в их уместности. См. главу 15.

Восстанавливаемость

Способность восстанавливать системы после разрушительных действий, например удаления критических файлов или системы недовольным сотрудником. См. главу 9.

Методы атакующего

Как описанные нами злоумышленники обычно атакуют? Это важно знать для того, чтобы понимать, как кто-то может поставить под угрозу ваши системы и как вы можете их защитить. Действия злоумышленников могут показаться вам сложными и сверхъестественными. Предсказать действия конкретного злоумышленника в любой день невозможно из-за разнообразия доступных методов атаки. Мы не можем перечислить здесь все возможные методы. К счастью, разработчики программ и систем используют все более обширное хранилище примеров и структур для решения такой проблемы. В этом разделе мы обсудим некоторые основы для изучения методов атакующего: анализ угроз, цепочки киберубийств (kill chains) и рассмотрим тактики, методы и процедуры (Tactics, Techniques and Procedures, TTP).

Анализ угроз

Многие охранные фирмы дают подробные описания атак, с которыми они сталкивались в реальной жизни. Эта *информация об угрозах* может помочь защитникам систем понять работу реальных злоумышленников и отражать их атаки. Информация об угрозах представлена в нескольких формах, и каждая преследует разные цели.

- *Письменные отчеты* объясняют, как происходили атаки, — это полезно для изучения хода и намерений злоумышленника. Эти отчеты создаются после практических ответных действий. Их качество может быть разным, так как зависит от опыта исследователей.
- *Индикаторы компрометации* (indicators of compromise, IOC) — конечные атрибуты атаки, такие как IP-адрес, на котором злоумышленник размещает фишинговый сайт, или контрольная сумма SHA256 для вредоносного бинарного файла. IOC часто структурированы с использованием общего формата¹ и получены с помощью автоматических каналов, поэтому их можно использовать для программной настройки систем обнаружения.
- *Отчеты о вредоносных программах* помогают понять возможности инструментов злоумышленника и могут стать источником IOC. Эти отчеты

¹ Например, многие инструменты поддерживают язык Structure Threat Information eXpression (STIX), чтобы стандартизировать документацию IOC, которая может передаваться между системами с использованием сервисов вроде Trusted Automated eXchange of Indicator Information (TAXII) (<https://github.com/TAXIIProject>).

создаются экспертами в бинарных файлах *обратного проектирования*, или *реверс-инжиниринга*, обычно с использованием стандартных инструментов торговли, таких как IDA Pro или Ghidra. Исследователи вредоносного ПО тоже используют эти отчеты.

Получая информацию об угрозах от авторитетной компании, занимающейся безопасностью, вы можете лучше понять действия злоумышленников, включая атаки, затрагивающие подобные организации в вашей отрасли. Зная типы атак, с которыми сталкиваются организации вроде вашей, вы будете в силах предотвратить то, с чем когда-нибудь можете столкнуться. Многие фирмы, занимающиеся разведкой угроз, тоже публикуют бесплатные ежегодные сводные отчеты и отчеты о тенденциях¹.

Cyber Kill Chains™

Один из способов подготовки к атакам — определение всех шагов, которые могут понадобиться злоумышленнику для достижения его целей. Некоторые исследователи безопасности используют формализованные структуры, такие как Cyber Kill Chain (цепочка киберубийств)², для соответствующего анализа атак. Подобные структуры могут помочь вам определить формальное развитие атаки, а также рассмотреть защитные средства. В табл. 2.3 показаны этапы предположительной атаки относительно некоторых уровней защиты.

Таблица 2.3. Цепь киберубийств гипотетической атаки

Стадия атаки	Пример атаки	Пример защиты
<i>Разведка:</i> слежка за жертвой, чтобы понять ее слабые места	Атакующий использует поисковую систему, чтобы найти адреса электронной почты сотрудников в целевой организации	Ознакомьте сотрудников с правилами интернет-безопасности

¹ Яркие примеры: ежегодный отчет Verizon по исследованиям распространения данных (<https://oreil.ly/x3hfo>) и ежегодный отчет CrowdStrike о глобальной угрозе (<https://oreil.ly/i1GOs>).

² Cyber Kill Chain (<https://oreil.ly/L0u6I>), задуманная (и зарегистрированная как ТМ) Lockheed Martin, является адаптацией традиционных структур военных атак. Она определяет семь этапов кибератак, но мы обнаружили, что их можно адаптировать. Некоторые исследователи сокращают количество до четырех или пяти ключевых этапов, как мы сделали здесь.

Стадия атаки	Пример атаки	Пример защиты
<i>Вход:</i> получение доступа к сети, системам или учетным записям, нужным для проведения атаки	Атакующий отправляет сотрудникам фишинговые письма, которые приводят к взлому учетных данных. Затем злоумышленник входит в службу виртуальной частной сети (VPN) организации, используя эти учетные данные	Используйте двухфакторную аутентификацию (например, ключи безопасности) для службы VPN. Разрешите VPN-подключения только от систем, управляемых организацией
<i>Горизонтальное движение:</i> перемещение между системами или учетными записями для получения дополнительного доступа	Злоумышленник удаленно входит в другие системы, используя скомпрометированные учетные данные	Разрешите сотрудникам входить только в свои системы. Нужна двухфакторная аутентификация для входа в многопользовательские системы
<i>Постоянство:</i> обеспечение постоянного доступа к скомпрометированным активам	В недавно скомпрометированных системах атакующий создает лазейку, предоставляющую ему удаленный доступ	Используйте белый список приложений, разрешающий запуск только авторизованного ПО
<i>Цели:</i> принятие мер по целям атаки	Злоумышленник крадет документы из сети и использует лазейку удаленного доступа для их экранирования	Включите наименее привилегированный доступ к конфиденциальным данным и мониторинг учетных записей сотрудников

Тактика, методы и процедуры

Методическая категоризация «тактика, методы и процедуры» (Tactics, Techniques and Procedures; TTPs) злоумышленника все чаще используется для систематизации методов атаки. Не так давно научно-исследовательская и техническая корпорация Массачусетского технологического института (MITRE) разработала фреймворк (базу знаний) ATT&CK (<https://attack.mitre.org/>) для более тщательной проработки этой идеи. Если вкратце, то фреймворк разделяет каждый этап Kill Chain на подробные фрагменты и предоставляет формальное описание того, как злоумышленник может выполнить каждый этап атаки. Например, на этапе доступа к учетным данным ATT&CK описывает, что файл `.bash_history` пользователя может содержать случайно введенные пароли, которые злоумышленник может получить, просто прочитав этот файл. Фреймворк ATT&CK предлагает

к рассмотрению сотни (потенциально тысячи) способов, которыми злоумышленники могут пользоваться, и благодаря этому защитники могут создавать препятствие для каждого метода атаки.

Оценка риска

Разобраться в профилях потенциальных противников и используемых ими методах непросто. Оценка риска, связанная с разными злоумышленниками, привела нас к следующим соображениям.

Вы можете не осознавать, что являетесь целью

Не сразу может быть очевидно, что ваша компания, организация или проект является потенциальной целью. Многие организации, несмотря на их размер или род занятий, могут стать объектами атак. В сентябре 2012 года компания Adobe, наиболее известная благодаря ПО, позволяющему создавать контент, сообщила, что злоумышленники проникли в ее сети (<https://oreil.ly/R4-8A>), намереваясь подписать свое вредоносное ПО с помощью официального сертификата подписи программного обеспечения компании. Это позволило злоумышленникам развернуть вредоносное ПО, которое не было обнаружено антивирусами и другими защитными программами. Подумайте, есть ли в вашей организации потенциально интересные для атакующего активы — либо для получения прямой выгоды, либо в рамках более масштабной атаки на кого-то еще.

Успешная атака необязательно должна быть изощренной

Даже если у злоумышленника много ресурсов и навыков, не думайте, что он всегда выберет самые трудные, дорогие или запутанные средства для достижения своих целей. Атакующие выбирают наиболее простые и экономически эффективные методы компрометации системы, соответствующие их целям. Например, многие заметные и эффективные операции по сбору информации основаны на простом фишинге (<https://oreil.ly/11AyT>) — обмане пользователя для получения его пароля. Поэтому, проектируя свои системы, обязательно учитывайте простые основы безопасности (например, использование двухфакторной аутентификации), прежде чем беспокоиться о запутанных и экзотических атаках (например, о лазейках в прошивке).

Не стоит недооценивать противника

Не думайте, что злоумышленник не может получить ресурсы для проведения дорогой или сложной атаки. Тщательно проанализируйте, сколько противник готов потратить на атаку. Необычная история о том, что АНБ

внедряет лазейки в аппаратное обеспечение Cisco, перехватывая поставки на пути к клиентам, показывает, как далеко пойдут для достижения своих целей хорошо финансируемые и талантливые атакующие¹. Но знайте, что эти случаи скорее исключение, чем норма.

Идентификация — это сложно

В марте 2016 года был обнаружен новый тип вымогателя — вредоносная программа Petya («Петя»). Она делает данные или системы недоступными до тех пор, пока жертва не заплатит выкуп. «Петя» оказалась финансово мотивированной. Год спустя исследователи нашли новую вредоносную программу, содержащую многие элементы оригинальной программы «Петя». Новое вредоносное ПО, получившее название NotPetya («НеПетя») (<https://oreil.ly/DGYDx>), очень быстро разошлось по всему миру.

Этот пример показывает, что мотивированные злоумышленники могут творчески скрывать свои мотивы и личность. Как в этом случае, маскируя себя под нечто потенциально более безобидное. Не всегда можно раскрыть личность и намерения злоумышленников. Поэтому мы рекомендуем вам сосредоточиться на том, как они работают (их ТТР).

Злоумышленники не всегда боятся быть пойманными

Даже если вам удастся отследить местонахождение и личность злоумышленника, может быть непросто привлечь его к юридической ответственности (особенно на международном уровне). Тем более если речь идет о государственных субъектах, работающих на правительство, которое может не желать выдавать их.

Резюме

Все атаки на безопасность можно отследить вплоть до личности. Мы рассмотрели некоторые распространенные профили злоумышленников, чтобы помочь вам определить, кто и почему может захотеть нацелиться на ваши сервисы. Это позволит вам расставить приоритеты для защиты.

Оцените, кто может нацелиться на ваше ПО. Каковы ваши активы? Кто покупает ваши товары или услуги? Могут ли ваши пользователи или их действия мотивировать злоумышленников? Как ваши защитные ресурсы сравниваются с наступательными ресурсами ваших потенциальных противников? Даже если

¹ *Greenwald G.* No Place to Hide: Edward Snowden, the NSA and the U.S. Surveillance State. — New York: Metropolitan Books, 2014.

атакующий хорошо финансирован, информация в остальной части книги поможет сделать вас более дорогой целью, чтобы свести на нет стимул для атаки. Не забывайте о менее заметном противнике — анонимность, местоположение, достаточное время и сложность судебного преследования могут сыграть преступнику на руку, позволяя ему нанести вам непропорционально большой ущерб. Подумайте о внутреннем риске, так как все организации сталкиваются как со злонамеренными, так и со случайными потенциальными угрозами со стороны инсайдеров. Предоставляя инсайдерам расширенный доступ, вы позволяете им нанести значительный ущерб.

Следите за угрозами разведки, выпущенными компаниями, которые занимаются безопасностью. Несмотря на эффективность метода многоэтапной атаки, она также открывает несколько точек контакта, где вы можете обнаружить и предотвратить этот процесс. Помните о сложных стратегиях атаки, но не забывайте, что простые, неизощренные атаки, такие как фишинг, могут быть до боли эффективными. Не стоит недооценивать своих противников или собственную ценность в качестве цели.

ЧАСТЬ II

Проектирование систем

Часть II посвящена наиболее рентабельному способу реализации требований безопасности и надежности: сделать это как можно раньше в жизненном цикле разработки ПО — при проектировании систем.

Несмотря на то что при проектировании продукт изначально должен быть надежным и безопасным, большая часть связанной с этими факторами функциональности, которую вы разработаете, будет добавлена в уже существующий продукт. В главе 3 есть пример того, как мы сделали уже работающие системы Google более безопасными и отказоустойчивыми. Вы можете улучшить свои системы, используя подобные модификации. Они будут намного эффективнее в сочетании с некоторыми из следующих принципов проектирования.

В главе 4 рассматривается естественная тенденция пренебрежения надежностью и безопасностью в пользу постоянного ускорения. Мы уверены, что функциональные и нефункциональные требования не обязательно должны противоречить друг другу.

Если вам интересно, с чего начать внедрение принципов безопасности и надежности в ваши системы, обратитесь к главе 5, в которой обсуждается, как

оценивать доступ на основе риска. Затем в главе 6 вы прочтете, как можно анализировать и понимать системы с помощью инвариантов и ментальных моделей. Для идентификации, авторизации и контроля доступа мы рекомендуем пользоваться многоуровневой системной архитектурой. Она базируется на стандартизированных платформах.

Чтобы реагировать на изменяющуюся картину рисков, вам нужно часто и быстро менять инфраструктуру и одновременно поддерживать высокую надежность сервера. В главе 7 приводятся практики, позволяющие приспособиться к краткосрочным, среднесрочным и долгосрочным изменениям, а также к неожиданным осложнениям, которые могут возникнуть при запуске сервиса.

Все эти рекомендации не будут полезны, если система не сможет противостоять серьезной неисправности или сбоям. В главе 8 обсуждаются стратегии поддержания работоспособности системы во время инцидентов, возможно, в ухудшенном режиме. Глава 9 рассматривает системы с точки зрения их восстановления после поломки. Наконец, глава 10 дает сценарий, в котором пересекаются надежность и безопасность, и иллюстрирует некоторые экономически эффективные методы защиты от сетевых атак на каждом уровне набора сервисов.

Пример из практики: безопасные прокси

*Якуб Вармуз и Ана Опреа
с Томасом Мауфером, Сюзанной Ландерс,
Роксаной Лозой, Полом Бланкиншипом
и Бетси Бейер*

Представьте, что злоумышленник хочет нарушить работу ваших систем. Или, может быть, ничего не подозревающий инженер с привилегированной учетной записью по ошибке вносит очень серьезные изменения. Так как вы хорошо понимаете свои системы и они рассчитаны на использование наименьшей привилегии и восстановление, влияние на вашу среду ограничено. При расследовании инцидентов и реагировании на них вы можете определить первопричину и принять соответствующие меры.

Подходит ли этот сценарий для вашей организации? Возможно, не все ваши системы соответствуют описанной картине. Поэтому вам нужен способ сделать работающую систему более безопасной и отказоустойчивой. Использование безопасных прокси — один из способов добиться этого.

Безопасные прокси в производственных средах

В целом прокси помогает удовлетворить новые требования к надежности и безопасности, не требуя сильных изменений в развернутых системах. Вместо изменения существующей системы вы можете просто использовать прокси для маршрутизации соединений, которые в противном случае были бы направлены в систему. Прокси может включать средства контроля, отвечающие новым требованиям к безопасности и надежности. В этом примере мы рассмотрим набор *безопасных прокси*, которые мы используем в Google, чтобы не позволять привилегированным администраторам случайно или злонамеренно вызывать проблемы в производстве.

Безопасные прокси — это структура, позволяющая уполномоченным лицам получать доступ или изменять состояние физических серверов, виртуальных машин

или определенных приложений. В Google мы используем безопасные прокси для проверки, утверждения и выполнения рискованных команд без установления SSH-соединения с системами. Пользуясь этими прокси, мы можем предоставить тонкий доступ для отладки проблем или ограничить количество перезапусков компьютера. Безопасные прокси — единая точка входа между сетями. Они являются ключевыми инструментами, позволяющими выполнять следующее:

- аудит каждой операции на проекте;
- контроль доступа к ресурсам;
- защиту производства от человеческих ошибок.

Zero Touch Prod (https://oreil.ly/_4rAo) — это проект в Google, требующий, чтобы каждое изменение в производственном коде, которое должно быть проведено с помощью автоматизации (а не людей), было предварительно проверено ПО или запущено с помощью надежного механизма аварийного доступа¹. Безопасные прокси входят в число инструментов, которые мы используем для достижения этих целей. По оценкам Google, с помощью Zero Touch Prod можно было бы предотвратить или смягчить около 13 % всех сбоев.

В модели безопасного прокси, представленной на рис. 3.1, вместо прямого общения с целевой системой клиенты общаются с прокси. В Google мы придерживаемся этого поведения, ограничивая целевую систему приемом вызовов только от прокси через конфигурацию. Эта конфигурация указывает, какие вызовы удаленных процедур (remote procedure call, RPC) на уровне приложений могут выполняться какими ролями клиента через списки управления доступом (access control list, ACL). После проверки прав доступа прокси отправляет запрос на выполнение через RPC целевым системам. Обычно в каждой целевой системе есть программа прикладного уровня, которая получает запрос и выполняет его в системе. Прокси регистрирует все запросы и команды, выданные системами, с которыми он взаимодействует.

Мы нашли много плюсов использования прокси для управления системами, будь клиент человеком, автоматизированной программой или и тем и другим. Прокси предоставляют следующее.

- Центральную точку для обеспечения многосторонней авторизации (multi-party authorization, МРА)², где мы принимаем решения о доступе для запросов, взаимодействующих с конфиденциальными данными.

¹ Механизм аварийного доступа позволяет обходить политики, чтобы инженеры могли быстро устранять простои. См. подраздел «Механизм аварийного доступа» в главе 5.

² МРА требует, чтобы еще один пользователь одобрил действие, прежде чем оно будет разрешено. См. подраздел «Многосторонняя авторизация (МРА)» главы 5.

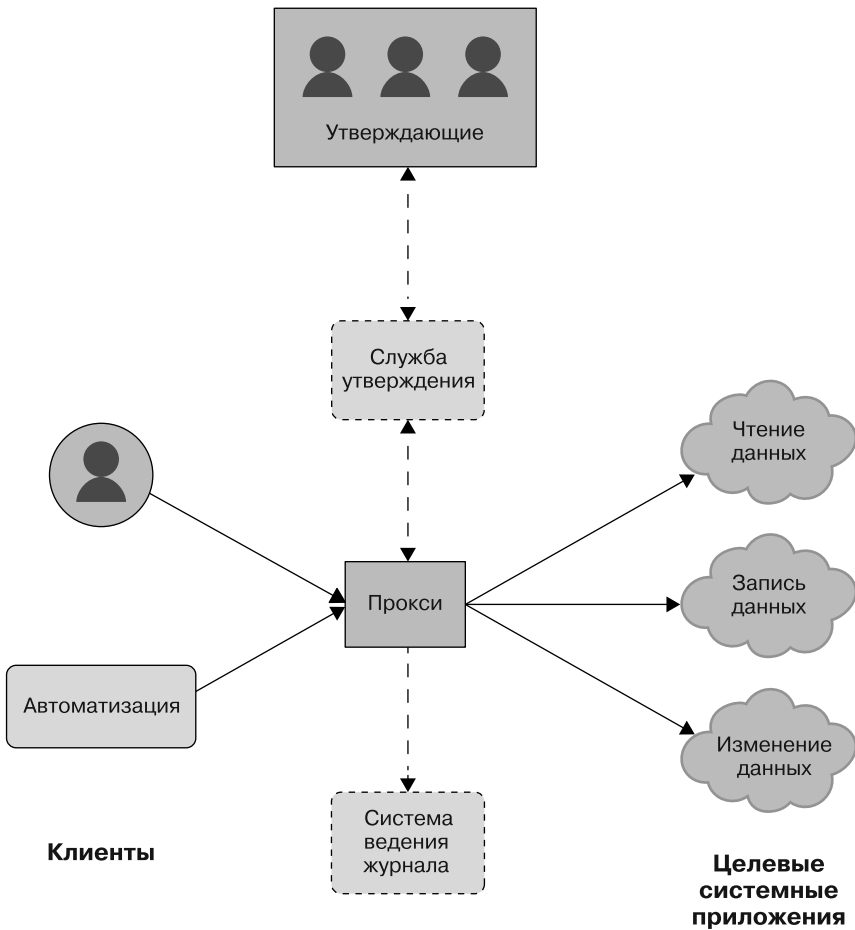


Рис. 3.1. Модель безопасных прокси

- Аудит административного использования, где мы можем отслеживать, когда и кем был выполнен данный запрос.
- Тарифные ограничения, когда изменения вроде перезапуска системы вступают в силу постепенно и мы можем потенциально ограничить радиус взрыва ошибки.
- Совместимость со сторонними целевыми системами с закрытым исходным кодом, где мы контролируем поведение компонентов (которые не можем изменять) с помощью дополнительной функциональности в прокси.
- Интеграция непрерывных улучшений, где мы добавляем усовершенствования безопасности и надежности в центральную прокси-точку.

У прокси есть некоторые недостатки и потенциальные подводные камни.

- Увеличение затрат с точки зрения техобслуживания и эксплуатационных накладных расходов.
- Единственная точка отказа — если либо сама система, либо одна из ее зависимостей недоступна. Мы смягчаем эту ситуацию, запуская несколько экземпляров, тем самым увеличивая избыточность. У всех зависимостей нашей системы абсолютно точно есть соглашение о гарантированном уровне обслуживания (service level agreement, SLA), и у команды, управляющей каждой из зависимостей, есть документированный аварийный контакт.
- Конфигурация политики для контроля доступа, которая сама по себе может содержать ошибки. Мы помогаем пользователям сделать правильный выбор, предоставляя шаблоны или автоматически генерируя настройки, безопасные по умолчанию. Создавая такие шаблоны или автоматизации, мы используем стратегии проектирования, описанные в части II.
- Центральное устройство, которое противник может взять под контроль. Вышеупомянутая конфигурация политики требует, чтобы система пересылала идентификатор клиента и выполняла любые действия от его имени. Сам прокси не дает высоких привилегий, потому что ни один запрос не выполняется под его ролью.
- Соппротивление изменениям, так как пользователи могут захотеть подключиться напрямую к производственным системам. Чтобы уменьшить разногласия, связанные с прокси, мы тесно сотрудничаем с инженерами для обеспечения их доступа к системам через механизм аварийного доступа в чрезвычайных ситуациях. Мы обсуждаем такие темы более подробно в главе 21.

Поскольку безопасный прокси применяется для добавления возможностей безопасности и надежности, связанных с контролем доступа, предоставляемые прокси интерфейсы вместе с целевой системой должны использовать одни и те же внешние API. В результате прокси не влияет на общее взаимодействие с пользователем. Безопасный прокси (если предположить, что он прозрачен) может просто пересылать трафик после выполнения некоторой пред- и постобработки для проверки и ведения журнала. В следующем разделе обсуждается один конкретный пример безопасного прокси, который мы используем в Google.

Tool Proxy в Google

Сотрудники Google выполняют большинство административных операций, пользуясь инструментами интерфейса командной строки (command-line interface, CLI) (<https://oreil.ly/7qk8Q>). Некоторые из них потенциально опасны.

Например, часть инструментов способны отключать сервер. Если такому инструменту передать неправильную область действия, вызов команды может случайно остановить несколько внешних интерфейсов сервиса, что в итоге приведет к отказу. Было бы сложно и дорого отслеживать каждый инструмент CLI, обеспечивать его централизованное ведение журнала и дополнительную защиту конфиденциальных действий. Чтобы решить эту проблему, Google создал *Tool Proxy*: бинарный файл, содержащий общий метод RPC, который внутренне запускает операции в указанной командной строке через копирование и выполнение. Все вызовы контролируются политикой, регистрируются для аудита и могут требовать МРА.

Пользуясь Tool Proxy, можно достичь одной из основных целей Zero Touch Prod: повысить безопасность производственной среды, не позволяя людям напрямую получать к ней доступ. Инженеры не могут запускать произвольные команды прямо на серверах. Вместо этого им нужно связаться с Tool Proxy.

В настройках мы определяем, кому и какие действия разрешены, пользуясь детальным набором политик, выполняющих авторизацию для метода RPC. Политика в примере 3.1 позволяет члену `group:admin` запускать в командной строке последнюю версию `borg` с любым параметром после того, как кто-то из `group:admin-leads` утверждает команду. Экземпляры Tool Proxy обычно развешиваются как задания Borg (<https://oreil.ly/ks1HD>).

Пример 3.1. Политика Tool Proxy в Borg

```
config = {
  proxy_role = 'admin-proxy'
  tools = {
    borg = {
      mpm = 'client@live'
      binary_in_mpm = 'borg'
      any_command = true
      allow = ['group:admin']
      require_mpa_approval_from = ['group:admin-leads']
      unit_tests = [{
        expected = 'ALLOW'
        command = 'file.borgcfg up'
      }]
    }
  }
}
```

Политика в примере 3.1 позволяет инженеру выполнить команду, чтобы остановить задание Borg в рабочей среде со своей рабочей станции, пользуясь такой строкой:

```
$ tool-proxy-cli --proxy_address admin-proxy borg kill ...
```

Эта команда отправляет RPC в прокси по указанному адресу, инициирующему следующую цепочку событий (рис. 3.2).

1. Прокси регистрирует все RPC и выполненные проверки, предоставляя простой способ аудита ранее выполненных административных действий.
2. Прокси проверяет политику, чтобы убедиться, что запускающий команду пользователь находится в `group:admin`.
3. Так как это чувствительная команда, запускается MPA и прокси ожидает авторизации от пользователя из `group:admin-leads`.
4. Если получено подтверждение, прокси выполняет команду, ожидает результата и присоединяет возвращаемый код, стандартный вывод (stdout) и стандартный вывод ошибок (stderr) к ответу RPC.

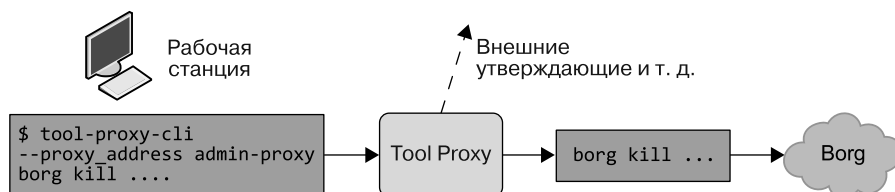


Рис. 3.2. Рабочий процесс использования Tool Proxy

Tool Proxy требует изменений процесса разработки: инженерам нужно добавить к их командам `tool-proxy-cli --proxy_address`. Чтобы привилегированные пользователи не могли обойти прокси, мы изменили сервер, разрешив только административные действия для `admin-proxy` и запретив любые прямые подключения вне ситуаций, в которых требуется аварийный доступ.

Резюме

Использование безопасных прокси — это один из способов добавить в систему журналирование и многостороннюю авторизацию. Так прокси могут сделать ваши системы безопаснее и надежнее. Данный подход может быть экономически эффективным для существующей системы, но он будет более надежным, если также придерживаться других принципов проектирования, описанных далее в части II. Как мы обсудим в главе 4, если вы начинаете новый проект, в идеале стоит построить свою системную архитектуру, используя фреймворки, которые интегрированы с журналированием и модулями контроля доступа.

Компромиссные решения при проектировании

*Кристоф Керн с Брайаном Густафсоном,
Полом Бланкиншипом
и Феликсом Грёбертом*

Часто бывает сложно согласовать потребности в безопасности и надежности с особенностями проекта и требованиями к стоимости. В этой главе обсуждается важность рассмотрения требований безопасности и надежности вашей системы на как можно более раннем этапе разработки ПО.

Мы начнем с разговора о связи между системными ограничениями и функциями продукта. После приведем два примера — сервис обработки платежей и инфраструктуру микросервисов — демонстрирующих общие компромиссы безопасности и надежности. В заключение мы поговорим о естественном стремлении отложить работу над безопасностью и надежностью и обсудим то, как ранние инвестиции в безопасность и надежность могут привести к устойчивому быстройдействию проекта и почему их не стоит откладывать на потом.

Итак, вы собираетесь создать (программный) продукт! В этом сложном путешествии вам придется подумать о многом, начиная от разработки высокоуровневых планов и заканчивая развертыванием кода.

Как правило, вы начинаете с обдумывания будущих функций продукта или сервиса. Это может принять форму высокоуровневой концепции для игры или набора бизнес-требований для облачного приложения, нацеленного на повышение производительности. Также вы разработаете высокоуровневые планы финансирования сервиса.

Чем дальше вы зашли в процессе проектирования, тем конкретнее будут идеи о форме продукта, появятся дополнительные требования и ограничения на разработку и реализацию приложения. Оформятся ваши ожидания

к функциональности продукта. Добавятся общие ограничения, такие как затраты на разработку и эксплуатацию. Вы столкнетесь с запросами и ограничениями в отношении безопасности и надежности. Вашему сервису понадобится определенный уровень доступности и надежности, также могут быть добавлены требования к безопасности, чтобы защитить конфиденциальные пользовательские данные, обрабатываемые приложением.

Некоторые из этих требований и ограничений могут конфликтовать друг с другом, и вам нужно будет найти правильный баланс между ними.

Цели и требования при проектировании

Требования к функциональным возможностям продукта обычно сильно отличаются от требований к безопасности и надежности. Давайте подробнее рассмотрим типы требований, с которыми вы столкнетесь при разработке продукта.

Функциональные требования

Требования к характеристикам (feature), также известные как *функциональные требования*¹, определяют основное назначение сервиса или приложения. Они описывают, как пользователь может выполнить определенную задачу или удовлетворить конкретную потребность. Такие требования часто выражаются в терминах *сценарии использования*, *пользовательские истории* или *пути пользователей* (<https://oreil.ly/yFvEU>) — это последовательности взаимодействий между пользователем и сервисом или приложением. *Критические требования* — это подмножество требований к характеристикам, важным для продукта или сервиса. Если результат проектирования не отвечает критическим требованиям или критической пользовательской истории, жизнеспособного продукта не выйдет.

Принимая проектировочные решения, вы в большей степени будете ориентироваться на требования к характеристикам. В конце концов, вы хотите создать систему или сервис, удовлетворяющие определенным потребностям для задуманной группы пользователей. Между разными требованиями часто приходится принимать компромиссные решения. Имея это в виду, полезно отличать критические требования от других требований к характеристикам.

¹ Более формальный подход см. в руководстве по системному проектированию MITRE (<https://oreil.ly/GD6cY>) и в стандарте ISO/IEC/IEEE 29148-2018(E) (<https://oreil.ly/GD6cY>).

Обычно ряд требований распространяется на все приложение или сервис, часто не отображаясь в пользовательских историях или требованиях к отдельным характеристикам. Вместо этого они указываются один раз в централизованной документации по требованиям или просто подразумеваются. Рассмотрим пример.

Все представления/страницы веб-интерфейса приложения должны:

- следовать общим рекомендациям по визуальному дизайну;
- соответствовать рекомендациям по реализации специальных возможностей;
- иметь нижний колонтитул со ссылками на политику конфиденциальности и условия оказания услуг (Terms of Service, ToS).

Нефункциональные требования

Некоторые категории требований сосредоточены не на специфичном поведении, а на общих атрибутах или поведении системы. Эти *нефункциональные требования* релевантны по отношению к обсуждаемой теме: безопасности и надежности. Например.

- Каковы исключительные обстоятельства, при которых кто-либо (внешний пользователь, агент службы поддержки пользователей или инженер по эксплуатации) может иметь доступ к определенным данным?
- Каков целевой уровень обслуживания (service level objectives, SLO) (<https://oreil.ly/EJNnj>) для таких показателей, как время безотказной работы или задержка отклика 95-го и 99-го процентиля? Как система реагирует при нагрузке выше определенного порога?

Балансируя потребности, лучше одновременно учитывать требования в областях, выходящих за рамки самой системы. Этот выбор может оказать существенное влияние на основные требования к системе. Подобные более широкие области включают следующее.

Эффективность и скорость разработки

Насколько эффективно разработчики могут создавать новые функции с учетом выбранного языка реализации, фреймворков, процессов тестирования и процессов сборки? Как эффективно разработчики могут понимать, модифицировать или отлаживать существующий код?

Скорость развертывания

Сколько времени пройдет от момента разработки функции до момента, когда эта функция станет доступной для пользователей/клиентов?

Характеристики в сравнении с эмерджентными свойствами

В функциональных требованиях прослеживается прямая связь между требованиями, кодом, который им удовлетворяет, и тестами, подтверждающими реализацию. Например.

Спецификация

В пользовательских историях или требованиях может быть указано, как авторизованный пользователь приложения может просматривать и изменять личные данные, связанные с его профилем (например, его имя и контактную информацию).

Реализация

Веб- или мобильное приложение, основанное на этой спецификации, обычно имеет код, конкретно относящийся к данному требованию, например:

- структурированные типы для представления данных профиля;
- код пользовательского интерфейса для представления и разрешения изменения данных профиля;
- серверные обработчики RPC- и HTTP-действий для получения данных профиля зарегистрированного пользователя из хранилища и принятия обновленной информации для записи в хранилище.

Проверка

Обычно проводится интеграционное тестирование, которое шаг за шагом проходит по указанным пользовательским историям. При этом может использоваться тестовый драйвер пользовательского интерфейса для заполнения и отправки формы «Изменить профиль», а после нужно убедиться, что представленные данные отображаются в ожидаемой записи БД. Возможно, для отдельных шагов пользовательской истории есть модульные тесты.

Нефункциональные требования, такие как требования к надежности и безопасности, часто гораздо сложнее определить. Было бы неплохо, если бы у вашего веб-сервера был флаг `--enable_high_reliability_mode` (--включить_режим_высокой_надежности) и, чтобы сделать приложение надежным, вам достаточно было бы просто установить его и дополнительно заплатить хостинг-провайдеру или облачному провайдеру за обслуживание. Но такого флага не существует, и в исходном коде любого приложения нет конкретного модуля или компонента, который «реализует» надежность.

НАДЕЖНОСТЬ И БЕЗОПАСНОСТЬ КАК ЭМЕРДЖЕНТНЫЕ СВОЙСТВА ПРОЕКТИРОВАНИЯ СИСТЕМЫ

Надежность — это прежде всего *эмерджентное*, то есть *независимое свойство* проектирования вашей системы и всего процесса разработки, развертывания и использования. Надежность складывается из таких факторов.

- Как ваш сервис в целом разбит на компоненты — например, микросервисы.
- Как доступность вашего сервиса связана с доступностью/надежностью его зависимостей, включая серверную часть, хранилище и базовую платформу.
- С помощью каких механизмов компоненты взаимодействуют (например, RPC, очереди сообщений или шины событий), как маршрутизируются запросы и как реализованы и настраиваются балансировка и сброс нагрузки.
- Как модульное тестирование, сквозное функциональное тестирование, проверка готовности к производству (production readiness review, PRR) (<https://oreil.ly/POJdF>), нагрузочное тестирование и аналогичные действия по проверке интегрированы в процесс разработки и развертывания.
- Как осуществляется мониторинг (https://oreil.ly/F_iBb) системы, предоставляют ли доступные средства мониторинга, метрики и журналы информацию, которая нужна, чтобы обнаруживать аномалии и сбои и реагировать на них.

Общее состояние безопасности вашего сервиса тоже не возникает из единственного «модуля безопасности». Это независимое свойство того, как спроектированы система и операционная среда, включая следующие компоненты, но не ограничиваясь ими.

- То, как большая система разбивается на подкомпоненты и как организованы доверительные отношения между этими компонентами.
- Языки программирования, платформы и фреймворки приложений/сервисов, на которых разрабатывается приложение.
- То, каким образом проектирование безопасности и проверки реализации, тестирование безопасности и подобные проверочные действия встроены в процесс разработки и развертывания ПО.
- Формы мониторинга безопасности, ведения журнала аудита, обнаружение аномалий и другие инструменты, доступные для ваших аналитиков безопасности и тех, кто реагирует на инциденты.

Найти правильный баланс между этими проектировочными целями сложно. Для обоснованных решений часто нужен большой опыт, но даже хорошо продуманные проектировочные решения в дальнейшем могут оказаться неверными. В главе 7 обсуждается, как подготовиться к неизбежному — пересмотру и адаптации.

Пример: документ о дизайне в Google

В Google используется шаблон проектной документации для управления разработкой новых функций и сбора отзывов от заинтересованных сторон перед началом технического проекта.

Разделы шаблона, относящиеся к надежности и безопасности, напоминают командам о необходимости задуматься о последствиях своего проекта и, если нужно, запустить процессы проверки готовности к производству или безопасности. Обзоры проекта иногда происходят за несколько кварталов до того, как инженеры официально начинают думать о стадии запуска.

ШАБЛОН ДИЗАЙН-ДОКУМЕНТА GOOGLE

Вот разделы шаблона дизайн-документа Google, связанные с надежностью и безопасностью.

Масштабируемость

Как масштабируется ваша система? Учитывайте как увеличение размера данных (если это уместно), так и увеличение трафика (если это уместно).

Рассмотрим текущую ситуацию с оборудованием: добавление дополнительных ресурсов может занять гораздо больше времени, чем вы думаете, или может оказаться слишком затратным. Какие начальные ресурсы вам понадобятся? Ориентируйтесь на высокую нагрузку, но помните, что использование большего количества ресурсов, чем нужно, будет препятствовать расширению вашего сервиса.

Избыточность и надежность

Обсудите, как система будет обрабатывать локальные потери данных и временные ошибки (например, временные сбои) и как каждая из них влияет на вашу систему.

Какие системы или компоненты требуют резервного копирования данных? Как данные резервируются? Как они восстанавливаются? Что происходит между моментом потери данных и моментом их восстановления?

Можно ли продолжать обслуживание в случае частичной потери? Можно ли восстановить только недостающие части резервных копий в хранилище?

Вопросы зависимости

Что произойдет, если зависимости от других сервисов будут недоступны в течение определенного периода времени?

Какие сервисы должны быть запущены для запуска вашего приложения? Не забывайте о тонких зависимостях, таких как разрешение имен с помощью DNS или проверка местного времени.

Вводите ли вы какие-либо циклы зависимости вроде блокировки системы, которая не может работать, если ваше приложение еще не запущено? Если есть сомнения, обсудите ваш вариант использования с командой, работающей над системой, от которой вы зависите.

Целостность данных

Как вы узнаете о повреждении или потере данных в хранилищах данных?

Какие источники потери данных могут быть обнаружены (ошибка пользователя, ошибки приложения, ошибки платформы хранения, сбой сайта/копии)?

Сколько времени нужно, чтобы заметить каждый из этих типов потерь?

Каков ваш план восстановления после каждой потери?

Требования SLA

Какие есть механизмы аудита и мониторинга поддержания уровня обслуживания вашего приложения?

Как вы можете гарантировать заявленный уровень надежности?

Вопросы безопасности и конфиденциальности

Наши системы регулярно атакуются. Подумайте о потенциальных атаках, имеющих отношение к этому проекту. Опишите худший сценарий, к которому они могут привести, а также контрмеры, которыми вы воспользуетесь для предотвращения или смягчения каждой атаки.

Перечислите все известные уязвимости или потенциально небезопасные зависимости.

Если по какой-либо причине ваше приложение не учитывает требования безопасности или приватности, прямо укажите это вместе с причиной.

Как только вы завершите ваш проектный документ, проведите быстрый обзор безопасности проекта. Он поможет избежать системных проблем безопасности, которые могут задержать или заблокировать окончательный обзор безопасности.

Балансировка требований

Атрибуты системы, удовлетворяющие требованиям безопасности и надежности, в основном являются эмерджентными свойствами. Поэтому они могут взаимодействовать как с требованиями к характеристикам, так и друг с другом. В результате становится трудно отдельно рассуждать о компромиссах, связанных с безопасностью и надежностью, как об отдельной теме.

СТОИМОСТЬ ДОБАВЛЕНИЯ НАДЕЖНОСТИ И БЕЗОПАСНОСТИ К СУЩЕСТВУЮЩИМ СИСТЕМАМ

Эмерджентная природа безопасности и надежности означает, что связанные с ними варианты проектирования обычно базовые и по своей природе похожи на основные архитектурные решения. Например, на такие, как использование реляционной БД или базы данных NoSQL для хранения информации, а также использование монолитной либо микросервисной архитектуры. Если система изначально не была разработана с учетом проблем безопасности и надежности, то в дальнейшем их будет сложно обеспечить. Система, не имеющая четко определенных и понятных интерфейсов между компонентами и содержащая запутанный набор зависимостей, скорее всего, будет менее доступна и подвержена ошибкам с последствиями для безопасности (см. главу 6). Никакие тестирования и исправления не смогут этого изменить.

Чтобы обеспечить соблюдение требований к безопасности и надежности в существующей системе, нужно сильно изменить конструкцию, провести серьезный рефакторинг или даже частично переписать код. На это может уйти много денег и времени. Кроме того, такие изменения могут потребоваться под давлением времени в ответ на происшествия с безопасностью или надежностью. Но быстрое внесение серьезных изменений в конструкцию развернутой системы влечет за собой появление еще больших недостатков. Поэтому важно учитывать требования к безопасности и надежности и соответствующие компромиссные решения на ранних этапах планирования программного проекта. В этих обсуждениях должны участвовать команды безопасности и SRE, если они есть в вашей организации.

В этом разделе представлен пример, иллюстрирующий возможные компромиссы. Кое-где мы углубляемся в технические детали, но они не очень важны сами по себе. Все нормативные, правовые и деловые соображения, касающиеся проектирования систем обработки платежей и их функционирования, для этого примера тоже не важны. Главная цель здесь в том, чтобы показать сложные взаимозависимости между требованиями. Другими словами, основное внимание уделяется не мельчайшим деталям по защите номеров кредитных карт, а мыслительному процессу при разработке системы со сложными требованиями к безопасности и надежности.

Пример: обработка платежей

Представьте, что вы создаете онлайн-сервис, который продает виджеты потребителям¹. Спецификация сервиса включает в себя историю пользователя. В ней указано, что пользователь может выбирать виджеты из онлайн-каталога, пользуясь мобильным или веб-приложением. Затем пользователь может приобрести выбранные виджеты, что требует предоставления данных для способа оплаты.

¹ Для целей примера неважно, что именно продается. Интернет-магазину может понадобиться оплата за товары, мобильной компании — оплата за транспортировку. На маркетплейсе можно купить физические товары, которые отправляются потребителям, а сервис заказа еды может облегчить доставку блюд навынос из местных ресторанов.

Соображения безопасности и надежности

Прием платежной информации подразумевает удовлетворение определенных требований к безопасности и надежности при проектировании системы и организационных процессов. Имена, адреса и номера банковских карт — конфиденциальные личные данные, требующие особых мер предосторожности¹. Ваша система должна будет соответствовать нормативным стандартам, в зависимости от применимой юрисдикции. Прием платежной информации может привести к тому, что сервис должен будет соответствовать требованиям отраслевых или нормативных стандартов безопасности, таких как стандарт безопасности данных индустрии платежных карт (PCI DSS) (<https://www.pcisecuritystandards.org/>).

Компрометация этой конфиденциальной пользовательской информации, особенно той, что позволяет идентифицировать личность (personally identifiable information, PII), может иметь серьезные последствия для проекта и даже для всей организации/компании. Вы можете потерять доверие своих пользователей и клиентов, в результате потеряв участие в их бизнесах. В последние годы государственные органы приняли законы и нормативные акты, возлагающие потенциально трудоемкие и дорогостоящие обязательства на компании, которые пострадали от утечек данных. Как отмечалось в главе 1, некоторые компании даже полностью прекратили свою деятельность из-за серьезных происшествий с безопасностью.

В отдельных случаях компромисс более высокого уровня при проектировании продукта может освободить приложение от обработки платежей. Например, продукт может быть приведен к рекламной или финансируемой общественными фондами модели. Для целей нашего примера мы будем придерживаться идеи, что принятие платежей является критическим требованием.

Использование стороннего поставщика услуг для обработки конфиденциальных данных

Часто лучший способ смягчить проблемы безопасности конфиденциальных данных — вообще не хранить эти данные (подробнее об этом — в главе 5). Можно попробовать организовать конфиденциальные данные так, чтобы они никогда не проходили через вашу систему, или проектировать системы так, чтобы они не хранили данные постоянно². Вы можете выбрать один из множества API

¹ См., например, статью: *McCallister E., Grance T., Scarfone K.* NIST Special Publication 800-122. Guide to Protecting the Confidentiality of Personally Identifiable Information (PII). 2010. <https://oreil.ly/T9G4D>.

² Обратите внимание, что уместность этого может зависеть от нормативно-правовой базы вашей организации. Такие вопросы выходят за рамки этой книги.

коммерческих платежных сервисов, чтобы интегрировать их с приложением и обрабатывать платежную информацию, транзакции и связанные с этим задачи (например, меры противодействия мошенничеству) на стороне поставщика.

Преимущества

В зависимости от обстоятельств, используя платежные сервисы, вы можете не накапливать свои знания для устранения рисков, а просто положиться на знания поставщика.

- Ваши системы больше не хранят конфиденциальные данные. Это снижает риск того, что уязвимость в ваших системах или процессах может привести к компрометации данных. Но компрометация на стороне поставщика все еще может поставить под угрозу данные ваших пользователей.
- В зависимости от обстоятельств и применимых требований вы можете упростить ваши договорные и нормативные обязательства в соответствии со стандартами безопасности платежной индустрии.
- Вам не нужно создавать и поддерживать инфраструктуру для защиты данных, находящихся в хранилищах вашей системы. Это может исключить значительный объем разработки и постоянных усилий по поддержке.
- Многие сторонние поставщики платежей предлагают контрмеры против мошеннических транзакций и услуги по оценке платежных рисков. Это может помочь снизить риски мошенничества с платежами без необходимости самостоятельно создавать и поддерживать базовую инфраструктуру.

С другой стороны, используя стороннего поставщика сервисов, вы увеличите свои расходы и риски.

Расходы и нетехнические риски

Очевидно, что поставщику нужно платить. Ваш выбор здесь будет зависеть от объема транзакций — экономически более эффективно будет обрабатывать часть транзакций внутри компании.

Нужно учитывать и стоимость разработки, основанную на зависимости от сторонней организации. Ваша команда должна научиться использовать API поставщика. Также может потребоваться отслеживание изменений/выпусков API по расписанию поставщика.

Риски надежности

Из-за передачи функций обработки платежей к вашему приложению добавляется новая зависимость — от стороннего сервиса. Дополнительные зависимости часто создают дополнительные возможные режимы сбоев. В сторонних

зависимостях вы не всегда можете контролировать такие причины. Например, запрос вашего заказчика «пользователь может купить выбранные им виджеты» может не выполняться, если сервис поставщика платежей недоступен или вышел из строя. Значимость риска зависит от соблюдения поставщиком платежей соглашений об уровне обслуживания (<https://oreil.ly/KZ03g>), которые вы заключили.

Вы можете устранить этот риск, введя в систему избыточность (см. главу 8). В этом случае — добавив альтернативного поставщика платежей, к которому ваш сервис может подключиться при сбое. Такая избыточность создает новые сложности и дополнительные затраты. Из-за того, что у двух поставщиков платежей, скорее всего, разные API, нужно спроектировать систему так, чтобы она могла взаимодействовать с обоими, учитывая все дополнительные инженерные и эксплуатационные затраты. Не стоит забывать и о повышенной подверженности ошибкам или нарушениям безопасности.

Риск надежности также можно снизить с помощью резервных механизмов на вашей стороне. Например, когда платежный сервис недоступен, можно вставить механизм организации очередей в канал связи с поставщиком платежей для буферизации данных транзакции. Это позволит продолжить пользовательскую историю «потока покупок» во время отключения платежного сервиса.

Добавляя механизм ведения очереди сообщений, будьте готовы к дополнительным сложностям и новым режимам сбоев самих очередей. Если очередь сообщений не предназначена для обеспечения надежности (например, она хранит данные только в энергозависимой памяти), вы можете потерять транзакции. Это новый вид риска. Все подсистемы, которые работают только в редких и исключительных обстоятельствах, обычно имеют проблемы с надежностью и скрытые ошибки.

Вы можете использовать более надежную реализацию очереди сообщений. Это могут быть либо системы хранения в памяти, распределенной по нескольким физическим местам, что представляет сложность, либо хранения на постоянном диске. Хранение данных на диске, даже в особых случаях, снова вызывает опасения касательно хранения конфиденциальных данных (риск компрометации, соображения соответствия и т. д.), которых вы пытались избежать в первую очередь. Некоторые платежные данные никогда не должны даже попадать на диск. Это затрудняет применение очереди повторов в этом сценарии, поскольку она основана на постоянном хранении информации.

В таком варианте развития событий вам придется учитывать атаки (в частности, атаки со стороны инсайдеров), намеренно разрывающие связь с поставщиком платежей для активации локального ведения очереди данных транзакции, которая после может быть скомпрометирована.

Так в итоге вы столкнетесь с угрозой безопасности, возникшей в результате вашей попытки снизить риски для надежности, которые появились из-за ваших попыток снизить риск для безопасности!

Риски безопасности

Использование в проекте сторонних сервисов тоже чревато некоторыми проблемами безопасности. Во-первых, вы доверяете конфиденциальные данные клиентов стороннему поставщику. Поэтому вы должны выбрать поставщика, уровень безопасности которого хотя бы равен вашему, а для этого нужно тщательно оценивать поставщиков перед началом работы с ними и в последующие периоды. Это непросто, и существуют сложные договорные, нормативные и юридические аспекты, которые выходят за рамки этой книги и которые нужно знать вашему юрисконсульту.

Во-вторых, для интеграции с сервисом поставщика вам нужно подключить библиотеку поставщика к вашему приложению. Здесь *ваши* системы могут стать уязвимыми из-за возможной уязвимости в этой библиотеке или одной из ее транзитивных зависимостей. Чтобы снизить этот риск, вы можете поместить библиотеку¹ в песочницу и подготовиться к быстрому развертыванию ее обновленных версий (см. главу 7). Чтобы избежать этих проблем, вы можете обратиться к поставщику, не требующему от вас привязки проприетарной библиотеки к вашему сервису (см. главу 6). Использование проприетарных библиотек можно избежать, если поставщик выставляет свой API, пользуясь открытым протоколом вроде REST + JSON, XML, SOAP или gRPC.

Может понадобиться включить библиотеку JavaScript в клиенте вашего веб-приложения для интеграции с поставщиком. Это позволяет избежать передачи данных о платежах через ваши системы даже временно. Вместо этого данные о платежах можно отправлять из браузера пользователя прямо в веб-сервис поставщика. Но эта интеграция вызывает те же проблемы, что и с библиотекой: код библиотеки поставщика работает с полными привилегиями в веб-источнике вашего приложения². Уязвимость в коде или компрометация сервера, обслуживающего эту библиотеку, может привести к компрометации вашего приложения. Чтобы снизить риск, изолируйте функциональность, связанную с платежами, и поместите в отдельный веб-источник или изолированный встроенный фрейм. Но эта тактика потребует от вас использования безопасного механизма межсистемной связи, снова вводящего сложность и дополнительные режимы сбоя. В качестве альтернативы поставщик платежей может предложить интеграцию,

¹ См., например, проект Sandboxed API (<https://oreil.ly/fx86y>).

² Подробнее об этом см.: *Zalewski M. The Tangled Web: A Guide to Securing Modern Web Applications.* — San Francisco, CA: No Starch Press, 2011.

основанную на перенаправлениях HTTP. Но от этого взаимодействие с пользователем может стать менее плавным.

Выбор проектировочных решений, связанных с нефункциональными требованиями, может иметь довольно далеко идущие последствия для технической экспертизы в конкретной области. Мы начали обсуждать компромисс, связанный со снижением рисков обработки платежных данных, а в конце задумались над соображениями, находящимися глубоко в сфере безопасности веб-платформы. Мы также столкнулись с договорными и нормативными проблемами.

Управление противоречиями и согласование целей

При предварительном планировании вы часто можете обеспечить выполнение важных нефункциональных требований, таких как безопасность и надежность, с разумными затратами и не отказываясь от функций. Возвращаясь к рассмотрению вопросов безопасности и надежности в контексте всей системы и процесса разработки и эксплуатации, можно отметить, что эти цели во многом соответствуют общим характеристикам качества ПО.

Пример: микросервисы и фреймворк веб-приложений Google

Рассмотрим развитие внутреннего фреймворка Google для микросервисов и веб-приложений. Основной задачей команды, создававшей этот фреймворк, было упростить разработку и использование приложений и сервисов для крупных организаций. При разработке команда внесла ключевую идею применения статических и динамических *проверок соответствия* для гарантии того, что код приложения соответствует разным руководствам по написанию кода и лучшим практикам. Например, проверка соответствия позволяет удостовериться, что все значения, передаваемые между контекстами параллельного выполнения, имеют неизменяемые типы. Такая практика сильно уменьшает вероятность ошибок при использовании параллелизма. Другой набор проверок соответствия создан для применения изоляционных ограничений между компонентами. Это значительно снижает вероятность того, что изменение одного компонента/модуля приложения приведет к ошибке в другом компоненте.

Приложения, построенные на этом фреймворке, имеют довольно жесткую и четко определенную структуру. Поэтому такая платформа обеспечивает готовую автоматизацию для многих распространенных задач разработки и развертывания — от создания каркаса для новых компонентов до автоматической настройки сред с непрерывной интеграцией (continuous integration, CI) и в значительной

степени автоматизированных развертываний производственного кода. Благодаря этим преимуществам фреймворк стал популярен среди разработчиков Google.

Какое отношение все это имеет к безопасности и надежности? Команда разработки фреймворка сотрудничала с SRE и бригадами безопасности на всех этапах проектирования и внедрения. Это позволило гарантировать, что передовые практики в области безопасности и надежности вплетены в структуру фреймворка, а не просто прикреплены в конце. Такой фреймворк отвечает за решение многих общих задач безопасности и надежности. Аналогичным образом он автоматически настраивает мониторинг рабочих показателей и содержит такие функции надежности, как проверка работоспособности и соответствие SLA.

Например, поддержка веб-приложений фреймворка позволяет обрабатывать самые распространенные типы уязвимостей веб-приложений¹. Благодаря сочетанию разработки API и проверок соответствия кода можно избежать случайного внедрения разработчиками многих распространенных типов уязвимостей в код приложения². Что же касается этих типов уязвимостей, фреймворк выходит за рамки «безопасности по умолчанию». Он полностью берет на себя ответственность за безопасность и гарантирует, что любое приложение, основанное на нем, не подвержено этим рискам. Мы обсудим, как это сделать, в главах 6 и 12.

ПРЕИМУЩЕСТВА НАДЕЖНОСТИ И БЕЗОПАСНОСТИ ФРЕЙМВОРКОВ ДЛЯ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Хорошо отлаженный и широко используемый фреймворк со встроенными функциями обеспечения надежности и безопасности — это беспроигрышный сценарий. Разработчики используют его, так как он упрощает создание приложений и автоматизирует общие задачи, делая повседневную работу проще и продуктивнее. Фреймворк — общая функциональная платформа, где инженеры по безопасности и SR-инженеры могут создавать новую функциональность, добавляя возможности для улучшенной автоматизации и увеличивая общую скорость проекта.

Разработка на основе фреймворков приводит к созданию более безопасных и надежных систем, так как фреймворки автоматически решают общие задачи безопасности и надежности. Кроме того, проверки безопасности и готовности к производству также становятся более эффективными. Если непрерывные сборки и тесты программного проекта зеленые (что указывает на то, что код прошел проверки соответствия на уровне платформы), вы можете быть уверены, что на него не влияют общие проблемы безопасности,

¹ См., например, топ-10 по мнению OWASP (<https://oreil.ly/O0kva>) и топ-25 наиболее опасных ошибок в программном обеспечении по мнению CWE/SANS (<https://oreil.ly/Fm6IJ>).

² См. *Kern C. Securing the Tangled Web* // Communications of the ACM. № 57 (9). P. 38–47. doi: 10.1145/2643134.

уже решенные в фреймворке. Аналогичным образом, останавливая релизы, когда израсходован бюджет ошибок (<https://oreil.ly/tOfnj>), система автоматизации развертывания фреймворка гарантирует, что сервис придерживается своих соглашений об уровне обслуживания (см. главу 16 в книге *Site Reliability Engineering* (<https://landing.google.com/sre/workbook/chapters/canarying-releases/>)). Теперь инженеры по безопасности и SR-инженеры могут сосредоточиться на более интересных задачах на уровне проектирования. Наконец, исправления ошибок и улучшения в коде, являющемся частью централизованно поддерживаемого фреймворка, автоматически распространяются на приложения каждый раз, когда они пересобираются (с учетом обновленных зависимостей) и разворачиваются¹.

Согласование требований к эмерджентным свойствам

Пример с фреймворком показывает, что цели, связанные с безопасностью и надежностью, часто хорошо согласуются с другими целями продукта, особенно с состоянием кода и проекта, а также с удобством сопровождения и долгосрочно установившейся скоростью проекта. Попытка же внедрить цели безопасности и надежности в конце часто приводит к увеличению рисков и затрат.

Приоритеты для безопасности и надежности могут совпадать с приоритетами в других областях.

- Как обсуждается в главе 6, проект системы, позволяющий людям эффективно и точно рассуждать об инвариантах и поведении системы, очень важен для безопасности и надежности. Понятность также является ключевым свойством кода и работоспособности проекта, а также опорой для скорости разработки. Понятную систему легче отлаживать и изменять (в первую очередь без ошибок).
- Проектирование для восстановления (см. главу 9) позволяет количественно определять и контролировать риски, связанные с изменениями и внедрением. Обычно обсуждаемые здесь принципы проектирования позволяют поддерживать более высокую скорость изменений (то есть скорость развертывания), чем можно было бы достичь.
- Безопасность и надежность требуют, чтобы мы проектировали системы с учетом меняющегося ландшафта (см. главу 7). Это делает проект более адаптируемым и позволяет не только оперативно устранять вновь возникающие уязвимости и предотвращать атаки, но и быстрее приспосабливаться к меняющимся требованиям бизнеса.

¹ В Google ПО обычно создается из главного каталога общего репозитория, что автоматически обновляет все зависимости при каждой сборке. См. *Potvin R., Levenberg J. Why Google Stores Billions of Lines of Code in a Single Repository // Communications of the ACM, 2016. № 59 (7). P. 78–87. <https://oreil.ly/jXTZM>.*

Начальная скорость по сравнению с установившейся скоростью

В командах, особенно небольших, есть тенденция откладывать проблемы безопасности и надежности на потом («Мы добавим безопасность и будем беспокоиться о масштабировании после того, как у нас появятся клиенты»). Обычно команды оправдывают это тем, что так будет лучше для «скорости». Они считают, что, потратив время на обдумывание и решение этих проблем, они замедлят разработку и введут недопустимые задержки в свой первый цикл выпуска.

Важно различать *начальную* и *установившуюся скорость*. Решение не учитывать такие критические требования, как безопасность, надежность и ремонтопригодность, в начале проектного цикла и правда может повысить скорость разработки на первом этапе. Но опыт показывает, что это *сильно замедлит вас* потом¹. Затраты на модернизацию конструкции на поздней стадии с учетом требований, которые проявляются как эмерджентные свойства, могут быть очень существенными. А внесение значительных изменений постфактум для устранения рисков безопасности и надежности может привести к еще большим рискам для безопасности и надежности. Поэтому чем раньше вы внедрите безопасность и надежность в свою командную культуру, тем лучше (подробнее об этом см. в главе 21).

Анализ ранней истории Интернета², а также разработки и развития базовых протоколов, таких как IP, TCP, DNS и BGP, позволяет сделать интересный вывод. Надежность и работоспособность сети даже в условиях перебоев в работе узлов³ и надежность связи, несмотря на склонность к сбоям⁴, были первостепенными целями проектирования ранних предшественников современного Интернета, таких как ARPANET.

Но безопасность нечасто упоминается в ранних интернет-публикациях и документации. Ранние сети были закрытыми, а узлами управляли проверенные

¹ См. обсуждение тактического программирования против стратегического в книге: Ousterhout J. A Philosophy of Software Design. — Palo Alto, CA: Yaknyam Press, 2018. Подобные наблюдения делает и Мартин Фаулер (<https://oreil.ly/Lc2eY>).

² См. RFC 2235 (<https://oreil.ly/UHIV6>) и статью: Leiner Barry M. et al. A Brief History of the Internet // ACM SIGCOMM Computer Communication Review, 2009. № 39 (5). P. 22–31. doi: 10.1145/1629607.1629613.

³ Baran P. On Distributed Communications Networks // IEEE Transactions on Communications Systems, 1964. № 12 (1). P. 1–9. doi: 10.1109/TCOM.1964.1088883.

⁴ Roberts L. G., Wessler B. D. Computer Network Development to Achieve Resource Sharing // Proceedings of the 1970 Spring Joint Computing Conference, 1970. P. 543–549. doi: 10.1145/1476936.1477020.

исследовательские и правительственные учреждения. Но в сегодняшнем открытом Интернете это условие не выполняется вообще — в сети действуют самые разные злоумышленники (см. главу 2).

Основополагающие протоколы Интернета — IP, UDP и TCP — не предусматривают ни аутентификацию отправителя, ни обнаружение злонамеренного изменения данных промежуточным узлом в сети. Многие высокоуровневые протоколы, такие как HTTP или DNS, уязвимы для различных атак злоумышленников в Сети. Со временем были разработаны безопасные протоколы или расширения протоколов для защиты от таких атак. Например, HTTPS расширяет HTTP, позволяя передавать данные по аутентифицированному безопасному каналу. На уровне IP IPsec криптографически проверяет подлинность одноранговых сетевых узлов и обеспечивает целостность и конфиденциальность данных. IPsec (<https://oreil.ly/Bie7A>) можно использовать для установки VPN-соединения через ненадежные IP-сети.

Однако широкое развертывание этих безопасных протоколов оказалось сложным. Интернет существует примерно 50 лет, и его значительное коммерческое использование началось примерно 25–30 лет назад. Но несмотря на это, большая часть веб-трафика все еще не использует HTTPS¹.

Возьмем другой пример компромисса между начальной и устоявшейся скоростью (в данном случае вне области безопасности и надежности). Рассмотрим процессы гибкой разработки (<https://oreil.ly/upg-w>). Основная их цель — повышение скорости разработки и развертывания — уменьшение задержки между созданием спецификации функции и развертыванием. Но гибкие рабочие процессы обычно основаны на достаточно зрелых методах модульного и интеграционного тестирования и определенной инфраструктуре непрерывной интеграции, требующей предварительных вложений в обмен на долгосрочные выгоды для скорости и стабильности.

В более общем смысле вы можете выбрать начальную скорость проекта и разработать первую итерацию веб-приложения без тестов и с процессом выпуска, который сводится к копированию архивов на производственные хосты. Так вы получите свою первую демоверсию быстро, но к третьему выпуску ваш проект, скорее всего, будет отставать от графика и окажется обременен техническими долгами.

Ранее мы затронули соответствие между надежностью и скоростью. Вложения в зрелый процесс непрерывной интеграции/непрерывного развертывания (continuous integration/continuous deployment, CI/CD) и инфраструктуру

¹ Felt A. P., Barnes R., King A., Palmer C., Bentzel C., Tabriz P. Measuring HTTPS Adoption on the Web // Proceedings of the 26th USENIX Conference on Security Symposium, 2017. P. 1323–1338. <https://oreil.ly/G1A9q>.

поддерживают частые производственные выпуски с управляемым и приемлемым риском надежности (см. главу 7). Но для настройки такого процесса нужны предварительные вложения:

- покрытие модульными и интеграционными тестами, достаточно надежное для обеспечения относительно низкого риска появления ошибок в производственных выпусках, не требующих серьезной работы квалифицированных специалистов;
- надежный конвейер CI/CD;
- часто используемая надежная инфраструктура для поэтапного развертывания и отката производственного кода;
- архитектура ПО, допускающая несвязанное развертывание (<https://oreil.ly/8E04K>) кода и конфигураций (например, «флаги функций»).

Такие инвестиции обычно небольшие, если осуществляются на ранних этапах проектирования. От разработчиков требуется лишь постоянно поддерживать хороший уровень тестирования и «зеленых» сборок. Если же процесс разработки характеризуется плохой автоматизацией тестирования, опирается на ручные этапы развертывания и циклы выпуска у него длительные, то чем сложнее будет становиться проект, тем медленнее будет разработка. На этом этапе улучшение системы тестирования и автоматизация выпусков обычно требуют много работы сразу и могут еще сильнее замедлить ваш проект. Кроме того, тесты, измененные для зрелой системы, чаще могут попасть в ловушку использования текущего ошибочного поведения, нежели правильного, предполагаемого поведения.

Такие вложения выгодны для проектов любых размеров. Но крупные организации могут ощутить большие преимущества в масштабе, так как затраты по многим проектам могут быть амортизированы. Тогда для инвестиций в отдельный проект нужно лишь использовать централизованно поддерживаемые фреймворки и рабочие процессы.

Выбирая ориентированные на безопасность решения, способствующие постоянно поддерживаемой скорости, отдавайте предпочтение фреймворку и рабочему процессу, которые обеспечивают защиту от соответствующих классов уязвимостей на уровне конструкции. Такой выбор поможет снизить или даже устранить риск внедрения подобных уязвимостей во время разработки и обслуживания кодовой базы вашего приложения (см. главы 6 и 12). Для этого не нужны значительные предварительные вложения. Следует только приложить дополнительные усилия по соблюдению ограничений фреймворка. Так вы сможете снизить риск незапланированных сбоев системы или ответных мер по безопасности, приводящих к беспорядку в графиках развертывания. Повышается вероятность того, что ваши проверки безопасности и проверки готовности к производству пройдут гладко.

Резюме

Проектировать и создавать безопасные и надежные системы непросто, ведь безопасность и надежность — это неотъемлемые свойства всего процесса разработки и эксплуатации. Приходится учитывать множество сложных вещей, большинство из которых на первый взгляд кажутся не относящимися к основным требованиям к характеристикам вашего сервиса.

В процессе проектирования вам придется пойти на большое количество компромиссов между безопасностью, надежностью и функциональными требованиями. Может показаться, что в большинстве случаев эти компромиссы будут противоречить друг другу. Вам захочется избежать проблем на ранних стадиях проекта и «разобраться с ними позже», но нередко такое решение приводит к большим затратам и риску для проекта: если сервис работает, то надежность и безопасность не являются обязательными. Если сервис не работает, вы можете потерять бизнес. А если сервис будет скомпрометирован, вам придется работать в режиме аврала, чтобы выйти из этой ситуации. Но при хорошем планировании и тщательном проектировании часто можно сбалансировать все три аспекта. Можно даже сделать это при небольших первоначальных дополнительных затратах и часто с меньшими инженерными усилиями в течение срока службы системы.

ГЛАВА 5

Принцип наименьших привилегий при проектировании

*Оливер Барретт, Аарон Джойнер
и Рори Уорд с Гаем Фишманом
и Бетси Бейер*

Принцип минимальных (наименьших) привилегий гласит, что пользователи должны иметь минимальный доступ, необходимый для выполнения задачи, независимо от того, предоставляется он людям или системам. Эффективнее всего добавить эти ограничения в начале жизненного цикла разработки, на этапе проектирования новых функций. Чем больше ненужных привилегий, тем шире пространство для возможных ошибок, уязвимостей или компромиссов. Это также приводит к рискам безопасности и надежности, которые дорого содержать или минимизировать в работающей системе.

В этой главе мы обсудим, как классифицировать доступ на основе риска, и рассмотрим лучшие практики для обеспечения наименьших привилегий. На примере распределения конфигурации можно увидеть компромиссы, которые эти методы влекут за собой в реальных условиях. После детального описания политики аутентификации и авторизации мы перейдем к расширенным средствам контроля авторизации, помогающим снизить риск взлома учетных записей, а также риск ошибок, допущенных дежурными инженерами. В конце мы обговорим компромиссы и противоречия, которые могут возникнуть при проектировании с наименьшими привилегиями. В идеальном мире люди, использующие системы, имеют благие намерения и выполняют свои рабочие задачи безопасно и без ошибок. К сожалению, в реальности все иначе. Инженеры могут совершать ошибки, их учетные записи могут быть скомпрометированы и т. д. Поэтому важно проектировать системы с наименьшими привилегиями: системы должны ограничивать доступ пользователей к данным и сервисам, нужным для выполнения поставленной задачи.

Компании часто надеются на благонамеренность инженеров и полагаются на них, чтобы они безупречно выполняли сложнейшую работу. Это крайне неразумно. В качестве упражнения подумайте о том ущербе, который вы можете нанести своей организации, если *захотите* сделать что-то плохое. Что вы могли бы сделать? Как бы вы это сделали? Обнаружили бы вас? Смогли бы вы замести следы? Или, даже если ваши намерения были благими, какую худшую

ошибку вы (или кто-то с таким же доступом) могли бы совершить? Сколько спонтанных, вводимых вручную команд используете вы или ваши коллеги при отладке, реагировании на сбой или экстренном исправлении неполадок? Может ли одна опечатка или ошибка при копировании и вставке вызвать новый сбой или ухудшить ситуацию?

Человек несовершенен, поэтому всегда нужно быть готовыми к худшему варианту развития событий. Мы рекомендуем постараться свести на нет влияние этих плохих действий при проектировании системы.

Даже если мы уверены в людях, получающих доступ к нашим системам, нужно ограничить их привилегии и доверие к их учетным данным. Всегда что-то может пойти не так. Люди будут совершать ошибки, делать опечатки в командах, рисковать и открывать фишинговые письма. Нельзя ожидать совершенства. Другими словами — цитируя максиму SRE — надежда не является стратегией.

Концепции и терминология

Прежде чем мы углубимся в лучшие практики проектирования и эксплуатации системы контроля доступа, создадим рабочие определения для некоторых специальных терминов, используемых в отрасли и Google.

Наименьшая привилегия

Наименьшая привилегия — это широкая концепция, хорошо зарекомендовавшая себя в индустрии безопасности. Лучшие практики высокого уровня в этой главе могут заложить основы системы, предоставляющей наименьшие привилегии, нужные для любой задачи или пути действий. Эта цель относится к людям, автоматизированным задачам и отдельным устройствам, входящим в распределенную систему. Цель наименьших привилегий должна распространяться на все уровни аутентификации и авторизации системы. В частности, наш рекомендуемый подход отвергает распространение неявных полномочий на инструментарий (как показано в разделе «Пример из практики: распределение конфигурации» далее в главе) и работает для обеспечения того, чтобы пользователи не имели внешних полномочий (https://oreil.ly/ff_EA) — например, возможности войти в систему с правами root — насколько это возможно.

Сеть с нулевым доверием

Принципы проектирования, которые мы обсуждаем, начинаются с создания *сети с нулевым доверием*. Это представление о том, что сетевое местоположение пользователя (находящегося в сети компании) не предоставляет никакого

привилегированного доступа. Например, подключение к сетевому порту в конференц-зале не дает большего доступа, чем подключение из других источников в Интернете. Вместо этого система предоставляет доступ на основе комбинации учетных данных пользователя и учетных данных устройства — того, что мы знаем о пользователе, и того, что мы знаем об устройстве. Google успешно внедрил крупномасштабную модель сети с нулевым доверием через свою программу BeyondCorp (<https://oreil.ly/8x9OJ>).

Zero Touch

Организация SRE в Google работает над созданием концепции наименьших привилегий через автоматизацию для перехода к тому, что мы называем интерфейсами Zero Touch (нулевые касания). Конкретная цель этих интерфейсов, таких как Zero Touch Production (ZTP), описанный в главе 3, и Zero Touch Networking (ZTN), в том, чтобы сделать Google более безопасным и сократить простои, исключив прямой доступ человека к производственным ролям. Вместо этого люди имеют косвенный доступ к производству через инструменты и автоматизацию, вносящие предсказуемые и контролируемые изменения в производственную инфраструктуру. Для такого подхода нужны обширная автоматизация, новые безопасные API и устойчивые многосторонние системы подтверждения.

Классификация доступа на основе риска

Любая стратегия по снижению риска всегда нуждается в компромиссах. Снижение риска, создаваемого людьми, влечет за собой дополнительные меры контроля или инженерные работы и может привести к негативным последствиям со стороны производительности. Это может увеличить время проектирования, объем изменений процесса, эксплуатационные работы или издержки. Ограничить эти расходы можно, четко определив сферу охвата и приоритеты того, что нужно защитить.

Не все данные и действия созданы равными. Структура вашего доступа может быть разной в зависимости от характера системы. Поэтому не нужно защищать весь доступ в одинаковой степени. Для применения наиболее подходящих средств контроля и избегания принципа «все или ничего» нужно классифицировать доступ, основываясь на воздействии, риске безопасности и/или критичности. Например, может быть, придется по-разному обрабатывать доступ к разным типам данных (общедоступные данные, данные компании, данные пользователей и криптографические секреты). Точно так же нужно относиться и к административным API, которые могут удалять данные иначе, чем API чтения для конкретного сервиса.

Ваши классификации должны быть четко определены, должны последовательно применяться и широко пониматься, чтобы люди могли разрабатывать системы и сервисы, «говорящие» на этом языке. Структура классификации будет зависеть от размера и сложности вашей системы. Возможно, вам понадобится только два или три типа, основанных на специальной маркировке, а может понадобится надежная и программируемая система для классификации частей вашей системы (группировки API, типы данных). Эти классификации могут применяться к хранилищам, API, сервисам или другим объектам, к которым пользователи могут обращаться в ходе работы. Убедитесь, что ваша платформа может обрабатывать наиболее важные объекты в системах.

После создания основы классификации нужно рассмотреть возможность управления для каждого элемента. Для этого учитывайте следующее.

- Кто должен иметь доступ?
- Насколько тщательно этот доступ должен контролироваться?
- Какой тип доступа нужен пользователю (чтение/запись)?
- Какие существуют средства контроля инфраструктуры?

Например, как показано в табл. 5.1, компании могут потребоваться три классификации: *открытая*, *конфиденциальная* и *особо секретная*. Компания может классифицировать средства контроля безопасности как *низкорисковые*, *средне-рисковые* или *высокорисковые* по уровню ущерба, который может нанести доступ, предоставленный ненадлежащим образом.

Таблица 5.1. Примеры классификаций доступа, основанных на риске

	Описание	Доступ на чтение	Доступ на запись	Доступ к инфраструктуре
Открытая	Открыта для всех в компании	Низкий риск	Низкий риск	Высокий риск
Конфиденциальная	Ограничена для групп с бизнес-целями	Средний/высокий риск	Средний риск	Высокий риск
Особо секретная	Нет постоянного доступа	Высокий риск	Высокий риск	Высокий риск

Должна быть административная возможность обойти обычные средства контроля доступа. Например, возможность понизить уровни ведения журнала, изменить требования к шифрованию, получить прямой SSH-доступ к компьютеру, перезапустить и перенастроить параметры сервиса или как-то иначе повлиять на доступность сервиса (-ов).

ПЕРЕСЕЧЕНИЯ НАДЕЖНОСТИ И БЕЗОПАСНОСТИ: РАЗРЕШЕНИЯ

С точки зрения надежности вы могли бы начать со средств контроля инфраструктуры. Нужно убедиться, что неавторизованные участники не могут завершать задачи, изменять списки управления доступом и нарушать настройки сервисов. Но имейте в виду, что с точки зрения безопасности чтение конфиденциальных данных часто тоже может нанести вред: чрезмерно широкие разрешения на чтение могут привести к массовой утечке данных.

Стремитесь к тому, чтобы создать структуру доступа, с помощью которой можно применять соответствующие средства контроля с правильным балансом производительности, безопасности и надежности. Наименьшая привилегия должна применяться ко всем видам доступа к данным и действиям. Работая с этой основополагающей структурой, давайте обсудим, как проектировать системы с учетом принципов и средств контроля с наименьшими привилегиями.

Лучшие практики

Для реализации модели с наименьшими привилегиями мы предлагаем несколько рекомендаций.

Небольшие функциональные API

Заставьте каждую программу хорошо выполнять одну задачу. Чтобы выполнить новую задачу, создавайте другие, а не усложняйте старые программы, добавляя новые функции.

Макэлрой, Пинсон и Таг (1978)¹

Как следует из этой цитаты, культура Unix сосредоточена вокруг небольших и простых инструментов, которые можно комбинировать. Поскольку современные распределенные вычисления превратились из единых вычислительных систем с разделением времени 1970-х годов в глобальные распределенные системы, подключенные к сети, советы авторов по-прежнему актуальны более

¹ *McIlroy M. D., Pinson E. N., Tague B. A. UNIX Time-Sharing System: Foreword // The Bell System Technical Journal, 1978. № 57 (6). P. 1899–1904. doi: 10.1002/j.1538-7305.1978.tb02135.x.*

чем 40 лет спустя. Чтобы адаптировать эту цитату к текущей вычислительной среде, можно сказать: «Сделайте так, чтобы каждая конечная точка API хорошо выполняла одну задачу». При создании систем, ориентирующихся на безопасность и надежность, избегайте открытых интерактивных интерфейсов. Вместо этого создавайте небольшие функциональные API. Такой подход позволяет применять классический принцип безопасности с наименьшими привилегиями и предоставлять минимальные разрешения, которые нужны для выполнения определенной функции.

Что именно мы подразумеваем под API? Каждая система имеет API: это просто пользовательский интерфейс, который она представляет. Одни API очень большие (например, POSIX API¹ или Windows API²), другие относительно небольшие (например, memcached³ и NATS⁴), а третьи просто крошечные (например, World Clock API (<http://worldclockapi.com/>), TinyURL⁵ и Google Fonts API⁶). Когда мы говорим об API распределенной системы, мы просто подразумеваем совокупность всех способов, которыми можно запрашивать или изменять ее внутреннее состояние. Проектирование API хорошо освещено в компьютерной литературе⁷. В этой главе вы узнаете, как можно проектировать и безопасно поддерживать защищенные системы, предоставляя конечным точкам API несколько четко определенных примитивов. Например, входные данные, которые вы оцениваете, могут быть операциями CRUD (Create, Read, Update, Delete — создание,

¹ POSIX — аббревиатура для Portable Operating System Interface (переносимый интерфейс ОС), стандартизированный интерфейс IEEE, предоставляемый большинством вариантов Unix. Общий обзор см. в Википедии (<https://oreil.ly/4vLTI>).

² Windows API включает в себя знакомые графические элементы и программные интерфейсы, такие как DirectX (<https://oreil.ly/TnmPu>), COM (<https://oreil.ly/PzTUF>) и т. д.

³ Memcached (<https://memcached.org/>) — это высокопроизводительная система кэширования объектов с распределенной памятью.

⁴ NATS (<https://oreil.ly/baCc4>) — пример базового API, построенного поверх текстового протокола, в отличие от сложного интерфейса RPC, такого как gRPC.

⁵ API TinyURL.com (<http://tinyurl.com/>) недостаточно документирован, но, по сути, это один GET-запрос для URL. Он возвращает сокращенный URL в качестве тела ответа. Это редкий пример изменяемого сервиса с крошечным API.

⁶ Fonts API (<https://oreil.ly/-Z4F4>) просто перечисляет доступные в данный момент шрифты. Он имеет ровно одну конечную точку.

⁷ Для начала мы бы посоветовали книгу: Гамма Э. и др. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер, 2021. См. также: Bloch J. How to Design a Good API and Why It Matters // Companion to the 21st ACM SIGPLAN Symposium on Object-Oriented Programming Systems, Languages and Applications, 2006. P. 506–507. doi: 10.1145/1176617.1176622.

чение, обновление и удаление) с уникальным идентификатором, а не API, который принимает язык программирования.

В дополнение к пользовательскому API обращайтесь особое внимание на административный API. Он не менее важен (возможно, даже более важен) для надежности и безопасности вашего приложения. Опечатки и ошибки при использовании этих API приводят к катастрофическим сбоям или раскрытию огромных объемов данных. Поэтому административные API наиболее привлекательны для атаки злоумышленников.

Административные API доступны только внутренним пользователям и инструментам. По этой причине, по сравнению с пользовательскими API, их можно быстрее и проще менять. Тем не менее, после того как внутренние пользователи и инструменты начнут создавать что-то на любом API, его изменение все равно будет затратно. Поэтому мы рекомендуем внимательно рассмотреть их проектирование. Административные API включают в себя следующее:

- API установки/разборки, такие как те, что используются для создания, установки и обновления ПО, или предоставления контейнера (-ов), в котором (-ых) оно запускается;
- API обслуживания и аварийного реагирования, такие как административный доступ для удаления поврежденных пользовательских данных или состояния, либо перезапуска некорректно работающих процессов.

Имеет ли значение размер API, когда речь заходит о доступе и безопасности? Рассмотрим уже знакомый пример: POSIX API — мы его уже упоминали как очень большой API. Этот API популярен, потому что он гибок и многим знаком. Как API управления производственными машинами, он чаще всего используется для относительно четко определенных задач, таких как установка пакета ПО, изменение файла конфигурации или перезапуск процесса демона.

Пользователи часто выполняют традиционную настройку и обслуживание хоста Unix¹ через интерактивный сеанс OpenSSH или с помощью инструментов, пишущих сценарии для API POSIX. Оба подхода предоставляют весь API POSIX для вызывающей стороны. Может быть сложно ограничить и проверить действия пользователя во время этого интерактивного сеанса. Это важно, если пользователь умышленно пытается обойти средства контроля или если подключенная рабочая станция взломана.

¹ Хотя мы используем хосты Unix как пример, этот шаблон не уникален для Unix. Традиционная установка хоста Windows и управление им происходит по такой же модели, где интерактивное представление Windows API обычно осуществляется через RDP вместо OpenSSH.

РАЗВЕ Я НЕ МОГУ ПРОСТО ПРОВЕРЯТЬ ИНТЕРАКТИВНЫЙ СЕАНС?

Многие простые подходы к аудиту, такие как захват команд, выполняемых `bash`, или перенос интерактивного сеанса в `script(1)`, могут показаться достаточными и всесторонними. Но такие варианты ведения журнала для интерактивных сеансов похожи на замки на дверях дома: они могут защитить только от добросовестных людей. К сожалению, злоумышленник, осведомленный об их существовании, может легко обойти большинство этих типов базового аудита сеанса.

Приведем пример — злоумышленник может обойти ведение журнала истории команд `bash`, открыв редактор, такой как `vim`, и выполнив команды из интерактивного сеанса (например, `!/bin/evilcmd`). Эта простая атака усложняет ваши попытки проверять вывод журнала `typescript` с помощью `script(1)`. Так происходит потому, что вывод будет искажен символами визуального контроля, используемыми `vim` и `ncurses` для отображения среды визуального редактирования. Существуют и более сильные механизмы, такие как захват некоторых или всех системных вызовов в фреймворке аудита ядра (`kernel's audit framework`). Но нужно понимать недостатки примитивных подходов и трудности в реализации более продвинутых.

Вы можете использовать разные механизмы, чтобы ограничить разрешения, предоставляемые пользователю через API POSIX¹, но эта необходимость — сильный недостаток предоставления очень большого API. Будет лучше сократить и разложить этот большой административный API на более мелкие части. После вы можете следовать принципу наименьших привилегий для предоставления разрешения только на конкретные действия, требуемые каким-либо конкретным вызывающим пользователем.



Не путайте открытый API POSIX с API OpenSSH. Можно использовать протокол OpenSSH и его средства контроля аутентификацией, авторизацией и аудитом (`authentication, authorization, auditing, AAA`), не раскрывая весь API POSIX, например, используя `git-shell` (<https://oreil.ly/gN12->).

Механизм аварийного доступа

Названный в честь оповещения о срабатывании пожарной сигнализации, указывающего пользователю «разбить стекло в экстренной ситуации», *механизм разбивания стекла* (*breakglass mechanism*), или *механизм аварийного доступа*, обеспечивает доступ к системе в чрезвычайной ситуации и полностью обходит

¹ Популярные варианты включают запуск в качестве непривилегированного пользователя и затем кодирование разрешенных команд через `sudo`, предоставляющего только необходимые `capabilities(7)` (<https://oreil.ly/7Zf70>) или использование фреймворка, такого как SELinux.

систему авторизации. Это может быть полезно для восстановления после непредвиденных обстоятельств. Чтобы получить дополнительную информацию, см. подразделы «Диагностика отказа в доступе» и «Механизмы постепенного отказа и аварийного доступа» далее.

Аудит

Аудит служит в первую очередь для обнаружения неправильного использования авторизации. Здесь могут учитываться действия системного оператора, злоупотребляющего своими полномочиями, компрометация учетных данных пользователя внешним исполнителем или вредоносное ПО, предпринимающее неожиданные действия против другой системы. Ваша способность проводить аудит и осмысленно обнаруживать сигнал среди шума во многом зависит от конструкции проверяемых систем.

- Насколько детализировано решение о контроле доступа, которое принимается или обходится? (Что? Где?)
- Как точно вы можете захватить и зафиксировать метаданные, связанные с запросом? (Кто? Когда? Почему?)

Следующие советы помогут разработать надежную стратегию аудита. В конечном счете ваш успех также будет зависеть от культуры, связанной с аудитом.

Сбор хороших журналов аудита

Использование небольших функционирующих API (как описано в подразделе «Небольшие функциональные API» выше в этой главе) обеспечивает хорошее преимущество возможностям аудита. Наиболее полезные журналы аудита фиксируют ряд детализированных действий, таких как «отправлена конфигурация с криптографическим хешем 123DEAD...BEEF456» или «выполнена команда <x>». Размышления о том, как отображать и обосновывать ваши административные действия для ваших клиентов, также могут помочь сделать ваши журналы аудита более описательными и, следовательно, более полезными внутри компании. Детальная информация журнала аудита четко показывает, какие действия пользователь совершал или не совершал — но обязательно сосредоточьтесь на захвате *полезных* частей действий.

Для исключительных обстоятельств нужен исключительный доступ. Это требует наличия сильной культуры аудита. Если вы обнаружите, что существующие небольшие функциональные плоскости API недостаточны для восстановления системы, у вас есть два варианта.

- Предоставить функциональность аварийного доступа. Тогда пользователь сможет открыть интерактивный сеанс для гибкого API.
- Разрешить пользователю иметь прямой доступ к учетным данным и исключить надлежащий аудит их использования.

В любом из этих случаев вы, возможно, не сможете создать подробный контрольный журнал. Регистрация того, что пользователь открыл интерактивный сеанс с большим API, не говорит о том, что конкретно он сделал. Мотивированный и знающий инсайдер может легко обойти многие решения, которые фиксируют журналы интерактивных сеансов, такие как запись истории команд `bash`. Даже если вы сможете записать полную расшифровку сеанса, эффективный аудит может оказаться довольно сложным: визуальные приложения, использующие `ncurses`, необходимо воспроизвести, чтобы они были удобочитаемыми, а такие функции, как мультиплексирование SSH, могут еще больше усложнить захват и понимание чередующегося состояния.

Чтобы избежать пространных API и/или частого использования механизма аварийного доступа, нужно создать культуру, которая ценит тщательный аудит. Это важно как по соображениям надежности, так и по соображениям безопасности, и вы можете использовать обе мотивации для обращения к ответственным сторонам. Две пары глаз помогают избежать опечаток и ошибок, и вы всегда должны быть защищены от одностороннего доступа к пользовательским данным в своих системах.

Создавая административные API и добавляя автоматизацию, команды должны спроектировать их так, чтобы по итогу облегчить аудит. Любой, кто регулярно обращается к производственным системам, должен быть заинтересован в совместном решении этих проблем и понимании ценности хорошего журнала аудита. Без культурного подкрепления аудиты могут превратиться в резиновые штампы, а использование механизма аварийного доступа может стать повседневным явлением, утратив ощущение важности или срочности. Благодаря культуре команды могут выбирать, создавать и использовать системы способами, поддерживающими аудит, чтобы такие события происходили лишь изредка и чтобы события аудита подвергались тщательному изучению, которого они заслуживают.

Выбор аудитора

После создания хорошего журнала аудита нужно выбрать подходящего человека для проверки записанных событий (надеемся, что редких). Аудитор должен иметь как правильный контекст, так и верную цель.

Во-первых, аудитор должен знать, что происходит при конкретном действии, и в идеале, почему субъекту необходимо было выполнить это действие. Поэтому аудитором обычно становится коллега в команде, менеджер или кто-то, знакомый с рабочими процессами, требующими выполнения этого действия. Найдите баланс между достаточным контекстом и объективностью. Например, внутренний рецензент может состоять в тесных отношениях с человеком, создавшим событие аудита, и/или стремиться добиться успеха в организации, а внешний частный аудитор может хотеть устроиться на работу в организации.

Во-вторых, выбор аудитора с правильной целью зависит от цели аудита. В Google мы выделяем две широкие категории аудита:

- аудиты для контроля использования лучших практик;
- аудиты для выявления нарушений безопасности.

Проще говоря, аудиты «лучших практик» поддерживают наши цели в области надежности. Например, команда SRE может выбрать аудит событий аварийного доступа дежурной сменой прошлой недели во время еженедельного собрания команды. Это создает культурное давление со стороны коллег для использования и улучшения административных API небольших сервисов — а не механизма аварийного доступа для доступа к более гибкому API экстренного использования. Доступ к аварийному механизму с широкой областью действия часто обходит некоторые или все проверки безопасности. Это может привести к более высокому риску человеческой ошибки.

Обычно Google распространяет обзоры событий аварийного доступа на уровне команды. Коллеги, выполняющие обзор, владеют контекстом, позволяющим им обнаруживать даже очень хорошо замаскированные действия. Это — ключ к предотвращению внутренних злоупотреблений и действий злонамеренных инсайдеров. Например, сотрудник хорошо подготовлен, чтобы заметить, что коллега неоднократно использует механизм аварийного доступа для доступа к необычному ресурсу, который ему на самом деле может быть и не нужен. Этот тип групповой проверки помогает выявить недостатки в административных API. Когда для конкретной задачи нужен доступ к аварийному механизму, это часто сигнализирует о необходимости обеспечить более надежный или безопасный способ выполнения этой задачи в рамках обычного API. Вы можете прочитать больше об этой теме в главе 21.

В Google мы склонны централизовать второй тип аудита, так как выявление внешних нарушений безопасности только выигрывает от широкого обзора. Продвинутый злоумышленник может скомпрометировать одну команду

и использовать этот доступ для компрометации другой команды, службы или роли. Каждая отдельная команда может не заметить пару аномальных действий, и у нее нет межкомандного представления, позволяющего связать точки между различными наборами действий.

Центральная аудиторская команда может иметь возможность создавать дополнительные оповещения и добавлять код для дополнительных событий аудита, которые широко не известны. Это особенно полезно при раннем обнаружении, но вы, возможно, не захотите широко раскрывать детали реализации. Для гарантии того, что механизмы аудита подходят и охватываются и документируются как положено, вам, вероятно, потребуется сотрудничать с другими отделами вашей организации (такими как юридический и кадровый).

Чтобы связать структурированные данные с событиями журнала аудита, мы в Google пользуемся *структурированным обоснованием*. Когда происходит событие, генерирующее запись в журнале аудита, мы можем связать его со структурированной ссылкой, такой как номер ошибки, номер заявки или номер обращения клиента. Это позволяет создавать программные проверки журналов аудита. Например, если персонал службы поддержки просматривает детали платежа или другие конфиденциальные данные, он может связать их с конкретным запросом клиента. Таким образом мы можем гарантировать, что наблюдаемые данные принадлежат клиенту, отправившему запрос.

Автоматизировать проверку записей в журнале было бы гораздо сложнее, если бы мы ориентировались на поля свободного ввода текста. Структурированное обоснование стало ключом к масштабированию наших аудиторских усилий. Оно обеспечивает централизованный контекст для команды аудита, который имеет решающее значение для эффективного аудита и анализа.

Тестирование и минимальные привилегии

Правильное тестирование — фундаментальное свойство любой хорошо спроектированной системы. У тестирования есть два важных аспекта в отношении наименьших привилегий.

- Тестирование наименьших *привилегий*, чтобы гарантировать, что доступ должным образом предоставляется только к необходимым ресурсам.
- Тестирование с наименьшими *привилегиями*, чтобы гарантировать, что инфраструктура для тестирования имеет только тот доступ, который ей необходим.

Тестирование наименьших привилегий

В контексте наименьших привилегий вы должны иметь возможность проверить, что четко определенные профили пользователей (например, аналитик данных, служба поддержки клиентов, SRE) имеют достаточные привилегии для выполнения своей роли, но не более.

Ваша инфраструктура должна позволять вам делать следующее.

- Описывать функции конкретного пользователя в своей рабочей роли. Это определяет минимальный доступ (API и данные) и тип доступа (чтение или запись, постоянный или временный), нужный для роли.
- Описывать набор сценариев, в которых профиль пользователя пытается выполнить действие в системе (например, чтение, массовое чтение, запись, удаление, массовое удаление, администрирование), и ожидаемый результат/влияние на систему.
- Запускать эти сценарии и сравнивать фактический результат/воздействие с ожидаемым.

В идеале для предотвращения негативного влияния на производственные системы, эти тесты выполняются до изменения кода или ACL. Если тестовое покрытие неполное, вы можете уменьшить чрезмерно широкий доступ с помощью мониторинга систем доступа и оповещения.

Тестирование с наименьшими привилегиями

Тесты должны позволять вам проверить ожидаемое поведение чтения/записи, не подвергая риску надежность сервиса, конфиденциальные данные или другие важные активы. Но если у вас нет подходящей тестовой инфраструктуры, учитывающей разные среды, клиентов, учетные данные, наборы данных и т. д., тесты, требующие чтения/записи данных или изменения состояния сервиса, могут быть рискованными.

Рассмотрим пример (мы вернемся к нему в следующем разделе) загрузки файла конфигурации в производственную среду. Первым делом при разработке стратегии тестирования для этой загрузки конфигурации вы должны предоставить отдельную среду с отдельными ключами и учетными данными. Это гарантирует, что ошибка в написании или выполнении теста не повлияет на производственную среду, например, перезаписав производственные данные или отключив производственный сервис.

Рассмотрим альтернативный вариант. Вы разрабатываете приложение для клавиатуры, позволяющее людям публиковать мемы одним нажатием. Вы хотите

проанализировать поведение и историю действий пользователей, чтобы автоматически рекомендовать мемы. Если у вас нет необходимой инфраструктуры тестирования, нужно предоставить аналитикам доступ на чтение/запись ко всему набору необработанных пользовательских данных в производственной среде для выполнения анализа и тестирования.

Правильная методология тестирования должна предусматривать, как ограничить доступ пользователей и снизить риск, но позволять аналитикам данных выполнять необходимые тесты. Нужен ли им доступ для записи? Можете ли вы использовать анонимизированные наборы данных для выполнения задач, которые они должны выполнять? Можно ли использовать тестовые учетные записи? Можно ли работать в тестовой среде с анонимизированными данными? Если этот доступ скомпрометирован, какие данные будут раскрыты?

КОМПРОМИСС БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: СРЕДА ТЕСТИРОВАНИЯ

Вместо создания тестовой инфраструктуры, точно отражающей производство, из соображения экономии времени и затрат вы можете использовать специальные тестовые учетные записи в производстве, чтобы изменения не влияли на реальных пользователей. Эта стратегия может работать в некоторых ситуациях, но может и сбивать с толку, когда дело доходит до аудита и ACL.

Вы можете приблизиться к тестовой инфраструктуре, начав с малого — не позволяйте лучшему быть врагом хорошего. Начните с размышлений о том, как вам проще всего:

- разделить среды и учетные данные;
- ограничить типы доступа;
- ограничить раскрытие данных.

На начальном этапе, возможно, вы сможете проводить кратковременные тесты на облачной платформе вместо создания всего стека инфраструктуры тестирования. Одним сотрудникам может потребоваться только чтение или временный доступ. В других случаях вы также можете использовать репрезентативные или анонимизированные наборы данных.

Все эти методики хорошо звучат в теории. Но вы можете поразиться потенциальным затратам на создание надлежащей инфраструктуры тестирования. Сделать это правильно стоит недешево. А теперь рассмотрите стоимость *отсутствия* надлежащей тестовой инфраструктуры. Можете ли вы быть уверены, что каждый тест критических операций не будет снижать производительность? Можете ли вы смириться с тем, что аналитики данных имеют привилегии для

доступа к конфиденциальным данным, которые могли бы не предоставляться при наличии инфраструктуры тестирования? Вы полагаетесь на идеальных людей с идеальными тестами, которые идеально выполняются?

Важно провести корректный анализ затрат и выгод для вашей конкретной ситуации. Может, и не нужно изначально строить «идеальное» решение. Тем не менее убедитесь, что вы создаете фреймворк, который будет использоваться. Людям необходимо тестировать. Если у вас нет подходящей среды для этого, они будут тестировать в производственной среде, обходя установленные вами средства контроля.

Диагностика отказа в доступе

В сложной системе, где соблюдается принцип наименьших привилегий, а доверие клиента должно быть завоевано с помощью третьего фактора, многосторонней авторизации или другого механизма (см. раздел «Расширенные средства контроля» этой главы), политика безопасности применяется на нескольких уровнях и с высокой степенью детализации. В результате отказы политик тоже могут происходить сложным образом.

Рассмотрим случай, когда применяется здравая политика безопасности и ваша система авторизации запрещает доступ. Мы придем к одному из трех возможных результатов.

- Запрос клиента правильно отклонен, ваша система ведет себя должным образом. Принцип наименьших привилегий применен — и все хорошо.
- Клиенту правильно отказано в доступе, но он может пользоваться расширенным средством контроля (например, многосторонней авторизацией) для получения временного доступа.
- Клиент считает, что ему было ошибочно отказано в праве доступа, и подает заявку в службу поддержки в вашу команду по политике безопасности. Такое может произойти, если клиент был недавно удален из авторизованной группы или если политика изменилась скрытым или неправильным образом.

Во всех случаях вызывающий не знает причины отказа. Но может ли система предоставить клиенту больше информации? В зависимости от уровня привилегий вызывающего, может.

Для клиента с ограниченными привилегиями или вообще без них отказ должен оставаться скрытым. Ведь если раскрыть информацию за пределами кода

ошибки 403 «Отказ в доступе» (или его эквивалента), то сведения о причинах отказа могут быть использованы для получения информации о системе и даже поиска способа получить доступ. Но если у вызывающего есть минимальные привилегии, вы можете предоставить токен, связанный с отказом. Клиент может воспользоваться этим токеном, чтобы вызвать расширенное средство контроля для получения временного доступа, или предоставить токен команде по политике безопасности через канал поддержки, чтобы они с его помощью провели диагностику проблемы. Более привилегированный вызывающий может получить токен, связанный с отказом, и некоторую информацию об исправлении. После этого клиент может попытаться исправить ситуацию, не связываясь со службой поддержки. Например, он может узнать, что в доступе отказано потому, что он должен быть членом определенной группы, после чего он может запросить доступ к этой группе.

Всегда есть противоречие между тем, сколько информации об исправлении необходимо предоставить, и тем, с какой перегрузкой поддержки может справиться команда по политике безопасности. Но, если вы предоставите слишком много информации, клиенты могут перестроить политику на основе информации об отказе, что облегчит злоумышленнику создание запроса, который использует политику непредусмотренным образом. Зная это, мы рекомендуем на ранних этапах реализации модели с нулевым доверием использовать токены, чтобы все клиенты обращались к каналу поддержки.

Механизмы постепенного отказа и аварийного доступа

В идеале вы всегда должны иметь дело с работающей системой авторизации, обеспечивающей разумную политику. Но в действительности вы можете столкнуться со сценарием, приводящим к большому количеству ошибочных отказов в доступе (возможно, из-за плохого обновления системы). Чтобы исправить это, вы должны иметь возможность обойти систему авторизации с помощью механизма аварийного доступа.

Пользуясь механизмом аварийного доступа, придерживайтесь следующих рекомендаций.

- Максимально ограничьте возможность использования этого механизма. Он должен быть доступен только вашей команде SRE, отвечающей за операционные SLA вашей системы.
- Механизм аварийного доступа для сетей с нулевым доверием должен быть доступен только в определенных местах. Эти места являются вашими *убежищами*, особыми точками с дополнительными средствами контроля физического

доступа, чтобы компенсировать возросшее доверие к их подключению (внимательный читатель заметит, что резервный механизм для сетей с нулевым доверием, стратегия недоверия к сетевому местоположению — это... доверительное сетевое местоположение, но с дополнительными средствами контроля физического доступа).

- Все виды использования механизма аварийного доступа должны тщательно контролироваться.
- Механизм аварийного доступа должен регулярно тестироваться командой (-ми), ответственной (-ми) за производственные сервисы, чтобы убедиться, что он работает, когда вам это нужно.

Когда механизм аварийного доступа будет успешно использован для восстановления доступа пользователей, ваши SRE и команда политик безопасности могут дополнительно диагностировать и устранять основную проблему. В главах 8 и 9 обсуждаются соответствующие стратегии.

Пример из практики: распределение конфигурации

Давайте обратимся к реальному примеру. Распределение файла конфигурации на набор веб-серверов — интересная задача для проектирования. Ее можно практически реализовать с помощью небольшого функционального API. Вот лучшие практики по управлению файлом конфигурации.

1. Сохраните файл конфигурации в системе контроля версий.
2. Проверьте изменения кода в файле.
3. Сначала автоматизируйте распределение на набор канареечных серверов, проверьте их работоспособность. После продолжайте проверять работоспособность всех хостов, постепенно отправляя файл на комплект веб-серверов¹. Этот шаг требует предоставления доступа к автоматизации для удаленного обновления файла конфигурации.

Есть множество подходов к представлению небольшого API, каждый из которых предназначен для функции обновления конфигурации ваших веб-серверов. Та-

¹ *Канареечное тестирование* — это постепенное внедрение в производство, начиная с небольшого набора конечных точек. Подобно канарейке в угольной шахте, такие тесты предупреждают, если что-то идет не так. См. главу 27 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/reliable-product-launches>).

блица 5.2 обобщает некоторые API и их компромиссы. В следующих разделах мы подробнее ознакомимся с каждой тактикой.

Таблица 5.2. API, обновляющие конфигурацию веб-сервера, и их компромиссы

	POSIX API через OpenSSH	API обновления ПО	Пользовательский ForceCommand для OpenSSH	Произвольный HTTP-получатель
Поверхность API	Большая	Различная	Малая	Малая
Существование	Вероятно	Да	Маловероятно	Маловероятно
Сложность	Высокая	Высокая	Низкая	Средняя
Способность масштабироваться	Умеренная	Умеренная, но многоразовая	Сложная	Умеренная
Возможность аудита	Плохая	Хорошая	Хорошая	Хорошая
Может выражать наименьшие привилегии	Плохо	Различно	Хорошо	Хорошо

Таблица показывает, насколько вероятно или маловероятно, что у вас уже есть API такого типа как часть существующего развертывания веб-сервера.

POSIX API через OpenSSH

Вы можете разрешить автоматике подключаться к хосту веб-сервера через OpenSSH, обычно логинясь как локальный пользователь, от имени которого работает веб-сервер. Затем она может записать файл конфигурации и перезапустить процесс веб-сервера. Это простой и распространенный шаблон, при котором используется административный, вероятно уже существующий, API, и поэтому требуется лишь небольшой дополнительный код. Но использование большого уже существующего административного API влечет за собой несколько рисков.

- Роль, выполняющая автоматизацию, может навсегда остановить веб-сервер, запустить вместо него другой бинарный файл, прочитать любые данные, к которым у нее есть доступ, и т. д.
- Ошибки в автоматизации неявно имеют достаточный доступ, чтобы вызвать скоординированное отключение всех веб-серверов.
- Компрометация учетных данных автоматизации эквивалентна компрометации всех веб-серверов.

API обновления ПО

Вы можете распространять конфигурацию как пакетное обновление ПО, используя тот же механизм, что и для обновления бинарного файла веб-сервера. Есть много способов упаковки и запуска бинарных обновлений с использованием API разных размеров. Простой пример — пакет Debian (`.deb`), извлекаемый из центрального репозитория периодическим вызовом `apt-get` из `cron`. Вы можете создать более сложный пример, используя один из шаблонов, описанных в следующих разделах, для запуска обновления (вместо `cron`), которое затем можно было бы повторно использовать как для конфигурации, так и для бинарного файла. По мере того как вы совершенствуете свой механизм распространения бинарных файлов для повышения надежности и безопасности, возрастают и преимущества вашей конфигурации, поскольку они вместе используют одну и ту же инфраструктуру. Любая работа, сделанная для того, чтобы управлять канареечным процессом, проверкой работоспособности или предоставлением подписей/доказательств/аудита, аналогично приносит пользу обоим.

Иногда потребности бинарных систем и систем обновления конфигурации не совпадают. Например, в своей конфигурации вы можете распространять список запрещенных IP-адресов, который должен собираться быстро и безопасно, и в то же время упаковывать бинарный файл веб-сервера в контейнер. В этом случае сборка, установка и демонтаж нового контейнера с той же скоростью, с которой нужно распространять обновления конфигурации, могут оказаться слишком дорогостоящим или разрушительным делом. Для конфликтующих требований такого типа может понадобиться два механизма распространения: один для бинарных файлов, другой — для обновлений конфигурации.

Для получения дополнительной информации об этом шаблоне см. главу 9.

Пользовательский параметр ForceCommand для OpenSSH

Вы можете написать короткий сценарий для выполнения следующих шагов.

1. Получить конфигурацию из `STDIN`.
2. Проверить ее на работоспособность.
3. Перезагрузить веб-сервер, чтобы обновить конфигурацию.

После вы можете развернуть эту команду через OpenSSH, связав определенные записи в файле `authorized_keys` с параметром `ForceCommand`¹. Эта стратегия

¹ ForceCommand — это параметр конфигурации, позволяющий ограничить определенную авторизованную идентичность для запуска только одной команды. Смотрите страницу руководства `ssh_config` (<https://oreil.ly/4ruSh>) для более подробной информации.

дает вызывающему очень маленький API для подключения через проверенный протокол OpenSSH. Единственное доступное действие там — предоставление копии файла конфигурации. Регистрация файла (или его хеша¹) фиксирует все действие в сеансе для последующего аудита.

Вы можете реализовать столько уникальных комбинаций «ключ/ForceCommand», сколько нужно. Однако такой шаблон может быть трудно масштабируемым для множества уникальных административных действий. Хотя вы можете создать текстовый протокол поверх OpenSSH API (например, git-shell (<https://oreil.ly/k4igi>)), и это приведет к созданию собственного механизма RPC. Скорее всего, вам лучше перейти к концу этого пути и создать систему поверх существующего фреймворка, такого как gRPC (<https://grpc.io/>) или Thrift (<https://thrift.apache.org/>).

Пользовательский HTTP-получатель (расширенный)

Вы можете написать небольшой расширяющий демон, принимающий конфигурацию. Он будет похож на решение с ForceCommand, но с использованием другого механизма AAA (например, gRPC с SSL, SPIFFE (<https://spiffe.io/>) или аналогичного). Для этого подхода не нужно менять обслуживающий бинарный файл. Он очень гибкий из-за введения большего количества кода и другого демона для управления.

Пользовательский HTTP-получатель (внутрипроцессный)

Вы можете изменить веб-сервер, чтобы он предоставлял API для прямого обновления его конфигурации (<https://oreil.ly/hu7yg>), получения ее и записи на диск. Это один из самых гибких подходов, сильно напоминающий способ управления конфигурацией в Google, но требующий добавления кода в обслуживающий бинарный файл.

Компромиссы

Все варианты, кроме самых больших, из табл. 5.2 дают возможность повысить уровень безопасности вашей автоматизации. Злоумышленник все еще может поставить под угрозу роль веб-сервера, загрузив произвольную конфигурацию.

¹ В масштабе может быть нецелесообразно регистрировать и хранить много дубликатов файла. Журналирование хеша позволяет сопоставить конфигурацию с системой контроля версий и обнаруживать неизвестные или неожиданные конфигурации при проверке журнала. И приятный бонус — если пространство позволяет, вы можете сохранить отклоненные конфигурации для помощи будущему расследованию. В идеале все конфигурации должны быть подписаны — это указывает на то, что они были получены из системы контроля версий с известным хешем или отклонены.

Но выбор меньшего API означает, что механизм загрузки не допустит неявного разрешения такой компрометации.

Возможно, у вас получится разработать проект с наименьшими привилегиями, подписывая конфигурацию независимо от загружающей ее автоматизации. Эта стратегия разделяет доверие между ролями и гарантирует, что, если роль автоматизации, загружающей конфигурацию, будет скомпрометирована, автоматизация тоже не сможет скомпрометировать веб-сервер, отправив вредоносную конфигурацию. Напомним совет Макэлроя, Пинсона и Тага — проектирование каждой части системы для выполнения одной задачи и выполнение этой задачи хорошо позволяет изолировать доверие.

Более детальная управляющая поверхность, представленная узким API, тоже может защитить от ошибок автоматизации. В дополнение к требованию подписи для проверки конфигурации вы можете потребовать bearer token¹ от центрального ограничителя нагрузки. Он должен быть создан независимо от вашей автоматизации и предназначен для каждого хоста в развертывании. Вы можете тщательно протестировать этот общий ограничитель нагрузки. Если он реализован независимо, ошибки, влияющие на автоматизацию развертывания, скорее всего, не повлияют на него. Независимый ограничитель нагрузки также удобно использовать повторно, так как он может ограничивать скорость развертывания конфигурации веб-сервера, двоичного развертывания того же сервера, перезагрузок сервера или любых других задач, в которые нужно добавить проверку безопасности.

Структура политик для решений аутентификации и авторизации

Аутентификация [сущ.]: проверка **личности** пользователя или процесса.

Авторизация [сущ.]: оценка того, **должен ли быть разрешен** запрос от конкретной аутентифицированной стороны.

В предыдущем разделе говорилось о разработке узкого административного API для вашего сервиса, позволяющего предоставлять как можно меньше привилегий для заданного действия. После создания этого API вы должны решить, как

¹ *bearer token (токен на предъявителя)* — это просто криптографическая подпись ограничителя нагрузки, которая может быть предоставлена любому, кто имеет открытый ключ ограничителя нагрузки. Можно использовать его для проверки того, что ограничитель нагрузки одобрил эту операцию в течение периода действия токена.

контролировать доступ к нему. Контроль доступа включает в себя два важных, но отдельных шага.

Во-первых, вы должны *аутентифицировать того*, кто подключается. Механизм аутентификации может варьироваться по сложности.

Простой: принятие имени пользователя, переданного в параметре URL

Пример: `/service?username=admin`.

Более сложный: предоставление заранее подготовленного секрета

Примеры: WPA2-PSK, файл cookie в HTTP.

Еще более сложный: сложное гибридное шифрование и схемы сертификатов

Примеры: TLS 1.3, OAuth.

Проще говоря, предпочтительнее использовать существующий механизм строгой криптографической аутентификации для распознавания стороны, вызывающей API. Результат этого решения об аутентификации обычно выражается в виде имени пользователя, общего имени, «принципала», «роли» и т. д. Для целей этого раздела мы используем *роль*, чтобы описать взаимозаменяемые результаты аутентификации.

Затем ваш код должен принять решение: *авторизована* ли эта роль для выполнения запрошенного действия? В вашем коде могут учитываться многие параметры запроса, например следующие.

Конкретное запрашиваемое действие

Примеры: URL, выполняемая команда, метод gRPC.

Аргументы запрошенного действия

Примеры: параметры URL, `argv`, запрос gRPC.

Источник запроса

Примеры: IP-адрес, метаданные сертификата клиента.

Метаданные аутентифицированной роли

Примеры: географическое расположение, юрисдикция, оценка риска с помощью машинного обучения.

Контекст на стороне сервера

Примеры: частота похожих запросов, доступная пропускная способность.

В оставшейся части этого раздела рассматриваются несколько методов, которые компания Google сочла полезными для улучшения и расширения основных требований к решениям аутентификации и авторизации.

Использование расширенных средств контроля авторизации

Использование списка контроля доступа (access control list, ACL) для ресурса — знакомый способ реализации решения об авторизации. Простейший ACL — это строка, соответствующая аутентифицированной роли, которая часто сочетается с группированием, например, группой ролей (такой как «администратор»), расширяющейся до большего списка ролей (имена пользователей). Когда сервис оценивает входящий запрос, он проверяет, является ли аутентифицированная роль членом ACL.

Более сложные варианты авторизации, такие как многофакторная (multi-factor authorization, MFA) или многосторонняя авторизация (multi-party authorization, MPA), требуют более сложного кода (подробную информацию о трехфакторной авторизации и MPA см. в разделе «Расширенные средства контроля» далее). Также некоторым организациям придется учитывать свои особые нормативные или договорные требования при разработке политик авторизации.

Этот код трудно правильно реализовать, и он быстро станет сложнее, если многие сервисы реализуют собственную логику авторизации. По нашему опыту, полезно отделить сложности решений по авторизации от разработки основного API и бизнес-логики. Это можно сделать с помощью фреймворков, предлагаемых AWS (<https://aws.amazon.com/iam>) или GCP (<https://cloud.google.com/iam>) для управления идентификацией и доступом (Identity & Access Management, IAM). В Google мы также широко используем разные варианты авторизации GCP для внутренних сервисов¹.

Благодаря структуре политики безопасности наш код может выполнять простые проверки (такие как «Может ли X получить доступ к ресурсу Y?») и оценивать их, сравнивая с предоставленной извне политикой. Чтобы добавить дополнительные средства контроля авторизацией к определенному действию, мы можем просто изменить соответствующий файл конфигурации политики. Эти низкие издержки обладают огромной функциональностью и преимуществами в скорости.

¹ Используемый нами вариант поддерживает внутренние примитивы аутентификации. Это позволяет избежать циклических проблем с зависимостями и т. д.

Вложение в широко используемый фреймворк авторизации

Вы можете включить масштабные изменения аутентификации и авторизации, задействовав общую библиотеку для реализации решений об авторизации и максимально широко используя согласованный интерфейс. Применяв этот совет по разработке модульного ПО в сфере безопасности, вы получите удивительные преимущества. Например.

- Вы можете добавить поддержку MFA или MPA для всех конечных точек сервиса одним изменением библиотеки.
- Затем вы можете применить эту поддержку для небольшого процента действий или ресурсов во всех сервисах через единственное изменение конфигурации.
- Вы можете повысить надежность, запрашивая MPA для всех потенциально небезопасных действий вроде запуска системы проверки кода. Такое улучшение процесса может усилить защиту от внутренних угроз (более подробно о типах злоумышленников см. в главе 2), упрощая быстрое реагирование на происшествя (путем обхода системы контроля версий и проверок кода) и не допуская широкого одностороннего доступа.

По мере роста вашей организации стандартизация станет вашим другом. Единая структура авторизации сделает команду мобильнее. Все больше людей знают, как кодировать и контролировать доступ с помощью общего фреймворка.

Избегание потенциальных ловушек

Разработать сложный язык политики авторизации непросто. Если язык политики слишком упрощен, она не достигнет своей цели. В конечном счете вы получите решения по авторизации, распространяющиеся как на политику фреймворка, так и на основную кодовую базу. Если язык политики слишком общий, сложно делать о ней какие-либо выводы. Для смягчения этих проблем вы можете применять стандартные методики проектирования программных API, например итеративный подход к проектированию. Но действуйте осторожно во избежание обеих этих крайностей.

Внимательно рассмотрите, как политика авторизации отправляется в бинарный файл (или вместе с ним). Возможно, вы захотите обновить политику, которая, вероятно, станет одной из наиболее чувствительных к безопасности частей конфигурации, независимо от бинарного файла. Дополнительное обсуждение распределения конфигурации см. в примере из практики в предыдущем разделе,

главах 9 и 14 этой книги, главе 8 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/release-engineering/>) и главах 14 и 15 в Site Reliability Workbook.

Следует помочь разработчикам приложений принять решения о политике, которые будут кодироваться на этом языке. Даже если вы избежите ловушек, описанных здесь, и создадите выразительный и понятный язык политики, чаще всего для этого потребуется сотрудничество между разработчиками приложений, реализующими административные API, и инженерами по безопасности и SR-инженерами с конкретными предметными знаниями о вашей производственной среде для правильного баланса между безопасностью и функциональностью.



Расширенные средства контроля

Несмотря на то что многие решения об авторизации представляют собой двоичный ответ «да»/«нет», в некоторых ситуациях полезна большая гибкость. Не требуйте строго «да» или «нет». Предохранительный клапан с надписью «возможно» вместе с дополнительной проверкой может значительно ослабить давление в системе. Многие из описанных здесь средств контроля можно комбинировать или использовать по отдельности. Разумное использование зависит от конфиденциальности данных, риска действия и существующих бизнес-процессов.

Многосторонняя авторизация (МРА)

Вовлечение другого человека — один из классических способов обеспечить правильное решение о доступе. Оно способствует культуре безопасности и надежности (см. главу 21). Такая стратегия предлагает несколько преимуществ.

- *Предотвращение ошибок* или непреднамеренных нарушений политики, которые могут привести к проблемам безопасности или конфиденциальности
- *Препятствование злоумышленникам* в попытках внести злонамеренные изменения. К ним относятся как сотрудники, которым грозит дисциплинарное взыскание, так и внешние хакеры, которые рискуют обнаружением.
- *Увеличение стоимости атаки* — нужна либо компрометация хотя бы еще одного человека, либо тщательно продуманное изменение, которое прошло бы экспертную оценку.
- *Аудит прошлых действий* для реагирования на происшествия или последующего «посмертного» анализа, при условии, что обзоры записываются постоянно и защищены от несанкционированного доступа.

- *Обеспечение комфорта клиентов.* Вашим клиентам может быть удобнее пользоваться вашим сервисом, зная, что ни один человек не может внести изменения в одиночку.

Многосторонняя авторизация (multi-party authorization, МРА) часто применяется для широкого уровня доступа. Например, когда нужно одобрение для присоединения к группе, предоставляющей доступ к производственным ресурсам, либо способность выступать в качестве заданной роли или учетных данных. Широкая МРА может быть ценным механизмом аварийного доступа, позволяющим выполнять очень необычные действия, для которых у вас может не быть определенных рабочих процессов. Там, где это возможно, вы должны стараться предоставлять более детальную авторизацию, дающую более устойчивые гарантии надежности и безопасности. Если вторая сторона одобряет действие в рамках небольшого функционального API (см. подраздел «Небольшие функциональные API» выше в этой главе), то она может быть гораздо более уверенной в том, что именно авторизует.

ПОТЕНЦИАЛЬНЫЕ ЛОВУШКИ

Убедитесь, что подтверждающие достаточно осведомлены для принятия обоснованного решения. Четко ли указана информация о том, кто и что делает? Вам может потребоваться привести параметры конфигурации и цели, чтобы показать, что делает команда. Это может быть особенно важно, если вы демонстрируете запросы на подтверждение на мобильных устройствах и пространства для подробностей на экране немного.

Социальное давление, связанное с подтверждением, также может привести к неправильным решениям. Например, инженеру может быть неудобно отклонять подозрительный запрос, если он получен от руководителя или старшего инженера. Чтобы смягчить давление, вы можете предоставить возможность передавать утверждения в службу безопасности или команду по расследованию постфактум. Или у вас может быть политика, при которой все (или часть) утверждения определенного типа проходят независимый аудит.

Прежде чем создавать многостороннюю систему авторизации, убедитесь, что технология и социальная динамика позволяют кому-то сказать «нет». Иначе система не имеет большой ценности.

Трехфакторная авторизация (3FA)

В большой организации МРА часто имеет один ключевой недостаток, которым может воспользоваться решительный и настойчивый злоумышленник. Все «несколько сторон» используют одинаковые централизованно управляемые рабочие

станции. Чем более однороден парк рабочих станций, тем больше вероятность того, что злоумышленник, который способен скомпрометировать одну рабочую станцию, скомпрометирует несколько или даже все из них.

Классический метод защиты парка рабочих станций от атак заключается в том, что пользователи поддерживают две совершенно разные рабочие станции: одну для общего пользования, например, для просмотра веб-страниц и отправки/получения электронной почты, и другую, более надежную, для связи с производственной средой. По нашему опыту, пользователи в конечном счете хотят получить одинаковые наборы функций и возможностей от этих рабочих станций, и поддержка двух наборов инфраструктуры рабочих станций для ограниченного круга пользователей, чьи учетные данные требуют такого повышенного уровня защиты, — задача затратная и трудная для постоянного выполнения. Как только эта проблема перестает быть приоритетной для руководства, люди становятся менее заинтересованными в поддержке инфраструктуры.

Чтобы снизить риск того, что одна скомпрометированная платформа подорвет всю авторизацию, требуется следующее.

- Поддержка как минимум двух платформ.
- Возможность утверждать запросы на двух платформах.
- Возможность усилить хотя бы одну платформу (предпочтительно).

Учитывая эти требования, еще один вариант — авторизация с защищенной мобильной платформы для некоторых очень рискованных операций. Для простоты и удобства вы можете разрешить создание RPC только с полностью управляемых настольных рабочих станций, а затем запрашивать трехфакторную авторизацию (three-factor authorization, 3FA) с мобильной платформы. Когда производственный сервис получает конфиденциальный RPC, фреймворк политики (описанный в разделе «Структура политик для решений аутентификации и авторизации» выше в этой главе) требует криптографически подписанного утверждения от отдельного сервиса 3FA. Затем этот сервис указывает, что он отправил RPC на мобильное устройство, тот был показан исходному пользователю и запрос был подтвержден.

Усиление мобильных платформ несколько проще, чем усиление рабочих станций общего назначения. Мы обнаружили, что пользователи обычно более терпимы к ограничениям безопасности на мобильных устройствах, таким как дополнительный мониторинг сети, разрешение только определенного набора приложений и подключение через ограниченное количество конечных точек HTTP. Эти политики также довольно легко реализовать через современные мобильные платформы.

Если у вас есть защищенная мобильная платформа, на которой можно отобразить предлагаемое производственное изменение, вы должны отправить запрос к этой платформе и показать его пользователю. В Google мы повторно используем инфраструктуру, отправляющую уведомления на телефоны Android, для авторизации и сообщения пользователям о попытках входа в Google. Если вы можете себе позволить такую роскошь, как подобная защищенная часть инфраструктуры, возможно, было бы полезно расширить ее для поддержки этого варианта использования. Но если не получится, создать базовое веб-решение тоже будет несложно. Ядро системы 3FA — простая служба RPC, получающая запрос на авторизацию и предоставляющая его для авторизации доверенному клиенту. Пользователь, запрашивающий RPC, который защищен с помощью 3FA, посещает URL-адрес сервиса 3FA со своего мобильного устройства и получает запрос на утверждение.

Важно различать, от каких угроз защищают МРА и 3FA, чтобы вы могли выбрать согласованную политику относительно того, когда и какую из них применять. МРА защищает от одностороннего инсайдерского риска и от компрометации отдельной рабочей станции (требуя повторного внутреннего одобрения). 3FA защищает от широкой компрометации внутренних рабочих станций, но не обеспечивает никакой защиты от внутренних угроз при изолированном использовании. Если требовать трехфакторной авторизации от инициатора и простой многосторонней авторизации от второй стороны, то можно обеспечить очень надежную защиту от комбинации большинства из этих угроз с небольшими организационными издержками.

3FA — НЕ 2FA (ИЛИ MFA)

Двухфакторная аутентификация (2FA) — хорошо знакомое нам подмножество многофакторной аутентификации. Это попытка объединить «что-то, что вы знаете» (пароль) с «чем-то, что у вас есть» (токен приложения или оборудования, который предоставляет криптографическое подтверждение присутствия), чтобы сформировать надежное решение для аутентификации. Подробнее о 2FA см. в тематическом исследовании «Пример: строгая двухфакторная аутентификация с использованием ключей безопасности FIDO» в главе 7.

Ключевое отличие в том, что 3FA пытается обеспечить более строгую *авторизацию* конкретного запроса, а не более строгую *аутентификацию* конкретного пользователя. Хотя мы признаем, что 3FA — немного неправильное название (это одобрение со стороны второй платформы, а не третьей), важно отметить, что «устройство 3FA» — это мобильное устройство, добавляющее дополнительную авторизацию для некоторых запросов, помимо ваших первых двух факторов.

Бизнес-обоснования

Как упоминалось в подразделе «Выбор аудитора» выше в этой главе, вы можете применить авторизацию, привязав доступ к структурированному бизнес-обоснованию, такому как ошибка, происшествие, заявка, идентификатор задачи или назначенная учетная запись. Но для построения логики проверки нужно выполнить больше дополнительной работы и внести изменения процесса для сотрудников, работающих по вызову или в отделе обслуживания клиентов.

В качестве примера рассмотрим процесс обслуживания клиентов. В небольших или молодых организациях бывает так, что базовая система может предоставлять представителям службы поддержки клиентов доступ ко всем записям клиентов либо по соображениям эффективности, либо из-за отсутствия средств контроля. Лучшим вариантом было бы заблокировать доступ по умолчанию и разрешить доступ к определенным данным только тогда, когда вы сможете проверить бизнес-потребности. Этот подход может представлять собой набор средств контроля, реализуемых с течением времени. Например, можно начать с разрешения доступа только представителям службы поддержки, которым назначена открытая задача. Со временем вы можете улучшить систему, чтобы разрешить доступ только к определенным клиентам и конкретным данным этих клиентов с их одобрения и в заданные сроки.

КОМПРОМИСС БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: УДОБСТВО ИСПОЛЬЗОВАНИЯ СИСТЕМЫ

Если инженерам слишком сложно находить подтверждающего каждый раз, когда они хотят выполнить действие с повышенными привилегиями, повышается риск потенциально опасного поведения. Например, они могут обойти принудительно введенные лучшие практики, предоставив общие бизнес-обоснования (например, «Команда требует доступ»), не соответствующие функциональным требованиям. Использование общих обоснований должно вызывать сигналы тревоги в системе аудита.

Временный доступ

Вы можете уменьшить риск принятия решения об авторизации, предоставив временный доступ к ресурсам. Эта стратегия может быть полезна, когда детализированные средства контроля доступны не для каждого действия, но вы все же хотите предоставить как можно меньше привилегий из имеющихся инструментов.

Вы можете предоставить временный доступ структурированным и запланированным способом (например, во время замены дежурных сотрудников или через истечение срока членства в группах) или способом по требованию, когда пользователи явно запрашивают доступ. Можно объединить временный доступ с запросом на многостороннюю авторизацию, доступ с бизнес-обоснованием или с другим средством контроля авторизации. Временный доступ также создает логическую точку для аудита, поскольку в таком случае вы четко ведете журнал о пользователях, имеющих доступ в любой момент времени. Он предоставляет данные о том, где происходит временный доступ, чтобы вы могли со временем расставлять приоритеты и уменьшать эти запросы.

Временный доступ также снижает внешние полномочия. Это одна из причин, по которой администраторы предпочитают `sudo` или «Запуск от имени администратора» работе от имени пользователя Unix `root` или учетной записи администратора в Windows. Если вы случайно вводите команду для удаления всех данных, то чем меньше у вас разрешений, тем лучше!

Прокси

Когда детальные средства контроля для серверных служб недоступны, вы можете вернуться к сильно контролируемому и ограниченному прокси-компьютеру (или *бастиону*). Только запросам от этих указанных прокси разрешается доступ к конфиденциальным сервисам. Этот прокси может ограничивать опасные действия, действия по ограничению скорости и выполнять более сложное ведение журнала.

Скажем, вам нужно выполнить экстренный откат плохого изменения. Существует бесконечное число способов внести плохое изменение и такое же количество способов его устранения. Поэтому нужные для выполнения отката шаги могут быть недоступны в предопределенном API или инструменте. Предоставьте системному администратору гибкость в разрешении чрезвычайной ситуации, но введите ограничения или дополнительные средства контроля, снижающие риск. Например.

- Может потребоваться утверждение для каждой команды.
- Администратор может подключаться только к соответствующим машинам.
- Компьютер, которым пользуется администратор, может не иметь доступа к Интернету.
- Вы можете включить более тщательное журналирование.

Как всегда, применение любого из этих средств контроля требует затрат на внедрение и эксплуатацию. Это мы обсудим в следующем разделе.

Компромиссы и противоречия

Внедрение модели доступа с наименьшими привилегиями поднимет уровень безопасности вашей организации. Но нужно понимать, что вы должны компенсировать выгоды, описанные в предыдущих разделах, потенциальными затратами на внедрение этой позиции. Здесь мы рассмотрим некоторые из них.

Повышенная сложность безопасности

Высокодетализированная система безопасности — мощный инструмент. Но он еще и сложен, поэтому им трудно управлять. Важно иметь полный набор инструментов и инфраструктуры, помогающих вам определять, анализировать, продвигать и отлаживать ваши политики безопасности, а также управлять ими. Иначе такая сложность может стать непреодолимой. Всегда стремитесь быть в состоянии ответить на следующие важные вопросы: «Имеет ли данный пользователь доступ к определенному сервису/фрагменту данных?» и «У кого есть доступ к данному сервису/фрагменту данных?».

Влияние на сотрудничество и корпоративную культуру

Хоть строгая модель с наименьшими привилегиями и подходит для конфиденциальных данных и сервисов, в других областях более мягкий подход может принести ощутимые выгоды.

Например, предоставление инженерам-программистам широкого доступа к исходному коду сопряжено с определенным риском. Это уравнивается тем, что инженеры смогут учиться на рабочем месте и вносить вклад в функции и исправления ошибок, выполняя не только отведенную им работу. Не так очевидно, что эта прозрачность также затрудняет инженеру написание неподходящего кода, который может остаться незамеченным.

Включение исходного кода и связанных с ним артефактов в процесс классификации данных может помочь создать принципиальный подход к защите конфиденциальных активов, одновременно извлекая выгоду из видимости менее конфиденциальных активов. Об этом вы можете подробнее прочитать в главе 21.

Качественные данные и системы, влияющие на безопасность

В среде с нулевым доверием, которая является основой модели наименьших привилегий, каждое детализированное решение по обеспечению безопасности

зависит от двух вещей: применяемой политики и контекста запроса. *Контекст* определяется большим набором данных (часть из них могут быть динамическими), которые могут повлиять на решение. Например, в данных могут быть указаны роль пользователя, группы, к которым он принадлежит, атрибуты клиента, выполняющего запрос, обучающая выборка, введенная в модель машинного обучения, или конфиденциальность API, к которому осуществляется доступ. Ознакомьтесь с системами, которые производят эти данные, чтобы убедиться, что качество данных, влияющих на безопасность, будет как можно более высоким. Из-за данных низкого качества решения по безопасности могут быть неверными.

Влияние на производительность пользователей

Позвольте своим пользователям осуществлять свои рабочие процессы максимально эффективно. Лучшая система безопасности — когда ваши конечные пользователи ее не замечают. Но введение новых трехфакторных и многоступенчатых этапов авторизации может повлиять на производительность пользователей, особенно если им приходится ждать получения авторизации. Вы можете облегчить пользователю жизнь и убедиться, что в новых действиях легко ориентироваться. Точно так же упростите для конечных пользователей способ разобраться в отказах в доступе — через диагностику и самообслуживание или с помощью быстрого доступа к каналу поддержки.

Влияние на сложность для разработчиков

Если во всей вашей организации принята модель с наименьшими привилегиями, разработчики должны ей соответствовать. Даже разработчикам, которые далеки от безопасности, должны быть доступны нужные концепции и политики. Поэтому вам следует предоставить учебные материалы и тщательно задокументировать свои API¹. Пока разработчики будут знакомиться с новыми требованиями, дайте им возможность легко и быстро связываться с инженерами по безопасности для анализа безопасности и консультаций. Развертывать стороннее ПО в такой среде надо осторожно, так как вам может понадобиться обернуть его в дополнительный слой, позволяющий обеспечить соблюдение политики безопасности.

¹ См. раздел «Структура политик для решений аутентификации и авторизации» этой главы.

Резюме

Использование модели с наименьшими привилегиями — самый безопасный способ добиться при проектировании сложной системы того, чтобы клиенты могли выполнять, что им нужно, но не более. Это мощная парадигма проектирования для защиты ваших систем и данных. Она предохраняет от злонамеренного или случайного нарушения, вызванного известными или неизвестными пользователями. В Google мы потратили много времени и усилий на внедрение этой модели. Вот ее ключевые компоненты.

- Всестороннее знание функциональности вашей системы. Вы можете классифицировать разные ее части исходя из имеющегося уровня риска безопасности, который несет каждая из них.
- Разделение вашей системы и доступа к данным на максимально возможном уровне на основе применения этой классификации. Для наименьших привилегий нужны небольшие функциональные API.
- Система аутентификации для проверки учетных данных пользователей при попытке доступа к вашей системе.
- Система авторизации, обеспечивающая четко определенную политику безопасности, которая может быть легко подключена к вашим тонко разделенным системам.
- Набор расширенных средств контроля для детальной авторизации. Они могут, например, обеспечить временное, многофакторное и многостороннее утверждение.
- Набор требований по использованию вашей системы для поддержки этих ключевых концепций. Как минимум необходимо следующее:
 - возможность проверять весь доступ и генерировать сигналы, чтобы вы могли определять угрозы и выполнять исторический анализ в ходе расследования;
 - средства для обоснования, определения, тестирования и отладки вашей политики безопасности, а также для обеспечения поддержки этой политики для конечного пользователя;
 - возможность обеспечить механизм аварийного доступа, когда ваша система не ведет себя так, как ожидалось.

Чтобы все эти компоненты могли легко применяться пользователями и разработчиками, не сильно влияя на их производительность, принимайте наимень-

шие привилегии максимально плавно. Это значит, что нужно контролировать состояние безопасности и взаимодействовать с пользователями и разработчиками, консультируя и поддерживая их по вопросам безопасности, определения политики и обнаружения угроз.

Хотя это может быть масштабным мероприятием, мы твердо верим, что это значительное улучшение по сравнению с существующими подходами к обеспечению безопасности.

ГЛАВА 6

Проектирование для понятности

*Жюльен Бёф, Кристоф Керн и Джон Риз
с Гаем Фишманом, Полом Бланкиништом,
Александрой Калвер, Сергеем Симаковым,
Питером Вальчевым и Дугласом Колишем*

Для уверенности в безопасности вашей системы и ее способности достигать поставленных целей уровня обслуживания (SLO) вам нужно управлять сложностью системы. Вы должны быть способны осмысленно рассуждать о системе, ее компонентах и их взаимодействии. Степень понятности системы может варьироваться в зависимости от различных свойств. Например, легко понять поведение системы при высоких нагрузках, но трудно понять ее поведение, когда она сталкивается со специально созданными (вредоносными) входными данными.

В этой главе обсуждается такое качество системы, как понятность, потому что она относится к каждому этапу ее жизненного цикла. Мы начнем с обсуждения того, как анализировать и понимать системы в соответствии с инвариантами и ментальными моделями. Вы увидите, что благодаря многоуровневой архитектуре, использующей для идентификации, авторизации и контроля доступа стандартизированные фреймворки, можно спроектировать понятную систему. После глубокого погружения в тему границ безопасности мы обсудим, как разработка ПО, особенно использование фреймворков приложений и API, может существенно повлиять на вашу способность рассуждать о свойствах безопасности и надежности.

В этой книге мы рассматриваем *понятность* системы как степень, в которой человек с соответствующим техническим опытом может точно и уверенно рассуждать о следующем:

- операционном поведении системы;
- инвариантах системы, включая безопасность и доступность.

Почему понятность важна?

Разработка системы таким образом, чтобы она была понятной, и поддержание этой понятности на протяжении длительного времени требует усилий. Как правило, эти усилия являются инвестицией, которая окупается в виде постоянной скорости реализации проекта (как обсуждалось в главе 4). В частности, понятная система имеет определенные преимущества.

Уменьшает вероятность появления уязвимостей в системе безопасности или сбоях отказоустойчивости

Каждый раз, когда вы модифицируете систему или программный компонент, например, добавляете функцию, исправляете ошибку или обновляете конфигурацию, существует риск. Вы можете случайно ввести новую уязвимость системы безопасности или поставить под угрозу эксплуатационную устойчивость системы. Если система будет непонятна, то, скорее всего, у инженера, изменяющего ее, будет больше шансов совершить ошибку. Этот инженер может неправильно истолковать поведение системы или не знать о скрытом, неявном или недокументированном требовании, конфликтующем с изменением.

Помогает эффективно реагировать на инциденты

В аварийных ситуациях важно, чтобы сотрудники служб реагирования могли быстро и точно оценить ущерб, локализовать инцидент, выявить и устранить первопричины. Запутанная, сложная для понимания система сильно затрудняет этот процесс.

Повышает доверие к утверждениям о состоянии безопасности системы

Утверждения о безопасности системы обычно выражаются в терминах *инвариантов*: свойств, которые должны сохраняться для *всех возможных* вариантов поведения системы. Они включают в себя реакцию системы на неожиданные взаимодействия с ее внешней средой, например, когда она получает искаженные или злонамеренно созданные данные. Другими словами, поведение системы при получении вредоносных входных данных не должно нарушать требуемое свойство безопасности. В сложной для понимания системе трудно или иногда невозможно с высокой степенью уверенности проверить верность таких утверждений. Часто тестирования бывает недостаточно для демонстрации того, что условие «для всех возможных поведений» выполняется. Тестирование обычно проверяет систему только на относительно небольшую долю возможных вариантов поведения,

соответствующих типичной или ожидаемой операции¹. Обычно, чтобы установить такие свойства, как инварианты, вам нужно полагаться на абстрактные рассуждения о системе.

Инварианты системы

Инвариант системы — это свойство, которое всегда верно, независимо от поведения среды системы (даже неправильного поведения). Система *полностью ответственна* за обеспечение того, чтобы желаемое свойство фактически было инвариантом. Даже при условии, что среда системы ведет себя неожиданно или злонамеренно. Эта среда включает в себя все, над чем у вас нет прямого контроля: от бесчестных пользователей, которые обращаются к интерфейсу вашего сервиса с вредоносными запросами, до сбоев оборудования, приводящих к случайным неполадкам. Одна из наших основных целей при анализе системы — определить, являются ли на самом деле конкретные желаемые свойства инвариантами.

Вот несколько примеров желаемых свойств безопасности и надежности системы.

- Только аутентифицированные и должным образом авторизованные пользователи могут получить доступ к постоянному хранилищу данных системы.
- Все операции с конфиденциальными данными в постоянном хранилище данных системы записываются в журнал аудита в соответствии с политикой аудита системы.
- Все значения, полученные за пределами границы доверия системы, соответствующим образом проверяются или кодируются перед передачей в API, которые подвержены уязвимостям внедрения (например, API SQL-запросов или API для построения разметки HTML).
- Количество запросов, полученных сервером системы, масштабируется относительно количества запросов, полученных ее фронтом.
- Если серверная часть системы не отвечает на запрос по истечении заранее определенного промежутка времени, то ее фронтенд изящно деградирует (<https://oreil.ly/bLTJN>) — например, возвращает приблизительный ответ.
- Когда нагрузка на какой-либо компонент больше, чем этот компонент может выдержать, то для снижения риска каскадного сбоя он будет обрабатывать ошибки перегрузки (<https://oreil.ly/7eJtF>), а не выходить из строя.

¹ Автоматическое нечеткое тестирование, особенно при наличии инструментария и руководства по покрытию, иногда может исследовать большую часть возможных вариантов поведения. Это подробно обсуждается в главе 13.

- Система может принимать вызовы удаленных процедур (RPC) только от заранее заданного набора систем и может отправлять RPC только заранее заданному набору систем.

Если ваша система допускает поведение, нарушающее желаемое свойство безопасности (если указанное свойство на самом деле не является инвариантом), то у нее есть слабое место или уязвимость в системе безопасности. Например, представьте, что свойство 1 из списка неверно для вашей системы, так как в обработчике запросов нет проверок доступа или эти проверки были реализованы неправильно. Теперь у вас есть уязвимость, позволяющая злоумышленнику получить доступ к личным данным ваших пользователей.

Теперь допустим, что ваша система не отвечает четвертому свойству: иногда она генерирует чрезмерное количество запросов на сервер для каждого входящего запроса интерфейса. Например, интерфейс может генерировать несколько повторных попыток в быстрой последовательности (и без соответствующего механизма отката), если первый запрос не выполняется или занимает слишком много времени. У вашей системы есть потенциальный недостаток доступности. Как только система достигнет этого состояния, ее интерфейс может полностью перегружать серверную часть. Из-за этого сервис не будет реагировать на запросы — один из сценариев отказа в обслуживании, вызванного самим собой.

Анализ инвариантов

При анализе того, соответствует ли система заданному инварианту, существует компромисс между потенциальным ущербом, причиняемым нарушениями этого инварианта, и количеством усилий, которые вы тратите на соответствие инварианту и проверку того, что он действительно выполняется. С одной стороны, эти усилия могут включать в себя выполнение нескольких тестов и чтение частей исходного кода для поиска ошибок — например, забытых проверок доступа — которые могут привести к нарушению инварианта. Такой подход не приводит к особенно высокой степени уверенности. Вполне вероятно, что поведение, которое не подверглось тестированию и углубленному анализу кода, будет содержать ошибки. Это говорит о том, что хорошо изученные распространенные классы программных уязвимостей, такие как SQL-инъекция, межсайтовые скрипты (XSS) и переполнение буфера, сохранили лидирующие позиции в списках «самых распространенных уязвимостей»¹. Отсутствие доказательств не является доказательством отсутствия.

¹ Например, опубликованных SANS (<https://oreil.ly/cYTHM>), MITRE (<https://oreil.ly/-XYhE>) и OWASP (<https://oreil.ly/eChGB>).

С другой стороны, вы можете выполнить анализ, основанный на доказуемо обоснованных, формальных рассуждениях. Система и ее заявленные свойства смоделированы в формальной логике, и вы создаете логическое доказательство (обычно с использованием помощника по автоматическому доказательству), что свойство справедливо для системы¹. Такой подход сложен и требует большой работы. Например, один из крупнейших проектов проверки ПО на сегодняшний день создал доказательство (<https://oreil.ly/qxVvk2>) исчерпывающей корректности и свойств безопасности реализации микроядра на уровне машинного кода. Этот проект занял около 20 человеко-лет усилий². Хотя формальная проверка становится практически применимой в определенных ситуациях, таких как наличие микроядра или сложного кода криптографической библиотеки³, она обычно неосуществима для крупномасштабных проектов разработки прикладного программного обеспечения.

В этой главе мы покажем вам практическую середину. Проектируя систему для понятности, вы можете поддержать принципиальные (но все еще неофициальные) аргументы, что система имеет определенные инварианты. И, приложив разумные усилия, вы сможете получить достаточно высокую степень уверенности в этих утверждениях. В Google мы обнаружили, что этот подход полезен для крупномасштабной разработки ПО и эффективен для уменьшения распространенных классов уязвимостей. Дополнительную информацию о тестировании и проверке см. в главе 13.

Ментальные модели

Людям трудно целостно рассуждать об очень сложных системах. На практике инженеры и эксперты в этой области часто строят ментальные модели, объясняющие соответствующее поведение системы, опуская ненужные детали. Для сложной системы вы можете построить несколько ментальных моделей, основанных друг на друге. Так, размышляя о поведении или инвариантах системы

¹ См.: *Murray T., Oorschot P. van.* BP: Formal Proofs, the Fine Print and Side Effects // Proceedings of the 2018 IEEE Cybersecurity Development Conference, 2018. P. 1–10. doi: 10.1109/SecDev.2018.00009.

² См.: *Klein G. et al.* Comprehensive Formal Verification of an OS Microkernel // ACM Transactions on Computer Systems, 2014. № 32 (1). P. 1–70. doi: 10.1145/2560537.

³ См., например: *Erbsen A. et al.* 2019. Simple High-Level Code for Cryptographic Arithmetic — With Proofs, Without Compromises // Proceedings of the 2019 IEEE Symposium on Security and Privacy, 2019. P. 73–90. doi: 10.1109/SP.2019.00005. Для другого примера см.: *Chudnov A. et al.* Continuous Formal Verification of Amazon s2n // Proceedings of the 30th International Conference on Computer Aided Verification, 2018. P. 430–446. doi: 10.1007/978-3-319-96142-2_26.

или подсистемы, вы можете отделить ее от окружающих деталей и базовых компонентов, а вместо этого использовать соответствующие ментальные модели.

Ментальные модели полезны, потому что они упрощают рассуждения о сложной системе. По этой же причине они ограничены. Если вы формируете ментальную модель, основанную на опыте взаимодействия с системой, работающей в определенных условиях, эта модель может не предсказать поведение системы в необычных сценариях. Проектирование безопасности и надежности сильно связано с анализом систем именно в таких необычных условиях. Например, при атаке или перегрузке системы или при отказе компонента.

Рассмотрим систему, пропускная способность которой обычно предсказуемо и постепенно растет с увеличением входящих запросов. Но, преступив границу определенного порога нагрузки, система может достичь состояния, когда она начинает реагировать совершенно иначе. Например, нехватка памяти может привести к перегрузке¹ на уровне виртуальной памяти или на уровне кучи (heap)/сборщика мусора (garbage collector). В результате система не сможет справиться с дополнительной нагрузкой. Слишком большая дополнительная нагрузка может даже *снизить* пропускную способность. Если система находится в таком состоянии во время устранения неполадок, вы можете быть серьезно введены в заблуждение ее слишком упрощенной ментальной моделью, если вы только явно не признаете, что эта модель больше не применима.

Проектируя системы, важно учитывать ментальные модели, которые инженеры по программному обеспечению, безопасности и надежности будут неизбежно создавать для себя. При разработке нового компонента для добавления в более крупную систему в идеале его естественно возникающая ментальная модель должна соответствовать сформированным ранее моделям для подобных существующих подсистем.

По возможности проектируйте системы так, чтобы их ментальные модели оставались предсказуемыми и полезными для работы в экстремальных или необычных условиях. Чтобы избежать перегрузки, вы можете настроить рабочие серверы так, чтобы они работали без подкачки виртуальной памяти на диск. Если производственный сервис не может выделить необходимую для ответа на запрос память, он может быстро вернуть ошибку предсказуемым образом. Даже если сервис с ошибками или неправильным поведением не может обработать ошибку выделения памяти и выходит из строя, вы можете по крайней мере четко связать ошибку с основной проблемой. В данном случае — с нехваткой памяти.

¹ См.: Denning P.J. Thrashing: Its Causes and Prevention // Proceedings of the 1968 Fall Joint Computer Conference, 1968. P. 915–922. doi: 10.1145/1476589.1476705.

Проектирование понятных систем

В оставшейся части этой главы обсуждаются конкретные меры, которые вы можете предпринять для понятности системы, чтобы поддерживать ее по мере развития. Начнем с рассмотрения вопроса о сложности.

Сложность и понятность

Основной враг понятности — *неуправляемая сложность*.

Некоторая сложность все же неизбежна. Это происходит из-за масштаба современных программных систем, особенно распределенных, и проблем, которые они решают. Например, в Google десятки тысяч инженеров работают в хранилище данных, содержащем более миллиарда строк кода. Этот код совместно реализует множество сервисов пользовательского интерфейса, а также серверные части и конвейеры данных, которые их поддерживают. Даже небольшие организации, предлагающие единственный продукт, могут реализовать большое количество функций и пользовательских историй посредством сотен тысяч строк кода, которые редактируются десятками или сотнями инженеров.

Рассмотрим Gmail как пример системы со значительной функциональной сложностью. Вы можете кратко описать Gmail как облачный почтовый сервис, но это краткое описание противоречит его сложности. Вот лишь часть из многих функций Gmail.

- Несколько фронтов и пользовательских интерфейсов (настольные и мобильные веб-приложения, встроенные мобильные приложения).
- Несколько API (<https://oreil.ly/RaYQx>), позволяющих сторонним разработчикам создавать расширения.
- Входящие и исходящие интерфейсы IMAP и POP.
- Обработка вложений, интегрированная с облачным сервисом хранения данных.
- Предоставление вложений во многих форматах, таких как документы и электронные таблицы.
- Веб-клиент с поддержкой автономной работы и встроенной инфраструктурой синхронизации.
- Фильтрация спама.
- Автоматическая категоризация сообщений.

- Системы для извлечения структурированной информации о рейсах, событиях календаря и т. д.
- Корректировка орфографии.
- Функции «Умный ответ» и «Умный ввод».
- Напоминания для ответа на сообщения.

Система с такими функциями сложнее, чем система без них. Но мы же не можем сказать менеджерам продуктов Gmail, что эти функции слишком усложняют работу, и попросить удалить их ради безопасности и надежности. Ведь эти функции обеспечивают ценность и определяют Gmail как продукт. Но если мы будем усердно работать, чтобы справиться с этой сложностью, система все-таки может быть достаточно безопасной и надежной.

Как упоминалось ранее, понятность имеет значение в контексте специфического поведения и свойств систем и подсистем. Наша цель в том, чтобы структурировать проект системы, отделяя и сдерживая эту внутреннюю сложность так, чтобы человек мог с высокой точностью рассуждать об этих *конкретных, релевантных свойствах и поведении системы*. Другими словами, мы должны специально управлять аспектами сложности, которые мешают пониманию.

Конечно, это легче сказать, чем сделать. В оставшейся части этого раздела мы рассмотрим общие источники неуправляемой сложности и соответствующего снижения понятности, а также паттерны проектирования, помогающие контролировать сложность и делать системы более понятными.

Несмотря на то, что наши основные проблемы связаны с безопасностью и надежностью, обсуждаемые нами паттерны не относятся к ним. По большей части они соответствуют общим методам проектирования ПО, нацеленным на управление сложностью и повышение понятности. Вы можете обратиться и к общим текстам по разработке систем и программного обеспечения, таким как *A Philosophy of Software Design* Джона Оустерхаута (Yaknyam Press, 2018).

Разрушение сложности

Для понимания всех аспектов поведения сложной системы вам нужно усвоить и поддерживать большую ментальную модель. Это занятие не из легких.

Сделайте систему понятнее, составив ее из более мелких компонентов. Вы должны уметь рассуждать о каждом компоненте изолированно и объединять их так, чтобы из свойств компонента можно было выводить свойства всей системы.

Такой подход позволяет устанавливать инварианты всей системы без необходимости представлять всю систему за раз.

Непросто воссоздать это на практике. Ваша способность устанавливать свойства подсистем и объединять их в общесистемные зависит от того, как вся система структурирована на компоненты, от характера интерфейсов и доверительных отношений между этими компонентами. Мы рассмотрим эти отношения и связанные с ними соображения в разделе «Архитектура системы» далее.

Централизованная ответственность за требования безопасности и надежности

Как обсуждалось в главе 4, требования безопасности и надежности часто горизонтально применяются ко всем компонентам системы. Например, в требовании безопасности может быть указано, что для любой операции, выполняемой в ответ на запрос пользователя, система должна выполнить некоторую общую задачу (например, ведение журнала аудита и сбор метрик) или проверить некоторые условия (например, аутентификацию и авторизацию).

Если каждый отдельный компонент отвечает за независимую реализацию общих задач и проверок, трудно определить, действительно ли результирующая система удовлетворяет этому требованию. Можно улучшить проект, сделав централизованный компонент (часто библиотеку или фреймворк) ответственным за общие функциональные возможности. Например, фреймворк сервиса RPC может гарантировать, что система реализует аутентификацию, авторизацию и ведение журнала для каждого метода RPC в соответствии с политикой, которая определяется централизованно для всего сервиса. При такой конструкции отдельные методы не отвечают за эти функции безопасности и разработчики приложений не могут забыть реализовать их или сделать это неправильно. Кроме того, специалист по проверке безопасности может понять средства контроля аутентификации и авторизации сервиса, не читая реализацию его каждого отдельного метода. Вместо этого достаточно понять фреймворк и проверить специфичную для сервиса конфигурацию.

Другой пример: для предотвращения каскадных сбоев под нагрузкой должны быть определены тайм-ауты и крайние сроки для входящих запросов. Любая логика, повторяющая сбои, вызванные перегрузкой, должна контролироваться строгими механизмами безопасности. Чтобы реализовать эти политики, вы можете полагаться на код приложения или сервиса для настройки крайних сроков подзапросов и таким образом обрабатывать сбои. Ошибка или упущение в любом соответствующем коде в одном приложении может снизить надежность всей системы. Вы можете сделать систему более устойчивой и понятной

с точки зрения надежности путем включения в базовую структуру сервиса RPC механизмов для поддержки автоматического распространения крайних сроков и централизованной обработки отмены запросов¹.

Эти примеры показывают два преимущества централизованной ответственности за требования безопасности и надежности.

Улучшенная понятность системы

Рецензенту достаточно заглянуть только в одно место, чтобы подтвердить правильность выполнения требования безопасности/надежности.

Повышение вероятности того, что результирующая система работает правильно

Благодаря этому подходу специальная реализация требования в коде приложения не может отсутствовать или быть неправильной.

Несмотря на первоначальные затраты на создание и проверку централизованной реализации как части фреймворка приложений или библиотеки, эти затраты амортизируются во всех приложениях, созданных на основе этой платформы.

Архитектура системы

Структурирование систем по уровням и компонентам — ключевой инструмент для управления сложностью. Пользуясь таким подходом, вы можете рассуждать о частях системы, вместо того чтобы понимать каждую деталь всей системы сразу.

Вам нужно тщательно продумать, как именно стоит разбить систему на компоненты и слои. Слишком тесно связанные компоненты так же трудно понять, как и монолитную систему. Чтобы сделать систему понятной, вы должны уделять столько же внимания границам и интерфейсам между компонентами, сколько и самим компонентам.

Опытные разработчики ПО знают, что система должна рассматривать входные данные из своей внешней среды (и последовательности взаимодействий с ней) как ненадежные и что она не может делать предположения относительно этих данных. В противоположность этому может возникнуть соблазн рассматривать программы, вызывающие внутренние API нижнего уровня (такие как API объектов внутрипроцессной службы или RPC, предоставляемые внутренними серверными микросервисами), в качестве заслуживающих доверия и полагаться

¹ Для получения дополнительной информации см. главу 11 в Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/managing-load/>).

на то, что эти вызывающие остаются в рамках документированных ограничений на использование API.

Предположим, что свойство безопасности системы зависит от правильной работы внутреннего компонента. Еще предположим, что его правильная работа зависит от предварительных условий, обеспечиваемых программами, вызывающими API компонента, таких как правильная последовательность операций или ограничения на значения параметров метода. Определение того, действительно ли система обладает желаемым свойством, требует не только понимания реализации API, но и понимания каждого места вызова API во всей системе и того, обеспечивает ли каждое такое место вызова требуемое предварительное условие.

Чем меньше предположений компонент делает о вызывающих его сущностях, тем проще рассуждать о нем изолированно. В идеале он вообще не делает таких предположений.

Если компонент вынужден делать предположения о сущностях, вызывающих его, важно явно отразить эти предположения при разработке интерфейсов или других ограничений среды. Это можно сделать путем сокращения набора сущностей, способных взаимодействовать с компонентом.

Понятные спецификации интерфейса

Структурированные интерфейсы, согласованные объектные модели и идемпотентные операции способствуют пониманию системы. Как описано в следующих разделах, эти соображения облегчают прогнозирование поведения на выходе и того, как интерфейсы будут взаимодействовать.

Выбирайте узкие интерфейсы, которые предоставляют меньше возможностей для интерпретации

Сервисы могут использовать множество различных моделей и фреймворков для предоставления доступа к интерфейсам. Вот несколько примеров:

- RESTful HTTP с JSON с OpenAPI;
- gRPC;
- Thrift;
- W3C Web Services (XML/WSDL/SOAP);
- CORBA;
- DCOM.

Некоторые из этих моделей очень гибкие, тогда как другие обеспечивают большую структурированность. Например, сервис, использующий gRPC или Thrift, определяет имя каждого поддерживаемого им метода RPC, а также типы его ввода и вывода. Напротив, сервис RESTful в свободной форме может принимать любой HTTP-запрос, а код приложения проверяет, что тело запроса является объектом JSON с ожидаемой структурой.

Фреймворки, поддерживающие определяемые пользователем типы (такие как gRPC, Thrift и OpenAPI), упрощают создание инструментов для некоторых функций. Например, перекрестные ссылки и проверки соответствия, улучшающие открытость и понятность API. Такие структуры обычно со временем обеспечивают и более безопасную эволюцию API. К примеру, OpenAPI имеет встроенную поддержку версий API. Буферы протоколов, используемые для объявления интерфейсов gRPC, обладают хорошо документированными рекомендациями (<https://oreil.ly/yRUQ3>), как обновлять определения сообщений для сохранения обратной совместимости.

Напротив, API, построенный на строках JSON произвольной формы, может быть сложным для понимания, если вы не проверите его код реализации и основную бизнес-логику. Этот неограниченный подход может нарушить безопасность или надежность. Если клиент и сервер обновляются независимо, они могут по-разному интерпретировать полезную нагрузку RPC, что может привести к сбою одного из них.

Отсутствие явной спецификации API тоже затрудняет оценку состояния безопасности сервиса. Если бы у вас не было доступа к определению API, было бы сложно создать автоматическую систему аудита безопасности для соотношения политик, описанных в структуре авторизации (например, политика авторизации Istio (<https://oreil.ly/DjOpK>)), с фактической площадью поверхности, предоставляемой сервисами.

Выбирайте интерфейсы, которые применяют общую объектную модель

Системы, управляющие несколькими типами ресурсов, могут извлечь выгоду из общей объектной модели. Пример системы с общей объектной моделью — Kubernetes. Вместо обработки каждого типа ресурса в отдельности здесь инженеры могут использовать единственную ментальную модель для понимания больших частей системы. Например.

- Каждому объекту в системе может быть гарантированно назначен набор предопределенных базовых свойств (инвариантов).

- Система может предоставлять стандартные способы создания, аннотирования, добавления ссылок и группировки объектов всех типов.
- Операции могут иметь согласованное поведение для всех типов объектов.
- Инженеры могут создавать пользовательские типы объектов для поддержки своих вариантов использования и могут рассуждать об этих типах, используя ту же ментальную модель, что и для встроенных.

У Google есть общие рекомендации по разработке ресурсо-ориентированных API (<https://oreil.ly/AyMVP>).

Обращайте внимание на идемпотентные операции

Идемпотентная операция дает один и тот же результат при многократном выполнении. Если человек нажимает кнопку второго этажа в лифте, лифт будет каждый раз подниматься на второй этаж. Повторное нажатие кнопки, даже несколько раз, не изменит результат.

В распределенных системах идемпотентность важна, потому что операции могут поступать не по порядку или ответ сервера после завершения операции может никогда не вернуться к клиенту. Если метод API идемпотентен, клиент может повторять операцию, пока не получит успешный результат. Если же метод не идемпотентен, системе может понадобиться вторичный способ, такой как опрос сервера, чтобы выяснить, существует ли недавно созданный объект.

Идемпотентность влияет на ментальные модели инженеров. Несоответствие между фактическим и ожидаемым поведением API влечет за собой ненадежные или неверные результаты. Предположим, что клиент хочет добавить запись в БД. Хотя запрос выполнен успешно, ответ не доставлен из-за сброса соединения. Если авторы клиентского кода считают, что операция идемпотентна, клиент, скорее всего, попытается повторить запрос. Но если операция на самом деле не идемпотентна, система создаст дублирующую запись.

Хотя неидемпотентные операции могут быть необходимы, идемпотентные часто приводят к более простой ментальной модели. Когда операция идемпотентна, инженерам (включая разработчиков и тех, кто реагирует на происшествия) не нужно отслеживать время начала операции. Они могут просто пытаться выполнить операцию, пока не узнают о ее успешном проведении.

Одни операции естественно идемпотентны, а другие вы можете сделать идемпотентными, реструктурировав их. В предыдущем примере БД могла бы попросить клиента добавлять уникальный идентификатор (например, UUID) для каждого

RPC, изменяющего ее. Если сервер получает запрос на новое изменение с тем же уникальным идентификатором, он знает, что операция дублируется, и может ответить соответствующим образом.

Понятные идентификационные данные, аутентификация и контроль доступа

Любая система должна уметь определить, кто имеет доступ к каким ресурсам, особенно если эти ресурсы конфиденциальны. Аудитор платежной системы должен понимать, какие инсайдеры имеют доступ к личной информации клиентов. Обычно системы имеют политики авторизации и контроля доступа, ограничивающие доступ данного объекта к данному ресурсу в данном контексте. В этом случае политика будет ограничивать доступ сотрудника к личным данным при обработке кредитных карт. Фреймворк аудита может регистрировать каждый полученный запрос на доступ. Позже вы можете автоматически анализировать журнал доступа как для обычной проверки, так и для расследования происшествий.

Идентификации

Идентификация — это набор атрибутов или идентификаторов, которые относятся к сущности. *Учетные данные* утверждают идентификацию объекта. Они могут принимать разные формы, такие как простой пароль, сертификат X.509 или токен OAuth2. Учетные данные обычно отправляются с использованием определенного *протокола аутентификации*, необходимого системам контроля доступа для идентификации объектов, которые обращаются к ресурсу. Идентификация объектов и выбор модели для нее могут быть сложными. Хотя несложно понять, как система распознает человеческие сущности (как клиентов, так и администраторов), большие системы должны уметь определять все возможные сущности.

Большие системы часто состоят из совокупности микросервисов, вызывающих друг друга, с участием человека или без него. Например, сервису базы данных может понадобиться периодически создавать моментальный снимок для сервиса диска более низкого уровня. Сервис диска может вызвать сервис квотирования, чтобы убедиться, что у сервиса базы данных достаточно выделенного пространства на диске для указанных данных. Или рассмотрим клиента, проходящего аутентификацию в сервисе заказа еды. Клиентская часть сервиса вызывает серверную. Она, в свою очередь, вызывает базу данных для получения пользовательских предпочтений в еде. В общем, *активные объекты* — это набор людей,

программных и аппаратных компонентов, взаимодействующих друг с другом в системе.



Традиционные методы обеспечения безопасности сети иногда используют IP-адреса как основной идентификатор для контроля доступа и для ведения журнала и аудита (например, правила брандмауэра). К сожалению, IP-адреса имеют недостатки в современных системах микросервисов. Из-за того, что им не хватает стабильности и безопасности (и их легко подделать), IP-адреса просто не предоставляют подходящий идентификатор для опознавания сервисов и моделирования уровня их привилегий в системе. Микросервисы развернуты в пулах хостов, а несколько сервисов размещены на одном хосте. Порты не предоставляют строгого идентификатора, так как со временем они могут использоваться повторно или, что еще хуже, произвольно выбираться различными сервисами, запущенными на хосте. Микросервис тоже может обслуживать разные экземпляры, работающие на разных хостах. Это означает, что IP-адрес не может служить стабильным идентификатором.

СВОЙСТВА ИДЕНТИФИКАТОРА, ОБЕСПЕЧИВАЮЩИЕ БЕЗОПАСНОСТЬ И НАДЕЖНОСТЬ

В целом подсистемы идентификации и аутентификации¹ отвечают за моделирование идентификаторов и отправку учетных данных, представляемых этими идентификаторами. «Значимая» идентификация должна:

Иметь понятные идентификаторы

Человек должен иметь возможность понимать, на кого/на что ссылается идентификатор, не проверяя внешний источник. Например, число наподобие 24245223 не очень понятно, а строка типа *widget-store-frontend-prod* четко определяет конкретную рабочую нагрузку. Ошибки в понятных идентификаторах легче обнаружить. Вы, вероятно, сделаете ошибку при изменении списка контроля доступа, если он будет в виде строки произвольных чисел, а не понятных человеку имен. Вредоносное намерение тоже будет более очевидным в понятных человеку идентификаторах, хотя администратор, контролирующий ACL, должен быть осторожным, проверяя идентификатор при добавлении — чтобы защититься от злоумышленников, использующих допустимые идентификаторы.

Не поддаваться фальсификации

Поддержка этого качества зависит от типа учетных данных (пароль, токен, сертификат) и протокола аутентификации, поддерживающего идентификацию. Имя пользователя/пароль или bearer token, используемые в открытом текстовом канале, легко подделать. Гораздо сложнее подделать сертификат с закрытым ключом, который поддерживается аппаратным модулем, таким как TPM (<https://oreil.ly/2PWij>), и используется в сеансе TLS.

¹ В общем случае есть несколько подсистем идентификации. Система может иметь одну подсистему идентификации для внутренних микросервисов и другую подсистему для администраторов-людей.

Иметь одноразовые идентификаторы

Идентификатор нельзя использовать повторно после того, как он был использован для данного объекта. Предположим, что системы контроля доступа компании пользуются адресами электронной почты как идентификаторами. Если привилегированный администратор покидает компанию и новому сотруднику назначается старый адрес электронной почты администраторов, он может наследовать многие из привилегий администратора.

Качество механизмов контроля доступа и аудита зависит от релевантности идентификаторов, используемых в системе, и их доверительных отношений. Присоединение значимого идентификатора ко всем активным объектам в системе — фундаментальный шаг для понятности с точки зрения как безопасности, так и надежности. Со стороны безопасности, идентификаторы помогают определить, у кого и к чему есть доступ. Со стороны надежности идентификаторы помогают планировать и обеспечивать использование общих ресурсов, таких как ЦП, память и пропускная способность сети.

В масштабах организации система идентификации усиливает общую ментальную модель и означает, что весь коллектив при обращении к субъектам может говорить на одном языке. Инженерам и аудиторам понимание дается сложнее при наличии конкурирующих систем идентификации для одних и тех же типов объектов, например, сосуществующих систем глобальных и локальных объектов.

Подобно внешним сервисам обработки платежей в примере заказа виджетов в главе 4, компании могут выводить свои подсистемы идентификации на внешний уровень. OpenID Connect (OIDC) (<https://openid.net/connect>) предоставляет платформу, позволяющую провайдеру утверждать идентификацию. Здесь организация не реализует свою систему идентификации, а несет ответственность только за настройку принимаемых поставщиков. Но, как и в случае со всеми зависимостями, нужно учитывать компромисс — здесь между простотой этой модели и имеющейся безопасностью и надежностью доверенных поставщиков.

Пример: модель идентификации для производственной системы Google

Google моделирует идентичность, используя различные типы активных объектов.

Администраторы

Люди (инженеры Google), предпринимающие действия для изменения состояния системы путем создания новой версии или изменения конфигурации.

Машины

Физические машины в центре обработки данных Google. На них запускаются программы, реализующие наши сервисы (например, Gmail), а также сервисы, в которых нуждается сама система (например, сервис внутреннего времени).

Рабочие нагрузки

Они планируются на машинах системой координации Borg, похожей на Kubernetes¹. Часто идентификация рабочей нагрузки отличается от идентификации компьютеров, на которых она выполняется.

Клиенты

Клиенты Google, получающие доступ к предоставляемым Google сервисам.

Администраторы — основа всех взаимодействий в производственной системе. При взаимодействии между рабочими нагрузками администраторы не могут активно изменять состояние системы, но они инициируют действие запуска рабочей нагрузки (которая может сама запустить другую рабочую нагрузку) на этапе инициализации.

Как описано в главе 5, вы можете пользоваться аудитом для отслеживания всех действий вплоть до самого администратора (или группы администраторов), чтобы установить ответственность и проанализировать уровень привилегий сотрудника. Значимые идентификаторы для администраторов и управляемых ими объектов делают аудит возможным.

Администраторы управляются глобальной службой каталогов, интегрированной с системой единого входа. Глобальная система управления командами может объединять администраторов для представления концепции команд.

Машины занесены в глобальный сервис инвентаризации/БД машин. В производственной сети Google машины получают адреса с использованием DNS. Нам также необходимо связать идентификаторы машины с администраторами для указания тех, кто может изменять ПО, запущенное на машине. На практике мы обычно объединяем группу, которая выпускает образ ПО машины, с группой, которая может войти в систему с правами администратора.

У каждой производственной машины в дата-центрах Google есть свой идентификатор. *Идентификатор* относится к типичному назначению машины. У лабораторных машин, предназначенных для тестирования, есть иденти-

¹ Для получения дополнительной информации о Borg см.: Verma A. et al. Large-Scale Cluster Management at Google with Borg // Proceedings of the European Conference on Computer Systems (EuroSys), 2015. <https://oreil.ly/zgKsd>.

фикационные данные, отличные от данных тех машин, на которых выполняются производственные рабочие нагрузки. Такие программы, как служебные процессы управления машиной, запускающие на ней основные приложения, ссылаются на эту идентификацию.

Рабочие нагрузки планируются на компьютерах с использованием структуры координации. У каждой рабочей нагрузки есть идентификатор, выбранный запрашивающей стороной. Система координации отвечает за то, чтобы объект, создающий запрос, имел право его сделать и мог планировать рабочую нагрузку, выполняемую с запрошенной идентификацией. Система координации ограничивает и выбор машин, на которых можно запланировать рабочую нагрузку. Сами рабочие нагрузки могут выполнять административные задачи, такие как управление группами, но не должны иметь прав администратора на базовом компьютере.

У идентификационных данных *клиентов* тоже есть специализированная подсистема идентификации. Внутренне эти идентификаторы используются каждый раз, когда служба выполняет действие от имени клиента. *Контроль доступа* объясняет, как идентификаторы клиентов работают при взаимодействии с идентификаторами рабочей нагрузки. Внешне Google предоставляет рабочие процессы OpenID Connect (<https://oreil.ly/vxJAP>). Они позволяют клиентам пользоваться своими идентификационными данными Google для аутентификации на конечных точках, не контролируемых Google (таких как *zoom.us*).

Аутентификация и безопасность переноса

Аутентификация и безопасность переноса — сложные техники, требующие специальных знаний в таких областях, как криптография, проектирование протоколов и операционных систем. Не каждый инженер сможет глубоко разобратся во всех этих темах.

Вместо этого инженеры должны понимать абстракции и API. Такая система, как Google Application Layer Transport Security (ALTS) (<https://oreil.ly/EsBfd>), обеспечивает автоматическую аутентификацию между сервисами и безопасность переноса приложений. Так, разработчикам приложений не нужно думать о том, как предоставляются учетные данные или какой конкретный криптографический алгоритм используется для защиты данных в соединении.

Ментальная модель для разработчика приложения проста.

- Приложение запускается со значимым идентификатором:
 - инструмент на рабочей станции администратора для доступа к продукту обычно запускается от имени этого администратора;

- привилегированный процесс на машине запускается с идентификатором этой машины;
- приложение, развернутое как рабочая нагрузка с использованием фреймворка координации, обычно выполняется с идентификатором рабочей нагрузки, специфичным для предоставляемых среды и сервиса (например, `myservice-frontend-prod`).
- ALTS обеспечивает безопасный перенос с нулевой конфигурацией.
- API для общих фреймворков управления доступом извлекает информацию об аутентифицированных партнерах.

ALTS и аналогичные системы, например модель безопасности Istio (<https://oreil.ly/17Jm6>), обеспечивают понятную аутентификацию и безопасность переноса.

Пока в структуре безопасности фреймворка в форме «от приложения к приложению» нет системного подхода, рассуждать о такой системе трудно или вовсе невозможно. Предположим, что разработчики приложений должны сами выбрать тип используемых учетных данных и идентификацию рабочей нагрузки, которую эти данные будут определять. Аудитор должен вручную прочитать весь код приложения, чтобы убедиться в правильности выполнения приложением аутентификации. Такой подход плох для безопасности — он не масштабируется, и некоторая часть кода будет либо непроверенной, либо неверной.

Контроль доступа

Использование фреймворков для кодирования и обеспечения соблюдения политик контроля доступа для входящих запросов к сервису поможет сделать глобальную систему понятной. Фреймворки укрепляют общие знания и дают единый способ описания политик, поэтому являются важной частью инструментария инженера.

Фреймворки могут обрабатывать сложные взаимодействия, такие как множественные идентификаторы, участвующие в передаче данных между рабочими нагрузками. Например, на рис. 6.1 показано следующее.

- Цепочка рабочих нагрузок, работающих как три идентификатора: *Вход*, *Клиент* и *Сервер*.
- Аутентифицированный пользователь делает запрос.

Для каждого звена в цепочке фреймворк должен определить, есть ли у рабочей нагрузки или клиента полномочия для запроса. Политики тоже должны быть достаточно ясными, чтобы можно было решить, какой идентификатор рабочей нагрузки может получать данные от имени клиента.

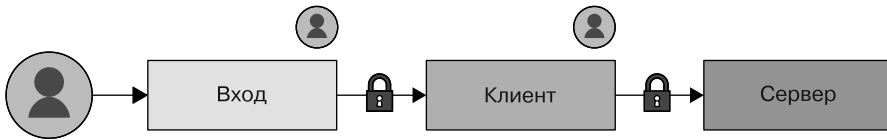


Рис. 6.1. Взаимодействия, связанные с передачей данных между рабочими нагрузками

Такие проверки будут понятны большинству инженеров, если у вас есть единый способ отразить эту внутреннюю сложность. Если бы у каждой сервисной команды была собственная специальная система для работы с одним и тем же сложным сценарием использования, добиться понятности было бы непросто.

Фреймворки помогают последовательно определить и применить декларативные политики контроля доступа, благодаря чему инженеры могут разрабатывать инструменты для оценки степени защиты сервисов и пользовательских данных в фреймворке. Если бы логика управления доступом была реализована специально на уровне кода приложения, разработка этого инструментария была бы невозможна.



Границы безопасности

Основа *доверенной вычислительной базы* (ДВБ) системы — «набор компонентов (аппаратное, программное обеспечение, человек и т. д.), правильного функционирования которых достаточно для обеспечения применения политики безопасности или отказ которых может привести к нарушению политики безопасности»¹. ДВБ должна поддерживать политику безопасности, даже если какой-либо объект *за пределами* ДВБ ведет себя неправильно или злонамеренно. Конечно, область за пределами ДВБ включает в себя внешнюю среду вашей системы (например, злоумышленники в Интернете), но эта область *также* включает части *вашей собственной системы*, которые не входят в ДВБ.

Интерфейс между ДВБ и «всеом остальным» называется *границей безопасности*. «Все остальное» — другие части системы, внешняя среда, клиенты, связанные с ней через сеть, и т. д. — взаимодействуют с ДВБ, связываясь через эту границу. Такая связь может быть выражена как межпроцессный канал связи, сетевые

¹ Anderson R.J. Security Engineering: A Guide to Building Dependable Distributed Systems. Hoboken, N. J.: Wiley, 2008.

пакеты и протоколы более высокого уровня (например, gRPC). ДВБ должна относиться ко всему пересекающему границу безопасности с подозрением — и к самим данным, и к другим деталям, например к порядку сообщений.

Части системы, формирующие ДВБ, зависят от используемой политики безопасности. Полезно задуматься о политиках безопасности и соответствующих ДВБ, необходимых для их поддержки на уровнях. Например, у модели безопасности ОС есть понятие «идентификация пользователя». Она предоставляет политики безопасности, предусматривающие разделение между процессами, выполняемыми под разными пользователями. В Unix-подобных системах процесс, работающий под пользователем А, не должен иметь возможность просматривать или изменять память или сетевой трафик, связанный с процессом, принадлежащим пользователю Б¹. На программном уровне ДВБ, обеспечивающая это свойство, состоит из ядра ОС и всех привилегированных и системных служебных процессов. ОС обычно опирается на механизмы, предоставляемые базовым оборудованием, таким как виртуальная память. Эти механизмы включены в ДВБ, которая относится к политикам безопасности, касающимся разделения пользователей на уровне ОС.

ПО сервера сетевых приложений (например, сервера, представляющего веб-приложение или API) — *не* часть ДВБ этой политики безопасности на уровне ОС, так как работает под непривилегированной ролью уровня ОС (такой как пользователь *httpd*). Но это приложение может применять собственную политику безопасности. Предположим, что у многопользовательского приложения есть политика безопасности, которая делает пользовательские данные доступными только через явные средства контроля совместным использованием документов. В этом случае код приложения (или его части) находится в ДВБ по отношению к этой политике безопасности уровня приложения.

Для уверенности в том, что система применяет требуемую политику безопасности, вы должны понимать всю ДВБ, относящуюся к этой политике безопасности, и уметь рассуждать о ней. По определению, сбой или ошибка в любой части ДВБ может привести к нарушению политики безопасности.

Рассуждение о ДВБ становится сложнее, когда она расширяется, чтобы включать больше кода и сложных компонентов. Поэтому важно сохранять ДВБ как можно меньшей и исключать любые компоненты, не участвующие в поддержке политики безопасности. Помимо ухудшения понятности, включение этих несвязанных компонентов в ДВБ добавляет риск.

¹ Это верно, если пользователь А не обладает правами администратора, и при ряде других конкретных условий. Например, если задействована общая память или если привилегии администратора заданы возможностями Linux.

Вернемся к нашему примеру из главы 4: веб-приложение, позволяющее пользователям покупать виджеты онлайн. Благодаря процессу проверки, запущенному пользовательским интерфейсом приложения, пользователи могут вводить данные кредитной карты и адреса доставки. Система хранит часть этой информации и передает другие сведения (например, данные кредитной карты) стороннему платежному сервису.

Мы хотим гарантировать, что только сами пользователи могут получить доступ к своим личным конфиденциальным данным, таким как адреса доставки. Мы будем использовать ДВБ как надежную вычислительную базу для этого свойства безопасности.

Используя один из множества популярных фреймворков, мы можем получить архитектуру, как на рис. 6.2¹.

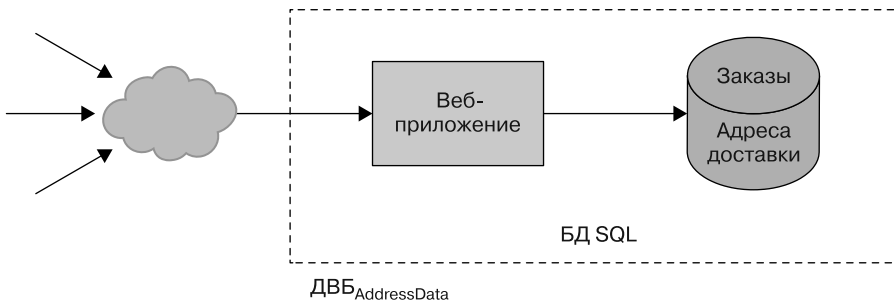


Рис. 6.2. Пример архитектуры приложения, которое продает виджеты

В этом проекте наша система состоит из монолитного веб-приложения и связанной базы данных. Приложение может использовать несколько модулей для реализации разных функций, но все они — часть одной и той же кодовой базы, и все приложение выполняется как один серверный процесс. Аналогично приложение хранит все свои данные в одной базе, и все части сервера имеют доступ на чтение и запись ко всей БД.

Одна из частей приложения обрабатывает оформление и покупку товаров из корзины, а другие хранят информацию, связанную с покупками. Еще часть функциональности работает с покупками, но сама не зависит от функциональности покупок (например, управление содержимым корзины). Тем не менее другие части приложения не имеют ничего общего с покупками (они поддерживают такие функции, как просмотр каталога виджетов или чтение и написание обзоров

¹ Для простоты примера на рис. 6.2 не показаны подключения к внешним поставщикам услуг.

на продукты). Все эти функции — часть одного сервера, и все эти данные хранятся в одной базе. Поэтому все приложение и его зависимости, например сервер БД и ядро ОС, являются частью ДВБ для обеспечения применения политики доступа к данным пользователя.

К рискам относится возможность внедрения SQL в коде поиска в каталоге, что позволит злоумышленнику получать конфиденциальные пользовательские данные, такие как имена или адреса доставки. Сюда же можно отнести уязвимость удаленного выполнения кода на сервере веб-приложений, например CVE-2010-1870 (<https://oreil.ly/y0xRI>), позволяющая злоумышленнику читать или изменять любую часть БД приложения.

Небольшие ДВБ и строгие границы безопасности

Мы можем усилить безопасность нашего проекта, разделив приложение на микросервисы. В этой архитектуре каждый микросервис обрабатывает автономную часть функциональности приложения и сохраняет данные в отдельной базе. Эти микросервисы обмениваются данными друг с другом через RPC и обрабатывают все входящие запросы как не обязательно заслуживающие доверия, даже если вызывающий объект — это другой внутренний микросервис.

Используя микросервисы, мы могли бы реструктурировать приложение, как показано на рис. 6.3.

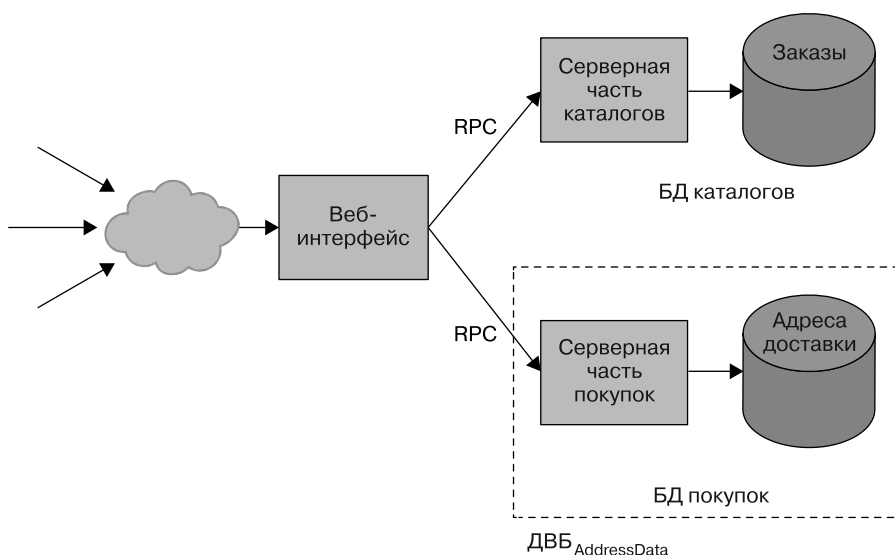


Рис. 6.3. Пример архитектуры микросервисов для приложения по продаже виджетов

Вместо монолитного сервера у нас теперь есть веб-приложение и отдельные серверные части для каталогов продуктов и функций, связанных с покупками. Каждая серверная часть имеет свою отдельную БД¹. Веб-интерфейс никогда не запрашивает БД напрямую. Вместо этого он отправляет RPC в соответствующую серверную часть. Например, веб-интерфейс запрашивает серверную часть каталогов для поиска элементов в каталоге или получения сведений о конкретном элементе. Аналогично внешний интерфейс отправляет RPC в серверную часть покупок, чтобы обработать процесс проверки корзины. Как мы уже обсуждали в этой главе, базовый микросервис и сервер БД могут полагаться на идентификацию рабочей нагрузки и протоколы аутентификации на уровне инфраструктуры, такие как ALTS, для аутентификации вызывающих компонентов и ограничения запросов авторизованными рабочими нагрузками.

В такой новой архитектуре база надежных вычислений для политики безопасности адресных данных меньше. В нее входит только серверная часть покупок и ее БД, а также связанные с ними зависимости. У злоумышленника больше не получится пользоваться уязвимостью в серверной части каталогов, чтобы получить данные о платежах, ведь серверная часть каталогов не может напрямую получить доступ к этим данным. Такой дизайн ограничивает влияние уязвимостей в главном компоненте системы (тема обсуждается в главе 8).

Границы безопасности и модели угроз

Размер и форма доверенной вычислительной базы будут зависеть от желаемого свойства безопасности и архитектуры вашей системы. Вы не можете просто нарисовать пунктирную линию вокруг компонента системы и назвать его ДВБ. Подумайте об интерфейсе компонента и о том, как он может неявно проверять остальную систему.

Представим, что в нашем приложении пользователь может просматривать и обновлять свои адреса доставки. Так как серверная часть сервиса покупок обрабатывает адреса доставки, она должна предоставлять метод RPC, позволяющий веб-интерфейсу получать и обновлять адрес доставки пользователя.

Если серверная часть позволяет веб-интерфейсу получать адрес доставки *любого* пользователя, то взломавший его злоумышленник может использовать этот метод RPC для изменения конфиденциальных данных всех пользователей или доступа

¹ В реальных условиях вы будете использовать одну БД с отдельными группами таблиц, к которым будут предоставлены соответствующие права доступа. Это обеспечивает разделение доступа к данным, в то же время позволяя базе обеспечивать свойства согласованности данных во всех таблицах, такие как ограничения внешнего ключа между содержимым корзины и элементами каталога.

к ним. Другими словами, если серверная часть сервиса покупок доверяет веб-интерфейсу больше, чем случайной третьей стороне, то веб-интерфейс — это часть ДВБ.

Другой вариант: серверная часть может потребовать у интерфейса предоставления тикета конечного пользователя (end-user context ticket, EUC) (<https://oreil.ly/0WkhS>), который аутентифицирует запрос в контексте конкретного запроса внешнего пользователя. EUC — внутренний краткосрочный тикет, который создается центральной службой аутентификации в обмен на внешние учетные данные, такие как cookie-файл аутентификации или связанный с данным запросом токен (например, OAuth2). Если серверная часть предоставляет данные только в ответ на запросы с действительным EUC, то у скомпрометировавшего внешний интерфейс злоумышленника *нет* полного доступа к серверной части сервиса покупок. Так происходит потому, что он не может получить EUC для любого произвольного пользователя. В худшем случае он может получить конфиденциальные данные пользователей, активно использующих приложение во время атаки.

Прежде чем рассмотреть пример того, как ДВБ соотносятся с рассматриваемой моделью угрозы, давайте подумаем, как эта архитектура связана с моделью безопасности веб-платформы¹. В этой модели безопасности *веб-источник* (полное имя хоста сервера, а также протокол и дополнительный порт) является областью доверия. Работающий в контексте данного источника JavaScript-код может изменять любую информацию, присутствующую или доступную для этого контекста. Браузеры же ограничивают доступ между контентом и кодом для разных источников на основе правил, называемых *политикой одного источника*.

Наш веб-интерфейс может обслуживать весь пользовательский интерфейс из единого веб-источника, такого как <https://widgets.example.com>. Это означает, что вредоносный сценарий, внедренный в наш источник через XSS-уязвимость² в пользовательском интерфейсе отображения каталога, может получить доступ к информации профиля пользователя и даже «покупать» элементы от его имени. Так, в модели угроз веб-безопасности ДВБ_{AddressData} снова включает весь веб-интерфейс.

Исправить эту ситуацию можно, сильнее разбив систему и установив дополнительные границы безопасности — в этом случае на основе веб-источников. Как показано на рис. 6.4, мы можем использовать два отдельных веб-интерфейса. Один из них реализует поиск и просмотр каталога и находится на <https://>

¹ *Zalewski M.* The Tangled Web: A Guide to Securing Modern Web Applications. San Francisco, CA: No Starch Press, 2012.

² Там же.

widgets.example.com, а второй отвечает за покупки и располагается на <https://checkout.example.com>¹. Теперь веб-уязвимости вроде XSS в пользовательском интерфейсе каталога не поставят под угрозу функциональность платежей, ведь эти функции отделены собственным веб-источником.

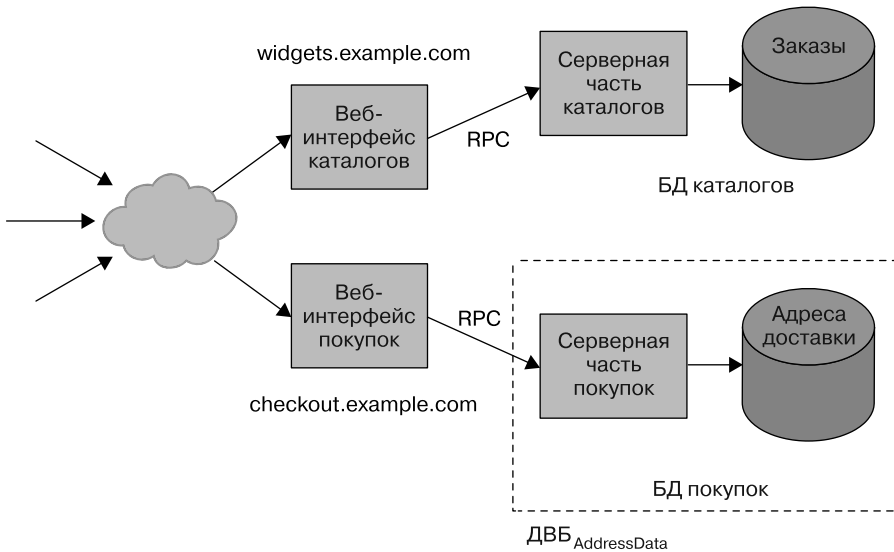


Рис. 6.4. Разбиение веб-интерфейса

ДВБ и понятность

ДВБ и границы безопасности не только обладают преимуществами безопасности, но и облегчают понимание систем. Чтобы квалифицироваться как ДВБ, компонент должен быть изолирован от остальной части системы. Он должен иметь четко определенный, чистый интерфейс, и вы должны иметь возможность рассуждать о правильности реализации ДВБ изолированно. Если правильность компонента зависит от допущений вне его контроля, то это нельзя назвать ДВБ.

ДВБ часто является сама для себя областью отказов. Это облегчает понимание того, как приложение может вести себя при ошибках, DoS-атаках или других операционных воздействиях. В главе 8 мы обсудим преимущества более глубокого разбиения системы.

¹ Важно настраивать веб-серверы так, чтобы интерфейс платежей также *не был* доступен, например, по адресу <https://widgets.example.com/checkout>.

Проектирование программного обеспечения

После структуризации большой системы на компоненты, разделенные границами безопасности, вам все равно придется иметь представление обо всем коде и подкомпонентах внутри заданной границы безопасности, которая часто является большой и сложной частью ПО. В этом разделе обсуждаются методы структурирования ПО, позволяющие дополнительно рассуждать об инвариантах на уровне небольших программных компонентов, таких как модули, библиотеки и API.

Использование фреймворков приложений для общих требований к сервису

Как уже говорилось ранее, фреймворки могут предоставлять части многократно используемой функциональности. У системы может быть фреймворк аутентификации, авторизации, фреймворк RPC, координации, мониторинга, фреймворк выпуска ПО и т. д. Они могут обеспечить большую гибкость — часто *слишком большую*. Все возможные комбинации фреймворков и способы их настройки могут быть непосильными для инженеров, взаимодействующих с сервисом, — разработчиков приложений и сервисов, владельцев сервисов, SR-инженеров и инженеров DevOps.

В Google мы создали фреймворки более высокого уровня для управления этой сложностью. Мы называем их *фреймворками приложений*. Иногда их называют фреймворками *с полным стеком* или фреймворками типа «*все включено*». У таких фреймворков есть канонический набор подфреймворков для отдельных частей функциональности с разумными конфигурациями по умолчанию и гарантией того, что все подфреймворки работают друг с другом. Благодаря фреймворку приложений пользователю не нужно выбирать и настраивать набор подфреймворков.

Предположим, что разработчик приложения предоставляет новый сервис со своим любимым фреймворком RPC. Он устанавливает аутентификацию, используя предпочитаемый фреймворк, но забывает настроить авторизацию и/или контроль доступа. Что касается функций, то новый сервис работает нормально. Но его небезопасно применять без политики авторизации. Любой аутентифицированный клиент (например, каждое приложение в системе) может вызывать этот новый сервис по желанию, нарушая принцип наименьших привилегий (см. главу 5). Эта ситуация может привести к серьезным проблемам безопасности. Например, один метод, предоставляемый сервисом, позволяет

вызывающему компоненту перенастраивать все сетевые коммутаторы в центре обработки данных!

Фреймворк приложения поможет избежать этой проблемы. Он гарантирует действующую политику авторизации для каждого приложения и обеспечивает безопасные значения по умолчанию, запрещая все по умолчанию неразрешенные действия для всех клиентов.

В общем, фреймворки приложений должны обеспечивать продуманный способ включения и настройки всех функций, необходимых разработчикам приложений и владельцам сервисов, в том числе (но не ограничиваясь ими):

- диспетчеризацию и переадресацию запросов, распространение крайних сроков;
- очистку пользовательского ввода и определение локали;
- аутентификацию, авторизацию и аудит доступа к данным;
- ведение журналов и создание отчетов об ошибках;
- управление работоспособностью, мониторинг и диагностику;
- применение квот;
- балансировку нагрузки и управление трафиком;
- развертывание бинарных файлов и конфигураций;
- интеграцию, предварительное тестирование и тестирование нагрузки;
- панели мониторинга и оповещения;
- планирование и предоставление ресурсов.

Платформа приложений решает такие связанные с надежностью проблемы, как мониторинг, оповещение, балансировка нагрузки и планирование ресурсов (см. главу 12). Таким образом, благодаря фреймворку приложения инженеры из нескольких отделов могут говорить на одном языке. Это повышает понимание и взаимодействие между командами.

Понимание сложных потоков данных

Многие свойства безопасности основаны на утверждениях о *значениях*, проходящих через систему.

Например, многие веб-сервисы пользуются URL-адресами для разных целей. Сначала представление URL-адресов в виде строк в системе кажется простым и понятным. Но код и библиотеки приложения могут неявно предполагать, что

URL-адреса сформированы правильно или что URL-адрес имеет определенную схему, например `https`. Такой код неправильный (и может содержать ошибки безопасности), если его вызвать с URL-адресом, нарушающим предположение. Другими словами, можно предположить, что исходящий код, получающий входные данные от ненадежных внешних абонентов, выполняет правильную и надлежащую проверку.

Но по строковому значению нельзя утверждать, что оно представляет правильно сформированный URL. Сам «строковый» тип определяет только то, что значение — это последовательность символов или кодовых точек определенной длины (детали зависят от языка реализации). Любые другие предположения о свойствах значения неявны. Поэтому для рассуждения о правильности кода ниже точки объявления нужно понимать весь код выше ее и то, выполняет ли этот код требуемую проверку.

Вы можете сделать рассуждения о свойствах данных, проходящих через большие и сложные системы, более понятными, если представите значение в виде определенного типа данных, соглашение которого определяет желаемое свойство. В более понятном проекте ваш код ниже точки объявления использует URL-адрес не в форме базового строкового типа, а в форме типа (реализованного, например, как класс `Java`), представляющего *правильно сформированный* URL-адрес¹. Соглашение такого типа может быть реализовано конструкторами или фабричными методами. Например, фабричный метод `Url.parse(String)` будет проводить проверку во время выполнения и будет либо возвращать экземпляр `Url` (представляющий правильно сформированный URL), либо сообщать об ошибке, либо выдавать исключение для искаженного значения.

Благодаря такой конструкции для понимания кода, используемого URL-адресом, корректность которого зависит от правильности его формирования, больше не нужно разбираться во всех вызывающих компонентах и в том, выполняют ли они соответствующую проверку. Вместо этого вам достаточно понять обработку URL, разобравшись с двумя меньшими частями. Во-первых, вы можете проверить реализацию типа `Url` *изолированно*. Можно заметить, что все конструкторы типа гарантируют корректность и соответствие всех экземпляров типа документированному соглашению. После можно *отдельно* рассуждать о правильности кода, использующего значение типа `Url`, применяя соглашение типа (то есть корректность) как предположение в своих рассуждениях.

Такое использование типов улучшает понятность, потому что это может значительно сократить объем кода, который нужно прочитать и проверить. Без типов

¹ Или, в более общем случае, URL-адрес, удовлетворяющий конкретным свойствам, таким как наличие определенной схемы.

вы должны иметь представление обо всем коде, использующем URL-адреса, и о коде, который транзитивно передает URL-адреса в виде простой строки. Представляя URL-адреса как тип, вы должны только понять реализацию проверки данных внутри `Url.parse()` (и аналогичных конструкторов и фабричных методов) и конечное использование `Url`. Вам не нужно понимать остальную часть кода приложения. Он просто передает экземпляры типа.

Реализация типа чем-то похожа на ДББ — она единолично отвечает за свойство «все URL правильно сформированы». Но в широко используемых языках реализации механизмы инкапсуляции для интерфейсов, типов или модулей обычно не представляют границы безопасности. Поэтому вы не можете рассматривать внутренние компоненты модуля как ДББ, которые способны противостоять вредоносному поведению кода вне модуля. Это связано с тем, что в большинстве языков код на внешней стороне границы модуля может изменять внутреннее его состояние (например, с помощью рефлексии или приведения типов). С инкапсуляцией типов вы можете разобраться в поведении модуля изолированно, но *только с предположением*, что окружающий код был написан сторонними разработчиками и что он выполняется в нескомпрометированной среде. На самом деле это разумное предположение на практике. Обычно оно гарантируется средствами управления на уровне организации и инфраструктуры. Например, средствами управления доступом к репозиторию, процессами проверки кода, усилением защиты сервера и т. п. Но если это предположение не верное, вашей команде безопасности придется рассмотреть получившийся наихудший сценарий (см. часть IV).

Вы можете использовать типы для рассуждения о более сложных свойствах, например о предотвращении уязвимостей внедрения (таких как внедрение XSS или SQL). Оно зависит от надлежащей проверки или кодирования любых внешних и потенциально вредоносных входных данных в момент между получением данных и их передачей в подверженный внедрению API.

Чтобы у приложения не было уязвимостей внедрения, требуется понимание всего кода и компонентов, связанных с передачей данных из внешних входных данных в *приемники внедрения* (то есть API, подверженные уязвимостям безопасности, если они представлены с недостаточно проверенными или закодированными входными данными). В типичных приложениях такие потоки данных могут быть очень сложными. Обычно потоки данных есть там, где значения принимаются внешним интерфейсом, проходят через один или несколько уровней серверных частей микросервиса, сохраняются в БД, а после считываются и используются для приемника внедрения. Распространенный класс уязвимостей в таком сценарии — ошибки *хранимых XSS*, когда ненадежные входные данные достигают приемников HTML-внедрений (таких как шаблон HTML или DOM API на стороне браузера) через постоянное хранилище, без проверки

или экранирования. Обычно человеку не по силам рассмотреть и понять объединения всех соответствующих потоков в большом приложении за разумный промежуток времени (даже при наличии инструментов).

Один из эффективных способов предотвращения таких уязвимостей при внедрении с высокой степенью достоверности — использование типов для различения значений, безопасных для применения в конкретном контексте приемника внедрения, например в запросах SQL или разметке HTML¹.

- Конструкторы и фабричные API для таких типов, как `SafeSql` или `SafeHtml`, отвечают за то, чтобы все экземпляры этих типов действительно были безопасны для использования в соответствующем контексте приемника (например, API запроса SQL или контекст рендеринга HTML). Эти API обеспечивают соглашения типов, комбинируя проверки потенциально ненадежных значений во время выполнения и проектирования правильного API. Конструкторы могут полагаться и на более сложные библиотеки, например, полноценные средства проверки HTML или системы шаблонов HTML, применяющие контекстно-зависимое экранирование или подтверждение данных, которые интерполируются в шаблон².
- Приемники модифицируются, чтобы принимать значения соответствующих типов. Соглашение типа утверждает, что его значения безопасны для использования в соответствующем контексте. Это делает типизированный API безопасным по построению. Например, при использовании API запросов SQL, принимающего только значения типа `SafeSql` (вместо `String`), можно не беспокоиться об уязвимостях внедрения SQL, так как все значения типа `SafeSql` безопасны для использования в качестве запроса SQL.
- Приемники могут допускать и значения базовых типов (таких как строки). Но в этом случае они не должны делать какие-либо предположения о безопасности значения в контексте внедрения. Вместо этого API приемника сам отвечает за проверку или кодирование данных, в зависимости от ситуации для гарантии того, что значение безопасно.

С такими конструкциями вы будете уверены в том, что *все приложение* не содержит внедрений SQL или XSS-уязвимостей и основано *исключительно* на понимании реализаций типов и типобезопасных API. Вам не нужно понимать или читать какой-либо код приложения, перенаправляющий значения этих

¹ См.: Kern C. Securing the Tangled Web. Communications of the ACM, 2014. № 57 (9). P. 38–47. doi: 10.1145/2643134.

² См.: Samuel M., Saxena P., Song D. Context-Sensitive AutoSanitization in Web Templating Languages Using Type Qualifiers // Proceedings of the 18th ACM Conference on Computer and Communications Security, 2011. P. 587–600. doi: 10.1145/2046707.2046775.

типов, ведь инкапсуляция типов гарантирует, что код приложения не сможет сделать недействительными инварианты типов, относящиеся к безопасности¹. Вам не нужно понимать и просматривать код приложения, использующий для создания экземпляров безопасные по построению конструкторы типов. Ведь эти конструкторы были разработаны для обеспечения соглашений своих типов без каких-либо предположений относительно поведения вызывающих их компонентов. В главе 12 мы детально обсудим этот подход.

Рассматриваем удобство использования API

Хорошая идея — рассмотреть влияние принятия и использования API на разработчиков вашей организации и их производительность. Если API громоздкие в использовании, разработчики примут их медленно или неохотно. У безопасных по построению API есть двойное преимущество: они делают ваш код понятнее и позволяют разработчикам сосредоточиться на логике приложения. Еще они автоматически встраивают безопасные подходы в культуру вашей организации.

Можно спроектировать библиотеки и фреймворки так, чтобы защищенные API приносили разработчикам чистую выгоду и содействовали культуре безопасности и надежности (глава 21). В обмен на принятие вашего безопасного API, в идеале следующего уже знакомым шаблонам и идиомам, ваши разработчики получают преимущество. Состоит оно в том, что они не несут ответственности за обеспечение инвариантов безопасности, связанных с использованием API.

Например, система шаблонов HTML для автоматического контекстного экранирования полностью ответственна за корректные проверки и экранирование всех данных, интерполированных в шаблон. Это мощный инвариант безопасности для всего приложения. Благодаря ему отображение любого такого шаблона приведет к XSS-уязвимостям, независимо от передаваемых шаблоном (потенциально вредоносных) данных.

С точки зрения разработчика, использование системы шаблонов HTML для автоматического контекстного экранирования похоже на использование обычного

¹ Как мы уже говорили, это утверждение справедливо только с предположением, что вся кодовая база приложения не вредоносна. То есть система типов полагается на защиту инвариантов при несущественных ошибках в других местах кодовой базы, но не при активном вредоносном коде, который может использовать рефлексии API для изменения закрытых полей типа. Эта проблема решается с помощью дополнительных механизмов безопасности, таких как проверки кода, контроль доступа и использование контрольных журналов на уровне исходного хранилища.

шаблона HTML. Вы предоставляете данные, а система шаблонов интерполирует их в заполнители в разметке HTML. Разница лишь в том, что вам больше не нужно беспокоиться о добавлении директив по экранированию или проверке.

Пример: безопасные криптографические API и криптографический фреймворк Tink

Криптографический код сильно подвержен незначительным ошибкам. У многих криптографических примитивов (таких как алгоритмы шифрования и хеширования) есть режимы катастрофических сбоев, необъяснимых для неопытных разработчиков. Например, иногда при неправильном сочетании шифрования и аутентификации (или без аутентификации вообще) злоумышленник может использовать сервис как «декодировщик» и восстановить открытый текст зашифрованных сообщений¹. Неопытный разработчик, не знающий основной техники атаки, не сможет заметить этот недостаток. Зашифрованные данные выглядят совершенно нечитаемыми, а код использует стандартный, рекомендуемый и безопасный шифр, такой как AES. Но из-за некорректного использования номинально защищенного шифра криптографическая схема небезопасна.

По нашему опыту, у кода с использованием криптографических примитивов, не разработанного и не проверенного опытными криптографами, обычно есть серьезные недостатки. Правильно использовать криптографию очень, очень сложно.

Множественные проверки безопасности Google привели к тому, что мы разработали Tink — фреймворк, позволяющий инженерам безопасно пользоваться криптографией (<https://oreil.ly/7G0mD>) в своих приложениях. Tink появился благодаря нашему обширному опыту взаимодействия с командами разработчиков продуктов Google, исправления уязвимостей в криптографических реализациях и предоставления простых API, которые могут безопасно использоваться инженерами без криптографического опыта.

Tink снижает вероятность распространенных криптографических ошибок и предоставляет безопасные API, которые легко использовать правильно. При проектировании и разработке Tink мы руководствовались следующими принципами.

Безопасность по умолчанию

Библиотека предоставляет API, который трудно использовать неправильно. Например, API не разрешает повторное использование одноразовых значе-

¹ См.: Rizzo J., Duong T. Practical Padding Oracle Attacks // Proceedings of the 4th USENIX Conference on Offensive Technologies, 2010. № 1–8. <https://oreil.ly/y-OYm>.

ний в счетчике с аутентификацией Галуа — довольно распространенная, но едва заметная ошибка. Она была обнаружена в RFC 5288 (<https://oreil.ly/3z4CT>), так как позволяет восстановить ключ аутентификации. Это приводит к полному отказу аутентификации в AES-GCM. Благодаря Project Wycheproof (<https://oreil.ly/7UhA7>) Tink использует проверенные и хорошо протестированные библиотеки.

Удобство использования

У фреймворка есть простой и удобный в использовании API. Поэтому разработчик ПО может сосредоточиться на желаемой функциональности, например, на реализации блочных и потоковых примитивов аутентифицированного шифрования с присоединенными данными (Authenticated Encryption с Associated Data, AEAD).

Читаемость и возможность аудита

Функциональность четко читается в коде, и Tink обеспечивает контроль над используемыми криптографическими схемами.

Расширяемость

Легко добавлять новые функции, схемы и форматы, например, через реестр для менеджеров ключей.

Динамичность

У Tink есть встроенная замена ключей, и он поддерживает удаление устаревших/испорченных схем.

Интеграция с внешними системами

Tink доступен на многих языках и платформах.

У Tink есть решение для управления ключами. Он интегрируется с облачной службой управления ключами (Cloud Key Management Service) (<https://oreil.ly/k5A3c>), службой управления ключами AWS (AWS Key Management Service) (<https://oreil.ly/nzkF9>) и хранилищем ключей на Android (Android Keystore) (<https://oreil.ly/PUkYz>). Многие криптографические библиотеки упрощают хранение закрытых ключей на диске и делают их добавление в исходный код (крайне нерекомендуемая практика) еще проще. Даже если вы выполняете действия «поиск ключей» и «поиск паролей», чтобы найти и удалить секреты из вашей кодовой базы и систем хранения, трудно полностью устранить неполадки, связанные с управлением ключами. Tink API не принимает ключи напрямую — вместо этого API поощряет использование службы управления ключами.

Google пользуется Tink, чтобы обезопасить данные многих продуктов. Теперь это рекомендуемый фреймворк для защиты данных в Google и при общении с третьими лицами. Она предоставляет абстракции с понятными свойствами (такими как «аутентифицированное шифрование»), подкрепленные хорошо спроектированными разработками, позволяя инженерам по безопасности сосредоточиться на аспектах криптографического кода более высокого уровня. А заботиться о низкоуровневых атаках на базовые криптографические примитивы им больше не нужно.

Здесь важно отметить, что Tink не может предотвратить высокоуровневые ошибки проектирования в криптографическом коде. Например, неопытный в криптографии разработчик ПО может выбрать защиту конфиденциальных данных через хеширование. Это небезопасно для данных, относящихся к набору (в криптографическом выражении), скромному по размеру (например, номера кредитных карт или социального страхования). Использовать криптографический хеш в таком сценарии вместо аутентифицированного шифрования — ошибка уровня разработки, которая проявляется выше уровня детализации API Tink. Обозреватель безопасности не может сделать вывод, что таких ошибок нет в приложении только потому, что в коде используется Tink вместо другой криптографической библиотеки.

Разработчики и проверяющие код ПО должны позаботиться о том, чтобы понять, какие свойства безопасности и надежности библиотека или фреймворк гарантирует, а какие нет. Tink предотвращает многие ошибки, которые могут привести к низкоуровневым криптографическим уязвимостям, но не предотвращает ошибки, основанные на использовании неправильного API шифрования (или неиспользовании шифрования вообще). Точно так же безопасный по построению веб-фреймворк предотвращает XSS-уязвимости, но не предотвращает ошибки безопасности в бизнес-логике приложения.

Резюме

Надежность и безопасность систем тесно связаны с их понятностью.

Хотя иногда «надежность» и синоним «доступности», этот атрибут действительно означает соблюдение всех критических гарантий проектирования системы — доступности, долговечности, инвариантов безопасности и многих других.

Для построения понятной системы мы рекомендуем создать ее из компонентов, имеющих четкие и ограниченные цели. Некоторые из них могут составлять свою надежную вычислительную базу и концентрировать ответственность за устранение угроз безопасности.

Мы обсудили и стратегии обеспечения желаемых свойств — таких как инварианты безопасности, устойчивость архитектуры и устойчивость данных — внутри и между этими компонентами. Такие стратегии включают в себя следующее:

- узкие, согласованные, типизированные интерфейсы;
- согласованные и тщательно реализованные стратегии аутентификации, авторизации и учета;
- четкое назначение идентификаторов активным объектам, независимо от того, являются ли они программными компонентами или людьми-администраторами;
- библиотеки фреймворков приложений и типы данных, инкапсулирующих инварианты безопасности для того, чтобы компоненты последовательно следовали рекомендациям.

Когда наиболее критические части вашей системы работают с перебоями, понятность системы может стать решающим фактором. SR-инженеры должны знать инварианты безопасности системы, чтобы выполнять свою работу. Иногда они могут отключить сервис во время происшествия, жертвуя доступностью ради безопасности.

ГЛАВА 7

Проектирование для меняющегося ландшафта

*Майя Качоровски, Джон Ланни
и Дениз Песел с Джен Барнасон,
Питером Даффом и Эмили Старк*

Ваша способность адаптироваться при выполнении целей уровня обслуживания, обещанных пользователям, зависит от устойчивости и гибкости возможностей надежности вашего сервиса. Такие инструменты и подходы, как частые сборки, выпуски с автоматизированным тестированием, контейнеры и микросервисы, позволят вам адаптироваться к кратко-, средне-, долгосрочным изменениям и к неожиданным сложностям, возникающим при запуске сервиса. В этой главе вы найдете множество примеров того, как Google изменил и адаптировал свои системы за последние годы, и прочтете об уроках, усвоенных нами на этом пути.

Большинство проектных решений, особенно связанных с архитектурой, проще и дешевле реализовать на этапе проектирования системы. Но многие из лучших практик, описанных в этой главе, могут быть реализованы на более поздних этапах.

«Изменения — это единственная константа»¹ — принцип, который, безусловно, относится к программному обеспечению. Чем больше количество и разнообразие используемых нами устройств, тем больше количество уязвимостей библиотек и приложений. Любое устройство или приложение может быть подвержено утечке данных, захвату бот-сетью или другим вариантам взлома.

У пользователей и контрольно-надзорных органов продолжают расти ожидания к безопасности и конфиденциальности. Это требует более строгих мер управления, таких как ограничения доступа для конкретных предприятий и систем аутентификации.

¹ Приписывается Гераклиту Эфесскому (<https://oreil.ly/SUdXz>).

Чтобы реагировать на эти виды уязвимостей, ожиданий и рисков, вы должны уметь часто и быстро менять свою инфраструктуру, предоставляя высоконадежное обслуживание, а это нелегкая задача. Достижение такого баланса часто сводится к тому, чтобы решить, когда и как быстро предоставить изменение.

Типы изменений безопасности

Есть много видов изменений, которые вы можете внести для улучшения состояния безопасности или устойчивости инфраструктуры безопасности, например:

- изменения в ответ на происшествия в области безопасности (см. главу 18);
- изменения в ответ на недавно обнаруженные уязвимости;
- изменения продукта или функций;
- изменения, вызванные внутренними факторами, для улучшения состояния безопасности;
- изменения, вызванные внешними причинами, например новые нормативные требования.

Для некоторых типов изменений, связанных с безопасностью, нужны дополнительные факторы. Для развертывания необязательной функции с целью сделать ее обязательной вам необходимо собрать достаточную обратную связь от первых пользователей и тщательно протестировать оборудование.

Если вы хотите изменить зависимость кода от поставщика или стороннего производителя, убедитесь, что новое решение соответствует требованиям безопасности.

Проектирование ваших изменений

На изменения безопасности распространяются те же основные требования к надежности и принципы разработки выпусков, что и на любые другие изменения ПО. Для получения дополнительной информации см. главу 4 в этой книге и главу 8 в книге *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/release-engineering/>). Временные рамки для развертывания изменений в системе безопасности могут различаться (см. раздел «Разные изменения: разные скорости, разные сроки» далее в этой главе), но общий процесс должен следовать тем же рекомендациям.

Все изменения должны быть:

Инкрементны

Вносите небольшие и автономные изменения. Избегайте искушения связать изменения с несвязанными улучшениями, такими как рефакторинг кода.

Документированы

Распишите ответы на «как» и «почему» для ваших изменений, чтобы другие могли понять суть изменения и срочность его внедрения. Ваша документация может включать в себя что-то или все из следующего:

- требования;
- системы и команды в области применения;
- уроки, извлеченные из доказательства концепции;
- обоснование решений (при необходимости переоценки планов);
- пункты связи для всех участвующих команд.

Протестированы

Проверьте изменения безопасности с помощью модульных и интеграционных тестов, где это возможно (дополнительную информацию о тестировании см. в главе 13). Завершите экспертную оценку для уверенности в том, что изменения работают в производстве.

Изолированы

Чтобы изолировать изменения друг от друга и избежать несовместимости версий, используйте флаги функций. Для получения дополнительной информации см. главу 16 в книге Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/canarying-releases/#separating-components-that-change-at-different-rates>). При отключении функций у базового бинарного файла не должно быть никаких изменений в поведении.

Квалифицированы

Внедрите свои изменения с помощью обычного процесса выпуска, проходя этапы квалификации до получения производственного или пользовательского трафика.

Постепенны

Раскатайте свои изменения постепенно, используя инструменты для канареечного тестирования. Вы должны видеть различия в поведении до и после изменений.

Эти методы предлагают использовать «медленный и устойчивый» подход к развертыванию. По нашему опыту, важен сознательный компромисс между скоростью и безопасностью. Риск может привести к более серьезным проблемам вроде повсеместного простоя или потери данных, если изменение что-то нарушит.

Архитектурные решения для простых изменений

Как спроектировать инфраструктуру и процессы, чтобы они реагировали на неизбежные изменения? Здесь мы обсуждаем стратегии, позволяющие делать систему более гибкой и вносить изменения с минимальными трудностями. Это приводит к формированию культуры безопасности и надежности (обсуждается в главе 21).

Постоянно обновляйте зависимости и регулярно пересобирайте код

Убедитесь, что ваш код ссылается на последние версии зависимостей. Это делает вашу систему менее восприимчивой к новым уязвимостям. Постоянное обновление ссылок на зависимости особенно важно для часто меняющихся проектов с открытым исходным кодом, таких как OpenSSL или ядро Linux. У многих крупных проектов с открытым исходным кодом есть хорошо разработанные планы реагирования на уязвимости безопасности и исправления, уточняющие, содержит ли новый выпуск критическое изменение безопасности, и переносимые в поддерживаемые версии. При обновленных зависимостях вы можете применить критическое исправление напрямую, а не объединять его с еще не готовыми изменениями или внести несколько исправлений.

Новые выпуски с их исправлениями безопасности не попадут в среду до повторной сборки. Чаще выполняйте сборку и повторное развертывание среды. Благодаря этому вы будете готовы развернуть новую версию, когда это потребуется, а при экстренном развертывании будут использованы последние изменения.

Частые выпуски с использованием автоматизированного тестирования

Один из основных принципов SRE — регулярно создавать и обновлять выпуски для облегчения аварийных изменений. Разбивая один большой выпуск на много малых, вы гарантируете, что каждый выпуск содержит меньше изменений, что с меньшей вероятностью потребует отката. Чтобы глубже изучить эту тему, см. магический цикл, изображенный на рис. 16.1 в книге Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/canarying-releases/#the-virtuous-cycle-of-ci-cd>).

Чем меньше у каждого выпуска изменений в коде, тем легче понять, что изменилось, и точно определить возможные проблемы. Если понадобится внедрить изменение безопасности, вы можете быть более уверенными в результате.

Автоматизируйте тестирование и проверку частых выпусков, чтобы полностью раскрыть их потенциал. Благодаря этому вы сможете автоматически создавать хорошие выпуски, не допуская попадания в производство сырых вариантов. Автоматическое тестирование дает дополнительную уверенность, когда нужно выпустить исправления, защищающие от критических уязвимостей.

Используя контейнеры¹ и микросервисы², вы можете уменьшить объем исправлений, наладить регулярные процессы выпуска и упростить понимание уязвимостей системы.

Использование контейнеров

Контейнеры отделяют бинарные файлы и библиотеки, необходимые для вашего приложения, от базовой операционной системы. Каждое приложение упаковано с собственными зависимостями и библиотеками, поэтому ОС хоста не должна включать их. Приложения станут более переносимыми, и вы сами сможете защитить их. Например, можно исправить уязвимость ядра в ОС хоста без изменения контейнера приложения.

Контейнеры должны быть неизменяемыми, то есть не меняться после развертывания. Вместо подключения к машине по протоколу SSH вы пересобираете и заново развертываете весь образ. Контейнеры недолговечны, поэтому они часто собираются и развертываются повторно.

Исправляйте образы в реестре контейнеров, а не контейнеры напрямую. Это позволит развернуть полностью исправленный образ контейнера как одно целое и сделает сам процесс таким же, как и ваш (очень частый) процесс развертывания кода — вместе с мониторингом, канареечным и прочим тестированием. В результате можно будет чаще добавлять исправления.

¹ Для получения дополнительной информации о контейнерах см. запись в блоге: *Lorenc D., Kaczorowski M.* Exploring Container Security (<https://oreil.ly/i9DTQ>). См. также: *Burns B. et al.* Borg, Omega and Kubernetes: Lessons Learned from Three Container-Management Systems Over a Decade // ACM Queue, 2016. № 14 (1). <https://oreil.ly/tDKBJ>.

² Для получения дополнительной информации о микросервисах см. «Пример 4. Запуск сотен микросервисов на общей платформе» в главе 7 книги *Site Reliability Workbook* (<https://landing.google.com/sre/workbook/chapters/simplicity/>).

По мере того, как эти изменения распространяются на каждую задачу, система плавно перемещает обслуживающий трафик в каждый новый экземпляр (см. «Пример 4. Запуск сотен микросервисов на общей платформе» в главе 7 книги *Site Reliability Workbook* (<https://landing.google.com/sre/workbook/chapters/simplicity/>)). Вы можете достичь таких результатов и избежать простоев при установке исправлений с голубыми/зелеными развертываниями (см. главу 16 в *Site Reliability Workbook* (<https://landing.google.com/sre/workbook/chapters/canarying-releases/>)).

Контейнеры можно использовать для обнаружения и исправления недавно появившихся уязвимостей. Из-за того, что контейнеры неизменяемы, содержащее в них будет доступным. Другими словами, вы на самом деле знаете, что работает в вашей среде, например, какие образы развернуты. Допустим, вы ранее развернули полностью исправленный образ, который может быть подвержен новой уязвимости. Вы можете использовать реестр для определения уязвимых версий и применения исправлений, чтобы не сканировать производственные кластеры напрямую.

Чтобы уменьшить потребность в таких специальных исправлениях, следите за возрастом контейнеров, запущенных в производстве, и регулярно их развертывайте, чтобы убедиться, что у старых контейнеров нет влияния на код. А чтобы избежать повторного развертывания старых неисправленных образов, разворачивайте в рабочей среде только недавно созданные контейнеры.

Использование микросервисов

Идеальная системная архитектура легко масштабируется, обеспечивает представление о производительности системы и позволяет управлять каждым потенциальным узким местом между сервисами в вашей инфраструктуре. Благодаря архитектуре микросервисов вы можете разделить рабочие нагрузки на более мелкие, более управляемые блоки для облегчения обслуживания и обнаружения. Вы можете независимо масштабировать, балансировать нагрузку и выполнять развертывание в каждом микросервисе. Это означает возможность более гибкого внесения изменений в инфраструктуру. Поскольку каждый сервис обрабатывает запросы отдельно, для обеспечения глубокой защиты вы можете использовать несколько защит независимо и последовательно (см. раздел «Защита в глубину» главы 8).

Благодаря микросервисам легче создавать сети с ограниченным или нулевым доверием. Это значит, что ваша система не станет доверять сервису только на основании того, что они находятся в одной сети (см. главу 6). Вместо использования модели безопасности по периметру с ненадежным внешним и доверенным

внутренним трафиком микросервисы пользуются доверием изнутри. У внутреннего трафика могут быть разные уровни доверия. Текущие тенденции движутся в сторону более сегментированной сети. Поскольку зависимость от одной внешней границы сети, например брандмауэра, устранена, сеть может быть дополнительно сегментирована сервисами. В крайнем случае можно реализовать сегментацию на уровне микросервисов без внутреннего доверия между сервисами.

Вторичный результат использования микросервисов — это взаимодействие инструментов безопасности. Поэтому некоторые процессы, инструменты и зависимости могут повторно использоваться несколькими командами. Для масштабирования вашей архитектуры может быть полезно объединить усилия по удовлетворению общих требований безопасности. Например, с помощью общих криптографических библиотек или общей инфраструктуры мониторинга и оповещения. Так вы можете разделить критически важные службы безопасности на отдельные микросервисы, которые обновляются и управляются небольшим количеством ответственных сторон. Помните: для достижения преимуществ безопасности архитектуры микросервисов нужны ограничения. Они необходимы для обеспечения максимально простой работы служб при сохранении требуемых свойств безопасности.

Благодаря использованию архитектуры микросервисов и процесса разработки команды могут решать вопросы безопасности стандартизированно на ранних этапах жизненного цикла разработки и развертывания — когда вносить изменения менее затратно. Так разработчики могут достичь безопасных результатов, не потратив на безопасность много времени.

Пример: модель внешнего интерфейса Google

Модель внешнего интерфейса Google использует микросервисы для обеспечения устойчивости и глубокой защиты¹. У разделения внешнего и внутреннего интерфейсов на разные уровни много преимуществ. Внешний интерфейс Google (Google Front End, GFE) используется как внешний для большинства сервисов Google, реализованных в виде микросервисов. Поэтому у таких служб нет прямого доступа в Интернет. GFE тоже ограничивает трафик для входящих HTTP(S), TCP и TLS-прокси; обеспечивает противодействие DDoS-атакам; распределяет и балансирует нагрузку на облачные сервисы Google².

¹ См. главу 2 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/production-environment/>).

² См. документ Encryption in Transit in Google Cloud (<https://oreil.ly/kZQNh>) для получения дополнительной информации.

GFE обеспечивает независимое разделение внешних и внутренних служб. Это дает преимущества для масштабируемости, надежности, гибкости и безопасности.

- Глобальная балансировка нагрузки помогает передавать трафик между GFE и внутренними компонентами. Например, мы можем перенаправить трафик во время простоя центра обработки данных, сократив время устранения неполадки.
- Уровни внутреннего и внешнего интерфейса могут иметь несколько своих внутренних подуровней. Каждый уровень является микросервисом, и мы можем сбалансировать нагрузку для всех. В результате можно просто добавить емкость, внести общие изменения или применить быстрые изменения к каждому микросервису.
- Если сервис перегружен, GFE может быть точкой смягчения, отбрасывая или поглощая соединения до того, как нагрузка достигнет серверных частей. Это означает, что не каждому уровню в архитектуре микросервисов нужна собственная защита от нагрузки.
- Принимать новые протоколы и требования безопасности относительно просто. GFE может обрабатывать соединения IPv6, даже если некоторые серверные части еще не готовы к ним. GFE упрощает управление сертификатами, выступая как точка подключения для различных общих сервисов, таких как SSL¹. Например, когда была обнаружена уязвимость (<https://oreil.ly/XDPI2>) при пересмотре SSL, контроль GFE за ограничением этих пересмотров защищал все сервисы, стоящие за ним. Быстрое внедрение шифрования безопасности переноса на уровне приложений (Application Layer Transport Security (<https://oreil.ly/IRkJI>)) показывает, как архитектура микросервисов облегчает принятие изменений. Команда безопасности Google внедрила библиотеку ALTS в свою библиотеку RPC для обработки учетных данных сервиса. Это обеспечило широкое внедрение без особой нагрузки на отдельные команды разработчиков.

Добиться преимуществ, описанных здесь, будет несложно в современном облачном мире. Используйте архитектуру микросервисов, создавая уровни управления безопасностью и управляя межсервисными коммуникациями путем объединения сервисов. Вы можете отделить обработку запросов от конфигурации для управления их обработкой. Этот тип преднамеренного разделения называется *разделением плоскости данных* (запросов) и *плоскости управления* (конфигурации). В этой модели плоскость данных позволяет фактически обрабатывать данные в системе, обеспечивая балансировку нагрузки, безопасность и наблюдаемость. Плоскость управления позволяет задавать политики и конфигурации для сервисов плоскости данных.

¹ Там же.

Разные изменения: разные скорости, разные сроки

Не все изменения происходят в одинаковые сроки или с одинаковой скоростью. На скорость внесения ваших изменений влияют несколько факторов.

Серьезность

Уязвимости обнаруживаются ежедневно, но не все они критически важны, активно используются или применимы к вашей инфраструктуре. Если в вашей ситуации все-таки сработал один из этих факторов, вы наверняка захотите выпустить исправление как можно быстрее. Ускорение сроков опасно и может привести к поломке системы. Иногда скорость важна, но медленные изменения более безобидны — так вы можете обеспечить продукту достаточную безопасность и надежность (в идеале можно применить критическое исправление безопасности независимо — так его можно провести быстро, но без излишнего ускорения любых других запланированных развертываний).

Зависимые системы и команды

Некоторые системные изменения могут зависеть от других команд, которым нужно внедрить новые политики или включить определенную функцию до развертывания. Ваше изменение может зависеть от внешней стороны — например, если нужно получить исправление от поставщика или если клиентская часть должна быть исправлена до серверной.

Важность

Значимость ваших изменений может повлиять на развертывание в рабочей среде. Несущественные изменения, улучшающие безопасность организации в целом, не обязательно такие же срочные, как критическое исправление. Такое несущественное изменение можно развернуть более постепенно. Например, от одной команды к другой. В зависимости от дополнительных факторов внесение изменений может не стоить риска. Можно отказаться от внедрения несрочных изменений в критические периоды производственной среды — например, в сезон праздничных покупок, когда изменения жестко контролируются.

Крайний срок

У некоторых изменений есть крайний срок. Изменение в нормативной базе может иметь конкретный срок введения в действие, или вам может понадобиться применить исправление до того, как на уязвимость будет наложено эмбарго (см. следующую врезку).

НАЛОЖЕНИЕ ЭМБАРГО НА УЯЗВИМОСТИ

Известные уязвимости, публично не раскрытые до этого, может быть сложно исправить. Информация, на которую *распространяется эмбарго*, не может быть раскрыта до определенной даты. Это может быть дата выпуска исправления или дата запланированного обнародования проблемы.

Если вы знакомы с информацией об уязвимости под эмбарго и внесение исправлений нарушит его, вы должны подождать публичного объявления, прежде чем вносить исправления. Если вы участвуете в реагировании на происшествия до объявления об уязвимости, поработайте с другими сторонами, чтобы договориться о дне, который подходит для процессов развертывания в большинстве организаций.

Нет строгого правила для определения скорости конкретного изменения. Оно может потребовать быстрой смены конфигурации и развертывания в одной организации и занять месяцы в другой. Пока одна команда может внести определенное изменение в соответствии с графиком, вашей организации может понадобиться целый ряд соглашений, чтобы полностью принять это изменение.

В следующих разделах мы обсудим три разных временных горизонта для изменений и приведем примеры, чтобы показать, как каждый вариант выглядел в Google:

- краткосрочное изменение для исправления новой уязвимости безопасности;
- среднесрочное изменение при постепенном принятии нового продукта;
- долгосрочное изменение по нормативным причинам — пришлось создавать новые системы для реализации изменения в Google.

Краткосрочное изменение: уязвимость нулевого дня

Недавно обнаруженные уязвимости часто нуждаются в краткосрочных действиях. *Уязвимость нулевого дня* — это уязвимость, известная некоторым злоумышленникам, но не обнародованная или не обнаруженная целевым поставщиком инфраструктуры. Обычно исправление либо еще не доступно, либо широко не применяется.

Есть много способов узнать о новых уязвимостях, которые могут повлиять на вашу среду. Например, регулярные проверки кода, сканирование внутреннего кода (см. подраздел «Очистка кода» в главе 12), нечеткое тестирование (см. раздел «Нечеткое тестирование» главы 13), внешние проверки, такие как тесты на проникновение, сканирование инфраструктуры и поиск ошибок программ.

В контексте краткосрочных изменений мы сосредоточимся на уязвимостях нулевого дня. Хотя Google часто реагирует на уязвимости, на которые наложено

эмбарго, например при разработке патчей, краткосрочные изменения для уязвимостей нулевого дня — обычное поведение для большинства таких организаций.



Уязвимости нулевого дня привлекают к себе много внимания (как внешне, так и внутри организации), но они не обязательно часто используются злоумышленниками. Не торопитесь реагировать на уязвимости нулевого дня в тот же день, а убедитесь, что у вас есть патч для «самых популярных» проблем, чтобы покрыть критические уязвимости за последние годы.

Обнаружив новую уязвимость, выполните сортировку для определения серьезности и влияния этой уязвимости. Например, уязвимость, позволяющая удалить выполнение кода, может считаться критической. Но влияние на вашу организацию может быть очень трудно определить. Какие системы используют этот конкретный бинарный файл? Развернута ли уязвимая версия в производстве? По возможности установите постоянный мониторинг и выдачу оповещений, чтобы определить, активно ли используется уязвимость.

Чтобы принять меры, вам нужно получить *патч* — новую версию уязвимого пакета или библиотеки с примененным исправлением. Сначала проверьте, действительно ли патч устраняет уязвимость. Для этого может пригодиться работающий эксплойт. Но имейте в виду, что даже если вы не можете вызвать уязвимость с его помощью, ваша система все равно может оказаться незащищенной (напомним, что отсутствие доказательств не является доказательством отсутствия). Например, примененный патч может устранить только один возможный вариант более широкого класса уязвимостей.

После проверки патча разверните его — в идеале в тестовой среде. Даже при ускоренных сроках развертывание должно быть постепенным, как и любые другие производственные изменения — с использованием тех же средств тестирования, канареечного тестирования и других инструментов (порядка часов или дней)¹. Благодаря постепенному развертыванию вы можете раньше обнаружить возможные проблемы, поскольку патч может неожиданно повлиять на ваши приложения. Например, приложение, использующее API, о котором вы не знали, может повлиять на характеристики производительности или вызвать другие ошибки.

Иногда невозможно напрямую исправить уязвимость. В этом случае лучший вариант — смягчить риск путем сдерживания или иного ограничения доступа к уязвимым компонентам. Это смягчение может быть временным, пока патч

¹ См. обсуждение постепенного и поэтапного развертывания в главе 27 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/reliable-product-launches/>).

не станет доступным, или постоянным, если вы не можете применить патч к своей системе — например, из-за требований к производительности. Если вы уже приняли меры по снижению риска для защиты вашей среды, вам даже не потребуется предпринимать никаких дальнейших действий.

Подробнее о реагировании на инциденты см. в главе 17.

Пример: Shellshock

Утром 24 сентября 2014 года Google Security узнала о публично раскрытой (<https://oreil.ly/mQFJj>), дистанционно управляемой уязвимости в bash (<https://oreil.ly/qbTqD>), которая позволяла выполнять код в уязвимых системах. После закрытия уязвимости мы сразу же попытались ее оценить.

Первоначальный отчет (<https://oreil.ly/4l2W4>) содержал запутанные технические детали, но в нем не было четкого описания эмбарго на обсуждение этого вопроса. Этот отчет, вместе с обнаружением нескольких подобных уязвимостей, вызвал путаницу в отношении характера и возможности атаки. Команда Google по реагированию на инциденты инициировала протокол «Черный лебедь» для устранения исключительной уязвимости и запланировала (<https://oreil.ly/v6DeI>) следующий порядок действий¹.

1. Определить системы в области действия уязвимости и относительный уровень риска для каждой из них.
2. По внутренним каналам связаться со всеми командами, которые могли быть затронуты.
3. Исправить все уязвимые системы как можно быстрее.
4. Сообщить внешним партнерам и клиентам о предстоящих действиях, в том числе о планах исправлений.

Мы не знали об этой проблеме до публичного раскрытия, поэтому рассматривали ее как уязвимость нулевого дня, требующую принятия мер в чрезвычайных ситуациях. В этом случае исправленная версия bash уже была доступна.

Команда оценила риск для разных систем и действовала соответственно.

- Мы посчитали, что огромное количество производственных серверов Google не подвергаются высокому риску. Их можно было легко исправить с помощью автоматического развертывания. После проверки и тестирования серверов мы исправили их гораздо быстрее, чем обычно.

¹ См. также видео на YouTube (<https://oreil.ly/boGtL>) об обсуждении этого события.

- Мы предположили, что множество рабочих станций Google подвержены более высоким рискам. К счастью, их можно было быстро исправить.
- Мы пришли к выводу, что несколько нестандартных серверов и унаследованная инфраструктура были в красной зоне риска и требовали ручного вмешательства. Мы отправили уведомления каждой команде с подробным описанием действий, которые они должны были предпринять. Это позволило нам распределить усилия по нескольким командам и легко отслеживать прогресс.

Параллельно команда разработала ПО для обнаружения уязвимых систем по периметру сети Google. Мы использовали его, чтобы провести оставшиеся необходимые исправления, и добавили эту функцию в стандартный мониторинг безопасности Google.

Проанализировав то, с чем мы хорошо (и не очень) справлялись во время этого реагирования, мы предлагаем ряд уроков для других команд и организаций.

- *Максимально стандартизируйте распространение программного обеспечения*, так чтобы было легко внести исправления при обнаружении неполадок. Это требует от владельцев сервисов понимания и принятия риска выбора нестандартного, неподдерживаемого распределения. Владелец сервиса должен нести ответственность за поддержку и исправление альтернативного варианта.
- *Используйте общедоступные стандарты распределения* — в идеале патч для развертывания уже будет в правильном формате. Так ваша команда может начать его проверку и тестирование быстрее, не перерабатывая его в соответствии с вашей конкретной средой.
- *Убедитесь, что вы можете ускорить используемый механизм для экстренного развертывания изменений (например, в случае возникновения уязвимости нулевого дня)*. Этот механизм должен обеспечивать более быструю проверку перед полным развертыванием в уязвимых системах. Мы не рекомендуем пропускать функциональную проверку вашей среды — сбалансируйте этот шаг с необходимостью защиты от уязвимости.
- *Убедитесь, что у вас есть средства для отслеживания хода развертывания, выявления неисправленных систем и определения уязвимости*. Наличие у вас необходимых инструментов может помочь при принятии решения о замедлении или ускорении в зависимости от текущего риска.
- *Подготовьте внешние сообщения как можно раньше при реагировании*. Вы же не хотите увязнуть во внутренних согласованиях с представителями PR-службы, когда СМИ потребует ответ.
- *Заранее разработайте план реагирования на происшествия или уязвимости для многократного использования* (см. главу 17), включая формулировки

для внешних коммуникаций. Если вы не уверены, что именно вам нужно, начните с анализа предыдущего происшествия.

- *Знайте, какие системы нестандартны или требуют особого внимания.* Отслеживая отклонения от стандарта, вы будете знать, для каких систем нужны предварительные уведомления и помощь в исправлении (если вы стандартизируете распространение ПО, используя наши рекомендации из первого пункта, отклонений не должно быть много).

Среднесрочные изменения: улучшение состояния безопасности

Команды, отвечающие за безопасность, часто вносят изменения для улучшения общего состояния безопасности среды и снижения рисков. Эти изменения обусловлены внутренними и внешними требованиями и сроками, и обычно их не нужно вводить внезапно.

Планируя изменения в области безопасности, выясните, какие группы и системы подвержены уязвимости, и определите, с чего лучше начинать. Следуя принципам SRE, изложенным в разделе «Проектирование ваших изменений» выше в этой главе, составьте план действий для постепенного развертывания. Каждый этап должен включать критерии успеха, которые должны выполняться перед переходом к следующему этапу.

Затронутые изменениями безопасности системы или команды не обязательно могут быть представлены в процентах от развертывания. Вместо этого вы можете выполнить поэтапное развертывание в зависимости от того, кто или что затронут и какие изменения следует внести.

Разверните команду изменений по группам с точки зрения того, на кого это влияет. Здесь группой может быть команда разработчиков, система или набор конечных пользователей. Скажем, вы могли бы начать с развертывания изменений в политиках устройств для пользователей, часто использующих систему, например, для отдела продаж. Благодаря этому можно быстро протестировать самые распространенные сценарии и получить реальную обратную связь. Если мы имеем дело с развертыванием по группам, существуют две конкурирующие стратегии.

- Начните с самого простого варианта использования, где вы получите наибольшую отдачу и докажете ценность своих действий.
- Начните с самого сложного варианта использования, который покрывает большинство ошибок и крайних случаев.

Если вам еще не хватает вовлеченности организации, начните с самого простого варианта. Если у вас есть поддержка руководства и ресурсы, лучше найти ошибки реализации и болевые точки на ранней стадии. Помимо организационных проблем, вы должны рассмотреть, какая стратегия эффективнее снизит риск в краткосрочной и в долгосрочной перспективе. Во всех случаях успешное подтверждение концепции помогает определить, как лучше двигаться вперед. Чтобы понять пользовательский опыт, вносящая изменения команда должна испытать все самостоятельно.

Вы можете внедрить само изменение постепенно. Например, можно поочередно выполнять все более строгие требования или сначала выставить изменение как необязательное. Там, где этого сделать нельзя, попробуйте рассмотреть возможность развертывания изменения как пробного прогона в режиме оповещения или аудита перед переключением в режим принудительного применения. Так пользователи могут почувствовать, как это их коснется, прежде чем изменение станет обязательным. Это поможет найти системы или пользователей, которые неправильно определены как вовлеченные в область действия, и пользователей или системы, для которых достижение соответствия будет особенно трудным.

Пример: строгая двухфакторная аутентификация с использованием ключей безопасности FIDO

Фишинг — серьезная проблема безопасности в Google. Хотя мы широко внедрили двухфакторную аутентификацию с использованием одноразовых паролей (one-time password, OTP), OTP по-прежнему подвержены перехвату как части фишинг-атаки. Мы предполагаем, что даже самые искушенные пользователи, хорошо подготовленные к борьбе с фишингом, все же подвержены захватам учетных записей. Это может происходить из-за запутанных пользовательских интерфейсов или человеческих ошибок. Для устранения этого риска начиная с 2011 года мы исследовали и протестировали несколько более надежных методов двухфакторной аутентификации (two-factor authentication, 2FA)¹. В конечном счете мы выбрали универсальные аппаратные токены безопасности (universal two-factor, U2F) из-за их свойств безопасности и удобства использования. Внедрение ключей безопасности для большой и распределенной по всему миру команды сотрудников Google требовало создания пользовательских интеграций и управления массовой регистрацией.

Оценка потенциальных решений и наш окончательный выбор входили в сам процесс изменений. Мы определили требования безопасности, конфиденциаль-

¹ См.: *Lang J. et al. Security Keys: Practical Cryptographic Second Factors for the Modern Web // Proceedings of the 2016 International Conference on Financial Cryptography and Data Security, 2016. P. 422–440. <https://oreil.ly/S2ZMU>.*

ности и удобства использования. После мы проверили потенциальные решения с пользователями, чтобы понять, что именно меняется, получить реальную обратную связь и измерить влияние изменений на повседневные рабочие процессы.

Чтобы решение было принято без проблем, помимо требований безопасности и конфиденциальности, оно должно соответствовать требованиям удобства использования. Это также очень важно для формирования культуры безопасности и надежности (см. главу 21). 2FA должна была быть легкой — быстрой и «бестолковой», чтобы было не просто использовать ее неправильно или небезопасно. Это требование было особенно критичным для SRE. В случае простоя 2FA не могла замедлить процессы реагирования. У внутренних разработчиков должна была быть возможность легко интегрировать решение 2FA в свои сайты с помощью простых API. Мы хотели получить эффективное решение для достижения идеального юзабилити. Чтобы оно масштабировалось для пользователей с несколькими учетными записями и не требовало дополнительного оборудования. Оно также должно было быть физически не требующим усилий, простым в изучении и легко восстанавливаемым в случае потери.

Оценив несколько вариантов, мы разработали ключи безопасности FIDO (<https://oreil.ly/UHbVu>). Хотя эти ключи и не отвечали всем нашим требованиям, в пилотных версиях они уменьшали общее время аутентификации и имели незначительную частоту ее неудачных попыток.

После окончательного решения мы развернули ключи безопасности для всех пользователей и отказались от поддержки OTP в Google. Это происходило в 2013 году. Для обеспечения широкого внедрения регистрация была с возможностью самообслуживания.

- Изначально многие пользователи выбирали ключи безопасности добровольно, потому что они были проще в использовании, чем существующие инструменты OTP. С ключами не требовалось набирать код со своего телефона или использовать физическое устройство OTP. Пользователям давали «нано» ключи безопасности, которые могли оставаться на USB-накопителях их ноутбуков.
- Чтобы получить ключ безопасности, пользователь мог перейти в любое место TechStop в любом офисе¹ (было сложно распространять устройства в глобальных офисах, ведь юристы должны были соблюдать требования по экспорту и импорту).
- Пользователи регистрировали свои ключи безопасности через сайт само-регистрации. TechStop предоставил помощь самым первым пользователям

¹ TechStops — это сервисы технической поддержки Google, описанные в блоге (<https://oreil.ly/BWn0->) Хесуса Луго (Jesus Lugo) и Лизы Мойк (Lisa Mauck).

и людям, нуждающимся в дополнительной поддержке. При первой аутентификации пользователи должны были применить существующую систему ОТР, поэтому ключи становились доверенными при первом использовании (TOFU).

- Пользователи могли зарегистрировать несколько ключей безопасности, чтобы не беспокоиться о потере единственного ключа. Этот подход обошелся дороже, но был строго согласован с нашей целью не заставлять пользователя носить дополнительное оборудование.

Во время развертывания команда столкнулась с некоторыми проблемами, например, с устаревшими прошивками. По возможности мы подходили к этим вопросам самостоятельно, в частности, разрешая пользователям самим обновлять микропрограмму ключа безопасности.

Но сделать ключи безопасности доступными для пользователей — только половина проблемы. Системы, использующие ОТР, тоже должны были перейти на ключи безопасности. В 2013 году многие приложения изначально не поддерживали эту новую технологию. Сначала команда сосредоточилась на поддержке приложений, ежедневно используемых сотрудниками Google, таких как инструменты внутреннего анализа кода и информационные панели. Там, где ключи безопасности не поддерживались (например, в случае управления сертификатами некоторых аппаратных устройств и сторонних веб-приложений), Компания Google работала напрямую с поставщиком для запроса и добавления поддержки. После мы столкнулись с длинным списком приложений. Поскольку все ОТР были сгенерированы централизованно, мы могли выяснить, какое приложение нужно использовать, отслеживая клиентов, выполняющих ОТР-запросы.

В 2015 году команда сосредоточилась на завершении развертывания и отказе от сервиса ОТР. Мы отправляли пользователям напоминания, когда они использовали ОТР вместо ключа безопасности, и в итоге заблокировали доступ через ОТР. Хотя мы имели дело практически со всем длинным списком приложений, запрашивающих ОТР, было еще несколько исключений, таких как настройка мобильного устройства. Тогда был создан веб-генератор ОТР для исключительных обстоятельств. Пользователи должны были подтвердить свою личность с помощью ключа безопасности — разумный режим отказа с немного большей временной нагрузкой. Мы успешно завершили внедрение ключей безопасности в масштабе всей компании в 2015 году.

Этот опыт позволил извлечь несколько полезных уроков, касающихся формирования культуры безопасности и надежности (см. главу 21).

Убедитесь, что выбранное вами решение подходит для всех пользователей

Очень важно, чтобы решение 2FA было доступно в том числе и пользователям с нарушениями зрения.

Сделайте изменения легкими для изучения и максимально простыми

Такое особенно применимо, если решение для пользователя удобнее, чем исходный вариант! Это значимо для действий или изменений, которые, как вы ожидаете, пользователь будет выполнять часто. В этом случае даже небольшая заминка может привести к значительным нагрузкам на пользователя.

Сделайте изменения самообслуживающимися, чтобы минимизировать нагрузку на центральную ИТ-команду

Для большого изменения, затрагивающего повседневную деятельность всех пользователей, важно, чтобы они могли легко регистрировать, удалять и устранять неполадки.

Дайте пользователю реальное доказательство того, что решение работает и отвечает его интересам

Четко объясните влияние изменений с точки зрения безопасности и снижения рисков и предоставьте пользователям возможность высказать свое мнение.

Как можно быстрее добавьте возможность обратной связи о несоблюдении правил

Отзыв может быть передан в виде ошибки аутентификации, системной ошибки или напоминания по электронной почте. Уведомление пользователя о том, что его действие не соответствовало требуемой политике, в течение нескольких минут или часов позволяет ему принять меры для устранения проблемы.

Отслеживайте прогресс и определяйте, как решить обширную проблему

Изучая запросы пользователей на ОТР для каждого приложения, мы можем определить, на каких приложениях нужно сосредоточиться. Используйте инструментальные панели для отслеживания прогресса и определения того, могут ли альтернативные решения с похожими свойствами безопасности работать в различных сценариях.

Долгосрочные изменения: внешнее требование

В некоторых ситуациях вам понадобится гораздо больше времени для развертывания изменения. К примеру, внутреннее изменение может потребовать значительных перестроек в архитектуре или системе либо более обширных модификаций отрасли в целом. Эти изменения могут быть обусловлены или ограничены внешними сроками или требованиями, и для их реализации может потребоваться несколько лет.

Приступая к реализации долгосрочного — на несколько лет — решения, вы должны четко определить и оценить прогресс в достижении вашей цели. Документация важна как для учета необходимых факторов, влияющих на процесс разработки (см. раздел «Проектирование ваших изменений» выше в этой главе), так и для обеспечения непрерывности. Люди, работающие над изменениями сегодня, могут покинуть компанию, и им придется сдать свою работу. Чтобы обеспечить постоянную поддержку руководства, важно сохранять документацию в актуальном состоянии с последним планом и статусом.

Установите соответствующие инструменты и средства отображения информации для измерения текущего прогресса. В идеале можно автоматически контролировать изменения, устраняя необходимость участия человека в цикле, если использовать проверку конфигурации или тестирование. Так же, как вы стремитесь к значительному тестовому покрытию для кода в вашей инфраструктуре, вы должны стремиться к покрытию для проверки соответствия затронутых изменением систем. Для эффективного масштабирования такого охвата эти средства должны быть самообслуживаемыми. Это позволит командам внедрять изменения и сами инструменты. Прозрачное отслеживание этих результатов помогает мотивировать пользователей и упрощает коммуникации и внутреннюю отчетность. Вместо того, чтобы дублировать работу, также стоит использовать этот единственный источник истины для отчетности руководству.

При проведении любых масштабных, долгосрочных изменений в организации не всегда удастся сохранять на одном уровне поддержку со стороны руководства. Чтобы со временем динамика не пропала, нужно мотивировать людей, создающих эти изменения. Установление конечных целей, отслеживание прогресса и демонстрация значимых результатов могут помочь командам прийти к финишу. Реализация всегда будет затягиваться, поэтому разработайте стратегию, подходящую для вашей ситуации. Если изменение не нужно (по правилам или по другим причинам), а достижение принятия на 80 или 90 % может ощутимо снизить риск безопасности, оно должно рассматриваться как благоприятное.

Пример: увеличение использования HTTPS

В последнее десятилетие использование HTTPS в сети резко возросло благодаря совместным усилиям команды Google Chrome, Let's Encrypt (<https://letsencrypt.org/>) и других организаций. HTTPS обеспечивает важные гарантии конфиденциальности и целостности для пользователей и веб-сайтов. Интерфейс имеет решающее значение для успеха веб-экосистемы — теперь он требуется как часть HTTP/2.

Чтобы продвигать использование HTTPS в Интернете, мы провели обширные исследования для разработки стратегии, по разным каналам связались с владельцами сайтов и привели для них серьезные доводы в пользу перехода. Долгосрочные, общесистемные изменения требуют продуманной стратегии и значительного планирования. Мы использовали подход, основанный на данных, для определения наилучшего способа охвата каждой группы заинтересованных сторон.

- Мы собрали данные о текущем использовании HTTPS во всем мире для выбора целевых областей.
- Мы опросили конечных пользователей, чтобы понять, как они воспринимают интерфейс HTTPS в браузерах.
- Мы измерили поведение сайта с целью определения функций веб-платформы, которые могут быть ограничены HTTPS для защиты конфиденциальности пользователей.
- Мы использовали тематические исследования, чтобы понять озабоченность разработчиков по поводу HTTPS и типов инструментов, которые мы могли бы создать для них.

Это не было разовым подходом: мы продолжали следить за показателями и собирать данные в течение многих лет, корректируя стратегию при необходимости. Например, так как мы постепенно выдавали предупреждения в Chrome для небезопасных страниц в течение нескольких лет, мы следили за показателями поведения пользователей, чтобы убедиться, что изменения интерфейса не вызвали неожиданных негативных эффектов.

Постоянное общение было ключом к успеху. Перед каждым изменением мы использовали все доступные каналы связи: блоги, списки рассылки для разработчиков, прессу, справочные форумы Chrome и партнерские встречи. Такой подход позволил максимально расширить охват, поэтому владельцы сайтов не были удивлены тем, что их подталкивали к переходу на HTTPS. Мы также индивидуально настроили наш охват на региональном уровне, уделив особое внимание Японии, когда поняли, что внедрение HTTPS отстает там из-за медленного распространения среди ведущих сайтов.

В итоге мы сосредоточились на создании и усилении стимулов, чтобы предоставить вескую причину для перехода на HTTPS. Даже разработчики, отвечающие за безопасность, не могли убедить свои организации без экономического обоснования перехода. К примеру, благодаря HTTPS веб-сайты могут использовать функции веб-платформы, такие как Service Worker (<https://oreil.ly/W5t4I>), фоновый скрипт, обеспечивающий автономный доступ, push-уведомления

и периодическую фоновую синхронизацию. Эти функции улучшают производительность и доступность и могут непосредственно влиять на бизнес-результаты. Организации увереннее тратились на переход на HTTPS, если чувствовали, что этот шаг соответствует их деловым интересам.

Как показано на рис. 7.1, пользователи Chrome на разных платформах теперь проводят более 90 % своего времени на HTTPS-сайтах. Ранее же этот показатель составлял всего 70 % для пользователей Chrome в Windows и 37 % для Chrome в Android. Такому росту способствовали бесчисленные сотрудники многих организаций — поставщиков браузеров, центров сертификации и веб-издателей. Эти организации координировали свою деятельность через органы стандартизации, научно-исследовательские конференции и путем открытого общения о проблемах и успехах, с которыми сталкивался каждый. Опыт Chrome в этом сдвиге позволил извлечь важные уроки о том, как внести вклад в изменения в рамках всей экосистемы.

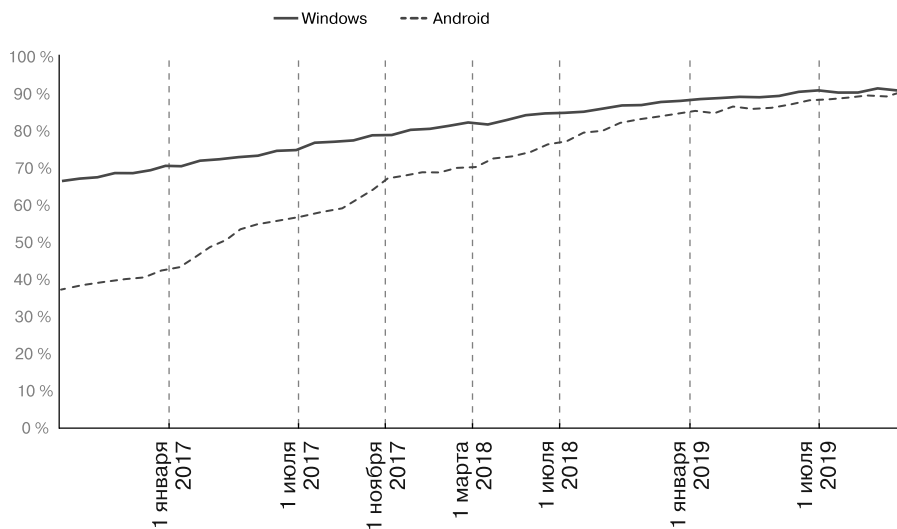


Рис. 7.1. Просмотры HTTPS-сайтов в Chrome в зависимости от платформы (в процентах)

Понять экосистему, прежде чем принимать стратегию

Мы основали нашу стратегию на количественных и качественных исследованиях, ориентируясь на широкий круг заинтересованных сторон, включая веб-разработчиков и конечных пользователей в разных странах и на разных устройствах.

Постоянно общаться, чтобы максимизировать охват

Мы использовали разные каналы связи для охвата самого широкого круга заинтересованных сторон.

Связывать изменения безопасности с бизнес-целями

Руководители были более лояльны к переходу на HTTPS, когда могли видеть явные выгоды для бизнеса.

Создайте отраслевой тренд

Несколько поставщиков браузеров и организаций одновременно поддерживали переход Интернета на HTTPS. Разработчики рассматривали HTTPS как отраслевой тренд.

Сложности: когда планы меняются

Лучшие планы в области безопасности и SRE часто ошибочны. Есть много причин, по которым вам может понадобиться либо ускорить изменение, либо замедлить его.

Часто нужно ускорить изменение из-за внешних факторов — из-за уязвимости, которая активно эксплуатируется. В этом случае следует ускорить развертывание, чтобы быстрее исправить системы. Будьте осторожны: ускорение не всегда лучший выбор для безопасности и надежности ваших систем. Подумайте, можете ли вы изменить порядок развертывания, чтобы быстрее охватить конкретные системы с более высоким риском. Возможно, подойдет другой способ, чтобы исключить доступ злоумышленников. Это можно сделать с помощью операций ограничения скорости или перевода особо важной системы в автономный режим.

Вы можете решить замедлить изменения. Такой подход обычно актуален в случае проблем с исправлением, таких как неожиданно высокая частота ошибок или другие ошибки развертывания. Если замедление изменений не решает проблему или вы не можете развернуть их полностью без негативного влияния на систему или пользователей, то откат, отладка проблемы и повторное развертывание — болезненный, но осторожный сценарий. Еще вы можете замедлить изменения, основываясь на обновленных бизнес-требованиях, например, изменениях внутренних сроков или задержек в появлении отраслевых стандартов (что вы имеете в виду, разве TLS 1.0 все еще используется?!).

В лучшем случае ваши планы поменяются до их реализации. В ответ вам просто нужно создавать новые планы! Вот некоторые возможные причины для изменения планов и подходящие для этого тактики.

Возможно, вам придется отложить изменение из-за внешних факторов

Если вы не можете начать исправление сразу после отмены эмбарго (см. предыдущий раздел «Разные изменения: разные скорости, разные сроки»), поработайте с командой по уязвимости, чтобы выяснить, есть ли в вашей ситуации какие-либо другие системы и возможно ли изменить график. В любом случае убедитесь, что у вас есть средства связи, с помощью которых можно проинформировать затронутых пользователей о новых планах.

Возможно, вам придется ускорить изменение, публично объявив о нем

Для уязвимости в условиях эмбарго вам придется подождать публичного объявления, прежде чем вносить исправления. Сроки могут измениться, если объявление выйдет раньше положенного, если взлом станет общеизвестным или уязвимость будет открыто использоваться. В этом случае может понадобиться начать исправление раньше. У вас должен быть план действий на каждом этапе в случае нарушения эмбарго.

Происшествие не сильно затрагивает вашу систему

Если уязвимость или изменение в основном затрагивают общедоступные веб-сервисы, а их количество в вашей организации ограничено, не спешите исправлять всю инфраструктуру. Исправьте затронутые части и замедлите скорость, с которой вы применяете исправления к другим частям системы.

Вы можете зависеть от внешних сторон

Большинство организаций зависят от третьих сторон в распространении исправленных пакетов и образов для уязвимостей или полагаются на доставку программного и аппаратного обеспечения в рамках изменения инфраструктуры. Если исправленная ОС недоступна или поставка необходимого оборудования откладывается, вы мало что можете сделать. Возможно, придется вносить изменения позже, чем предполагалось изначально.

Пример: растущая область действия — Heartbleed

В декабре 2011 года в OpenSSL была добавлена поддержка функции контрольных сигналов SSL/TLS вместе с нераспознанной ошибкой (<https://xkcd.com/1354>), позволяющей серверу или клиенту получить доступ к 64 Кбайт личной памяти другого сервера или клиента. В апреле 2014 года эту ошибку обнаружили сотрудник Google, Нил Мехта, и инженер, работающий в Codenomicon (компания по кибербезопасности). Оба сообщили об этом в проект OpenSSL. Команда

OpenSSL добавила изменение в коде для исправления ошибки и разработала план ее публичного раскрытия. Codenomicon сделала всеобщее объявление, что удивило многих в сообществе безопасности, и запустила пояснительный сайт heartbleed.com. Это первое использование говорящего названия¹ и логотипа вызвало неожиданно большой интерес в СМИ.

Команды инфраструктуры Google имели ранний доступ к исправлению, находившемуся под эмбарго, и перед запланированным публичным раскрытием уже незаметно исправили несколько ключевых внешних систем, непосредственно обрабатывающих трафик TLS. Но другие внутренние команды не знали об этой проблеме.

Как только ошибка стала общеизвестной, средства ее обработки быстро проектировались в таких средах, как Metasploit (<https://www.metasploit.com/>). Теперь, в условиях ускорения сроков, многим командам в Google нужно было спешно исправлять свои системы. Их служба безопасности воспользовалась автоматическим сканированием для обнаружения дополнительных уязвимых систем и высылала затронутым командам инструкции по исправлению и отслеживанию прогресса. Раскрытие памяти значило утечку закрытых ключей. Это означало, что в ряде сервисов нужно менять ключи. Команда безопасности уведомляла затронутые команды и отслеживала их прогресс в централизованной электронной таблице.

История с Heartbleed иллюстрирует ряд важных уроков.

Спланируйте худший сценарий нарушения эмбарго или его отмены на ранней стадии

Хотя в идеале рассматривается ответственное раскрытие информации, может случиться непредвиденное (а симпатичные логотипы могут привлечь интерес СМИ). Выполните как можно больше предварительной работы и быстро приступайте к исправлению самых уязвимых систем независимо от соглашений о раскрытии (которые требуют внутренней секретности в отношении информации, находящейся под эмбарго). Если вы сможете получить исправление заранее, то сможете и развернуть его до публичного объявления. Если это невозможно, все равно стоит по максимуму протестировать и проверить исправление, чтобы обеспечить плавный процесс развертывания.

Подготовьтесь к быстрому и масштабному развертыванию

Используйте непрерывные сборки для гарантии того, что вы можете перекомпилировать код в любое время, применяя канареечное тестирование для проверки без дополнительных разрушений.

¹ Heart bleed — дословно «сердце кровью обливается». — *Примеч. пер.*

Регулярно заменяйте ключи шифрования и другие секреты

Замена ключа (<https://oreil.ly/TFb4b>) — лучшая практика для ограничения потенциальной области действия взлома при компрометации ключа. Регулярно выполняйте эту операцию и убедитесь, что ваши системы по-прежнему работают так, как задумано (подробности см. в главе 9 книги Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/incident-response/>)). Это дает гарантию того, что замена любых скомпрометированных ключей не будет стоить титанических усилий.

Убедитесь, что у вас есть канал связи с конечными пользователями — как внутренний, так и внешний

Если внести изменение не удастся или оно вызывает непредвиденное поведение, вам необходимо быстро предоставлять обновления.

Резюме

Важно понимать, в чем различие между разными видами изменений в системе безопасности. Нужно, чтобы представители затронутых отделов знали, что от них ожидается и какую поддержку вы можете предложить.

В следующий раз, когда вам будет поручено внести изменения в безопасность инфраструктуры, сделайте глубокий вдох и составьте план. Начните с малого или найдите добровольцев, желающих проверить изменения. Вам нужна обратная связь для понимания того, с чем сталкиваются пользователи. Если планы меняются, не паникуйте и не удивляйтесь.

Такие стратегии проектирования, как частые развертывания, контейнеризация и использование микросервисов, облегчают как упреждающие улучшения, так и меры по смягчению последствий в чрезвычайных ситуациях. Продуманный проект и обновляемая документация, ориентированные на изменения, сохраняют систему работоспособной, делают нагрузку вашей команды более контролируемой и — как вы узнаете в следующей главе — повышают устойчивость системы.

Проектирование для устойчивости

*Виталий Шипицын, Митч Адлер,
Золтан Егид и Пол Бланкиншип
с Иисусом Климентом, Джесси Янг,
Дугласом Колишем и Кристофом Керном*

Хороший системный проект включает планирование устойчивости: способность защищать от атак и противостоять необычным обстоятельствам, нагружающим вашу систему и влияющим на ее надежность.

На ранней стадии разработки вы должны подумать о полной или частичной поддержке системы в работе при столкновении с несколькими инцидентами одновременно.

Мы начнем эту главу с истории из древнего мира, где защита в глубину могла бы спасти империю. После мы поговорим о современных стратегиях защиты в глубину на примере Google App Engine.

Описанные в этой главе решения влекут разные затраты на внедрение и различаются в зависимости от размера организации. Если у вас небольшая организация, мы предлагаем сосредоточиться на контролируемой деградации, контроле радиуса взлома и разбиения систем на отдельные области отказов. По мере роста вашей организации мы предлагаем пользоваться постоянной проверкой для подтверждения и повышения устойчивости системы.

«Устойчивость» — характеристика системы, описывающая ее способность противостоять серьезной неисправности или сбоям. Отказоустойчивые системы могут автоматически восстанавливаться после сбоев в некоторых частях — или при сбое всей системы — и нормально работать после устранения проблем. Сервисы в отказоустойчивой системе в идеале должны работать в течение всего происшествия, возможно, в ухудшенном режиме. Рассмотрение устойчивости на каждом уровне проектирования системы помогает защитить ее от непредвиденных сбоев и сценариев атак.

Есть разница между проектированием системы для обеспечения устойчивости и для восстановления (подробно рассматривается в главе 9). Устойчивость тесно

связана с восстановлением. Но если восстановление сосредоточено на способности исправлять системы *после* их поломки, то устойчивость — это разработка систем, *задерживающих* или *выдерживающих* поломку. Системы, разработанные с упором и на устойчивость, и на восстановление, лучше способны восстанавливаться после сбоев и почти не нуждаются в человеческом вмешательстве.

Принципы проектирования для устойчивости

Характеристики устойчивости системы основаны на принципах проектирования, рассмотренных в части II. Чтобы оценить устойчивость системы, вы должны хорошо понимать, как она спроектирована и построена. Вам следует тщательно согласовывать другие качества проектирования, описанные в этой книге, — наименьшие привилегии, понятность, адаптивность и восстановление — для усиления стабильности и устойчивости вашей системы.

В этой главе будут рассмотрены следующие подходы, характеризующие отказоустойчивую систему.

- Проектируем каждый уровень в системе как независимый. Этот подход обеспечивает глубокую защиту каждого уровня.
- Расставляем приоритеты для каждой функции и рассчитываем ее стоимость, чтобы отделить важные функции, требующие их поддержания независимо от нагрузки, которую испытывает система, от функций менее важных, которые можно регулировать или отключать, если возникнут проблемы или ресурсы будут ограничены. После можно определить, где могут быть эффективно применены ограниченные ресурсы системы и как максимизировать возможности ее обслуживания.
- Разделяем систему по четко определенным границам для обеспечения независимости изолированных функциональных частей. Так будет легче создавать дополнительное защитное поведение.
- Используем резервирование разделов для защиты от локальных сбоев. Для глобальных сбоев нужны разделы, обеспечивающие разные свойства надежности и безопасности.
- Сокращаем время реакции системы, автоматизировав как можно больше мер по обеспечению устойчивости. Работаем над обнаружением новых режимов отказов, которым было бы на пользу создание новой автоматизации или улучшение существующей.
- Поддерживаем эффективность системы, проверяя ее свойства устойчивости — автоматические ответы и любые другие атрибуты устойчивости.

Защита в глубину

Защита в глубину подразумевает несколько уровней внешних границ защиты. В результате злоумышленники имеют ограниченную видимость системы, и им становится труднее взломать ее.

Троянский конь

История троянского коня, рассказанная Вергилием в «Энеиде», предостерегает от опасностей неадекватной защиты. После десяти бесплодных лет осады города Трои греческая армия строит большого деревянного коня, которого преподносит троянцам в дар. Конь минует стены Трои, и люди, прячущиеся в нем, выскакивают наружу. Они подрывают оборону города изнутри, после чего открывают городские ворота для всей греческой армии, которая разрушает город.

Представьте, как закончилась бы эта история, если бы город провел защиту в глубину. Для начала, воины Трои могли бы осмотреть подарок внимательнее и обнаружить обман. Если бы злоумышленникам все-таки удалось проникнуть через городские ворота, они могли бы столкнуться со вторым уровнем защиты. Например, конь мог быть закрыт в безопасном внутреннем дворе без доступа к остальной части города.

Что говорит нам история 3000-летней давности о безопасности в масштабе или даже о безопасности в целом? Во-первых, если вы пытаетесь понять стратегии, необходимые для защиты и поддержки системы, сначала нужно понять саму атаку. Если мы рассмотрим город Трои как систему, то можем проанализировать шаги атакующих (этапы атаки) для выявления слабых мест, которые защита в глубину может устранить.

На высоком уровне мы можем разделить троянскую атаку на четыре этапа.

1. *Моделирование угроз и обнаружение уязвимостей* — оцените цель и конкретно определите ее слабые места и недостатки. Нападавшие не могли открыть городские ворота снаружи, но могли ли они открыть их изнутри?
2. *Развертывание* — определите условия для атаки. Нападавшие создали и доставили объект, который троянцы в итоге пропустили через городские стены.
3. *Выполнение* — проведите фактическую атаку, пользуясь предыдущими этапами. Воины вышли из троянского коня и открыли городские ворота, чтобы впустить греческую армию.
4. *Компромисс* — после успешного выполнения атаки происходит повреждение и начинается смягчение.

У троянцев были возможности сорвать атаку на каждом этапе до компромисса, и они дорого заплатили за то, что упустили их. Аналогично защита в глубину вашей системы может снизить ущерб, который вы можете понести при взломе системы.

Моделирование угроз и обнаружение уязвимостей

Как преступники, так и защитники могут оценить цель на наличие слабых мест. Злоумышленники проводят разведку целей, находят слабые места, а затем моделируют атаки. Защитники должны сделать все возможное для ограничения информации, раскрываемой злоумышленниками во время разведки. Но защитники не могут полностью предотвратить эту разведку. Поэтому они должны обнаружить ее и использовать в качестве сигнала. В случае с троянским конем защитники могли бы насторожиться при вопросах незнакомцев о защите ворот. И после проявили бы особую осторожность, обнаружив у городских ворот большого деревянного коня.

Принятие к сведению этих запросов незнакомцев равносильно сбору информации об угрозах. Есть много способов сделать это для ваших систем, и вы даже можете выбрать некоторые из них. Например.

- Мониторинг системы на предмет сканирования портов и приложений.
- Отслеживание DNS-регистрации URL-адресов, похожих на ваши, — злоумышленник может использовать их для фишинговых атак.
- Покупка данных разведки угроз.
- Создание команды разведки угроз для изучения и пассивного мониторинга действий известных и вероятных угроз для вашей инфраструктуры. Мы не рекомендуем небольшим компаниям инвестировать в такой подход, но по мере роста организации он может стать экономически эффективным.

Защищая систему, вы знаете ее изнутри. Поэтому ваша оценка может быть более точной, чем разведка злоумышленника. Это критический момент: понимая слабые стороны вашей системы, вы можете эффективнее их защищать. И чем лучше вы понимаете методы злоумышленников, тем сильнее будет этот эффект. Остерегайтесь появления слепых зон — не недооценивайте возможность атаки тех сторон системы, которые, по вашему мнению, не имеют особого значения.

Развертывание атаки

Если вы знаете, что злоумышленники проводят разведку системы, очень важно приложить максимум усилий для обнаружения и прекращения атаки. Представьте, что троянцы решили не вводить деревянного коня в городские ворота, потому что он был создан кем-то, кому они не доверяли. Вместо этого они могли бы тщательно осмотреть троянского коня, прежде чем пропустить его внутрь, или просто подожгли бы его.

В наше время вы можете выявить потенциальные атаки, пользуясь проверкой сетевого трафика, обнаружением вирусов, контролем выполнения ПО, защищенными песочницами¹ и надлежащим предоставлением привилегий для подачи сигналов об аномальном использовании.

Выполнение атаки

Когда вы не можете предотвратить все развертывания противников, ограничьте радиус потенциальных атак. Если бы защитники окружили троянского коня, нападавшим было бы сложнее выбраться из укрытия незамеченными. В кибервойнах эта тактика (более подробно описанная в подразделе «Уровни среды выполнения» далее в этой главе) называется «песочницей».

Компромисс

Когда троянцы проснулись и увидели врагов, стоящих над их кроватями, они поняли, что город подвергся нападению. Это понимание пришло как раз после фактического компромисса. Многие банки столкнулись с похожей ситуацией в 2018 году после взлома их инфраструктуры EternalBlue (<https://oreil.ly/wNI2u>) и WannaCry (<https://oreil.ly/irovS>).

Ваша реакция на происшествие определяет, как долго ваша инфраструктура останется под угрозой.

Анализ Google App Engine

Рассмотрим защиту в глубину в современных условиях: Google App Engine. Google App Engine позволяет пользователям размещать код приложения и масштабировать его по мере увеличения нагрузки без управления сетями,

¹ Защищенные песочницы обеспечивают изолированную среду для ненадежного кода и данных.

компьютерами и операционными системами. На рис. 8.1 показана упрощенная схема архитектуры App Engine в его первые годы. За безопасность кода приложения отвечает разработчик, а за защиту среды выполнения Python/Java и базовой ОС отвечает Google.

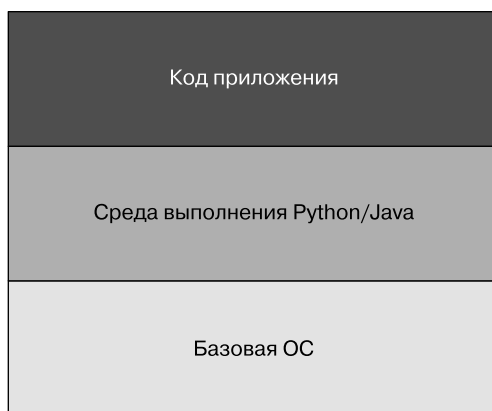


Рис. 8.1. Упрощенное представление архитектуры Google App Engine

Изначально для реализации Google App Engine были нужны особые методы изоляции процессов. В то время Google применял традиционную изоляцию пользователей POSIX как стратегию по умолчанию (посредством отдельных пользовательских процессов). Но мы решили, что запуск кода каждого пользователя на независимой виртуальной машине был слишком неэффективным для запланированного уровня принятия. Нам нужно было понять, как запускать сторонний ненадежный код так же, как и любые другие задачи в инфраструктуре Google.

Рискованные API

Первоначальное моделирование угроз для App Engine выявило несколько проблемных областей.

- Доступ к сети был проблематичным. До этого момента все приложения, работающие в производственной сети Google, считались доверенными и аутентифицированными компонентами инфраструктуры. Из-за того, что мы внедряли в эту среду произвольный, ненадежный сторонний код, нам была нужна стратегия для изоляции внутренних API и доступа к сети от App Engine. Еще нужно было помнить, что сам App Engine работал в той же инфраструктуре и поэтому зависел от доступа к тем же API.

- На компьютерах с пользовательским кодом нужен доступ к локальной файловой системе. По крайней мере, он был ограничен каталогами, принадлежащими пользователю. Это помогло защитить среду выполнения и снизить риск вмешательства предоставленных пользователем приложений в приложения других пользователей на том же устройстве.
- Использование ядра Linux означало, что App Engine подвергся большой атаке, которую нужно было минимизировать. Например, мы хотели предотвратить как можно больше случаев локального повышения привилегий.

Для решения этих проблем мы сначала рассмотрели ограничение доступа пользователей к каждому API. Наша команда избавилась от встроенных API операций ввода/вывода для взаимодействия по сети и файловой системе во время выполнения. Мы заменили встроенные API на «безопасные» версии, обращенные к другой облачной инфраструктуре, а не напрямую управляющие средой выполнения.

Для запрета добавления преднамеренно удаленных возможностей мы не разрешали пользователям применять предоставленный ими скомпилированный байт-код или общие библиотеки. Пользователи должны были зависеть от предоставленных нами методов и библиотек, в дополнение к множеству разрешенных только для среды выполнения реализаций с открытым исходным кодом.

Уровни среды выполнения

Мы также провели тщательный аудит базовых реализаций объектов данных среды выполнения для функций, которые могли привести к ошибкам повреждения памяти. В результате мы добавили несколько исправлений ошибок в каждой из запущенных сред.

Мы предполагали, что некоторые из этих защитных мер будут неуспешны, ведь мы вряд ли сможем найти и предсказать все используемые состояния в выбранных средах выполнения. Мы решили специально адаптировать среду выполнения Python для компиляции в битовый код Native Client (NaCL). Благодаря NaCL мы смогли предотвратить многие случаи повреждения памяти и атак потока управления, упущенных при углубленном аудите и усилении кода.

Нас не очень устраивало то, что NaCL будет содержать все рискованные ошибки кода и неполадки. Поэтому мы добавили второй уровень песочницы `ptrace` для фильтрации и оповещения о непредвиденных системных вызовах и параметрах. В любой непредвиденной ситуации выполнение сразу прекращалось, посылалось высокоприоритетное оповещение вместе с соответствующими записями активности.

За следующие пять лет команда обнаружила несколько случаев аномальной активности из-за эксплуатации в одном из режимов. В каждом случае уровень с песочницей давал нам преимущество перед злоумышленниками (позже мы подтвердили, что это были исследователи безопасности), а наши несколько уровней песочницы содержали их действия и используемые параметры.

Функционально реализация Python в App Engine включает уровни песочницы, показанные на рис. 8.2.

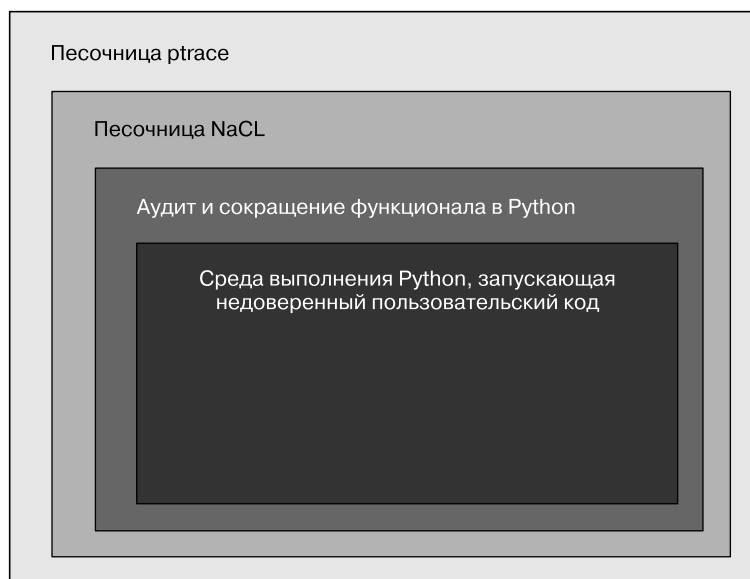


Рис. 8.2. Уровни песочницы реализации Python в App Engine

Уровни App Engine дополняют друг друга: каждый новый предвосхищает слабые места или вероятные сбои предыдущего. Когда активация защиты проходит сквозь уровни, сигналы о компрометации становятся сильнее. Это позволяет нам сосредоточиться на возможных атаках.

Мы применили тщательный и многоуровневый подход к безопасности для Google App Engine, но внешняя помощь в защите среды¹ нам пригодилась. В дополнение к нашей команде, обнаружившей аномальную активность, сторонние исследователи выявили несколько попыток взлома по другим направлениям. Мы благодарны исследователям, которые нашли и обнародовали пробелы.

¹ В Google есть программы вознаграждения за найденные ошибки (<https://oreil.ly/ZQGNW>).

Управление деградацией

Проектируя для защиты в глубину, мы предполагаем, что системные компоненты или даже целые системы могут выйти из строя. У сбоев может быть много причин: физический ущерб, сбой оборудования или сети, неправильная настройка ПО, ошибка или компрометация безопасности. В случае сбоя компонента воздействие может распространиться на все системы, зависящие от него. Общий пул аналогичных ресурсов тоже уменьшается. Например, сбой дисков уменьшают общую емкость хранилища, сбой сети уменьшают пропускную способность и увеличивают задержку, а сбой ПО уменьшают вычислительную емкость в масштабе всей системы. Сбои могут усугубляться — например, из-за нехватки памяти может произойти сбой ПО.

Такие нехватки ресурсов или внезапный всплеск входящих запросов, например, вызванный эффектом Slashdot (<https://oreil.ly/Z1UL8>), неправильной конфигурацией или отказом в обслуживании, могут привести к перегрузке системы. Когда нагрузка больше ее емкости, отклик неизбежно начинает ухудшаться. Это может привести к полной поломке и недоступности системы. Если такой сценарий не был запланирован, вы не будете знать, где система может сломаться, но это, скорее всего, будет не самое безопасное ее место.

Для контроля деградации нужно выбрать системные свойства, которые вы будете отключать или расширять при возникновении трудных обстоятельств, при этом делая все возможное для защиты безопасности системы. Если вы *намеренно* проектируете несколько вариантов ответа для таких обстоятельств, система может использовать настраиваемые контрольные точки, а не ломаться произвольно. Вместо того чтобы страдать от каскадных сбоев и справляться с последующим хаосом, система может реагировать, *постепенно деградируя*. Вот несколько способов сделать так.

- Освободите ресурсы и уменьшите частоту неудачных операций, отключив редко используемые, наименее важные функции или затратную функциональность. После вы можете применить освобожденные ресурсы для сохранения важных функций и особенностей. К примеру, большинство систем, принимающих соединения TLS, поддерживают криптосистемы Elliptic Curve (ECC) и RSA. В зависимости от реализации вашей системы одна из них будет дешевле, обеспечивая при этом сопоставимую безопасность. Программное обеспечение ECC — менее ресурсоемкое для операций с закрытым ключом¹. Отключение поддержки RSA, если система ограничена

¹ См.: Singh S. R., Khan A. K., Singh S. Performance Evaluation of RSA and Elliptic Curve Cryptography // Proceedings of the 2nd International Conference on Contemporary Computing and Informatics, 2016. P. 302–306. doi: 10.1109/IC3I.2016.7917979.

в ресурсах, освободит место для большего количества соединений при более низкой стоимости ЕСС.

- Старайтесь, чтобы системные меры реагирования вступили в силу быстро и автоматически. Это проще всего сделать при наличии серверов, находящихся под вашим прямым контролем, где вы можете произвольно переключать рабочие параметры любого масштаба или степени детализации. Клиентами пользователей сложнее управлять. У них долгие циклы развертывания, потому что клиентские устройства могут откладывать или не получать обновления. Кроме того, разнообразие клиентских платформ увеличивает вероятность отката мер реагирования из-за непредвиденных несовместимостей.
- Определите, какие системы имеют решающее значение для миссии вашей компании, их относительную важность и взаимозависимости. Возможно, вам придется сохранить минимальные характеристики этих систем пропорционально их относительной стоимости. Например, у Gmail есть «простой режим HTML». Он отключает симпатичный стиль пользовательского интерфейса и автозаполнение поиска, но позволяет пользователям продолжать открывать почтовые сообщения. Сбои сети, ограничивающие пропускную способность в регионе, могут лишить приоритета даже этот режим, если это позволит мониторингу безопасности сети продолжать защищать пользовательские данные.

Если эти изменения помогают системе воспринимать нагрузку или отказ, они обеспечивают критическое дополнение ко всем другим механизмам обеспечения устойчивости и дают респондентам больше времени для реагирования. Делайте важные и трудные выборы заранее, а не под давлением во время происшествия. Как только в отдельных системах будут разработаны четкие стратегии деградации, станет легче расставить приоритеты деградации в более широком масштабе, в разных системах или областях продукта.

КОМПРОМИСС БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: УПРАВЛЕНИЕ ДЕГРАДАЦИЕЙ

Не забывайте учитывать роль безопасности, оценивая критичность вашего сервиса. Нужно определить приемлемую степень повышенного риска. Например, допустимо ли отключение 2FA (<https://oreil.ly/aIKg0>) во время входа в систему? Когда 2FA была новой технологией, а пользователи еще только знакомились с ней, в некоторых сервисах можно было разрешать входить в систему без нее. С другой стороны, критический сервис может выбрать отключение всех входов в систему и полный отказ при недоступности 2FA. Например, банк может предпочесть отключение обслуживания вместо разрешения несанкционированного доступа к счетам клиентов.

Разделение затрат на сбои

Любая неудачная операция сопряжена с некоторыми издержками. Сбой загрузки данных с мобильного устройства на сервер приложения потребляет вычислительные ресурсы и пропускную способность сети для настройки RPC и передачи некоторых данных. Если вы сумеете перестроить свои потоки так, чтобы они выходили из строя раньше или дешевле, то сможете уменьшить издержки, связанные со сбоями, или вовсе их избежать.

Чтобы обосновать затраты на отказы, сделайте следующее.

Определите общие затраты на отдельные операции

Например, вы можете собирать показатели воздействия на процессор, память или пропускную способность во время нагрузочного тестирования конкретного API. Если время ограничено, в первую очередь сосредоточьтесь на самых важных операциях — по критичности или частоте.

Определите, на каком этапе операции эти расходы понесены

Вы можете проверить исходный код или использовать инструменты разработчика для сбора данных самоанализа (например, браузеры предлагают отслеживание этапов запроса). Вы даже можете использовать код для моделирования ошибок на разных этапах.

При наличии собранной информации об эксплуатационных издержках и точках отказа вы можете подумать об изменениях, помогающих отложить дорогостоящие операции до более успешного разрешения проблемы в системе.

Вычислительные ресурсы

Вычислительные ресурсы, потребляемые неудачной операцией — от ее начала до сбоя, — недоступны для любых других операций. Такой эффект умножается, если клиенты пытаются снова получить ответ в случае сбоя. Это даже может привести к каскадному отказу системы. Чтобы быстрее высвободить вычислительные ресурсы, проверяйте условия ошибок на более ранних этапах выполнения. Например, нужно проверить достоверность запросов на доступ к данным до того, как система выделит память или начнет их чтение/запись. Файлы cookie SYN (<https://oreil.ly/EaL2N>) позволяют избежать выделения памяти для запросов на подключение TCP, исходящих от поддельных IP-адресов. CAPTCHA может помочь защитить самые дорогостоящие операции от злоупотребления.

В широком смысле, если сервер узнает, что его состояние ухудшается (например, по сигналам системы мониторинга), вы можете заставить его переключиться

в режим «хромой утки»¹: он продолжает обслуживание, но говорит вызывающим объектам, что нужно снизить скорость или прекратить отправку запросов. Этот подход позволяет посылать более удобные сигналы, к которым может адаптироваться общая среда, и одновременно минимизировать ресурсы, перенаправляемые на ошибки обслуживания.

Есть вероятность того, что несколько экземпляров сервера не будут использоваться из-за внешних факторов. Запускаемые ими сервисы могут быть «истощены» или изолированы из-за нарушения безопасности. При отслеживании таких условий ресурсы сервера могут быть временно освобождены для повторного использования другими сервисами. Но прежде чем перераспределять ресурсы, убедитесь в том, что все полезные для судебной экспертизы данные будут защищены.

Пользовательский опыт

Взаимодействия системы с пользователем должны иметь приемлемый уровень поведения в ухудшенных условиях. Идеальная система информирует пользователей о том, что ее сервисы могут работать со сбоями, но позволяет им продолжать взаимодействовать с работоспособными частями. Системы могут использовать разные протоколы подключения или аутентификации и авторизации или конечные точки для сохранения функционального состояния. Пользователи должны быть в курсе всего, что касается сбоя данных или угрозы безопасности. Небезопасные для использования функции должны быть явно отключены.

Автономный режим в приложении для совместной работы в Интернете может сохранить основные функции, возможность показывать обновления других пользователей или интеграцию с функциями чата, несмотря на временную потерю соединения. В приложении чата со сквозным шифрованием пользователи могут периодически менять свой ключ шифрования, используемый для защиты связи. Такое приложение сохранит доступ ко всем предыдущим сообщениям, так как это изменение не повлияет на их подлинность.

Пример плохого дизайна: весь ГПИ перестает отвечать на запросы из-за истечения времени ожидания одного из его RPC к серверу. Представьте себе мобильное приложение, которое подключается к серверу при запуске, чтобы отображать только самый свежий контент. Серверная часть может быть недоступной просто потому, что пользователь устройства намеренно отключил соединение. Тем не менее пользователи не увидят даже ранее кэшированные данные.

¹ Это описано в главе 20 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/load-balancing-datacenter/>).

Чтобы найти решение, обеспечивающее удобство использования и производительность в ухудшенном режиме, может потребоваться исследование и разработка пользовательского опыта (UX).

Скорость смягчения

Скорость восстановления системы после сбоя влияет на стоимость этого сбоя. Время отклика включает период между внесением смягчающих изменений и временем обновления и восстановления последнего затронутого экземпляра компонента. Избегайте размещения критических точек отказа в таких компонентах, как клиентские приложения, которые сложнее контролировать.

Вернемся к более раннему примеру мобильного приложения, обновляющего данные при запуске. При таком дизайне подключение к серверу превращается в критическую зависимость. В этой ситуации начальные проблемы усугубляются медленной и неконтролируемой скоростью обновления приложений.

Развертывание механизмов реагирования

Идеальная система должна активно реагировать на ухудшающиеся условия с помощью безопасных, предварительно запрограммированных мер, максимизирующих эффективность реагирования при минимизации рисков для безопасности и надежности. Автоматизированные измерения обычно работают лучше людей. Люди реагируют медленнее, могут не иметь доступа к сети или безопасности для выполнения операции и не так хороши в решении задач с несколькими переменными. Но люди должны оставаться в курсе событий для противовеса и сдерживания угроз и принимать решения в непредвиденных или нестандартных обстоятельствах.

Давайте подробно рассмотрим управление чрезмерной нагрузкой — из-за потери обслуживающей способности, доброкачественных всплесков трафика или даже DoS-атак. Реакция людей может быть замедлена, а трафик может перегружать серверы настолько, что это может привести к каскадным сбоям и глобальному сбою сервиса. Создание защиты путем постоянного перераспределения ресурсов серверов приводит к пустой трате денег и не гарантирует безопасного реагирования. Вместо этого серверы должны корректировать реакцию на нагрузку в зависимости от текущих условий. Здесь вы можете использовать две конкретные стратегии автоматизации:

- сброс нагрузки через возврат ошибок, а не обслуживание запросов;
- регулирование количества клиентов при помощи задержки ответов до достижения крайнего срока запроса.

На рис. 8.3 показан всплеск трафика, превышающий пропускную способность. Рисунок 8.4 иллюстрирует эффекты сброса и регулирования нагрузки для управления скачком нагрузки. Обратите внимание на следующее.

- Кривая — это запросы в секунду, а область под ней — общее количество запросов.
- Пробелы показывают обработанный без сбоев трафик.
- Область с обратной косой чертой — ухудшение трафика (некоторые запросы не выполнены).
- Заштрихованные области — это отклоненный трафик (все запросы не выполнены).
- Область с косой чертой иллюстрирует трафик, подлежащий расстановке приоритетов (важные запросы выполнены).

На рис. 8.3 показано, как система может на самом деле аварийно завершить работу. Это приведет к большему влиянию на объем (количество потерянных запросов) и время (длительность простоя увеличивается после скачка трафика). На рис. 8.3 можно различить и неконтролируемый характер ухудшенного трафика (область с обратной косой чертой) до сбоя системы. На рис. 8.4 показано, что система со сбросом нагрузки отклоняет меньше трафика, чем на рис. 8.3 (область с перекрестной штриховкой), а оставшаяся часть трафика обрабатывается без сбоев (область с пробелами) или отклоняется при более низком приоритете (область с косой чертой).

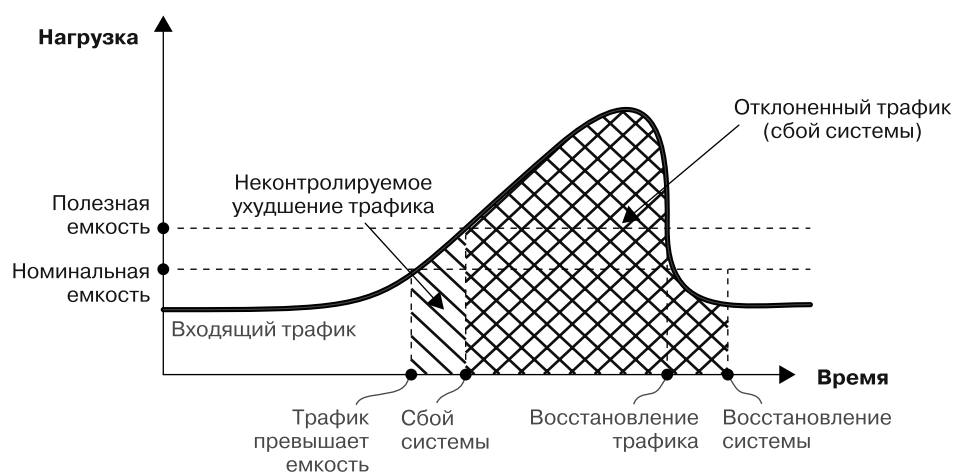


Рис. 8.3. Полный сбой и возможный каскадный сбой из-за скачка нагрузки

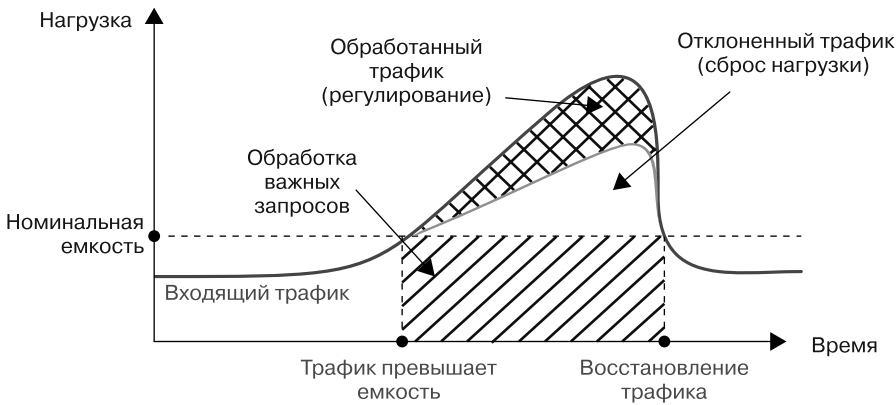


Рис. 8.4. Использование сброса и регулирования нагрузки для управления скачком трафика

Сброс нагрузки

Основная задача обеспечения устойчивости сброса нагрузки (описанная в главе 22 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/addressing-cascading-failures/>)) — стабилизация компонентов при максимальной нагрузке. Это может быть особенно полезно для сохранения критически важных для безопасности функций. Когда нагрузка на компонент начинает превышать его производительность, нужно, чтобы он обслуживал ошибки для всех чрезмерных запросов, а не выходил из строя. Из-за сбоя могут быть недоступны *все* ресурсы компонента, а не только ресурсы для избыточных запросов. Когда эти ресурсы исчезнут, нагрузка просто переместится в другое место. Это может вызвать каскадный сбой.

Сброс нагрузки позволяет освободить ресурсы сервера до достижения ее предельной емкости и сделать эти ресурсы доступными для более важной работы. Для выбора запросов, которые нужно отбросить, у сервера должно быть представление о приоритете и стоимости запроса. Вы можете задать политику, определяющую количество запросов каждого типа, которые следует отбрасывать, основываясь на приоритете и стоимости запроса и текущем использовании сервера. Назначьте приоритеты запроса на основе его критичности или зависимостей с точки зрения бизнеса (критические функции безопасности должны получить высокий приоритет). Вы можете измерить или эмпирически оценить стоимость запроса¹. Эти измерения должны быть сопоставимы с измерениями использования сервера, такими как использование ЦП и (возможно) памяти. Затраты на вычислительные запросы должны быть экономичными.

¹ См. главу 21 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/handling-overload/>).

Регулирование

Регулирование (описанное в главе 21 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/handling-overload/>)) косвенно изменяет поведение клиента, задерживая текущую операцию для откладывания будущих. После получения запроса сервер может подождать, прежде чем обработать его, или дожидаться отправки ответа клиенту сразу после обработки. Такой подход снижает частоту запросов, получаемых сервером от клиентов (если клиенты отправляют запросы последовательно). Это означает, что вы можете перенаправить ресурсы, сохраненные во время ожидания.

Подобно сбросу нагрузки, вы можете определить политики для применения регулирования для проблемных или для всех клиентов. Приоритет запроса и его стоимость важны при выборе запросов, которые нужно ограничить.

Автоматический ответ

Благодаря статистике использования сервера мы можем определить, когда следует применить такие средства контроля, как сброс нагрузки и регулирование. Чем больше загружен сервер, тем меньше трафика или нагрузки он может обработать. Если для активации средств контроля нужно слишком много времени, запросы с более высоким приоритетом могут быть отброшены или ограничены.

Для эффективного управления механизмами деградации вам может понадобиться центральный внутренний сервис. Вы можете превратить деловые соображения о критически важных функциях и стоимости отказа в политики и сигналы для этого сервиса. Такой сервис может собирать эвристические данные о клиентах и сервисах для распространения обновленных политик на все серверы почти в реальном времени. Затем серверы могут применять эти политики в соответствии с правилами использования.

Некоторые возможности автоматического ответа.

- Внедрение систем балансировки нагрузки, способных реагировать на сигналы регулирования и переместить трафик на серверы с более низкой нагрузкой.
- Обеспечение защиты от DoS, которая может помочь при ответе вредоносным клиентам, если регулирование неэффективно или наносит ущерб системе.
- Использование отчетов сброса нагрузки для критически важных сервисов, чтобы подготовиться к переходу на альтернативные компоненты (стратегия, которую мы обсудим позже в этой главе).

Вы можете пользоваться автоматизацией, чтобы обнаружить сбои самостоятельно. Сервер, определяющий свою неспособность обслуживать некоторые

или все классы запросов, может перейти в режим полного сброса загрузки. Автономное обнаружение желательно, потому что не всегда нужно полагаться на внешние сигналы (возможно, имитируемые злоумышленником), чтобы все серверы одновременно не вышли из строя.

Реализуя постепенное ухудшение, важно определить и записать уровни деградации системы, независимо от причины проблемы. Эта информация полезна для диагностики и отладки. Сообщение о фактическом сбросе или регулировании нагрузки (самостоятельном или направленном) поможет вам оценить общее состояние, пропускную способность и обнаружить ошибки или атаки. Эта информация нужна и для оценки текущей оставшейся емкости системы и влияния пользователей. Другими словами, стоит знать, насколько повреждены отдельные компоненты и вся система и какие действия могут понадобиться. После происшествия может быть полезна оценка эффективности ваших механизмов деградации.

Автоматизируйте ответственно

Чтобы механизмы автоматического реагирования непреднамеренно не повредили безопасность и надежность системы, соблюдайте осторожность, создавая их.

Сбой безопасности и сбой защиты

Проектируя систему для обработки отказов, соблюдайте баланс между оптимизацией для надежности — «при отказе открыт» (надежность) и для безопасности — «при отказе закрыт» (безопасность)¹.

- Чтобы максимизировать *надежность*, система должна противостоять сбоям и работать в обычном режиме, насколько это возможно в условиях неопределенности. Даже если целостность системы нарушена, пока ее конфигурация жизнеспособна, оптимизированная для доступности система будет продолжать обслуживание. Если ACL не удалось загрузить, по умолчанию ACL равны «разрешить все».
- Для обеспечения максимальной *безопасности* система должна полностью блокироваться в условиях неопределенности. Если система не может

¹ Понятия «при отказе открыт» и «при отказе закрыт» относятся к сервису, *продолжающему работать* (будучи надежным) или *закрывающемуся* (будучи безопасным) соответственно. Термины «при отказе открыт» и «при отказе закрыт» часто используются взаимозаменяемо с терминами «отказоустойчивость» и «защищенность от взлома», как описано в главе 1.

проверить свою целостность — независимо от того, скрыта часть настроек на неисправном диске или злоумышленник изменил настройки для взлома — ей нельзя доверять и она должна защищать себя, насколько это возможно. Если ACL не удалось загрузить, ACL по умолчанию — «запретить все».

Эти принципы надежности и безопасности явно расходятся. Для устранения противоречия каждая организация должна сначала определить свое минимальное не подлежащее обсуждению состояние безопасности. После этого нужно найти способы обеспечения требуемой надежности критических функций сервисов безопасности. Например, сеть, настроенная на отбрасывание пакетов с низким QoS — качеством обслуживания (Quality of Service), может потребовать, чтобы ориентированный на безопасность трафик RPC был помечен для особого QoS. Это предотвратит отбрасывание пакетов. Для RPC-серверов, ориентированных на безопасность, может понадобиться специальная маркировка во избежание нехватки ЦП планировщиками рабочей нагрузки.

КОМПРОМИСС БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: МЕХАНИЗМЫ РЕАГИРОВАНИЯ

Критически важные для безопасности операции не должны открываться при отказе. Из-за их открытия при сбое злоумышленник может снизить безопасность системы, например, используя только DoS-атаки. Но это не означает, что критически важные для безопасности операции вообще не подходят для контролируемой деградации. Более доступный альтернативный компонент (см. подраздел «Типы компонентов» далее в этой главе) может заменить собой регулярного контроля безопасности. Если в нем предусмотрены *более строгие* меры безопасности, попытки нарушить регулярные меры становятся невыгодными для преступника. Это эффективно повышает устойчивость системы.

Вовлечение людей

Иногда люди должны участвовать в принятии решений по ухудшению обслуживания. Способность основанных на правилах систем оценивать что-либо ограничена predetermined правилами. Автоматизация не работает при непредвиденных обстоятельствах, не соответствующих ни одному из predetermined ответов системы. Автоматический ответ тоже может привести к непредвиденным обстоятельствам из-за ошибки в коде. Чтобы разрешить человеку вмешиваться в решения этих и подобных ситуаций, нужно предусмотрительно проектировать системы.

Во-первых, вы должны предотвратить отключение службами автоматизации сервисов, используемых сотрудниками для восстановления инфраструктуры (см. раздел «Экстренный доступ» главы 9). Важно спроектировать защиту для этих систем, чтобы даже DoS-атаки не могли полностью предотвратить доступ.

Например, SYN-атака не должна мешать открывать TCP-соединение для SSH-сеанса. Обязательно реализуйте альтернативы с минимумом зависимостей и постоянно проверяйте их возможности.

Не допускайте, чтобы автоматизация вносила неконтролируемые изменения в политики значительной величины (например, один сервер теряет *все* RPC) или значительной области (*все* серверы теряют некоторые RPC). Вместо этого подумайте об изменении бюджета. Он не обновляется автоматически, когда автоматизация исчерпывает все ресурсы. Человек должен увеличить бюджет или принять другое решение.

Управление радиусом взлома

Вы можете добавить еще один уровень в свою стратегию защиты в глубину, ограничив область действия каждой части вашей системы. Рассмотрим сегментацию сети. В прошлом у организации была обычная сеть, в которой содержались все ее ресурсы (компьютеры, принтеры, хранилище, базы данных и т. д.). Эти ресурсы были видны любому пользователю или сервису в этой сети, и доступ контролировался самим ресурсом.

Самый распространенный способ повышения безопасности на сегодняшний день — *сегментирование* сети и предоставление доступа к каждому сегменту определенным классам пользователей и сервисов. Этого можно достичь с помощью виртуальных локальных сетей (VLAN) с сетевыми ACL, представляющими собой простое стандартное решение. Вы можете контролировать трафик в каждом сегменте и управлять их взаимодействием. Еще можно ограничить доступ каждого сегмента к «необходимой информации».

Сегментация сети — хороший пример общей идеи разделения на части, которая обсуждалась в главе 6. *Разделение на части* — это намеренное создание небольших отдельных операционных блоков (разделов) и ограничение доступа к каждому из них. Хорошая идея — разделить большинство составляющих ваших систем — серверы, приложения, хранилище и т. д. При использовании настройки одной сети злоумышленник, скомпрометировавший учетные данные пользователя, может получить доступ к любому устройству в сети. Но при использовании разделения нарушение безопасности или перегрузка трафика в одном разделе не ставят под угрозу все остальные.

Контроль радиуса взлома означает разделение воздействия события аналогично отсекам корабля, обеспечивающим его устойчивость к затоплению. Проектируя в целях устойчивости, вы должны создать отдельные барьеры, ограничивающие злоумышленников и случайные сбои. Эти барьеры позволят вам лучше

адаптировать и автоматизировать ответы. Вы можете использовать их для создания областей отказов, обеспечивающих избыточность компонентов и изоляцию сбоев, как описано в разделе «Области отказов и избыточность».

Разделы помогают при карантинных мероприятиях, уменьшая необходимость для респондентов балансировать между защитой и сохранением доказательств. Некоторые разделы могут быть изолированы и заморожены для анализа во время восстановления других. Разделы создают естественные границы для замены и исправления во время реагирования на происшествия. Отсек может быть отброшен для сохранения оставшейся части системы.

Для контроля радиуса взлома у вас должен быть способ установления границ, при котором вы будете уверены в их безопасности. Рассмотрим задание, выполняемое в производственной среде, как один раздел¹. Это задание должно разрешать некоторый доступ (вы хотите, чтобы раздел был полезен), но не неограниченный доступ (вы хотите защитить раздел). Ограничение доступа к заданию зависит от вашей способности распознавать конечные точки в производственной среде и подтверждать их идентификацию.

Это можно сделать с помощью аутентифицированных удаленных вызовов процедур, идентифицирующих обе стороны в одном соединении. Для защиты идентификационных данных сторон от подделки и сокрытия их содержимого от сети эти RPC используют взаимно аутентифицированные соединения, способные удостоверить идентификационные данные обеих сторон, подключенных к сервису. Чтобы разрешить конечным точкам принимать более обоснованные решения о других разделах, вы можете добавить дополнительную информацию, которую конечные точки публикуют вместе с идентификацией. Например, информацию о местоположении, чтобы отклонить нелокальные запросы.

После создания механизмов для генерации разделов вы столкнетесь с трудным компромиссом. Нужно ограничить свои операции достаточным разбиением на разделы полезного размера, но не создавать *слишком большое* разделение. Один сбалансированный подход к разделению будет рассматривать каждый метод RPC как отдельный раздел. Это выравнивает разделы по логическим границам приложения, и их количество будет напрямую зависеть от количества системных функций.

Нужно тщательнее рассмотреть разделение, контролирующее приемлемые значения параметров методов RPC. Хотя это приведет к ужесточению мер безопасности, число возможных нарушений для каждого метода RPC пропорционально количеству клиентов RPC. Это усложняет все функции системы

¹ См. главу 2 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/production-environment/>) для описания производственной среды в Google.

и требует координации изменений в клиентском коде и политике сервера. С другой стороны, разделами, охватывающими целый сервер (независимо от его сервисов RPC или их методов), намного проще управлять, но они менее ценны. При обдумывании этого компромисса нужно советоваться с командами по управлению инцидентами и операциями, чтобы оценить выбор типов разделов и проверить его полезность.

Несовершенные разделы, неидеально охватывающие все крайние случаи, тоже могут быть ценными. Например, в процессе обнаружения крайних случаев злоумышленник может совершить ошибку и тем самым выдать себя. Любое время, которое потребуется такому противнику, чтобы покинуть раздел, — это дополнительное время для действий вашей команды реагирования на инциденты.

Команды по управлению инцидентами должны планировать и практиковать тактику герметизации разделов для ограничения радиуса взлома или действий злоумышленника. Отключение части вашей производственной среды — серьезный шаг. Хорошо спроектированные разделы позволяют командам по управлению происшествиями выполнять действия, пропорциональные происшествиям, поэтому не обязательно переводить всю систему в автономный режим.

При реализации разделения вы сталкиваетесь с компромиссом между тем, чтобы все клиенты совместно использовали один экземпляр данного сервиса¹, и запуском отдельных экземпляров сервиса, поддерживающих отдельных клиентов или их подмножества.

Запуск двух виртуальных машин (ВМ), каждая из которых контролируется разными не доверяющими друг другу объектами, на одном оборудовании сопряжен с определенным риском. Возможны уязвимости нулевого дня на уровне виртуализации или незначительные утечки информации между виртуальными машинами. Некоторые клиенты могут решить устранить эти риски, разделив развертывания на основе физического оборудования². Для упрощения такого подхода многие облачные провайдеры предлагают развертывание на выделенном оборудовании для каждого клиента. Здесь стоимость сокращения использования ресурсов отражается в надбавке к цене.

Разделение повышает устойчивость системы до тех пор, пока в ней есть механизмы для его поддержания. Отслеживать эти механизмы и обеспечивать

¹ Этот экземпляр сервиса будет по-прежнему обслуживаться многими репликами базового сервера, но будет функционировать как единый логический раздел.

² Например, Google Cloud Platform предлагает единый режим — эксклюзивный доступ к единственному узлу (<https://oreil.ly/anLXq>).

их работу непросто. Для предотвращения регрессии обязательно проверьте, действительно ли операции, запрещенные через границы разделения, терпят неудачу (см. «Непрерывная проверка»). Так как операционная избыточность опирается на разделение (описанное в разделе «Области отказов и избыточность» далее), ваши механизмы проверки могут охватывать как запрещенные, так и ожидаемые операции.

Google проводит разделение на части по ролям, местоположению и времени. Когда злоумышленник пытается скомпрометировать разделенную систему, потенциальный масштаб любой отдельной атаки значительно уменьшается. Если система скомпрометирована, команды управления происшествиями могут отключить только ее части для устранения последствий, оставив другие в рабочем состоянии. В следующих разделах подробно рассматриваются типы разделения.

Разделение по ролям

Большинство современных систем на базе микросервисов дают возможность пользователям запускать задания под определенными *ролями*, иногда называемыми *учетными записями сервисов*. Затем заданиям предоставляются учетные данные, позволяющие им проходить проверку подлинности для других микросервисов в сети с их конкретными ролями. Если злоумышленник скомпрометирует одно задание, он сможет действовать в сети под соответствующей ролью. Так как теперь преступник может получить доступ ко всем данным, к которым имеют доступ другие задания, запущенные под той же ролью, это означает, что он скомпрометировал и другие задания.

Для ограничения области такой компрометации разные задания обычно должны выполняться под разными ролями. К примеру, если у вас есть два микросервиса, которым нужен доступ к двум разным классам данных (скажем, к фотографиям и текстовым чатам), их использование под разными ролями может повысить устойчивость вашей системы, даже если два микросервиса разрабатываются и управляются одной командой.

Разделение по местоположению

Разделение по местоположению помогает ограничить воздействие атакующего дополнительным измерением: местом работы микросервиса. Вы можете запретить злоумышленнику, физически скомпрометировавшему один центр обработки данных, читать данные во всех других центрах обработки. Еще вы можете настроить систему так, чтобы права доступа самых влиятельных пользо-

вателей-администраторов были ограничены только определенными регионами для уменьшения инсайдерского риска.

Самый очевидный способ добиться разделения по местоположению — запускать одни и те же микросервисы под разными ролями в разных местах (например, в центрах обработки данных или разных регионах облачной среды, которые также обычно соответствуют разным физическим местоположениям). После вы можете пользоваться привычными механизмами контроля доступа для защиты экземпляров одного сервиса в разных местах, так же, как и при защите сервисов, работающих под разными ролями.

Разделение по местоположению помогает противостоять атаке, которая затрагивает разные места. Благодаря криптографическим разделам на основе местоположения можно ограничивать доступ к приложениям и их хранимым данным определенными местоположениями, включающими радиус локальных атак.

Физическое местоположение — естественная граница разделения, так как многие неблагоприятные события связаны с местонахождением. Например, стихийные бедствия ограничиваются регионом, как и другие локальные сбои вроде перебоев с оптоволокном, неполадок с электроэнергией или пожаров. Вредоносные атаки, требующие физического присутствия злоумышленника, тоже ограничиваются местами, в которые он может попасть, и почти никто, кроме наиболее способных злоумышленников (например, на уровне государства), не имеет возможности отправлять взломщиков в несколько мест одновременно.

Точно так же степень подверженности риску может зависеть от характера физического местоположения. Риск конкретных видов стихийных бедствий меняется в зависимости от географического расположения. Кроме того, риск проникновения злоумышленника в здание и обнаружения открытого сетевого порта для подключения в стандартном офисе, где работает много людей и возможны посетители, выше, чем риск проникновения в центр обработки данных с жестко контролируемым физическим доступом.

Зная об этом, при проектировании систем учитывайте местоположение, чтобы гарантировать, что локализованные воздействия будут ограничены системами в этом регионе. Это позволит вашей мультирегиональной инфраструктуре продолжать работать. Убедитесь, что у сервиса, предоставляемого серверами в нескольких регионах, нет критической зависимости от сервера, размещенного в единственном центре данных. Точно так же важно проверить, что физическая компрометация одного местоположения не позволяет злоумышленнику легко скомпрометировать другие. Подключение к офису и к открытой точке доступа в конференц-зале не должно давать сетям злоумышленников доступ к рабочим серверам в вашем дата-центре.

Выравнивание физической и логической архитектуры

При разделении архитектуры на логические области отказов и безопасности важно выравнивать соответствующие физические границы с логическими. Будет полезно сегментировать сеть как по рискам сетевого уровня (таким как сети, подверженные вредоносному интернет-трафику, и доверенные внутренние сети), так и по рискам физических атак. В идеале у вас должно быть разделение сети между корпоративной и производственной средами, физически расположенными в отдельных зданиях. Вы можете дополнительно разделить корпоративную сеть, отделив области с высоким трафиком посетителей, например залы конференций и собраний.

Физическая атака (кража или взлом сервера) может дать злоумышленнику доступ к важным секретам, ключам шифрования или учетным данным, которые позволят ему проникнуть в ваши системы. Помня об этом, вы можете логически разделить распределение секретов, ключей и учетных данных на физических серверах для снижения риска физической компрометации.

Если у вас есть веб-серверы в нескольких физических центрах обработки данных, то вместо того, чтобы использовать один сертификат для *всех* серверов, вы можете развернуть отдельный на каждом сервере или совместно использовать сертификат только между серверами в одном месте. Это позволит вам более гибко реагировать на физический компромисс одного центра обработки данных. Вы можете ограничить его трафик и отозвать только развернутые в этом дата-центре сертификаты. После чего вы можете перевести центр обработки данных в автономный режим для реагирования на происшествие и восстановления, одновременно обслуживая трафик оставшихся центров обработки данных. Если бы у вас был только один сертификат, развернутый на всех серверах, пришлось бы очень быстро заменить сертификаты на них всех, даже на фактически не скомпрометированных.

Изоляция доверия

Хотя сервисам для правильной работы может потребоваться обмен данными через границы местоположения, им может понадобиться и отклонять запросы из местоположений, с которыми не ожидается связи. Для этого вы можете ограничить связь по умолчанию и разрешить только ожидаемую через границы местоположения. Маловероятно, что все API любого сервиса будут использовать одинаковый набор ограничений местоположения. Пользовательские API обычно открыты глобально, тогда как API плоскости управления чаще всего ограничены. Поэтому понадобится детализированное (по вызовам API) управление разрешенными местоположениями. Благодаря созданию инструментов, облег-

чающих для любого конкретного сервиса измерение, определение и применение ограничений местоположения для отдельных API, команды могут использовать свои знания о каждом сервисе для изоляции местоположения.

Для ограничения связи на основе местоположения все идентификаторы должны включать метаданные местоположения. Система управления заданиями Google сертифицирует и запускает их в производственной среде. Когда система сертифицирует задание для выполнения в определенном разделе, она создает соответствие между меткой данных местоположения этого раздела и сертификатом задания. У каждого местоположения есть своя копия системы управления заданиями, сертифицирующая их для запуска в этом местоположении. Устройства в этом местоположении принимают задания только из данной системы. Это позволяет предотвратить проникновение злоумышленника в границы раздела и воздействие на другие локации. Сравните этот подход с использованием единого централизованного органа. Если бы для всей компании Google существовала только одна система управления заданиями, ее местоположение было бы весьма ценной информацией для злоумышленника.

После установления доверительной изоляции мы можем расширить ACL для хранимых данных, включив в них ограничения местоположения. Так мы можем отделить места для хранения (куда мы помещаем данные) от мест для доступа (где можно извлекать или изменять данные). Это открывает возможность довериться физической безопасности по сравнению с доступом через API. Иногда дополнительное требование физического оператора позволяет предотвратить удаленную атаку.

Для контроля нарушений в разделах у Google есть корень доверия (root of trust) в каждом местоположении. Он охватывает список доверенных корней и местоположений, которые они представляют, на все используемые устройства. Благодаря этому каждое устройство может обнаружить несанкционированный доступ в разных местах. Мы можем отозвать идентификацию местоположения, распространив обновленный список на все устройства и объявив местоположение ненадежным.

Ограничение доверия на основе местоположения

В Google мы решили спроектировать инфраструктуру нашей корпоративной сети так, чтобы местоположение не подразумевало никакого доверия. Вместо этого, по примеру сети с нулевым доверием в нашей инфраструктуре BeyondCorp (см. главу 5), рабочей станции можно доверять на основе выданного для отдельного устройства сертификата и утверждений о его конфигурации (например, об обновленном ПО). Подсоединение ненадежного компьютера к сетевому порту

офисного уровня назначит его ненадежной гостевой VLAN. Только авторизованные рабочие станции (аутентифицированные по протоколу 802.1x) назначаются соответствующей VLAN рабочей станции.

Мы решили не полагаться даже на физическое местоположение для установления доверия к серверам в центрах обработки данных. Один из мотивирующих примеров был получен после оценки красной командой среды центра обработки данных. В этом упражнении член красной команды поместил беспроводное устройство на подставку и быстро подключил его к открытому порту для дальнейшего проникновения во внутреннюю сеть центра обработки данных за пределами здания. Когда участники вернулись убрать за собой после выполнения упражнения, то обнаружили, что внимательный техник дата-центра тщательно закрепил кабели точки доступа. Очевидно, ему не понравилась неаккуратная установка устройства и он даже не задумался, зачем оно вообще установлено. Эта история показывает сложность определения доверия на основе физического местоположения, даже в пределах физически защищенной области.

В производственной среде Google, как и в проекте BeyondCorp, аутентификация между производственными сервисами базируется на доверительных отношениях между компьютерами на основе учетных данных для каждого устройства. Производственная среда Google не будет доверять вредоносному дополнению к неавторизованному устройству.

Изоляция конфиденциальности

После создания системы для изоляции доверия нужно изолировать ключи шифрования для гарантии того, что данные, защищенные с помощью корня шифрования в одном месте, не будут скомпрометированы в результате утечки ключей шифрования в другом месте. Например, если один из филиалов компании скомпрометирован, злоумышленники не смогут читать данные из других филиалов.

У Google есть базовые ключи шифрования, защищающие деревья ключей. Они охраняют данные в состоянии покоя за счет переноса и получения ключей.

Для изоляции шифрования и переноса ключей в местоположении убедитесь, что корневые ключи доступны только для правильного местоположения. Для этого нужна система распространения, которая размещает корневые ключи только в правильных местах. Система доступа к ключам должна использовать доверительную изоляцию для гарантии того, что эти ключи не будут доступны для объектов, не находящихся в соответствующем месте.

Пользуясь этими принципами, в данном местоположении можно использовать ACL на локальных ключах для предотвращения расшифровки данных удаленными злоумышленниками. Это происходит, даже если у злоумышленников есть доступ к зашифрованным данным (с помощью внутренней компрометации или эксфильтрации). Переход от глобального дерева ключей к локальному должен быть постепенным. Хотя любая часть дерева может независимо перемещаться из глобальной области в локальную, изоляция для листа или ветви дерева не будет завершена, пока все ключи над ним не перейдут в локальную область.

Разделение по времени

Наконец, полезно ограничить возможности противника по времени. Самый распространенный сценарий — злоумышленник взломал систему и украл ключ или учетные данные. Если вы периодически заменяете свои ключи и учетные данные и срок действия старых ключей истекает, для получения новых секретов злоумышленник должен все еще находиться в системе. Это дает вам новые возможности для обнаружения взлома. Даже если вы никогда не обнаружите кражу ключей, их замена будет иметь решающее значение. Вы можете закрыть путь, использованный злоумышленником для получения доступа к ключу или учетным данным, во время обычной работы по обеспечению безопасности (например, при исправлении уязвимости).

Как мы обсуждаем в главе 9, замена ключей и учетных данных, а также истечение срока их действия требуют осторожных компромиссов. Истечение срока действия учетных данных на основе системных часов может вызвать проблемы, если есть сбой, не позволяющий перейти к новым учетным данным до истечения срока действия старых. Чтобы разделение по времени было полезным, сбалансируйте частоту замены ключей и риск простоя или потери данных в случае отказа механизма замены.



Области отказов и избыточность

До сих пор мы изучали проектирование систем, корректирующих свое поведение в ответ на атаки и сдерживающих последствия атак с помощью разделения на части. Для устранения полных сбоев компонентов системы проектирование должно учитывать избыточность и отдельные области отказов. Мы надеемся, что эта тактика поможет уменьшить последствия неудач и предотвратить полный крах. Особенно важно смягчить сбои критических

компонентов, ведь любая система, зависящая от неисправных критических компонентов, тоже подвержена риску полного отказа.

Вместо постоянного предотвращения всех сбоев вы можете создать сбалансированное решение для своей организации, сочетая следующие подходы.

- Разбивайте системы на независимые области отказов.
- Уменьшите вероятность единственной основной причины, влияющей на компоненты в нескольких областях отказа.
- Создайте избыточные ресурсы, компоненты или процедуры, способные заменить отказавшие.

Области отказов

Область отказов — один из вариантов контроля радиуса взлома. Вместо структурного разделения по ролям, местоположению или времени области отказов достигают функциональной изоляции, разделяя систему на несколько эквивалентных, но полностью независимых копий.

Функциональная изоляция

Для клиентов область отказов выглядит как единая система. Если потребуется, любой раздел может взять на себя работу системы во время простоя. Из-за того что у раздела есть только часть системных ресурсов, он может поддерживать лишь часть емкости системы. В отличие от управления разделениями по ролям, расположению и времени, использование областей отказов и поддержание их изоляции требуют постоянных усилий. Они повышают устойчивость системы, чего не делают другие средства управления радиусом взлома.

Области отказов помогают защитить системы от глобального воздействия, ведь одно событие обычно не затрагивает все области сразу. Но в особых случаях значимое событие может нарушить работу нескольких или даже всех областей отказов. К примеру, вы можете представить устройства хранения данных (жесткие диски или твердотельные накопители) как области отказов. Несмотря на отказ любого из устройств, вся система хранения продолжает работать, так как она создает новую реплику данных в другом месте. Если происходит сбой большого количества устройств хранения данных и не хватает запасных для обслуживания реплик, дальнейшие сбои могут привести к потере данных в системе хранения.

Изоляция данных

Подготовьтесь к возможному получению неверных данных в источнике или отдельных областях отказов. Каждому экземпляру области отказов нужна собственная копия данных для его функционирования независимо от других областей. Для достижения изоляции данных мы советуем двойной подход.

Во-первых, вы можете ограничить способ обновления данных в области отказов. Система принимает новые данные только после того, как они пройдут все проверки на предмет типичных и безопасных изменений. Некоторые исключения требуют дополнительного обоснования, а механизм аварийного доступа¹ может позволить новым данным поступать в область отказов. В результате вы сможете успешно предотвратить атаки злоумышленников или программные ошибки, чтобы в систему не были внесены разрушительные изменения.

Рассмотрим изменения ACL. Человеческая ошибка или ошибка в ПО, генерирующем ACL, может привести к появлению пустого ACL. Это означает запрет доступа для всех². Такое изменение может вызвать сбой системы. Точно так же преступник может попытаться расширить зону действия, добавив в ACL предложение «разрешить все».

В Google у отдельных сервисов обычно есть конечная точка RPC для приема новых данных и передачи сигналов. Такие среды программирования, как в главе 12, содержат API для управления версиями снимков данных и оценки их достоверности. Клиентские приложения могут использовать логику фреймворка для проверки безопасности новых данных. Централизованные службы передачи данных контролируют качество для обновления данных. Службы передачи данных проверяют, откуда их взять, как упаковать и когда отправлять упакованные данные. Для защиты автоматизации от массового сбоя Google ограничивает глобальные изменения с помощью квот для каждого приложения. Мы запрещаем действия, изменяющие несколько приложений одновременно или слишком быстро меняющие ресурсы приложения в течение определенного времени.

Во-вторых, разрешение системам записывать последнюю удачную конфигурацию на диск делает их устойчивыми к потере доступа к API конфигурации. Они могут использовать сохраненную конфигурацию. Многие системы Google

¹ Механизмы аварийного доступа — это механизмы, способные обойти политики. Это позволяет инженерам быстро устранять простои. См. подраздел «Механизм аварийного доступа» в главе 5.

² Системы, использующие ACL, должны закрываться при сбое (защищаться) с доступом, явно предоставленным записями ACL.

сохраняют старые данные в течение ограниченного периода времени на случай повреждения новейших данных. Это еще один пример защиты в глубину, позволяющий обеспечить долгосрочную устойчивость.

Практические аспекты

Даже разделение системы только на две области отказов дает существенные преимущества.

- Наличие двух областей отказов обеспечивает возможности регрессии А/В и ограничивает радиус действия изменений системы до одной области отказов. Чтобы реализовать эту функцию, используйте одну область отказа как канареечную и создайте политику, не допускающую одновременного обновления обеих областей.
- Географически разделенные области отказов могут обеспечить изоляцию в случае стихийных бедствий.
- Вы можете использовать разные версии ПО в разных областях отказов, уменьшая вероятность того, что одна ошибка повредит все серверы или испортит все данные.

Объединение данных и функциональная изоляция улучшают общую устойчивость и управление происшествиями. Такой подход ограничивает риск изменений данных, которые принимаются без обоснования. При возникновении проблем изоляция задерживает их распространение в отдельных функциональных единицах. Это дает другим защитным механизмам больше времени для обнаружения и реагирования, что очень полезно во время реагирования на происшествия, когда важна каждая минута. Выдвигая несколько возможных исправлений параллельно в разные области отказов, вы можете независимо оценить, какие из них имеют желаемый эффект. Так можно избежать случайного запуска спешного обновления с ошибочным «исправлением» в глобальном масштабе. Это может привести к дальнейшему ухудшению работы всей системы.

У областей отказов есть эксплуатационные издержки. Даже простому сервису с несколькими областями отказов нужна поддержка нескольких копий конфигураций сервиса, основанных на идентификаторах областей отказов. Для этого необходимо следующее.

- Обеспечение согласованности конфигураций.
- Защита всех конфигураций от одновременного повреждения.
- Скрытие от клиентских систем разделения на области отказов, чтобы клиент случайно не подключился к определенной области.

- Потенциальное разбиение всех зависимостей. Одно общее изменение зависимостей может случайно распространиться на все области отказов.

Отметим, что область отказов может полностью потерпеть неудачу, даже если только одна из важных составляющих системы выйдет из строя. В итоге вы сначала разбили исходную систему на области отказов, чтобы она могла работать, даже если копии области полностью провалились. Но создание областей отказов просто сдвигает проблему на один уровень вниз. В следующем разделе обсуждается, как можно использовать альтернативные компоненты для снижения риска полного отказа всех областей.

Типы компонентов

Устойчивость области отказа выражается в общей надежности ее компонентов и их зависимостей. Чем больше областей отказов, тем устойчивее будет вся система. Но эта повышенная устойчивость компенсируется эксплуатационными затратами на поддержание все большего количества областей отказов.

Для улучшения устойчивости попробуйте замедлять или останавливать разработку новых функций, получая взамен больше стабильности. Избегая добавления новой зависимости, вы избегаете и возможных сбоев. Если вы перестанете обновлять код, новых ошибок будет меньше. Но даже если остановить разработку новых функций, вам все равно придется реагировать на случайные изменения состояния вроде уязвимостей в системе безопасности и увеличения пользовательского спроса.

Очевидно, что прекращение разработки новых функций — нерабочая стратегия для большинства организаций. В следующих разделах мы поделимся альтернативными подходами к балансировке надежности и ценности. В целом есть три класса надежности сервисов: высокая производительность, высокая доступность и низкая зависимость.

Компоненты с высокой производительностью

Компоненты, которые вы создаете и используете в обычной деятельности, составляют ваш сервис с *высокой производительностью*. Это потому, что основной набор, обслуживающий ваших пользователей, состоит из таких компонентов. Именно здесь ваш сервис поглощает всплески пользовательских запросов или потребления ресурсов из-за новых функций. Компоненты с высокой пропускной способностью тоже поглощают трафик DoS, пока не вступит в силу смягчение последствий DoS или не начнется постепенная деградация.

Так как эти компоненты наиболее критически важны для вашего сервиса, вы должны сначала сосредоточить свои усилия на них. Например, следуя рекомендациям по планированию емкости, развертыванию ПО и конфигурации и многому другому, как описано в части III книги SRE (<https://landing.google.com/sre/sre-book/toc/index.html>) и части II Site Reliability Workbook (<https://landing.google.com/sre/workbook/toc/>).

Компоненты с высокой доступностью

Если в вашей системе есть компоненты, сбой которых влияет на всех пользователей или влечет другие серьезные последствия — компоненты с высокой производительностью, описанные в предыдущем разделе, — вы можете уменьшить риски, развернув копии этих компонентов. У этих копий *высокая доступность*, если они обеспечивают значительно более низкую вероятность сбоев.

Для более низкой вероятности сбоев копии должны быть настроены с меньшим количеством зависимостей и ограниченной скоростью изменений. Благодаря такому подходу будет меньше сбоев инфраструктуры или операционных ошибок, нарушающих работу компонентов. Например, вы можете сделать следующее.

- Использовать данные, кэшированные в локальном хранилище во избежание зависимости от удаленной БД.
- Использовать старый код и настройки, чтобы избежать недавних ошибок в новых версиях.

Запуск компонентов с высокой доступностью требует дополнительных ресурсов, расходы на которые масштабируются пропорционально размеру набора компонентов. Соотношение цены/качества зависит от того, должны ли компоненты с высокой доступностью поддерживать всю пользовательскую базу или только ее часть. Похожим образом настройте возможности постепенного снижения производительности для каждого компонента с высокой пропускной способностью и высокой доступностью. Это позволит тратить меньше ресурсов для более быстрой контролируемой деградации.

Компоненты с низкой зависимостью

Если сбой в компонентах с высокой доступностью неприемлемы, сервис с *низкой зависимостью* будет следующим уровнем устойчивости. Для низкой зависимости нужна альтернативная реализация с минимумом зависимостей. Эти минимальные зависимости и будут низкими. Общий набор сервисов, процессов или заданий, способных потерпеть неудачу, настолько мал, насколько позволяют потребности и расходы бизнеса.

Сервисы с высокой пропускной способностью и доступностью могут обслуживать большие пользовательские базы и предлагать богатые функции благодаря уровням взаимодействующих платформ (виртуализация, контейнеризация, планирование, фреймворки приложений). Эти уровни помогают масштабировать систему, позволяя сервисам быстро добавлять или перемещать узлы. Но несмотря на это они еще и обнаруживают более высокие показатели перебоев в работе по мере увеличения бюджетов ошибок на сотрудничающих платформах¹. Для сервисов с низкой зависимостью нужно упростить стек обслуживания, пока они не смогут принять совокупный бюджет ошибок стека. Упрощение же стека обслуживания приведет к необходимости удалять функции.

Для компонентов с низкой зависимостью необходимо выяснить, возможно ли создать альтернативу для критического компонента, чтобы критические и альтернативные компоненты не имели общих областей отказов. В конце концов, успех резервирования обратно пропорционален вероятности того, что одна и та же первопричина повлияет на оба компонента.

Рассматривайте пространство хранения данных как основной строительный блок распределенной системы. Может понадобиться хранить запасные локальные копии данных на случай, когда серверы RPC для хранения данных недоступны. Но общий подход к хранению локальных копий данных не всегда практичен. Эксплуатационные расходы увеличиваются при поддержании избыточных компонентов, а пользы от дополнительных компонентов обычно нет.

На практике вы получаете небольшой набор компонентов с малыми зависимостями и ограниченными пользователями, функциями и затратами. Но они могут быть доступны для временных нагрузок или восстановления, что повышает их надежность. Когда большинство полезных функций обычно полагаются на множественные зависимости, сильно ухудшенный сервис лучше недоступного.

Представьте устройство, для которого предполагается, что операции только для записи или чтения доступны по сети. В домашней системе безопасности к таким операциям относятся запись журналов событий (только для записи) и поиск номеров экстренных служб (только для чтения). Для взлома нужно будет отключить домашнее интернет-соединение, что нарушает систему безопасности. Для противостояния такому типу сбоев вы настраиваете систему безопасности на использование локального сервера, реализующего те же API, что и удаленный сервис. Локальный сервер записывает журналы событий в локальное хранилище, обновляет удаленный сервис и повторяет неудачные попытки. Он же отвечает

¹ См. главу 3 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/embracing-risk/>).

на запросы по поиску экстренных телефонных номеров. Этот список номеров периодически обновляется с удаленного сервиса. С точки зрения домашней безопасности система работает как положено, пишет журналы и получает номера экстренных служб. Скрытый стационарный телефон с низкой зависимостью может предоставлять возможности набора номера как резервной копии для отключенного беспроводного соединения.

В качестве примера в масштабе предприятия глобальный сбой сети — один из самых страшных типов сбоев. Он влияет на функциональность сервиса и на способность респондентов устранять сбой. Большие сети управляются динамически и подвергаются большему риску глобальных отключений. Создание альтернативной сети, полностью исключающей повторное использование тех же элементов, что и в основной сети — соединения, коммутаторы, маршрутизаторы, домены маршрутизации или программное обеспечение SDN, — требует тщательного проектирования. Такая конструкция должна предусматривать на конкретное и узкое подмножество вариантов использования и рабочих параметров. Это позволяет вам сосредоточиться на простоте и понятности. Экономия на этой редко используемой сети приведет к ограничению доступных функций и пропускной способности. Несмотря на ограничения, этих результатов достаточно. Наша цель здесь — поддерживать только самые важные функции и только часть обычной пропускной способности.

Контроль избыточности

Избыточные системы настроены так, чтобы иметь больше одного варианта для каждой из своих зависимостей. Управлять выбором между этими вариантами непросто, и преступники могут использовать различия между избыточными системами, например, подталкивая систему к менее безопасному варианту. Помните, что гибкая конструкция обеспечивает безопасность и надежность, не жертвуя одним ради другого. Во всяком случае, более высокий уровень безопасности у альтернативы с низкой зависимостью может быть сдерживающим фактором для злоумышленников, которые надеются на износ вашей системы.

Стратегии восстановления после отказа

Предоставление набора серверов, обычно с помощью технологий балансировки нагрузки, повышает устойчивость к отказам серверной части. Нецелесообразно полагаться на один сервер RPC. Система будет зависеть каждый раз, когда нужно будет перезапустить этот сервер. Для простоты система обычно рассматривает избыточные серверные части как *взаимозаменяемые*, если все серверные модули обеспечивают одинаковое поведение функций.

Система, нуждающаяся в разных параметрах *надежности* (для одного и того же набора функциональных характеристик), должна опираться на отдельный набор взаимозаменяемых серверов, обеспечивающих желаемую надежность. Сама она должна реализовать логику, чтобы определить, какой набор поведений использовать и когда, например, с помощью флагов. Это дает вам полный контроль над надежностью системы, особенно во время восстановления. Сравните этот подход с запросом поведения, обладающим низкой зависимостью от одного сервера с высокой доступностью. Используя RPC, вы можете предотвратить попытки серверной части установить связь с недоступной зависимостью среды выполнения. Если зависимость при выполнении будет зависимостью при запуске, система перезапустится после происшествия.

Момент переключения на компонент с лучшей стабильностью зависит от ситуации. Если цель — автоматическое переключение при сбое, вы должны устранить различия в доступной емкости, используя средства, описанные в разделе «Управление деградацией» выше в этой главе. После отработки отказа такая система переключается на использование политик регулирования и отключения нагрузки, настроенных для альтернативного компонента. Чтобы система восстановилась после обновления затронутого компонента, предоставьте способ отключить этот отказ. В некоторых случаях вам понадобится стабилизировать колебания или точно контролировать восстановление после отказа.

Распространенные сложности

Мы отыскивали несколько распространенных сложностей при работе с альтернативными компонентами независимо от того, имеют они высокую доступность или низкую зависимость.

Со временем вы можете использовать альтернативные компоненты для нормальной работы. Любая зависимая система, которая начинает рассматривать альтернативные системы как резервную копию, перегружает их во время сбоя. Из-за этого альтернативная система может стать неожиданной причиной отказа в обслуживании. Есть и другая проблема. Неиспользование альтернативных компонентов, когда они необходимы, приводит к устареванию и неожиданным отказам.

Другая ловушка — неконтролируемый рост зависимости от других сервисов или количества требуемых вычислительных ресурсов. Системы могут развиваться по мере изменения требований пользователей, а разработчики добавляют новые функции. Со временем, чем больше будет зависимостей и зависимых компонентов, тем менее эффективно системы будут использовать ресурсы. Копии с высокой доступностью могут отставать от наборов с высокой производительностью. Сервисы с низкой зависимостью могут потерять согласованность

и воспроизводимость, если их предполагаемые рабочие ограничения непрерывно не контролируются и не проверяются.

Переход на альтернативные компоненты не должен нарушать целостность или безопасность системы. Рассмотрим следующие сценарии, в которых правильный выбор зависит от обстоятельств вашей организации.

- У вас есть сервис с высокой доступностью, выполняющий шестинедельный код по соображениям безопасности (для защиты от недавних ошибок). Но этот же сервис требует срочного исправления безопасности. Какой риск вы бы выбрали: не применять исправление или потенциально нарушить код с ним?
- Начальная зависимость сервиса удаленных ключей для извлечения закрытых, расшифровывающих данные ключей может стать слабой зависимостью из-за хранения закрытых ключей в локальном хранилище. Такой подход может создать неприемлемый риск для этих ключей? Способно ли увеличение частоты замены ключей в достаточной степени нейтрализовать этот риск?
- Вы определяете, что можете освободить ресурсы, уменьшив периодичность обновления нечасто изменяемых данных (например, ACL, списков отзыва сертификатов или метаданных пользователя). Нужно ли освобождать эти ресурсы, даже если это потенциально дает злоумышленнику больше времени для внесения изменений в данные или позволяет этим изменениям дольше оставаться незамеченными?

Наконец, убедитесь, что ваша система не сможет автоматически восстановиться в неподходящее время. Если показатели устойчивости автоматически снижают производительность системы, то эти же меры можно автоматически отключить. Но если вы применили аварийное переключение вручную, не давайте автоматизации отменить его. Система может быть помещена в карантин из-за уязвимости безопасности, или ваша команда может смягчить каскадный сбой.



Непрерывная проверка

Как с точки зрения надежности, так и с точки зрения безопасности мы хотим быть уверены в том, что наши системы ожидаемо поведут себя и в нормальных, и в непредвиденных обстоятельствах. Мы хотим быть уверены, что новые функции или исправления ошибок постепенно не разрушают многоуровневые механизмы устойчивости системы. Ничто не может

заменить действительное использование системы и подтверждение ее правильной работы.

Проверка направлена на наблюдение за системой в реалистичных, но контролируемых условиях — проверяются рабочие процессы в пределах одной или между несколькими системами¹. В отличие от хаос-инжиниринга (<https://oreil.ly/Fvx4L>), носящего исследовательский характер, проверка подтверждает конкретные свойства и поведение системы, описанные здесь и в главах 5, 6 и 9. Регулярные проверки гарантируют, что результаты остаются ожидаемыми и что сами проверки все еще функциональны.

Есть небольшие хитрости, как сделать проверку *значимой*. Для начала вы можете использовать некоторые концепции и практики, описанные в главе 15. Например, выбрать область проверки и измерить эффективные системные атрибуты. После вы можете постепенно расширять область действия проверки, создавая, обновляя или удаляя элементы. Вы можете извлечь полезную информацию из реальных происшествий. Эти детали — истина о поведении вашей системы. Они часто показывают нужные изменения в дизайне или пробелы в охвате проверки. Помните, что по мере изменения бизнес-факторов отдельные сервисы могут развиваться и изменяться. Это может привести к появлению несовместимых API или непредвиденных зависимостей.

Общая стратегия обслуживания проверки включает в себя следующее.

1. Обнаружение новых сбоев.
2. Реализацию проверок для каждого сбоя.
3. Повторное выполнение всех проверок.
4. Постепенный отказ от проверок, если соответствующие функции или варианты поведения больше не существуют.

Для обнаружения значимых сбоев используйте следующие источники.

- Регулярные отчеты об ошибках от пользователей и сотрудников.
- Нечеткое тестирование и подобные подходы (описанные в главе 13).
- Внедрение сбоев (сродни инструменту «Обезьяна хаоса» (Chaos Monkey tool) (<https://oreil.ly/fvSKQ>)).
- Аналитическое суждение экспертов в данной области, обслуживающих ваши системы.

¹ Это отличается от модульных, интеграционных и нагрузочных тестов, описанных в главе 13.

Создание фреймворка автоматизации поможет вам запланировать несовместимые проверки для запуска в разное время, чтобы они не конфликтовали друг с другом. Вам стоит отслеживать и устанавливать периодические проверки автоматизации для выявления нарушенных или скомпрометированных действий.

Основные направления проверки

Полезно проверять целые системы и комплексное взаимодействие между их сервисами. Но из-за высокой стоимости проверки реакции на сбой целых систем, обслуживающих реальных пользователей, приходится идти на компромисс. Проверка меньших системных реплик обычно доступнее и по-прежнему дает информацию, которую невозможно получить, проверяя отдельные части системы в изоляции. Например, вы можете сделать следующее.

- Рассказать, как вызывающие компоненты реагируют на серверную часть RPC, которая выдает ответ медленно или становится недоступной.
- Посмотреть, что происходит при нехватке ресурсов и возможно ли получить экстренную квоту на ресурсы, когда их потребление резко возрастает.

Другое практическое решение — использовать журналы для анализа взаимодействия между системами и/или их компонентами. При выполнении системой разделения операции, которые пытаются пересечь границы разделения по роли, местоположению или времени, не должны выполняться. Если в ваших журналах зафиксированы неожиданные успехи — отметьте их. Анализ журнала нужно проводить постоянно. Это позволит вам наблюдать за реальным поведением системы во время проверки.

Проверьте атрибуты принципов проектирования безопасности: наименьшие привилегии, понятность, адаптивность и восстановление. Проверка восстановления особенно важна, потому что усилия по восстановлению обязательно подразумевают действия человека. Люди непредсказуемы, и модульные тесты не сумеют проверить человеческие навыки и привычки. Проверяя схему восстановления, рассмотрите удобочитаемость инструкций по восстановлению и эффективность и совместимость его различных процессов.

Проверка атрибутов безопасности означает выход за рамки обеспечения правильных ответов системы. Убедитесь, что в коде или конфигурации нет известных уязвимостей. Активное тестирование на проникновение развернутой системы дает представление об устойчивости системы извне, часто выявляя векторы атак, которые разработчики не обнаружили.

Взаимодействия с компонентами с низкой зависимостью заслуживают особого внимания. По определению, такие компоненты развертываются в самых критических обстоятельствах, и альтернатив для них нет. К счастью, у хорошо спроектированной системы ограниченное число компонентов с низкой зависимостью. Это делает возможным определение проверок для всех критических функций и взаимодействий. Использовать компоненты с низкой зависимостью будет выгодно, *только* если они работают по мере необходимости. Основывайте ваши планы восстановления на компонентах с низкой зависимостью. Нужно также проверить их использование людьми для возможной ситуации, когда система ухудшится до этого уровня.

Проверка на практике

В этом разделе описаны несколько сценариев, использованных Google, для демонстрации различных подходов к непрерывной проверке.

Внедрение ожидаемых изменений поведения

Вы можете проверить реакцию системы на сброс и ограничение нагрузки, введя изменение поведения на сервер и проследив, все ли затронутые клиентские и серверные части реагируют как надо.

Например, Google реализует серверные библиотеки и управляющие API, позволяющие нам добавлять произвольные задержки или сбои к любому серверу RPC. Мы пользуемся этой функцией, когда будет рассматривать упражнения по готовности к авариям. Эти эксперименты могут проводиться в любое время. Используя этот подход, мы изучаем изолированные методы RPC, целые компоненты или более крупные системы и специально ищем признаки каскадных сбоев. Начиная с небольшого увеличения задержки, мы строим пошаговую функцию для имитации полного отключения. Графики мониторинга четко отражают пошаговые изменения в задержках ответов так же, как и при реальных проблемах. Сопоставляя эти временные шкалы с сигналами мониторинга от клиентов и внутренних серверов, мы можем наблюдать распространение эффекта. Если частота ошибок сильно возрастает по сравнению с шаблонами из предыдущих этапов, мы знаем, что нужно отступить, сделать паузу и проанализировать, является ли поведение неожиданным.

Важно иметь надежный механизм для быстрой и безопасной отмены введенного поведения. Если есть сбой, пусть даже причина не связана с проверкой, правильное решение — сначала прервать эксперименты, а после оценить, когда можно повторить попытку.

Используйте аварийные компоненты как часть нормальных рабочих процессов

Уверенность в том, что система с низкой зависимостью или высокой доступностью работает правильно и готова к запуску в производство, крепнет, если мы видим, что она выполняет намеченные функции. Чтобы проверить готовность, мы добавляем небольшую долю реального трафика или реальных пользователей в проверяемую систему.

Системы с высокой доступностью (а иногда и системы с низкой зависимостью) проверяются через зеркальное отображение запросов. Клиенты отправляют два одинаковых запроса: один для компонента с высокой пропускной способностью, другой — для компонента с высокой доступностью. Изменяя код клиента или внедряя серверную часть, способную дублировать один входной поток трафика в два эквивалентных¹, вы можете сравнить ответы и сообщить о различиях. Когда расхождения в ответах сильно отличаются от ожидаемых, службы мониторинга отправляют оповещения. Могут быть некоторые различия, например, если резервная система имеет более старые данные или функции. Поэтому клиент должен использовать ответ от системы с высокой производительностью. Если только не происходит ошибка или клиент явно настроен на игнорирование этой системы, что может быть в чрезвычайных ситуациях. Для отзеркаливания запросов нужно не только изменение кода на клиенте, но и возможность настроить поведение отзеркаливания. Потому такую стратегию легче развернуть на клиентских интерфейсах или серверах, чем на устройствах конечных пользователей.

Системы с низкой зависимостью (а иногда и системы с высокой доступностью) лучше подходят для проверки реальными пользователями, чем для отзеркаливания запросов. Это связано с тем, что системы с низкой зависимостью сильно отличаются от своих менее надежных аналогов функциями, протоколами и емкостями системы. В Google дежурные инженеры используют системы с низким уровнем зависимости как неотъемлемую часть своих обязанностей. Мы пользуемся этой стратегией по нескольким причинам.

- В замене дежурных участвует много инженеров, но не одновременно. Так как круг людей, участвующих в проверке, ограничен.
- Когда инженеры на связи, им может понадобиться использование аварийных путей. Хорошо отработанное использование систем с малой зависимостью сокращает время, необходимое дежурному инженеру для переключения на использование этих систем в реальной аварийной ситуации, и предотвращает риск непредвиденной неправильной настройки.

¹ Это похоже на то, что команда `tee` в Unix делает для `stdin`.

Переход дежурных инженеров к использованию только систем с низкой зависимостью может осуществляться постепенно и разными способами, в зависимости от критических бизнес-задач каждой системы.

Разделение без возможности зеркально отображать трафик

Вместо отзеркаливания запросов вы можете разделять их между непересекающимися наборами серверов. Это целесообразно, если нельзя выполнить зеркальное отображение запросов. Например, когда у вас нет контроля над клиентским кодом, но возможна балансировка нагрузки на уровне маршрутизации запросов. Поэтому запросы на разделение работают только тогда, когда альтернативные компоненты используют одинаковые протоколы, как это часто бывает с версиями компонента с высокой пропускной способностью и доступностью.

Другое применение этой стратегии — распределение трафика по множеству областей отказов. Если ваша балансировка нагрузки нацелена на одну область отказов, вы можете провести эксперименты конкретно с ней. Так как у области отказов ресурсов меньше, для атаки и получения устойчивых ответов нужно меньше нагрузки. Вы можете количественно оценить влияние вашего эксперимента, сравнив сигналы мониторинга от других областей отказов. Добавляя сброс и регулирование нагрузки, вы дополнительно повышаете качество вывода из эксперимента.

Превысить расходы, но не полагаться на «авось»

Назначенная клиентам, но не использованная квота — пустая трата ресурсов. Поэтому для максимального использования ресурсов многие сервисы подписываются на чуть больше ресурсов, чем нужно. Гибкая система отслеживает приоритеты, поэтому она может высвобождать ресурсы с более низким приоритетом для удовлетворения спроса на ресурсы с более высоким. Но вы должны проверить, может ли система действительно высвобождать эти ресурсы надежно и за приемлемое время.

У Google когда-то был сервис, которому требовалось много дискового пространства для пакетной обработки. Пользовательские сервисы имеют приоритет над пакетной обработкой, и для них выделяют значительные дисковые резервы на случай пиков использования. Мы разрешили сервису пакетной обработки использовать ненужные пользовательским сервисам диски при определенных условиях: любые диски в кластере должны быть полностью освобождены в течение X часов. Разработанная нами стратегия проверки состояла в том, чтобы периодически убирать сервис пакетной обработки из кластера, отмерять нужное

для перемещения время и исправлять любые новые проблемы, обнаруженные при каждой попытке. Это не было симуляцией. Наша проверка гарантировала, что у инженеров, которые обещали SLO за X часов, были настоящие доказательства и реальный опыт.

Эти проверки дорогие, но большинство затрат покрывается автоматизацией. С помощью балансировки нагрузки ограничиваются затраты на управление ресурсами в исходном и целевом расположениях. Если предоставление ресурсов в целом автоматизировано, как в случае с облачными сервисами, нужно только запустить сценарии или инструкции для выполнения серии запросов к автоматизации.

Для небольших сервисов или компаний стратегия периодической повторной балансировки применяется аналогично. Уверенность в предсказуемом реагировании на изменения нагрузки приложений — часть основы архитектуры ПО, которая может служить глобальной базе пользователей.

Измерение циклов замены ключей

Замена ключей проста, но на практике она может принести неприятные сюрпризы, включая полное отключение обслуживания. Проверяя работу замены ключей, вы должны искать как минимум два разных результата.

Задержка замены ключей

Время, необходимое для завершения одного цикла замены.

Подтвержденная потеря доступа

Уверенность в том, что старый ключ полностью бесполезен после замены.

Мы рекомендуем периодически заменять ключи, чтобы не терять готовность к экстренной их замене, вызванной компрометацией безопасности. Заменять ключи стоит даже тогда, когда это не нужно. Если процесс замены стоит дорого, ищите способы снизить его стоимость.

В Google мы убедились, что измерение задержки замены ключей помогает решить несколько задач.

- Вы узнаете, действительно ли каждый сервис, использующий ключ, может обновить свою конфигурацию. Возможно, сервис не был разработан для замены ключей, или все же был, но это никогда не тестировалось — или изменение нарушило то, что раньше работало.
- Вы узнаете, сколько времени нужно каждому сервису для замены ключа. Это может быть таким же стандартным действием, как изменение файла

и перезапуск сервера, или он может разрастись до масштабов постепенного развертывания во всех регионах мира.

- Вы узнаете, как динамика системы задерживает процесс замены ключей.

Измерение задержки замены ключей помогло нам сформировать реалистичное ожидание всего цикла, как в обычных, так и в чрезвычайных ситуациях. Учитывайте возможные откаты (вызванные заменой ключей или другими событиями), заморозку изменений для сервисов вне бюджета ошибок и постепенные развертывания из-за областей отказов.

Не всегда можно проверить потерю доступа, используя старый ключ. Доказать, что все экземпляры старого ключа были уничтожены, сложно. Поэтому в идеале вы можете показать, что попытка использовать старый ключ не удалась, а после можно будет уничтожить его. Если этот подход непрактичен, можете положиться на механизмы запрета списка ключей (например, SAC). При наличии основного центра сертификации и хорошего мониторинга вы можете создавать оповещения, если в ACL указаны метка или серийный номер старого ключа.

Практические советы: с чего начать

Проектирование отказоустойчивых систем — необычная задача. Она требует времени и энергии и отвлекает от другой важной работы. Тщательно обдумайте компромиссные решения в соответствии с желаемой степенью устойчивости. После из широкого спектра описанных нами опций выберите несколько или множество решений, отвечающих вашим запросам.

Приводим в порядке затрат.

1. Области отказов и контроль радиуса взлома недорогие из-за их относительно статического характера, но они предлагают значительные улучшения.
2. Сервисы с высокой доступностью — следующее наиболее экономически эффективное решение.

Рассмотрите следующие варианты.

1. Попробуйте развертывание функций снижения и регулирования нагрузки, если масштаб вашей организации или неприятие риска оправдывают вложения в активную автоматизацию для обеспечения устойчивости.
2. Оцените эффективность вашей защиты от DoS-атак (см. главу 10).
3. Если вы создаете решение с низкой зависимостью, представьте процесс или механизм, обеспечивающий постепенный переход к низкой зависимости.

Часто сложно противостоять отказу инвестировать в повышение устойчивости. Ведь лучшее доказательство устойчивости — отсутствие проблем. Эти аргументы могут помочь.

- Развертывание областей отказов и средств управления радиусом взлома возымеет длительный эффект в будущих системах. Методы изоляции могут обеспечивать хорошее разделение областей отказов в работе. После настройки они неизбежно усложняют проектирование и развертывание излишне связанных или хрупких систем.
- Регулярные упражнения и применение техник по замене ключей не только означают готовность к происшествиям в области безопасности, но и обеспечивают общую криптографическую гибкость. Например, вы будете знать, что можете обновить примитивы шифрования.
- Небольшие дополнительные затраты на развертывание экземпляров сервиса с высокой доступностью позволяют легко проверить, насколько можно улучшить доступность сервиса. Отказаться от этого тоже недорого.
- Возможности отключения и регулирования нагрузки наряду с другими подходами, описанными в разделе «Управление деградацией» выше в этой главе, удешевляют ресурсы, которые компания должна поддерживать. В итоге видимые для пользователя улучшения часто применяются к самым ценным функциям продукта.
- Контроль деградации значительно способствует скорости и эффективности первой реакции при защите от DoS-атак.
- Решения с низкой зависимостью дороже, но редко применяются на практике. Чтобы определить, стоят ли они того, узнайте, сколько времени займет установка всех зависимостей критически важных для бизнеса сервисов. Теперь вы можете сравнить затраты и решить, стоит ли инвестировать свое время во что-то другое.

Независимо от собранных вами решений по устойчивости ищите доступные способы их постоянного подтверждения. Не пытайтесь сократить затраты — от этого может пострадать эффективность. Выгода от инвестиций в проверку в том, чтобы в долгосрочной перспективе зафиксировать общую стоимость всех других инвестиций в устойчивость. Если вы автоматизируете эти методы, команда разработчиков и служба поддержки смогут работать над предоставлением новых преимуществ. Затраты на автоматизацию и мониторинг в идеале будут поддерживаться другими усилиями и продуктами, нужными вашей компании.

Возможно, вам часто не хватает денег или времени, которые можно инвестировать в устойчивость. В следующий раз, когда вы сможете потратить больше этих ограниченных ресурсов, обратите внимание на оптимизацию затрат на уже развернутые механизмы устойчивости. Если вы уверены в их качестве и эффективности, рассмотрите дополнительные варианты.

Резюме

В этой главе мы обсудили разные способы обеспечить стабильность безопасности и надежности системы, начиная с этапа проектирования. Важный выбор для многих характеристик должен делать именно человек. Мы можем оптимизировать некоторые варианты с помощью автоматизации, но для других все еще нужны люди.

Стабильность характеристик надежности помогает сохранить самые важные функциональные возможности системы. Поэтому она в состоянии выдерживать чрезмерную нагрузку или значительные сбои. Если система выходит из строя, ее функционал позволяет увеличить время, нужное для организации, предотвращения большего ущерба или ручного восстановления. Устойчивость помогает системам противостоять атакам и защищает от попыток получить долгосрочный доступ. Если злоумышленник проникнет в систему, благодаря ее конструктивным особенностям, таким как контроль радиуса взлома, можно будет ограничить урон.

Обоснуйте свои стратегии проектирования при защите в глубину. Проверяйте безопасность системы так же, как и время безотказной работы и надежности. Защита в глубину подобна избыточности $N + 1$. Вы не доверяете всю пропускную способность своей сети одному маршрутизатору или коммутатору, так зачем доверять одному межсетевому экрану или другим средствам защиты? Проектируя защиту в глубину, всегда принимайте и проверяйте сбои на разных уровнях безопасности. Это могут быть сбои в защите по внешнему периметру, компрометация конечной точки, инсайдерская атака и т. д. Планируйте горизонтальные перемещения для их остановки.

Даже если вы разрабатываете свои системы для обеспечения устойчивости, вполне возможно, что в какой-то момент устойчивость упадет и ваша система сломается. В следующей главе обсуждается, что происходит *после* этого: как восстановить поврежденные системы и как минимизировать ущерб, вызванный поломками.

ГЛАВА 9

Проектирование для восстановления

*Аарон Джойнер, Джон Маккьюн
и Виталий Шипицын
с Константиносом Неофиту,
Джесси Янгом и Кристиной Беннетт*

У всех сложных систем есть проблемы с надежностью, и преступники пытаются заставить их выйти из строя или иначе отклониться от их предполагаемой функции. На ранних этапах разработки продукта вы должны предвидеть такие сбои и планировать неизбежный процесс восстановления.

В этой главе мы обсуждаем такие стратегии, как использование ограничения скорости и нумерация версий. Мы подробно рассмотрим такие компромиссы, как откат и использование механизмов отзыва, способных вернуть систему в рабочее состояние, но при этом вновь добавить уязвимости в системе безопасности. Независимо от выбора подхода к восстановлению важно знать, в какое состояние должна вернуться система и что делать при невозможности получения доступа к этому известному исправному состоянию с помощью существующих методов.

Современные распределенные системы подвержены множеству типов сбоев, которые являются результатом как непреднамеренных ошибок, так и преднамеренных действий злоумышленника. В случае накопления ошибок, редких сбоев или злонамеренных действий преступников люди должны вмешаться для восстановления даже самых безопасных и отказоустойчивых систем.

Восстановить отказавшую или скомпрометированную систему в стабильное и безопасное состояние можно сложными непредвиденными способами. Например, откат нестабильного выпуска может привести к появлению уязвимостей

безопасности. Выпуск новой версии для исправления уязвимости безопасности приведет к проблемам с надежностью. Подобные рискованные меры по смягчению урона влекут за собой принятие более тонких компромиссов. К примеру, если принимать решение о скорости внедрения изменений, быстрое развертывание с большей вероятностью позволит опередить действия злоумышленников, но и ограничит количество тестов, которые вы можете выполнить. В результате вы можете развернуть новый код с критическими ошибками.

Нехорошо начинать учитывать эти тонкости — и неготовность вашей системы справиться с ними — во время стрессового инцидента, связанного с безопасностью или надежностью. Только осознанные проектные решения могут сделать вашу систему гибкой и надежной, что необходимо для естественной поддержки разных потребностей в восстановлении. В этой главе рассматриваются принципы проектирования, которые мы сочли эффективными при подготовке наших систем и облегчении восстановления. Многие из них применяются в разных масштабах, от систем планетарного уровня до сред прошивки на отдельных устройствах.

От чего мы восстанавливаемся?

До того, как мы рассмотрим стратегии проектирования по облегчению восстановления, разберем некоторые сценарии, приводящие к необходимости восстановления системы. Они делятся на несколько основных категорий: случайные ошибки, непреднамеренные ошибки, вредоносные действия и программные ошибки.

Случайные ошибки

Все распределенные системы построены на базе физического оборудования, и все физическое оборудование дает сбой. Ненадежный характер физических устройств и непредсказуемая физическая среда, в которой они работают, приводят к случайным ошибкам. Чем больше физического оборудования, поддерживающего систему, тем вероятнее, что в распределенной системе появятся случайные ошибки. Устаревание оборудования тоже приводит к большему количеству ошибок.

Одни случайные ошибки легче исправить, чем другие. Полный сбой или изоляция некоторой части системы, такой как источник питания или важный сетевой маршрутизатор, является одним из самых простых сбоев, с которыми

нужно справиться¹. Сложнее бороться с кратковременным повреждением, вызванным неожиданным переключением битов². Непросто справиться и с долгосрочным повреждением, вызванным ошибками в инструкции на одном ядре многоядерного процессора. Эти ошибки особенно коварны, если происходят незаметно.

Непредсказуемые события вне системы тоже могут стать причиной ошибки в современных цифровых средах. Торнадо или землетрясение способны привести к внезапной и постоянной потере определенной части системы. Отказ электростанции или подстанции, неисправность ИБП или батареи могут поставить под угрозу подачу электроэнергии на устройства. Это может привести к падению или скачкам напряжения, что повлечет за собой повреждение памяти или другие временные ошибки.

Непреднамеренные ошибки

Все распределенные системы прямо или косвенно управляются людьми, и все люди ошибаются. *Непреднамеренные ошибки* — ошибки, совершенные людьми без злого умысла. Вероятность человеческих ошибок зависит от типа задачи. Грубо говоря, чем сложнее задача, тем чаще происходят ошибки³. Внутренний анализ простоев в работе Google с 2015 по 2018 год показал, что значительная часть отключений (хотя и не большинства) была вызвана односторонними действиями человека, не подлежавшими технической или процедурной проверке безопасности.

Люди могут совершать ошибки в отношении любой части вашей системы. Поэтому учитывайте возможность возникновения человеческих ошибок во всем стеке инструментов, систем и рабочих процессов в жизненном цикле системы. Случайные ошибки могут повлиять на вашу систему случайным, внешним по отношению к ней образом. Например, если экскаватор, используемый для не связанных с системой работ, повредит волоконно-оптический кабель.

¹ Теорема Брюэра (https://oreil.ly/WP_FI) описывает некоторые компромиссы, связанные с масштабированием распределенных систем, и их последствия.

² Неожиданное переключение битов может быть вызвано отказом оборудования, помехами других систем или даже космическими лучами. В главе 15 сбои оборудования рассматриваются более подробно.

³ Целая область исследований, известная как анализ надежности человека (Human Reliability Analysis, HRA), регистрирует возможность человеческих ошибок при выполнении определенной задачи. Для получения дополнительной информации см. вероятностную оценку риска (<https://oreil.ly/fGTNa>) Комиссии по регулированию ядерной энергетики США.

Ошибки программного обеспечения

Вспомните обсуждаемые нами ранее типы ошибок с изменениями дизайна и/или ПО. Перефразируя классическую цитату¹ и ее следствие², все ошибки могут быть решены с помощью программного обеспечения ... за исключением ошибок в нем. Программные ошибки — это вид ошибок, допущенных при разработке ПО. Это особый отложенный случай непреднамеренных ошибок. В вашем коде будут ошибки, и вам нужно будет их исправить. Некоторые базовые и хорошо описанные принципы проектирования — например, модульное проектирование ПО, тестирование, проверка кода, входных и выходных данных зависимых API — помогают устранить ошибки. В главах 6 и 12 эти темы рассматриваются подробнее.

Иногда программные ошибки похожи на другие типы ошибок. Например, автоматизация без проверки безопасности может внести внезапные и кардинальные изменения в производство, имитируя действия злоумышленника. Ошибки ПО увеличивают и другие типы ошибок. Например, ошибки датчиков, возвращающих неожиданные значения, которые ПО не может обработать как положено, или неожиданное поведение, похожее на злонамеренную атаку, когда пользователи обходят неисправный механизм в процессе обычной работы.

Вредоносные действия

Люди могут сознательно действовать против ваших систем. Они могут быть привилегированными и хорошо осведомленными инсайдерами с плохими намерениями. Злонамеренные субъекты — целая категория людей, активно работающих над нарушением контроля безопасности и надежности вашей системы (систем) или имитирующих случайные, непреднамеренные или другие виды ошибок. Автоматизация может уменьшить, но не устранить необходимость участия человека. По мере масштабирования и увеличения сложности вашей распределенной системы размер обслуживающей ее организации должен масштабироваться вместе с ней (в идеале, сублинейным образом). В то же время вероятность того, что кто-то из членов этой организации нарушит оказанное им доверие, тоже возрастает.

Эти нарушения доверия могут исходить от инсайдера, злоупотребляющего своими законными полномочиями в системе. К примеру, читая пользовательские

¹ «Все проблемы в информатике могут быть решены с помощью другого уровня косвенного обращения». Дэвид Уилер.

² «...за исключением проблемы слишком большого количества уровней косвенного обращения». Неизвестный автор.

данные, не относящиеся к его работе, разглашая или раскрывая секреты компании, или даже активно работая, чтобы вызвать длительное отключение. У человека может быть недостаточно времени для принятия верного решения, или же он может искренне желать причинить вред, он может стать и жертвой атаки социальной инженерии и даже подвергнуться принуждению извне (<https://xkcd.com/538>).

Сторонняя компрометация системы тоже может привести к злонамеренным ошибкам. В главе 2 подробно описывается круг возможных злоумышленников. Что касается проектирования системы, то стратегии смягчения последствий одинаковы независимо от того, будет ли злоумышленник инсайдером или посторонним, компрометирующим системные учетные данные.

Принципы проектирования для восстановления

В следующих разделах можно увидеть принципы проектирования для восстановления на основе нашего многолетнего опыта работы с распределенными системами. Список неполный — мы дадим рекомендации для дальнейшего изучения. Эти принципы применимы ко всем организациям, а не только к предприятиям масштаба Google. В целом при разработке восстановления важно не иметь предубеждений в отношении широты и разнообразия потенциальных проблем. Другими словами, не тратьте время на беспокойство о том, как классифицировать нюансы пограничных случаев ошибок, а сосредоточьтесь на готовности оправиться от них.

Проектирование должно быть максимально быстрым (в соответствии с политикой)

Восстанавливая систему до рабочего состояния после ее компрометации или сбоя, вы будете испытывать большое давление. Но механизмы, используемые вами для быстрых изменений в системах, сами имеют риск неправильных и слишком быстрых изменений, что усугубляя проблему. Если ваши системы злонамеренно скомпрометированы, не спешите предпринимать действия по восстановлению или очистке, ведь они могут вызвать другие проблемы. Например, так злоумышленник может узнать о том, что он был обнаружен¹. Мы выявили несколько подходов, эффективных для выбора компромиссов,

¹ Подробное обсуждение вопроса реагирования при компрометации и основные проблемы определения компрометации систем восстановления см. в главе 18. В главе 7 приводятся дополнительные проектные решения и примеры, описывающие, как выбрать соответствующий уровень.

связанных с проектированием систем для поддержки переменных скоростей восстановления.

Для восстановления системы после любого из описанных четырех классов ошибок или во избежание необходимости восстановления нужно иметь возможность изменить состояние системы. Создавая механизм обновления (например, процесс развертывания ПО/прошивки, процедур управления изменениями настроек или сервисов пакетного планирования), проектируйте систему обновлений так, чтобы она работала настолько быстро, насколько это может понадобиться (или еще быстрее, в пределах практичности). После добавьте средства контроля для ограничения скорости изменения в соответствии с текущей политикой для рисков и нарушений. Есть несколько преимуществ, позволяющих отделить вашу способность выполнять развертывание от политики уровней и частот для него.

Потребности и политики развертывания в любой организации со временем меняются. Например, сначала компания могла выполнять развертывание ежемесячно, а не по вечерам или выходным. Если система развертывания ориентирована на изменение политики, оно может повлечь за собой сложный рефакторинг и последующие изменения кода. Если проект системы развертывания вместо этого четко отделяет время и скорость изменения от его действия и содержания, гораздо легче приспособиться к неизбежным изменениям политики, управляющим временем и скоростью изменения.

Может случиться так, что полученная в середине развертывания новая информация повлияет на ваш ответ. Представьте, что в ответ на обнаруженную внутреннюю уязвимость безопасности вы развертываете внутренне разработанное исправление. Обычно не нужно развертывать такое изменение достаточно быстро, чтобы не повышать риск дестабилизации сервиса. Но ваш расчет риска может измениться в ответ на изменение ландшафта (см. главу 7). Если в середине развертывания вы обнаружите, что уязвимость стала общеизвестной и теперь активно используется, вы можете ускорить процедуру.

Рано или поздно внезапное или неожиданное событие изменит уровень риск, на который вы готовы пойти. В результате может понадобиться вывести изменение очень быстро. Примеры могут меняться от ошибок безопасности (ShellShock¹, Heartbleed² и т. д.) до обнаружения активной компрометации. *Проектируйте вашу систему аварийной отправки изменений так, чтобы это была ваша обычная система отправки, но включенная на максимум.* Это означает, что

¹ См. CVE-2014-6271 (<https://oreil.ly/mRhGN>), CVE-2014-6277 (<https://oreil.ly/yyf6K>), CVE-2014-6278 (<https://oreil.ly/7ii2u>) и CVE-2014-7169 (<https://oreil.ly/0ZD04>).

² См. CVE-2014-0160 (<https://oreil.ly/cJTQ8>).

ваша обычная система развертывания и система аварийного отката — это одно и то же. Мы часто говорим, что не проверенные заранее экстренные практики могут не сработать в нужный момент. Включение обычной системы в режим обработки чрезвычайных ситуаций означает, что вам не нужно обслуживать две отдельные системы отправки изменений и что система аварийного выпуска часто используется¹.

Если для реагирования на стрессовую ситуацию нужно только изменить ограничения скорости для быстрого внесения изменения, у вас будет гораздо больше уверенности в том, что инструмент развертывания сработает как нужно. Теперь вы можете сосредоточиться на других неизбежных рисках, таких как потенциальные ошибки в быстро развернутых изменениях, или на устранении уязвимостей, которые злоумышленники могли использовать для доступа к вашим системам.

ИЗОЛЯЦИЯ МЕХАНИЗМА ОГРАНИЧЕНИЯ СКОРОСТИ ДЛЯ ПОВЫШЕНИЯ НАДЕЖНОСТИ

Как защитить вашу систему от нежелательного быстрого выполнения, если она *может* очень быстро разворачивать изменения? Одна из стратегий — создать механизм ограничения скорости как независимый, автономный, одноцелевой микросервис, ограничивающий скорость изменения конкретной системы или систем. Сделайте этот микросервис максимально простым и поддающимся строгому тестированию. Например, ограничивающий скорость микросервис может предоставить криптографический токен с коротким сроком службы, подтверждающий, что микросервис проверил и одобрил конкретное изменение в определенное время.

Сервис ограничения скорости может быть и отличной точкой для сбора журналов изменений в целях аудита. Хотя дополнительная сервисная зависимость добавляет еще одну движущую часть к полной системе развертывания, мы приходим к компромиссу: разделение явно сообщает, что активация изменения должна быть отделена от его скорости (для безопасности). Так инженеры не смогут обходить сервис ограничения скорости, потому что другой код этого ограничения, предлагающий избыточную функциональность, скорее всего, будет явным признаком небезопасного дизайна.

Мы усвоили эти уроки по мере развития нашего внутреннего дистрибутива Linux. До недавних пор Google устанавливал все устройства в центрах обработки данных с «базовым», или «золотым», образом, содержащим известный набор статических файлов. Для каждой машины было несколько конкретных настроек, таких как имя хоста, конфигурация сети и учетные данные. Наша политика заключалась в том, чтобы каждый месяц развертывать новый «базовый» образ на всех устройствах. В течение нескольких лет мы создавали набор инстру-

¹ Следствие этого принципа: если у вас есть рабочая методология для чрезвычайной ситуации, сделайте ее стандартной.

ментов и систему обновления ПО для этой политики и рабочего процесса. Мы собирали все файлы в сжатый архив, набор изменений проверялся старшим SR-инженером, а устройства постепенно обновлялись для принятия нового образа.

Мы разработали наши инструменты развертывания в соответствии с этой политикой и спроектировали их для сопоставления определенного базового образа с набором устройств. Мы также представили язык конфигурации, чтобы выразить, как изменить этот образ в течение нескольких недель, а после использовали несколько механизмов для наложения исключений поверх базового образа. Одно исключение содержало исправления безопасности для растущего числа отдельных пакетов ПО. Так как список исключений становился все сложнее, не было большого смысла в том, чтобы инструменты следовали ежемесячной схеме.

В итоге мы решили отказаться от предположения о ежемесячном обновлении базового образа. Мы разработали более детализированные блоки выпуска, соответствующие каждому программному пакету. Еще мы создали новый чистый API, указывающий точный набор пакетов для установки, по одному устройству за раз, поверх существующего механизма развертывания. Как показано на рис. 9.1, этот API отделил ПО, определяющее несколько различных аспектов.

- Развертывание и скорость, с которой должен был меняться каждый пакет.
- Хранилище настроек, которое определило текущие настройки всех устройств.
- Исполнительный механизм развертывания, управляющий применением обновления для каждого устройства.

В итоге мы могли разрабатывать каждый аспект независимо. Затем мы использовали существующее хранилище настроек для определения конфигурации всех пакетов, применяемых к каждому устройству, и создали систему развертывания для отслеживания и обновления независимых развертываний каждого пакета.

Отделив сборку образа от ежемесячной политики развертывания, мы могли обеспечить более широкий диапазон скоростей выпуска для разных пакетов. Но, хотя стабильные и последовательные развертывания на большинстве устройств еще сохранялись, некоторые тестовые устройства могли следовать последним сборкам всего ПО. Более того, разделение политик открыло новые возможности использования всей системы. Теперь мы используем его для регулярного распространения подмножества тщательно проверенных файлов по всем устройствам. Мы можем использовать наши обычные инструменты для аварийных выпусков, просто отрегулировав некоторые ограничения скорости и одобрив выпуск одного типа пакета для ускорения его работы. Результат стал проще, полезнее и безопаснее.

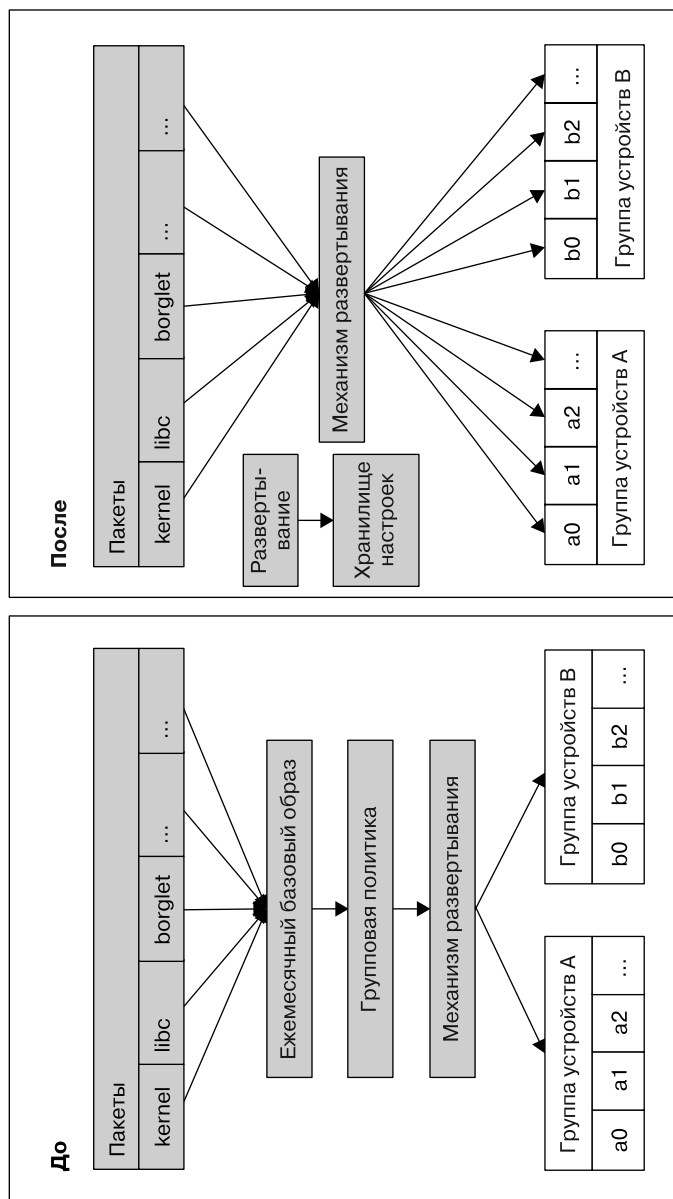


Рис. 9.1. Эволюция нашего рабочего процесса развертывания пакетов на устройствах

Ограничение зависимости от внешних представлений о времени

Обычное время суток, которое можно узнать, глядя на часы, является формой состояния. Вы не можете изменить то, как ваша система проходит через какое-то время. Поэтому в любом месте, где используется системное время, возможны проблемы с выполнением восстановления. Из-за несоответствия между временем восстановления и временем последней нормальной работы системы она может повести себя неожиданно. Например, восстановление, включающее в себя воспроизведение транзакций с цифровой подписью, может завершиться ошибкой. Такое происходит, если некоторые транзакции подписаны сертификатами с истекшим сроком действия. Чтобы это предотвратить, спроектируйте процесс восстановления, учитывая исходную дату транзакции при проверке сертификатов.

Зависимость вашей системы от времени, которое вы не можете контролировать, может привести к проблемам безопасности или надежности. К этой категории можно отнести множество типов ошибок: программные, такие как Y2K (<https://oreil.ly/zV9E0>), переход на новую эпоху Unix (https://oreil.ly/heY_0), или непреднамеренные, когда разработчики выбирают время истечения срока действия сертификата в будущем, чтобы это «больше не было их проблемой». Открытые или неаутентифицированные NTP-соединения (<https://oreil.ly/9IG8s>) тоже небезопасны, если злоумышленник может контролировать сеть. Фиксированное смещение даты или времени в коде демонстрирует «запах» кода (<https://oreil.ly/zxfz2>), что указывает на возможность создания бомбы замедленного действия.



Связывание событий с системным временем часто может быть антипаттерном. Вместо него мы рекомендуем использовать что-то из следующего.

- Уровни.
- Вручную добавленные представления о прогрессе, такие как номера периодов или версий.
- Списки достоверности.

Как упоминалось в главе 8, система безопасности транспорта ALTS от Google не использует сроки годности в своих цифровых сертификатах. Вместо этого она полагается на систему аннулирования. Активный список аннулирования состоит из массивов данных, определяющих диапазоны действительных и аннулированных серийных номеров сертификатов. Он работает независимо от системного времени. Для достижения целей изоляции вы можете периодически обновлять списки аннулирования для создания временных интервалов. Вы можете экстренно создать новый список аннулирования, чтобы отозвать сертификаты, если подозреваете, что у злоумышленника есть доступ к базовым

ключам. Вы также можете остановить периодические отправки изменений в необычных обстоятельствах, чтобы начать отладку или экспертизу. См. подраздел «Использование явного механизма отката» далее в этой главе, чтобы подробнее в этом разобраться.

Выбор зависящего от системного времени решения тоже может привести к появлению слабых мест в безопасности. Из-за ограничений надежности вы можете захотеть отключить проверку достоверности сертификата для выполнения восстановления. Но в этом случае лечение хуже болезни — лучше пропустить срок действия сертификата (пару SSH-ключей, разрешающих вход в систему для доступа к кластеру серверов), чем проверку достоверности. Одно примечательное исключение: системное время *подойдет* для преднамеренно ограниченного доступа. Например, вы можете потребовать ежедневного прохождения повторной аутентификации большинством сотрудников. В таких случаях важно знать, как исправить систему без зависимости от системного времени.

Использование абсолютного времени может привести к проблемам во время восстановления после сбоев или при попытках восстановления БД с монотонно возрастающим временем восстановления после повреждения. Этот процесс может потребовать исчерпывающего воспроизведения транзакций (что быстро становится невозможным по мере роста наборов данных) или попытки скоординированного отката времени в нескольких системах. Более простой пример: сопоставление журналов в системах с неточными представлениями о времени обременяет инженеров ненужной работой. Так случайные ошибки становятся более распространенными.

Вы можете устранить зависимости от системного времени с помощью эпох или версий. Для этого нужно, чтобы все части системы координировались вокруг целочисленного значения, представляющего прямую последовательность «действительных» элементов против «истекших». Эпохой может быть целое число, хранящееся в компоненте распределенных систем (блокировки или локальное состояние устройства), которое постепенно увеличивается (допускается перемещение только вперед) в соответствии с политикой. Чтобы увеличить для систем скорость отправки выпусков, вы можете спроектировать эти выпуски так, чтобы обеспечить быстрое увеличение эпохи. Один сервис может отвечать за объявление текущей эпохи или инициирование ее изменения. Вы можете остановить изменение эпохи в случае неприятностей, пока не обнаружите и не исправите проблему. Вернемся к нашему более раннему примеру с открытым ключом. Хотя сертификаты могут устареть, у вас не будет соблазна полностью отключить их проверку, поскольку вы можете остановить продвижение по эпохе. Эпохи имеют некоторые сходства со схемой MASVN, обсуждаемой в подразделе «Минимально допустимые номера версий безопасности» далее в этой главе.



Сильно увеличивающиеся значения эпох могут выйти за пределы допустимых значений. Будьте осторожны с тем, насколько быстро ваша система внедряет изменения, и тем, сколько промежуточных значений эпох или версий вы можете пропустить.

Злоумышленник с временным контролем над вашей системой может нанести длительный ущерб, резко ускорив продвижение по эпохе. Популярное решение этой проблемы — выбор значения эпохи с большим диапазоном и задание базового предела скорости, например 64-разрядного целочисленного значения, ограниченного приращением не более одного раза в секунду. Жесткое кодирование ограничения скорости — исключение из нашей предыдущей рекомендации по разработке — разворачивать изменения как можно быстрее и добавлять политику для определения скорости. Но в этом случае трудно представить причину, по которой состояние системы меняется чаще раза в секунду, так как вы будете иметь дело с миллиардами лет. Эта стратегия имеет смысл и потому, что 64-разрядное целое число обычно не требует больших затрат памяти на современном оборудовании.

Даже в случаях, когда желательно ожидать истечения системного времени, попробуйте просто измерить истекшее время, не сверяясь с фактическим. Ограничение скорости будет работать, даже если система не знает о фактическом времени.



Откаты — компромисс между безопасностью и надежностью

Первый шаг к восстановлению во время реагирования на происшествие — смягчение последствий инцидента, обычно путем безопасного отката любых подозрительных изменений. Большая часть производственных проблем, требующих вмешательства человека, возникает самостоятельно (см. подразделы «Непреднамеренные ошибки» и «Ошибки программного обеспечения» в начале этой главы). Это значит, что предполагаемое изменение в системе содержит ошибку или неверную конфигурацию, которая служит причиной происшествия. Во время таких происшествий нужно как можно быстрее и безопаснее вернуть системы к последнему известному исправному состоянию.

В других случаях следует *предотвратить* откат. При исправлении уязвимостей безопасности вы часто соревнуетесь со злоумышленниками, пытаясь установить патч до того, как они воспользуются этой уязвимостью. После успешного развертывания патча и подтверждения его стабильности вам нужно предотвратить применение отката злоумышленниками. Они повторно вводят уязвимость, оставляя при этом возможность добровольного отката. Поскольку сами патчи

безопасности тоже изменяют код, они могут содержать собственные ошибки или уязвимости.

В свете этих соображений может быть сложно определить подходящие для отката условия. ПО прикладного уровня — более простой пример. Системное ПО, такое как ОС или служебный процесс управления привилегированными пакетами, может легко завершать и перезапускать задачи или процессы. Вы можете собирать имена нежелательных версий (обычно уникальные строки меток, числа или хеши¹) в список запретов, который потом можно включить в политики выпуска системы развертывания. Еще вы можете управлять списком разрешений и создавать свою автоматизацию, чтобы включить в этот список развернутое прикладное ПО.

Сложнее будет с привилегированными или низкоуровневыми системными компонентами, отвечающими за обработку собственных обновлений. Мы называем эти компоненты *самообновляющимися*. Например, служебный процесс управления пакетами, который обновляет сам себя, перезаписывая собственный исполняемый файл и запуская его заново. Еще пример: образ прошивки, такой как BIOS, который перепрошивает новый образ поверх самого себя, а затем принудительно перезагружается. Если эти компоненты злонамеренно изменены, они могут активно препятствовать самообновлению. Требования к реализации, зависящие от аппаратного обеспечения, усложняют задачу. Вам нужны механизмы контроля отката, работающие даже для этих компонентов, но само предполагаемое поведение может быть сложно определить. Чтобы лучше оценить проблему, рассмотрим два примера политик и их недостатки.

Разрешать произвольные откаты

Это решение небезопасно, так как любой фактор, побуждающий вас выполнить откат, может снова привести к известной уязвимости безопасности. Чем старше или заметнее уязвимость, тем больше вероятность того, что стабильные способы ее повторного добавления будут легкодоступны.

Никогда не разрешать откат

Такое решение исключает путь возврата к известному стабильному состоянию и позволяет только переходить к более новым. Это ненадежно, потому что, если в обновлении есть ошибка, вы больше не можете откатиться до последней известной исправной версии. Этот подход неявно требует, чтобы система сборки генерировала новые версии, к которым можно переходить, добавляя время и необязательные зависимости в инфраструктуру разработки и выпуска.

¹ Например, криптографический хеш (такой как SHA256) полного образа программы или прошивки.

У этих двух крайних подходов есть альтернативы, предлагающие практические компромиссы. К ним относятся:

- использование списков запретов;
- использование номеров версий безопасности (Security Version Number, SVN) и минимально допустимых номеров версий безопасности (Minimum Acceptable Security Version Number, MASVN);
- замена ключей подписи.

В следующем обсуждении мы предполагаем, что во всех случаях у обновлений есть криптографическая подпись.

Списки запретов

По мере обнаружения ошибок и уязвимостей в версиях выпуска вы можете создать список запретов для предотвращения повторной активации известных плохих версий, жестко кодируя список запретов в самом компоненте. В следующем примере мы добавляем его как `Release[DenyList]`. После обновления компонента до недавно выпущенной версии он отказывается от обновления до версии, не включенной в список:

```
def IsUpdateAllowed(self, Release) -> bool:  
    return Release[Version] not in self[DenyList]
```

К сожалению, это решение подходит только для непреднамеренных ошибок. Потому что жестко запрограммированные списки запретов представляют неразрешимый компромисс между безопасностью и надежностью. Если список запретов всегда оставляет возможность для отката хотя бы к одному более старому, заведомо хорошо известному образу, схема уязвима для *распаковки*. Преступник может постепенно откатывать версии, пока не получит более старую, содержащую известную уязвимость, которую он может использовать. Этот сценарий сводится к экстремальному «разрешению произвольных откатов», описанному ранее, с промежуточными переходами.

С другой стороны, настройка списка запретов для полного предотвращения откатов критических обновлений безопасности приводит к экстремальному состоянию «никогда не разрешать откат» с сопутствующими ошибками надежности.

Если вы восстанавливаетесь после происшествия, связанного с безопасностью или надежностью, когда несколько устройств могут обновляться одновременно, жестко запрограммированные списки запретов — хороший выбор для настройки системы и избегания непреднамеренных ошибок. Добавить одну версию в список — быстро и просто, так как почти не влияет на достоверность других версий.

Но для противостояния злонамеренным атакам вам нужна более надежная стратегия.

Лучшее решение при использовании списка запретов — кодировать его вне самообновляющегося компонента. В следующем примере мы используем для этого `ComponentState[DenyList]`. Этот список запретов сохраняется во всех обновлениях и версиях компонентов. Так происходит потому, что он не зависит от какого-либо отдельного выпуска, но компоненту все еще нужна логика для поддержки списка запретов.

Каждый выпуск может разумно кодировать наиболее полный список запретов, известный на момент выпуска: `Release[DenyList]`. Потом логика сервиса объединяет эти списки и сохраняет их локально (обратите внимание, что мы пишем `self[DenyList]` вместо `Release[DenyList]`, чтобы указать, что `self` установлен и активно работает):

```
ComponentState[DenyList] = ComponentState[DenyList].union(self[DenyList])
```

Проверьте предварительные обновления на достоверность по списку и откажитесь от обновлений из списка (не ссылайтесь явно на `self`, ведь его вклад в список запретов уже отражен в `ComponentState`, где он остается даже после установки следующих версий):

```
def IsUpdateAllowed(self, Release, ComponentState) -> bool:
    return Release[Version] not in ComponentState[DenyList]
```

Теперь вы можете сознательно решить компромисс между безопасностью и надежностью в соответствии с политикой. Когда вы определяете, что включать в `Release[DenyList]`, взвесьте риски атак распаковки и нестабильного выпуска.

У этого подхода есть и недостатки даже при кодировании списков запретов в структуре данных `ComponentState`, которая поддерживается вне самообновляющегося компонента.

- Даже если список запретов вне назначения вашей централизованной системы развертывания, вы все равно должны отслеживать и учитывать его.
- Если запись добавлена в список запретов случайно, вы можете удалить ее оттуда. Но добавление возможности удаления может повлечь риск атаки распаковки.
- Список запретов может беспрепятственно увеличиваться, что в итоге приведет к ограничению размера хранилища. Как вы управляете сборкой мусора (https://oreil.ly/_TsJS) в списке запретов?

Минимально допустимые номера версий безопасности

По мере добавления новых записей списки запретов становятся большими и громоздкими. Вы можете использовать отдельный номер версии безопасности, записанный как `Release[SVN]`, для удаления более старых записей из списка запретов, но помешать их установке. В результате вы уменьшаете когнитивную нагрузку на людей, ответственных за систему.

Сохранение `Release[SVN]` независимо от других обозначений версии позволяет логически следить за множеством выпусков, не требуя дополнительного пространства для списка запретов. Всякий раз, когда вы применяете критическое исправление безопасности и демонстрируете его стабильность, увеличивайте `Release[SVN]`, отмечая этап безопасности, используемый для регулирования решений об откате. Простой индикатор состояния безопасности каждой версии означает гибкость для проведения обычного тестирования и квалификации выпуска. Также это означает уверенность в том, что вы сможете быстро и безопасно принимать решения об откате во время обнаружения ошибок или проблем со стабильностью.

Помните о необходимости помешать злоумышленникам как-то откатить ваши системы до известных сломанных или уязвимых версий¹. Не позволяйте этим людям закрепиться в вашей инфраструктуре и использовать эту точку опоры для предотвращения восстановления. Вы можете использовать `MASVN` для определения отметки, ниже которой ваши системы никогда не должны работать². Это должно быть упорядоченное значение (не криптографический хеш), лучше простое целое число. Вы можете управлять `MASVN` по аналогии со списком запретов.

- Каждая версия выпуска содержит значение `MASVN`, отражающее допустимые версии на момент выпуска.
- Вы поддерживаете глобальное значение вне системы развертывания, записанное как `ComponentState[MASVN]`.

В качестве предварительного условия для применения обновления все выпуски содержат логику, проверяющую, что `Release[SVN]` пробного обновления по

¹ «Известные сломанные» версии могут быть результатом успешных, но необратимых изменений (например, существенного изменения схемы) или ошибок и уязвимостей.

² Распространенный пример аккуратно управляемых номеров версий безопасности есть в микрокоде Intel SVN, который используется для смягчения проблемы безопасности CVE-2018-3615 (<https://oreil.ly/fN9f3>).

крайней мере так же высок, как `ComponentState[MASVN]`. С помощью псевдокода это можно выразить так:

```
def IsUpdateAllowed(self, Release, ComponentState) -> bool:
    return Release[SVN] >= ComponentState[MASVN]
```

Операция обслуживания для глобального `ComponentState[MASVN]` не является частью процесса развертывания. Вместо этого оно происходит при первой инициализации новой версии. В каждом выпуске вы жестко кодируете целевой `MASVN` — `MASVN`, который нужно применить для этого компонента во время создания выпуска, записанного как `Release[MASVN]`.

Когда новый выпуск развертывается и выполняется впервые, он сравнивает свой `Release[MASVN]` (записанный как `self[MASVN]`, чтобы ссылаться на установленный и работающий выпуск) с `ComponentState[MASVN]`. Если `Release[MASVN]` выше, чем существующий `ComponentState[MASVN]`, то `ComponentState[MASVN]` обновляется до нового большего значения. Фактически эта логика запускается каждый раз при инициализации компонента, но `ComponentState[MASVN]` изменяется только после успешного увеличения `Release[MASVN]`. С помощью псевдокода это можно выразить следующим образом:

```
ComponentState[MASVN] = max(self[MASVN], ComponentState[MASVN])
```

Эта схема может эмулировать любую из упомянутых ранее крайних политик:

- разрешать произвольный откат — никогда не изменяйте `Release[MASVN]`;
- никогда не разрешать откат — измените `Release[MASVN]` на шаг с `Release[SVN]`.

На практике `Release[MASVN]` часто возникает в выпуске $i + 1$, после выпуска, устраняющего проблему безопасности. Это гарантирует, что $i - 1$ или более старые версии никогда больше не будут использоваться. Поскольку `ComponentState[MASVN]` является внешним по отношению к самой версии, версия i больше не допускает понижения до $i - 1$ после установки $i + 1$, даже несмотря на то, что она позволяла это при первоначальном развертывании.

На рис. 9.2 показаны примеры значений для последовательности из трех выпусков и их влияние на `ComponentState[MASVN]`.

Для уменьшения уязвимости безопасности в выпуске $i - 1$ выпуск i содержит исправление безопасности и инкрементный `Release[SVN]`. `Release[MASVN]` не изменяется в выпуске i , потому что даже исправления безопасности могут иметь ошибки. После того, как релиз i станет стабильным в производственном процессе, следующий релиз $i + 1$ увеличивает `MASVN`. Это означает, что исправление безопасности теперь обязательно и выпуски без него запрещены.

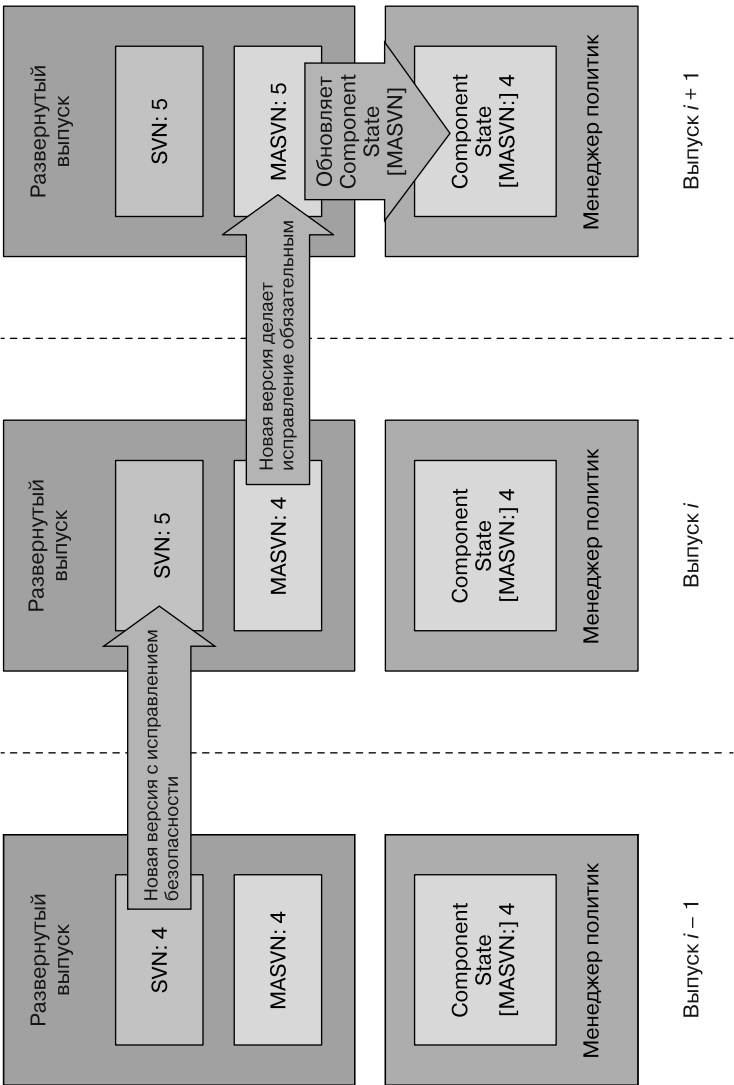


Рис. 9.2. Последовательность из трех выпусков и их влияние на ComponentState [MASVN]

В соответствии с принципом разработки «как можно быстрее» схема MASVN отделяет политику для разумных целей отката от выполняющей откаты инфраструктуры. Технически можно ввести определенный API в компоненте самообновления и получить команду от централизованной системы управления развертыванием для увеличения `ComponentState[MASVN]`. С помощью этой команды вы можете вызвать `ComponentState[MASVN]` для компонентов, получающих обновление в конце процесса развертывания, после проверки выпуска на достаточном количестве устройств для уверенности в том, что он будет работать как запланировано. Подобный API будет полезен при реагировании на активную компрометацию или особенно серьезную уязвимость, где скорость критична, а рискоустойчивость для проблем доступности выше обычного.

До сих пор в этом примере не использовался специальный API для изменения `ComponentState`. `ComponentState` — это набор значений, влияющий на вашу способность восстанавливать системы с помощью обновлений или откатов. Он локален для компонента и внешний по отношению к централизованной части автоматизации. Фактическая последовательность версий ПО / прошивки, используемая каждым отдельным компонентом, может изменяться в зависимости от набора похожих или идентичных устройств, в условиях одновременной разработки, тестирования, канареечного анализа и развертывания. Одни компоненты или устройства могут принять полный набор выпусков, а другие нужно много раз откатывать. Третьи могут претерпеть минимальные изменения и перейти от сломанной или уязвимой версии к следующему стабильному, полноценному выпуску.

Таким образом, использование MASVN — это полезный метод, который можно сочетать со списком запретов для самообновляющихся компонентов. В этом сценарии вы можете очень быстро запустить список запретов — возможно, в условиях реагирования на происшествие. Потом выполните обслуживание MASVN в более спокойных условиях, чтобы собрать лишнее в списке запретов и окончательно исключить (для каждого экземпляра компонента) все уязвимые или устаревшие выпуски, которые больше не предназначены для повторного выполнения в этом экземпляре компонента.

Замена ключей подписи

Многие самообновляющиеся компоненты включают поддержку криптографической аутентификации предварительных обновлений. Другими словами, часть цикла выпуска для компонента включает в себя криптографическую подпись этого выпуска. Такие компоненты часто содержат жестко закодированный спи-

сок известных открытых ключей или поддержку независимой БД ключей, как часть `ComponentState`. Например:

```
def IsUpdateAllowed(self, Release, KeyDatabase) -> bool:  
    return VerifySignature(Release, KeyDatabase)
```

Откат можно предотвратить, изменив набор открытых ключей, которым доверяет компонент, обычно чтобы удалить старый или скомпрометированный ключ или ввести новый для подписи будущих выпусков. Старые выпуски становятся недействительными, так как более новые больше не доверяют общедоступному ключу проверки подписи, нужному для верификации подписей в более старых выпусках. Но заменой ключей нужно управлять тщательно, ведь внезапное переключение с одного ключа подписи на другой может подвергнуть системы риску возникновения проблем с надежностью.

Вы можете заменять ключи постепенно, вводя новый ключ проверки подписи обновлений ($k + 1$) вместе со старым (k) и разрешив обновления, которые аутентифицируются с помощью любого из них. Как только новый ключ докажет свою стабильность, объявите ключ k как недоверенный. Для такой схемы нужна поддержка нескольких подписей на артефакте выпуска и нескольких ключей проверки при верификации подлинности предполагаемых обновлений. Преимущество в том, что замена ключей подписи — лучшая практика управления криптографическими ключами — выполняется регулярно и может сработать при необходимости во время происшествия.

Замена ключей поможет вам восстановиться после очень серьезной компрометации, после которой злоумышленнику удастся временно получить контроль над выпуском, подписать и развернуть его с максимальным значением `Release[MASVN]`. В этом типе атаки, установив `ComponentState[MASVN]` на максимальное значение, злоумышленник вынуждает вас установить `Release[SVN]` на максимальное значение, чтобы использовать будущие выпуски, тем самым делая всю схему MASVN бесполезной. В ответ вы можете отозвать скомпрометированный открытый ключ в новых выпусках, подписанных новым ключом, и добавить выделенную логику для распознавания необычно высокого `ComponentState[MASVN]` и его сброса. Эта логика сама по себе тонкая и потенциально опасная, поэтому вы должны использовать ее осторожно и намеренно отзывать любые выпуски, включающие ее сразу после завершения задачи.

Эта глава не охватывает всю сложность реагирования на происшествия при серьезной и целенаправленной компрометации. См. главу 18 для получения подробных сведений.

Откат прошивки и другие аппаратно-ориентированные ограничения

Аппаратные устройства с их собственной прошивкой — такие как компьютер и его BIOS или сетевая карта (NIC) и ее прошивка — распространенные примеры самообновляющихся компонентов. Такие устройства создают дополнительные проблемы для надежных схем MASVN или замены ключей, о которых мы кратко расскажем здесь. Эти сведения важны для восстановления, ведь они позволяют включить масштабируемое или автоматическое восстановление после потенциально вредоносных действий.

Иногда однократно программируемые (one-time-programmable, OTP) устройства, такие как предохранители, используются ПЗУ или прошивкой для реализации однонаправленной схемы MASVN путем сохранения `ComponentState[MASVN]`. У этих схем значительные риски надежности, потому что их откат невозможен. Благодаря дополнительным программным уровням можно устранить ограничения физического оборудования. Например, поддерживаемый OTP `ComponentState[MASVN]` покрывает небольшой специализированный загрузчик, имеющий собственную логику MASVN и эксклюзивный доступ к отдельной изменяемой области хранения MASVN. Затем этот загрузчик предоставляет более надежную семантику MASVN программному стеку более высокого уровня.

Для проверки подлинности подписанных обновлений аппаратные устройства иногда используют память OTP для хранения открытых ключей (или их хешей) и информации об аннулировании, связанной с ними. Число поддерживаемых замен ключей или их откатов обычно сильно ограничено. В этих случаях общий шаблон снова заключается в использовании открытого ключа в кодировке OTP и информации об аннулировании для проверки небольшого загрузчика. Этот загрузчик содержит свой уровень проверки и логики управления ключами, аналогичный примеру MASVN.

При работе с большим набором аппаратных устройств, активно использующих эти механизмы, управление запасными частями может быть проблемой. Части, которые остаются запасными многие годы, прежде чем они будут развернуты, обязательно будут иметь очень старую прошивку на момент первого использования. Эта прошивка должна быть обновлена. Если старые ключи полностью не используются, а новые выпуски подписаны только новыми ключами, которых не было на момент создания запасной части, новое обновление не будет проверено.



Одно из решений — провести устройства через последовательность обновлений, убедившись, что они затрагивают все выпуски, доверяющие как старому, так и новому ключу во время их замены. Другое решение — поддержка нескольких подписей на выпуск. Даже если более новые образы (и устройства, обновленные для запуска этих образов) не доверяют более старому ключу проверки, новые образы могут все еще иметь подпись для старого ключа. Только старые версии прошивки могут проверять эту подпись — желаемое действие, которое позволяет им восстанавливаться при отсутствии обновлений.

Подумайте, сколько ключей может использоваться в течение всего срока службы устройства, и убедитесь, что на нем достаточно места. Некоторые продукты FPGA поддерживают несколько ключей для аутентификации или шифрования своих битовых потоков¹.



Использование явного механизма отката

Главная задача системы отката — удалить какой-либо доступ или функцию. При активной компрометации система отката может спасти ситуацию, позволяя вам быстро отозвать захваченные злоумышленником учетные данные и восстановить контроль над своими системами. Но после того, как система отката установлена, случайное или злонамеренное поведение может привести к последствиям для надежности и безопасности. Рассмотрите эти вопросы на этапе проектирования. В идеале система отката должна служить своей цели всегда, не создавая слишком много собственных рисков безопасности и надежности.

Ознакомимся подробнее с механизмом отката. Рассмотрим неудачный, но распространенный сценарий. Вы обнаружили, что злоумышленник каким-то образом получил контроль над действительными учетными данными (такими как пара SSH-ключей клиента, разрешающая вход в систему для доступа к кластеру серверов), и нужно откатить эти полномочия².



Откат — сложная тема, касающаяся многих аспектов устойчивости, безопасности и восстановления. В этом разделе рассматриваются только некоторые его аспекты, относящиеся к восстановлению. Дополнительные темы для изучения: когда использовать списки разрешений и запретов, как чисто заменять устойчивые к сбоям сертификаты и как безопасно отменять изменения во время развертывания. В других главах книги вы найдете советы по многим из этих тем, но имейте в виду, что ни одна хорошая книга не может постоянно ссылаться сама на себя.

¹ Одним из примеров является аппаратный корень поддержки доверия в устройствах Xilinx Zynq Ultrascale+ (<https://oreil.ly/hfydr>).

² Немедленный откат учетных данных — не всегда лучший выбор. Обсуждение реакции на компрометацию см. в главе 17.

Централизованная служба отзыва сертификатов

Вы можете воспользоваться централизованной службой отзыва сертификатов. Этот механизм устанавливает приоритеты безопасности, требуя от ваших систем взаимодействия с централизованной БД, в которой хранится информация о достоверности сертификатов. Тщательно отслеживайте и поддерживайте эту базу для обеспечения безопасности ваших систем, ведь она становится официальным хранилищем записей, для которых действительны сертификаты. Этот подход напоминает создание отдельного сервиса ограничения скорости, независимого от сервиса, предназначенного для внесения изменений, как обсуждалось выше. Но у требования связи с БД достоверности сертификатов есть недостаток. Если база данных не будет работать, все другие зависимые системы тоже выйдут из строя. Есть сильное искушение не закрывать систему при сбое, даже если БД достоверности сертификата недоступна, чтобы другие системы тоже были открыты. Действуйте осторожно!

Открытие при сбое

Открытие при сбое предотвращает блокировку и упрощает восстановление, но и создает опасный компромисс. Эта стратегия обходит важную защиту доступа при неправильном использовании или атаке. Даже сценарии частичного аварийного открытия могут вызвать проблемы. Представьте, что система, от которой зависит БД достоверности сертификатов, выходит из строя. Предположим, что база зависит от сервиса времени или эпохи, но принимает все правильно подписанные учетные данные. Если БД достоверности сертификата не может получить доступ к сервису времени/эпохи, то злоумышленник, выполняющий простую атаку типа «отказ в обслуживании», перегрузив сетевые ссылки на сервис времени/эпохи, может повторно использовать даже очень старые отозванные учетные данные. Эта атака работает, потому что отозванные сертификаты действительны, пока продолжается DoS-атака. Преступник может проникнуть в вашу сеть или найти новые способы распространения по ней, пока вы пытаетесь восстановить систему.

Вместо открытия при сбое лучше распространить известные достоверные данные из сервиса отката на отдельные серверы в форме списка отката, который узлы могут кэшировать локально. Узлы будут использовать имеющуюся информацию, пока не получат более качественные данные. Этот выбор намного безопаснее политики тайм-аута и открытия при сбое.

Непосредственная обработка аварийных ситуаций

Для быстрого отзыва ключей и сертификатов вы можете разработать инфраструктуру, чтобы напрямую обрабатывать аварийные ситуации через развертывание изменений в файлах *authorized_users* или списке отката ключей (Key Revocation List, KRL) сервера¹. Это решение проблематично для восстановления по нескольким причинам.



Особенно заманчиво управлять файлами *authorized_keys* или *known_hosts* напрямую, когда используется небольшое количество узлов. Но это плохо масштабируется и разбрасывает проверку данных по всему набору устройств. Трудно гарантировать, что данный набор ключей был удален из файлов на всех серверах, особенно если эти файлы — единственный источник истины.

Вместо прямого управления файлами *authorized_keys* или *known_hosts* обеспечьте согласованность процессов обновления, централизованно управляя ключами и сертификатами и распределяя состояние по серверам через список отката. Развертывание явных списков отката — это возможность максимально снизить неопределенность в рискованных ситуациях, когда вы двигаетесь с максимальной скоростью. Вы можете использовать свои обычные механизмы для обновления и мониторинга файлов, включая механизмы ограничения скорости, на отдельных узлах.

Устранение зависимости от точных представлений о времени

У использования откатов есть еще одно преимущество. При проверке сертификата этот подход устраняет зависимость от точных представлений о времени. Независимо от причины неправильное время плохо повлияет на проверку сертификата. Старые сертификаты могут резко снова стать действительными, пропуская злоумышленников, а правильные могут внезапно не пройти проверку. Это приведет к простоям в обслуживании. Такие осложнения могут усугубить и без того стрессовую компрометацию или происшествие.

Лучше, чтобы проверка сертификата вашей системы зависела от аспектов, которыми вы управляете напрямую. Это может быть отправка файлов, содержащих открытые ключи или списки откатов. Системы, передающие файлы,

¹ Файл KRL — компактное бинарное представление того, какие ключи, выпущенные центром сертификации (CA), были отозваны. Для получения дополнительной информации см. справочную страницу `ssh-keygen(1)` (<https://oreil.ly/rRqkZ>).

сами файлы и основной источник истины будут защищаться, поддерживаться и контролироваться гораздо лучше, чем распределение времени. Далее восстановление сводится к простой отправке файлов, и для мониторинга нужно будет только проверить правильность этих файлов — это стандартный процесс, уже используемый в ваших системах.

Откат учетных данных в масштабе

Используя явный откат, учитывайте последствия масштабируемости. Будьте осторожны с масштабируемым откатом. Злоумышленник, частично скомпрометировавший ваши системы, может использовать это как мощный инструмент для дальнейшего отказа в обслуживании. Возможно, он воспользуется этим даже для отката всех действительных учетных данных в инфраструктуре. Продолжая упомянутый ранее пример с SSH, преступник может попытаться отозвать все сертификаты хоста SSH, но ваши устройства должны поддерживать такие операции, как обновление файла KRL, чтобы применить новую информацию об откате. Как защитить эти операции от злоупотреблений?

При обновлении файла KRL слепая замена старого файла новым может стать проблемой¹. Одна отправка может откатить все действительные учетные данные во всей инфраструктуре. Попробуйте заставить целевой сервер оценить новый KRL перед применением и отказаться от любого обновления, отменяющего его учетные данные. KRL, откатывающий учетные данные всех хостов, игнорируется ими. Поскольку лучшая стратегия злоумышленника — откатить половину ваших устройств, в худшем случае у вас есть гарантия того, что по крайней мере половина ваших устройств будет работать после каждой отправки KRL. Гораздо проще восстановить половину инфраструктуры, чем всю².

Как избежать рискованных исключений

У крупных распределенных систем из-за их размера могут быть проблемы с распространением списков откатов. Эти проблемы могут ограничивать скорость развертывания нового списка откатов и замедлять отклик при удалении скомпрометированных учетных данных.

¹ Хотя эта глава посвящена *восстановлению*, важно учитывать *устойчивость* операций вроде этой. При замене важного файла конфигурации в системе POSIX, такой как Linux, нужно быть осторожным, обеспечивая надежное поведение при сбоях или неполадках. Попробуйте использовать системный вызов `renameat2` с флагом `RENAME_EXCHANGE`.

² Последовательные злонамеренные отправки KRL могут расширить негативное влияние, но ограничения по скорости и ширине отправок все еще сильно расширяют возможности ответа.

Возможно, вы захотите создать специальный список «срочных» откатов для устранения этого недостатка. Но это решение неидеально. Из-за того, что список будет использоваться редко, нет гарантии того, что механизм сработает в нужный момент. Лучшее решение — отменить список откатов, чтобы требовалось постоянное его обновление. Так, откат учетных данных во время чрезвычайной ситуации требует обновления только подмножества данных. Последовательное использование сегментирования данных означает, что ваша система всегда использует многосторонний список отзывов и вы применяете одни и те же механизмы в обычных и чрезвычайных ситуациях.

Остерегайтесь добавления «специальных» учетных записей (например, учетных записей, предоставляющих прямой доступ старшим сотрудникам), обходящих механизмы отката. Эти записи очень привлекательны для злоумышленников — если они будут успешно атакованы, то все ваши механизмы отката станут неэффективными.

Найдите свое предполагаемое состояние, вплоть до байтов

Для восстановления после любой категории ошибок (будь то случайные, непреднамеренные, злонамеренные или программные) нужно вернуть систему в известное состояние. Вы можете упростить это, если действительно знаете, каким должно быть предполагаемое состояние системы, и можете прочитать развернутое состояние. Это может показаться очевидным, но незнание предполагаемого состояния — распространенный источник проблем. Чем тщательнее вы кодируете целевое состояние и уменьшаете изменяемое состояние на каждом уровне — для каждого сервиса, хоста, устройства и т. д. — тем легче будет распознать момент возврата в допустимое рабочее состояние.

Тщательное кодирование предполагаемого состояния — основа превосходной автоматизации, безопасности, обнаружения вторжений и восстановления до известного состояния.

ЧТО ТАКОЕ СОСТОЯНИЕ

*Состояние системы содержит всю информацию, нужную системе для выполнения желаемой функции. Даже у систем, описанных как «не имеющие состояния» (например, простой REST веб-сервис, который сообщает, является ли число нечетным или четным), есть состояние. Например, версия кода, порт, подслушивающий сервис, и то, запускается ли сервис автоматически при запуске хоста/ВМ/контейнера. Для целей этой главы *состояние* описывает не только работу системы в ответ на последовательность запросов, но и конфигурацию и настройку среды самого сервиса.*

Управление хостом

Допустим, вы управляете отдельным хостом, таким как физическое или виртуальное устройство или даже простой докерный контейнер. Вы настроили инфраструктуру для автоматического восстановления, и это приносит много пользы, ведь эффективно решает многие потенциальные проблемы, например, те, которые подробно обсуждаются в главе 7 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/automation-at-google/>). Для включения автоматизации вам нужно кодировать состояние этого отдельного сервера, включая выполняемые на нем рабочие нагрузки. Вы кодируете достаточно этой информации, чтобы автоматизация могла безопасно вернуть машину в рабочее состояние. В Google мы применяем эту парадигму на каждом уровне абстракции, вверх и вниз по аппаратному и программному стеку.

Система Google, распределяющая пакеты ПО нашего хоста по набору устройств, как описано в подразделе «Проектирование должно быть максимально быстрым (в соответствии с политикой)» выше в этой главе, постоянно отслеживает все состояние системы необычным способом. Каждая машина всегда просматривает свою локальную файловую систему, поддерживая ассоциативный массив, содержащий имя и криптографическую контрольную сумму каждого файла в системе. Мы собираем эти массивы с помощью центральной службы и сравниваем их с назначенным набором пакетов (предполагаемым состоянием) для каждого устройства. Если найдено отклонение предполагаемого состояния от текущего, мы добавляем эту информацию в список отклонений.

Стратегия отслеживания состояния машины дает значительные преимущества для восстановления. Она объединяет разные средства восстановления в один процесс. Если космический луч случайно повреждает часть диска, мы обнаруживаем несоответствие контрольной суммы и исправляем отклонение. Если предполагаемое состояние устройства изменяется из-за того, что развертывание ПО для одного компонента непреднамеренно меняет файл для другого, изменяя содержимое этого файла, мы устраняем отклонение. Если кто-то пытается изменить локальную конфигурацию компьютера вне обычного инструментария управления и процесса проверки (случайно или злонамеренно), мы тоже исправляем это отклонение. Можно было бы исправлять отклонения через переоснащение всей системы — неискусственный и простой в реализации, но более разрушительный в масштабе подход.

Помимо записи состояния файлов на диске, у многих приложений есть соответствующее состояние в памяти. Автоматическое восстановление должно использовать оба состояния. К примеру, когда запускается служебный процесс SSH, он считывает свою конфигурацию с диска и не перезагружает ее, пока не получит соответствующее указание. Для гарантии того, что состояние в памяти обновляется по мере необходимости, каждый пакет должен содержать

идемпотентную команду `post_install`¹, которая выполняется всякий раз, когда исправляется отклонение в его файлах. `post_install` пакета OpenSSH перезапускает служебный процесс SSH. Подобная команда `pre_rm` очищает любое состояние в памяти перед удалением файлов. Эти простые механизмы могут поддерживать состояние всего устройства в оперативной памяти, сообщать и исправлять отклонения.

Кодирование этого состояния позволяет автоматизации проверять каждое отклонение на наличие вредоносных несоответствий. Обширная информация о состоянии ваших устройств также очень важна во время криминалистического анализа, следующего за происшествием в области безопасности. Она поможет вам лучше понять действия и намерения злоумышленника. Представим, что он нашел способ размещения вредоносного shell-кода на некоторых ваших устройствах, но не смог взломать систему мониторинга и восстановления, отменившую неожиданные изменения на одной или нескольких машинах. Преступнику будет намного сложнее скрыть свои следы, потому что центральная служба будет замечать и регистрировать отклонения, о которых сообщает хост².

Так, все изменения состояния равны на этом уровне абстракции. Вы можете автоматизировать, защитить и проверить все изменения состояния подобным образом, рассматривая откат бинарного файла, не прошедший канареечный анализ, и аварийное развертывание обновленного бинарного файла `bash` как обычные изменения. Пользуясь одной инфраструктурой, вы можете принимать согласованные решения о политиках относительно скорости применения каждого изменения. Ограничения скорости на этом уровне позволяют защитить от непреднамеренных столкновений между типами изменений и установить их максимально допустимую скорость.

Прошивка устройства

Для обновления прошивки вы можете записывать состояние дальше вниз по стеку. Отдельные части оборудования в современном компьютере имеют собственные параметры ПО и конфигурации. Для безопасности и надежности вы должны отслеживать версию прошивки каждого устройства. В идеале нужно

¹ Понятия `post_install` и `pre_rm` заимствованы из Debian (<https://oreil.ly/H9q9p>). Система управления пакетами Google использует подход посложнее: она не допускает отдельную или частично неудачную настройку и установку. Любое изменение пакета будет успешным, или машина полностью откатится до предыдущего состояния. В случае сбоя отката устройство проходит процесс исправления для повторной установки и возможной замены оборудования. Этот подход позволяет устранить большую часть сложности состояний пакета.

² Подробнее о том, как реагировать на компрометацию, см. в главе 17.

зафиксировать все параметры, доступные для этой прошивки, и убедиться, что для них выбраны ожидаемые значения.

Управляя прошивкой и ее конфигурацией на компьютерах Google, мы используем те же системы и процессы, что и для управления обновлениями ПО нашего хоста и анализа отклонений (см. предыдущий пункт «Управление хостом»). Автоматизация надежно распределяет предполагаемое состояние для всех прошивок в виде пакета, сообщает о любых отклонениях и исправляет их в соответствии с политикой ограничения скорости и другими политиками обработки сбоев.

Предполагаемое состояние часто не подвергается прямому воздействию локального мониторящего файловую систему служебного процесса, не обладающего специальными знаниями о прошивке устройства. Мы отделяем сложность взаимодействия с оборудованием от этого процесса. Мы позволяем каждому пакету ссылаться на проверку активации — сценарий или бинарный файл в пакете, который периодически запускается для определения правильности установки пакета. Они делают все необходимое для связи с оборудованием и сравнения версии прошивки и параметров конфигурации. После они сообщают о любых неожиданных отклонениях. Эта функциональность полезна для восстановления, ведь она позволяет экспертам в данной области (то есть владельцам подсистем) предпринимать соответствующие шаги для устранения проблем. Например, автоматизация отслеживает целевое, текущее состояние и отклонения. Если машина предназначена для запуска BIOS версии 3, но работает под управлением BIOS версии 1, автоматизация не имеет никакого мнения о версии BIOS 2. Сценарии управления пакетами определяют, могут ли они обновить BIOS сразу до версии 3 или уникальные ограничения требуют последовательной установки нескольких версий.

Несколько конкретных примеров показывают, почему управление предполагаемым состоянием важно и для безопасности, и для надежности. Для взаимодействия с внешними источниками времени Google использует специальное оборудование от поставщиков (например, системы GNSS/GPS и атомные часы), чтобы обеспечить точный хронометраж в производстве — предварительное условие для Spanner¹. Наше оборудование хранит две разные версии прошивки на двух разных чипах устройства. Для правильной работы этих источников времени нам нужно тщательно настроить версии прошивки. Еще одна сложность в том, что у некоторых версий прошивки есть известные ошибки, влияющие на

¹ Spanner (<https://oreil.ly/YGCjO>) — это глобально распределенная БД Google, поддерживающая внешне согласованные распределенные транзакции. Она требует очень жесткой синхронизации времени между центрами обработки данных.

способ обработки дополнительных секунд и другие крайние случаи. Без поддержки прошивки и настройки на этих устройствах мы не сможем предоставить точное время в производстве. Важно, чтобы управление состоянием охватывало и резервный, вторичный или другой неактивный код или конфигурацию, способную внезапно стать активной во время восстановления. Так как восстановление — не лучшее время для начала выяснения того, какие ошибки находятся в неактивном образе. Если устройства загружаются, но аппаратное обеспечение часов не предоставляет достаточно точное время для запуска наших сервисов, то система недостаточно восстановилась.

Другой пример: у современного BIOS множество параметров, критически важных для безопасности загружаемой (и загруженной) системы. Может понадобиться, чтобы порядок загрузки предпочел SATA загрузочным устройствам USB, чтобы предотвратить злонамеренную загрузку системы с USB-накопителя. Более продвинутые развертывания отслеживают и поддерживают БД ключей, позволяющую подписывать обновления BIOS, как для управления заменой ключей, так и для защиты от взлома. Если во время восстановления в вашем загрузочном устройстве произошел сбой, вы не хотите обнаружить, что BIOS завис в ожидании ввода с клавиатуры, потому что вы забыли отслеживать и указывать настройки для дополнительного загрузочного устройства.

Глобальные сервисы

Самый высокий уровень абстракции в вашем сервисе и наиболее устойчивые части вашей инфраструктуры — например, хранение, именование и идентификация — могут быть самыми трудными областями для восстановления. Парадигма захвата состояния тоже применима к этим высоким уровням стека. При создании или развертывании новых глобальных одноэлементных систем вроде Spanner или Hadoop (<https://hadoop.apache.org/>) убедитесь, что поддерживаются несколько экземпляров, даже если вы никогда не планируете использовать более одного, и даже для самого первого развертывания. Помимо резервного копирования и восстановления вам может понадобиться повторная сборка нового экземпляра всей системы для восстановления данных в ней.

Вместо ручной настройки сервисов вы можете написать обязательную автоматизацию включения или использовать декларативный высокоуровневый язык конфигурации (например, инструмент настройки контейнерного управления, такой как Terraform). В этих сценариях вы должны фиксировать состояние создания сервиса. Это аналогично тому, как разработка через тестирование фиксирует предполагаемое поведение вашего кода, которое затем управляет реализацией и помогает прояснить публичный API. Обе практики приводят к улучшению обслуживания систем.

Популярность контейнеров, которые часто создаются герметично и развертываются из исходного кода, означает, что состояние многих строительных блоков глобальных служб фиксируется по умолчанию. Хотя и здорово автоматически получать «большую часть» информации о состоянии службы, но не забывайте о ложном чувстве безопасности. Для восстановления структуры с нуля нужна сложная цепочка зависимостей. Это может привести к обнаружению непредвиденных проблем с емкостью или циклических зависимостей. При работе с физической инфраструктурой спросите себя: достаточно ли у вас запасных машин, дискового пространства и емкости сети для запуска второй копии инфраструктуры? Если вы работаете с большой облачной платформой, такой как GCP или AWS, вы можете приобрести столько физических ресурсов, сколько нужно, но достаточно ли у вас квоты для использования этих ресурсов в короткие сроки? У ваших систем органически выросли какие-либо взаимозависимости, мешающие чистому запуску с нуля? Полезно провести аварийное тестирование в контролируемых условиях, чтобы убедиться в вашей готовности к непредвиденным обстоятельствам¹.

Постоянные данные

*Никто не заботится о резервных копиях, все заботятся только о восстановлении*²

Пока что мы сосредоточены на безопасном восстановлении инфраструктуры, необходимой для работы вашего сервиса. Для некоторых сервисов без сохранения состояния этого достаточно. Но многие сервисы хранят постоянные данные, что создает особые проблемы. Есть много информации о задачах резервного копирования и восстановления постоянных данных³. Здесь мы обсудим ключевые аспекты, касающиеся безопасности и надежности.

Для защиты от упомянутых ранее типов ошибок, особенно от вредоносных, для ваших резервных копий нужен такой же уровень защиты целостности, как

¹ См.: *Krishnan K.* Weathering the Unexpected. ACM Queue, 2012. № 10 (9). <https://oreil.ly/vJ66c>.

² Несколько вариантов этого превосходного совета были приписаны многим уважаемым инженерам. Самая старая версия, которую мы могли найти в печати (в книге из библиотеки одного из авторов), принадлежит У. Кертису Престону (W. Curtis Preston) из Unix Backup & Recovery (<http://shop.oreilly.com/product/9781565926424.do>) (O'Reilly). Он приписывает цитату Рону Родригесу (Ron Rodriguez): «Никому нет дела до того, сможешь ли ты сделать резервную копию, — всех интересует только восстановление».

³ Для начала см. доклад Кристины Беннетт на SREcon18 — Tradeoffs in Resiliency: Managing the Burden of Data Recoverability (<https://oreil.ly/V-FEC>).

и для основного хранилища. Злонамеренный инсайдер может изменить содержимое ваших резервных копий и принудительно восстановить поврежденную. Даже если у вас есть надежные криптографические подписи, показывающие ваши резервные копии, они бесполезны, если ваши инструменты восстановления не проверяют их как часть процесса восстановления или если ваши внутренние средства контроля доступом как следует не ограничивают набор людей, которые могут создать подписи вручную.

Разделите защиту безопасности постоянных данных, где это возможно. Если вы обнаружите повреждение в своей копии данных, изолируйте его от наименьшего из возможных подмножеств. Восстановление 0,01 % ваших постоянных данных пройдет быстрее, если вы можете определить это подмножество резервных данных и проверить их целостность, не считывая и не проверяя остальные 99,99 %. Эта способность становится важнее по мере увеличения размера постоянных данных. Но если вы следуете лучшим практикам проектирования больших распределенных систем, разделение обычно происходит естественным образом. Расчет размера частей для разделения часто требует компромисса между хранением и вычислительными издержками. Учитывайте и влияние размера блока на среднее время восстановления.

Примите во внимание то, как часто ваша система требует частичного восстановления. Подумайте, сколько общей инфраструктуры используется вашими системами, участвующими в восстановлении и переносе данных. Перенос обычно похож на частичное восстановление с низким приоритетом. Если каждая миграция данных — на другой компьютер, блок, кластер или центр обработки — работает и укрепляет уверенность в критически важных частях вашей системы восстановления, вы будете знать, что задействованная инфраструктура с большей вероятностью будет работать и будет понятна, когда это необходимо.

Восстановление данных может создавать собственные проблемы безопасности и конфиденциальности. Удаление данных — необходимая и часто юридически обязательная функция для многих систем¹. Убедитесь, что ваши системы восстановления не позволяют случайно восстановить предположительно уничтоженные данные. Помните о различии между удалением ключей шифрования и удалением зашифрованных данных. Вы можете сделать данные недоступными, уничтожив соответствующие ключи шифрования, но такой подход требует разделить ключи, используемые для разных типов данных, так, чтобы это было совместимо с требованиями удаления отдельных данных.

¹ См. политику хранения данных Google (<https://oreil.ly/abNZP>).

Проектирование для тестирования и непрерывной проверки

Как обсуждалось в главе 8, непрерывная проверка может помочь в поддержании надежной системы. Ваша стратегия тестирования должна содержать тесты процессов восстановления. Процессы восстановления представляют собой необычные задачи в незнакомых условиях, и без тестирования возможны непредвиденные проблемы. Например, если вы автоматизируете создание чистого экземпляра системы, хороший тест может выявить, что конкретный сервис будет иметь только один глобальный экземпляр. Таким образом, он поможет выявить ситуации, когда сложно создать второй экземпляр для восстановления этого сервиса. Подумайте о тестировании возможных сценариев восстановления для верного баланса между эффективностью тестирования и действительным состоянием производственного кода.

Вы можете рассмотреть и возможность тестирования редких ситуаций, когда восстановление особенно сложно. В Google мы реализуем протокол управления криптографическими ключами в разнообразном наборе сред: процессоры Arm и x86, UEFI и встроенное ПО, Microsoft Visual C++ (MSVC), компиляторы Clang, GCC и т. д. Мы знали, что отработать все режимы сбоя для этой логики будет непросто. Даже при серьезных инвестициях в комплексное тестирование трудно реально эмулировать аппаратные сбои или прерывание связи. Вместо этого мы решили реализовать базовую логику единожды переносимым, независимым от компилятора и от размеров способом. Мы тщательно протестировали логику и уделили внимание проектированию интерфейса для абстрактных внешних компонентов. Для фальсификации отдельных компонентов и проявления их поведения при сбое мы создали интерфейсы для чтения и записи байтов из флэш-памяти, хранения криптографических ключей и для примитивов мониторинга производительности. Такой метод тестирования условий среды выдержал испытание временем, так как он явно фиксирует классы отказов, от которых нам и нужно восстановиться.

Ищите способы укрепить доверие к вашим методам восстановления с помощью непрерывной проверки. Восстановление включает человеческие действия, а люди ненадежны и непредсказуемы. Только модульные тесты или непрерывная интеграция/доставка/развертывание не могут выявить ошибки, возникающие из-за человеческих навыков или привычек. В дополнение к проверке эффективности и функциональной совместимости рабочих процессов восстановления проверьте читаемость и простоту инструкций по восстановлению.

Экстренный доступ

Описанные в этой главе методы восстановления основаны на способности исполнителя взаимодействовать с системой, и мы выступили за процессы восстановления, использующие те же основные службы, что и обычные операции. Но вам может понадобиться специальное решение для развертывания во время *полного* отказа обычных методов доступа.

АВАРИЙНЫЙ ДОСТУП ОЧЕНЬ ВАЖЕН ДЛЯ НАДЕЖНОСТИ И БЕЗОПАСНОСТИ

Экстренный доступ — крайний случай, когда мы не можем переоценить важность надежности и безопасности — нет других уровней для смягчения отказа. При экстренном доступе должен предоставляться абсолютный минимум всех технологий, используемых респондентами для доступа к основным административным интерфейсам вашей компании (например, доступ на уровне администратора к сетевым устройствам, ОС или средствам администрирования приложений) и для связи с другими респондентами во время наиболее серьезных сбоев. Он должен также сохранять контроль доступа в максимально возможной степени. Предусмотрите сети, через которые могут подключаться респонденты, любые системы контроля доступа и инструменты восстановления, которые могут вам понадобиться.

Обычно у организаций уникальные потребности и возможности для экстренного доступа. Ключевой момент — нужно иметь план и заранее созданные механизмы, поддерживающие и защищающие этот доступ. Нужно знать и об уровнях системы вне вашего контроля — любые сбои в них нельзя обработать, даже если они влияют на остальную часть системы. Тогда вам придется остаться в стороне, пока кто-то еще не исправит сервисы, от которых зависит ваша компания. Чтобы минимизировать влияние отключений сторонних производителей на ваш сервис, поищите возможные экономически эффективные резервы, которые вы можете развернуть на любом уровне своей инфраструктуры. Конечно, может не быть никаких экономически эффективных альтернатив, или вы уже достигли верхнего SLA, которое гарантируется вашим поставщиком услуг. Тогда помните, что система доступна в той мере, в которой доступна сумма ее зависимостей¹.

Стратегия удаленного доступа Google базируется на развертывании автономных критически важных сервисов на территориально распределенных блоках. Для закрепления усилий по восстановлению мы стремимся контролировать удаленный доступ, эффективную локальную связь, альтернативные сети и укрепленные

¹ См.: *Treynor B. et al.* The Calculus of Service Availability. ACM Queue, 2017. № 15 (2). <https://oreil.ly/It4-h>.

важные точки в инфраструктуре. Во время глобального сбоя, так как каждый блок остается доступным по крайней мере для некоторой части респондентов, они могут начать фиксировать сервисы, к которым имеют доступ, а после радиально расширить процесс восстановления. То есть при невозможности глобального сотрудничества можно попытаться решить проблемы в небольших произвольных областях. Несмотря на то что у респондентов может не хватить контекста, чтобы узнать, где они больше всего нужны, и есть риск расхождения областей, этот подход может сильно ускорить восстановление.

Контроль доступа

Службы контроля доступа организации не должны становиться единими точками отказа для всего удаленного доступа. В идеале вы сможете реализовать альтернативные компоненты, без одинаковых зависимостей, но для их надежности могут понадобиться разные решения безопасности. Хотя их политики доступа должны быть одинаково сильными, они могут быть менее удобными и/или иметь ухудшенный набор функций по техническим или практическим причинам.

Учетные данные удаленного доступа не могут зависеть от обычных сервисов учетных данных, ведь они полагаются на зависимости, которые могут быть недоступны. Поэтому вы не можете получить учетные данные доступа из динамических компонентов инфраструктуры доступа, таких как единый вход (SSO) или интегрированные поставщики удостоверений, если не можете заменить их реализациями с низкой зависимостью. Выбор срока действия этих данных — сложный компромисс в области управления рисками. Эффективная практика принудительного применения учетных данных для краткосрочного доступа пользователей или устройств становится бомбой замедленного действия, если сбой превышает этот срок. Поэтому придется увеличивать срок действия сервисов учетных данных для увеличения продолжительности любых ожидаемых отключений, несмотря на дополнительную угрозу безопасности (см. подраздел «Разделение по времени» в главе 8). Если вы выдаете учетные данные удаленного доступа по фиксированному расписанию, а не активируете их по требованию в начале сбоя, он может начаться как раз в момент окончания срока действия учетных данных.

При авторизации пользователя или устройства во время доступа к сети у любой зависимости от динамических компонентов есть риски, аналогичные тем, с которыми сталкивается служба учетных данных. Все больше сетей пользуются динамическими протоколами¹. Поэтому вам может потребоваться предоставить

¹ Например, программно-конфигурируемые сети (software-defined networking, SDN).

более статичные альтернативы. Список доступных сетевых провайдеров может ограничить ваши возможности. Если возможны выделенные сетевые подключения со статическим контролем доступа к сети, убедитесь, что их периодические обновления не нарушают маршрутизацию или авторизацию. Может быть очень важно реализовать достаточный мониторинг для определения места потери доступа к сети или чтобы отличить проблемы доступа к сети от проблем на уровнях над ней.

ПОЛНОМОЧИЯ GOOGLE ДЛЯ КОНТРОЛЯ ДОСТУПА

Корпоративная сеть Google достигает своих свойств нулевого доверия при помощи автоматических оценок доверия каждого устройства сотрудника, краткосрочных учетных данных из службы единого входа с двухфакторной авторизацией и многосторонней авторизацией для некоторых административных операций. Каждая из этих функций реализована в ПО с необычными зависимостями. Отказ любой из них может помешать доступу всех сотрудников, даже отвечающих за происшествия. Чтобы учесть эту возможность, мы представили альтернативные учетные данные, которые будут доступны в автономном режиме, и развернули альтернативные алгоритмы аутентификации и авторизации. Эти положения обеспечивали соответствующую степень безопасности и значительно сокращали общий набор зависимостей. Чтобы ограничить новую поверхность атаки, издержки и влияние относительно худшего удобства использования, эти положения ограничены для людей, нуждающихся в немедленном доступе, в то время как остальная часть компании ждет, пока они исправят обычные службы контроля доступа.

Связь

Каналы экстренной связи — следующий критический фактор реагирования на чрезвычайные ситуации. Что делать вызывающим, если их обычный сервис чата недоступен? Что, если он будет взломан или прочитан злоумышленником?

Выберите средство общения (например, Google Chat, Skype или Slack) с наименьшим количеством зависимостей и достаточно удобное для ваших команд респондентов. Если эта технология передана на аутсорсинг, есть ли у респондентов доступ к системам, даже если системные уровни вне вашего контроля нарушены? Телефонные мосты, хотя они и неэффективны, тоже используются — как вариант старой школы, хотя они все чаще развертываются с использованием IP-телефонии, зависящей от Интернета. Инфраструктура ретранслируемого интернет-чата (Internet Relay Chat, IRC) надежна и автономна, если ваша компания хочет развернуть свое решение, но в ней нет некоторых аспектов безопасности. Вы все равно должны убедиться в доступности ваших IRC-серверов во время перебоев в работе сети. Когда каналы связи размещены вне вашей инфраструктуры, вы можете задаться вопросом, гарантируют ли провайдеры достаточную аутентификацию и конфиденциальность для нужд вашей компании.

Привычки респондентов

Из-за уникальности технологий экстренного доступа часто нужно прибегать к практическим действиям, отличным от обычных повседневных операций. Если эти технологии не будут использоваться повсеместно в вашей компании, респонденты могут не знать, что с ними делать в чрезвычайной ситуации, и вы потеряете их преимущества. Интегрировать альтернативы с низкой зависимостью может быть сложно, но это только часть проблемы. Человеческое замешательство в условиях стресса в сочетании с редко используемыми процессами и инструментами могут вызвать сложность, способную затруднить любой доступ. Другими словами, люди, а не технологии, могут сделать инструменты аварийного доступа неэффективными¹.

Чем меньше будет различие между нормальными и аварийными процессами, тем больше респондентов смогут действовать по привычке. Это освобождает их когнитивные способности и позволяет сосредоточиться на отличиях. В итоге устойчивость организации к сбоям может улучшиться. Например, в Google мы сосредоточились на Chrome, его расширениях и любых средствах контроля и инструментах, связанных с ним, в качестве единой платформы, достаточной для удаленного доступа. Благодаря введению аварийного режима в расширениях Chrome мы добились минимально возможного увеличения когнитивной нагрузки сразу, сохранив возможность позже интегрировать его в большее количество расширений.

Чтобы ваши респонденты регулярно применяли методы экстренного доступа, вводите политики, интегрирующие экстренный доступ в повседневные привычки дежурных инженеров, и постоянно проверяйте удобство использования соответствующих систем. Определите и установите минимальный период между необходимыми тренировками. Руководитель может отправлять уведомления по электронной почте, когда члену команды нужно выполнить требуемые задачи по обновлению учетных данных или обучению. Тренер может и отказаться от тренировки, если решит, что этот человек регулярно участвует в подобных действиях. Это повышает уверенность в том, что при аварии у остальной части команды будут соответствующие полномочия и необходимое обучение будет пройдено. Или же сделайте практику с механизмами аварийного доступа и любыми связанными с ними процессами обязательной для вашего персонала.

Убедитесь в наличии соответствующей документации — политики, стандарты и инструкции. Люди склонны забывать детали, которые редко используются,

¹ Инструменты аварийного доступа — это механизмы, способные обходить политики, что позволяет инженерам быстро устранять простои. См. подраздел «Механизм аварийного доступа» в главе 5.

и такая документация снимет стресс и сомнения под давлением. Обзоры и схемы архитектуры тоже полезны для сотрудников, реагирующих на происшествия. Они позволяют людям, не знакомым с предметом, быстро освоиться без особой помощи от экспертов в этой области.

Неожиданные преимущества

Описанные в этой главе принципы, основанные на отказоустойчивом проектировании, улучшают способность вашей системы к восстановлению. Неожиданные преимущества, помимо надежности и безопасности, помогут вам убедить вашу организацию принять эти методы. Рассмотрим сервер, разработанный для механизмов аутентификации, отката, блокировки и проверки обновлений прошивки. С помощью этих примитивов вы можете уверенно восстановить машину после обнаруженной компрометации. Теперь рассмотрите возможность использования этого устройства в облачном хостинг-сервисе «на чистом сервере», где поставщик хочет очищать и перепродавать устройства, используя автоматизацию. Устройства, спроектированные с учетом необходимости восстановления, уже обладают безопасным и автоматизированным решением.

Преимущества усиливаются в отношении безопасности цепочки поставок. Когда машины собираются из множества разных составляющих, вы будете уделять меньше внимания безопасности цепочки поставок для компонентов, целостность которых восстанавливается автоматически. Ваши операции при первой отправке просто требуют запуска процедуры восстановления. Бонус повторного использования процедуры восстановления: вы регулярно пользуетесь критически важными возможностями восстановления, поэтому ваши сотрудники готовы действовать в случае возникновения аварии.

Проектирование систем для восстановления считается сложной темой. Его ценность для бизнеса доказывается, только когда система находится вне запланированного состояния. Но учитывая, что мы рекомендуем операционным системам использовать бюджет ошибок для максимизации эффективности затрат¹, мы ожидаем, что такие системы будут регулярно находиться в состоянии ошибки. Мы надеемся, что ваши команды постепенно начнут инвестировать в механизмы ограничения скорости или отката как можно раньше в процессе разработки. Подробную информацию о том, как повлиять на вашу организацию, см. в главе 21.

¹ Для получения дополнительной информации о бюджетах ошибок см. главу 3 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/embracing-risk/>).

Резюме

В этой главе рассмотрены разные аспекты проектирования систем для восстановления. Мы объяснили, почему системы должны быть гибкими с точки зрения скорости, с которой они внедряют изменения. Эта гибкость позволяет по возможности медленно разворачивать изменения и избегать скоординированных сбоев. Благодаря гибкости вы можете также быстро и уверенно разворачивать изменения, когда нужно идти на больший риск для достижения целей безопасности. Возможность отката изменений важна для создания надежных систем. Но иногда вам может понадобиться предотвратить откат к небезопасным или достаточно старым версиям. Понимание, мониторинг и воспроизведение состояния вашей системы в максимально возможной степени — с помощью версий ПО, памяти, системных часов и т. д. — ключ к надежному восстановлению системы в любое ранее работавшее состояние и проверке соответствия текущего состояния вашим требованиям безопасности. В крайнем случае экстренный доступ позволяет респондентам оставаться на связи, оценивать систему и смягчать ситуацию. Продуманное управление политикой восстановления способствует стабильности и надежности повседневных действий.

Защита от DoS-атак

Дамиан Менчер с Виталием Шипицыным
и Бетси Бейер

Безопасность и надежность пересекаются, если активный злоумышленник выводит из строя систему, проведя атаку типа «отказ в обслуживании» (DoS). Отказ в обслуживании может быть вызван и неожиданными обстоятельствами — от обрыва оптоволоконного кабеля экскаватором до некорректного запроса, приводящего к сбою сервера, — и нацелен на любой уровень стека. Обычно это проявляется как внезапный всплеск использования сервиса. Несмотря на то, что вы можете применить некоторые меры защиты к существующим системам, для минимизации последствий DoS-атаки часто нужно тщательно проектировать систему. В этой главе обсуждаются некоторые стратегии защиты от DoS-атак.

Специалисты по безопасности часто думают о системах, с которыми они взаимодействуют, с точки зрения *атаки* и *защиты*. Но для типичной атаки вида «отказ в обслуживании» экономика предлагает более понятные термины: злоумышленник пытается заставить *спрос* на конкретный сервис превысить *предложение* емкости этого сервиса. В итоге возможностей сервиса становится недостаточно для обслуживания пользователей. Далее организация должна решить, понести ли еще большие расходы, пытаясь отразить атаку, или увеличить время простоя (и соответствующие финансовые потери) до прекращения атаки.

Хотя некоторые отрасли чаще подвергаются DoS-атакам, чем другие, так можно атаковать любой сервис. *DoS-вымогательство*, финансовая атака, в которой злоумышленник угрожает сломать сервис, если ему не заплатить, наносит удар практически без разбора¹.

¹ Некоторые вымогатели начинают с небольшой «демонстрационной атаки», чтобы заставить свою жертву платить. Почти во всех случаях преступники не способны генерировать более масштабную атаку и не будут создавать дальнейших угроз, если их требования игнорируются.

Стратегии атаки и защиты

И у преступников, и у защитников ограниченные ресурсы, которые они должны эффективно использовать для достижения своих целей. Разрабатывая защитную стратегию, начните с понимания стратегии противника, чтобы раньше него найти слабые места в вашей защите. С этим пониманием вы можете создавать защиту для известных атак и разрабатывать системы с гибкостью для быстрого предотвращения новых.

Стратегия атакующего

Чтобы превзойти возможности своей цели, злоумышленник должен сосредоточиться на эффективном использовании своих ограниченных ресурсов. Умный противник может помешать сервисам более мощного оппонента.

Типичный сервис имеет несколько зависимостей. Рассмотрим поток типичного пользовательского запроса.

1. DNS-запрос предоставляет IP-адрес сервера, который должен получать трафик пользователя.
2. Сеть переносит запрос на сервисные интерфейсы.
3. Интерфейсы интерпретируют пользовательский запрос.
4. Сервисные серверы предоставляют функциональность базы данных для пользовательских ответов.

Атака, способная успешно нарушить любой из этих шагов, нарушит и работу сервиса. Многие начинающие злоумышленники пытаются повлиять на поток запросов приложений или сетевой трафик. Более искушенный может генерировать запросы, на которые затратно отвечать, например, злоупотребляя поисковыми функциями, которые есть на многих веб-сайтах.

Из-за того, что одного устройства не хватает для нарушения работы большого сервиса (который часто поддерживается несколькими), решительный злоумышленник разработает инструменты, позволяющие использовать мощность многих устройств в так называемой атаке с *распределенным отказом в обслуживании* (distributed denial-of-service, DDoS). Для DDoS-атаки злоумышленник может либо скомпрометировать уязвимые машины и объединить их в *бот-сеть*, либо запустить *усиленную атаку*.

УСИЛЕННЫЕ АТАКИ



Во всех типах связи запрос обычно генерирует ответ. Представьте, что в магазин фермерских товаров, в котором продаются удобрения, поступает заказ на погрузку навоза. Предприятие отправит кучу коровьего навоза по указанному адресу. Но что, если личность заявителя была подделана? Ответ отправится не туда, а получатель не будет рад неожиданной доставке.

Усиленная атака работает по тому же принципу. Но вместо одного запроса злоумышленник подделывает повторяющиеся запросы с одного адреса на тысячи серверов. Трафик ответа вызывает распределенную отраженную DoS-атаку на поддельный IP-адрес.

Хотя сетевые поставщики должны предотвращать подделку исходных IP-адресов своих клиентов (которые аналогичны обратному адресу) в исходящих пакетах, не все они применяют это ограничение и не все последовательны в его применении. Злоумышленники могут использовать пробелы в действиях поставщиков для отражения небольших запросов от открытых серверов, которые потом возвращают жертве еще больший ответ. Есть несколько протоколов, обеспечивающих эффект усиления, таких как DNS, NTP и memcache¹.

Хорошая новость в том, что большинство атак с усилением легко идентифицировать, так как усиленный трафик поступает из хорошо известного исходного порта. Вы можете эффективно защищать свои системы, пользуясь сетевыми ACL, ограничивающими трафик UDP (протоколов пользовательских датаграмм) от злоупотребляющих протоколов².

Стратегия защитника

Хорошо вооруженный защитник может поглощать атаки, просто излишне выделяя весь свой стек, но с большими затратами. Центры обработки данных, заполненные энергоемкими машинами, обходятся дорого, а постоянное выделение ресурсов для отражения самых крупных атак невозможно. Автоматическое масштабирование может пригодиться для сервисов, построенных на облачной платформе с достаточной пропускной способностью. Но обычно защитникам нужно использовать другие экономически эффективные подходы для защиты своих сервисов.

¹ См.: Rosrow C. Amplification Hell: Revisiting Network Protocols for DDoS Abuse // Proceedings of the 21st Annual Network and Distributed System Security Symposium, 2014. doi: 10.14722/ndss.2014.23233.

² Протоколы на основе TCP также могут быть использованы для этого типа атак. Для обсуждения см.: Kührer M. et al. Hell of a Handshake: Abusing TCP for Reflective Amplification DDoS Attacks // Proceedings of the 8th USENIX Workshop on Offensive Technologies, 2014. <https://oreil.ly/0JCPP>.

Разрабатывая лучшую стратегию защиты от DoS, учитывайте время разработки — отдавайте приоритет стратегиям, оказывающим наибольшее влияние. Может быть соблазнительно сосредоточиться на устранении вчерашнего сбоя, но ориентация на время может привести к быстрому изменению приоритетов. Вместо этого используйте подход модели угроз, чтобы сосредоточить свои усилия на самом слабом звене в цепочке зависимостей. Сравните угрозы в зависимости от количества компьютеров, которые злоумышленник должен будет контролировать, чтобы вызвать видимые пользователю сбои.



Мы используем термин *DDoS* для обозначения DoS-атак, эффективных только из-за их распределенной природы и использующих либо большую бот-сеть, либо усиленную атаку. Термин *DoS* применяется для обозначения атак, которые могут быть получены с одного хоста. Разница важна при проектировании защиты, так как вы часто можете развернуть защиту DoS на уровне приложений, тогда как защита DDoS часто использует фильтрацию в инфраструктуре.

Проектирование для защиты

Все силы идеальной атаки сосредоточены на одном ограниченном ресурсе, таком как пропускная способность сети, процессор или память сервера приложений, или на элементе серверной части, такой как база данных. Вашей целью должна быть защита каждого из этих ресурсов наиболее эффективным способом.

По мере проникновения вредоносного потока сообщений в систему он становится более сфокусированным и дорогостоящим для смягчения последствий. Поэтому важная особенность конструкции здесь — многоуровневая защита, при которой каждый новый уровень защищает предыдущий. Здесь мы рассмотрим варианты проектирования с защитой систем на двух основных уровнях: общая инфраструктура и отдельные сервисы.

Защищаемая архитектура

У большинства сервисов общая инфраструктура — пиринговые мощности, балансировщики сетевой нагрузки и балансировщики нагрузки приложений.

Общая инфраструктура — естественная среда для обеспечения общей защиты. Пограничные маршрутизаторы могут сдерживать атаки с высокой пропускной способностью, защищая основную сеть. Балансировщики сетевой нагрузки могут сдерживать атаки с переполнением пакетов для защиты балансировщиков нагрузки приложений. Балансировщики нагрузки приложений могут подавлять специфичные для приложений атаки до того, как трафик достигнет внешних интерфейсов сервисов.

Многоуровневая защита рентабельна, так как вам нужно только планировать пропускную способность внутренних уровней для стилей DoS-атак, способных нарушить защиту внешних уровней. Исключение трафика атаки как можно раньше экономит пропускную способность и вычислительную мощность. Развертывая ACL на границе сети, вы можете отбросить подозрительный трафик, прежде чем он сможет использовать пропускную способность внутренней сети. Развертывание кэширующих прокси-серверов на границе сети сильно экономит затраты и сократит задержки для конечных пользователей.



Правила межсетевого экранирования с сохранением состояния — не самая подходящая защита первой линии для производственных систем, получающих входящие соединения¹. Злоумышленник может провести атаку с исчерпанием состояния, когда количество неиспользуемых соединений заполняет память брандмауэра с включенным отслеживанием соединений. Вместо этого используйте списки управления доступом маршрутизатора для ограничения трафика необходимыми портами, не нагружая систему с отслеживанием состояния данными.

Внедрение средств защиты в общей инфраструктуре обеспечивает экономию за счет масштаба. Хотя предоставление значительных возможностей защиты для какого-либо отдельного сервиса может быть нерентабельным, общие средства защиты позволяют охватывать широкий спектр сервисов за раз. На рис. 10.1 показано, как направленная на один сайт атака вызвала для этого сайта больший объем трафика, чем обычно. Но этот объем все еще управляем, по сравнению с объемом трафика, полученного всеми пятью сайтами, защищенными Project Shield (<https://projectshield.withgoogle.com/>). У коммерческих служб по снижению риска DoS похожий подход к пакетированию для обеспечения экономически эффективного решения.

Особенно крупная DDoS-атака может нанести непоправимый урон дата-центру. Любая стратегия защиты должна гарантировать, что мощь распределенной атаки не будет сосредоточена на отдельном компоненте. Вы можете использовать балансировщики сетевой нагрузки и приложений для постоянного мониторинга входящего трафика и его распределения в ближайший центр обработки данных, имеющий доступные ресурсы для предотвращения этого типа перегрузки².

¹ Межсетевые экраны с сохранением состояния, выполняющие отслеживание соединений, лучше всего использовать для защиты серверов, которые отправляют исходящий трафик.

² Они могут отбрасывать трафик, если сервис находится в состоянии глобальной перегрузки.

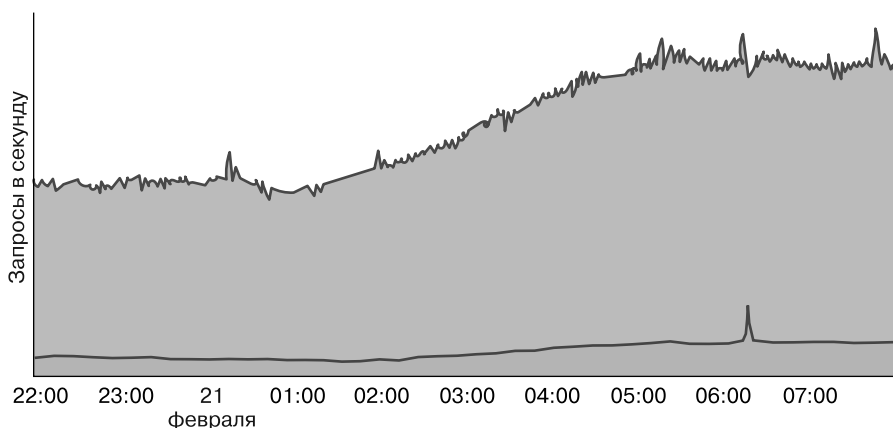


Рис. 10.1. DDoS-атака на сайт, защищенный Project Shield, если смотреть (сверху) с точки зрения отдельного сайта и (снизу) с точки зрения балансировщиков нагрузки Project Shield

Вы можете защитить общую инфраструктуру, не полагаясь на реактивную систему, используя *anycast* (<https://oreil.ly/m0HRU>), метод, при котором IP-адрес объявляется из нескольких мест. При использовании этой техники каждое местоположение привлекает трафик от расположенных рядом пользователей. В результате распределенная атака будет рассредоточена по всему миру, поэтому она не сможет объединить силы на каком-либо отдельном центре данных.

Защищаемые сервисы

Проектирование веб-сайта или приложения может сильно повлиять на вид защиты сервиса. Хотя постепенное ухудшение сервиса в условиях перегрузки обеспечивает лучшую защиту, можно сделать несколько простых изменений для повышения устойчивости к атакам и обеспечения серьезной экономии средств при нормальной работе.

Используйте прокси-кэширование

Использование *Cache-Control* и связанных заголовков позволит проводить повторные запросы контента, обрабатываемые прокси, чтобы каждый запрос не обязательно обращался к серверу приложения. Это относится к большинству статических изображений и даже может применяться к домашней странице.

Избегайте ненужных запросов приложений

Каждый запрос потребляет ресурсы сервера, поэтому лучше минимизировать их количество. Если веб-страница содержит несколько маленьких значков,

эффективнее разместить их все в одном (большем) изображении — техника, известная как *спрайт*¹. Дополнительное преимущество в том, что уменьшение количества запросов, отправляемых реальными пользователями сервису, уменьшит количество ложных срабатываний при выявлении вредоносных ботов.

Минимизируйте выходную пропускную способность

Во время попыток переполнить входную пропускную способность атака может привести к ее перегрузке, запросив большой ресурс. Изменение размера изображения до необходимого позволит сохранить выходную пропускную способность и сократить время загрузки страницы для пользователей. Еще один вариант — ограничение скорости или снижение приоритета неизбежно крупных ответов.

Предотвращение атак

Хотя хорошо защищенная архитектура дает возможность противостоять многим DoS-атакам, для предотвращения крупных или сложных атак вам может понадобиться дополнительная защита.

Мониторинг и оповещение

Время разрешения перебоев определяется двумя факторами: средним временем обнаружения (mean time to detection, MTTD) и средним временем восстановления (mean time to repair, MTTR). DoS-атака может привести к резкому увеличению загрузки ЦП сервера или к исчерпанию памяти приложением во время последовательных запросов. Для быстрой диагностики главной причины, в дополнение к использованию процессора и памяти, отслеживайте частоту запросов.

Оповещение о необычно высокой частоте запросов может четко указать команде реагирования на атаку. Но убедитесь в небезосновательности ваших оповещений. Если атака не наносит вреда пользователю, часто лучше просто принять ее. Оповещайте сотрудников, только когда спрос превышает пропускную способность сервиса и при запуске автоматической DoS-защиты.

Принцип предупреждения только в случае, когда может потребоваться вмешательство человека, применим и к атакам сетевого уровня. Многие Synflood-атаки

¹ Один из наших сервисных дизайнов закруглял углы для всех элементов пользовательского интерфейса. В первоначальном виде браузер извлекал изображения для каждого из четырех углов. Изменив сайт так, чтобы он загружал круг, а затем разделял изображение на стороне клиента, мы сэкономили 10 миллионов запросов в день.

могут быть поглощены автоматически, но на них нужно обратить внимание при запуске Syncookie¹. Точно так же широкополосные атаки заслуживают внимания, только если канал переполняется.

Постепенная деградация

При невозможности поглощения атаки вы должны минимизировать воздействие на пользователя.

Во время крупной атаки вы можете использовать сетевые ACL для регулирования подозрительного трафика, обеспечивая эффективный переключатель для немедленного ограничения трафика атаки. Важно не блокировать подозрительный трафик постоянно, чтобы вы могли сохранять видимость системы и минимизировать риск воздействия на допустимый трафик, соответствующий сигнатуре атаки. Если умный противник может имитировать допустимый трафик, регулирования будет недостаточно. Вы можете использовать средства контроля качеством обслуживания (quality-of-service, QoS) для определения приоритетности важного трафика. Использование более низкого QoS для менее важного трафика, такого как пакетные копии, может при необходимости высвободить пропускную способность для очередей с более высоким QoS.

При перегрузке приложения могут вернуться в ухудшенный режим. Google справляется с перегрузкой такими способами.

- Blogger работает в режиме только для чтения, отключая комментарии.
- Веб-поиск продолжает работать с ограниченным набором функций.
- DNS-серверы отвечают на столько запросов, сколько могут, но они спроектированы так, чтобы не зависать при любой нагрузке.

Дополнительные сведения об обработке перегрузки см. в главе 8.

Система защиты от DoS

Автоматизированные средства защиты, такие как регулирование верхних IP-адресов, обслуживание вызовов JavaScript или CAPTCHA, могут быстро и последовательно смягчить атаку. Так у команды реагирования на происше-

¹ При Synflood-атаке запросы на TCP-соединение отправляются с высокой скоростью, но не завершаются. Если на принимающем сервере не реализован механизм защиты, ему не хватит памяти для отслеживания всех входящих подключений. Распространенная защита — использование Syncookie, предоставляющих механизм без сохранения состояния для проверки новых соединений.

ствия есть время, чтобы понять проблему и определить, оправдано ли настраиваемое смягчение.

Автоматизированную систему защиты от DoS-атак можно разделить на два компонента.

Обнаружение

У системы должно быть четкое представление о входящем трафике с максимально возможной детализацией. Для этого нужна статистическая выборка на всех конечных точках с агрегированием до центральной системы управления. Система управления выявляет аномалии, способные указать на атаки, работая в сочетании с балансировщиками нагрузки, понимающими возможности сервиса, чтобы определить, оправдан ли ответ.

Ответ

Система должна иметь возможность реализовать защитный механизм, например, предоставляя набор IP-адресов для блокировки.

В любой крупномасштабной системе неизбежны ложноположительные (и ложноотрицательные) срабатывания. Особенно при блокировке по IP-адресу, ведь для нескольких устройств характерно использование одного сетевого адреса (например, при использовании трансляции сетевых адресов). Для уменьшения побочного ущерба других пользователей с тем же IP-адресом используйте CAPTCHA, чтобы реальные пользователи могли обходить блоки уровня приложения.

РЕАЛИЗАЦИЯ CAPTCHA



Обход CAPTCHA должен предоставить пользователю долгосрочное исключение, чтобы для следующих запросов не требовалось заново вводить CAPTCHA. Вы можете реализовать CAPTCHA без ввода дополнительных состояний сервера, используя cookie-файл браузера. Но обязательно настройте cookie-файл исключений для защиты от злоупотребления. Файлы cookie исключений в Google содержат такую информацию.

- Псевдоанонимный идентификатор, благодаря которому мы можем выявлять злоупотребления и отзываться исключения.
- Тип решенной проблемы, что позволяет нам требовать более сложных вызовов для более подозрительного поведения.
- Отметка времени решения проблемы, благодаря которой мы можем перестать пользоваться старыми файлами cookie.
- IP-адрес, решивший проблему, не позволяющий бот-сети совместно использовать одно исключение.
- Подпись, гарантирующая невозможность подделки файла cookie.

Учитывайте режимы сбоя в вашей системе защиты от DoS — проблемы могут быть вызваны атакой, изменением конфигурации, несвязанным сбоем инфраструктуры или другими причинами.

Система предотвращения DoS должна быть сама устойчивой к атакам. Поэтому избегайте зависимостей от производственной инфраструктуры, на которые могут повлиять DoS-атаки. Этот совет распространяется не только на сервис, но и на инструменты команды реагирования на происшествия и процедуры связи. DoS-атаки могут повлиять на Gmail или Google-документы, поэтому у Google есть резервные способы связи и хранилище инструкций.

Атаки часто приводят к немедленным отключениям. Постепенная деградация снижает влияние перегруженного сервиса, но лучше, если система предотвращения DoS может реагировать в считанные секунды, а не минуты. Эта характеристика создает естественную напряженность из-за лучшей практики медленного развертывания изменений для защиты от простоев. Но есть компромисс: прежде чем развертывать изменения повсеместно, мы можем отменить их все (включая автоматические ответы) в подмножестве нашей производственной инфраструктуры. Это не займет много времени — в некоторых случаях не больше одной секунды!

При выходе из строя центрального контроллера не всегда можно закрыть систему при сбое (это заблокирует весь трафик, приводящий к отключению) или открыть ее (это пропустит продолжающуюся атаку). Вместо этого мы переходим к статическому отказу. Это значит, что политика не меняется. Теперь система управления может завершить работу во время атаки (которая фактически произошла в Google!) не отключаясь. Из-за статического отказа механизм DoS не обязательно должен быть так же доступен, как инфраструктура внешнего интерфейса, что снижает затраты.

Стратегический ответ

При ответе на сбой может возникнуть желание реагировать оперативно и пытаться фильтровать текущий трафик атаки. Этот подход быстрый, но не оптимальный. Злоумышленники могут сдаться после неудачной первой попытки, но что, если они этого не сделают? Противник может без ограничений исследовать оборону и строить обходные пути. Стратегический ответ предпочитает не информировать средства злоумышленника о состоянии ваших систем. Например, однажды мы получили атаку, тривиально идентифицированную заголовком `User-Agent: I AM BOTNET` («я — бот-сеть»). Если бы мы просто отбросили весь

трафик с этой строкой, мы бы научили нашего противника использовать более правдоподобный *User-Agent*, такой как *Chrome*. Вместо этого мы перечислили IP-адреса, отправляющие этот трафик, и перехватили *все* их запросы с CAPTCHA в течение определенного периода времени. Из-за такого подхода противнику стало сложнее пользоваться А/В-тестированием (<https://oreil.ly/xuQrD>), чтобы узнать, как мы изолировали трафик атаки. Это заблаговременно заблокировало бот-сеть, так что даже отправка другого *User-Agent* не помогла бы.

Понимая возможности и цели злоумышленника, вы можете направлять вашу защиту. Небольшая усиленная атака говорит о том, что ваш противник может быть ограничен одним сервером, с которого он отправляет поддельные пакеты. Но DDoS-атака HTTP, запрашивающая одну и ту же страницу несколько раз, указывает на то, что у противника есть доступ к бот-сети. Иногда «атака» может быть непреднамеренной — ваш злоумышленник может просто пытаться вскрыть ваш сайт, неразумно его используя. В этом случае позаботьтесь о том, чтобы веб-сайт нельзя было с легкостью вскрыть.

Помните, что вы не одиноки — другие разработчики сталкиваются с такими же угрозами. Подумайте о сотрудничестве с другими организациями для улучшения средств защиты и возможности реагирования. Провайдеры по предотвращению DoS могут очищать некоторые типы трафика, сетевые провайдеры могут выполнять фильтрацию в восходящем направлении, а сообщество сетевых операторов может выявлять и фильтровать источники атак.

Борьба с внутренними атаками

Войдя в азарт, вы можете захотеть победить противника. Но что, если противника нет? У внезапного увеличения трафика есть и другие причины.

Поведение пользователя

В основном пользователи принимают независимые решения, и их поведение усредняется в плавную кривую спроса. Но внешние события могут синхронизировать их поведение. Например, если ночное землетрясение разбудит всех жителей населенного пункта, они могут внезапно обратиться к своим устройствам с целью найти информацию о безопасности, написать в социальные сети или позвонить друзьям. Эти одновременные действия могут привести к регистрации внезапного увеличения использования — как пик трафика, показанный на рис. 10.2.

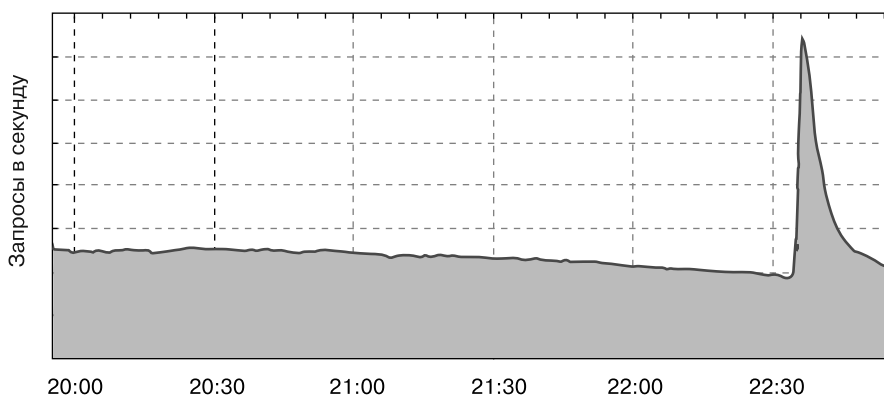


Рис. 10.2. Веб-трафик, измеряемый в HTTP-запросах в секунду, в инфраструктуре Google, обслуживающей пользователей в районе залива Сан-Франциско в ночь 14 октября 2019 года, когда в регионе произошло землетрясение магнитудой 4,5

БОТ ИЛИ НЕТ?

В 2009 году веб-поиск Google обнаружил сильный всплеск трафика, который длился около минуты. Несмотря на выходные, несколько SR-инженеров начали расследование. Мы получили очень странные результаты: все запросы содержали немецкие слова, начинавшиеся с одной буквы. Мы предположили, что это была бот-сеть, проводящая атаку на основе словаря.

Примерно через 10 минут атака повторилась (но с другим набором символов, с которых начиналось каждое слово), потом еще. Проведя углубленный анализ, мы начали сомневаться в нашем первоначальном подозрении по нескольким причинам.

- Запросы исходили от устройств в Германии.
- Запросы поступили из ожидаемой зоны локализации браузеров.

Интересно, может ли этот трафик исходить от реальных пользователей? Что заставило бы их вести себя столь необычно?

Позже мы выяснили: это было телевизионное игровое шоу. Конкурсантам давали начало слова и предлагали найти слово с ним, которое будет давать наибольшее количество результатов поиска в Google. Зрители в это время находились дома.

Мы вернулись к этой «атаке», изменив дизайн и запустив функцию, предлагающую завершение слов при вводе текста.

Поведение клиента при повторных попытках

Некоторые «атаки» бывают непреднамеренными и просто могут быть вызваны неправильным поведением ПО. Что произойдет, если клиент ожидает получить ресурс с вашего сервера, а он возвращает ошибку? Разработчик может подумать, что лучше всего повторить попытку. Это приведет к циклу, если сервер все еще обрабатывает ошибки. Если этот цикл затронет множество клиентов, восстановление после сбоя затруднится¹.

Клиентское ПО должно быть тщательно спроектировано во избежание циклов повторных попыток. В случае сбоя сервера клиент может повторить попытку, но он должен реализовать экспоненциальный откат. Например, удвоение периода ожидания при каждой неудачной попытке. Такой подход ограничивает количество запросов к серверу, но его недостаточно — сбой может синхронизировать запросы всех клиентов, вызывая многократные всплески большого трафика. Чтобы не было синхронных повторов, каждый клиент должен получать случайное время ожидания, называемое *флуктуацией*. В Google мы реализуем экспоненциальный откат с флуктуацией в большинстве клиентских программ.

Что вы можете сделать без контроля над клиентом? Это общая проблема для людей, работающих на официальных DNS-серверах. При сбое частота повторных попыток с допустимых рекурсивных DNS-серверов приведет к сильному увеличению трафика. Обычно он может быть в 30 раз больше, чем при нормальном использовании. Это затруднит восстановление после сбоя. Часто может быть трудно найти основную причину: операторы могут думать, что DDoS-атака — это причина, а не следствие. В таком сценарии лучше всего ответить на столько запросов, сколько возможно, поддерживая работоспособность сервера с помощью регулирования восходящего потока запросов. Каждый успешный ответ позволит клиенту выйти из цикла повторных попыток, и проблема будет решена.

Резюме

Каждый онлайн-сервис должен готовиться к DoS-атакам, даже если вы считаете, что они маловероятны. У каждой организации есть предел поглощаемого трафика, и задача защитника в максимально эффективном предотвращении атак, превышающих допустимую емкость.

¹ См. главу 22 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/addressing-cascading-failures/>).

Не забывайте об экономических ограничениях вашей DoS-защиты. Простое поглощение атаки может быть дорогим. Вместо этого, начиная с этапа проектирования, пользуйтесь экономически эффективными методами предотвращения. При атаке рассмотрите все варианты, включая блокировку проблемного хостинг-провайдера (который может содержать несколько реальных пользователей) или кратковременное отключение и объяснение ситуации пользователям. И помните, что «атака» может быть непреднамеренной.

Для реализации защиты на каждом уровне обслуживающего стека нужно сотрудничать с несколькими командами. Для некоторых команд DoS-защита не может быть главным приоритетом. Чтобы получить их поддержку, сосредоточьтесь на экономии средств и организационных упрощениях, которые может обеспечить система смягчения последствий DoS. Планирование емкости может быть сосредоточено на реальных потребностях пользователей, а не на поглощении самых крупных атак на каждом уровне стека. Фильтрация известных вредоносных запросов с использованием брандмауэра веб-приложений (web application firewall, WAF) позволяет команде безопасности сосредоточиться на новых угрозах. При обнаружении уязвимостей на уровне приложений одна система может блокировать попытки эксплуатации, поэтому команда разработчиков может подготовить исправление.

Благодаря тщательной подготовке вы можете определить функциональные возможности и режимы отказа сервиса на ваших условиях, а не на условиях противника.

ЧАСТЬ III

Внедрение систем

После того как вы проанализировали и спроектировали системы, пришло время для реализации ваших планов. Иногда реализация — это покупка готового решения. В главе 11 представлен один пример мыслительного процесса команды Google перед созданием специального программного решения.

Часть III посвящена внедрению безопасности и надежности на этапе реализации жизненного цикла разработки программного обеспечения. Глава 12 повторяет идею того, что фреймворки упростят ваши системы. Добавление статического анализа и нечеткого тестирования, как описано в главе 13, улучшит код. В главе 14 обсуждается, почему вы должны вкладываться в проверяемые сборки и дополнительные средства контроля. Меры предосторожности, связанные с программированием, сборкой и тестированием, имеют ограниченный эффект, если злоумышленники могут обойти их, достигнув производственной среды.

Даже если вся ваша цепочка поставки ПО устойчива к сбоям безопасности и надежности, вам все равно придется анализировать системы при возникновении проблем. В главе 15 обсуждается выверенный баланс, который нужно соблюдать между предоставлением доступа к отладке и требованиями безопасности хранения и доступа к журналам.

Пример: проектирование, разработка и поддержка публично доверенного центра сертификации

*Энди Уорнер, Джеймс Кастен,
Роб Смитс, Петр Кучарский
и Сергей Симаков*

SR-инженерам, разработчикам и администраторам иногда нужно взаимодействовать с центрами сертификации (ЦС), чтобы получить сертификаты для шифрования и аутентификации или для обработки таких функций, как VPN и подпись кода. Это исследование посвящено тому, как Google разрабатывает, внедряет и поддерживает общедоступный доверенный ЦС при помощи лучших практик, описанных в книге. Этот путь был обусловлен внутренними потребностями в масштабируемости, требованиями соответствия, безопасности и надежности.

Общие сведения о публично доверенных центрах сертификации

Публично доверенные центры сертификации действуют как якоря доверия для транспортного уровня Интернета. Они выпускают сертификаты для безопасности на транспортном уровне (Transport Layer Security, TLS)¹, S/MIME² и другие распространенные сценарии распределенного доверия. Это набор ЦС, которым браузеры, ОС и устройства доверяют по умолчанию. Так, написание

¹ Последняя версия TLS описана в RFC 8446 (<https://oreil.ly/dB0au>).

² Использование стандарта S/MIME (Secure/Multipurpose Internet Mail Extensions) — это распространенный метод шифрования содержимого электронной почты.

и поддержание публично доверенного ЦС поднимает ряд проблем безопасности и надежности.

Чтобы стать публично доверенным и поддерживать этот статус, ЦС должен соответствовать ряду критериев, охватывающих разные платформы и варианты использования. Доверенные ЦС должны пройти аудит на соответствие стандартам, таким как WebTrust, и тем, что установлены такими организациями, как Европейский институт стандартизации в области связи (ETSI) (<https://www.etsi.org/>). Публично доверенные центры сертификации тоже должны соответствовать целям базовых требований форума по ЦС/браузерам (<https://oreil.ly/gfdBF>). Эти оценки проверяют логические и физические средства контроля безопасности, процедуры и практики. Типичный публично доверенный ЦС тратит на эти аудиты не менее одного квартала в год. У большинства браузеров и ОС есть собственные уникальные требования, которым ЦС должен соответствовать, прежде чем ему будут доверять по умолчанию. ЦС должны быть адаптируемыми к изменениям требований и подстраиваться под изменения процесса или инфраструктуры.

Вашей организации вряд ли понадобится создавать публично доверенный центр сертификации¹. Большинство компаний полагаются на третьи стороны для получения общедоступных сертификатов TLS, сертификатов для подписания кода и других типов сертификатов, требующих широкого доверия пользователей. Цель такого примера не в том, чтобы показать вам, как создать публичный ЦС, а в том, чтобы выделить некоторые из наших выводов, которые могут найти отклик в проектах в вашей среде.

- Выбор языка программирования и решение использовать сегментацию или контейнеры при обработке созданных третьими сторонами данных обезопасили общую среду.
- Тщательное тестирование и усиление кода — как кода, который мы сгенерировали сами, так и стороннего — были решающими для избавления от серьезных проблем надежности и безопасности.
- Наша инфраструктура стала безопаснее и надежнее, когда мы уменьшили сложность проектирования и заменили ручные действия на автоматизированные.
- Понимание нашей модели угроз позволило создать механизмы проверки и восстановления, позволяющие нам лучше подготовиться к аварии.

¹ Мы понимаем, что многие организации создают и используют частные ЦС, пользуясь общими решениями, такими как службы сертификации Microsoft AD. Обычно они предназначены только для внутреннего использования.

Зачем нам нужен доверенный ЦС

Потребности нашего бизнеса в публично доверенном ЦС со временем изменялись. На заре Google мы приобрели все публичные сертификаты у стороннего центра сертификации. У этого подхода были три требующие решения проблемы.

Зависимость от третьих лиц

Бизнес-требования, определяющие высокий уровень доверия — например, чтобы предлагать облачные сервисы клиентам, — означали, что нам нужны строгие проверки и контроль над выпуском и обработкой сертификатов. Даже если мы проводили обязательные аудиты в экосистеме ЦС, мы не были уверены в соответствии третьих сторон высоким стандартам безопасности. Заметные упущения в безопасности в публично доверенных ЦС укрепили наши взгляды на нее¹.

Потребность в автоматизации

Google владеет тысячами доменов, обслуживающими пользователей по всему миру. В рамках наших вездесущих усилий по TLS (см. подраздел «Пример: увеличение использования HTTPS» в главе 7) мы хотели защитить каждый принадлежащий нам домен и часто менять сертификаты. Нашей целью было и предоставление клиентам простого способа получения сертификатов TLS. Автоматизировать получение новых сертификатов было сложно, потому что у сторонних публично доверенных ЦС часто не было расширяемых API или они не предоставляли SLA ниже наших потребностей. В итоге большая часть процесса запроса для этих сертификатов включала подверженные ошибкам ручные методы.

Стоимость

Учитывая миллионы TLS-сертификатов, которые компания Google хотела использовать для собственных веб-ресурсов и от имени клиентов, анализ затрат показал, что выгоднее было бы разрабатывать, внедрять и поддерживать свой ЦС, чем продолжать получать сертификаты от третьих сторон.

Создать или купить?

После решения Google об использовании доверенного ЦС мы встали перед выбором: покупать коммерческое ПО для работы ЦС или писать свое. В итоге мы решили разработать ядро ЦС самостоятельно, с возможностью интегрировать

¹ DigiNotar обанкротился (<https://oreil.ly/nwNnG>) после того, как злоумышленники скомпрометировали и неправильно использовали его ЦС.

открытые и коммерческие решения, если понадобится. Помимо ряда значимых факторов за этим решением стояло несколько основных причин.

Прозрачность и валидация

Коммерческие решения для ЦС часто не сопровождались аудитом кода или цепочки поставок, необходимым нам для такой критически важной инфраструктуры. Несмотря на то, что наше программное обеспечение ЦС было интегрировано с библиотеками с открытым исходным кодом и использовало сторонний проприетарный код, его разработка и тестирование дали нам уверенность в создаваемой системе.

Возможности интеграции

Мы хотели упростить внедрение и обслуживание ЦС через интеграцию с безопасной критически важной инфраструктурой Google. Например, мы могли настроить регулярное резервное копирование в Spanner (<https://oreil.ly/ZnhV->), добавив одну строку в файл конфигурации.

Гибкость

Более широкое интернет-сообщество разрабатывает новые инициативы, обеспечивающие повышенную безопасность экосистемы. Certificate Transparency (<https://www.certificate-transparency.org/>) — инструмент для мониторинга и аудита сертификатов, а также проверка домена с использованием DNS, HTTP и других методов¹ — два канонических примера. Мы хотели первыми внедрить подобные инициативы, и пользовательский ЦС лучше всего подходил для быстрого добавления этой гибкости.

Вопросы проектирования, реализации и обслуживания

Для обеспечения безопасности нашего ЦС мы создали трехуровневую связанную архитектуру. В ней каждый уровень отвечает за свою часть процесса выдачи: анализ запроса сертификата, функции регистрационного органа (маршрутизация и логика) и подписание сертификата. Каждый уровень состоит из микросервисов с четко определенными обязанностями. Мы разработали архитектуру с двойной зоной доверия. Здесь ненадежный ввод обрабатывается в среде, отличной от среды для критических операций. Эта сегментация создает тщательно определенные границы, помогающие понять и упростить анализ. Архитектура

¹ Хороший справочник с рекомендациями по проверке домена — базовые требования форума по ЦС/браузерам (<https://oreil.ly/OkYRq>).

тоже уменьшает возможность атаки. Каждый компонент ограничен в функциональности, поэтому злоумышленник, получающий к нему доступ, тоже будет ограничен в функциональности, на которую он может повлиять. Для получения дополнительного доступа противнику придется обойти дополнительные контрольные точки.

Простота — главный принцип каждого разработанного и реализованного микро-сервиса. В течение срока службы ЦС мы постоянно проводим рефакторинг каждого компонента с учетом простоты. Мы строго тестируем и проверяем код (как наш, так и сторонний) и данные. Мы упаковываем код в контейнеры, если это повысит безопасность. Далее подробнее описывается наш подход к решению вопросов безопасности и надежности с помощью качественного проектирования и выбора реализации.

РАЗВИВАЮЩИЕСЯ ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИИ

В этом тематическом исследовании несколько идеализированная картина хорошего проектирования системы. На самом деле обсуждаемые нами проектные решения были созданы в течение почти десятилетия, в течение трех отдельных итераций проектирования. Новые требования и практики появлялись за большой промежуток времени. Часто невозможно спроектировать и реализовать все аспекты безопасности и надежности с самого начала проекта. Но уже в начале важно установить хорошие принципы. Вы можете начать с малого и непрерывно перебирать свойства безопасности и надежности системы для достижения долгосрочной устойчивости.

Выбор языка программирования

Выбор языка программирования для частей системы, принимающих произвольный ненадежный ввод, был важным аспектом проектирования. По итогу мы решили написать ЦС на смеси Go и C++ и выбрали язык для каждого подкомпонента в зависимости от его цели. И Go, и C++ совместимы с хорошо протестированными криптографическими библиотеками, показывают отличную производительность и предлагают сильную экосистему фреймворков и инструментов для выполнения общих задач.

Так как Go безопасен в части доступа к памяти (memory-safe), у него есть дополнительные преимущества безопасности при обработке ЦС произвольного ввода. Например, запросы на подпись сертификата (Certificate Signing Requests, CSR) (<https://oreil.ly/8YkPI>) — это ненадежные входные данные в ЦС. CSR могут поступать от одной из наших потенциально безопасных внутренних систем или от интернет-пользователя (возможно, даже злоумышленника). У связанных с памятью

уязвимостей в коде, анализирующем стандарт DER (Distinguished Encoding Rules, формат кодирования, используемый для сертификатов)¹, долгая история. Поэтому мы хотели использовать безопасный для памяти язык, обеспечивающий дополнительную безопасность. Go (https://oreil.ly/WM_zw) отвечает всем этим требованиям.

C++ не безопасен для памяти, но обладает хорошей совместимостью для критических подкомпонентов системы, особенно для определенных компонентов базовой инфраструктуры Google. Для защиты этого кода мы запускаем его в безопасной зоне и проверяем все данные. Например, для обработки CSR мы анализируем запрос в Go перед его передачей подсистеме C++ для той же операции, а после сравниваем результаты. Если есть расхождение, обработку не продолжаем.

Мы применяем передовые методы обеспечения безопасности и удобочитаемости (<https://oreil.ly/m8dug>) для всего кода на C++, а централизованный инструментарий Google обеспечивает разные способы уменьшить время сбора данных и выполнения. К ним относятся следующие.

W^X (<https://oreil.ly/9gNIa>)

Сводит на ноль известный трюк использования `mmaping` с `PROT_EXEC` путем копирования `shell`-кода и перехода в эту память. Такая мера защиты не приводит к снижению производительности процессора или памяти.

Scudo Allocator (<https://oreil.ly/xpo5t>)

Распределитель кучи в пользовательском режиме.

SafeStack (<https://oreil.ly/EPwod>)

Техника повышения безопасности, оберегающая от атак, основанных на переполнении буфера стека.

Сложность в сравнении с понятностью

В качестве защитной меры мы явно выбрали реализацию нашего ЦС с ограниченной функциональностью по сравнению с полным набором доступных в стандартах опций (см. раздел «Проектирование понятных систем» главы 6). Нашим основным вариантом использования была выдача сертификатов для стандартных веб-сервисов с часто используемыми атрибутами и расширениями. Наша оценка вариантов коммерческого ПО и ПО с открытым исходным

¹ У базы данных Miter CVE (<https://cve.mitre.org/>) сотни уязвимостей, обнаруженных в различных обработчиках DER.

кодом для ЦС показала, что их попытки приспособить эзотерические атрибуты и ненужные нам расширения привели к затруднившему проверку усложнению системы, более подверженному ошибкам. Поэтому мы решили написать ЦС с ограниченной функциональностью и большей понятностью для более легкого отслеживания ожидаемых входных и выходных данных.

Мы постоянно работаем над упрощением архитектуры ЦС, чтобы сделать ее более понятной и обслуживаемой. В одном случае мы поняли, что в нашей архитектуре создано слишком много разных микросервисов и это привело к увеличению затрат на обслуживание. Мы планировали получить преимущества модульного сервиса с четко определенными границами, но обнаружили, что проще объединить некоторые части системы. В другом случае мы поняли, что наши проверки ACL для вызовов RPC каждый раз были реализованы вручную. Это могло привести к ошибкам разработчика и рецензента. Мы реорганизовали кодовую базу для централизации проверки ACL и исключения возможности добавления новых RPC без ACL.

Защита сторонних компонентов и компонентов с открытым исходным кодом

Наш пользовательский центр сертификации опирается на сторонний код в форме коммерческих модулей и библиотек с открытым исходным кодом. Нам требовалось проверить, укрепить и упаковать этот код в контейнеры. Для начала мы сосредоточились на нескольких известных и широко используемых пакетах с открытым исходным кодом, использующих ЦС. Даже пакеты с открытым исходным кодом, широко применяемые в контексте безопасности и поступающие от опытных в плане защиты разработчиков или организаций, подвержены уязвимостям. После тщательного анализа безопасности каждого из них мы представили исправления для устранения обнаруженных проблем. По возможности мы подвергали тестированию все сторонние компоненты и компоненты с открытым исходным кодом. Оно подробно описано в следующем разделе.

Использование двух безопасных зон — одной для обработки ненадежных данных и одной для обработки конфиденциальных операций — тоже дает нам многоуровневую защиту от ошибок или вредоносных вставок в код. Анализатор CSR использует библиотеки с открытым исходным кодом X.509 и работает как микросервис в недоверенной зоне в контейнере Borg¹. Это обеспечивает дополнительный уровень защиты от проблем в этом коде.

¹ Контейнеры Borg описаны в статье: *Verma A. et al. Large-Scale Cluster Management at Google with Borg* // Proceedings of the 10th European Conference on Computer Systems, 2015. P. 1–17. doi: 10.1145/2741948.2741964.

Мы должны были обеспечить и безопасность стороннего закрытого исходного кода. Для запуска публично доверенного ЦС нужно воспользоваться аппаратным модулем безопасности (*hardware security module*, HSM) — выделенным криптографическим процессором, предоставляемым коммерческим поставщиком в качестве хранилища, защищающего ключи ЦС. Мы хотели предоставить дополнительный уровень проверки для полученного от поставщика кода, взаимодействующего с HSM. Как и во многих решениях, предоставляемых извне, мы могли использовать ограниченное количество видов тестирования. Для защиты систем от таких проблем, как утечки памяти, мы предприняли следующие шаги.

- Создали части ЦС, которые должны были защищенно взаимодействовать с библиотеками HSM, так как знали о потенциальном риске для входных и выходных данных.
- Запустили сторонний код в *nsjail* (<https://oreil.ly/QaE4s>), облегченном механизме изоляции процессов.
- Сообщили о найденных проблемах поставщику.

Тестирование

Для обеспечения чистоты проекта мы пишем модульные и интеграционные тесты (см. главу 13), охватывая широкий спектр сценариев. Ожидается, что члены команды напишут эти тесты как часть процесса разработки, и проверка кода гарантирует соблюдение этой практики. Мы тестируем не только ожидаемое поведение, но и негативные условия (о ненаступлении события или несовершении действия). Каждые несколько минут мы генерируем условия выдачи тестовых сертификатов — одни отвечают всем требованиям, а другие содержат вопиющие ошибки. К примеру, мы явно проверяем, что точные сообщения об ошибках вызывают сигналы тревоги, когда их выдает неавторизованное лицо. Благодаря сборнику положительных и отрицательных условий тестирования мы можем очень быстро выполнять надежное сквозное тестирование во всех новых развертываниях программного обеспечения ЦС.

С помощью централизованных наборов инструментов разработки ПО в Google мы получаем преимущества интегрированного автоматизированного тестирования кода перед отправкой и после сборки. Как обсуждается в подразделе «Интеграция статического анализа в рабочий процесс» в главе 13, все изменения кода в Google проверяются Tricorder, нашей платформой статического анализа. Мы подвергаем код ЦС различным средствам очистки кода, таким как AddressSanitizer (ASAN) и ThreadSanitizer, для выявления распространенных ошибок (см. раздел «Динамический анализ программ» главы 13). Кроме того, мы выполняем целенаправленное нечеткое тестирование кода ЦС (см. раздел «Нечеткое тестирование» главы 13).

ИСПОЛЬЗОВАНИЕ ПЕРЕДЕЛОК ДЛЯ ЗАЩИТЫ ЦС

В рамках наших постоянных усилий по улучшению безопасности мы участвуем в инженерно-технических *переделках*. Переделки — инженерная традиция Google, когда мы на некоторое время откладываем все обычные рабочие задачи и объединяемся для достижения общей цели. Одна из таких переделок была направлена на нечеткое тестирование, при котором мы интенсивно тестировали части ЦС. При этом в анализаторе X.509 в Go мы обнаружили проблему, способную привести к сбою приложения. Мы извлекли ценный урок: относительно новая библиотека Go для анализа X.509 не была проверена так же, как ее старшие коллеги (такие как OpenSSL и BoringSSL). Этот опыт показал, как специальные переделки (в данном случае *перетесты*) позволяют выявить свет на библиотеки, требующие дальнейшего тестирования.

Устойчивость набора ключей ЦС

Наиболее серьезный риск для ЦС — кража или неправильное использование наборов ключей. Большинство обязательных мер безопасности для публично доверенного ЦС решают общие проблемы, способные привести к такому злоупотреблению. Они содержат стандартные рекомендации, такие как использование HSM и строгий контроль доступа.

Мы храним основные наборы ключей ЦС в автономном режиме и защищаем их несколькими уровнями физической защиты, требующими двухсторонней авторизации для каждого уровня доступа. Для повседневной выдачи сертификатов мы используем промежуточные ключи, доступные только онлайн. Это стандартная практика в отрасли. Процесс включения публично доверенного ЦС в экосистему (то есть в браузеры, телевизоры и мобильные телефоны) может занять годы. Поэтому замена ключей в рамках процесса восстановления после компрометации (см. подраздел «Замена ключей подписи» в главе 9) не простое или своевременное действие. Как вы понимаете, потеря или кража набора ключей может привести к серьезным сбоям. Для защиты от этого сценария мы разрабатываем другой основной набор ключей в экосистеме (распространяя его среди браузеров и других клиентов, использующих зашифрованные соединения), чтобы при необходимости обмениваться им.

Проверка данных

Помимо потери набора ключей, ошибки выдачи — самые серьезные ошибки, которые может допустить ЦС. Мы стремились спроектировать наши системы так, чтобы ограничить влияние человеческого восприятия на проверку или выдачу. Это значит, что мы можем сосредоточить наше внимание на правильности и надежности кода и инфраструктуры ЦС.

Непрерывная проверка (см. раздел «Непрерывная проверка» главы 8) гарантирует ожидаемое поведение системы. Для реализации такой проверки в публично доверенном ЦС в Google мы автоматически пропускаем сертификаты через инструменты статического анализа на нескольких этапах процесса выдачи¹. Инструменты статического анализа проверяют наличие шаблонных ошибок, гарантируя, что сертификаты имеют действительный срок действия или что `subject:commonName` допустимой длины. После подтверждения сертификата мы вписываем его в журналы прозрачности сертификатов. Это позволяет проводить постоянную публичную проверку. Для окончательной защиты от злонамеренного выпуска мы используем несколько независимых систем ведения журналов, благодаря которым можем сравнить две системы запись за записью для обеспечения согласованности. Эти журналы подписываются до попадания в хранилище для дальнейшей безопасности и последующей проверки при необходимости.

Резюме

Центры сертификации — пример инфраструктуры, предъявляющей строгие требования к безопасности и надежности. Использование лучших практик для реализации инфраструктуры, изложенных в этой книге, приведет к долгосрочным положительным результатам для безопасности и надежности. Эти принципы должны быть частью разработки на раннем этапе, но можно использовать их для улучшения систем и по мере развития.

¹ Например, ZLint (<https://github.com/zmap/zlint>) — это инструмент статического анализа, написанный на Go, проверяющий, что содержимое сертификата соответствует RFC 5280 и требованиям форума по ЦС/браузерам.

ГЛАВА 12

Написание кода

*Михал Чапинский и Юлиан Бангерт
с Томасом Мауфером
и Кавитой Гулиани*

В коде неизбежно будут ошибки. Но избежать распространенных уязвимостей безопасности и проблем с надежностью можно при помощи усиленных фреймворков и библиотек, устойчивых к таким проблемам.

В этой главе мы делимся шаблонами разработки ПО, которые нужно применять во время реализации проекта. Сначала мы рассмотрим пример с серверным RPC, изучим, как фреймворки помогают автоматически обеспечивать требуемые свойства безопасности и противостоять типичным антипаттернам надежности. Мы сосредоточимся на простоте кода — научимся контролировать накопление технического долга и при необходимости делать рефакторинг кодовой базы. В конце вас ждут советы о том, как правильно выбрать инструменты и использовать по максимуму языки разработки.

Безопасность и надежность не могут быть легко интегрированы в ПО. Поэтому важно учитывать их при разработке на самых ранних этапах. Добавление этих функций после запуска болезненно, менее эффективно и может потребовать изменения других важных предположений о кодовой базе (подробнее это описано в главе 4).

Первый и самый важный шаг в уменьшении проблем безопасности и надежности — обучение разработчиков. Но даже самые подготовленные инженеры могут ошибаться — эксперты по безопасности могут писать небезопасный код, а SR-инженеры могут пропускать проблемы с надежностью. Трудно одновременно учитывать множество соображений и компромиссов, связанных с созданием безопасных и надежных систем, особенно если вы ответственны и за создание ПО.

Не полагайтесь только на разработчиков для проверки кода на предмет безопасности и надежности. Вместо этого поручите SR-инженерам и экспертам по безопасности анализировать код и программные разработки. У такого подхода тоже есть недостатки. Проверки кода вручную не найдут всех пробелов, и ни один рецензент не обнаружит *все* потенциальные проблемы безопасности. В силу своего опыта или интересов рецензенты могут стремиться найти новые классы атак, проблемы высокого уровня в проектных решениях или интересные недостатки в криптографических протоколах. А вот просмотр сотен шаблонов HTML на наличие ошибок межсайтового скриптинга (XSS) или проверка логики обработки ошибок для каждого RPC в приложении — не самые интересные задачи.

Проверки кода могут найти не все уязвимости, но у них есть и другие преимущества. Развитая культура проверок побуждает разработчиков структурировать свой код так, чтобы можно было легко просматривать свойства безопасности и надежности. В этой главе мы обсуждаем стратегии, позволяющие сделать эти свойства очевидными для рецензентов и внедрения автоматизации в процесс разработки. Эти стратегии помогут разгрузить команду разработчиков. Тогда они смогут сосредоточиться на других вопросах. Это приведет к формированию культуры безопасности и надежности (см. главу 21).

Основы обеспечения безопасности и надежности

Как уже говорилось в главе 6, безопасность и надежность приложения зависят от заданных для домена инвариантов. Например, приложение защищено от атак с использованием внедрения SQL, если все его запросы к БД состоят только из управляемого разработчиком кода, а внешние входные данные передаются через привязки параметров запроса. Веб-приложение может предотвратить XSS-атаки, если весь пользовательский ввод, вставленный в HTML-формы, правильно экранирован или не позволяет встраивать любой исполняемый код.

РАСПРОСТРАНЕННЫЕ ИНВАРИАНТЫ БЕЗОПАСНОСТИ И НАДЕЖНОСТИ

Почти у любого многопользовательского приложения есть инварианты безопасности, определяющие, какие действия с данными пользователи могут производить. Каждое действие должно последовательно поддерживать эти инварианты. Для предотвращения каскадных сбоев в распределенной системе приложение должно следовать разумным политикам, таким как откат повторных попыток при сбое RPC. Точно так же, чтобы избежать повреждения памяти и проблем с безопасностью, программы на C++ должны обращаться только к допустимым местам в памяти.

В теории вы можете создать безопасное и надежное ПО, тщательно написав код приложения, поддерживающий эти инварианты. Но по мере роста числа желаемых свойств и размера кодовой базы это будет сделать почти невозможно. Неразумно ожидать от разработчика, что он станет экспертом во всех этих областях или будет постоянно сохранять бдительность при написании или обзоре кода.

При самостоятельном просмотре каждого изменения людям будет сложно поддерживать глобальные инварианты, ведь рецензенты не всегда могут отслеживать общий контекст. Если рецензенту нужно знать, какие параметры функции передаются пользователем, а какие аргументы содержат только контролируемые разработчиком доверенные значения, он должен быть знаком со всеми переходными вызывающими функциями. Рецензенты вряд ли смогут сохранить это состояние в долгосрочной перспективе.

Лучший подход — обеспечение безопасности и надежности в общих фреймворках, языках и библиотеках. В идеале библиотеки предоставляют только интерфейс, препятствующий написанию кода с общими классами уязвимостей безопасности. Несколько приложений могут использовать все библиотеки или фреймворки. Когда эксперты решают проблему, они избавляются от нее во всех приложениях, поддерживаемых фреймворком. Это позволяет лучше масштабировать такой инженерный подход. Вероятность проникновения будущих уязвимостей будет ниже, если вместо проверки вручную использовать централизованный защищенный фреймворк. Конечно, ни один фреймворк не может защитить от всех уязвимостей безопасности, и злоумышленники все еще могут обнаружить непредвиденный класс атак ошибки в реализациях фреймворков. Но если вы найдете новую уязвимость, ее можно будет устранить в одном месте (или нескольких), а не по всей кодовой базе.

Приведем конкретный пример: внедрение SQL (SQL injection, SQLI) занимает первое место в списках распространенных уязвимостей безопасности OWASP (<https://oreil.ly/TnBaK>) и SANS (<https://oreil.ly/RWvPF>). По нашему опыту, когда вы используете защищенную библиотеку данных, такую как `TrustedSqlString` (см. подраздел «Уязвимости SQL-внедрений: `TrustedSqlString`» далее в этой главе), эти типы уязвимостей становятся незначительными. Типы делают эти предположения явными и автоматически применяются компилятором.

Преимущества использования фреймворков

Большинство приложений имеют схожие строительные блоки для обеспечения безопасности (аутентификация и авторизация, ведение журнала, шифрование данных) и надежности (ограничение скорости, балансировка нагрузки, логика повторных попыток). Разработка и поддержка таких блоков с нуля

для каждого сервиса дорого стоит и приводит к разным ошибкам в каждом отдельном сервисе.

Благодаря фреймворкам можно повторно использовать код. Чтобы не учитывать все влияющие на функциональность аспекты безопасности и надежности, разработчикам нужно только настроить конкретный блок. Например, разработчик может указать, какая информация из учетных данных входящего запроса важна для авторизации, не беспокоясь о достоверности этой информации, ведь она подтверждается фреймворком. Разработчик может указать и необходимые для регистрации данные, не беспокоясь о хранении или копировании. Фреймворки облегчают и распространение обновлений, так как достаточно применить обновление только в одном месте.

Используя фреймворки, вы повышаете производительность всех разработчиков в организации. Это способствует формированию культуры безопасности и надежности (см. главу 21). Группа экспертов эффективнее спроектирует и разработает строительные блоки для фреймворка, чем отдельная команда самостоятельно реализует функции безопасности и надежности. Если команда безопасности занимается криптографией, все остальные могут использовать эти знания. Разработчики, использующие фреймворки, могут не беспокоиться о внутренних деталях, а вместо этого сосредоточиться на бизнес-логике приложения.

Фреймворки сильнее повышают производительность, предлагая инструменты, с которыми легко интегрироваться. Они могут предоставлять инструменты, автоматически экспортирующие основные операционные метрики, такие как общее количество запросов, количество неудачных запросов с разбивкой по типу ошибки или задержке каждого этапа обработки. Вы можете воспользоваться этими данными для создания автоматических панелей мониторинга и оповещений для сервиса. Фреймворки упрощают интеграцию с инфраструктурой балансировки нагрузки. Поэтому сервис может автоматически перенаправлять трафик с перегруженных экземпляров или ускорять работу новых при большой нагрузке. В итоге построенные поверх фреймворков сервисы показывают более высокую надежность.

Использование фреймворков упрощает работу с кодом. Оно четко отделяет бизнес-логику от общих функций. Благодаря этому разработчики могут уверенно делать заявления о безопасности или надежности сервиса. В целом фреймворки снижают сложность — когда код между несколькими сервисами однороднее, легче следовать общепринятым практикам.

Необязательно всегда разрабатывать собственные фреймворки. Во многих случаях лучше повторно использовать существующие решения. Почти любой

специалист по безопасности посоветует вам не проектировать и не внедрять свою криптографическую среду. Вместо этого вы можете взять хорошо зарекомендовавшую себя и широко используемую среду, такую как Tink (обсуждается в пункте «Пример: безопасные криптографические API и криптографический фреймворк Tink» в главе 6).

До использования конкретного фреймворка оцените его состояние безопасности. Мы предлагаем использовать активно поддерживаемые фреймворки и постоянно обновлять зависимости кода для добавления последних обновлений безопасности для любого кода, от которого зависит ваш собственный.

Рассмотрим практический пример, демонстрирующий преимущества фреймворков: фреймворк для создания серверов RPC.

Пример: фреймворк для серверов RPC

У многих серверов RPC схожая структура. Они обрабатывают специфичную для запросов логику и выполняют следующие действия:

- регистрация;
- аутентификация;
- авторизация;
- регулирование (ограничение скорости).

Вместо реализации этой функциональности для каждого отдельного сервера RPC используйте фреймворк, способный скрыть детали реализации этих строительных блоков. Тогда разработчики просто должны настроить каждый шаг в соответствии с потребностями своего сервиса.

На рис. 12.1 показана возможная архитектура фреймворка, основанная на предварительно определенных *перехватчиках* (*interceptors*), отвечающих за каждый из ранее упомянутых этапов. Вы можете использовать перехватчики и для пользовательских шагов. Каждый перехватчик определяет действие, которое должно быть произведено *до* и *после* выполнения конкретной логики RPC. Каждый этап может сообщить об ошибке, препятствующей дальнейшему внедрению перехватчиков. Но когда это происходит, *следующие* шаги каждого уже использованного перехватчика выполняются в обратном порядке. Фреймворк между перехватчиками может прозрачно выполнять дополнительные действия. Например, экспортировать статистику ошибок или метрики производительности. Эта архитектура приводит к четкому разделению логики, выполняемой на каждом этапе, благодаря чему повышается простота и надежность.

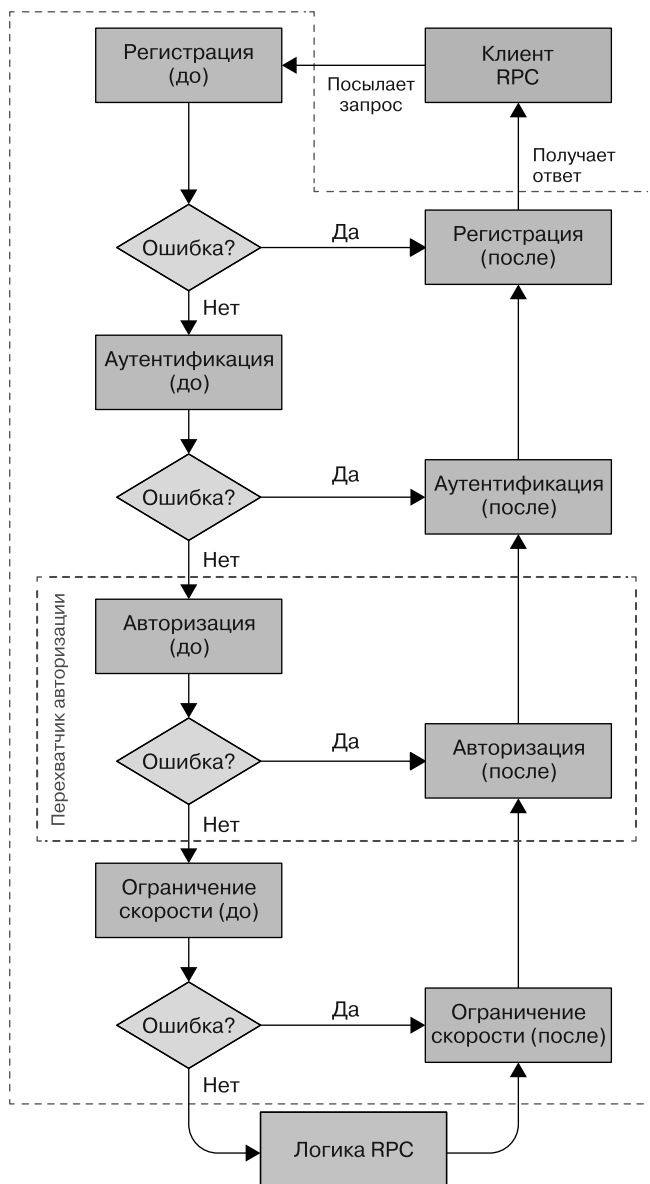


Рис. 12.1. Поток управления в потенциальном фреймворке для серверов RPC: типичные этапы инкапсулированы в predetermined перехватчики. Для примера выделена авторизация

В этом примере *предшествующий* этап перехватчика ведения журнала может регистрировать вызов, а *последующий* — состояние операции. Теперь, если запрос не авторизован, логика RPC не выполняется, но ошибка «Отказано в доступе»

регистрируется правильно. Далее система вызывает этапы перехватчиков аутентификации и регистрации (даже если они пусты) и только *после* отправляет ошибку клиенту.

Перехватчики разделяют состояние через *объект контекста*, который они передают друг другу. Например, *перед* этапом аутентификации перехватчик может обрабатывать все криптографические операции, связанные с обработкой сертификатов (обратите внимание на повышенную безопасность от повторного использования специализированной криптографической библиотеки, а не самостоятельной ее реализации). После система упаковывает извлеченную и проверенную информацию о вызывающем объекте во вспомогательный объект, добавленный ею в контекст. Следующие перехватчики могут легко получить доступ к этому объекту.

Далее фреймворк может использовать объект контекста для отслеживания времени выполнения запроса. Если на каком-то этапе ясно, что запрос не будет выполнен раньше установленного срока, система может автоматически отменить его. Уведомив клиента, вы повысите надежность и сэкономите ресурсы.

Хороший фреймворк должен позволять работать с зависимостями сервера RPC. Например, с другим сервером, отвечающим за хранение журналов. Вы можете зарегистрировать их как мягкие или жесткие зависимости, и фреймворк будет постоянно отслеживать их доступность. Если он обнаруживает, что жесткая зависимость недоступна, фреймворк может остановить сервис. После этого он сообщает о собственной недоступности и автоматически перенаправляет трафик в другие экземпляры.

Рано или поздно перегрузка, проблемы с сетью или другие неполадки приведут к недоступности зависимости. Часто лучше повторить запрос, но выполняйте повторы осторожно, чтобы избежать *каскадного сбоя* (схожего с падением домино)¹. Самое популярное решение — повторная попытка *экспоненциального отката*². Хороший фреймворк должен обеспечивать поддержку такой логики, а не требовать от разработчика реализации логики для каждого вызова RPC.

Среда, корректно обрабатывающая недоступные зависимости и перенаправляющая трафик во избежание перегрузки сервиса или его зависимостей, повышает надежность самого сервиса и всей экосистемы. Эти улучшения требуют минимального участия разработчиков.

¹ См. главу 22 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/addressing-cascading-failures/>), чтобы подробнее узнать о каскадных сбоях.

² Также описано в главе 22 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/addressing-cascading-failures/>).

Фрагменты кода. Примеры с 12.1 по 12.3 показывают перспективы разработчика серверов RPC при работе с фреймворком, ориентированным на безопасность или надежность. Они написаны на Go и используют буферы протоколов Google (<https://oreil.ly/yzES2>).

Пример 12.1. Начальные определения типов (предшествующий этап перехватчика может изменить контекст. Перехватчик аутентификации может добавить проверенную информацию о вызывающем объекте)

```
type Request struct {
    Payload proto.Message
}

type Response struct {
    Err error
    Payload proto.Message
}

type Interceptor interface {
    Before(context.Context, *Request) (context.Context, error)
    After(context.Context, *Response) error
}

type CallInfo struct {
    User string
    Host string
    ...
}
```

Пример 12.2. Пример перехватчика авторизации, разрешающего только запросы от пользователей из белого списка

```
type authzInterceptor struct {
    allowedRoles map[string]bool
}

func (ai *authzInterceptor) Before(ctx context.Context, req *Request)
(context.Context, error) {
    // Переменная callInfo была заполнена фреймворком
    callInfo, err := FromContext(ctx)
    if err != nil { return ctx, err }

    if ai.allowedRoles[callInfo.User] { return ctx, nil }
    return ctx, fmt.Errorf("Unauthorized request from %q", callInfo.User)
}

func (*authzInterceptor) After(ctx context.Context, resp *Response) error {
    return nil // После обработки RPC здесь больше нечего делать
}
```

Пример 12.3. Пример перехватчика регистрации, который регистрирует каждый входящий запрос (этап «до»), а потом все неудавшиеся запросы с их статусами (этап «после»). `WithAttemptCount` — это предоставляемая фреймворком опция вызова RPC, реализующая экспоненциальный откат

```
type logInterceptor struct {
    logger *LoggingBackendStub
}

func (*logInterceptor) Before(ctx context.Context,
    req *Request) (context.Context, error) {
    // Переменная callInfo была заполнена фреймворком
    callInfo, err := FromContext(ctx)
    if err != nil { return ctx, err }
    logReq := &pb.LogRequest{
        timestamp: time.Now().Unix(),
        user: callInfo.User,
        request: req.Payload,
    }
    resp, err := logger.Log(ctx, logReq, WithAttemptCount(3))
    return ctx, err
}

func (*logInterceptor) After(ctx context.Context, resp *Response) error {
    if resp.Err == nil { return nil }

    logErrorReq := &pb.LogErrorRequest{
        timestamp: time.Now().Unix(),
        error: resp.Err.Error(),
    }
    resp, err := logger.LogError(ctx, logErrorReq, WithAttemptCount(3))
    return err
}
```

Распространенные уязвимости безопасности

В больших кодовых базах несколько классов составляют большинство уязвимостей безопасности, несмотря на постоянные усилия по обучению разработчиков и внедрению анализа кода. OWASP и SANS публикуют списки общих классов уязвимостей. В табл. 12.1 перечислены десять самых распространенных рисков уязвимости в соответствии с OWASP (<https://oreil.ly/bUZq8>) и некоторые возможные подходы к снижению каждого из них на базовом уровне.

Таблица 12.1. Десять самых распространенных рисков уязвимости в соответствии с OWASP

Уязвимость	Методы защиты из фреймворков
Внедрение [SQL]	TrustedSQLString (см. следующий подраздел)
Проблемы с аутентификацией	Нужна аутентификация с использованием хорошо проверенного механизма, такого как OAuth, до направления запроса приложению (см. подраздел «Пример: фреймворк для серверов RPC» выше в главе)
Раскрытие конфиденциальных данных	Используйте разные типы (вместо строк) для хранения и обработки конфиденциальных данных, таких как номера кредитных карт. Этот подход может ограничить преобразования для предотвращения утечки данных и обеспечения надлежащего шифрования. Фреймворки могут дополнительно гарантировать прозрачную защиту при передаче, например HTTPS с LetsEncrypt. Криптографические API, такие как Tink, могут предоставлять подходящее секретное хранилище, например, загрузку ключей из облачной системы управления ключами вместо файла конфигурации
Внешние сущности XML (XXE)	Используйте анализатор XML без включенного XXE. Убедитесь, что эта рискованная функция отключена в поддерживающих ее библиотеках*
Проблемы с контролем доступа	Это сложные проблемы, потому что они часто зависят от приложения. Используйте фреймворк, требующий от каждого обработчика запросов или RPC наличия четко определенных ограничений контроля доступа. По возможности передайте учетные данные конечного пользователя серверу и примените политику контроля доступа на сервере
Неправильная настройка безопасности	Используйте стек технологий, который по умолчанию обеспечивает безопасные конфигурации и ограничивает или не допускает рискованные параметры конфигурации. Например, используйте веб-среду, которая не выводит информацию об ошибках в производственном интерфейсе. Для включения всех функций отладки пользуйтесь одним флагом. Настройте фреймворк развертывания и мониторинга так, чтобы этот флаг не был включен для обычных пользователей. Флаг <code>environment</code> в Rails — один из примеров такого подхода
Межсайтовый скриптинг (XSS)	Используйте защищенную от XSS систему шаблонов (см. подраздел «Предотвращение XSS: SafeHtml» далее)
Небезопасные преобразования	Используйте библиотеки преобразований, которые созданы для обработки ненадежных входных данных, таких как буферы протоколов (https://oreil.ly/hlezU)
Использование компонентов с известными уязвимостями	Выберите популярные библиотеки, которые активно поддерживаются. Не выбирайте компоненты с неисправленными или медленно исправляемыми проблемами безопасности. Также см. раздел «Уроки оценки и создания фреймворков» далее в главе
Недостаточное ведение журнала и мониторинг	Вместо того, чтобы полагаться на специальное ведение журнала, регистрируйте и отслеживайте запросы и другие события в зависимости от ситуации в библиотеке низкого уровня. См. пример перехватчика регистрации, описанный в предыдущем разделе

* См. таблицу предотвращения XXE (<https://oreil.ly/AOYev>) для получения дополнительной информации.

Уязвимости SQL-внедрений: TrustedSqlString

Внедрение SQL (<https://xkcd.com/327>) — распространенный класс уязвимостей без-опасности. Когда непроверенные фрагменты строк вставляются в SQL-запрос, злоумышленники могут вводить команды для БД. Ниже приведена простая веб-форма для сброса пароля:

```
db.query("UPDATE users SET pw_hash = '" + request["pw_hash"]
+ "' WHERE reset_token = '" + request.params["reset_token"] + "'")
```

Здесь запрос пользователя направляется на сервер с определенным `reset_token`, специфичным для его учетной записи. Но из-за конкатенации строк злонамеренный пользователь может создать свой `reset_token` с дополнительными командами SQL (например, `' or username = 'admin'`) и *внедрить* этот токен на сервер. В результате можно сбросить хеш пароля другого пользователя — в этом случае учетной записи администратора.

Уязвимости SQL-внедрений сложнее обнаружить в обширных кодовых базах. Движок БД может помочь предотвратить уязвимости внедрения SQL, предоставив связанные параметры и подготовленные операторы:

```
Query q = db.createQuery(
    "UPDATE users SET pw_hash = @hash WHERE token = @token");
q.setParameter("hash", request.params["hash"]);
q.setParameter("token", request.params["token"]);
db.query(q);
```

Простое руководство по использованию подготовленных операторов не приводит к масштабируемому процессу безопасности. Вам нужно будет проинформировать каждого разработчика об этом правиле, а специалисты по безопасности должны будут просмотреть весь код приложения для проверки согласованного использования подготовленных утверждений. Вместо этого вы можете спроектировать API базы данных так, чтобы смешивание пользовательского ввода и SQL стало невозможным. К примеру, вы можете создать отдельный тип с именем `TrustedSqlString` и принудительно установить, что все строки SQL-запроса создаются из входных данных, контролируемых разработчиком. В Go вы можете реализовать тип таким образом:

```
struct Query {
    sql strings.Builder;
}
type stringLiteral string;
// Вызывайте эту функцию только со строковыми литеральными параметрами
func (q *Query) AppendLiteral(literal stringLiteral) {
```

```
q.sql.writeString(literal);
}
// q.AppendLiteral("foo") будет работать, q.AppendLiteral(foo) – нет
```

Эта реализация гарантирует, что содержимое `q.sql` полностью конкатенировано из строковых литералов, присутствующих в вашем исходном коде, и пользователь не может предоставить свои строковые литералы. Для обеспечения этого соглашения в масштабе вы можете использовать механизмы конкретного языка, чтобы убедиться, что `AppendLiteral` вызывается только со строковыми литералами. Например:

В Go

Используйте пакетно-закрытый псевдоним типа (`stringLiteral`). Код за пределами пакета не может ссылаться на этот псевдоним. Но строковые литералы неявно преобразуются в этот тип.

В Java

Используйте средство проверки подверженного ошибкам кода `Error Prone` (<https://errorprone.info/>), предоставляющее аннотацию `@CompileTimeConstant` для параметров.

В C++

Используйте конструктор шаблона, зависящий от каждого символьного значения в строке.

Вы можете найти похожие механизмы для других языков.

Некоторые функции нельзя создать самостоятельно только при помощи константы времени компиляции. Например, приложение для анализа данных, по своей конструкции выполняющее произвольные SQL-запросы, предоставленные пользователем — владельцем данных. Для обработки сложных случаев использования в Google мы применяем способы обойти ограничения типа только с разрешения инженера по безопасности. Наш API базы данных имеет отдельный пакет `unsafequery`, экспортирующий отдельный тип `unsafequery.String`, который может быть создан из произвольных строк и добавлен в SQL-запросы. Только небольшая часть наших запросов использует непроверенные API. Бремя анализа новых применений SQL-запросов, не безопасных по своей сути, и других ограниченных шаблонов API возлагается на одного (чередующегося) инженера на полставки из сотен или тысяч активных разработчиков. См. следующий раздел «Уроки оценки и создания фреймворков», чтобы узнать о других преимуществах рассмотренных исключений.

Предотвращение XSS: SafeHtml

Подход к безопасности на основе типов, рассмотренный в предыдущем разделе, не специфичен для SQL-внедрений. Google использует более сложную версию того же проектного решения для уменьшения уязвимости межсайтового скриптинга в веб-приложениях¹.

XSS-уязвимости возникают, когда веб-приложение создает ненадежный ввод без предварительной очистки. Например, приложение может интерполировать контролируемое злоумышленником значение `$address` в HTML-фрагмент, такой как `<div>$address</div>`, который виден другому пользователю. Далее атакующий может установить `$address` как `<script>exfiltrate_user_data();</script>` и выполнить произвольный код в контексте страницы другого пользователя.

У HTML нет эквивалента привязки параметров запроса. Вместо этого ненадежные значения должны быть как следует очищены или экранированы до их вставки в HTML-страницу. Кроме того, различные атрибуты и элементы HTML имеют разную семантику. Поэтому разработчикам приложений нужно обрабатывать значения по-разному в зависимости от используемого контекста. Например, контролируемый злоумышленником URL-адрес может вызвать выполнение кода по схеме `javascript:`.

Система типов позволяет учитывать эти требования: можно вводить разные типы для значений, предназначенных для разных контекстов. Например, `SafeHtml` для представления содержимого HTML-элемента и `SafeUrl` для URL-адресов, по которым можно безопасно переходить. Каждый из типов служит (неизменяемой) оберткой вокруг строки. Соглашения поддерживаются конструкторами, доступными для каждого типа. Они составляют надежную кодовую базу, отвечающую за обеспечение свойств безопасности приложения.

Google создал разные библиотеки для разных вариантов использования. Отдельные HTML-элементы могут быть созданы с помощью методов конструктора, которым нужен правильный тип для каждого значения атрибута, и `SafeHtml` для содержимого элемента. Система шаблонов со строгим контекстным экранированием гарантирует соглашение `SafeHtml` для более сложного HTML. Он выполняет следующие действия.

1. Анализирует частичный HTML-код в шаблоне.
2. Определяет контекст для каждой точки подстановки.

¹ Эта система подробнее описана в статье: *Kern C. Securing the Tangled Web. Communications of the ACM, 2014. № 57 (9). P. 38–47. <https://oreil.ly/drZss>.*

3. Требуется, чтобы программа передала значение правильного типа либо правильно экранировала или очистила ненадежные строковые значения.

Например, если у вас есть следующий шаблон замыкания:

```
{template .foo kind="html"><script src="{url}"></script>{/template}
```

попытка использовать строковое значение для `$url` не удастся:

```
templateRendered.setMapData(ImmutableMap.of("url", some_variable));
```

Вместо этого разработчик должен предоставить значение `TrustedResourceUrl`, например:

```
templateRenderer.setMapData(
    ImmutableMap.of("x", TrustedResourceUrl.fromConstant("/script.js"))
).render();
```

Вы не захотите встраивать HTML в веб-интерфейс приложения, если он происходит из ненадежного источника, ведь это приведет к легко эксплуатируемой XSS-уязвимости. Вместо этого воспользуйтесь средством очистки HTML, анализирующим его и совершающим проверки во время выполнения, чтобы определить, что каждое значение соответствует соглашению. Средство очистки удаляет не соответствующие соглашению элементы или те, для которых невозможно проверить соглашение во время выполнения. Вы можете использовать средство очистки для взаимодействия с другими системами, не использующими безопасные типы, потому что многие фрагменты HTML при очистке неизменны.

Разные библиотеки построения HTML направлены на разные компромиссы между производительностью разработки и читаемостью кода. Но все они обеспечивают одинаковые соглашения и должны быть в равной степени заслуживающими доверия (за исключением ошибок в их надежных реализациях). Фактически для уменьшения нагрузки на обслуживание в Google мы генерируем функции построения на разных языках из декларативного файла конфигурации. В нем перечислены HTML-элементы и необходимые соглашения для значений каждого атрибута. Некоторые из наших средств очистки HTML и систем шаблонов используют одинаковый файл конфигурации.

Зрелая реализация с открытым исходным кодом безопасных типов для HTML доступна в шаблонах замыкания (<https://oreil.ly/VrN4w>). Сейчас предпринимаются усилия по внедрению защиты на основе типов (<https://oreil.ly/VrN4w>) в качестве веб-стандарта.

Уроки оценки и создания фреймворков

Ранее мы обсуждали, как структурировать библиотеки для установки свойств безопасности и надежности. Но вы не можете элегантно выразить все такие свойства в процессе проектирования API, а иногда даже не можете легко изменить API. Это можно сделать при взаимодействии со стандартизированным DOM API, предоставляемым браузером.

Вместо этого введите проверки во время компиляции, чтобы разработчики не использовали рискованные API. Плагины для популярных компиляторов, такие как Error Prone (<https://errorprone.info/>) для Java и Tsetse (<https://tsetse.info/>) для TypeScript, могут запрещать рискованные шаблоны кода.

Опыт показал, что ошибки компилятора обеспечивают немедленную и действенную обратную связь. Встроенные инструменты (например, средства статического анализа кода) или инструменты для проверки кода во время рецензирования обеспечивают обратную связь намного позже. Обычно к тому времени, когда код отправляется на проверку, у разработчиков есть готовая рабочая версия. Не очень приятно узнавать, что нужно выполнить какие-то преобразования для использования строго типизированного API в конце разработки.

Намного проще предоставить разработчикам ошибки компилятора или более быстрые механизмы обратной связи, такие как плагины IDE, подчеркивающие проблемный код. Разработчики быстро решают проблемы с компиляцией. Позже им приходится исправлять другие найденные несоответствия, такие как тривиальные опечатки и синтаксические ошибки. Разработчики уже работают над конкретными строками кода, поэтому у них есть полный контекст и вносить изменения становится проще — например, изменить строковый тип на `SafeHtml`.

Чтобы упростить разработчику задачу, предложите автоматические исправления. Они станут отправной точкой для безопасного решения. Например, когда вы обнаруживаете вызов функции с SQL-запросом, то можете автоматически вставить `TrustedSqlBuilder.fromConstant` с параметром запроса. Даже если полученный код не компилируется (запрос может быть строковой переменной, а не константой), разработчики знают, что делать. Им не нужно беспокоиться о механических деталях API, ища нужную функцию, добавляя правильные декларации импорта и т. д.

Поскольку цикл обратной связи быстрый и исправить каждый шаблон несложно, разработчики охотнее пользуются безопасными по своей природе API, даже когда мы не можем доказать, что их код небезопасен, или когда они пишут хороший безопасный код с использованием небезопасных API. Наш опыт от-

личается от информации, описанной в исследовательской литературе, которая фокусируется на снижении ложноположительных и ложноотрицательных показателей¹.

Мы поняли, что фокусировка на этих показателях часто приводит к сложным проверкам, требующим гораздо больше времени для получения результатов. Например, проверка может анализировать потоки данных всей программы через сложное приложение. Часто разработчикам сложно объяснить, как устранить обнаруженную с помощью статического анализа проблему. Так происходит потому, что работу средства проверки труднее объяснить, чем простое синтаксическое свойство. Понимание результатов занимает столько же времени, сколько и поиск ошибки в GDB (отладчик GNU). Но исправление ошибки безопасности типов во время компиляции при написании нового кода обычно сложнее исправления обычной ошибки типа.

Простые, безопасные, надежные библиотеки для общих задач

Может быть сложно создать безопасную библиотеку, охватывающую всевозможные варианты использования и надежно обрабатывающую каждый из них. Разработчик приложения, работающий с системой шаблонов HTML, может написать такой шаблон:

```
<a onclick="showUserProfile('{{username}}');">Show profile</a></pre>

```

Чтобы защититься от XSS, если `username` контролируется злоумышленником, система шаблонов должна включать три разных уровня контекста: одинарные кавычки строки внутри JavaScript в атрибуте HTML-элемента. Трудно создать систему шаблонов, способную обрабатывать все возможные комбинации крайних случаев, и использовать эту систему тоже будет непросто. В других областях эта проблема может стать еще сложнее. Бизнес-требования могут диктовать, кто может выполнять действие, а кто нет. Если ваша библиотека авторизации так же выразительна (и ее трудно анализировать), как язык программирования общего назначения, вы не сможете выполнить все требования разработчиков.

Вместо этого вы можете начать с простой небольшой библиотеки, охватывающей только общие случаи. Простые библиотеки легче объяснить, документировать и использовать. Это упрощает взаимодействие разработчиков и помогает убедить других разработчиков использовать защищенную библиотеку. Иногда лучше

¹ См.: Bessey A. *et al.* A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World // Communications of the ACM, 2010. № 53 (2). P. 66–75. doi: 10.1145/1646353.1646374.

предлагать разные библиотеки, оптимизированные для разных вариантов использования. Например, у вас могут быть как системы шаблонов HTML для сложных страниц, так и библиотеки для построения коротких фрагментов.

Вы можете приспособить другие варианты использования с проверенным экспертами доступом к неограниченной, рискованной библиотеке, обходящей гарантии безопасности. Если вы видите повторяющиеся похожие запросы для варианта использования, поддержите эту функцию в изначально безопасной библиотеке. Как мы заметили в подразделе «Уязвимости SQL-внедрений: `TrustedSqlString`» ранее в этой главе, нагрузка при рецензировании обычно управляема.

Из-за небольшого объема запросов на рецензирование специалисты по безопасности могут подробно изучить код и предложить обширные улучшения, а обзоры — уникальные сценарии использования, мотивирующие их и предотвращающие ошибки из-за повторов и усталости. Исключения действуют и как механизм обратной связи: если разработчикам постоянно нужны исключения для варианта использования, авторам библиотеки стоит рассмотреть возможность создания отдельной библиотеки для этого варианта использования.

Стратегия внедрения

Наш опыт показал, что использование типов для свойств безопасности полезно новому коду. У приложений, созданных в одной широко используемой внутренней веб-среде Google, изначально разработанной с безопасными типами для HTML, было меньше зарегистрированных XSS-уязвимостей (на два порядка), чем у приложений, написанных без них, несмотря на тщательное рецензирование кода. Несколько обнаруженных уязвимостей были вызваны компонентами приложения, не использующими безопасные типы.

Адаптировать существующий код для использования безопасных типов труднее. Даже если вы начинаете с новой кодовой базы, вам нужна стратегия для миграции унаследованного кода. Вы можете обнаружить новые классы проблем безопасности и надежности, от которых нужно защититься, или может понадобиться усовершенствование существующих соглашений.

Мы экспериментировали с несколькими стратегиями рефакторинга существующего кода. Обсудим два самых успешных подхода в следующих подразделах. Для этих стратегий нужно получить доступ и изменить весь исходный код приложения. Большая часть исходного кода Google хранится в одном репозитории¹ с централизованными процессами для внесения, сборки, тестирования

¹ Potvin R., Levenberg J. Why Google Stores Billions of Lines of Code in a Single Repository // Communications of the ACM, 2016. № 59 (7). P. 78–87. doi: 10.1145/2854146.

и отправки изменений. Рецензенты тоже применяют общие стандарты удобочитаемости и организации кода. Это уменьшает сложность изменения незнакомой кодовой базы. В других средах масштабные рефакторинги могут быть сложнее. Так легче получить общее согласие, поэтому каждый владелец кода готов принять изменения в своем исходном коде, что помогает формированию культуры безопасности и надежности.



В руководстве по стилю Google для всей компании есть концепция *читабельности* языка: подтверждение того, что инженер понимает лучшие практики Google и стиль написания кода для данного языка. Читаемость — основа качества кода. Инженер должен либо сделать код на языке, с которым работает, читаемым, либо получить обратную связь от человека, который может эту читаемость обеспечить. Если код очень сложен или критически важен, проверять его должны отдельные разработчики. Такой способ может быть наиболее продуктивным и эффективным для улучшения качества кодовой базы.

Постепенное внедрение

Часто невозможно исправить всю кодовую базу за раз. Разные компоненты могут быть в разных репозиториях. Создание, просмотр, тестирование и отправка одного изменения, затрагивающего несколько приложений, часто ненадежны и подвержены ошибкам. Вместо этого в Google мы изначально освобождаем устаревший код от принудительного применения и рассматриваем существующие небезопасные использования API по одному.

Если у вас уже есть API базы данных с функцией `doQuery(String sql)`, вы можете добавить перегрузку `doQuery(TrustedSqlString sql)` и ограничить небезопасную версию для существующих вызывающих объектов. С помощью фреймворка Error Prone вы можете добавить аннотацию `@RestrictedApi(whitelistAnnotation={LegacyUnsafeStringQueryAllowed.class})` и `@LegacyUnsafeStringQueryAllowed` для всех существующих вызывающих объектов.

Далее, внедрив *Git-хуки*, анализирующие каждое изменение, вы можете запретить новому коду использовать перегрузку строк. Другой вариант — ограничить видимость небезопасного API. Например, белые списки видимости Bazel (<https://oreil.ly/ajmrr>) позволят пользователю вызывать API, только когда член команды безопасности одобрит запрос (pull request, PR). Если ваша кодовая база активно разрабатывается, она будет органично двигаться в направлении безопасного API. По достижении точки, в которой только малая часть вызывающих объектов использует устаревший API на основе строк, вы можете вручную очистить оставшийся код. На этом этапе кодовая база будет защищена от SQL-внедрений.

Унаследованные преобразования

Часто лучше объединить все ваши механизмы исключения в одну функцию, очевидную в читаемом исходном коде. Можно создать функцию, принимающую произвольную строку и возвращающую безопасный тип. С ее помощью вы сможете заменить все вызовы API со строковым типом на более точные. Обычно типов будет намного меньше, чем функций, их потребляющих. Вместо того чтобы ограничивать и контролировать удаление многих устаревших API (например, каждого DOM API, использующего URL), удалите только одну устаревшую функцию преобразования для каждого типа.

Простота — залог безопасного и надежного кода

Стремитесь к чистоте и простоте кода. Есть несколько публикаций на эту тему¹, поэтому здесь мы сосредоточимся на двух незамысловатых историях, опубликованных в блоге Google Testing (<https://testing.googleblog.com/>). В обеих историях раскрыты стратегии, позволяющие избежать быстрого увеличения сложности кода.

Избегайте многоуровневого вложения

Многоуровневое вложение — это распространенный антипаттерн, приводящий к простым ошибкам. Если ошибка будет на самом распространенном пути кода, ее зафиксируют модульные тесты. Но они не всегда проверяют пути обработки ошибок в многоуровневом вложенном коде. Ошибка может привести к снижению надежности (например, в случае сбоя сервиса при неправильной обработке ошибки) или к уязвимости в системе безопасности (например, ошибке проверки авторизации при неправильной обработке).

Можете ли вы обнаружить ошибку в коде на рис. 12.2? Обе версии равнозначны².

Ошибки «wrong encoding» и «unauthorized» меняются местами. Это легче увидеть в реорганизованной версии, так как проверки происходят сразу после обработки ошибок.

¹ См.: Ousterhout J. A Philosophy of Software Design. Palo Alto, CA: Yaknyam Press, 2018.

² Источник: Karpilovsky E. Code Health: Reduce Nesting, Reduce Complexity. 2017. <https://oreil.ly/PO1QR>.

<pre> response = stub.Call(rpc, request) if rpc.status.ok(): if response.GetAuthorizedUser(): if response.GetEnc() == 'utf-8': if response.GetRows(): vals = [ParseRow(r) for r in response.GetRows()] avg = sum(vals) / len(vals) return avg, vals else: raise ValueError('no rows') else: raise AuthError('unauthorized') else: raise ValueError('wrong encoding') else: raise RpcError(rpc.ErrorText()) </pre>	<pre> response = stub.Call(request, rpc) if !rpc.status.ok(): raise RpcError(rpc.ErrorText()) if not response.GetAuthorizedUser(): raise ValueError('wrong encoding') if response.GetEnc() != 'utf-8': raise AuthError('unauthorized') if not response.GetRows(): raise ValueError('no rows') vals = [ParseRow(r) for r in response.GetRows()] avg = sum(vals) / len(vals) return avg, vals </pre>
--	---

Рис. 12.2. Ошибки сложнее обнаружить в коде
с несколькими уровнями вложенности

Устранение «запахов» YAGNI

Иногда разработчики перерабатывают решения, добавляя функциональность, которая может пригодиться в будущем, «на всякий случай». Это противоречит принципу YAGNI (You Aren't Gonna Need It, «вам это не нужно») (<https://oreil.ly/K4Oan>), по которому нужно реализовывать только необходимый на данный момент код. Код YAGNI добавляет ненужную сложность, потому что его необходимо документировать, тестировать и поддерживать. Рассмотрим следующий пример¹:

```

class Mammal { ...
    virtual Status Sleep(bool hibernate) = 0;
};

class Human : public Mammal { ...
    virtual Status Sleep(bool hibernate) {
        age += hibernate ? kSevenMonths : kSevenHours;
        return OK;
    }
};

```

Код `Human::Sleep` должен обрабатывать случай, когда `hibernate` имеет значение `true`, хотя все вызывающие объекты всегда должны передавать значение `false`.

¹ Источник: *Eaddy M.* Code Health: Eliminate YAGNI Smells. 2017. <https://oreil.ly/NYr7y>.

Вызывающие объекты должны обрабатывать возвращенное состояние, даже если оно всегда должно быть равно ОК. Вместо этого, пока вам не понадобятся классы, отличные от Human, этот код можно упростить до следующего:

```
class Human { ...  
    void Sleep() { age += kSevenHours; }  
};
```

Если предположения разработчика о возможных требованиях к будущей функциональности верны, можно легко добавить эту функциональность позже, следуя принципу *поздней разработки и проектирования*. В нашем примере будет проще создать интерфейс Mammal с более подходящим общим API при обобщении на основе нескольких существующих классов.

Подытожим: отказ от кода YAGNI ведет к повышению надежности, а более простой код влечет за собой уменьшение числа ошибок безопасности, меньшее количество возможностей совершать ошибки и уменьшение времени, затрачиваемого разработчиком на поддержку неиспользуемого кода.

Погашение технического долга

Разработчики обычно отмечают места, требующие дополнительного внимания, с помощью аннотаций TODO или FIXME. В краткосрочной перспективе эта привычка может ускорить добавление самых важных функций и позволить команде справиться с работой раньше, но она несет *технический долг*. Это не обязательно плохая практика, если у вас есть четкий процесс (и выделенное время) для погашения такого долга.

Технический долг может содержать ошибочную обработку исключительных ситуаций и введение в код слишком сложной логики (часто написанной для обхода других областей технического долга). Любое из этих действий может привести к проблемам с безопасностью и надежностью, которые редко обнаруживаются во время тестирования (из-за недостаточного охвата редких случаев) и в итоге становятся частью производственной среды.

Есть разные способы погашения технического долга.

- Ведение информационных панелей с показателями работоспособности кода. Они вариативны: от простых информационных панелей, показывающих охват тестами или количество и средний возраст TODO, до более сложных, включающих такие показатели, как *цикломатическая сложность* (https://oreil.ly/_N25V) или *индекс удобства сопровождения* (https://oreil.ly/_N25V).

- Использование инструментов анализа: средства статического анализа кода для обнаружения распространенных дефектов вроде мертвого кода, ненужные зависимости или специфические для языка ошибки. Часто такие инструменты могут автоматически исправить ваш код.
- Отправка уведомлений, когда показатели работоспособности кода падают ниже определенных пороговых значений или когда число автоматически обнаруживаемых проблем становится слишком большим.

Важно сохранять командную культуру, которая поддерживает хорошее состояние кода и фокусируется на нем. Здесь немаловажно лидерство. Вы можете запланировать регулярные недели *исправлений*, в течение которых разработчики сосредоточатся на улучшении работоспособности кода и исправлении ошибок, а не на добавлении новых функций. Вы можете поддерживать постоянный вклад в улучшение качества кода в команде с помощью бонусов или других способов поощрения.

Рефакторинг

Рефакторинг — самый эффективный способ сохранить кодовую базу чистой и простой. Он нужен даже для исправной кодовой базы — при расширении существующего набора функций, изменении сервера и т. д.

Рефакторинг особенно полезен при работе со старыми, унаследованными кодовыми базами. Его первый шаг — измерение покрытия кода и увеличение этого покрытия до достаточного уровня¹. Чем выше покрытие, тем выше ваша уверенность в безопасности рефакторинга. Но даже стопроцентное тестирование не может гарантировать успех, потому что тесты могут быть бессмысленны. Вы можете решить эту проблему с помощью других видов тестирования, таких как *нечеткое тестирование*, описанное в главе 13.



Независимо от причин рефакторинга вы всегда должны следовать одному золотому правилу: *никогда не смешивайте рефакторинг и функциональные изменения в одном коммите в хранилище кода*. Рефакторинг изменений важен и сложен для понимания. Если коммит тоже содержит функциональные изменения, риск того, что автор или рецензент могут пропустить ошибки, возрастает.

¹ Широкий выбор инструментов покрытия кода. Для обзора смотрите список на Stackify (<https://oreil.ly/-w6DM>).

Полный обзор методов рефакторинга выходит за рамки этой книги. Для получения дополнительной информации по этой теме есть отличная книга¹ Мартина Фаулера и статьи Райта и др. (2013)², Вассермана (2013)³, Потвина и Левенберга (2016).

Безопасность и надежность по умолчанию

Вы можете использовать не только фреймворки с надежными гарантиями, но и другие средства автоматического повышения уровня безопасности и надежности приложения, а также принципы командной культуры, о которой вы узнаете больше в главе 21.

Выберите правильные инструменты

Выбор языка, фреймворка и библиотек — сложная задача, на которую часто влияет сочетание таких факторов, как:

- интеграция с существующей кодовой базой;
- доступность библиотек;
- навыки или предпочтения команды разработчиков.

Учитывайте огромное влияние выбора языка на безопасность и надежность проекта.

Использование безопасных для памяти языков

В BlueHat Israel в феврале 2019 года Мэтт Миллер из Microsoft заявил, что около 70 % всех уязвимостей связаны с проблемами безопасности памяти⁴. Эта статистика оставалась неизменной в течение последних 12 лет.

¹ *Fowler M.* Refactoring: Improving the Design of Existing Code. Boston, MA, Addison-Wesley, 2019.

² *Wright H. et al.* Large-Scale Automated Refactoring Using Clang // Proceedings of the 29th International Conference on Software Maintenance, 2013. P. 548–551. doi: 10.1109/ICSM.2013.93.

³ *Wasserman L.* Scalable, Example-Based Refactorings with Refaster // Proceedings of the 2013 ACM Workshop on Refactoring Tools, 2013. P. 25–28. doi: 10.1145/2541348.2541355.

⁴ *Miller M.* Trends, Challenges and Strategic Shifts in the Software Vulnerability Mitigation Landscape. BlueHat IL, 2019. <https://goo.gl/vKM7uQ>.

В презентации 2016 года Ник Кралевиц из Google сообщил, что 85 % всех ошибок в Android (включая ошибки в ядре и других компонентах) были вызваны ошибками управления памятью (слайд 54)¹. Кралевиц заключил: «Нам нужно двигаться к языкам, безопасным для памяти». Используя любой язык с высокоуровневым управлением памятью (например, Java или Go), а не язык с большими сложностями в распределении памяти (например, C/C++), вы можете избежать всего этого класса уязвимостей безопасности (и надежности). Вы можете использовать средства очистки кода, обнаруживающие большинство ошибок управления памятью (см. подраздел «Очистка кода» далее).

Используйте строгую типизацию и статическую проверку типов

В языке *со строгой типизацией* «когда объект передается из вызывающей функции в вызываемую, его тип должен быть совместим с типом, объявленным в вызываемой функции»². Без этого требования язык называется *слабо типизированным*. Вы можете включить проверку типов во время компиляции (*статическая проверка типов*) или выполнения (*динамическая проверка типов*).

Преимущества строгой типизации и статической проверки типов можно заметить при работе на больших кодовых базах с несколькими разработчиками. Так, вы можете применять инварианты и устранять широкий спектр ошибок во время компиляции, а не во время выполнения. Это приводит к созданию более надежных систем, меньшему количеству проблем безопасности и более эффективному коду в производственной среде.

А вот при использовании динамической проверки типов (например, в Python) вы почти ничего не можете сказать о коде, если он не покрыт тестами на 100 %. Это неплохо, но на практике встречается редко. Рассуждение о коде усложняется в слабо типизированных языках, что часто приводит к неожиданному поведению. Например, в JavaScript каждый литерал по умолчанию обрабатывается как строка: `[9, 8, 10].sort()` -> `[10, 8, 9]`³. В обоих этих случаях, так как инварианты не применяются во время компиляции, вы можете фиксировать ошибки только во время тестирования. Это помогает чаще обнаруживать проблемы надежности и безопасности в производственной среде, а не во время разработки, особенно в нечасто используемых путях кода.

¹ *Krlevich N.* The Art of Defense: How Vulnerabilities Help Shape Security Features and Mitigations in Android. BlackHat, 2016. <https://oreil.ly/16rCq>.

² *Liskov B., Zilles S.* Programming with Abstract Data Types // Proceedings of the ACM SIGPLAN Symposium on Very High Level Languages, 1974. P. 50–59. doi: 10.1145/800233.807045

³ Еще больше сюрпризов в JavaScript и Ruby см. в «Молниеносной беседе Гэри Бернхардта из CodeMash» 2012 (<https://oreil.ly/M69rg>).

Если вы хотите использовать языки с динамической проверкой типов или слабой типизацией по умолчанию, используйте описанные ниже расширения для повышения надежности кода. Они предлагают поддержку более строгой проверки типов, и вы можете постепенно добавлять их в существующие кодовые базы:

- Pytype для Python (https://oreil.ly/_AAvo);
- TypeScript для JavaScript (<https://www.typescriptlang.org/>).

Использование строгих типов

Использование нетипизированных примитивов (таких как строки или целые числа) приведет к следующим проблемам:

- передаче концептуально недопустимых параметров в функции;
- нежелательным неявным преобразованиями типов;
- сложной для понимания иерархии типов¹;
- запутанным единицам измерения.

Первая ситуация возникает, если у примитивного типа параметра функции недостаточно контекста, и поэтому вызов функции становится запутанным. Например:

- Для функции `AddUserToGroup(string, string)` неясно, указывается имя группы в качестве первого или второго аргумента.
- Каков порядок высоты и ширины при вызове конструктора `Rectangle(3.14, 5.67)`?
- `Circle(double)` ожидает радиус или диаметр?

Документация позволяет исправить двусмысленность, но разработчики все еще могут ошибиться. Модульные тесты дают возможность выявить большинство из этих ошибок при должной осмотрительности, но некоторые могут раскрыться только во время выполнения.

Используя сильные типы, вы можете обнаружить эти ошибки во время компиляции. Вернемся к нашему более раннему примеру — требуемые вызовы будут выглядеть так:

- `Add(User("alice"), Group("root-users"))`
- `Rectangle(Width(3.14), Height(5.67))`
- `Circle(Radius(1.23))`

¹ См.: *Eaddy M.* Code Health: Obsessed with Primitives?, 2017. <https://oreil.ly/0DvJL>.

где `User`, `Group`, `Width`, `Height` и `Radius` — обертки строгого типа вокруг строковых или примитивов чисел с плавающей точкой. Этот подход не так подвержен ошибкам и делает код более самодокументируемым — в этом контексте в первом примере достаточно вызвать функцию `Add`.

Во второй ситуации неявные преобразования типов могут привести к следующему:

- усечению при преобразовании из целочисленных типов в большие и меньшие;
- потере точности при преобразовании в типы с плавающей точкой большего или меньшего размера;
- неожиданному созданию объекта.

Иногда компилятор сообщает о первых двух проблемах (например, при использовании синтаксиса прямой инициализации `{}` в C++), но многие случаи будут упущены. Использование строгих типов защищает ваш код от ошибок такого типа, не фиксируемых компилятором.

Теперь рассмотрим случай сложной для понимания иерархии типов:

```
class Bar {
public:
    Bar(bool is_safe) {...}
};

void Foo(const Bar& bar) {...}

Foo(false); // Скорее всего, все в порядке, но знает ли разработчик,
            // что был создан объект Bar?
Foo(5);     // Будет создан объект Bar(is_safe := true), но,
            // скорее всего, случайно
Foo(NULL);  // Будет создан объект Bar(is_safe := false), опять же,
            // скорее всего, случайно
```

Три вызова здесь будут скомпилированы и выполнены, но будет ли результат операции соответствовать ожиданиям разработчиков? По умолчанию компилятор C++ пытается неявно (*принудительно*) привести параметры к типам аргументов функции. В этом случае компилятор попытается использовать тип `Bar`, обладающий удобным конструктором с одним значением, принимающим параметр типа `bool`. Большинство типов C++ неявно приводятся к `bool`.

Иногда подразумевается неявное приведение в конструкторах (например, при преобразовании значений с плавающей точкой в класс `std::complex`), но в большинстве ситуаций это может быть опасно. Для предотвращения опасных результатов сделайте конструкторы с одним значением *явными*, например

`explicit Bar(bool is_safe)`. Заметьте, что последний вызов приведет к ошибке компиляции при использовании `nullptr`, а не `NULL`, потому что не будет неявного преобразования в `bool`.

Наконец, неразбериха в единицах измерения — бесконечный источник ошибок, которые можно охарактеризовать так.

Безвредные

Код, устанавливающий таймер на 30 секунд вместо 30 минут, потому что программист не знал, какие единицы использует `Timer(30)`.

Опасные

Самолет AirCanada «Gimli Glider» (<https://oreil.ly/5r61w>) вынужден был совершить аварийную посадку после того, как наземный экипаж рассчитал необходимое топливо в фунтах вместо килограммов, оставив только половину.

Дорогие

Ученые потеряли орбитальный аппарат Mars Climate Orbiter за 125 миллионов долларов (<https://oreil.ly/ZMbIO>), потому что две отдельные команды разработчиков использовали разные единицы измерения (британской системы и метрические).

Использование строгих типов позволит решить эту проблему: можно внедрить единицу измерения и представлять только абстрактные понятия, такие как отметка времени, длительность или вес. Такие типы обычно выполняют следующее.

Разумные операции

Складывание двух временных меток — не полезная операция, но при их вычитании мы получаем длительность, которая будет полезна для многих случаев использования. Складывание двух длительностей или весов тоже полезно.

Перевод единиц измерения

`Timestamp::ToUnix, Duration::ToHours, Weight::ToKilograms`.

Некоторые языки предоставляют такие абстракции изначально: примеры содержат пакет `time` (<https://golang.org/pkg/time>) в Go и библиотеку `chrono` (<http://www.wg21.link/p0355>) в будущем стандарте C++20. Для других языков может понадобиться специальная реализация.

В блоге Fluent C++ (<https://oreil.ly/Urmzl>) подробнее обсуждаются применение строгих типов и примеры реализации в C++.

Очистка кода

Автоматическая проверка отсутствия типичных ошибок управления памятью или параллелизма очень полезна. Вы можете выполнить ее предварительно для каждого списка изменений или как часть автоматизации непрерывной сборки и тестирования. Список ошибок для проверки зависит от языка. В этом разделе показаны некоторые решения для C++ и Go.

C++: Valgrind или Google Sanitizer

C++ обеспечивает низкоуровневое управление памятью. Как мы уже упоминали, ошибки управления памятью — основная причина проблем безопасности. Они могут привести к таким сценариям сбоя, как:

- чтение нераспределенной памяти (до `new` или после `delete`);
- чтение за пределами выделенной памяти (сценарий атаки переполнения буфера);
- чтение неинициализированной памяти;
- утечки памяти, когда система теряет адрес выделенной памяти или не освобождает неиспользуемую память на ранних этапах.

Valgrind (<http://www.valgrind.org/>) — это популярный фреймворк, позволяющий разработчикам отлавливать такие ошибки, даже если модульные тесты их не обнаруживают. Преимущество Valgrind в предоставлении виртуальной машины, интерпретирующей предоставленный бинарный файл. Благодаря этому пользователям не нужно перекомпилировать код для его использования. Инструмент Helgrind (<https://oreil.ly/mBSSw>) в Valgrind может дополнительно обнаруживать типичные ошибки синхронизации, такие как:

- злоупотребление API `pthread` в POSIX (например, разблокировка незаблокированного или удерживаемого другим потоком мьютекса);
- потенциальные взаимоблокировки, возникающие из-за проблем с упорядочением блокировок;
- гонки данных, вызванные доступом к памяти без надлежащей блокировки или синхронизации.

Пакет Google Sanitizers (<https://oreil.ly/qqdMy>) предлагает разные компоненты, способные находить все те же проблемы, которые может обнаружить Callgrind от Valgrind (профилировщик кэша и ветвления):

- AddressSanitizer (ASan) находит ошибки памяти (переполнение буфера, использование после освобождения, неверный порядок инициализации);

- LeakSanitizer (LSan) выявляет утечки памяти;
- MemorySanitizer (MSan) обнаруживает чтение системой неинициализированной памяти;
- ThreadSanitizer (TSan) обнаруживает гонки данных и взаимоблокировки;
- UndefinedBehaviorSanitizer (UBSan) находит ситуации с неопределенным поведением (использование указателей с неправильным выравниванием; переполнение целых чисел со знаком; преобразование в, из или между типами с плавающей точкой, переполняющее выбранный тип).

Основное преимущество пакета Google Sanitizers — скорость: он в десять раз быстрее (<https://oreil.ly/iyxQ1>), чем Valgrind. Популярные IDE, такие как CLion (<https://oreil.ly/iyxQ1>), тоже обеспечивают встроенную интеграцию с Google Sanitizers. В следующей главе мы подробнее остановимся на средствах очистки и других инструментах динамического анализа программ.

Go: Race Detector

Go предназначен для предотвращения проблем с повреждением памяти, типичных для C++, но он все еще может страдать от гонки данных. Race Detector в Go (<https://oreil.ly/RU46m>) может выявить эти условия.

Резюме

В этой главе разобрано несколько принципов, помогающих разработчикам в проектировании и реализации более безопасного и надежного кода. Мы рекомендуем применять фреймворки в качестве мощной стратегии. Они повторно используют проверенные строительные блоки для чувствительных областей кода, подверженных проблемам надежности и безопасности: аутентификации, авторизации, регистрации, ограничения скорости и связи в распределенных системах. Фреймворки тоже помогут повысить производительность труда разработчиков — как пишущих фреймворк, так и его использующих. Они сильно упростят понимание кода. Дополнительные стратегии для написания безопасного и надежного кода содержат стремление к простоте, выбор правильных инструментов, использование строгих, а не примитивных типов, и постоянную очистку кодовой базы.

Вложение дополнительных усилий в безопасность и надежность при разработке ПО окупается в долгосрочной перспективе и сокращает усилия, затрачиваемые на проверку приложения или устранение проблем после развертывания.

Тестирование

*Фил Эймс и Франьо Иванчич
с Верой Хаас и Джен Барнасон*

Заслуживающая доверия система устойчива к сбоям и отвечает своим задокументированным целям уровня обслуживания, включающим гарантии безопасности. Надежное тестирование и анализ ПО — полезные инструменты для снижения рисков отказов. Поэтому на этапе реализации проекта на них нужно обращать особое внимание.

Здесь мы обсудим несколько видов тестирования, включая модульное и интеграционное. Мы дополнительно углубимся в темы безопасности, такие как статический и динамический анализ программ и нечеткое тестирование, повышающие устойчивость ПО ко всевозможным входным данным.

Даже если разрабатывающие ПО инженеры будут осторожны, некоторых ошибок и упущений невозможно избежать. Неожиданные комбинации входных данных могут вызвать их повреждение или привести к проблемам с доступностью, таким как в примере «запрос смерти» в главе 22 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/addressing-cascading-failures/>). Ошибки кодирования могут вызвать проблемы с безопасностью, в частности переполнение буфера и уязвимости межсайтового скриптинга. Проще говоря, есть много способов вызвать сбой программного обеспечения.

Методы, которые мы сейчас обсудим, применяются на разных этапах и в разных контекстах разработки ПО. Помимо этого, они требуют разных затрат и дают разные выгоды¹. Например, *нечеткое тестирование* — отправка случайных запросов в систему — может помочь укрепить эту систему с точки зрения безопасности и надежности. Этот метод позволит уловить утечки информации и уменьшить

¹ Мы рекомендуем ознакомиться с главой 17 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/testing-reliability/>), чтобы оценить перспективы надежности.

количество ошибок при обслуживании, вызывая множество крайних случаев в сервисе. Для выявления возможных ошибок в сложноисправимых системах вам нужно выполнить тщательное предварительное тестирование.

Модульное тестирование

Модульное тестирование может повысить безопасность и надежность системы, выявляя широкий спектр ошибок в отдельных программных компонентах перед выпуском. Этот метод включает разбиение программных компонентов на более мелкие автономные «блоки» без внешних зависимостей и последующее тестирование каждого из них. Модульные тесты содержат код, который выполняется для заданного блока с разными входными данными, предварительно выбранными инженером. Популярные фреймворки модульного тестирования доступны для многих языков. Очень распространены системы на основе архитектуры xUnit (<https://oreil.ly/jZgl5>).

Фреймворки, следующие модели xUnit, позволяют выполнять общий код настройки и разбора для каждого тестового метода. Они определяют роли и обязанности для отдельных компонентов фреймворка тестирования. Это помогает стандартизировать формат результатов тестирования. Так другие системы получают подробную информацию о том, что именно пошло не так. Популярные примеры содержат JUnit для Java, GoogleTest для C++, go2xunit для Golang и встроенный модуль unittest в Python.

Пример 13.1 показывает модульный тест (<https://oreil.ly/4Dkod>), написанный с использованием фреймворка GoogleTest.

Пример 13.1. Модульный тест, написанный с использованием фреймворка GoogleTest, для функции, проверяющей, является ли предоставленный аргумент простым числом

```
TEST(IsPrimeTest, Trivial) {  
    EXPECT_FALSE(IsPrime(0));  
    EXPECT_FALSE(IsPrime(1));  
    EXPECT_TRUE(IsPrime(2));  
    EXPECT_TRUE(IsPrime(3));  
}
```

Модульные тесты обычно выполняются локально, как часть рабочих процессов проектирования. Это делается для того, чтобы обеспечить быструю обратную связь с разработчиками до того, как они отправят изменения в кодовую базу. В конвейерах с непрерывной интеграцией/развертыванием (continuous integration/continuous delivery, CI/CD) модульные тесты часто выполняются до объединения изменений с основной ветвью репозитория. Так мы пытаемся предотвратить изменения кода, нарушающие поведение, на которое полагаются другие команды.

Написание эффективных модульных тестов

Качество и полнота модульных тестов могут существенно повлиять на надежность вашего ПО. Модульные тесты должны быть быстрыми и надежными. Так инженерам будет проще узнать, нарушило ли их изменение ожидаемое поведение системы. Создание и поддержка модульных тестов гарантируют, что охватываемая тестами система будет работать так, как предполагается. Как отмечалось в главе 9, ваши тесты должны быть герметичными. Если тест не может каждый раз давать одинаковые результаты в изолированной среде, на него нельзя с уверенностью полагаться.

Рассмотрим систему, управляющую количеством хранимых байтов, которое команда может использовать в центре данных или регионе. Допустим, что система позволяет командам запрашивать дополнительную квоту, если в центре данных есть свободные нераспределенные байты. Простой модульный тест может включать проверку запросов на квоту в наборе частично занятых воображаемыми группами кластеров, отклоняя запросы, превышающие доступную емкость хранилища. Ориентированные на безопасность модульные тесты могут проверять обработку запросов с отрицательными объемами байтов или обработку кодом переполнения емкости для больших транзакций, приводящих к значениям квоты, близким к пределам возможностей типов переменных, используемых для их представления. Другой модульный тест может проверить, возвращает ли система сообщение об ошибке при отправке злонамеренного или искаженного ввода.

Полезно протестировать один и тот же код с разными параметрами или данными среды, такими как начальная квота в нашем примере. Для сведения объема дублирующегося кода к минимуму платформы или языки модульного тестирования часто предоставляют способ вызова одного теста с разными параметрами. Это помогает уменьшить количество дубликатов стандартного кода, что может упростить рефакторинг.

Когда писать модульные тесты

Распространенная стратегия — написание тестов вскоре после написания кода, чтобы тесты проверяли, работает ли код должным образом. Эти тесты обычно загружаются в том же коммите, что и новый код. В них часто обрабатываются случаи, вручную проверенные инженером, пишущим код. Наше приложение из примера для управления хранением может потребовать, чтобы «большая квота могла быть запрошена только администраторами выставления счетов для группы владельцев сервиса». Вы можете разбить эти требования на несколько модульных тестов.

В организациях, практикующих рецензирование кода, рецензент может дважды проверить тесты для уверенности в том, что они достаточно надежны для

поддержания качества кодовой базы. Рецензент может заметить, что, хотя новые тесты добавлены для новых изменений, они могут пройти успешно даже в случае удаления или деактивации нового кода. Если рецензент может заменить оператор вроде `if (condition_1 || condition_2)` на `if (false)` или `if (true)` в новом коде и все новые тесты пройдут успешно, тогда они могут пропустить важные тестовые случаи. Дополнительную информацию об опыте Google по автоматизации такого рода *мутационного тестирования* вы можете найти в статье Петрович и Иванкович (2018)¹.

Вместо того, чтобы писать тесты *после* написания кода, методологии разработки на основе тестирования (test-driven development, TDD) побуждают инженеров писать модульные тесты на основе установленных требований и ожидаемого поведения *перед* написанием кода. При тестировании новых функций или исправлении ошибок тесты не пройдут до полной реализации поведения. Как только функция будет реализована и тесты пройдут успешно, инженеры смогут перейти к следующей функции, где процесс повторяется.

Для существующих проектов, не построенных с использованием моделей TDD, обычно применяется стратегия медленной интеграции и улучшения покрытия тестами в ответ на сообщения об ошибках или попытке повысить доверие к системе. Но достижение полного покрытия тестами не означает, что в проекте нет ошибок. Неизвестные крайние случаи или не полностью реализованная обработка ошибок все еще могут вызвать неправильное поведение.

Вы можете написать модульные тесты в ответ на внутреннее ручное тестирование или проверку кода. Можно добавить эти тесты во время стандартной разработки и проверки или во время таких этапов, как проверка безопасности перед запуском в производстве. Новые модульные тесты могут проверить работу предполагаемого исправления ошибок и что последующий рефакторинг не приведет к повторному появлению той же ошибки. Этот тип тестирования особенно важен, если о коде сложно делать какие-то выводы, а потенциальные ошибки влияют на безопасность. Например, при написании проверок контроля доступа в системе со сложной моделью разрешений.



Чтобы охватить как можно больше сценариев, вы потратите больше времени на написание тестов, чем на написание тестируемого кода, особенно работая с нестандартными системами. Это время окупается в долгосрочной перспективе, ведь раннее тестирование улучшает качество кодовой базы, уменьшая количество крайних случаев для отладки.

¹ Petrović G., Ivanković M. State of Mutation Testing at Google // Proceedings of the 40th International Conference on Software Engineering, 2018. P. 163–171. doi: 10.1145/3183519.3183521.

Как модульное тестирование влияет на код

Для улучшения полноты тестов может понадобиться разработка нового кода, содержащего положения о тестировании. Можно и реорганизовать старый код, чтобы сделать его более тестируемым. Обычно рефакторинг содержит предоставление способа перехвата вызовов к внешним системам. Используя эту возможность, вы можете тестировать код разными способами. Например, проверяя, что код вызывает перехватчик правильное число раз и с правильными параметрами.

Подумайте, как можно протестировать фрагмент кода, открывающий заявки в удаленной системе отслеживания, когда выполняются определенные условия. Создание настоящей заявки каждый раз при запуске модульного теста будет генерировать ненужную активность. Еще эта стратегия тестирования может случайно провалиться, если система отслеживания заявок недоступна. А это не отвечает цели быстрых и надежных результатов тестирования. Переписывая этот код, вы можете удалить прямые вызовы службы отслеживания заявок и заменить их абстракцией, например, интерфейсом для объекта `IssueTrackerService`. Реализация для тестирования может записывать данные при получении вызова, такие как «Создать заявку». Тест может проверять эти метаданные, чтобы сделать вывод об успешном или неудачном прохождении. Производственная же реализация будет подключаться к удаленным системам и вызывать открытые методы API.

Такое изменение кода сильно снижает «бесполезность» теста, зависящего от реальных систем. Эти системы полагаются на негарантированное поведение — на внешнюю зависимость или порядок элементов при извлечении из некоторых типов контейнеров. Поэтому нестабильные тесты часто мешают разработчику, вместо того чтобы помогать. Попробуйте исправлять плохие тесты по мере их возникновения. Иначе разработчики могут привыкнуть игнорировать результаты тестирования при проверке изменений.



Эти абстракции и их реализации называются *моками* (*имитации, mocks*), *стабами* (*заглушки, stubs*) или *фейками* (*подделки, fakes*). Инженеры иногда используют эти слова взаимозаменяемо, несмотря на различия концепций по сложности реализации и функциям. Проследите, чтобы все сотрудники в вашей организации вкладывали единый смысл в эти слова. Если вы практикуете анализ кода или используете руководства по стилю кодирования — предоставьте определения, которые команды могут согласовать, чтобы избежать путаницы.

Легко попасть в ловушку чрезмерной абстракции, где тесты механически проверяют предположения о порядке вызовов функций или их аргументов. Чрезмерно

абстрагированные тесты часто бесполезны, так как они «тестируют» реализацию потока управления в языке, а не интересующее вас поведение систем.

Если вам приходится полностью переписывать свои тесты каждый раз при изменении метода, пересмотрите тесты или даже архитектуру самой системы. Во избежание постоянных переписываний тестов попросите знакомых с сервисом инженеров предоставить подходящие тестовые реализации для любых нестандартных задач тестирования. Это выгодно как ответственной за систему команде, так и инженерам, тестирующим код. Владеющая абстракцией команда может гарантировать, что та отслеживает набор функций сервиса по мере его развития, и у команды, использующей абстракцию, теперь есть более реалистичный компонент, который можно использовать в тестах¹.

ПРОВЕРКА ПРАВИЛЬНОСТИ

Тщательно разработанные наборы тестов могут оценить правильность различных программ, выполняющих одинаковую задачу. Это может быть полезно при специализированных условиях. У компиляторов часто есть наборы тестов, которые фокусируются на крайних случаях эзотерических языков программирования. Один из таких примеров — набор «усложненных тестов» (<https://oreil.ly/nn6u0>) компилятора GNU.

Другой пример — набор тестовых векторов Wycheproof (<https://oreil.ly/ecCr9>). Он предназначен для проверки правильности реализации криптографического алгоритма в отношении определенных известных атак. Wycheproof использует преимущества стандартизированного интерфейса Java — Java Cryptography Architecture (JCA) — для доступа к криптографическим функциям. Авторы криптографического ПО пишут реализации для JCA, а криптографические провайдеры JCA обрабатывают вызовы криптографических функций. Предоставленные тестовые векторы можно использовать и в других языках программирования.

Есть и наборы тестов, цель которых — проверить все правила RFC или проанализировать определенный формат мультимедиа. Инженеры, пытающиеся спроектировать анализаторы упрощенной замены, могут полагаться на такие тесты для гарантии того, что старая и новая реализации совместимы и дают равнозначные наблюдаемые результаты.

Интеграционное тестирование

Интеграционное тестирование шире отдельных модулей и абстракций. Оно заменяет реализации тестовых абстракций вроде баз данных или сетевых сервисов настоящими реализациями. В итоге интеграционные тесты используют более полные пути кода. Вы должны инициализировать и настроить эти и другие за-

¹ Для подробного обсуждения распространенных ошибок в модульном тестировании, встречающихся в Google, см.: *Wright H., Winters T. All Your Tests Are Terrible: Tales from the Trenches. CppCon 2015.* <https://oreil.ly/idleN>.

висимости, поэтому интеграционное тестирование может проходить медленнее и тщательнее модульного. Для выполнения теста этот подход пользуется настоящими переменными, такими как задержка сети, так как сервисы обмениваются данными напрямую.

Перейдя от тестирования отдельных низкоуровневых блоков кода к тестированию их взаимодействия друг с другом, вы будете более уверены в правильном поведении системы.

Формы интеграционного тестирования определяются сложностью проверяемых зависимостей. Если нужные для него зависимости несложные, интеграционный тест может выглядеть как базовый класс, устанавливающий несколько общих зависимостей (например, БД в предварительно настроенном состоянии), из которых расширяются другие тесты. Интеграционные тесты могут усложняться по мере разрастания сервисов, требуя от контролирующих систем создания инициализации или настройки зависимостей для поддержки теста. У Google есть команды, сосредоточенные только на инфраструктуре, обеспечивающей стандартизированную настройку интеграционного тестирования для общих сервисов инфраструктуры. Для организаций, использующих систему непрерывной сборки и развертывания, такую как Jenkins (<https://jenkins.io/>), интеграционные тесты могут выполняться отдельно от модульных тестов или параллельно им, в зависимости от размера кодовой базы и количества доступных тестов в проекте.



Создавая интеграционные тесты, помните о принципах, обсуждаемых в главе 5: убедитесь, что требования к данным и системам в отношении доступа к тестам не угрожают безопасности. Может быть заманчиво отразить реальные БД в тестовых средах, ведь базы предоставляют богатый набор реальных данных. По возможности избегайте этого антипаттерна, так как они могут содержать конфиденциальные данные, которые будут доступны любому, кто запускает тесты. Такая реализация несовместима с принципом наименьших привилегий и может угрожать безопасности. Вместо этого заполните эти системы открытыми испытательными данными. Этот подход упрощает очистку тестовых сред до известного чистого состояния, что уменьшает возможность ошибок в интеграционных тестах.

Написание эффективных интеграционных тестов

Интеграционные тесты, как и модульные, могут зависеть от выбора проектного решения. Продолжим наш предыдущий пример с системой отслеживания заявок и просто проверим с помощью модульного теста, был ли вызван метод для отправки заявки в удаленный сервис. Интеграционный тест использовал бы настоящую клиентскую библиотеку. Чтобы не создавать ложные ошибки

в производственном коде, интеграционный тест будет взаимодействовать с конечной точкой тестирования. Тестовые случаи будут использовать логику приложения со входами, иницилирующими вызовы тестового экземпляра. Управляющая логика может потом запросить тестовый экземпляр, чтобы напрямую убедиться в успешном выполнении внешних видимых действий.

Для понимания причины невыполнения интеграционных тестов во время выполнения всех модульных нужно много времени и энергии. Правильное ведение журнала в ключевые логические моменты ваших интеграционных тестов поможет вам отладить систему и понять, где происходят сбои. Но имейте в виду, что интеграционные тесты выходят за рамки отдельных блоков, изучая взаимодействие между компонентами. Поэтому они могут только ограниченно сообщить вам, насколько хорошо эти блоки будут соответствовать вашим ожиданиям в других сценариях. Это одна из многих причин, почему использование каждого типа тестирования в жизненном цикле разработки повышает ценность системы — одна форма тестирования не может заменить другую.



Динамический анализ программ

Благодаря *анализу программ* пользователи могут выполнять много полезных действий. Например, профилирование производительности, проверка корректности, связанной с безопасностью, создание отчетов о покрытии кода тестами и устранение неиспользуемой логики. Как мы покажем дальше, вы можете выполнять программный анализ *статически*, чтобы исследовать ПО, не запуская его. Здесь мы сосредоточены на *динамических* подходах. Динамический анализ программ проверяет ПО, запуская его в виртуализированных или эмулируемых средах, для выходящих за рамки простого тестирования целей.

Профилировщики производительности (используются для выявления проблем с производительностью в программах) и генераторы отчетов о покрытии кода тестами — самые известные типы динамического анализа. В прошлой главе мы показали инструмент динамического анализа программ Valgrind (<http://www.valgrind.org/>). Он объединяет виртуальную машину и разные инструменты для интерпретации бинарного файла и проверки на наличие различных общих ошибок при выполнении. В этом разделе рассматриваются подходы динамического анализа, основанные на поддержке компилятора (часто называемые *инструментацией*) для выявления связанных с памятью ошибок.

С помощью компиляторов и средств динамического анализа программ можно настраивать инструментарий для сбора статистики времени выполнения бинарных файлов, генерируемых компиляторами, таких как информация о профилирова-

нии производительности, информация о покрытии кода тестами и оптимизации на основе профилирования. Компилятор добавляет инструкции и обратные вызовы в библиотеку среды выполнения серверного кода, определяющей и собирающей соответствующую информацию при выполнении бинарного файла. Здесь мы сосредоточимся на ошибках, связанных с безопасным использованием памяти для программ на C/C++.

Пакет Google Sanitizers предоставляет инструменты динамического анализа на основе компиляции. Изначально они были разработаны как часть инфраструктуры компилятора LLVM (<https://llvm.org/>) для фиксации типичных ошибок программирования и теперь тоже поддерживаются GCC и другими компиляторами. Например, AddressSanitizer (ASan) (<https://oreil.ly/NkxYL>) находит ряд распространенных ошибок, связанных с памятью (доступ за пределы памяти) в программах на C/C++. К другим популярным средствам анализа кода относятся следующие.

UndefinedBehaviorSanitizer (<https://oreil.ly/fRXLV>)

Во время выполнения ставит метки при неопределенном поведении.

ThreadSanitizer (<https://oreil.ly/b6-wy>)

Обнаруживает условия гонки.

MemorySanitizer (<https://oreil.ly/u9Jfh>)

Обнаруживает чтение неинициализированной памяти.

LeakSanitizer (<https://oreil.ly/Z9O5m>)

Обнаруживает утечки памяти и другие типы утечек.

Новые аппаратные функции позволяют помечать адреса памяти, поэтому есть предложения (<https://oreil.ly/8BXt4>) использовать эти новые функции для повышения производительности ASan.

ASan обеспечивает высокую производительность, создавая специально оснащенный бинарный файл анализируемой программы. Во время компиляции ASan добавляет инструкции для выполнения обратных вызовов в предоставленную среду выполнения средства анализа. Он поддерживает метаданные о выполнении программы, например, какие адреса памяти действительны для доступа. ASan использует теньюю память для записи информации о том, безопасен ли байт для доступа программы. Он также использует добавленные компилятором инструкции для проверки теневого памяти, когда программа пытается прочитать или записать этот байт. ASan обеспечивает пользовательские реализации выделения и освобождения памяти (`malloc` и `free`). Например, функция `malloc`

выделяет дополнительную память прямо перед и после возвращенной запрошенной области памяти. Это создает область буферной памяти, позволяющую ASan легко сообщать о переполнении и потере буфера с точной информацией о том, что и где пошло не так. Для этого ASan помечает эти области (называемые *красными зонами*) как «отравленные». Точно так же ASan помечает освобожденную память, благодаря чему можно обнаруживать ошибки использования после освобождения.

В следующем примере показан простой запуск ASan с использованием компилятора Clang. Оболочка управляет инструментом и запускает определенный входной файл с ошибкой использования после освобождения, возникающей, когда читается адрес памяти, принадлежащий ранее выделенной области. При взломе этот тип доступа может использоваться как строительный блок. Опция `-fsanitize=address` включает инструментацию ASan:

```
$ cat -n use-after-free.c
1 #include <stdlib.h>
2 int main() {
3   char *x = (char*)calloc(10, sizeof(char));
4   free(x);
5   return x[5];
6 }

$ clang -fsanitize=address -O1 -fno-omit-frame-pointer -g use-after-free.c
```

По завершении компиляции возникает сообщение об ошибке, которое ASan создает при выполнении сгенерированного бинарного файла (для краткости мы пропустили полное сообщение об ошибке ASan). Заметьте, что ASan решает отчетам об ошибках указывать информацию об исходном файле, такую как номера строк, с помощью симболизатора LLVM. Это описано в разделе «Симболизация отчетов» документации Clang. Из отчета мы видим, что ASan находит однобайтовый доступ для чтения при использовании после выделения памяти (отмечено в примере). Сообщение об ошибке содержит информацию об исходном выделении, освобождении и последующем недопустимом использовании памяти:

```
% ./a.out
=====
==142161==ERROR: AddressSanitizer: heap-use-after-free on address
0x602000000015
at pc 0x00000050b550 bp 0x7ffc5a603f70 sp 0x7ffc5a603f68
READ of size 1 at 0x602000000015 thread T0
#0 0x50b54f in main use-after-free.c:5:10
#1 0x7f89ddd6452a in __libc_start_main
#2 0x41c049 in _start
```



```
0x60200000015 is located 5 bytes inside of 10-byte region
[0x60200000010,0x6020000001a)
```

```
freed by thread T0 here:
```

```
#0 0x4d14e8 in free
#1 0x50b51f in main use-after-free.c:4:3
#2 0x7f89ddd6452a in __libc_start_main
```

```
previously allocated by thread T0 here:
```

```
#0 0x4d18a8 in calloc
#1 0x50b514 in main use-after-free.c:3:20
#2 0x7f89ddd6452a in __libc_start_main
```

```
SUMMARY: AddressSanitizer: heap-use-after-free use-after-free.c:5:10 in main
[...]
==142161==ABORTING
```

КОМПРОМИССЫ ПРОИЗВОДИТЕЛЬНОСТИ ПРИ ДИНАМИЧЕСКОМ АНАЛИЗЕ ПРОГРАММ

Инструменты динамического анализа программ дают разработчикам полезную информацию о корректности и других аспектах, таких как производительность и покрытие кода тестами. Из-за этой обратной связи понижается производительность: компилируемые инструментарием бинарные файлы могут выполняться гораздо медленнее, чем встроенные. В результате многие проекты добавляют конвейеры с улучшенными средствами анализа в свои существующие системы CI/CD, но используют их реже — например, ночью. Это поможет выявить другие трудно определяемые ошибки, вызванные проблемами повреждения памяти. Другие CI/CD конвейеры, основанные на анализе программ, собирают дополнительные сигналы для разработчика, такие как ночные показатели покрытия кода тестами. Со временем вы можете использовать эти сигналы для получения разных метрик работоспособности кода.



Нечеткое тестирование

Нечеткое тестирование (часто называемое *фаззингом*) — это метод, дополняющий стратегии тестирования. Оно содержит использование механизма *фаззинга* (или *фаззера*) для генерации большого количества потенциальных входных данных, которые затем передаются через *драйвер фаззинга* к *цели фаззинга* (коду, обрабатывающему входные данные). После чего фаззер анализирует способ обработки ввода системой. Сложные входные данные, обрабатываемые всеми видами ПО, популярны для нечеткого тестирования. Например, анализаторы файлов, реализации алгоритмов сжатия, реализации сетевых протоколов и аудиокодеки.

ПРЕИМУЩЕСТВА БЕЗОПАСНОСТИ И НАДЕЖНОСТИ НЕЧЕТКОГО ТЕСТИРОВАНИЯ

Одна из причин проведения нечеткого тестирования — поиск таких ошибок, как повреждение памяти, несущих последствия для безопасности. Фаззинг может идентифицировать входные данные, вызывающие исключения во время выполнения. Это может вызвать каскадный отказ обслуживания в таких языках, как Java и Go.

Фаззинг полезен для тестирования устойчивости сервиса. Google регулярно выполняет ручные и автоматизированные тесты аварийного восстановления. Автоматизированные запуски важны для контролируемого поиска регрессий. Если система дает сбой при неправильном вводе или возвращает ошибку при использовании специального символа, результаты могут повлиять на бюджет ошибок¹, что явно не удовлетворит клиентов. Практики, называемые *хаос-инжинирингом* (<https://oreil.ly/SfYxy>), помогают автоматически идентифицировать такие слабости, внедряя в систему разные виды ошибок — например, задержки и сбои обслуживания. Simian Army (<https://oreil.ly/GZmUW>) от Netflix — один из первых примеров набора инструментов, способных выполнить это тестирование. Некоторые из его компонентов теперь внедрены в другие инструменты разработки выпусков, такие как Spinnaker (<https://oreil.ly/HpYx2>).

Использовать нечеткое тестирование можно и для оценки разных реализаций одной функциональности. Если вы хотите перейти из библиотеки А в библиотеку Б, фаззер может генерировать входные данные, передавать их в каждую библиотеку для обработки и сравнивать результаты. Он способен сообщать о любом несоответствующем результате как о «сбое». Это поможет инженерам определить, какие незначительные изменения в поведении могут возникнуть. Такое действие сбоя при разных выходах реализуется как часть драйвера фаззинга, как видно в фаззере BigNum в OpenSSL (<https://oreil.ly/jWQsI>)².

Нечеткое тестирование может выполняться бесконечно, поэтому невозможно блокировать каждое изменение, основываясь на результатах расширенного теста. Это значит, что, когда фаззер обнаруживает ошибку, она может быть уже проверена. Лучше, чтобы другие стратегии тестирования или анализа предотвратили ошибку в первую очередь. Поэтому нечеткое тестирование — это дополнение, генерирующее не предоставленные инженерами тестовые наборы. В качестве дополнительного преимущества в другом модульном тесте можно использовать сгенерированные входные данные, идентифицирующие ошибки в цели фаззинга для гарантии того, что последующие изменения не ухудшат исправление.

¹ См. главу 2 книги Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/implementing-slos/>).

² Цель фаззинга сравнивает результаты двух реализаций модульного возведения в степень в OpenSSL и будет считаться проваленной, если результаты отличаются друг от друга.

Как работают механизмы фаззинга

Механизмы фаззинга различаются по сложности и уровню детализации. На нижнем уровне находится метод, часто называемый *тупым фаззингом*. Он просто считывает байты из генератора случайных чисел и передает их в цель фаззинга в попытке найти ошибки. Механизмы фаззинга становятся умнее благодаря интеграции с наборами инструментов компилятора. Теперь они могут генерировать более интересные и значимые тестовые образцы при помощи возможностей инструментария компилятора, обсуждавшихся ранее. Использовать как можно больше механизмов фаззинга — хорошая практика. Особенно те, которые можно интегрировать в свой инструментарий сборки, и отслеживать такие показатели, как процент покрытия кода тестами. Если в какой-то момент покрытие кода перестает повышаться, нужно выяснить, почему фаззер не может достичь других областей кода.

Некоторые механизмы фаззинга принимают словари интересных ключевых слов из спецификаций или грамматик четко определенных протоколов, языков и форматов (таких как HTTP, SQL и JSON). После механизм фаззинга может генерировать входные данные, которые будут приняты тестируемой программой, ведь они могут быть просто отклонены сгенерированным кодом синтаксического анализатора, если в них есть недопустимые ключевые слова. Предоставив словарь, вы действительно можете создать такой код, который хотите проверить нечетким тестированием. Иначе вы можете получить код, отклоняющий входные данные на основе недействительных токенов, который никогда не находит интересных ошибок.

Такие механизмы фаззинга, как Peach Fuzzer (https://oreil.ly/_n1KP), дают возможность автору драйвера фаззинга программно определять формат ввода и ожидаемые отношения между полями. Поэтому механизм фаззинга может генерировать контрольные примеры, нарушающие эти отношения. Такие механизмы обычно принимают набор образцов входных файлов, называемых *начальными данными*, дающих то, что ожидает тестируемый с помощью фаззинга код. Механизм фаззинга изменяет эти начальные входные данные в дополнение к любым другим поддерживаемым стратегиям генерации входных данных. Некоторые пакеты ПО поставляются с образцами файлов (например, MP3-файлы для аудиобиблиотек или JPEG-файлы для обработки изображений) как часть их существующих наборов тестов. Такие примеры файлов будут отличными кандидатами в начальные данные. Иначе вы можете создать исходные данные из реальных или созданных вручную файлов. Исследователи

в области безопасности публикуют и начальные данные для популярных форматов файлов, к примеру:

- OSS-Fuzz (<https://oreil.ly/K39Q2>);
- Fuzzing Project (<https://oreil.ly/ywq1N>);
- American Fuzzy Lop (AFL) (<https://oreil.ly/mJBh1>).

Последние улучшения в цепочках инструментов компилятора привели к большим успехам в создании более интеллектуальных механизмов фаззинга. В C/C++ такие компиляторы, как LLVM Clang, могут обрабатывать код (как обсуждалось ранее), чтобы механизм фаззинга мог наблюдать, какой код выполняется во время обработки конкретного входного образца. Когда механизм фаззинга находит новый путь кода, он сохраняет образцы, инициировавшие путь кода, и использует их для генерации будущих. Для правильного отслеживания путей выполнения и увеличения покрытия кода другим языкам или механизмам фаззинга может понадобиться специальный компилятор, такой как afl-gcc для AFL (<https://github.com/google/AFL>) или go-fuzz-build для механизма go-fuzz (<https://github.com/dvyukov/go-fuzz>).

Если входные данные вызывают сбой в пути анализируемого кода во время их генерации механизмом фаззинга, он записывает ввод вместе с метаданными, извлеченными из программы в состоянии сбоя. Они могут содержать такую информацию, как трассировка стека, указывающая на вызвавшую сбой строку кода, или схема памяти процесса в данный момент. Благодаря этому инженеры получают подробную информацию о причине сбоя. Она может помочь им понять его природу, подготовить исправления или определить приоритеты ошибок. Например, во время выбора расстановки приоритетов для исправлений разных типов проблем нарушение доступа для записи будет важнее нарушения доступа для чтения. Это способствует формированию культуры безопасности и надежности (см. главу 21).

Реакция вашей программы на запуск механизмом фаззинга потенциальной ошибки зависит от множества обстоятельств. Механизм фаззинга эффективнее всего при обнаружении ошибок, если это приводит к согласованным и четко определенным событиям. Например, к получению сигнала или выполнению какой-либо функции в случае повреждения памяти или неопределенного поведения. Эти функции могут явно оповещать механизм фаззинга о случаях достижения системой определенного состояния ошибки. Многие из упомянутых ранее анализаторов работают именно так.

При помощи некоторых механизмов фаззинга можно установить верхнюю границу времени для обработки определенных сгенерированных входных данных.

Если ситуация тупиковая или выполнение бесконечного цикла приводит к тому, что входные данные превышают ограничение по времени, фаззер классифицирует образец как «сбой». Он сохраняет этот образец для дальнейшего изучения, чтобы команды разработчиков могли предотвратить DoS-уязвимости, способные сделать сервис недоступным.

«ЗАВЕДОМО БЕЗОПАСНЫЕ» ФУНКЦИИ

Иногда безобидные ошибки могут мешать нечеткому тестированию. Так может произойти с кодом, запускающим неопределенную семантику поведения, особенно в программах на C/C++. Например, C++ не определяет, что происходит в случае переполнения знакового целого числа (<https://oreil.ly/LmLCV>). Допустим, что функция использует легко достижимое знаковое целое число, вызывающее сбой `UndefinedBehaviorSanitizer`. Если значение отбрасывается или не используется в таких контекстах, как определение размеров выделения или индексация, у приложения нет никаких последствий для безопасности или надежности. Но этот «мелкий сбой» может помешать нечеткому тестированию достичь более интересного кода. Если невозможно выполнить исправление кода, перейдя к беззнаковым типам или усилив границы, нужно вручную пометить функции как «заведомо безопасные», чтобы отключить определенные анализаторы — например, `__attribute__((no_sanitize("undefined")))` для выявления более глубоких ошибок. Такой подход может привести к ложноотрицательным результатам, поэтому добавляйте аннотации вручную только после тщательного анализа и рассмотрения.

Ошибка Heartbleed (CVE-2014-0160), приводящая к утечке памяти на веб-серверах (включая память, содержащую сертификаты TLS или файлы cookie), может быть быстро выявлена с помощью нечеткого тестирования при использовании правильного драйвера фаззинга и анализатора. GitHub-репозиторий `fuzzer-test-suite` от Google (<https://oreil.ly/f1J7X>) содержит пример `Dockerfile`, показывающий успешную идентификацию ошибки. Вот выдержка из отчета ASan об ошибке Heartbleed, появившейся из-за вызова функции `__asan_memcpy`, добавленного плагином компилятора анализатора (выделено):

```
==19==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x629000009748
at pc
0x0000004e59c9 bp 0x7ffe3a541360 sp 0x7ffe3a540b10
READ of size 65535 at 0x629000009748 thread T0
#0 0x4e59c8 in __asan_memcpy /tmp/final/llvm.src/projects/compilerrt/
lib/asan/asan_interceptors_memintrinsics.cc:23:3
#1 0x522e88 in tls1_process_heartbeat
/root/heartbleed/BUILD/ssl/t1_lib.c:2586:3
#2 0x58f94d in ssl3_read_bytes /root/heartbleed/BUILD/ssl/s3_pkt.c:1092:4
#3 0x59418a in ssl3_get_message /root/heartbleed/BUILD/ssl/s3_both.c:457:7
#4 0x55f3c7 in ssl3_get_client_hello
```

```

/root/heartbleed/BUILD/ssl/s3_srvr.c:941:4
#5 0x55b429 in ssl3_accept /root/heartbleed/BUILD/ssl/s3_srvr.c:357:9
#6 0x51664d in LLVMFuzzerTestOneInput /root/FTS/openssl-
1.0.1f/target.cc:34:3
[...]

0x629000009748 is located 0 bytes to the right of 17736-byte region
[0x629000005200,
0x629000009748)
allocated by thread T0 here:
#0 0x4e68e3 in __interceptor_malloc /tmp/final/llvm.src/projects/compiler-rt/
lib/asan/asan_malloc_linux.cc:88:3
#1 0x5c42cb in CRYPTO_malloc /root/heartbleed/BUILD/crypto/mem.c:308:8
#2 0x5956c9 in freelist_extract /root/heartbleed/BUILD/ssl/s3_both.c:708:12
#3 0x5956c9 in ssl3_setup_read_buffer
/root/heartbleed/BUILD/ssl/s3_both.c:770
#4 0x595cac in ssl3_setup_buffers
/root/heartbleed/BUILD/ssl/s3_both.c:827:7
#5 0x55bff4 in ssl3_accept /root/heartbleed/BUILD/ssl/s3_srvr.c:292:9
#6 0x51664d in LLVMFuzzerTestOneInput /root/FTS/openssl-
1.0.1f/target.cc:34:3
[...]

```

Первая часть выходных данных описывает тип проблемы (в нашем случае `heap-buffer-overflow`, нарушение прав чтения) и символическую легко читаемую трассировку стека, указывающую на точную строку кода, которая читает за пределами выделенного размера буфера. Вторая часть содержит метаданные о соседней области памяти и о том, как она была выделена, чтобы помочь инженеру проанализировать проблему и понять, как процесс достиг недопустимого состояния.

Это можно сделать благодаря инструментарию компилятора и средствам анализа. Но у этого инструментария есть ограничения. Нечеткое тестирование с анализаторами не работает, если некоторые части ПО написаны от руки по соображениям производительности. Компилятор не может обработать код ассемблера, потому что плагины анализатора работают на более высоком уровне. Поэтому необрабатываемый рукописный код сборки может стать причиной ложных срабатываний или необнаруженных ошибок.

Можно провести нечеткое тестирование без анализаторов. Но это уменьшает способность обнаруживать недопустимые состояния программы и доступные для анализа сбоя метаданные. Чтобы фаззинг генерировал полезную информацию, если вы не используете анализатор, программа должна достичь сценария «неопределенного поведения», а после сообщить об этом состоянии ошибки внешнему механизму фаззинга (обычно при сбое или завершении работы).

Иначе неопределенное поведение будет незамеченным. Точно так же и если вы не используете ASan или похожие инструменты, фаззер может не определять состояния, когда память была повреждена, но не использована так, чтобы операционная система завершила процесс.

При работе с библиотеками, доступными только в бинарном виде, инструментарий компилятора вам не подойдет. Некоторые механизмы фаззинга, такие как American Fuzzy Lop, тоже интегрируются с эмуляторами процессора — например, QEMU, для анализа интересных инструкций на уровне процессора. Вы можете выбрать этот тип интеграции для нужных вам бинарных библиотек из-за скорости. Так механизм фаззинга будет понимать, какие пути кода могут генерироваться определенными входными данными по сравнению с другими. Но этот подход не так полезен при обнаружении ошибок, как исходный код, созданный с помощью инструкций анализатора, добавляемых компилятором.

Многие современные механизмы фаззинга, такие как libFuzzer (<https://oreil.ly/uRzhZ>), AFL (<https://oreil.ly/gJ64J>) и Honggfuzz (<https://oreil.ly/b418b>), используют комбинацию ранее описанных методов или их разновидности. Можно создать один драйвер фаззинга, взаимодействующий с несколькими механизмами. В этом случае лучше периодически перемещать интересные входные выборки, сгенерированные каждым, обратно в начальные данные, настроенные для использования другими механизмами фаззинга. Один механизм может успешно получать входные данные, сгенерированные другим, изменяя их и вызывая сбой.

Написание эффективных драйверов фаззинга

Для конкретизации концепций нечеткого тестирования мы подробнее расскажем о драйвере фаззинга с использованием фреймворка, предоставляемого механизмом libFuzzer от LLVM, входящего в состав компилятора Clang. Конкретно этот фреймворк удобен тем, что другие механизмы фаззинга (такие как Honggfuzz и AFL) тоже работают с точкой входа libFuzzer. При использовании этого фреймворка автору фаззера нужно написать только один драйвер, реализующий прототип функции:

```
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size);
```

Соответствующие механизмы фаззинга будут генерировать последовательности байтов и вызывать ваш драйвер, способный передать входные данные в код, который нужно протестировать.

Задача механизмов фаззинга — провести его цель через драйвер как можно быстрее, используя как можно больше уникальных и интересных входных данных.

Для обеспечения воспроизводимых сбоев и быстрого нечеткого тестирования избегайте в своих драйверах фаззинга всего, что описано ниже.

- Недетерминированное поведение. Использование генераторов случайных чисел или специфическое многопоточное поведение.
- Медленные операции. Ведение журнала консоли или дисковый ввод-вывод. Вместо этого попробуйте создать «дружественные к фаззеру» сборки, отключающие эти медленные операции, или воспользуйтесь файловой системой на основе памяти.
- Преднамеренные сбои. Идея нечеткого тестирования в том, чтобы найти неизвестные сбои — механизм фаззинга не сможет явно определить преднамеренно заданные ошибки.

Избегайте всего этого и в других типах тестирования, описанных в этой главе.

Не следует и использовать любые специализированные проверки целостности (такие как CRC32 или дайджесты сообщений), которые злоумышленник может «исправить» в сгенерированном входном образце. Механизм фаззинга вряд ли когда-либо выдаст действительную контрольную сумму и пройдет проверку целостности без специальной логики. Принято использовать флаги препроцессора компилятора, такие как `-DFUZZING_BUILD_MODE_UNSAFE_FOR_PRODUCTION`, чтобы включить это дружественное к фаззеру поведение и помочь воспроизвести идентифицированные с помощью фаззинга сбои.

Пример фаззера

В этом разделе представлены шаги по написанию фаззера для простой библиотеки C++ с открытым исходным кодом под названием Knusperli (<https://oreil.ly/1zV0T>). Knusperli — это JPEG-декодер, способный принимать широкий диапазон входных данных, кодировать загруженные пользователем файлы или обрабатывать изображения (включая потенциально вредоносные) из Интернета.

У Knusperli удобный интерфейс для нечеткого тестирования: функция, принимающая последовательность байтов (JPEG) и параметр размера, а также параметр, контролирующий, какие разделы изображения анализировать. Для ПО, которое не предоставляет такой простой интерфейс, можно воспользоваться вспомогательными библиотеками, такими как FuzzedDataProvider (<https://oreil.ly/HnrdZ>), чтобы преобразовать последовательность байтов в полезные значения для нужного интерфейса. В нашем примере драйвер фаззинга использует эту функцию (<https://oreil.ly/zTtl->):

```
bool ReadJpeg(const uint8_t* data, const size_t len, JpegReadMode mode,
             JPEGData* jpg);
```


Knusperli собирается с помощью системы сборки Bazel (<https://bazel.build/>). Изменяя файл `.bazelrc`, вы можете создать удобный быстрый способ сборки целей с использованием разных средств анализа и напрямую собирать фаззеры на основе libFuzzer. Вот пример для ASan:

```
$ cat ~/.bazelrc
build:asan --copt -fsanitize=address --copt -O1 --copt -g -c dbg
build:asan --linkopt -fsanitize=address --copt -O1 --copt -g -c dbg
build:asan --copt -fno-omit-frame-pointer --copt -O1 --copt -g -c dbg
```

На этом этапе вы сможете создать версию инструмента с включенным ASan:

```
$ CC=clang-6.0 CXX=clang++-6.0 bazel build --config=asan :knusperli
```

Вы можете добавить правило в файл `BUILD` для фаззера, который мы собираемся написать:

```
cc_binary(
  name = "fuzzer",
  srcs = [
    "jpeg_decoder_fuzzer.cc",
  ],
  deps = [
    ":jpeg_data_decoder",
    ":jpeg_data_reader",
  ],
  linkopts = ["-fsanitize=address,fuzzer"],
)
```

Простой пример 13.2 показывает, как может выглядеть драйвер фаззинга.

Пример 13.2. jpeg_decoder_fuzzer.cc

```
1 #include <cstdint>
2 #include <cstdlib>
3 #include "jpeg_data_decoder.h"
4 #include "jpeg_data_reader.h"
5
6 extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t sz) {
7   knusperli::JPEGData jpg;
8   knusperli::ReadJpeg(data, sz, knusperli::JPEG_READ_HEADER, &jpg);
9   return 0;
10 }
```

Собрать и запустить драйвер фаззинга можно с помощью таких команд:

```
$ CC=clang-6.0 CXX=clang++-6.0 bazel build --config=asan :fuzzer
$ mkdir synthetic_corpus
$ ASAN_SYMBOLIZER_PATH=/usr/lib/llvm-6.0/bin/llvm-symbolizer bazel-bin/fuzzer \
  -max_total_time 300 -print_final_stats synthetic_corpus/
```

Предыдущая команда запускает фаззер на пять минут с помощью пустого набора начальных данных. LibFuzzer помещает интересные сгенерированные образцы в каталог *synthetic_corpus/* для использования в будущих сеансах фаззинга. Получаем следующие результаты:

```
[...]
INFO:    0 files found in synthetic_corpus/
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than
4096 bytes
INFO: A corpus is not provided, starting from an empty corpus
#2  INITED cov: 110 ft: 111 corp: 1/1b exec/s: 0 rss: 36Mb
[...]
#3138182    DONE  cov: 151 ft: 418 corp: 30/4340b exec/s: 10425 rss: 463Mb
[...]
Done 3138182 runs in 301 second(s)
stat::number_of_executed_units: 3138182
stat::average_exec_per_sec:    10425
stat::new_units_added:        608
stat::slowest_unit_time_sec:   0
stat::peak_rss_mb:           463
```

Добавление файла JPEG — например, узора из цветных полос, показываемого на телевидении, — улучшает ситуацию. Эти единственные начальные данные улучшают покрытие выполненных блоков кода на 10 % (показатель cov):

```
#2  INITED cov: 169 ft: 170 corp: 1/8632b exec/s: 0 rss: 37Mb
```

Чтобы покрыть еще больше кода, мы можем использовать разные значения для параметра `JpegReadMode`. Допустимые значения (<https://oreil.ly/h4ok1>):

```
enum JpegReadMode {
    JPEG_READ_HEADER, // только основные заголовки
    JPEG_READ_TABLES, // заголовки и таблицы (квантовы, Хаффмана...)
    JPEG_READ_ALL,    // все
};
```

Чтобы не писать три разных фаззера, можно хешировать подмножество входных данных и использовать результат для реализации разных комбинаций библиотечных функций в одном фаззере. Будьте осторожны — используйте достаточно входных данных для создания различных выходных данных хеша. Если формат файла требует, чтобы первые *N* байтов входных данных выглядели одинаково, используйте хотя бы на один больше, чем *N*, чтобы определить, какие байты будут влиять на выбор параметров.

В других подходах нужно использовать `FuzzedDataProvider` для разделения входных данных или выделения первых нескольких байтов данных для задания параметров библиотеки. Оставшиеся байты передаются как входные данные

в цель фаззинга. Хеширование входных данных может привести к совершенно другой конфигурации при изменении даже одного бита. Учитывая это, другие подходы к разделению входных данных позволяют механизму фаззинга лучше отслеживать взаимосвязь между выбранными параметрами и поведением кода. Не забывайте, что эти разные подходы повлияют на удобство использования потенциальных существующих входных данных. Представьте, что вы создаете новый псевдоформат и при этом полагаетесь на первые несколько байтов для задания параметров библиотеки. В итоге вы больше не сможете легко использовать все существующие в мире файлы JPEG как возможные начальные входные данные. Единственное решение — предварительно обработать файлы для добавления начальных параметров.

Чтобы подробнее изучить настройку библиотеки при помощи сгенерированных входных данных, мы используем количество бит в первых 64 байтах входных данных для выбора `JpegReadMode`, как показано в примере 13.3.

Пример 13.3. Фаззинг с разделением входных данных

```
#include <stddef>
#include <stdint>
#include "jpeg_data_decoder.h"
#include "jpeg_data_reader.h"

const unsigned int kInspectBytes = 64;
const unsigned int kInspectBlocks = kInspectBytes / sizeof(unsigned int);

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t sz) {
    knusperli::JPEGData jpg;
    knusperli::JpegReadMode rm;
    unsigned int bits = 0;

    if (sz <= kInspectBytes) { // Отказ при слишком маленьких входных данных
        return 0;
    }

    for (unsigned int block = 0; block < kInspectBlocks; block++) {
        bits +=
            __builtin_popcount(reinterpret_cast<const unsigned int *>(data)[block]);
    }

    rm = static_cast<knusperli::JpegReadMode>(bits %
                                             (knusperli::JPEG_READ_ALL + 1));

    knusperli::ReadJpeg(data, sz, rm, &jpg);

    return 0;
}
```

При использовании цветной полосы как единственного набора входных данных в течение пяти минут этот фаззер дает такие результаты:

```
#851071 DONE cov: 196 ft: 559 corp: 51/29Kb exec/s: 2827 rss: 812Mb
[...]
Done 851071 runs in 301 second(s)
stat::number_of_executed_units: 851071
stat::average_exec_per_sec: 2827
stat::new_units_added: 1120
stat::slowest_unit_time_sec: 0
stat::peak_rss_mb: 812
```

Количество запусков в секунду сократилось. Так произошло, потому что эти изменения позволяют использовать больше возможностей библиотеки, поэтому драйвер фаззинга покрывает гораздо большее количество кода (на что указывает увеличение параметра `cov`). Если запустить фаззер без каких-либо ограничений по времени, он будет продолжать генерировать входные данные до бесконечности, пока не достигнет состояния ошибки анализатора. В этот момент система выдаст отчет, похожий на рассмотренный нами ранее для ошибки Heartbleed. Далее вы сможете внести изменения в код, пересобрать и запустить бинарный файл фаззера, созданный с помощью сохраненного артефакта, чтобы воспроизвести сбой или убедиться, что изменение кода устранило проблему.

Непрерывное нечеткое тестирование

Написав несколько сценариев фаззера и регулярно запуская их в кодовой базе по мере ее разработки, вы будете получать ценную обратную связь для инженеров. Непрерывный конвейер может генерировать ежедневные сборки фаззеров в вашей кодовой базе для использования системой, запускающей их, собирающей информацию о сбоях и регистрирующей ошибки в системе отслеживания заявок. Команды разработчиков могут использовать результаты для выявления уязвимостей или устранения основных причин, заставляющих сервис пропускать SLO.

Пример: ClusterFuzz и OSSFuzz

ClusterFuzz (<https://oreil.ly/10wuR>) — пример реализации масштабируемой инфраструктуры фаззинга с открытым исходным кодом, выпущенной Google. Он управляет наборами виртуальных машин, выполняющих задачи фаззинга, и предоставляет веб-интерфейс для просмотра информации о фаззерах. ClusterFuzz не собирает фаззеры, но ожидает непрерывный конвейер сборки/интеграции, который передаст их в Google Cloud Storage Bucket. У него есть такие функции, как обработка наборов входных данных, выявление дубликатов сбоев и управление жизненным циклом для обнаруженных сбоев. Эмпиризм, используемый ClusterFuzz для выявления дубликатов сбоев, основан на состоянии

программы в момент сбоя. Сохраняя вызывающие сбой образцы, ClusterFuzz может периодически повторно проверять эти проблемы, чтобы определить, воспроизводятся ли они по-прежнему, и автоматически закрывать проблему, если последняя версия фаззера больше не может вызвать сбой на проблемном образце.

Вы можете использовать метрики, показанные веб-интерфейсом ClusterFuzz, чтобы понять, насколько хорошо работает данный фаззер. Доступные метрики зависят от того, что экспортируют встроенные в ваш конвейер сборки механизмы фаззинга (по состоянию на начало 2020 года ClusterFuzz поддерживает libFuzzer и AFL). В документации ClusterFuzz есть инструкции по извлечению информации о покрытии кода тестами из фаззеров, созданные с поддержкой покрытия кода Clang. Там же есть и записи о том, как преобразовать эту информацию в формат, который можно сохранить в контейнере облачного хранилища данных Google и отобразить во внешнем интерфейсе. Использование этой функциональности для изучения покрытого фаззером кода поможет вам при выборе дополнительных улучшений набора входных данных или драйвера фаззинга.

OSS-Fuzz (<https://oreil.ly/tWlyz>) сочетает современные методы фаззинга (<https://oreil.ly/yIaKz>) с масштабируемым распределенным выполнением ClusterFuzz, размещенным на облачной платформе Google. Он находит уязвимости безопасности и проблемы со стабильностью и сообщает о них разработчикам. В течение пяти месяцев после запуска в декабре 2016 года OSS-Fuzz обнаружил более тысячи ошибок (<https://oreil.ly/r-jx6>), а с тех пор распознал еще десятки тысяч.

После интеграции (<https://oreil.ly/AReAc>) проекта с OSS-Fuzz инструмент использует непрерывное и автоматическое тестирование для нахождения проблем спустя несколько часов после добавления измененного кода в репозиторий верхнего уровня, до того, как они затронут пользователей. Объединяя и автоматизируя наши инструменты фаззинга, мы в Google собрали процессы в единый рабочий на основе OSS-Fuzz. Эти интегрированные проекты OSS выигрывают и от того, что они рассматриваются (https://oreil.ly/_TJc) как внутренними инструментами Google, так и внешними инструментами фаззинга. Благодаря такому подходу мы смогли расширить покрытие кода и быстрее обнаруживать ошибки, улучшая состояние безопасности проектов Google и экосистемы проектов с открытым исходным кодом.



Статический анализ программ

Статический анализ — это способ анализа и рассмотрения компьютерных программ путем проверки их исходного кода без выполнения или запуска. Статические анализаторы разбирают исходный код и создают внутреннее представление программы, подходящее для автоматического анализа. Так можно обнаружить потенциальные ошибки в исходном коде

до его проверки или развертывания в производственной среде. Есть много инструментов (<https://oreil.ly/p3yP1>) для разных языков и инструменты для межъязыкового анализа.

У инструментов статического анализа есть разные компромиссы между глубиной и стоимостью анализа исходного кода. Самые мелкие анализаторы выполняют простые сопоставления с образцом на основе текстового или абстрактного синтаксического дерева (abstract syntax tree, AST). Другие методы полагаются на рассуждения о семантических конструкциях программы, основываясь на потоке управления программы и потоке данных.

Инструменты приводят разные компромиссы анализа между ложноположительными (неправильные предупреждения) и ложноотрицательными (пропущенные предупреждения) результатами. Компромиссы неизбежны, отчасти из-за фундаментального ограничения статического анализа. Статическая проверка любой программы — неразрешимая проблема (https://oreil.ly/4CPo_). Невозможно разработать алгоритм, способный определить, будет ли выполняться любая программа без нарушения какого-либо свойства.

Поставщики инструментов учитывают это ограничение и сосредоточены на создании полезных сигналов для разработчиков на разных этапах создания программ. В зависимости от точки интеграции для механизмов статического анализа допустимы разные компромиссы в отношении скорости анализа и ожидаемой обратной связи. Инструмент статического анализа, интегрированный в систему проверки кода, будет нацелен только на недавно разработанный исходный код. Он будет выдавать точные предупреждения, ориентированные на вероятные проблемы. Для исходного кода, окончательно анализируемого перед развертыванием для критически важной для безопасности программы (например, для таких областей, как авиационное программное обеспечение или ПО для медицинских устройств с потенциальными государственными требованиями сертификации), может понадобиться более формальный и строгий анализ¹.

В следующих разделах вы ознакомитесь с методами статического анализа, приспособленными к разным потребностям на разных этапах процесса разработки. Мы выделяем инструменты автоматической проверки кода, инструменты на основе абстрактной интерпретации (этот процесс иногда называют *глубоким статическим анализом*) и более ресурсоемкие подходы — формальные методы. Мы обсудим внедрение статических анализаторов в рабочие процессы.

¹ См.: Bozzano M. et al. Formal Methods for Aerospace Systems // Cyber-Physical System Design from an Architecture Analysis Viewpoint. Singapore: Springer, 2017.

Инструменты автоматической проверки кода

Инструменты автоматической проверки кода выполняют синтаксический анализ исходного кода с учетом особенностей языка и правил использования. Они (еще их называют *линтерами*) обычно не моделирует такое сложное программное поведение, как межпроцедурный поток данных. Они выполняют неглубокий анализ, поэтому инструменты легко масштабируются до произвольных размеров кода. Часто они могут выполнить анализ исходного кода примерно за то же время, которое нужно для компиляции. Инструменты проверки кода легко расширяемы — вы можете просто добавить новые правила, охватывающие многие типы ошибок, особенно связанных с языковыми функциями.

В последние годы инструменты проверки кода были сосредоточены на стилистических изменениях и удобочитаемости. Так получилось из-за того, что такие предложения по улучшению кода более приемлемы для разработчиков. Многие организации применяют проверки стиля и формата по умолчанию для поддержания целостной кодовой базы, которой проще управлять в больших командах разработчиков. Там же регулярно проводятся проверки, выявляющие возможные запахи кода (<https://oreil.ly/ONE8f>) и вероятные ошибки.

В следующем примере основное внимание уделяется инструментам, выполняющим конкретный тип анализа — сопоставление с шаблоном AST. AST — это древовидное представление исходного кода программы, основанное на синтаксической структуре языка программирования. Компиляторы обычно разбирают входной файл кода на такое представление, а потом используют его в процессе компиляции. Например, AST может содержать узел в виде конструкции `if-then-else`, который имеет три дочерних узла: один для условного оператора `if`, один, представляющий поддерево для ветви `then`, и еще один, представляющий поддерево из ветви `else`.

Error Prone (<https://errorprone.info/>) для Java и Clang-Tidy (https://oreil.ly/qFh_k) для C/C++ широко используются в проектах Google. Инженеры могут использовать оба этих анализатора для добавления пользовательских проверок. На начало 2018 года (<https://oreil.ly/gKJDy>) 162 автора добавили 733 проверки в Error Prone. Для определенных типов ошибок как Error Prone, так и Clang-Tidy могут предложить шаблоны исправлений. Некоторые компиляторы (например, Clang и MSVC) тоже поддерживают разработанные сообществом основные рекомендации по C++ (<https://oreil.ly/y2Eqd>). С помощью библиотеки поддержки рекомендаций (Guideline Support Library, GSL) и самих рекомендаций можно предотвратить многие частые ошибки в программах на C++.

С помощью инструментов сопоставления с шаблоном в AST пользователи могут добавлять новые проверки, записывая правила в анализируемое дерево.

Рассмотрим предупреждение `absl-string-findstartswith` (<https://oreil.ly/3w5sM>) в Clang-Tidy. Инструмент пытается улучшить читаемость и производительность кода, проверяющего совпадения префикса строки, используя API `string::find` (<https://oreil.ly/tg1HX>) в C++. Clang-Tidy рекомендует использовать API `StartsWith`, предоставляемый ABSL (<https://oreil.ly/lmrox>). Для выполнения анализа инструмент создает шаблон поддерева AST, сравнивающий выходные данные API `string::find` в C++ с целочисленным значением 0. Инфраструктура Clang-Tidy предоставляет инструмент для поиска шаблонов поддерева AST в представлении AST анализируемой программы.

Рассмотрим фрагмент кода:

```
std::string s = "...";
if (s.find("Hello World") == 0) { /* do something */ }
```

Предупреждение `absl-string-find-startWith` Clang-Tidy помечает этот фрагмент и предлагает изменить код таким образом:

```
std::string s = "...";
if (absl::StartsWith(s, "Hello World")) { /* do something */ }
```

Чтобы предложить исправление, Clang-Tidy (выражаясь концептуально) дает возможность изменить поддерево AST с учетом шаблона. Левая сторона рис. 13.1 показывает соответствие шаблону AST (для ясности поддерево AST упрощено). Если инструмент находит соответствующее поддерево AST в дереве AST исходного кода, он уведомляет об этом разработчика. Узлы AST содержат информацию о строках и столбцах. Так инструмент может сообщать о конкретном предупреждении разработчику.

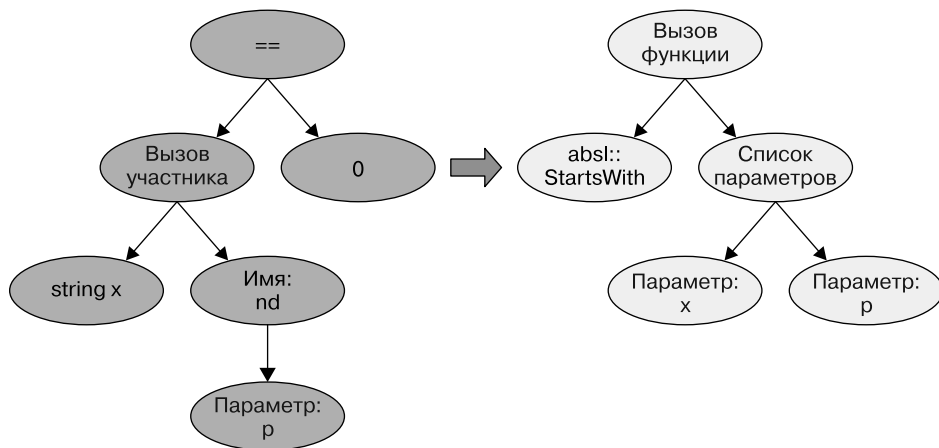


Рис. 13.1. Совпадение с шаблоном AST и предложение по замене

В дополнение к проверкам производительности и читабельности Clang-Tidy предоставляет множество общих проверок шаблонов ошибок. Попробуйте запустить Clang-Tidy для следующих входных файлов¹:

```
$ cat -n sizeof.c
1 #include <string.h>
2 const char* kMessage = "Hello World!";
3 int main() {
4 char buf[128];
5 memcpy(buf, kMessage, sizeof(kMessage));
6 return 0;
7 }

$ clang-tidy sizeof.c
[...]
Running without flags.
1 warning generated.
sizeof.c:5:32: warning: 'memcpy' call operates on objects of type 'const char'
while the size is based on a different type 'const char *'
[clang-diagnostic-sizeof-pointer-memaccess]
memcpy(buf, kMessage, sizeof(kMessage));
    ^
sizeof.c:5:32: note: did you mean to provide an explicit length?
memcpy(buf, kMessage, sizeof(kMessage));

$ cat -n sizeof2.c
1 #include <string.h>
2 const char kMessage[] = "Hello World!";
3 int main() {
4 char buf[128];
5 memcpy(buf, kMessage, sizeof(kMessage));
6 return 0;
7 }

$ clang-tidy sizeof2.c
[...]
Running without flags.
```

Два входных файла отличаются только объявлением типа `kMessage`. Когда `kMessage` определяется как указатель на инициализированную память, `sizeof(kMessage)` возвращает размер типа указателя. Поэтому Clang-Tidy выдает предупреждение `clang-Diagnostics-sizeof-pointer-memaccess`. Когда `kMessage` имеет тип `const char[]`, операция `sizeof(kMessage)` возвращает ожидаемую длину, а Clang-Tidy не выдает предупреждение.

Для некоторых проверок шаблонов, помимо предупреждающего сообщения, Clang-Tidy может предложить исправления кода. Предупреждение `absl-string-find-startsWith` в Clang-Tidy, приведенное выше, — один из примеров такого

¹ Вы можете установить Clang-Tidy при помощи стандартных менеджеров пакетов. Обычно используется название `clang-tidy`.

поведения. В правой части рис. 13.1 показана замена на уровне AST. Когда такие предложения доступны, вы можете позволить Clang-Tidy автоматически применять их к входному файлу, используя параметр командной строки `--fix`.

Вы можете использовать автоматически применяемые предложения для обновления кодовой базы, пользуясь исправлением `modernize` в Clang-Tidy. Рассмотрим последовательность команд, показывающую шаблон `modernize-use-nullptr` (<https://oreil.ly/9K8vD>). Последовательность находит экземпляры нулевых констант, используемых для назначения или сравнения указателей, и заменяет их на `nullptr`. Для запуска всех проверок `modernize` мы используем Clang-Tidy с опцией `--check=modernize-*`, а после применяем предложения к входному файлу с помощью `--fix`. В конце последовательности команд мы выделяем четыре изменения, выводя преобразованный файл (выделено):

```
$ cat -n nullptr.cc
1 #define NULL 0x0
2
3 int *ret_ptr() {
4     return 0;
5 }
6
7 int main() {
8     char *a = NULL;
9     char *b = 0;
10    char c = 0;
11    int *d = ret_ptr();
12    return d == NULL ? 0 : 1;
13 }
```

```
$ clang-tidy nullptr.cc -checks=modernize-* --fix
[...]
Running without flags.
4 warnings generated.
nullptr.cc:4:10: warning: use nullptr [modernize-use-nullptr]
    return 0;
        ^
        nullptr
nullptr.cc:4:10: note: FIX-IT applied suggested code changes
    return 0;
        ^
nullptr.cc:8:13: warning: use nullptr [modernize-use-nullptr]
    char *a = NULL;
            ^
            nullptr
nullptr.cc:8:13: note: FIX-IT applied suggested code changes
    char *a = NULL;
            ^
nullptr.cc:9:13: warning: use nullptr [modernize-use-nullptr]
```

```

char *b = 0;
    ^
    nullptr
nullptr.cc:9:13: note: FIX-IT applied suggested code changes
char *b = 0;
    ^
nullptr.cc:12:15: warning: use nullptr [modernize-use-nullptr]
return d == NULL ? 0 : 1;
    ^
    nullptr
nullptr.cc:12:15: note: FIX-IT applied suggested code changes
return d == NULL ? 0 : 1;
    ^
clang-tidy applied 4 of 4 suggested fixes.

```

```

$ cat -n nullptr.cc
1 #define NULL 0x0
2
3 int *ret_ptr() {
4     return nullptr;
5 }
6
7 int main() {
8     char *a = nullptr;
9     char *b = nullptr;
10    char c = 0;
11    int *d = ret_ptr();
12    return d == nullptr ? 0 : 1;
13 }

```

У других языков похожие автоматизированные инструменты проверки кода. GoVet (<https://oreil.ly/m815w>) анализирует исходный код на Go, выявляя распространенные подозрительные конструкции, Pylint (<https://www.pylint.org/>) анализирует код на Python, а Error Prone дает возможности анализа и автоматического исправления для программ на Java. В следующем примере кратко показано, как запустить Error Prone с помощью правила сборки Bazel (выделено). В Java операция вычитания `i - 1` для переменной `i` типа `Short` возвращает значение типа `int`. Поэтому операция `remove` невозможна:

```

$ cat -n ShortSet.java
1 import java.util.Set;
2 import java.util.HashSet;
3
4 public class ShortSet {
5     public static void main (String[] args) {
6         Set<Short> s = new HashSet<>();
7         for (short i = 0; i < 100; i++) {
8             s.add(i);
9             s.remove(i - 1);
10        }
11        System.out.println(s.size());

```

```

12  }
13  }

$ bazel build :hello
ERROR: example/myproject/BUILD:29:1: Java compilation in rule
'//example/myproject:hello'
ShortSet.java:9: error: [CollectionIncompatibleType] Argument 'i - 1' should
not be
passed to this method;
its type int is not compatible with its collection's type argument Short
  s.remove(i - 1);
           ^
(see http://errorprone.info/bugpattern/CollectionIncompatibleType)
1 error

```

Интеграция статического анализа в рабочий процесс

Хорошей практикой будет запустить быстрые инструменты статического анализа как можно раньше при разработке. Очень важно найти ошибки раньше, ведь стоимость их исправления сильно вырастает после помещения в репозиторий с исходным кодом или развертывания в производственной среде.

Порог для интеграции инструментов статического анализа в ваш конвейер CI/CD низок. От этого производительность инженеров может повыситься. Разработчик может получить сообщение об ошибке и предложение о том, как исправить разыменование нулевого указателя. Если при этом код нельзя сохранить, то нельзя забыть об исправлении проблемы, тем самым случайно вызвав сбой системы. Раскрыть конфиденциальную информацию тоже будет сложно. Это положительно скажется на культуре безопасности и надежности (см. главу 21).

Для этого Google разработала платформу анализа программ Tricorder¹ и ее версию с открытым исходным кодом — Shipshape (<https://github.com/google/shipshape>). Tricorder статически анализирует около 50 000 изменений кода в день. Платформа запускает много инструментов анализа программ разных типов и выдает предупреждения разработчикам во время проверки кода. Это помогает им оценивать предложения. Инструменты нацелены на предотвращение удобочитаемого кода, который легко исправить, с низким уровнем воспринимаемых пользователем ложных срабатываний (не более 10 %).

С Tricorder пользователи могут запускать много разных инструментов анализа программ. На начало 2018 года платформа содержала 146 анализаторов, охватывающих более 30 исходных языков. Большая часть этих анализаторов была предоставлена разработчиками Google. Общедоступные инструменты

¹ См.: *Caitlin Sadowski et al. Lessons from Building Static Analysis Tools at Google* // Communications of the ACM, 2018. № 61 (4). P. 58–66. doi: 10.1145/3188720.

статического анализа несложные. Большинство анализаторов под управлением Tricorder — это инструменты автоматической проверки кода. Они подходят для разных языков и могут проверять соответствие рекомендациям по стилю кода и находить ошибки. Как мы уже говорили, Error Prone и Clang-Tidy могут предлагать исправления в определенных сценариях. Далее программист может применить эти исправления одним нажатием кнопки.

На рис. 13.2 показаны результаты анализа Tricorder заданного входного файла Java, представленного рецензенту. Результаты показывают два предупреждения: одно от линтера Java и одно от Error Prone. Tricorder измеряет уровень воспринимаемых пользователем ложных срабатываний. Так рецензент может предоставлять обратную связь по появившимся предупреждениям, кликая по ссылке **Not useful** («Не подходит»). Команда Tricorder пользуется этими сигналами для отключения отдельных проверок. Для отдельных предупреждений рецензент может отправить автору кода запрос **Please fix** («Пожалуйста, исправьте»).



Рис. 13.2. Результаты статического анализа кода от Tricorder во время рецензирования

На рис. 13.3 показаны автоматически добавленные изменения кода, предложенные Error Prone во время рецензирования.

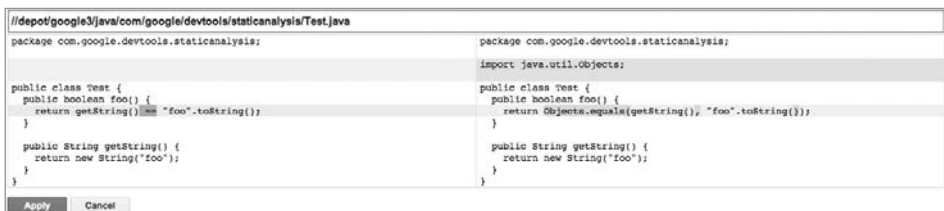


Рис. 13.3. Предпросмотр исправленной версии кода после предупреждения Error Prone с рис. 13.2

ОБРАТНЫЙ ИНЖИНИРИНГ И ГЕНЕРАЦИЯ ТЕСТОВЫХ ДАННЫХ

Методы программного анализа, как статические, так и динамические, использовались не только для тестирования и обеспечения правильности кода, но и для обратной разработки ПО. Она может быть полезна при попытке понять поведение бинарных файлов с недоступным исходным кодом. Обратный инжиниринг (реверс-инжиниринг) обычно используется, когда инженер по безопасности пытается понять, как работает потенциально вредоносный бинарный файл. Этот анализ часто содержит использование *декомпиляторов* или *дизассемблеров*. Дизассемблер переводит машинный язык в код ассемблера, а декомпилятор переводит программу с машинного языка в исходный код. Нет гарантии, что эти инструменты могут заново создать заданный исходный код. Но сгенерированный код может помочь инженеру по безопасности для понимания программы и ее поведения. Один из популярных инструментов обратной разработки — Ghidra (<https://ghidra-sre.org/>).

Некоторые инженеры используют способы анализа программ для генерации тестовых данных. Одна из техник, которая в последнее время стала популярной, — *конколическое тестирование* (*concolic testing*). Она объединяет регулярное выполнение (называемое *конкретным*, так как использует конкретные значения) с методами символического анализа (отсюда и название «конколическое» — *конкретное + символическое*). Далее пользователи могут автоматически генерировать тестовые входные данные, гарантированно показывающие некоторые пути выполнения в целевой программе. Конколические тесты могут обеспечить эту гарантию, выполняя эту программу с конкретным вводом (например, целочисленное значение 123) и затеняя каждый шаг выполнения формулой, соответствующей наблюдаемым выражениям и ветвям. Вы можете заменить конкретное входное значение 123 символом *a*. По мере выполнения в каждой точке ветвления, например, в операторе *if*, конколическое тестирование спрашивает решатель задач удовлетворения ограничений о возможности другого значения для *a*, которое приведет к переходу на альтернативную ветвь. С каждым полученным входным значением вы можете запускать новые конколические проверки, увеличивающие покрытие ветви. KLEE (<https://klee.github.io/>) — один из популярных инструментов конколического тестирования.

Абстрактная интерпретация

Основанные на абстрактной интерпретации инструменты статически выполняют семантический анализ поведения программы¹. Этот метод успешно применялся для проверки критически важного программного обеспечения — например, ПО для управления полетом². Рассмотрим простой пример программы, генерирующей десять наименьших положительных четных целых чисел. Во время обычного выполнения программа генерирует целочисленные значения 2, 4, 6, 8, 10, 12, 14, 16, 18 и 20. Для обеспечения эффективного стати-

¹ См.: Cousot P., Cousot R. Static Determination of Dynamic Properties of Programs // Proceedings of the 2nd International Symposium on Programming, 1976. P. 106–130. <https://oreil.ly/4xLgB>.

² Souyris J. et al. Formal Verification of Avionics Software Products // Proceedings of the 2nd World Conference on Formal Methods, 2009. P. 532–546. doi: 10.1007/978-3-642-05089-3_34.

ческого анализа такой программы нужно суммировать все возможные значения при помощи компактного представления, охватывающего все наблюдаемые значения. Используя интервал или область диапазона, мы можем представить все наблюдаемые значения, подменяя их абстрактными значениями интервала [2, 20]. Область интервала позволяет статическому анализатору эффективно рассуждать обо всех программах, просто запоминая самые низкие и самые высокие возможные значения.

Убедитесь, что все возможные варианты поведения программы фиксируются. Для этого важно покрыть все наблюдаемые значения абстрактным представлением. Но этот подход вводит приближение, способное привести к *неточности* или ложным предупреждениям. Например, если мы хотим гарантировать, что заданная программа никогда не выдаст значение 11, анализ с использованием целочисленной области приведет к ложноположительному результату.

Использующие абстрактную интерпретацию статические анализаторы обычно вычисляют абстрактное значение для каждой программной точки. Для этого они полагаются на представление программы в *графе потока управления* (control-flow graph, CFG). CFG обычно используются при оптимизации компилятора и статическом анализе программ. Каждый узел в нем — это базовый блок программы, соответствующий последовательности операторов программы, всегда выполняемых по порядку. То есть внутри этой последовательности операторов нет переходов, а в середине нет целей перехода. Ребра в CFG — это управляющая логика программы, где скачок происходит через внутрипроцедурную (из-за операторов `if` или циклов) или межпроцедурную управляющую логику из-за вызовов функций. Заметьте, что представление CFG используется фаззерами для покрытия кода тестами (обсуждалось ранее). Например, libFuzzer отслеживает, какие основные блоки и ребра покрыты во время фаззинга. При помощи этой информации фаззер решает, учитывать ли эти входные данные для будущих изменений.

Основанные на абстрактной интерпретации инструменты выполняют семантический анализ, основанный на потоке данных и управляющей логике в программах, часто при вызове функций. Поэтому они работают дольше, чем ранее обсуждавшиеся инструменты автоматической проверки кода. Хотя инструменты автоматической проверки кода можно интегрировать в интерактивные среды разработки, например в редакторы кода, абстрактная интерпретация не интегрируется так же. Поэтому разработчики периодически запускают основанные на абстрактной интерпретации инструменты для определенного кода (например, ночью), анализируя только измененный код и повторно используя результаты анализа для кода без изменений.

Во многих инструментах абстрактная интерпретация применяется для разных языков и свойств. С помощью инструмента Frama-C (<https://frama-c.com/>) можно

находить типичные ошибки выполнения и нарушения утверждений, включая переполнение буфера, ошибки сегментации из-за висячих или нулевых указателей и деление на ноль в написанных на C программах. Как мы уже обсуждали, эти ошибки, особенно связанные с памятью, могут нести последствия для безопасности. Infer (<https://fbinfer.com/>) анализирует выполняемые программами изменения памяти и указателей и может находить такие ошибки, как висячие указатели в Java, C и других языках. AbsInt (<https://www.absint.com/>) способен анализировать выполнение задач для худших сценариев в системах реального времени. С помощью программы App Security Improvement (ASI) (<https://oreil.ly/60tIV>) можно выполнить сложный межпроцедурный анализ каждого Android-приложения, загруженного в Google Play Store, обеспечивая безопасность и защиту. При обнаружении уязвимости ASI отмечает ее и предлагает способы устранить проблему. Рисунок 13.4 показывает пример предупреждения безопасности. На начало 2019 года эта программа предложила более миллиона исправлений приложений (<https://oreil.ly/my8fa>) в Play Store для более чем 300 000 разработчиков приложений.

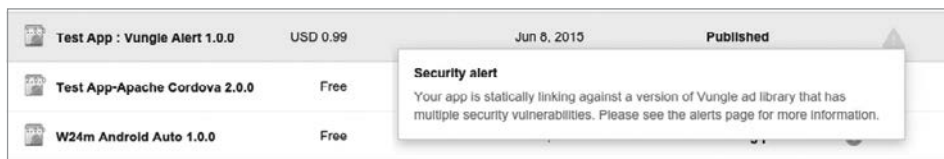


Рис. 13.4. Оповещение об улучшении безопасности приложения

Формальные методы

С помощью *формальных методов* пользователи могут указывать важные свойства для программных или аппаратных систем. Большинство из них — это *свойства безопасности*. Они указывают на то, что определенное плохое поведение никогда не должно наблюдаться. Примером «плохого поведения» могут быть утверждения в программах. Есть и *свойства живучести*, позволяющие пользователям указывать желаемый результат. Например, что отправленное задание печати в итоге будет обработано принтером. Разработчики, использующие формальные методы, могут проверять эти свойства для конкретных систем или моделей и даже разрабатывать такие системы при помощи подходов, *основанных на правильном построении*. В подразделе «Анализ инвариантов» в главе 6 говорится о дороговизне подходов, основанных на формальных методах. Так происходит потому, что для этих подходов нужно предварительно описать интересующие системные требования и свойства. Они должны быть определены математически строго и формально.

Подходы, основанные на формальных методах, успешно внедрились в аппаратные средства проектирования и проверки¹. Сейчас стандартная практика при проектировании оборудования — использование формальных или полуформальных инструментов, предоставляемых поставщиками средств автоматизации проектирования электроники (electronic design automation, EDA). Эти методы успешно применяются и к специализированному ПО (критические системы безопасности или анализ криптографических протоколов). Например, подход, основанный на формальных методах, постоянно анализирует криптографические протоколы, используемые в TLS в коммуникациях компьютерных сетей².

Резюме

Тестирование программного обеспечения для обеспечения безопасности и надежности — большая тема, и мы рассмотрели только малую часть. Стратегии тестирования, представленные здесь, и практика написания безопасного кода для устранения целых классов ошибок (см. главу 12) — ключевые параметры надежного масштабирования команд в Google. Они сводят к минимуму простои и проблемы с безопасностью. Создавайте ПО с учетом тестируемости на самых ранних этапах и проводите всестороннее тестирование на протяжении всей разработки.

Сейчас мы хотим подчеркнуть ценность полной интеграции всех этих методов тестирования и анализа в ваши рабочие процессы и конвейеры CI/CD. Сочетая и регулярно используя эти методы в кодовой базе, вы сможете быстрее выявлять ошибки. Вы повысите уверенность в своей способности обнаруживать и предотвращать ошибки при развертывании приложений — тема, которая будет рассмотрена в следующей главе.

¹ См.: Kern C., Greenstreet M. R. Formal Verification in Hardware Design: A Survey // ACM Transactions on Design Automation of Electronic Systems, 1999. № 4 (2). P. 123–193. doi: 10.1145/307988.307989. См. также: Hunt Jr. et al. Industrial Hardware and Software Verification with ACL2 // Philosophical Transactions of The Royal Society A Mathematical Physical and Engineering Sciences, 2017. doi: 10.1098/rsta.2015.0399.

² См.: Chudnov A. et al. Continuous Formal Verification of Amazon s2n // Proceedings of the 30th International Conference on Computer Aided Verification, 2018. P. 430–446. doi: 10.1007/978-3-319-96142-2_26.

Развертывание

*Джеремиа Спрадлин и Марк Лодато,
Сергей Симаков и Роксана Лоза*

Работает ли код в производственной среде именно так, как вы предполагаете? Вашей системе нужны средства контроля для обнаружения и предотвращения небезопасных развертываний. Само развертывание вносит изменения в систему, и любое из них может стать проблемой надежности или безопасности. Избежать развертывания небезопасного кода можно, если внедрить средства контроля на ранних этапах жизненного цикла разработки ПО. Эта глава начинается с определения модели угроз в цепочке поставок программного обеспечения и рекомендаций по защите от этих угроз. После мы углубимся в такие расширенные стратегии смягчения угроз, как использование верифицируемых сборок и политик развертывания на основе происхождения. В самом конце вы получите несколько практических советов о том, как внедрить такие изменения.

Ранее мы рассмотрели, как учитывать безопасность и надежность при написании и тестировании кода. Но у кода нет реального влияния на систему до сборки и развертывания. Поэтому важно тщательно продумать безопасность и надежность всех элементов процесса сборки и развертывания. Может быть трудно определить безопасность развернутого артефакта, просто осматривая его. Контроль на разных этапах цепочки поставок ПО повысит вашу уверенность в безопасности программного артефакта. Например, анализ кода (код-ревью) снизит вероятность ошибок и удержит злоумышленников от вредоносных изменений, а автоматизированные тесты могут повысить уверенность в правильности работы кода.

Эффект средств контроля, построенных вокруг инфраструктуры исходного кода, сборки и тестирования, ограничен, если злоумышленники могут обойти их, развернув код прямо в системе. Поэтому системы должны отклонять развертывания, проходящие мимо правильной цепочки поставок ПО. Для выполнения этого требования у каждого шага в цепочке поставок должно быть доказательство его правильного выполнения.

Концепции и терминология

Цепочка поставок программного обеспечения — процесс написания, сборки, тестирования и развертывания программной системы. Эти шаги содержат типичные обязанности системы управления версиями (version control system, VCS), конвейера непрерывной интеграции (continuous integration, CI) и непрерывного развертывания (continuous delivery, CD).

Хотя детали реализации разные для каждой компании и команды, у большинства процесс выглядит примерно так, как показано на рис. 14.1.

1. Код должен быть проверен в системе контроля версий.
2. Затем код собирается из проверенной версии.
3. После сборки бинарный файл должен быть протестирован.
4. Далее код разворачивается в среде, где происходит его настройка и запуск.

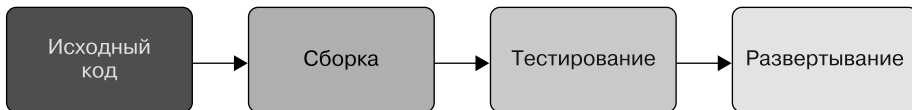


Рис. 14.1. Общее представление о типичной цепочке поставок ПО

Даже если ваша цепочка поставок сложнее, чем эта модель, вы можете разбить ее на основные составляющие. На рис. 14.2 есть конкретный пример того, как типичный конвейер развертывания выполняет эти шаги.

Разработайте цепочку поставок ПО, чтобы смягчить угрозы для системы. В этой главе основное внимание уделяется снижению угроз со стороны инсайдеров (или злоумышленников, притворяющихся ими) независимо от их намерений. К примеру, инженер из лучших побуждений может случайно собрать код, содержащий непроверенные изменения, или внешний злоумышленник может попытаться развернуть нужный ему исполняемый файл при помощи привилегий скомпрометированной учетной записи инженера. Оба этих сценария для нас равнозначны.

В этой главе мы определяем этапы цепочки поставок программного обеспечения в широком смысле.

Сборка — это любое преобразование входных артефактов в выходные, где *артефакт* — это любая часть данных, например файл, пакет, коммит в Git или образ виртуальной машины (ВМ). *Тест* — это особый случай сборки, где выходной артефакт — это логический результат, обычно «успех» или «неудача», а не какой-либо файл или исполняемый сценарий.

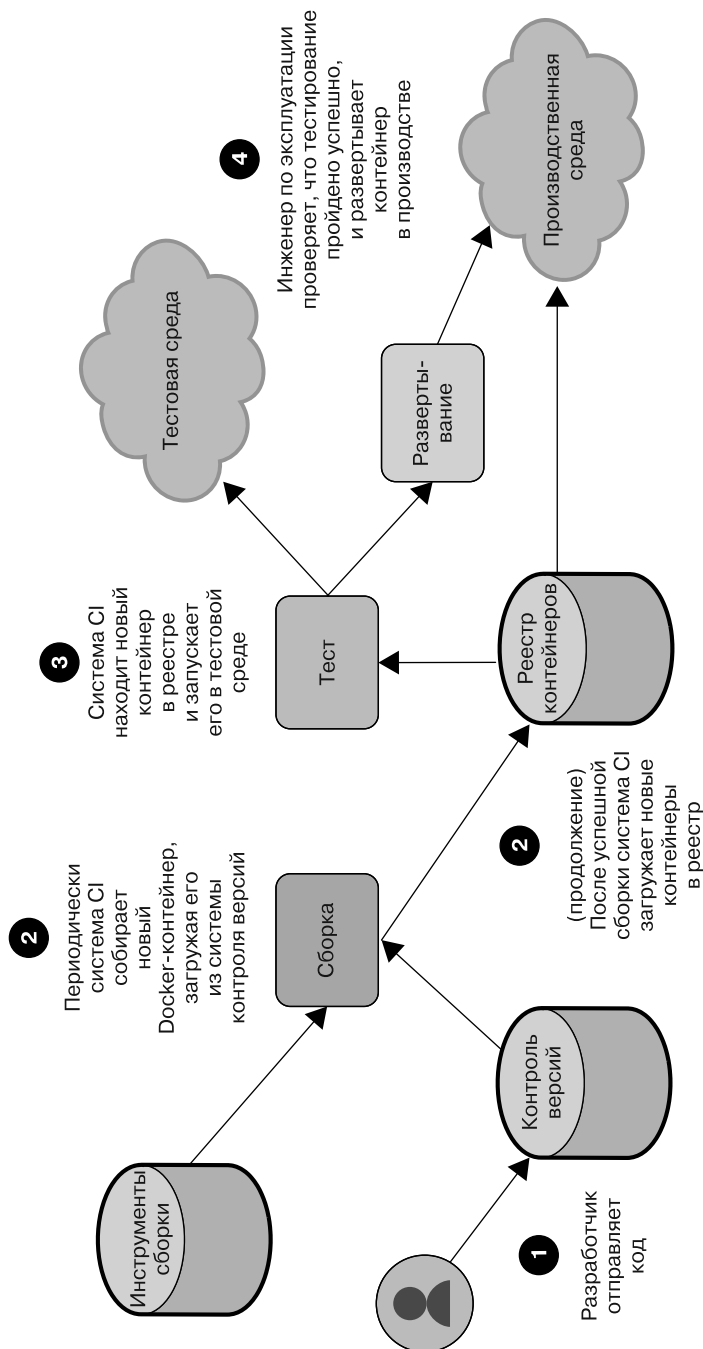


Рис. 14.2. Типичное развертывание облачных сервисов на основе контейнеров

Сборки могут быть объединены, а артефакт протестирован несколько раз. Например, процесс выпуска может сначала «собрать» бинарные файлы из исходного кода, затем «собрать» образ Docker из бинарных файлов, а потом «протестировать» образ, запустив его в среде разработки.

Развертывание — это любое присвоение какого-либо артефакта какой-либо среде. Каждое из следующих действий можно рассматривать как развертывание.

- Загрузка кода.
 - Ввод команды, заставляющей сервер загружать и запускать новый бинарный файл.
 - Обновление объекта развертывания Kubernetes для получения нового образа Docker.
 - Запуск виртуальной или физической машины, загружающей исходное программное или микропрограммное обеспечение.
- Обновление конфигурации.
 - Выполнение команды SQL для изменения схемы БД.
 - Обновление объекта развертывания Kubernetes для изменения флага командной строки.
- Публикация пакета или иных данных, которые будут применяться другими пользователями.
 - Загрузка пакета deb в apt-репозиторий.
 - Загрузка образа Docker в реестр контейнеров.
 - Загрузка APK в Google Play Store.

Изменения после развертывания не будут обсуждаться в этой главе.

Модель угроз

Перед защитой цепочки поставок вашего ПО от угроз вы должны определить возможных противников. Здесь мы рассмотрим следующие три типа противников. Организация или тип системы могут влиять на этот список:

- добросовестные инсайдеры, которые могут совершать ошибки;
- злонамеренные инсайдеры, которые пытаются получить больше доступа, чем позволяет их роль;
- сторонние злоумышленники, скомпрометировавшие компьютер или учетную запись одного или нескольких инсайдеров.

Глава 2 описывает профили злоумышленника и дает руководство о моделировании защиты от инсайдерского риска.

Представьте себя в роли злоумышленника и определите все способы, которыми он может подорвать цепочку поставок ПО для компрометации вашей системы. Ниже приведены примеры распространенных угроз. Вы можете изменить этот список под себя, чтобы отразить конкретные угрозы для вашей организации. Для простоты мы используем термин «инженер», обозначая добросовестных инсайдеров, а «злоумышленник» — для обозначения злонамеренных инсайдеров и атакующих извне.

- Инженер вносит изменение, случайно приводящее к уязвимости в системе.
- Злоумышленник вносит изменение, которое оказывается лазейкой или приводит к какой-либо другой преднамеренной уязвимости в системе.
- Инженер случайно проводит сборку из локально модифицированной версии кода, содержащей непроверенные изменения.
- Инженер развертывает бинарный файл с вредоносной конфигурацией. Например, в изменении есть функции отладки, предназначенные только для тестирования.
- Злоумышленник развертывает модифицированный бинарный файл в производственной среде, что приводит к утечке клиентских учетных данных.
- Злоумышленник изменяет ACL облачного хранилища, что приводит к утечке данных.
- Злоумышленник крадет ключ целостности, используемый для подписи ПО.
- Инженер разворачивает старую версию кода с известной уязвимостью.
- Система CI неправильно настроена. Это позволяет отправлять запросы на сборку из произвольных репозиторий с исходным кодом. В результате злоумышленник может создать репозиторий с вредоносным кодом.
- Злоумышленник загружает в систему CI свой сценарий сборки, получая ключ подписи. Далее он использует этот ключ для подписи и развертывания вредоносного бинарного файла.
- Злоумышленник обманывает систему CD при помощи взломанного компилятора или инструмента сборки, создающего вредоносный бинарный файл.

После составления полного списка возможных противников и угроз соотнесите определенные угрозы с уже принятыми мерами. Вы должны задокументировать любые ограничения текущих стратегий смягчения последствий. Так вы сможете увидеть полную картину потенциальных рисков в вашей системе. В первую очередь обратите внимание на угрозы без соответствующих мер по снижению риска или угрозы, для которых существующие меры сильно ограничены.

Лучшие практики

Следующие рекомендации помогут вам устранить угрозы, заполнить любые пробелы в безопасности, найденные в модели угроз, и постепенно улучшить состояние безопасности цепочки поставок ПО.

Сделайте рецензирование обязательным

Рецензирование кода — это практика, когда другой (или несколько) человек проверяет изменения в исходном коде до их тестирования и развертывания¹. В дополнение к повышению безопасности кода рецензирование дает много преимуществ для программного проекта. Оно способствует обмену знаниями и обучению, прививает нормы кодирования, улучшает читаемость кода и уменьшает количество ошибок². Все это помогает в создании культуры безопасности и надежности (подробнее об этой идее см. в главе 21).

С точки зрения безопасности рецензирование кода — форма многосторонней авторизации³. Это значит, что никто не имеет права самостоятельно вносить изменения. Как описано в главе 5, многосторонняя авторизация обеспечивает множество преимуществ безопасности.

Рецензирование должно быть обязательным. Злоумышленник просто не сможет удержаться, если будет знать, что его код не будет проверен! Рецензирование должно быть полным, чтобы не пропустить явный проблемный код. Рецензент должен понимать детали любого изменения и его последствия для системы или запрашивать у автора разъяснения. Иначе процесс может обернуться штамповкой проверок на автомате⁴.

Многие общедоступные инструменты позволяют выполнять обязательные проверки кода. Можно настроить GitHub, GitLab или BitBucket так, чтобы они требовали определенного количества подтверждений для каждого запроса с из-

¹ Проверка кода применяется и к изменениям в файлах конфигурации. См. подраздел «Относитесь к настройкам как к коду» далее.

² См.: *Sadowski C. et al.* Modern Code Review: A Case Study at Google // Proceedings of the 40th International Conference on Software Engineering, 2018. P. 181–190. doi: 10.1145/3183519.3183525.

³ В сочетании с применением к настройкам тех же правил, что и к коду, и политиками развертывания, описанными в этой главе, рецензирование становится базой многосторонней системы авторизации для произвольных систем.

⁴ Подробнее об обязанностях рецензента кода см. в подразделе «Культура рецензирования» в главе 21.

менениями. Вы можете использовать и автономные системы рецензирования, такие как Gerrit или Phabricator, настроив хранилище исходного кода на прием запросов только от заданной системы рецензирования.

У рецензирования кода есть ограничения в отношении безопасности, как описано во введении к главе 12. Поэтому его лучше применять как одну из мер безопасности «защиты в глубину» наряду с автоматизированным тестированием (описанным в главе 13) и рекомендациями из главы 12.

Автоматизируйте

В идеале автоматизированные системы должны выполнять большинство шагов в цепочке поставок ПО¹. У автоматизации есть ряд преимуществ. Она может обеспечить целостный, воспроизводимый процесс для сборки, тестирования и развертывания ПО. Уменьшение ручной работы в цикле помогает предотвратить ошибки и снижает трудозатраты. При запуске автоматизации цепочки поставок ПО изолированная система более защищена от вредоносных действий со стороны злоумышленников.

Рассмотрим сценарий, где инженеры вручную собирают «производственные» бинарные файлы на своих рабочих станциях по мере необходимости. В этом случае появляется много возможностей для ошибок. Инженеры могут случайно собрать неправильную версию кода или включить в него непроверенные изменения, не прошедшие рецензирования. Теперь злоумышленники, взломавшие компьютер инженера, могут намеренно перезаписать локально созданные бинарные файлы вредоносными версиями. Оба этих сценария можно предотвратить с помощью автоматизации.

Безопасно добавить автоматизацию может быть сложно, так как автоматизированная система сама по себе может создать другие прорехи в безопасности. Во избежание самых распространенных классов уязвимостей мы рекомендуем следующее.

Перенесите все этапы сборки, тестирования и развертывания в автоматизированные системы

Как минимум запишите все шаги. Это позволяет и людям, и средствам автоматизации выполнять одинаковые действия, что обеспечит согласованность. Для этого можно использовать системы CI/CD (например, Jenkins (<https://jenkins.io/>)). Рассмотрите возможность создания политики, требующей

¹ Цепочка шагов не обязательно должна быть полностью автоматизированной. Человек обычно может запускать этап сборки или развертывания. Но у него не должно быть возможности как-либо повлиять на поведение системы на этом этапе.

автоматизации всех новых проектов, так как может быть трудно добавлять автоматизацию к уже существующим системам.

Требуйте экспертной оценки для всех изменений настроек в цепочке поставок программного обеспечения

Обрабатывать настройки как код (в скором времени мы обсудим это) — лучший способ достижения этой цели. Обязательная проверка уменьшает шансы на ошибку и увеличивает стоимость вредоносных атак.

Изолируйте автоматизированную систему для предотвращения вмешательства администраторов или пользователей

Это самый сложный шаг, и детали его реализации выходят за рамки этой главы. Если вкратце, то рассмотрите все способы, как администратор может внести изменения без проверки. Например, настроив конвейер CI/CD напрямую или используя SSH для запуска команд на компьютере. Для каждого способа рассмотрите возможность предотвращения такого доступа без экспертной оценки.

Больше об изоляции автоматической системы сборки см. в подразделе «Верифицируемые сборки» далее в этой главе.

Автоматизация — беспроектный вариант, снижающий трудозатраты и повышающий надежность и безопасность. Автоматизируйте все, что можно автоматизировать!

Проверяйте не только людей, но и артефакты

У средств контроля инфраструктурой исходного кода, сборки и тестирования ограниченный эффект, если злоумышленники могут обойти их путем развертывания системы прямо в рабочей среде. Мало проверить, *кто* инициировал развертывание, потому что этот кто-то может ошибиться или преднамеренно внедрить вредоносное изменение¹. Вместо этого среды развертывания должны проверять, *что* развертывается.

Среды развертывания должны требовать доказательства того, что система прошла все автоматизированные этапы процесса развертывания. Люди должны быть неспособны обойти автоматизацию, пока какое-либо средство контроля не проверит это действие. Если вы работаете в Google Kubernetes Engine (GKE), то можете использовать двоичную авторизацию (<https://oreil.ly/0jsVi>), чтобы по

¹ При этом такие проверки полномочий все еще важны для принципа наименьших привилегий (см. главу 5).

умолчанию принимать только образы, подписанные вашей системой CI/CD, и отслеживать журнал аудита кластера Kubernetes для уведомлений, когда кто-то использует механизм аварийного доступа для развертывания несовместимого образа¹.

Ограничение этого подхода в том, что он предполагает, что все компоненты вашей настройки безопасны. Например, что система CI/CD принимает запросы на сборку только от разрешенных в производственной среде источников, что ключи подписи (если они используются) доступны только для систем CI/CD и т. д. В разделе «Расширенные стратегии смягчения угроз» далее в этой главе описывается более надежный способ проверки желаемых свойств напрямую с меньшим количеством неявных допущений.

Относитесь к настройкам как к коду

Настройки сервиса, как и его код, важны для безопасности и надежности. Поэтому все рекомендации для контроля версий кода и проверки изменений можно применить и к настройкам. Относитесь к настройкам как к коду, требуя, чтобы изменения в них были проверены, прорецензированы и протестированы до развертывания, как и любое другое изменение².

Пример: на вашем внешнем сервере есть опция конфигурации для указания внутреннего интерфейса. Выбор тестовой версии сервера в производственном интерфейсе вызвал бы серьезную проблему безопасности и надежности.

В качестве более практического примера рассмотрим систему, использующую Kubernetes и сохраняющую конфигурацию в файле YAML (<https://yaml.org/>) при помощи системы управления версиями³. Процесс развертывания вызывает бинарный файл `kubect1` и передает файл YAML, который развертывает утвержденную конфигурацию. Ограничение этого процесса использованием только «одобренного» YAML — YAML из системы управления версиями с обязательным рецензированием — помогает избежать неправильной настройки сервиса.

¹ Механизм аварийного доступа может обходить политики, позволяя инженерам быстро устранять простои. См. подраздел «Механизм аварийного доступа» в главе 5.

² Эта концепция подробнее обсуждается в главе 8 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/release-engineering/>) и в главах 14 (<https://landing.google.com/sre/workbook/chapters/configuration-design/>) и 15 (<https://landing.google.com/sre/workbook/chapters/configuration-specifications/>) книги Site Reliability Engineering. Все эти рекомендации применимы и здесь.

³ YAML — это язык конфигурации (<https://oreil.ly/UKo2t>), используемый Kubernetes.

Можно повторно использовать все средства контроля и рекомендации из этой главы для защиты настроек вашего сервиса. Обычно проще повторно использовать эти подходы, чем выбирать другие способы защиты изменений настроек после развертывания. Для этого часто нужно устанавливать отдельную систему многосторонней авторизации.

Создание версий и рецензирование кода — более распространенная практика, чем создание версий и рецензирование настроек. Даже организации, реализующие настройки как код, обычно не рассматривают настройки так же строго. Инженеры знают, что не следует создавать рабочую версию бинарного файла из локально модифицированной копии исходного кода. Те же инженеры не будут так же внимательны перед развертыванием изменения в настройках, не сохранив сначала изменения в системе управления версиями и не запросив рецензию.

Для реализации настроек как кода нужны изменения в вашей культуре, инструментах и процессах. В культурном отношении уделяйте больше внимания процессу рецензирования. Технически нужны инструменты, позволяющие легко сравнивать предлагаемые изменения (например, `diff`, `grep`) и отменять их вручную в случае чрезвычайной ситуации¹.

НЕ ПРОВЕРЯЙТЕ СЕКРЕТНЫЕ ДАННЫЕ!

Пароли, криптографические ключи и токены авторизации часто нужны для работы сервиса. Безопасность системы зависит от сохранения конфиденциальности этих секретов. Полная их защита не будет здесь обсуждаться, но мы выделили несколько важных советов.

- Никогда не храните секретные данные в системах управления версиями и не встраивайте их в исходный код. Может понадобиться встраивать *зашифрованные* секретные данные в исходный код или переменные среды, например, для их дешифрования и внедрения системой сборки. Это удобно, но может усложнить централизованное управление секретными данными.
- По возможности храните секретные данные в подходящей системе управления или шифруйте их с помощью системы управления ключами, такой как Cloud KMS (<https://cloud.google.com/kms>).
- Строго ограничивайте доступ к секретным данным. Предоставляйте его только сервисам и только при необходимости. Никогда не предоставляйте людям прямой доступ к секретным данным. Человеку обычно нужен доступ к паролям, а не секретным данным приложения. Если это возможно, создайте отдельные учетные данные для пользователей и сервисов.

¹ Вы должны регистрировать и проверять ручные переопределения, чтобы помешать преступнику использовать их в качестве вектора атаки.

Защита в соответствии с моделью угроз

Теперь, когда мы определили некоторые рекомендации, мы можем сопоставить их с описанными ранее угрозами. Оценивая процессы внедрения безопасности с точки зрения вашей конкретной модели угроз, спросите себя: нужно ли следовать всем рекомендациям? Достаточно ли они смягчают угрозы? В табл. 14.1 приведены примеры угроз, соответствующие меры по их смягчению и возможные ограничения.

Таблица 14.1. Примеры угроз, меры по смягчению и потенциальных ограничений

Угроза	Смягчение	Ограничения
Инженер отправляет изменение, которое случайно вводит уязвимость в систему	Проверка кода и автоматизированное тестирование (см. главу 13). Такой подход сильно снижает вероятность ошибок	
Злоумышленник вносит изменение, добавляющее лазейку в систему, или вводит какую-либо другую преднамеренную уязвимость	Рецензирование кода. Это увеличивает стоимость атак и вероятность обнаружения — злоумышленник должен тщательно обработать изменение, чтобы оно прошло проверку	Не защищает от сговора инсайдеров или внешних злоумышленников, способных скомпрометировать несколько внутренних учетных записей
Инженер случайно проводит сборку из локально модифицированной версии кода, содержащей непроверенные изменения	Сборка с помощью автоматизированной системы CI/CD, всегда извлекающая данные из заданного исходного хранилища	
Инженер разворачивает вредоносные настройки. Например, изменение содержит функции отладки, которые предназначались только для тестирования	Относитесь к настройкам как к исходному коду и требуйте для них такого же уровня рецензирования	Не все настройки могут рассматриваться как код
Злоумышленник разворачивает модифицированный бинарный файл в производственной среде, что приводит к утечке учетных данных клиента	Производственная среда должна требовать доказательств того, что бинарный файл был создан системой CI/CD. Система CI/CD настраивается на извлечение исходного кода только из заданного репозитория	Злоумышленник может выяснить, как обойти это требование при помощи процедуры аварийного развертывания (см. раздел «Практические советы» этой главы). Предотвратит это помогут правильное ведение журнала и аудит

Угроза	Смягчение	Ограничения
Злоумышленник изменяет ACL облачного хранилища, что приводит к утечке данных	Рассматривайте ACL ресурса как настройки. Облачное хранилище должно допускать изменения настроек только в процессе развертывания. Поэтому люди должны быть неспособны вносить изменения	Не защищает от сговора инсайдеров или внешних злоумышленников, способных скомпрометировать несколько внутренних учетных записей
Злоумышленник крадет ключ целостности, используемый для подписи ПО	Сохраните ключ целостности в системе управления, поддерживающей замену ключей, которая настроена так, чтобы ключ был доступен только системе CI/CD. Для получения дополнительной информации см. главу 9. Предложения конкретно для сборки см. в следующем разделе «Расширенные стратегии смягчения угроз»	

На рис. 14.3 показана обновленная цепочка поставок ПО, содержащая перечисленные в предыдущей таблице угрозы и меры по их смягчению.

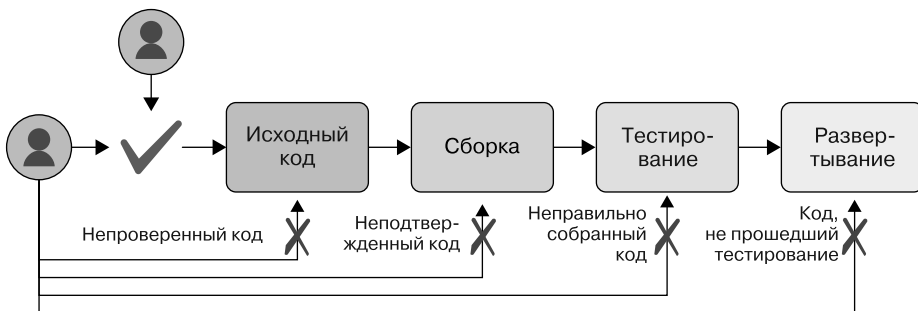


Рис. 14.3. Типичная цепочка поставок программного обеспечения — злоумышленники не должны иметь возможность обойти процесс

Нам еще предстоит сопоставить несколько угроз с мерами по их смягчению из рекомендаций.

- Инженер разворачивает старую версию кода с известной уязвимостью.
- Система CI неправильно настроена и разрешает запросы на сборку из произвольных репозиторий с исходным кодом. В итоге преступник может создать исходный репозиторий с вредоносным кодом.

- Злоумышленник загружает в систему CI свой сценарий сборки, извлекающий ключ подписи. Далее он использует этот ключ для подписи и разворачивания вредоносного бинарного файла.
- Злоумышленник обманывает систему CD при помощи взломанного компилятора или инструмента сборки, создающего вредоносный бинарный файл.

Для предотвращения этих угроз внедрите дополнительные средства контроля, которые мы рассмотрим в следующем разделе. Только вы можете решить, стоит ли учитывать эти угрозы в вашей организации.

ДОВЕРЕННЫЙ СТОРОННИЙ КОД

Современная разработка ПО обычно предполагает использование стороннего кода и систем с открытым исходным кодом. Если ваша организация использует такие зависимости, нужно выяснить, как снизить сопутствующие риски.

Если вы полностью доверяете людям, поддерживающим проект, процесс проверки кода, систему контроля версий и процесс импорта/экспорта, защищенный от несанкционированного доступа, то импортировать сторонний код в сборку просто. Загружайте его так, как будто он был предоставлен любой из ваших систем контроля версий.

Но если эти люди не полностью заслуживают вашего доверия или если проект не гарантирует проверки кода, тогда добавьте уровень проверки кода перед сборкой. Вы можете даже сохранить локальную копию стороннего кода и просматривать все предлагаемые исправления самостоятельно.

Сложность проверки будет зависеть от уровня вашего доверия к поставщику. Важно понимать сторонний код, который вы используете, и применять тот же уровень строгости к стороннему коду, что и к собственному.

Всегда отслеживайте зависимости для отчетов об уязвимостях и быстрой установки исправлений безопасности.



Расширенные стратегии смягчения угроз

Вам могут понадобиться комплексные меры по снижению риска для устранения некоторых более сложных угроз в цепочке поставок ПО. Рекомендации в этом разделе нельзя назвать стандартными для всей отрасли, поэтому для их применения может потребоваться создание настраиваемой инфраструктуры. Эти рекомендации лучше всего подходят для крупных и/или особо чувствительных к безопасности организаций и могут не пригодиться для небольших организаций с низким уровнем инсайдерских рисков.

Бинарное происхождение

У каждой сборки должно быть *бинарное происхождение* (*binary provenance*) — информация о том, как был создан бинарный артефакт: входные данные, преобразование и объект, выполнивший сборку.

Чтобы было понятнее, зачем это нужно, предположим, что вы расследуете инцидент безопасности и видите, что развертывание произошло в течение определенного периода времени. Нужно определить, связано ли развертывание с инцидентом. Стоимость обратной разработки бинарного кода была бы очень высокой. Намного проще проверить исходный код с учетом изменений в управлении версиями. Но как вы узнаете, из какого исходного кода пришел бинарный файл?

Даже если вы думаете, что такие расследования безопасности вам не пригодятся, бинарное происхождение будет полезно для политик развертывания на основе происхождения. Это мы обсудим далее в разделе.

Что указывать в бинарном происхождении

Информация, которую нужно добавить в происхождение, зависит от допущений, встроенных в вашу систему, и сведений, которые могут понадобиться потребителям. Для включения расширенных политик развертывания и проведения специального анализа добавьте к происхождению следующие поля.

Подлинность (обязательное)

Неявная информация о сборке — какая система провела ее и почему можно доверять происхождению. Обычно в этом случае используется криптографическая подпись, защищающая остальные поля бинарного происхождения¹.

Выходные данные (обязательное)

Выходные артефакты, к которым относится это бинарное происхождение. Обычно каждый вывод идентифицируется криптографическим хешем содержимого артефакта.

Входные данные

То, что вошло в сборку. С помощью этого поля верификатор сможет связать свойства исходного кода со свойствами артефакта. Добавьте следующую информацию.

¹ Обратите внимание, что подлинность подразумевает целостность.

Источники

«Основные» входные артефакты для сборки, такие как дерево исходного кода, в котором выполнялась команда сборки верхнего уровня. Например: «Коммит в Git 270f...ce6d из <https://github.com/mysql/mysqlserver>»¹ или «файл `foo.tar.gz` с содержимым SHA-256 78c5 ... 6649».

Зависимости

Все остальные нужные для сборки артефакты, такие как библиотеки, инструменты сборки и компиляторы, не полностью указанные в источниках. Каждый из них может повлиять на целостность сборки.

Команда

Используемая для запуска сборки команда. Например: `bazel build //main:hello-world`. В идеале это поле структурировано для автоматического анализа, поэтому пример может быть таким — `{"bazel": {"command": "build", "target": "//main:hello_world"}}`.

Среда

Любая другая важная для воспроизведения сборки информация, такая как детали архитектуры или переменные среды.

Входные метаданные

Иногда компоновщик может читать метаданные о входных данных, которые могут понадобиться последующим системам. Он может включать временную метку исходного коммита, которую система оценки политики потом использует во время развертывания.

Отладочная информация

Любая дополнительная информация, ненужная для безопасности, но полезная для отладки, например, идентификатор машины, на которой выполнялась сборка.

Версионность

Временная метка сборки и номер версии формата происхождения часто полезны для учета будущих изменений. Можно пометить старые сборки как недействительные или изменить формат, с возможностью отката при атаке.

¹ Идентификаторы коммитов в Git — это криптографические хеши, обеспечивающие целостность всего дерева исходного кода.

Вы можете опустить неявные поля или те, что указываются в самом исходном коде. В формате происхождения Debian нет команды `build` — всегда используется `dpkg-buildpackage`.

У входных артефактов должны быть *идентификатор* (URI) и *версия* (криптографический хеш). Обычно идентификатор используется для проверки подлинности сборки — проверки того, что код поступил из правильного исходного хранилища. Версия полезна для разных целей, таких как специальный анализ, создание воспроизводимых сборок и проверка связанных этапов сборки, где выходные данные этапа i — это выходные данные для этапа $i + 1$.

Помните о поверхности атаки. Проверяйте все, что не проверено системой сборки (и подразумевается подписью) или не содержится в исходном коде (и проверено коллегами) в нисходящем направлении. Если инициировавший сборку пользователь может указать произвольные флаги компилятора, верификатор должен проверить их. Например, флаг `-D` в GCC позволяет пользователю перезаписывать произвольные символы и полностью изменять поведение бинарного файла. Точно так же, если пользователь может указать свой компилятор, тогда верификатор должен убедиться, что использовался «правильный». Чем больше проверок может выполнить процесс сборки, тем лучше.

Хороший пример бинарного происхождения — в формате `deb-buildinfo` (https://oreil.ly/WNUw_) в Debian. Для получения обобщенных рекомендаций см. документацию проекта «Воспроизводимые сборки» (Reproducible Builds) (<https://oreil.ly/Y5VFW>). Рассмотрите JSON Web Tokens (JWT) (<https://jwt.io/>), где представлен стандартный способ подписывания и кодирования этой информации.

ПОДПИСАНИЕ КОДА

Подписание кода (<https://oreil.ly/f4gdr>) часто используется как механизм безопасности для повышения доверия к бинарным файлам. Но будьте осторожны, применяя его, ведь ценность подписи полностью зависит от того, что она представляет и насколько хорошо защищен ключ подписи.

Рассмотрим случай, когда можно доверять бинарному файлу Windows, если он имеет любую действительную подпись `Authenticode`. Чтобы обойти этот контроль, злоумышленник может либо купить (<https://oreil.ly/EiiiU>), либо украсть (https://oreil.ly/j_OCo) действующий сертификат подписи, который стоит от нескольких сотен до нескольких тысяч долларов (в зависимости от типа). Хотя этот подход ценен для безопасности, его выгода ограничена.

Для повышения эффективности подписания кода явно перечислите принимаемых подписчиков и заблокируйте доступ к соответствующим ключам подписи. Еще нужно убедиться, что среда, в которой происходит подписание кода, защищена, чтобы злоумышленник не мог злоупотреблять процессом подписи и подписывать свои вредоносные бинарные файлы. Считайте процесс получения действительной подписи кода «развертыванием» и следуйте рекомендациям из этой главы.

Политики развертывания на основе происхождения

В подразделе «Проверяйте не только людей, но и артефакты» выше мы рекомендовали, чтобы официальный конвейер автоматизации сборки проверял, что именно развертывается. Как вы проверяете правильность настройки конвейера? А что, если вы хотите добавить конкретные гарантии для некоторых сред развертывания, которые не применимы к другим средам?

Вы можете использовать явные политики развертывания, описывающие предполагаемые свойства каждой среды для решения этих проблем. Далее среды могут сопоставлять эти политики с двоичным происхождением артефактов, развернутых в них.

У такого подхода есть несколько преимуществ по сравнению с подходом, основанным только на подписывании.

- Он уменьшает количество неявных допущений в цепочке поставок ПО, облегчая анализ и гарантируя правильность.
- В нем разъясняются обязательства каждого шага в цепочке поставок ПО, что снижает вероятность неправильной настройки.
- Он позволяет использовать по одному ключу подписи на каждый шаг сборки, а не на среду развертывания, так как теперь можно использовать бинарное происхождение для принятия решений о развертывании.

Предположим, что у вас есть архитектура микросервисов и нужно гарантировать, что каждый микросервис может быть собран только из кода, переданного в репозиторий с исходным кодом микросервиса. При использовании подписи кода вам будет нужен один ключ на репозиторий с исходным кодом, а система CI/CD должна будет выбрать правильный ключ подписи на основе репозитория. Недостаток этого подхода в том, что сложно проверить, соответствует ли конфигурация системы CI/CD этим требованиям.

Используя основанные на происхождении политики развертывания, система CI/CD создает бинарное происхождение, в котором указывается репозиторий с исходным кодом, всегда подписанный одним ключом. В политике развертывания для каждого микросервиса указывается, какой репозиторий разрешен. Проверять правильность теперь намного проще, чем при подписании кода, ведь политика развертывания описывает свойства всех микросервисов в одном месте.

Правила, перечисленные в политике развертывания, смягчат угрозы для вашей системы. Обратитесь к модели угроз для выбранной системы. Какие правила можно определить для их смягчения? Вот несколько примеров правил, которые вы можете захотеть реализовать.

- Исходный код был передан в систему контроля версий и проверен.
- Исходный код поступил из определенного местоположения, такого как конкретная цель сборки и репозиторий.
- Сборка производилась через официальный конвейер CI/CD (см. подраздел «Верифицируемые сборки» далее).
- Код прошел тестирование.
- Двоичный файл был явно разрешен для этой среды развертывания. Не разрешайте «тестировать» двоичные файлы в производственной среде.
- Версия кода или сборки достаточно свежая¹.
- В коде нет известных уязвимостей, о чем свидетельствует недавнее сканирование безопасности².

Фреймворк in-toto (<https://in-toto.github.io/>) предоставляет один стандарт для реализации политик происхождения.

Реализация решений для политик

При реализации собственного механизма для политик развертывания на основе происхождения нужно выполнить три шага.

1. Убедитесь в *подлинности происхождения*. Этот шаг неявно проверяет целостность происхождения, предотвращая его подделку или взлом. Обычно это означает проверку того, что происхождение было криптографически подписано определенным ключом.
2. Убедитесь, что *происхождение относится к артефакту*. Этот шаг неявно проверяет целостность артефакта, гарантируя, что злоумышленник не может применить «хорошее» происхождение к «плохому» артефакту. Это означает сравнение криптографического хеша артефакта со значением из полезной нагрузки происхождения.
3. Убедитесь в *соответствии происхождения всем правилам*.

Простой пример такого процесса — правило, требующее, чтобы артефакты были подписаны определенным ключом. Эта единственная проверка выполняет все три шага: проверяет действительность подписи, что артефакт применяется к подписи и что подпись присутствует.

¹ В подразделе «Минимально допустимые номера версий безопасности» в главе 9 обсуждается откат до уязвимых версий.

² Вам может понадобиться подтверждение того, что Cloud Security Scanner (<https://oreil.ly/mgTi7>) не обнаружил ничего в тестовом экземпляре, выполняющем эту версию кода.

Рассмотрим пример посложнее: «Образ Docker должен быть собран из репозитория `mysql/mysql-server` в GitHub». Допустим, что ваша система сборки использует ключ K_b для подписания происхождения сборки в формате JWT. В этом случае схема полезной нагрузки токена будет следующей, где `sub` — это URI RFC 6920 (https://oreil.ly/_8zJm):

```
{
  "sub": "ni:///sha-256;...",
  "input": {"source_uri": "..."}
}
```

Чтобы оценить, удовлетворяет ли артефакт этому правилу, движок должен проверить следующее.

- Подпись JWT проверяется с использованием ключа K_b .
- `sub` соответствует хешу SHA-256 артефакта.
- `input.source_uri` — это `"https://github.com/mysql/mysql-server"`.

Верифицируемые сборки

Мы называем сборку *верифицируемой*, если созданное ей бинарное происхождение заслуживает доверия¹. Верифицируемость — в глазах смотрящего. Доверяете вы конкретной системе сборки или нет — зависит от модели угроз и от того, как система сборки вписывается в общую историю безопасности организации.

Подумайте, подходят ли следующие примеры нефункциональных требований вашей организации². Добавьте все требования, отвечающие вашим нуждам.

- Если одна рабочая станция скомпрометирована, целостность двоичного происхождения или выходных артефактов не будет скомпрометирована.
- Злоумышленник не может незаметно изменять происхождение или выходные артефакты.
- Одна сборка не может влиять на целостность другой, как параллельная, так и последовательная.
- Сборка не может предоставлять происхождение с ложной информацией. Например, происхождение не может утверждать, что артефакт был собран из коммита `abc...def` в Git, если он был собран из `123...456`.

¹ Напомним, что чистые подписи все еще считаются «двоичным происхождением», как описано в предыдущем разделе.

² См. раздел «Цели и требования при проектировании» главы 4.

- Не администраторы не могут настраивать такие определенные пользователем этапы сборки, как Makefile или сценарий Jenkins Groovy, так, чтобы это нарушало любое требование из списка.
- Снимок всех исходных артефактов доступен в течение как минимум N месяцев после сборки. Это позволяет проводить возможные расследования.
- Сборка воспроизводима (см. врезку «Герметичная, воспроизводимая или верифицируемая?» далее). Это может понадобиться, даже если верифицируемая архитектура сборки не нуждается в этом условии, как определено в следующем разделе. К примеру, воспроизводимые сборки могут быть полезны для независимой проверки двоичного происхождения артефакта при обнаружении инцидента или уязвимости.

Верифицируемые архитектуры сборки

Цель верифицируемой системы сборки — повысить доверие верификатора к двоичному происхождению, создаваемому этой системой. Независимо от конкретных требований к верифицируемости определяются три основные архитектуры.

Доверенный сервис сборки

Нужно, чтобы исходная сборка была выполнена сервисом сборки, которому верификатор доверяет. Это значит, что доверенный сервис сборки подписывает бинарное происхождение ключом, доступным только этому сервису.

У такого подхода есть преимущества, связанные с необходимостью сборки только единожды и без воспроизводимости (см. врезку «Герметичная, воспроизводимая или верифицируемая?» далее). Google использует эту модель для внутреннихборок.

Самостоятельные повторные сборки

Верификатор воспроизводит сборку на лету, чтобы проверить бинарное происхождение. Например, если оно утверждает, что получено из коммита `abc...def` в Git, верификатор выбирает коммит в Git, повторно запускает перечисленные в двоичном происхождении команды сборки и проверяет, что выходные данные побитово идентичны проверяемому артефакту. Чтобы получить дополнительную информацию о воспроизводимости, см. следующую врезку.

Этот подход может привлекать, потому что вы доверяете только самому себе, но он не масштабируется. На сборки уходят минуты или часы, тогда как решения по развертыванию должны приниматься за миллисекунды. Это требует от сборки ее полной воспроизводимости, что не всегда практично. См. врезку ниже для получения дополнительной информации.

Сервис повторных сборок

Верификатор требует, чтобы некоторый кворум «сборщиков» воспроизвел сборку и подтвердил подлинность двоичного происхождения. Это гибрид двух предыдущих вариантов. На практике этот подход обычно означает, что каждый сборщик отслеживает репозиторий пакетов, предварительно пересобирает каждую новую версию и сохраняет результаты в БД. Далее верификатор просматривает записи в N различных БД, на которые указывает криптографический хеш рассматриваемого артефакта. Такие проекты с открытым исходным кодом, как Debian (<https://oreil.ly/zNZ7G>), используют эту модель, когда модель централизованного управления невозможна или нежелательна.

ГЕРМЕТИЧНАЯ, ВОСПРОИЗВОДИМАЯ ИЛИ ВЕРИФИЦИРУЕМАЯ?

Концепции воспроизводимых и герметичных сборок тесно связаны с воспроизводимыми сборками. Здесь терминология не стандартизирована¹, поэтому мы предлагаем такие определения:

Герметичность

Все входные данные для сборки полностью определены заранее, вне процесса сборки. В дополнение к исходному коду это требование распространяется на все компиляторы, инструменты, библиотеки и любые другие входные данные, способные повлиять на сборку. Все ссылки должны быть однозначными — в виде полностью разрешенных номеров версий или криптографических хешей. Информация о герметичности регистрируется как часть исходного кода, но допустимо, чтобы эта информация существовала извне, например, в файле Debian `.buildinfo` (<https://oreil.ly/O7EIS>).

Герметичные сборки обладают такими преимуществами.

- Позволяют анализировать входные данные сборки и применять правила политики. Примеры от Google содержат обнаружение уязвимого ПО, которое требует исправления, используя БД Common Vulnerabilities and Exposures (CVE), обеспечение соответствия лицензиям с открытым исходным кодом и предотвращение использования запрещенного политикой ПО (известной небезопасной библиотеки).
- Гарантируют целостность стороннего импорта. К примеру, проверяя криптографические хеши зависимостей или требуя, чтобы все запросы использовали HTTPS и поступали из надежных репозиториях.
- Позволяют выбирать нужный код. Вы можете исправить ошибку, исправив код, повторно собрав двоичный файл и применив его без каких-либо посторонних изменений в поведении, например, вызванных использованием другой версии компилятора. Отбор кода сильно снижает риск, связанный с аварийными выпусками, которые могут не проходить столько испытаний и проверок, сколько обычные.

¹ В книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/release-engineering/#hermetic-builds-nqslhnd>) термины «герметичный» и «воспроизводимый» взаимозаменяются. Проект «Воспроизводимые сборки» (Reproducible Builds) (<https://reproducible-builds.org/>) определяет *воспроизводимость* так же, как и в этой главе, но в их примерах «*воспроизводимый*» иногда означает «*проверяемый*».

В примерах герметичных сборок есть Bazel (<https://bazel.build/>) при запуске в режиме песочницы и npm (<https://www.npmjs.com/>) при использовании *package-lock.json*.

Воспроизводимость

Выполнение одинаковых команд сборки с одинаковыми входными данными гарантирует получение побитово идентичных выходов. Для воспроизводимости почти всегда нужна герметичность¹.

У воспроизводимых сборок есть ряд преимуществ.

- *Верифицируемость* — верификатор может определить бинарное происхождение артефакта, воспроизводя сборку самостоятельно или при помощи кворума сборщиков, как описано в начале подраздела «Верифицируемые сборки» выше.
- *Герметичность* — невоспроизводимость часто указывает на негерметичность. Непрерывное тестирование на воспроизводимость поможет быстро найти негерметичность и обеспечить все описанные ранее преимущества герметичности.
- *Кэширование сборки* — воспроизводимые сборки позволяют лучше кэшировать промежуточные артефакты сборки в больших графах, таких как в Bazel.

Чтобы сделать сборку воспроизводимой, удалите все, что может быть недетерминированным, и предоставьте всю нужную для воспроизведения сборки информацию (известную как *buildinfo*). Если компилятор добавляет метку времени в выходной артефакт, установите для нее фиксированное значение или включите ее в *buildinfo*. В большинстве случаев требуется целиком указать полный набор инструментов и ОС. Разные версии обычно производят немного разные результаты. За практическими советами обращайтесь на сайт Reproducible Builds (<https://reproducible-builds.org/>).

Верифицируемость

Вы можете достоверно определить бинарное происхождение артефакта, например, информацию о том, из каких источников он был собран. Желательно (но не обязательно), чтобы верифицируемые сборки тоже были воспроизводимыми и герметичными.

Реализация верифицируемых сборок

Независимо от того, является верифицируемый сервис сборки «доверенным сервисом сборки» или «сервисом повторных сборок», помните о ряде важных соображений проектирования.

На базовом уровне почти все системы CI/CD работают в соответствии с шагами на рис. 14.4. Сервис принимает запросы, выбирает любые нужные входные данные, выполняет сборку и записывает их в систему хранения.

¹ Как противоположный пример рассмотрим процесс сборки, выбирающий последнюю версию зависимости во время сборки, но в остальном выдающий идентичные выходные данные. Этот процесс воспроизводим, пока две сборки происходят примерно в одно время, но он не герметичен.

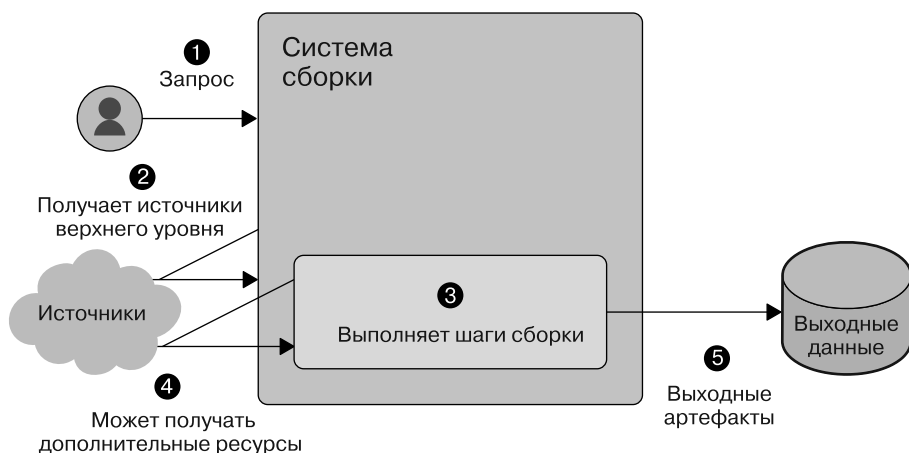


Рис. 14.4. Базовая система CI/CD

С такой системой вы можете относительно легко добавить подписанное происхождение к выходным данным, как показано на рис. 14.5. Для небольшой организации с моделью «централизованного сервиса сборки» этот дополнительный шаг подписания может помочь в решении проблем безопасности.

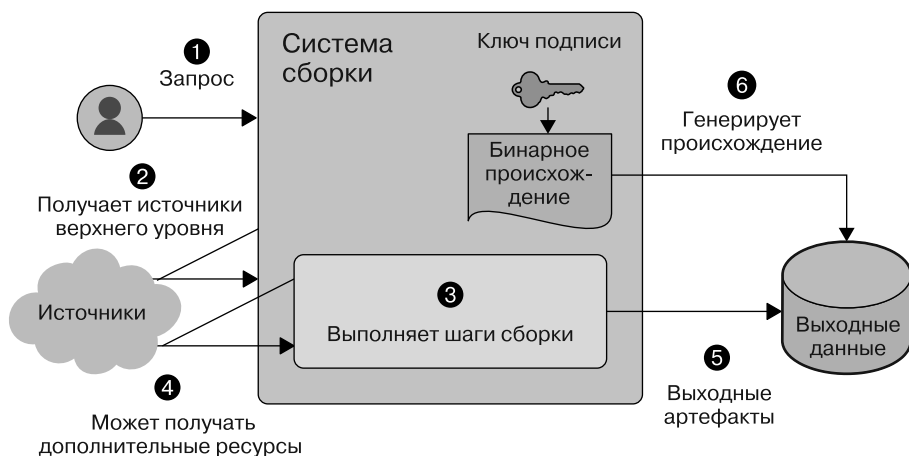


Рис. 14.5. Добавление подписи к существующей системе CI/CD

Чем больше будет ваша компания и чем больше ресурсов для вложения в безопасность у вас будет, тем больше у вас будет потребность в устранении еще двух рисков безопасности: ненадежные входные данные и входные данные без проверки подлинности.

Ненадежные входные данные

Злоумышленники могут использовать свои входные данные для взлома процесса сборки. Во многих сервисах сборки пользователи без прав администратора могут определять произвольные команды для выполнения во время процесса сборки. Например, через `Jenkinsfile`, `travis.yml`, `Makefile` или `BUILD`. Эта функциональность нужна для поддержки широкого спектра сборок для организации. Но с точки зрения безопасности эта функция является «Удаленным выполнением кода» (Remote Code Execution, RCE). Вредоносная команда сборки, запускаемая в привилегированной среде, может выполнить следующие действия.

- Украсть ключ подписи.
- Добавить ложную информацию в происхождение.
- Изменить состояние системы, влияя на последующие сборки.
- Управлять другой сборкой, происходящей параллельно данной.

Даже если пользователям не разрешено определять собственные шаги, компиляция очень сложна и дает широкие возможности для уязвимостей RCE.

Уменьшить эту угрозу вы можете, разделив привилегии. Используйте доверенный процесс-оркестратор для установки начального известного исправного состояния, запуска сборки и создания подписанного источника при завершении сборки. По желанию оркестратор может получить описанные в следующем подразделе входные данные для устранения угроз. Все пользовательские команды сборки должны выполняться в другой среде, без доступа к ключу подписи или другим специальным привилегиям. Вы можете создать эту среду разными способами — например, через песочницу на том же компьютере, что и дирижер, или запустив ее на отдельном компьютере.

Входные данные без проверки подлинности

Даже если пользователь и этапы сборки заслуживают доверия, большинство сборок зависят от других артефактов. Любая такая зависимость — это поверхность, через которую злоумышленники могут взломать сборку. Если система сборки извлекает зависимость по HTTP без TLS, преступник может выполнить атаку через посредника для изменения зависимости при передаче.

Поэтому мы рекомендуем герметичные сборки (см. врезку «Герметичная, воспроизводимая или верифицируемая?» выше). Процесс сборки должен объявить все входные данные заранее, и только оркестратор должен их получить. Герметичные сборки дают гораздо большую уверенность в правильности указанных в происхождении входных данных.

Как только вы учтете ненадежные и непроверенные входные данные, ваша система будет похожа на рис. 14.6. Такая модель более устойчива к атакам, чем простая на рис. 14.5.

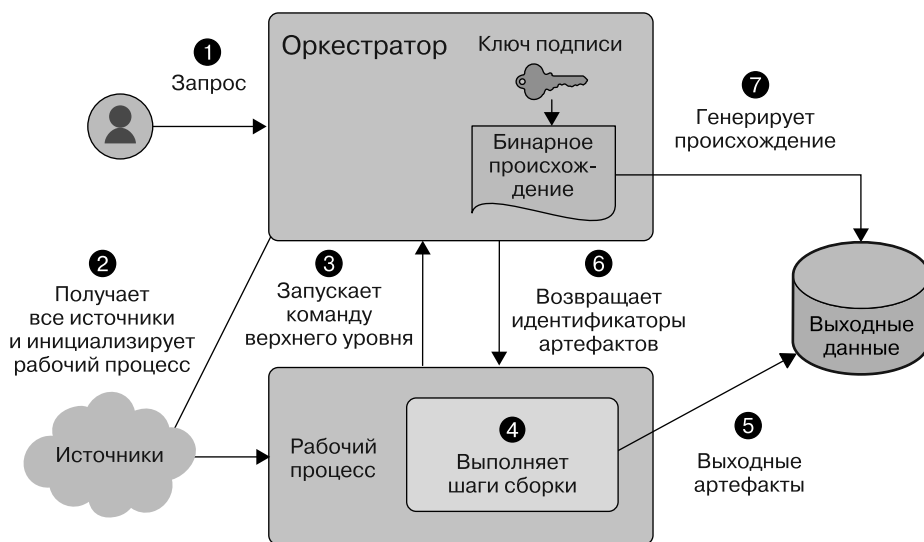


Рис. 14.6. «Идеальная» конструкция CI/CD, учитывающая риски, связанные с ненадежными входными данными и данными без проверки подлинности

Пункты контроля развертывания

Чтобы «проверять артефакты, а не только людей», решения по развертыванию должны приниматься в надлежащих пунктах контроля среды развертывания. Здесь *пункт контроля* — это точка, через которую должны проходить все запросы на развертывание. Злоумышленники могут обойти решения о развертывании, не происходящие в пунктах контроля.

Рассмотрим Kubernetes как пример для настройки пунктов контроля развертывания на рис. 14.7. Допустим, нужно проверить все развертывания на модулях в конкретном кластере Kubernetes. Главный узел мог бы стать хорошим пунктом контроля, потому что все развертывания должны проходить через него. Чтобы задать правильный пункт контроля, настройте рабочие узлы на прием запросов только от главного. Так атакующие не смогут развертываться прямо на рабочих узлах¹.

¹ В действительности должен быть какой-то способ развертывания ПО на самом узле — загрузчике, операционной системе, программном обеспечении Kubernetes и т. д. И у этого механизма развертывания должна быть собственная реализация политики, которая полностью отличается от реализации для модулей.

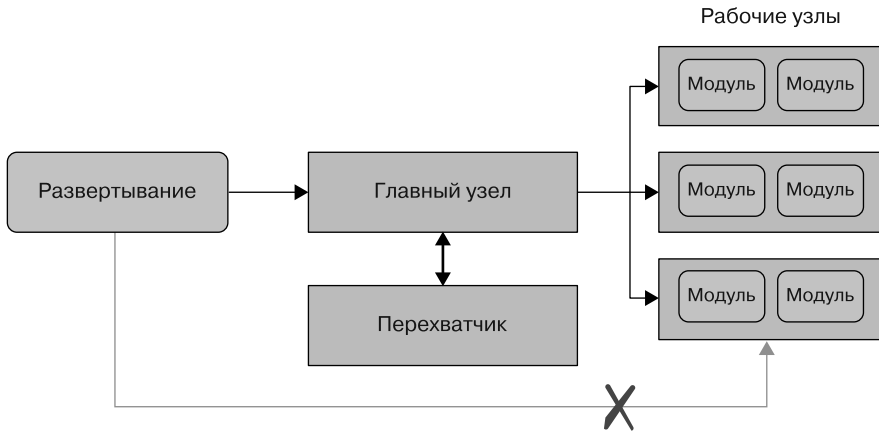


Рис. 14.7. Архитектура Kubernetes — все развертывания должны проходить через главный узел

В идеале пункт контроля принимает решение о политике напрямую или через RPC. Для этой цели у Kubernetes есть веб-перехватчик Admission Controller (<https://oreil.ly/Bm04C>). При использовании Google Kubernetes Engine Binary Authorization (<https://oreil.ly/YxiJX>) предоставляет встроенный перехватчик и много дополнительных функций. Даже если вы не пользуетесь Kubernetes, вы можете изменить точку «перехвата» для принятия решения о развертывании.

В качестве альтернативы разместите прокси перед пунктом контроля и примите решение о политике в нем, как показано на рис. 14.8. Для этого нужно настроить точку «перехвата», чтобы разрешить доступ только через прокси. Иначе злоумышленник может обойти прокси, передав данные напрямую в точку перехвата.

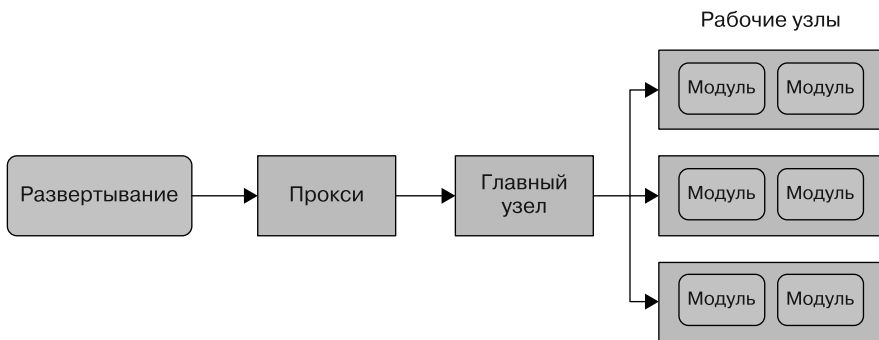


Рис. 14.8. Альтернативная архитектура с использованием прокси для принятия решений о политике

Проверки после развертывания

Даже после применения политики развертывания или проверки подписей во время развертывания всегда желательно добавить ведение журнала и проверки после развертывания по следующим причинам.

- *Политики могут измениться.* В этом случае механизм проверки должен пересмотреть существующие развертывания в системе, чтобы убедиться, что они по-прежнему соответствуют новым политикам. Это очень важно при первом включении политики.
- Возможно, запрос был разрешен для выполнения из-за недоступности службы принятия решений. Такое *отказоустойчивое* проектирование часто требуется для обеспечения доступности сервиса, особенно при первом внедрении функции принудительного исполнения.
- Оператор мог использовать *механизм аварийного доступа*, чтобы обойти решение в случае чрезвычайной ситуации, как описано в следующем разделе.
- Пользователям нужен способ *протестировать* возможные изменения политики перед их закреплением, чтобы убедиться, что существующее состояние не нарушит новую версию политики.
- По причинам, аналогичным сценарию «при отказе открыт», пользователям может понадобиться режим *пробного запуска*, когда система всегда разрешает запросы во время развертывания, но отслеживает потенциальные проблемы.
- Следователи могут запрашивать информацию после инцидента для *судебного расследования*.

Точка принятия решения должна регистрировать достаточно информации, чтобы верификатор мог оценить политику после развертывания¹. Регистрация полного запроса обычно нужна, но ее не всегда достаточно. Если для оценки политики требуется какое-то другое состояние, журналы должны содержать его. Мы столкнулись с этой проблемой при реализации проверки Borg после развертывания. Запросы «задания» содержат ссылки на существующие «ресурсы»

¹ В идеале записи журнала должны быть очень надежны и защищены от взлома, даже в случае перебоев в работе или нарушения в системе. Предположим, что главный узел Kubernetes получает запрос, а сервер ведения журнала недоступен. Главный узел может временно сохранить журнал на локальный диск. Что делать, если локальное устройство выходит из строя до того, как сервер ведения журнала снова заработает? Что делать, если у устройства недостаточно места для хранения? Это сложные случаи, решение для которых еще разрабатывается.

и «пакеты», поэтому нам пришлось объединить три источника журналов — задания, ресурсы и пакеты — для получения полного состояния, необходимого для принятия решения¹.

Практические советы

За прошлые годы мы усвоили несколько уроков при реализации верифицируемых сборок и политик развертывания в разных контекстах. Большинство из них посвящены не фактическому выбору технологий, а тому, как внедрить надежные, простые в отладке и понятные изменения. В этом разделе есть несколько практических советов, которые, мы надеемся, вам пригодятся.

Двигайтесь небольшими шагами

Для обеспечения безопасной, надежной и согласованной цепочки поставок ПО нужно внести много изменений — от сценариев этапов сборки до реализации происхождения сборки и обработки настроек как кода. Координировать все это непросто. Ошибки или отсутствующая функциональность в этих средствах контроля тоже могут представлять высокий риск для производительности проектирования. В худшем случае ошибка может вызвать сбой в работе вашего сервиса.

Успех будет выше, если вы сосредоточитесь на обеспечении конкретного аспекта цепочки поставок в одно время. Так вы можете снизить риск сбоев и помочь коллегам изучить новые рабочие процессы.

Предоставьте информативные сообщения об ошибках

При отклонении развертывания сообщение об ошибке должно четко объяснять причину сбоя и как можно исправить ситуацию. К примеру, если артефакт отклонен из-за того, что он был создан из неверного исходного URI, есть два варианта исправления: обновить политику для разрешения этого URI или повторить

¹ `alloc` (сокращенно от *allocation* (выделение ресурсов)) в Borg — это зарезервированный на устройстве набор ресурсов, в котором один или несколько наборов Linux-процессов могут быть запущены в контейнере. В пакетах есть двоичные файлы заданий Borg и файлы данных. Полное описание Borg см. в статье: *Verma A. et al. Large-Scale Cluster Management at Google with Borg* // *Proceedings of the 10th European Conference on Computer Systems*, 2015. P. 1–17. doi: 10.1145/2741948.2741964.

сборку с правильным URI. Ваш механизм принятия решений о политике должен предоставлять пользователю информативную обратную связь, с подобными предложениями. Простой ответ «не соответствует политике» приведет пользователя в замешательство.

Учитывайте эти пути при разработке архитектуры и языка политик. Из-за некоторых вариантов проектных решений предоставление информативной обратной связи для пользователей может усложниться. Поэтому постарайтесь уловить проблемы заранее. Один из наших ранних прототипов языка политики предлагал большую гибкость в выражении политик, но не позволял предоставлять информативные сообщения об ошибках. В итоге мы отказались от этого подхода в пользу очень ограниченного языка, учитывающего более полезные сообщения об ошибках.

Убедитесь в однозначности происхождения

Верифицируемая система сборки Google изначально загружала бинарное происхождение в БД асинхронно. Затем во время развертывания механизм политики просматривал происхождение в базе данных с хеш-артефактом в качестве ключа.

Этот подход в основном работал отлично, но мы столкнулись с серьезной проблемой: пользователи могут создавать артефакт несколько раз, а это приводит к нескольким записям для одного хеша. Рассмотрим случай с пустым файлом: у нас были буквально миллионы записей о происхождении, привязанных к хешу пустого файла, так как многие сборки создавали пустой файл как часть своего вывода. Для проверки такого файла наша система должна была узнать, соответствует ли политике *какая-либо* из записей о происхождении. Это привело к двум проблемам.

- Если не удалось найти подходящую запись, мы не могли предоставить информативные сообщения об ошибках. Например, вместо того чтобы вернуть сообщение «исходный URI был равен X, но в политике говорилось, что это должен быть Y», приходилось возвращать что-то вроде «Ни одна из этих 497 129 записей не соответствовала этой политике». Это был плохой механизм взаимодействия с пользователем.
- Время верификации было линейным по отношению к количеству возвращаемых записей. Это заставило нас превзойти SLO — задержку 100 мс — на несколько порядков!

Мы столкнулись с проблемами при асинхронной загрузке в БД. Сбой загрузки может пройти незаметно, и тогда наш механизм политики отклонит развертывание. Пользователи не понимали, почему загрузка была отклонена. Мы могли бы решить эту проблему, сделав загрузку синхронной, но это сделало бы нашу систему сборки менее надежной.

Поэтому мы настоятельно рекомендуем сделать происхождение однозначным. По возможности вместо использования БД *распространяйте происхождение в соответствии с артефактом*. Это увеличивает надежность системы, уменьшает задержки и облегчает отладку. Например, система, использующая Kubernetes, может добавить аннотацию, которая передается в веб-перехватчик Admission Controller.

Создавайте однозначные политики

Как и рекомендуемый нами подход к происхождению артефакта, политика, применяемая к конкретному развертыванию, должна быть однозначной. Проектируйте систему так, чтобы к каждому заданному развертыванию применялась только одна политика. Рассмотрим другой вариант: если подходят две политики, нужно ли применять обе или можно применить только одну из них? Легче вообще избежать этого вопроса. Если вы хотите применить глобальную политику ко всей организации, задайте ее как метаполитику: выполните проверку на соответствие всех отдельных политик некоторым глобальным критериям.

Добавьте механизм аварийного доступа при развертывании

При чрезвычайной ситуации иногда нужно обойти политику развертывания. Инженеру может понадобиться перенастроить внешний интерфейс для отвлечения трафика от отказавшего внутреннего интерфейса, и изменение конфигурации как кода может быть слишком долгим для развертывания через обычный конвейер CI/CD. С помощью обходящего политику механизма аварийного доступа инженеры могут быстро устранить простои, а это способствует культуре безопасности и надежности (см. главу 21).

Злоумышленники могут взломать механизм аварийного доступа, поэтому все его развертывания должны отправлять аварийные сигналы и быстро проверяться.

Чтобы сделать аудит практичным, не используйте события аварийного доступа часто — если их слишком много, вредоносную деятельность может быть невозможно отличить от допустимого использования.

Пересмотр защиты в соответствии с моделью угроз

Теперь мы можем сопоставить дополнительные меры по смягчению угроз с ранее не устраненными угрозами, как показано в табл. 14.2.

Таблица 14.2. Дополнительные меры по смягчению сложных угроз

Угроза	Смягчение
Инженер развертывает старую версию кода с известной уязвимостью	Политика развертывания требует, чтобы код был проверен на наличие уязвимостей в течение последних <i>N</i> дней
Система CI неправильно настроена и разрешает сборку запросов из произвольных репозиториев с исходным кодом. В итоге злоумышленник может создать свой репозиторий с вредоносным кодом	Система CI генерирует бинарное происхождение, описывающее, из какого репозитория оно было получено. В производственной среде применяется политика развертывания, требующая от происхождения доказательств, что развернутый артефакт пришел из утвержденного репозитория
Злоумышленник загружает в систему CI свой сценарий сборки, извлекающий ключ подписи. Далее он использует этот ключ для подписи и развертывания вредоносного бинарного файла	Верифицируемая система сборки разделяет привилегии, поэтому у компонента, выполняющего пользовательские сценарии сборки, нет доступа к ключу подписи
Злоумышленник обманывает систему CD при помощи взломанного компилятора или инструмента сборки, создающего вредоносный двоичный файл	Для герметичных сборок разработчики должны явно указывать выбор компилятора и инструмента сборки в исходном коде. Он рецензируется, как и любой другой код

С помощью подходящих мер безопасности в цепочке поставок ПО вы можете смягчать даже самые сложные угрозы.

Резюме

С рекомендациями из этой главы вы сможете укрепить цепочку поставок ПО от разных внутренних угроз. Проверка кода и автоматизация — важная тактика предотвращения ошибок и увеличения затрат на атаки злоумышленников. Обработка настроек как кода расширяет эти преимущества для настроек, традиционно

получающих меньше внимания, чем код. Средства управления развертыванием на основе артефактов, особенно связанные с двоичным происхождением и верифицируемыми сборками, обеспечивают защиту от опытных злоумышленников и предполагают масштабирование по мере роста организации.

Вместе эти рекомендации помогают гарантировать, что код, который вы написали и протестировали (следуя принципам в главах 12 и 13), и есть тот, который фактически развернут в производстве. Но несмотря на все усилия, код не всегда будет вести себя ожидаемо. Если это происходит, вы можете использовать некоторые стратегии отладки из следующей главы.

ГЛАВА 15

Исследование систем

*Пит Наттолл, Мэтт Линтон
и Дэвид Сейдман с Верой Хаас,
Джулией Сарацино и Амайей Букер*

Большинство систем в конечном итоге выходят из строя. Ваша способность исследовать сложную систему зависит от ряда факторов. Помимо наличия доступа к адекватным записям журнала и источникам информации для отладки, вам нужны соответствующие знания. Еще важно спроектировать свои системы ведения журнала с учетом защиты и контроля доступа. В этой главе мы пройдемся по методам отладки и предложим несколько стратегий выхода из тупиковых ситуаций. Далее мы обсудим различия между отладкой системной проблемы и исследованием проблемы безопасности. После этого мы рассмотрим компромиссы, которые следует учитывать при принятии решения о том, какие журналы нужно сохранить. Наконец, мы рассмотрим, как обеспечить безопасность и надежность этих ценных источников информации.

В идеальном мире все системы совершенны, а у пользователей только лучшие намерения. В реальности вы столкнетесь с ошибками и должны будете провести исследование по безопасности. Наблюдая за работой системы на протяжении долгого времени, вы будете определять области для улучшения и места для обновления и оптимизации процессов. Для всего этого нужны методы отладки и исследования, и не обойтись без соответствующего доступа к системе.

Но даже предоставление доступа только для чтения в целях отладки создает риск злоупотребления этим доступом. Для устранения этого риска нужны подходящие механизмы безопасности. Важно тщательно соблюдать баланс между потребностями разработчиков и операторов в отладке, требованиями к безопасности хранения и доступа к конфиденциальным данным.



В этой главе мы пользуемся термином «отладчик» для обозначения человека, устраняющего проблемы с ПО, а не для GDB (отладчик GNU) (<https://oreil.ly/F182Z>) или подобных инструментов. Мы используем термин «мы» для обозначения авторов этой главы, а не Google в целом, если нет других уточнений.

От отладки до исследования

Вдруг я осознал, что значительная часть моей оставшейся жизни будет потрачена на поиск ошибок в моих собственных программах.

Морис Уилкс (Maurice Wilkes), Memoirs of a Computer Pioneer (MIT Press, 1985)

У отладки плохая репутация. Ошибки появляются, когда все и так плохо. Иногда трудно оценить, когда ошибка будет исправлена или система будет «достаточно хороша» для использования большим количеством людей. Большинству разработчиков будет сподручнее написать новый код, чем взяться за отладку существующих программ. Некоторые и вовсе считают отладку невыгодной. Но она нужна и иногда даже интересна, если рассматривать ее с точки зрения изучения новых фактов и инструментов. Отладка делает нас лучшими программистами, чем мы есть, и напоминает, что иногда мы не настолько умные, как думаем.

Пример: временные файлы

Рассмотрим сбой, который мы (авторы) отладили два года назад¹. Исследование началось, когда мы получили предупреждение о том, что в базе данных Spanner (<https://oreil.ly/ZYr1W>) заканчивается квота хранилища. Мы прошли процесс отладки, ответив на следующие вопросы.

1. Что вызвало нехватку памяти в БД?

Быстрая сортировка показала, что проблема в накоплении множества мелких файлов, созданных в огромной распределенной файловой системе Google, Colossus (<https://oreil.ly/dkocj>). Скорее всего, это было вызвано изменением трафика пользовательских запросов.

2. Что создавало все эти мелкие файлы?

Рассмотренные нами метрики сервиса показали, что файлы были получены из-за недостатка памяти на сервере Spanner. Согласно нормальному поведению, последние записи (обновления) были помещены в буфер. Когда серверу не хватило памяти, он сбрасывал данные в файлы Colossus. К сожалению,

¹ Хотя сбой и был в большой распределенной системе, люди, обслуживающие небольшие и автономные системы, увидят в этом что-то знакомое. Например, при сбоях, связанных с единственным почтовым сервером, у которого недостаточно места на жестком диске!

у каждого сервера в зоне Spanner был только небольшой объем памяти для размещения обновлений. Поэтому вместо того, чтобы сбрасывать управляемое количество сжатых файлов большего размера¹, каждый сервер сбрасывал множество мелких файлов в Colossus.

3. Где использовалась память?

Каждый сервер запускался как задание Borg (в контейнере), ограничивающее доступную ему память². Чтобы определить, где внутри ядра была использована память, мы напрямую выполнили команду `slabtop` на производственном устройстве. Было выявлено, что кэш записей каталога (`dentry` — от `directory entry`) использовал больше всего памяти.

4. Почему кэш-память была настолько большой?

Мы сделали обоснованное предположение, что сервер базы данных Spanner создавал и удалял огромное количество временных файлов — по несколько для каждой операции очистки. Каждая операция увеличивала размер кэш-памяти, усугубляя проблему.

5. Как мы можем подтвердить нашу гипотезу?

Для проверки этой теории мы написали и запустили на Borg программу воспроизведения ошибки, создающую и удаляющую файлы в цикле. Спустя несколько миллионов файлов кэш-память записей каталога использовала всю память своего контейнера, что подтвердило гипотезу.

6. Была ли это ошибка ядра?

Мы исследовали ожидаемое поведение ядра Linux и определили, что оно кэширует отсутствие файлов — некоторым системам сборки эта функция нужна для обеспечения достаточной производительности. При нормальной работе ядро удаляет записи из кэш-памяти после заполнения контейнера. Но сервер Spanner неоднократно сбрасывал обновления, поэтому контейнер никогда не становился настолько заполненным, чтобы вызывать очистку. Мы решили эту проблему, указав, что временные файлы не нужно кэшировать.

¹ Spanner сохраняет данные в виде дерева с логарифмическим слиянием (Log-Structured Merge, LSM). Для получения дополнительной информации об этом формате см. статью: *Luo C., Carey M. J. LSM-Based Storage Techniques: A Survey*. 2018. Статья в arXiv — arXiv:1812.07527v3 (<https://oreil.ly/DjWJn>).

² Подробнее о Borg см.: *Verma A. et al. Large-Scale Cluster Management at Google with Borg* // *Proceedings of the 10th European Conference on Computer Systems*, 2015. P. 1–17. doi: 10.1145/2741948.2741964.

В этом процессе отладки применены многие концепции, обсуждаемые в этой главе. Но самый важный вывод здесь — *мы исправили эту проблему*, и вы тоже сможете! Решение и устранение проблемы не требовало никакой магии — только медленного и структурированного исследования. Вот его характеристики.

- После обнаружения признаков деградации в системе мы отладили проблему при помощи журналов и инфраструктуры мониторинга.
- Мы смогли отладить проблему, хотя она возникла в пространстве ядра и в коде, с которым отладчики раньше не работали.
- Мы не замечали проблему до этого сбоя, хотя она была в течение нескольких лет.
- Ни одна часть системы не была сломана. Все детали работали как положено.
- Разработчики сервера Spanner были удивлены тем, что временные файлы могут занимать память еще долго после удаления файлов.
- Мы смогли отладить использование памяти ядром с помощью инструментов, предоставленных разработчиками ядра. Мы никогда не использовали эти инструменты раньше, но смогли сравнительно быстро продвинуться вперед, потому что уже были знакомы с методами отладки.
- Изначально мы приняли проблему за ошибку пользователя и изменили свое мнение только после изучения данных.
- Разрабатывая гипотезу и создавая способ проверить нашу теорию, мы подтвердили первопричину, прежде чем вносить изменения в систему.

Методы отладки

В этом разделе описаны некоторые методы систематической отладки¹. Отладка — это навык, который вы можете освоить и применить на практике. В главе 12 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/effective-troubleshooting/>) есть два требования для успешной отладки.

- Знайте, как должна работать система.
- Будьте систематичны: собирайте данные, выдвигайте гипотезы о причинах и проверяйте теории.

¹ Вам может быть интересна статья *What Does Debugging a Program Look Like?* в блоге (<https://oreil.ly/J2U1R>) Джулии Эванс.

Первое из этих требований — самое сложное. Возьмем стандартный пример системы, созданной одним разработчиком, который внезапно покидает компанию, не передав никому знания о системе. Она может продолжать работать месяцами, но однажды таинственно сломается, и никто не сможет это исправить. Некоторые советы дальше могут помочь, но ничто не заменит понимания системы заранее (см. главу 6).

Отличия лошадей от зебр

Когда вы слышите стук копыт, то представляете лошадь или зебру? Преподаватели иногда задают этот вопрос студентам-медикам, которые учатся сортировать и диагностировать заболевания. Это напоминание о распространенности большинства болезней — скорее всего, это будут лошади, а не зебры. Понятно, почему это полезный совет для студентов-медиков. Не нужно сразу сводить симптомы к редкому заболеванию, когда оно может быть обычным и простым в лечении.

Но в больших системах опытные инженеры будут наблюдать как общие, *так* и редкие события. Создатели компьютерных систем могут (и должны) работать так, чтобы полностью устранить все проблемы. По мере масштабирования системы и устранения операторами общих проблем редкие будут появляться все чаще. Цитируя Брайана Кантрилла (<https://oreil.ly/eYfUO>): «Со временем лошади будут найдены, останутся только зебры».

Рассмотрим очень редкую проблему повреждения памяти при инвертировании разрядов. Современный модуль памяти с исправлением ошибок с вероятностью менее 1 % в год сталкивается с неисправимым инвертированием разрядов, способным привести к сбою системы¹. Инженер, отлаживающий неожиданный сбой, не подумает: «Держу пари, это было вызвано крайне маловероятной электрической неисправностью микросхем памяти!» Но в очень большом масштабе эти редкости могут быть очень вероятны. Гипотетический облачный сервис 25 000 устройств может использовать память на 400 000 микросхем ОЗУ. Учитывая вероятность неустраняемых ошибок *на чип* в 0,1 % в год, в масштабах сервиса это может привести к 400 случаям в год. Работающие с сервисом люди будут наблюдать сбой памяти ежедневно.

Отладка таких редких событий сложна, но достижима с помощью правильных данных. Вот еще один пример: инженеры Google однажды заметили, что не-

¹ Schroeder B., Pinheiro E., Weber W.-D. DRAM Errors in the Wild: A Large-Scale Field Study // ACM SIGMETRICS Performance Evaluation Review, 2009. № 37 (1). doi: 10.1145/2492101.1555372.

которые чипы ОЗУ выходили из строя чаще, чем ожидалось. При помощи данных активнов они нашли источник неисправных модулей DIMM (модулей памяти) и смогли отследить модули вплоть до поставщика. После тщательной отладки и исследования инженеры определили главную причину: экологический сбой в стерильном помещении на одном заводе, где производились модули DIMM. Эта проблема была «зеброй» — редкой ошибкой, видимой только в масштабе.

По мере роста сервиса сегодняшняя странная ошибка в следующем году может стать распространенной проблемой. В 2000 году повреждение оборудования памяти стало неожиданностью для Google (<https://oreil.ly/CicjH>). Сегодня такие сбои — обычное явление, которое устраняется с помощью комплексной проверки целостности и других мер надежности.

В последние годы мы столкнулись и с другими зебрами.

- Два запроса веб-поиска, хешированные с одним и тем же 64-битным ключом кэша, приводят к замене одного результата запроса другим.
- C++ преобразовал `int64` в `int` (только 32 бита), что привело к проблемам после 2^{32} запросов (подробнее об этой ошибке см. в пункте «Очистите код» далее в этой главе).
- Ошибка в алгоритме распределенной перебалансировки возникала, когда код запускался одновременно на сотнях серверов.
- Кто-то оставил нагрузочный тест запущенным в течение недели, что медленно снизило производительность. В итоге мы определили, что устройство постепенно нагружается от проблем с выделением памяти и это приводит к деградации. Мы смогли обнаружить это только потому, что недолгое испытание длилось дольше обычного.
- Изучение медленных тестов в C++ показало, что время загрузки динамического компоновщика было сверхлинейным по отношению к количеству загруженных совместно используемых библиотек. При 10 000 совместно используемых библиотеках запуск `main` может занять несколько минут.

Работая с меньшими, более новыми системами, ожидайте появления лошадей (общих ошибок). При работе со старыми, более крупными и стабильными системами будут появляться зебры (редкие ошибки) — операторы уже заметили и исправили общие ошибки, которые появлялись со временем. Проблемы чаще возникают в новых частях системы.

ПОВРЕЖДЕНИЕ ДАННЫХ И КОНТРОЛЬНЫЕ ПРОВЕРКИ

Есть множество причин повреждения памяти. Аппаратные проблемы, вызванные факторами окружающей среды, — только один из возможных вариантов. Проблемы с ПО, например, когда один поток записывает, а другой читает данные, тоже могут привести к повреждению памяти. Разработчикам программного обеспечения лучше не принимать каждую странную ошибку за аппаратную проблему.

Современный DRAM обеспечивает несколько способов защиты от повреждения памяти.

- RAM с коррекцией ошибок (error-correcting code, ECC) исправляет большинство повреждений.
- Обнаруженные неисправимые ошибки вызывают исключение на устройстве, что позволяет ОС действовать самостоятельно.
- Операционная система может выдавать сообщения о критических ошибках или завершать процесс, ссылающийся на поврежденную память. Из-за любого из этих событий система не может использовать недопустимое содержание памяти.

Контрольные суммы — это числа, полученные из данных для обнаружения изменений в них. Они могут защитить от непредвиденных сбоев оборудования и ПО. При использовании контрольных сумм имейте в виду следующее.

- Компромисс между стоимостью ЦП контрольной суммы и обеспечиваемым уровнем защиты.
- Циклические проверки избыточности (cyclic redundancy check, CRC) защищают от однобитового инвертирования разрядов, а криптографические хеши — от атак злоумышленников. CRC дешевле с точки зрения процессорного времени.
- Область действия контрольной суммы.

Представьте клиента, отправляющего ключ и значение по сети на сервер, сохраняющий значение на диске. Контрольная сумма уровня файловой системы защищает только от ошибок файловой системы или диска. Контрольная сумма сгенерированного клиентом значения защищает от ошибок сети и сервера. Но рассмотрим ошибку, когда клиент пытается получить значение с сервера. Клиент отправляет ключ, но неисправный сервер считывает с диска неправильное значение и его контрольную сумму и возвращает эти неверные значения клиенту. Область действия контрольной суммы ограничена только значением, ее проверка все еще проходит, несмотря на ошибку. Более подходящее решение — добавлять ключ как в значение, так и в контрольную сумму. Это поможет избежать случаев, когда ключ и возвращаемое значение имеют неверную контрольную сумму.

Выделите время для отладки и исследований

Исследования безопасности (которые мы обсудим позже) и отладка — это много часов непрерывной работы. Сценарий с временными файлами из предыдущего раздела требовал от 5 до 10 часов отладки. Во время серьезного происшествия дайте отладчикам и следователям возможность сосредоточиться, избавив их от ежеминутных отчетов.

Отладка поощряет медленные, методичные, настойчивые подходы, где люди перепроверяют свою работу и предположения и готовы копать глубже. Проблема с временными файлами предлагает негативный пример отладки. Первый респондент изначально выявил сбой, вызванный пользовательским трафиком, и обвинял пользователей в плохом поведении системы. В то время команда испытывала операционную перегрузку и была просто завалена сообщениями из-за медленно загружающихся страниц.



В главе 17 книги Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/overload/>) обсуждается снижение операционных перегрузок. В главе 11 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/being-on-call/>) предлагается поддерживать объем задач и вызовов до двух на смену, чтобы дать специалистам время на углубленное изучение проблем.

Запишите свои наблюдения и ожидания

Запишите то, что видите. Отдельно запишите теории, даже те, от которых вы отказались. Это даст несколько преимуществ.

- Так можно определить структуру исследования, что помогает вспомнить шаги, предпринятые в ходе исследования. В начале отладки вы не знаете, сколько времени понадобится для решения проблемы — поиск может занять пять минут или пять месяцев.
- Другой отладчик может прочитать ваши заметки, понять их и быстро принять участие в исследовании. Ваши заметки помогут сотрудникам избежать дублирования работы и вдохновить других на поиски новых путей исследования. Подробнее об этой теме см. в разделе «Волшебная сила отрицательных результатов» главы 12 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/effective-troubleshooting/>).
- Для возможных проблем с безопасностью полезно вести журнал всех доступов и шагов исследования. Позже вам может потребоваться доказать (иногда в суде), какие действия были совершены злоумышленником, а какие — исследователями.

После записи ваших наблюдений отметьте, что вы ожидали наблюдать и почему. Ошибки часто скрываются в промежутке между вашей ментальной моделью системы и ее реализацией. В примере с временными файлами разработчики предположили, что удаление файла удаляет все ссылки на него.

Знайте, как ваша система должна себя вести

Часто отладчики начинают отлаживать то, что на самом деле является ожидаемым поведением системы. Вот несколько примеров из нашего опыта.

- Двоичный файл с названием `abort` в конце кода `shutdown`. Новые разработчики увидели вызов `abort` в журналах и начали отладку вызова, не заметив, что интересный сбой фактически был причиной вызова `shutdown`.
- Браузер Chrome при запуске пытается разрешить три случайных домена (например, `cegzauxkwefark.local`), чтобы определить, не является ли сеть незаконным вмешательством в DNS. Даже внутренняя исследовательская команда Google ошибочно приняла эти разрешения DNS за вредоносное ПО, пытающееся определить имя хоста сервера управления и контроля.

Отладчикам нужно научиться отбрасывать эти нормальные события, независимо от того, выглядят они уместными или подозрительными. Исследователи безопасности сталкиваются с дополнительными проблемами постоянного уровня фоновых шумов и активных противников, которые могут пытаться скрыть свои действия. Часто приходится фильтровать рутинную шумовую активность, такую как автоматические взломы входа в систему по SSH, ошибки аутентификации, вызванные неправильно вводимыми паролями пользователей, и сканирование портов, до появления более серьезных проблем.

Один из способов понять нормальное поведение системы — установить базовый уровень ее поведения на момент, когда вы не подозреваете о каких-то проблемах. Если у вас уже есть проблема, вы можете вывести исходные данные, изучив историю журналов за период до ее появления.

В главе 1 мы описали глобальный сбой YouTube, вызванный изменением универсальной библиотеки журналов. Оно привело к нехватке памяти на серверах (out of memory, OOM), что стало причиной сбоя. Библиотека широко использовалась в Google, поэтому наши исследователи заинтересовались, повлияло ли отключение на количество OOM для всех других задач в Borg. Несмотря на то, что в тот день у нас было много состояний OOM, мы смогли сравнить эти данные с базовыми за предыдущие две недели. Они показали, что у Google *ежедневно* возникает много условий OOM. Хотя ошибка была серьезной, она не сильно повлияла на показатель OOM для задач в Borg.

Не принимайте отклонение от лучших практик за норму. Часто со временем ошибки становятся «нормальным поведением», и вы перестаете их замечать. К примеру, мы когда-то работали на сервере, потратившем ~10 % своей памяти на фрагментацию кучи. Мы много лет утверждали, что ~10 % — ожидаемая и приемлемая величина потерь. Но позднее мы изучили профиль фрагментации и быстро нашли основные возможности для экономии памяти.

Операционная перегрузка (<https://oreil.ly/L144H>) и повышенная утомляемость приводят к появлению слепых пятен, что позже нормализует отклонения. Во избежание этого мы активно прислушиваемся к новичкам в команде. А чтобы «чутье» разработчиков не ослабевало, мы перемещаем людей из дежурных групп в группы реагирования и наоборот. Написание документации и объяснение коллегам тоже поможет проверить ваше понимание системы. Еще мы используем красные команды (см. главу 20) для проверки слепых зон.

Воспроизведите ошибку

Если это возможно, попытайтесь воспроизвести ошибку за пределами производственной среды. У этого подхода два главных преимущества.

- Вы не влияете на системы, обслуживающие реальных пользователей, поэтому можете аварийно завершить работу системы и повредить данные насколько захотите.
- Конфиденциальные данные не используются, поэтому можно привлечь к расследованию много людей, не касаясь проблем безопасности данных. Вы можете добавить неподходящие для реальных пользовательских данных операции и возможности вроде дополнительного внедрения журналов.

Иногда нельзя выполнить отладку вне производственной среды. Возможно, ошибка срабатывает только в масштабе, или ее нельзя изолировать. Пример с временными файлами — одна из таких ситуаций: мы не смогли воспроизвести ошибку с полным стеком обслуживания.

Изолируйте проблему

Если вы можете воспроизвести проблему, следующий шаг должен изолировать ее — в идеале до наименьшего подмножества кода, в котором она все еще проявляется. Это можно сделать, отключив компоненты или временно закомментировав подпрограммы до обнаружения проблемы.

В примере с временными файлами, когда мы заметили, что управление памятью странно работает на всех серверах, нам больше не нужно было отлаживать все компоненты на каждом уязвимом устройстве. Другой пример: рассмотрим один сервер (из большого кластера систем), который внезапно начинает возвращать большие задержки или ошибки. Этот сценарий — стандартный тест ваших средств мониторинга, ведения журналов и других систем наблюдения. Можете ли вы быстро найти один неисправный сервер среди множества серверов в системе? Подробнее об этом — в подразделе «Что делать, если вы зашли в тупик» далее.

Можно изолировать проблемы в коде. Конкретный пример — мы недавно исследовали использование памяти для программы с очень ограниченным бюджетом памяти. В частности, мы исследовали отображения памяти для стеков потоков. Мы предполагали, что у всех потоков одинаковый размер стека, поэтому для нас было неожиданностью, что у разных стеков потоков много разных размеров. Некоторые были довольно большими, что могло привести к потреблению большой части нашего бюджета памяти. Начальная область отладки содержала ядро, glibc — библиотеку GNU в C, библиотеку потоков Google и весь код, запускающий потоки. Простой пример, основанный на `pthread_create` из glibc, создавал стеки потоков одинакового размера, поэтому мы могли не рассматривать ядро и glibc как источники разных размеров. Далее мы изучили запускающий потоки код и обнаружили, что многие библиотеки случайно выбирают размер потока, что объясняло изменение размеров. Узнав об этом, мы смогли сэкономить память, сосредоточившись на нескольких потоках с большими стеками.

Помните о взаимосвязанности и причинно-следственных связях

Иногда отладчики могут подумать, что у двух событий с похожими симптомами или начавшихся одновременно одинаковая основная причина. Но взаимосвязанность — это не всегда причинно-следственная связь. Две проблемы могут возникать одновременно, но по разным причинам.

Некоторые взаимосвязи просты. Например, увеличение задержки приведет к уменьшению пользовательских запросов просто потому, что пользователи дольше ждут ответа системы. Если команда постоянно обнаруживает взаимосвязи, кажущиеся простыми при детальном рассмотрении, это может указывать на пробелы в понимании работы системы. Как в примере с временными файлами — если вы знаете, что неудачное удаление файлов приводит к заполнению дисков, наличие взаимосвязи вас не удивит.

Наш опыт показал, что исследование взаимосвязей часто полезно, в том числе и тех, что возникают в начале сбоев. Понять потенциальные причины можно поразмышляв: «*X сломан, Y сломан, Z сломан; что у этих троих общего?*» Мы успешно применяли и инструменты, основанные на корреляции. Например, мы развернули систему, автоматически соотносящую проблемы устройства с задачами в Borg, выполняющимися на этом устройстве. В итоге мы часто могли определить подозрительное задание в Borg, вызывающее широко распространенную проблему. Этот вид автоматизированного инструментария выдает более эффективные, статистически более строгие и более быстрые взаимосвязи, чем человеческие наблюдения.

Ошибки могут проявляться во время развертывания — см. главу 12 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/effective-troubleshooting/>). В простых ситуациях проблемы могут скрываться в развер-

тываемом новом коде, но развертывания могут вызывать и скрытые ошибки в старых системах. Тогда отладка может ошибочно сосредоточиться на новом развертываемом коде, а не на скрытых проблемах. Систематическое исследование — определение того, что происходит и почему — помогает в этих случаях. В одном из примеров у старого кода производительность была хуже, чем у нового, что приводило к случайным помехам системы в целом. Когда производительность улучшилась, другие части системы стали перегружаться. Сбой был связан с новым развертыванием, но оно не было основной причиной.

Проверьте свои гипотезы на реальных данных

Возможно, при отладке вы захотите найти первопричину проблем, прежде чем вы действительно рассмотрите систему. Если дело касается проблем с производительностью, такие действия приведут к появлению слепых зон, ведь проблемы часто кроются в коде, который отладчики долгое время не рассматривали. Когда-то мы отлаживали медленно работающий веб-сервер. Мы думали, что проблема где-то в серверной части, но профилировщик (<https://oreil.ly/3YrvQ>) показал, что запись каждого возможного фрагмента ввода на диск и дальнейший вызов синхронизации приводили к огромным задержкам. Это было обнаружено только тогда, когда мы отбросили изначальные предположения и углубились в систему.

Наблюдаемость — это возможность определить действия системы с помощью изучения ее результатов. Готовые решения для отслеживания, например Dapper (<https://oreil.ly/9qDWj>) и Zipkin (<https://zipkin.io/>), очень полезны для такой отладки. Сеансы отладки начинаются с базовых вопросов, таких как «Можете ли вы найти медленную трассировку в Dapper?»¹



Для начинающих будет сложно определить, какой инструмент лучше подходит для работы, или какие вообще инструменты существуют. Книга *Systems Performance*² Брендана Грегга дает полный обзор инструментов и методов. Это потрясающий справочник для отладки производительности.

Перечитайте документацию

Рассмотрите следующее руководство из документации Python (<https://oreil.ly/PudXU>).

Между операторами сравнения нет подразумеваемых связей. Истинность `x==y` не означает, что `x!=y` ложно. Поэтому при определении `__eq__()` нужно также определить `__ne__()`, чтобы операторы вели себя ожидаемо.

¹ Для этих инструментов часто нужна настройка, которую мы обсудим далее в подразделе «Что делать, если вы зашли в тупик».

² Грегг Б. Производительность систем. — СПб.: Питер, 2023.

Недавно команда Google потратила много времени на отладку оптимизации внутренней панели инструментов. Зайдя в тупик, они перечитали документацию и обнаружили ясно написанное предупреждение, объясняющее, почему оптимизация вообще никогда не работала. Люди настолько привыкли к медленной работе панели, что не заметили абсолютную неэффективность оптимизации¹. Поначалу ошибка казалась большой. Команда даже подумала, что это проблема самого Python. Обнаружив предупреждающее сообщение, они поняли, что это не зебра, а лошадь — их код никогда не работал.

Практика!

Навыки отладки остаются свежими, только если вы часто ими пользуетесь. Вы можете ускорить свои исследования и сохранить в памяти данные нами советы, вспомнив о соответствующих инструментах и журналах. Регулярная отработка отладки тоже помогает автоматизировать общие и утомительные части процесса, например проверку журналов. Чтобы лучше справляться с отладкой (или держать себя в тонусе), практикуйтесь и сохраняйте код, который вы пишете во время отладочных сессий.

В Google мы официально практикуем отладку с помощью регулярных крупномасштабных тестов аварийного восстановления (называемых DiRT или программой Disaster Recovery Testing)² и тестов на проникновение в систему безопасности (см. главу 16). Не такие крупные тесты, в которых в течение часа участвуют один-два инженера, проще в настройке и все еще очень полезны.

Что делать, если вы зашли в тупик

Что делать, если вы изучали проблему в течение нескольких дней и до сих пор не знаете ее причину? Возможно, она проявляется только в производственной среде, и вы не можете воспроизвести ошибку, не затрагивая настоящих пользователей. Может быть, при смягчении проблемы вы потеряли важную отладочную информацию, заменяя журналы. Или же природа проблемы не способствует полезному ведению журнала. Как-то раз мы отлаживали проблему, когда в контейнере не хватило оперативной памяти, а ядро выдавало SIGKILL для всех процессов в контейнере, остановив запись всех журналов. Без этих записей мы не смогли бы отладить проблему.

¹ Это еще один пример нормализации отклонения, когда люди привыкают к неоптимальному поведению!

² См.: *Krishnan K.* Weathering the Unexpected // *ACM Queue*, 2012. № 10 (9). <https://oreil.ly/xFPfT>.

Основная стратегия в таких ситуациях — улучшение процесса отладки. Иногда продвинуться вперед помогает использование методов для разработки постмортемов (как описано в главе 18). Многие системы работают годами или десятилетиями. Поэтому усилия по улучшению отладки почти всегда стоят того. В этом разделе описываются некоторые подходы к улучшению существующих методов отладки.

Улучшите наблюдаемость

Иногда вам нужно определить, что делает конкретный фрагмент кода. Используется ли эта ветка кода? Используется ли эта функция? Может ли эта структура данных быть большой? Является ли этот сервер медленным в 99-м процентиле? Какие серверы используют этот запрос? Для этих ситуаций вам нужна лучшая видимость системы.

Иногда помогают такие простые методы, как добавление более структурированного ведения журнала для улучшения наблюдаемости. Однажды мы исследовали систему, показавшую, что мониторинг обслуживает слишком много ошибок 404¹, но веб-сервер не регистрировал их. После добавления дополнительного ведения журнала для веб-сервера мы обнаружили, что вредоносная программа пытается извлечь ошибочные файлы из системы.

Для других улучшений отладки нужны серьезные инженерные усилия. К примеру, для отладки сложной системы, такой как Bigtable (<https://oreil.ly/31cv1>), нужны сложные инструменты. Главный узел Bigtable — центральный координатор зоны Bigtable. Он хранит список серверов и экранов в оперативной памяти, и эти критические секции защищены несколькими мьютексами. По мере роста разветвляемости Bigtable в Google главный узел Bigtable и эти мьютексы стали узким местом для масштабирования. Для лучшего понимания возможной проблемы мы внедрили оболочку вокруг мьютекса, отображающую статистику — глубину очереди и время удержания мьютекса.

Такие средства отслеживания, как Dapper и Zipkin, очень полезны для подобной сложной отладки. Предположим, что у вас есть дерево RPC с внешним интерфейсом, вызывающим сервер, который вызывает другой сервер, и т. д. В корне каждого дерева RPC есть уникальный идентификатор. Далее каждый сервер регистрирует трассировки о RPC, которые он получает, отправляет и т. д. Dapper собирает все трассировки централизованно и объединяет их с помощью идентификатора. Так отладчик может видеть все серверы, затронутые запросом пользователя. Мы обнаружили, что Dapper важен для понимания задержки в распределенных системах. Точно так же Google встраивает простой веб-сервер

¹ 404 — стандартный код ошибки HTTP для сообщения «файл не найден».

почти в каждый двоичный файл, чтобы иметь представление об их поведении. На сервере находятся конечные точки отладки, которые предоставляют счетчики, символизированный вывод всех запущенных потоков, запущенные RPC и т. д. Для получения дополнительной информации см. статью Хендерсон (2017)¹.



Наблюдаемость не заменяет понимание вашей системы. Она не заменяет и критическое мышление при отладке (к сожалению!). Мы часто добавляем больше журналов и счетчиков, чтобы увидеть, что делает система. Но действительную картину мы сможем увидеть только после отхода на шаг назад и тщательного анализа проблемы.

Наблюдаемость — большая и быстро развивающаяся тема, полезная не только для отладки². Если у вас небольшая фирма с ограниченными ресурсами для разработчиков, то рассмотрите возможность использования систем с открытым исходным кодом или приобретения стороннего ПО для наблюдения.

Сделайте перерыв

На время отвлекитесь от проблемы и вернитесь к ней с новыми силами. Если вы старательно отлаживали ошибку и зашли в тупик, сделайте перерыв: выпейте немного воды, выйдите на улицу, сделайте несколько упражнений или почитайте книгу. Ошибки иногда открываются с новой стороны после хорошего сна. У главного специалиста нашей экспертизы в лаборатории команды хранится виолончель. Если он действительно погряз в проблеме, то он может отойти в лабораторию примерно на 20 минут, чтобы поиграть, а потом возвращается — снова энергичный и сфокусированный (https://oreil.ly/axc_Y). Еще один исследователь держит под рукой гитару, а некоторые предпочитают хранить на столе блокноты, чтобы можно было рисовать или создавать глупые анимированные изображения, а потом поделиться ими с командой, когда понадобится подобная встряска для ума.

Не забудьте поддерживать хорошую атмосферу в команде. Когда вы отойдете на перерыв, сообщите коллегам, что вам нужна перезарядка и вы следуете рекомендациям. Еще полезно документировать, куда вы забрались в результате исследования и почему застряли. Это поможет другому исследователю продолжить вашу работу, а вам — вернуться туда, где вы остановились. В главе 17 вы найдете больше советов по поддержанию морального состояния.

¹ *Henderson F.* Software Engineering at Google. 2017. Статья в arXiv: 1702.01715v2 (<https://oreil.ly/2-6pU>).

² Подробный обзор этой темы см. в блоге Синди Шридахарен в статье *Monitoring in the time of Cloud Native* (<https://oreil.ly/n6-j9>).

Очистите код

Иногда вы подозреваете, что в вашем коде есть ошибка, но не видите ее. Общее улучшение качества кода может помочь в этой ситуации. Как уже упоминалось в этой главе, мы однажды отлаживали фрагмент кода, который не сработал после 2^{32} запросов, потому что C++ преобразовывал `int64` в `int` (только 32 бита) и обрезаł его. Хотя компилятор может предупредить вас о таких преобразованиях с помощью `-Wconversion` (<https://oreil.ly/BUpuH>), мы не обратили внимания на это предупреждение, потому что в нашем коде было много допустимых преобразований. Благодаря очистке кода мы заметили предупреждение компилятора, обнаружили больше возможных ошибок и предотвратили новые проблемы, связанные с преобразованием.

Вот еще несколько советов по очистке.

- Покрытие кода можно улучшить модульными тестами. Отметьте функции, в которых предположительно могут быть ошибки или которые ведут себя непредсказуемо (см. главу 13 для получения дополнительной информации).
- Для параллельных программ используйте средства очистки (<https://oreil.ly/GJJq9>) (см. подраздел «Очистка кода» в главе 12) и добавляйте аннотации к мьютексам (<https://oreil.ly/z1BQk>).
- Улучшите обработку ошибок. Часто для выявления проблемы достаточно добавить больше контекста вокруг ошибки.

Удалите это!

Иногда в устаревших системах могут быть ошибки, особенно если разработчики не успели ознакомиться с кодовой базой или техническое обслуживание не было проведено как следует. Устаревшая система может быть скомпрометирована или может приводить к новым ошибкам. Вместо отладки или усиления устаревшей системы попробуйте ее удалить.

Удаление устаревшей системы улучшит состояние безопасности. Например, с одним из авторов этой книги однажды связался (через программу вознаграждения за нахождение уязвимостей Google, как описано в главе 20) исследователь безопасности, обнаруживший проблему в одной из устаревших систем нашей команды. Мы ранее изолировали ее от своей сети, но довольно долго не обновляли систему. Новые члены команды не знали о существовании устаревшей системы. В ответ на открытие исследователя мы решили просто удалить систему. Нам больше не требовалась большая часть предоставляемых ею функций, и мы смогли заменить ее более простой современной.



Будьте внимательны при переписывании устаревших систем. Спросите себя, почему переписанная система будет работать лучше старой. Иногда нужно переписать систему, потому что проще добавить новый код, чем отлаживать старый. Есть более веские причины для замены систем: иногда требования к системе обновляются, и при небольшом объеме работы старую систему можно удалить. Возможно, вы чему-то научились из первоначальной системы и можете использовать эти знания для улучшения новой.

Остановитесь, когда что-то пойдет не так

Многие ошибки сложно найти, потому что источник и его последствия могут быть далеко друг от друга в системе. Недавно мы столкнулись с проблемой, из-за которой сетевое оборудование искажало внутренние ответы DNS для сотен устройств. Программы выполняют поиск DNS для устройства `exa1`, но получают адрес `exa2`. Две наши системы по-разному реагировали на эту ошибку.

- Одна система (сервис архивирования) подключалась к `exa2` (неправильному компьютеру). Но после она проверила, является ли устройство, к которому она подключена, ожидаемым. Имена устройств не совпадали, поэтому задача сервиса архивирования не могла быть выполнена.
- Другая система, собирающая метрики устройства, делала это с неправильного устройства, `exa2`. После система запустила задачу исправления ошибок на `exa1`. Мы обнаружили это поведение только тогда, когда технический специалист указал, что его попросили починить пятый диск устройства, на котором не было пяти дисков.

Из этих двух реакций предпочтительнее первая. Когда проблемы и их последствия далеко друг от друга в системе — когда сеть вызывает ошибки на уровне приложений — закрытие приложения при сбое может предотвратить последствия (подозрение на сбой диска в неправильной системе). В главе 8 мы подробнее обсуждали, нужно ли открывать или закрывать систему при сбое.

Улучшите контроль доступа и авторизации даже для неконфиденциальных систем

Может возникнуть ситуация, когда «на одной кухне слишком много поваров». То есть вы можете столкнуться с отладкой ситуации, когда предполагаемый источник ошибки — множество людей, что затрудняет выявление причины. Однажды мы ответили на сбой, вызванный поврежденной строкой БД, и не смогли найти источник поврежденных данных. Чтобы исключить возможность того, что кто-то мог записать данные в производственную БД по ошибке, мы снизили количество предоставлявших доступ ролей и запрашивали обоснования

для любого доступа со стороны человека. Несмотря на неконфиденциальность данных, внедрение стандартной системы безопасности помогло нам предотвратить и расследовать будущие ошибки. К счастью, мы смогли восстановить эту строку базы данных из резервной копии.

Совместная отладка: способ обучения

Многие команды разработчиков обучаются отладке, работая коллективно над реальными проблемами в реальном времени (или в режиме видеоконференции). Совместная отладка помогает не только поддерживать навыки опытных отладчиков, но полезна и для установления психологической безопасности новых членов команды. Они смогут увидеть, как лучшие отладчики в команде зашли в тупик, отступили или как-то иначе борются с проблемой. Это показывает им, что быть неправым и испытывать трудности — нормально¹. Подробнее об обучении безопасности см. в главе 21.

Следующие правила помогут вам улучшить процесс обучения.

- Ноутбук должен быть открыт только у двух человек:
 - «оператор», выполняющий действия, запрошенные другими отладчиками;
 - «стенографист».
- Каждое действие должно быть определено аудиторией. Только оператор и стенографист могут использовать компьютеры, но они не определяют предпринятые действия. Таким образом, участники не выполняют отладку в одиночку, а только представляют ответ, не делаясь своими мыслительными процессами и действиями по устранению неполадок.

Команда коллективно определяет одну или несколько проблем для изучения, но никто в комнате не должен знать решение этой проблемы заранее. Каждый человек может запросить оператора выполнить действие для устранения проблемы (например, открыть панель мониторинга, просмотреть журналы, перезагрузить сервер и т. д.). Вся команда присутствует, чтобы засвидетельствовать предложения, поэтому каждый может узнать об инструментах и методах, предлагаемых другими участниками. Даже очень опытные члены команды могут почерпнуть что-то новое.

Как описано в главе 28 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/accelerating-sre-on-call/>), некоторые команды применяют имитационные упражнения «Колесо неудачи». Они могут быть либо теоретическими,

¹ См. статью в блоге Джулии Розовски: The Five Keys to a Successful Google Team (<https://oreil.ly/gpxoL>) и статью Чарльза Дьюхигга в «Нью-Йорк Таймс»: What Google Learned from Its Quest to Build the Perfect Team (<https://oreil.ly/YJmwk>).

либо практическими, когда тестирующий вызывает ошибку в системе. В этих сценариях тоже есть две роли:

- «создатель теста» — конструирует и представляет тест;
- «приемщик теста» — пытается решить проблему с помощью своих товарищей по команде.

Некоторые команды предпочитают для поэтапных упражнений безопасную среду. Но для практических упражнений с «Колесом неудачи» нужна нестандартная настройка, тогда как в большинстве систем всегда есть настоящая проблема для коллективной отладки. Независимо от подхода важно, чтобы все были включены в процесс обучения. Так каждый будет чувствовать себя в безопасности и сможет активно участвовать.

Совместная отладка и упражнение «Колесо несчастья» — отличные способы представить новые методы в команде и закрепить лучшие практики. Люди могут увидеть, насколько полезны методы в реальных ситуациях, часто для самых сложных проблем. Команды собирают некоторые практические проблемы отладки. Это делает их действия эффективнее во время настоящего кризиса.

Чем отличаются исследования безопасности от отладки

Мы ожидаем отлаженных систем от каждого, но советуем, чтобы компрометации системы исследовали только обученные и опытные специалисты по безопасности и экспертизе. Когда грань между «исследованием ошибок» и «проблемой безопасности» неясна, можно сотрудничать между двумя командами специалистов.

Исследование ошибок часто начинается, когда система сталкивается с проблемой. Оно сосредоточено на происшествиях в системе: какие данные были отправлены, что произошло с этими данными и как сервис начал действовать вопреки своему назначению. *Исследования безопасности* начинаются немного иначе и быстро переходят к таким вопросам, как: что делал пользователь, отправивший эту работу? За что еще отвечает этот пользователь? Есть ли настоящий злоумышленник в системе? Что он будет делать дальше? Если вкратце, то отладка более сфокусирована на коде, а исследование безопасности может быстро сосредоточиться на атакующем злоумышленнике.

Ранее рекомендованные шаги для устранения проблем могут мешать во время исследований безопасности. У добавления нового кода, удаления устаревших систем и т. д. могут быть непредвиденные побочные эффекты. Мы отреагировали на ряд происшествий, когда отладчик удалял файлы, которые обычно не принадлежали системе, для разрешения ошибочного поведения — и оказывалось,

что они были введены злоумышленником, который так был предупрежден об исследовании. Однажды злоумышленник даже ответил тем же, удалив всю систему!

Если вы подозреваете компрометацию безопасности, ваше исследование может обрести новое свойство срочности. Вероятность намеренного изменения системы поднимает вопросы, кажущиеся серьезными и насущными. Что это за злоумышленник? Какие другие системы могут попасть в зону действия взлома? Нужно ли оповещать правоохранительные органы? Чем больше организация занимается вопросами обеспечения безопасности, тем сложнее становится исследование безопасности (эта тема рассматривается в главе 17). Эксперты из других отделов (например, юристы) могут участвовать в этом до начала вашего собственного исследования. В общем, момент, когда вы подозреваете, что система безопасности скомпрометирована — подходящее время для получения помощи от специалистов по безопасности.

ПЕРЕСЕЧЕНИЕ БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: ПРИЗНАНИЕ КОМПРОМЕТАЦИИ

Во время исследования вы можете столкнуться со многими несоответствиями, которые могут быть просто странностями в системе, требующей исправления. Но если эти несоответствия начинают накапливаться или вы сталкиваетесь с их беспричинным увеличением — это может быть признаком вмешательства злоумышленника в систему. Вот примерный список несоответствий, способных вызвать подозрения о возможной компрометации.

- Нужные для отладки журналы и данные отсутствуют, усечены или повреждены.
- Вы начали свое исследование, изучая поведение системы, но обнаружили, что предположительно рассуждаете о действиях *учетной записи* или *пользователя*.
- Система ведет себя так, что вы не можете объяснить это как случайность. Например, веб-сервер продолжает выдавать интерактивные оболочки, хотя он выходит из строя.
- У файлов есть характеристики, кажущиеся ненормальными для типа файла (например, вы найдете файл с именем `rick_roll.gif`, но его размер составляет 1 Гбайт и имеет заголовки ZIP или TAR).
- Похоже, что файлы были преднамеренно скрыты, неправильно названы известными способами (например, *explore.exe* в Windows) или запутаны как-то иначе.

Иногда сложно решить, когда закончить расследование и объявить о происшествии в сфере безопасности. Многие инженеры склонны «не устраивать скандал», что обостряет проблемы. Ошибки могут быть не связаны с безопасностью, но продолжать исследования до момента доказательства обратного не всегда правильно. Помните о принципе «лошадей и зебр»: большинство проблем — это ошибки, а не злонамеренные действия. Но нужно следить за чередованием черных и белых полос.

Сбор подходящих и полезных записей журналов

По сути, журналы и выводы сбоев системы — это просто собранная информация, улучшающая понимание произошедшего в системе и помогающая исследовать как случайные, так и преднамеренные проблемы.

Перед запуском любого сервиса важно рассмотреть виды данных, которые сервис будет хранить от имени пользователей, и пути доступа к ним. Представим, что любое действие, дающее доступ к данным или системе, может быть предметом будущих исследований и что кому-то нужно будет провести аудит этого действия. Расследование любой проблемы сервиса или безопасности сильно зависит от ведения журналов.

Наше обсуждение «журналов» относится к структурированным, имеющим временные метки записям из систем. Во время расследований аналитики могут сильно полагаться и на другие источники данных, такие как выводы ядра, выводы памяти или трассировки стека. Обращайтесь с этими системами так же, как и с журналами. Структурированные журналы полезны для многих бизнес-целей — например, для формирования счетов за пользование сервисом. Но здесь мы сосредоточимся на структурированных журналах, собранных для исследований безопасности — информации, которую вам нужно собрать сейчас, чтобы она была доступна для решения проблемы в будущем.

Сделайте журналы неизменяемыми

Система, которую вы определяете для сбора журналов, должна быть неизменяемой. Записи журнала должно быть трудно изменить при создании (но не невозможно, см. следующий раздел «Примите во внимание конфиденциальность»), и изменения должны иметь неизменяемый контрольный журнал с историей. Злоумышленники обычно стараются стереть следы своей активности в системе из всех источников журналов. Часто для предотвращения этого используется удаленная запись журналов в централизованный и распределенный сервер. Это увеличивает рабочую нагрузку злоумышленника: помимо компрометации исходной системы, ему нужно компрометировать удаленный сервер журналов. Всегда тщательно защищайте систему ведения журналов.

До эпохи современных вычислений сверхкритические серверы подключались напрямую к линейному принтеру, такому как на рис. 15.1, печатающему записи журнала на бумаге по мере их создания. Чтобы стереть собственные следы, удаленному злоумышленнику понадобился бы кто-то, кто мог бы физически удалить бумагу из принтера и сжечь ее!

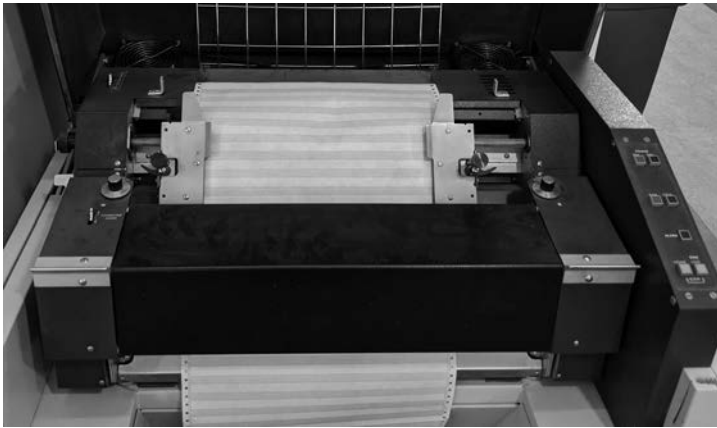


Рис. 15.1. Линейный принтер

Примите во внимание конфиденциальность

Сохранение конфиденциальности все более важно при проектировании систем. Хотя наша книга не о конфиденциальности, вам придется учитывать локальные правила и политики вашей организации при разработке журналов для исследований безопасности и отладки. Обязательно проконсультируйтесь с кем-либо из коллег из отделов конфиденциальности и права в вашей организации по этому вопросу. Вот некоторые темы для обсуждения.

Глубина ведения журнала

Журналы должны быть максимально полными и полезными для любого исследования. При исследовании безопасности может понадобиться изучить каждое действие пользователя (или злоумышленника, использующего его учетную запись) внутри системы, хост, с которого он вошел в систему, и точное время событий. Действуйте в соответствии с политикой организации о том, какую информацию можно записывать, учитывая, что многие методы сохранения конфиденциальности не позволяют передавать определенные данные пользователя в журналы.

Хранение

Для некоторых исследований полезно хранить журналы длительное время. Согласно исследованию 2018 года, большинству организаций в среднем нужно около 200 дней (<https://oreil.ly/vAunm>) для обнаружения компрометации системы. Исследования инсайдерских рисков в Google опирались на журналы безопасности ОС, созданные несколько лет назад. Обсудите в вашей организации время хранения журналов.

Управление доступом и аудитом

Многие из средств контроля, рекомендуемые нами для защиты данных, применяются и к журналам. Обязательно защищайте журналы и метаданные так же, как любые другие данные. См. главу 5, где описаны подходящие стратегии.

Анонимизация или псевдонимизация данных

Анонимизация ненужных компонентов данных — по мере их написания или спустя определенный период времени — все более распространенный способ сохранения конфиденциальности при обработке журналов. Вы даже можете реализовать эту функцию так, чтобы исследователи и отладчики не могли определить, кем является данный пользователь, но могли четко построить график действий этого пользователя на протяжении всего сеанса для целей отладки. Анонимизация — это сложно. Посоветуйтесь со специалистами по конфиденциальности и изучите доступную литературу по этой теме¹.

Шифрование

Вы можете реализовать ведение журнала с условиями конфиденциальности при помощи асимметричного шифрования данных. Этот метод шифрования идеально подходит для защиты данных журнала: он использует нечувствительный «открытый ключ», который любой может использовать для безопасной записи данных, но для его расшифровки нужен секретный (закрытый) ключ. Такие варианты разработки, как ежедневно обновляющиеся пары ключей, позволяют отладчикам получать небольшие поднаборы данных журнала из недавней активности системы, в то же время предотвращая получение кем-либо больших объемов данных журнала. Обязательно тщательно обдумайте способ хранения ключей.

Определите, какие журналы безопасности нужно сохранить

Специалисты по безопасности часто предпочитают иметь как можно больше журналов, но будьте избирательны в том, что вы регистрируете и сохраняете. Хранение слишком большого количества журналов может дорого стоить (как обсуждается в подразделе «Бюджет на ведение журнала» далее), а отбор слишком больших наборов данных замедлит работу исследователя и использует много ресурсов. Здесь мы обсудим некоторые типы журналов, которые вы, возможно, захотите вести и сохранять.

¹ См.: Ghiasvand S., Ciorba F. M. Anonymization of System Logs for Privacy and Storage Benefits. 2017. Статья в arXiv arXiv:1706.04337 (https://oreil.ly/c_a0N). См. статью Яна Линдквиста о псевдонимизации персональных данных (<https://oreil.ly/W3OFr>) для соответствия Общему регламенту по защите данных (General Data Protection Regulation, GDPR).

Журналы операционной системы

У большинства современных ОС есть встроенные журналы. В Windows есть журналы событий Windows, а в Linux и Mac — журналы syslog и auditd. У многих устройств, входящих в комплект от поставщика (таких как системы камер, средства управления средой и панели пожарной сигнализации), есть стандартная ОС, создающая журналы (например, как в Linux) под капотом. Встроенные фреймворки ведения журналов полезны для исследований, и их использование почти не требует усилий, ведь они часто включены по умолчанию или легко настраиваются. Некоторые механизмы, такие как auditd, по умолчанию не включены из соображений производительности. Но их можно включить для использования в реальных условиях.

Локальные агенты

Многие компании выбирают включение дополнительных возможностей ведения журналов через установку *системы обнаружения локальных угроз* (host intrusion detection system, HIDS) или *локальных агентов* на рабочие станции и серверы.

АНТИВИРУСНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

Антивирусное программное обеспечение сканирует файлы на наличие шаблонов, указывающих на известное вредоносное ПО (например, уникальную для конкретного вируса последовательность кода), и ищет подозрительное поведение (например, попытку изменить конфиденциальные системные файлы). Эксперты не всегда согласны с ценностью и использованием антивирусного ПО в современных условиях безопасности¹. Мы считаем, что антивирусная защита на каждом конечном компьютере стала менее полезной, так как угрозы стали более изощренными. Если антивирусное программное обеспечение написано плохо, оно может сильнее угрожать безопасности системы.

Современные локальные агенты (иногда называемые «агентами следующего поколения») пользуются новейшими методами для обнаружения все более сложных угроз. Некоторые агенты объединяют моделирование поведения системы и пользователя, машинное обучение и анализ угроз для выявления ранее неизвестных атак. Другие больше сосредоточены на сборе дополнительных данных о работе системы, которые могут быть полезны для автономного обнаружения и отладки. Некоторые, такие как OSQuery (<https://osquery.io/>) и GRR (<https://github.com/google/grr>), обеспечивают видимость системы в режиме реального времени.

¹ См. презентацию Джоксана Копета: Breaking Antivirus Software (<https://oreil.ly/alqtv>) на 44CON 2014.

Локальные агенты всегда влияют на производительность и часто могут вносить разногласия между конечными пользователями и командами разработчиков. Чем больше данных может собирать агент, тем сильнее будет его влияние на производительность — из-за более глубокой интеграции платформы и большей обработки в серверной среде. Некоторые агенты работают как часть ядра, некоторые — как приложения пользовательского пространства. У агентов ядра функциональность больше и поэтому они эффективнее, но могут страдать от проблем с надежностью и производительностью в попытках не отставать от изменений функциональности ОС. Агенты, которые запускаются как приложения, намного проще в установке и настройке, еще у них меньше проблем с совместимостью. Ценность и производительность локальных агентов сильно различаются, поэтому мы советуем тщательно оценить агент перед его использованием.

Приложения для ведения журнала

Приложения для ведения журнала — независимо от того, поставляются ли они в комплекте, такие как SAP и Microsoft SharePoint, имеют ли открытый исходный код или написаны на заказ, — создают журналы, которые можно собирать и анализировать. Потом эти журналы можно использовать для настраиваемого обнаружения и дополнения данных исследования. Мы используем журналы приложений из Google Drive (<https://oreil.ly/Fhckk>), чтобы определить, загружал ли скомпрометированный компьютер конфиденциальные данные.

При разработке пользовательских приложений важно сотрудничество между специалистами по безопасности и разработчиками. Оно может гарантировать, что приложения регистрируют действия, относящиеся к безопасности, такие как запись данных, изменения владельца или состояния и действия, связанные с учетной записью. Как мы упоминали в пункте «Улучшите наблюдаемость» выше в этой главе, использование инструментов ведения журнала для ваших приложений может облегчить отладку при выявлении эзотерических проблем безопасности и надежности, которые иначе было бы сложно разделить.

Облачные системы ведения журнала

Все чаще организации переносят части своих бизнес-процессов или процессов разработки в облачные сервисы. Начиная с данных в приложениях по принципу «программа как услуга» (Software-as-a-Service, SaaS) и заканчивая виртуальными машинами, на которых выполняются критические рабочие нагрузки, связанные с обслуживанием клиентов. Все эти сервисы представляют уникальные поверхности атаки и генерируют уникальные записи журнала. Например, злоумышленник может скомпрометировать учетные данные для облачного проекта, развернуть новые контейнеры в кластере Kubernetes и использовать

эти контейнеры для кражи данных из доступных хранилищ кластера. Модели облачных вычислений обычно запускают новые экземпляры ежедневно. Это усложняет обнаружение угроз в облаке.

Что касается обнаружения подозрительной активности, то здесь у облачных сервисов есть как преимущества, так и недостатки. Используя такие службы, как Google BigQuery, легко и недорого собирать и хранить большие объемы данных журналов и даже запускать правила обнаружения прямо в облаке. Облачные сервисы Google предлагают встроенные решения для ведения журнала, такие как Cloud Audit Logs (<https://oreil.ly/6SUwV>) и Stackdriver Logging (<https://oreil.ly/6SUwV>). С другой стороны, из-за большого количества видов облачных сервисов сложно идентифицировать, включить и централизовать все нужные журналы. Разработчики могут легко создавать новые файлы в облаке, поэтому многим компаниям сложно распознать все свои облачные файлы. Поставщики облачных услуг могут заранее определить список доступных журналов, и эти параметры могут быть недоступны для настройки. Учитывайте ограничения ведения журнала от вашего провайдера и потенциальные слепые зоны.

УЧЕТ ВАШИХ АКТИВОВ

Создавая конвейеры для исследования проблем в системе, подумайте о создании набора инструментов для использования этой информации в целях идентификации облачных активов, включая те, о которых вы не знаете. Пример: интеграция с финансовой системой для поиска счетов, выплачиваемых поставщикам облачных услуг, может выявить компоненты системы, требующие дальнейшего внимания. В статье об архитектуре BeyondCorp (<https://oreil.ly/7JXZx>) в Google мы упомянули набор инструментов под названием «Сервис инвентаризации устройств (Device Inventory Service)», который мы используем для обнаружения активов, потенциально принадлежащих Google (или содержащих наши данные). Создание аналогичного сервиса и его распространение в облачных средах — один из способов определения таких типов активов.

Разное коммерческое ПО, которое часто находится в облаке, предназначено для обнаружения атак на облачные сервисы. Большинство известных облачных провайдеров предлагают интегрированные сервисы обнаружения угроз, такие как Google Event Threat Detection (<https://oreil.ly/yJdVI>). Многие компании объединяют такие сервисы с внутренне разработанными правилами обнаружения или сторонними продуктами.

Агенты безопасности облачного доступа (cloud access security broker, CASB) — заметная категория технологий обнаружения и предотвращения угроз. CASB работают как посредники между конечными пользователями и облачными службами, обеспечивая контроль безопасности и ведение журнала. CASB может

помешать вашим пользователям загружать определенные типы файлов, или же он может записывать информацию о каждом загруженном пользователем файле. У многих CASB есть функция обнаружения, предупреждающая команду обнаружения о возможном злонамеренном доступе. Вы можете интегрировать журналы из CASB в пользовательские правила обнаружения.

Ведение журнала и обнаружение на основе сети

С конца 1990-х годов использование *систем обнаружения сетевых вторжений* (network intrusion detection system, NIDS) и *систем предотвращения вторжений* (intrusion prevention system, IPS), захватывающих и проверяющих сетевые пакеты, можно назвать обычной техникой обнаружения и ведения журнала. IPS блокируют и некоторые атаки. Они могут собирать информацию о том, какие IP-адреса обменивались трафиком, наряду с ограниченной информацией о нем, например размером пакета. Некоторые IPS могут записывать все содержимое определенных пакетов на основе настраиваемых критериев, например пакетов, отправляемых в системы высокого риска. Другие могут находить вредоносные действия в режиме реального времени и отправлять оповещения соответствующей команде. Эти системы очень полезны, и у них мало недостатков, поэтому мы настоятельно рекомендуем их почти для любой организации. Но тщательно продумайте, кто может эффективно сортировать генерируемые ими оповещения.

Журналы DNS-запросов — тоже полезные сетевые источники. Они позволяют увидеть, разрешил ли какой-либо компьютер в компании имя хоста. Вы можете захотеть узнать, выполнил ли какой-либо хост в вашей сети DNS-запрос для известного вредоносного имени хоста, или вы можете проверить ранее разрешенные домены, чтобы определить каждое устройство, затронутое контролирующим систему злоумышленником. Команда безопасности может использовать «провалы» DNS, ложно разрешающие известные вредоносные домены. Поэтому атакующие не смогут эффективно их использовать. Далее системы обнаружения оповещают разработчиков о случаях, когда пользователи получают доступ к домену «провала».

Вы можете использовать журналы из любых веб-прокси, используемых для внутреннего или выходного трафика. К примеру, веб-прокси может пригодиться для сканирования веб-страниц на наличие признаков фишинга или известных шаблонов уязвимостей. При использовании прокси для обнаружения важно рассмотреть вопрос о конфиденциальности сотрудников и обсудить использование журналов прокси с командой юристов. В общем, адаптируйте обнаружение как можно ближе к вредоносному контенту, чтобы уменьшить данные сотрудников, получаемые при обработке оповещений.

Бюджет на ведение журнала

Для отладки и исследования нужны дополнительные ресурсы. В одной из систем, с которой мы работали, было 100 Тбайт журналов, которые никогда не использовались. Ведение журнала потребляет много ресурсов, а журналы часто обрабатываются реже при отсутствии проблем. Поэтому у вас может возникнуть желание меньше вкладывать в инфраструктуру ведения журналов и отладки. Чтобы такого не было, планируйте бюджет на ведение журнала заранее, учитывая, сколько данных вам может понадобиться для решения проблемы с сервисом или устранения происшествия в сфере безопасности.

Современные системы ведения журнала часто содержат реляционную систему данных (например, Elasticsearch или BigQuery) для быстрого и простого запроса данных в режиме реального времени. Стоимость этой системы растет вместе с количеством необходимых для хранения и индексации событий, количеством компьютеров для обработки и запрашивания данных и требуемым пространством хранения. Поэтому при длительном сохранении данных полезно расположить журналы из соответствующих источников по приоритетам. Это важное компромиссное решение: если злоумышленник умеет скрывать свои следы, вам может понадобиться время на обнаружение происшествия. Если вы храните журналы доступа только в течение недели, вы вообще не сможете исследовать взлом!

ПЕРЕСЕЧЕНИЕ БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: ВЫДЕЛЕНИЕ БЮДЖЕТА ДЛЯ ДОЛГОСРОЧНОГО ХРАНЕНИЯ ЖУРНАЛА

Ваш бюджет может не позволять долго хранить данные журнала. Что делать в таком случае, чтобы хранить столько журналов, сколько захотите? Для уравнивания затрат на хранение и потребностей в хранении мы рекомендуем *корректно* спроектировать журналы, а не останавливать сбор журналов при достижении емкости хранилища.

Обобщение данных все более распространено для постепенного снижения качества журналов. Например, вы могли бы создать систему журналов, выделяющую 90 % хранилища для хранения журналов в полном виде и резервирующую оставшиеся 10 % для упрощенного объединения некоторых данных. Чтобы не было нужно перечитывать и анализировать уже записанные журналы (это требует больших вычислительных ресурсов), ваша система может записывать два файла журналов во время сбора данных. В первом будут данные с полной точностью, а во втором — в объединенном виде. Когда ваше хранилище будет заполнено, вы можете удалить журналы большего размера, чтобы освободить место и сохранить файлы меньшего размера как данные с более долгим сроком хранения. Вы можете выполнять полные захваты пакетов и данных сетевого потока, удаляя захваты через N дней, но сохраняя данные сетевого потока в течение года. В этом случае для журналов понадобится меньше затрат на хранение, но они все же обеспечат ключевой анализ информации, передаваемой хостами.

Для сбора журналов, ориентированных на безопасность, мы предлагаем такие стратегии инвестирования.

- Сосредоточьтесь на журналах с хорошим соотношением сигналов и шумов. Брандмауэры обычно блокируют множество пакетов, большинство из которых безвредны. Даже вредоносные пакеты, заблокированные брандмауэром, могут не стоить вашего внимания. Сбор журналов для этих заблокированных пакетов может использовать огромную пропускную способность и объем хранилища практически без пользы.
- По возможности сжимайте журналы. В большинстве журналов много дублированных метаданных, поэтому сжатие обычно очень эффективно.
- Разделяйте хранилище на «горячее» и «холодное». Вы можете выгружать старые журналы в дешевое автономное облачное хранилище («холодное хранилище»), сохраняя более свежие или связанные с известными происшествиями на локальных серверах, для немедленного использования («горячее хранилище»). Также вы можете хранить сжатые простые журналы долгое время, но помещать только самые последние в дорогую реляционную БД с полной индексацией.
- Разумно заменяйте записи журнала. Обычно лучше сначала удалить самые старые журналы, но вы можете сохранить важные на более длительное время.

Надежный и безопасный доступ при отладке

Для устранения неполадок часто нужен доступ к системам и данным, которые они хранят. Может ли злонамеренный или скомпрометированный отладчик увидеть конфиденциальную информацию? Можно ли устранить сбой системы безопасности (помните: все системы выходят из строя!)? Убедитесь в надежности и безопасности ваших систем отладки.

Надежность

Ведение журнала — это еще одно место, где система может выйти из строя. Например, ей может не хватить места на диске для хранения журналов. Открытие при отказе в этом примере влечет за собой другой компромисс. Такой подход может сделать вашу систему устойчивее, но злоумышленник может нарушить механизм ведения журнала.

Планируйте ситуации, когда вам может понадобиться отладка или восстановление самих систем безопасности. Примите во внимание компромиссы, чтобы

убедиться, что система не блокируется полностью, но вы можете сохранить ее в безопасности. В этом случае вы можете рассмотреть вопрос об автономном сохранении в безопасном месте набора учетных данных только для экстренных случаев. Эти данные при использовании должны посылать высоконадежные аварийные сигналы. Например, недавний выход из строя сети Google (<https://oreil.ly/hxrpj3>) вызвал большую потерю пакетов. Когда респонденты пытались получить внутренние учетные данные, система проверки подлинности не смогла получить доступ к одному серверу и в итоге закрылась. Но благодаря экстренным учетным данным респонденты смогли аутентифицировать и исправить сеть.

Безопасность

Одна из систем, с которой мы работали, была направлена на поддержку пользователей по телефону и позволяла администраторам просматривать интерфейс так, как видит его пользователь. Для отладки эта система была замечательной — можно было четко и быстро воспроизвести проблему пользователя. Но такой тип системы дает слишком много возможностей. Отладочные конечные точки — от места, где происходит перевоплощение администратора в пользователя, до места доступа к БД — должны быть защищены.

Часто для отладки необычного поведения системы не нужен доступ к пользовательским данным. При диагностировании проблем с трафиком часто достаточно ТСП данных о скорости и качестве байтов в проводе для диагностики проблем. Шифрование данных при передаче защитит их от любых попыток просмотра извне. У этого подхода есть отличный побочный эффект: при необходимости множество специалистов может получить доступ к выводам пакетов. Но одна из возможных ошибок — обработка метаданных как неконфиденциальных. Злоумышленник все еще может многое узнать о пользователе из метаданных, отслеживая связанные шаблоны доступа. Например, отмечая, что один пользователь обращается к сайту адвоката по бракоразводным процессам и к сайту знакомств в одном и том же сеансе. Тщательно оцените риски, связанные с обработкой метаданных как неконфиденциальных.

Кроме того, иногда для анализа *требуются* фактические данные. К примеру, для поиска часто используемых записей в БД и выяснения причины частого использования этих обращений. Однажды мы отладили проблему хранения низкого уровня, вызванную тем, что одна учетная запись получала тысячи писем в час. В подразделе «Сеть с нулевым доверием» в главе 5 вы найдете больше информации об управлении доступом в таких ситуациях.

Резюме

Отладка и исследования — важные аспекты управления системой. Основные моменты в этой главе.

- *Отладка* — важное действие, благодаря которому вы можете добиться большего, используя систематические методы, а не догадки. Вы значительно упростите отладку, внедрив инструменты или ведение журнала для обеспечения прозрачности системы. Постоянно практикуйте отладку, чтобы отточить свои навыки.
- *Исследования безопасности* отличаются от отладки. Это совокупность разных команд, тактик и рисков. В вашей исследовательской команде должны быть опытные специалисты по безопасности.
- *Централизованное ведение журнала* полезно для целей отладки, критически важно для исследований и часто полезно для бизнес-анализа.
- *Повторяйте одни и те же действия*, просматривая недавние исследования и спрашивая себя, какая информация помогла бы вам отладить или исследовать проблему. Отладка — это процесс постоянного улучшения. Вы будете регулярно добавлять новые источники данных и искать способы улучшения наблюдаемости.
- *Проектируйте для безопасности*. Вам нужны журналы. Отладчикам нужен доступ к системам и хранимым данным. Но с увеличением объема хранимых данных журналы и конечные точки отладки могут стать целями для злоумышленников. Разработайте системы ведения журнала для сбора важной информации и получения надежных разрешений, привилегий и политик для доступа к этим данным.

Как отладка, так и исследования безопасности часто зависят от внезапных озарений и простой удачи. Даже лучшие отладчики иногда остаются в неведении. Помните, что фортуна благосклонна к тем, кто подготовлен. Внедрите ведение журнала и создайте систему, способную их индексировать и исследовать, — и вы сможете в полной мере воспользоваться шансами, которые возникнут у вас на пути. Удачи!

ЧАСТЬ IV

Обслуживание систем

У организаций, готовых к неудобным ситуациям, больше шансов на успешное преодоление критической проблемы.

Нельзя заранее спланировать все сценарии, способные нарушить работу вашей организации. Но первые шаги на пути к общей стратегии планирования действий в случае бедствия, как описано в главе 16, конкретны и доступны. Они включают в себя создание команды реагирования на происшествия, предварительную настройку ваших систем, обучение сотрудников перед происшествием и тестирование систем и планов реагирования — этапы, которые подготовят вас к антикризисному управлению (тема главы 17). При работе с происшествиями в сфере безопасности люди с разными навыками и ролями должны иметь возможность сотрудничать и эффективно общаться для поддержания работоспособности систем.

После атаки ваша организация должна будет взять на себя управление восстановлением и справиться с последствиями, как описано в главе 18. Предварительное планирование на этих этапах поможет вам подготовить подробный ответ и узнать, что случилось, чтобы предотвратить повторение инцидента.

Для более полной картины мы добавили несколько конкретных контекстных примеров.

- В главе 16 вы узнаете о том, как в Google был создан план реагирования на разрушительное землетрясение в районе залива Сан-Франциско.
- В главе 17 рассказывается о том, как инженер обнаружил, что нераспознанная служебная учетная запись была добавлена в ранее не используемый облачный проект.
- В главе 18 обсуждаются компромиссы между отражением атаки злоумышленника, смягчением его действий и внесением долгосрочных изменений при столкновении с крупномасштабной фишинговой атакой.

Планирование аварийных ситуаций

*Майкл Робинсон и Шон Нунан
с Алексом Брэмли и Кавитой Гулиани*

В системах неизбежно происходят сбои надежности или инциденты безопасности, и вы должны быть готовы к их устранению. Мы рекомендуем проводить мероприятия по планированию аварийных ситуаций ближе к концу цикла разработки, после завершения этапа внедрения.

Эта глава начинается с анализа риска бедствий — важного шага для разработки гибкого плана реагирования на бедствия. Далее мы пройдем этапы подготовки команды реагирования на происшествия и дадим советы об определении предварительных действий, которые можно выполнить до возникновения аварии. В конце мы подробно расскажем о тестировании организации до катастрофы и приведем несколько примеров, показывающих, как Google готовился к некоторым конкретным сценариям.

Сложные системы могут выходить из строя по-разному — от неожиданных перебоев в обслуживании до атак злоумышленников с целью получения несанкционированного доступа. Вы можете предвидеть и предотвратить некоторые из них с помощью методов обеспечения надежности и безопасности, но в долгосрочной перспективе сбой не избежать.

Не надейтесь, что система переживет катастрофу или атаку или что ваши сотрудники смогут принять разумные ответные меры. Планирование катастроф гарантирует, что вы постоянно работаете над улучшением способности восстанавливаться после катастроф. Хорошая новость в том, что первые шаги к разработке комплексной стратегии прагматичны и доступны.

Определение «бедствия»

Вряд ли вы узнаете о бедствии, когда оно уже будет в полном разгаре. Прежде чем наткнуться на охваченное огнем здание, вы увидите дым или почувствуете его запах — какой-то небольшой указатель, не обязательно говорящий о катастрофе. Огонь распространяется постепенно, и вы не осознаете всю крайность ситуации, пока не окажетесь в гуще событий. Так же иногда небольшое событие, например, ошибка учета, упомянутая в главе 2, может привести к полномасштабному реагированию на происшествие.

Бедствия бывают разных форм.

- *Стихийные бедствия*, в том числе землетрясения, торнадо, наводнения и пожары. У них разная степень воздействия на систему.
- *Сбои инфраструктуры* — сбои компонентов или неправильные настройки. Их не всегда легко выявить, и их влияние может колебаться от малого до большого.
- *Перебои в обслуживании или поставках*, которые наблюдаются клиентами или другими заинтересованными сторонами.
- *Ухудшение работы сервисов*, работающих вблизи пороговых значений. Это состояние иногда трудно определить.
- *Внешний злоумышленник*, способный незаконно получить доступ к системе на длительный срок, прежде чем его обнаружат.
- *Несанкционированное разглашение конфиденциальных данных*.
- *Срочные уязвимости безопасности*, требующие немедленного применения патчей для исправления новых критических уязвимостей. Эти события рассматриваются как неизбежные атаки безопасности (см. подраздел «Компрометации и ошибки» в главе 17).

В этой и следующей главах мы используем термины *бедствие* и *кризис* взаимозаменяемо для обозначения любой ситуации, способной потребовать объявления о происшествии и принятия мер реагирования.

Стратегии динамического реагирования на бедствия

Возможных бедствий может быть много, но разработка гибких планов реагирования на них позволит вам адаптироваться к быстро меняющимся ситуациям. Заранее продумав сценарии, с которыми вы можете столкнуться, вы уже сделаете первый шаг к их подготовке. Как и в случае с любым набором навыков, можно отточить навыки реагирования на бедствия, планируя, практикуя и повторяя процедуры, пока они не войдут в привычку.

Реакция на происшествие не похожа на катание на велосипеде. Без обычной практики респондентам трудно сохранить хорошую мышечную память. Недостаток практики может привести к раздробленным реакциям и увеличению времени восстановления. Поэтому нужно регулярно отрабатывать и уточнять планы реагирования на бедствия. Чем лучше у вас будут отработаны навыки управления инцидентами, тем легче экспертам в данной области будет действовать во время реагирования на происшествия — если эти навыки вошли в привычку, им будет несложно следить за процессом.

Полезно разбить план реагирования на этапы: немедленное реагирование, краткосрочное восстановление, долгосрочное восстановление и возобновление обслуживания. На рис. 16.1 показаны основные этапы, связанные с аварийным восстановлением.

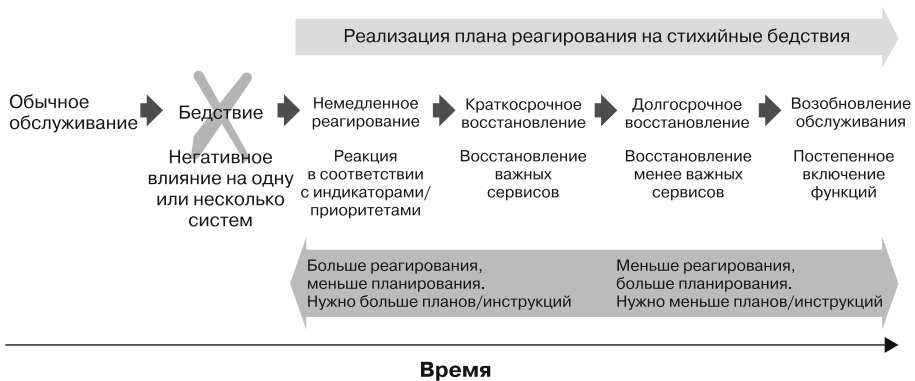


Рис. 16.1. Этапы реагирования на аварийное восстановление

На этапе краткосрочного восстановления должна быть разработка *критериев выхода* из происшествия. То есть критериев для объявления о завершении реагирования на инцидент. Успешное восстановление — это восстановление сервиса до полного рабочего состояния. Но базовое решение может иметь новую конструкцию, обеспечивающую такой же уровень обслуживания. Критерии могут тоже потребовать полного смягчения угроз безопасности, выявленных анализом рисков.

Разрабатывая стратегию, направленную на немедленное реагирование, краткосрочное и долгосрочное восстановление и возобновление работы, ваша организация может подготовиться, выполнив следующие действия.

- Анализ возможных бедствий, способных либо слегка затронуть организацию, либо оказать сильное влияние.
- Создание команды реагирования.

- Создание планов реагирования и подробных инструкций.
- Настройка систем должным образом.
- Тестирование процедур и систем.
- Добавление отзывов о результатах испытаний и оценок.

Анализ риска катастроф

Проведение анализа риска катастроф — первый шаг к определению важнейших операций организации, отсутствие которых приведет к общему нарушению. В основные операционные функции входит не только использование важных базовых систем, но и их базовых зависимостей, таких как сетевые компоненты и компоненты прикладного уровня. Анализ риска бедствий должен выявить следующее.

- Системы, способные вывести из строя обслуживание при повреждении или отключении. Вы можете классифицировать их как необходимые, важные или несущественные.
- Ресурсы — технологические или человеческие — понадобятся для реагирования на происшествие.
- Сценарии бедствия, которые могут возникнуть в каждой из ваших систем. Сценарии могут быть сгруппированы по вероятности, частоте возникновения и влиянию на обслуживание (низкое, среднее, высокое или критическое).

Вы можете интуитивно проводить оценку операций, но более формализованный подход к оценке рисков позволит избежать шаблонного мышления и выделить неочевидные риски. Для тщательного анализа ранжируйте риски, с которыми сталкивается организация, при помощи стандартизированной матрицы, учитывающей возможность возникновения каждого риска и его влияние на организацию. В приложении А есть пример матрицы оценки рисков, которую организации любых размеров могут адаптировать к особенностям своих систем.

Рейтинги риска обеспечивают хорошее эмпирическое правило о том, где в первую очередь сосредоточить внимание. Просмотрите список рисков и определите резко выделяющиеся значения после ранжирования. Например, маловероятное событие может быть оценено как критическое из-за возможного урона. Вы можете попросить эксперта пересмотреть оценку для выявления рисков со скрытыми факторами или зависимостями.

Ваша оценка риска может меняться в зависимости от местонахождения активов организации. Сайт в Японии или на Тайване должен учитывать воздействие тай-

фунов, а сайт в юго-восточной части США — принимать во внимание ураганы. Рейтинги риска могут меняться по мере зрелости организации и добавления таких отказоустойчивых систем, как резервные интернет-каналы и источники питания. Крупные организации должны проводить оценки рисков как на глобальном уровне, так и на уровне отдельных сервисов. Не мешает и периодически проверять и обновлять эти оценки по мере изменения операционной среды. Благодаря оценке рисков, определяющей, какие системы нуждаются в защите, вы сможете создать команду реагирования, подготовленную с помощью инструментов, процедур и обучения.

Создание команды реагирования на происшествия

Есть несколько способов создать команду реагирования на происшествия (incident response, IR).

- Через создание специальной IR-команды, занятой полный рабочий день.
- Распределяя IR-обязанности по между сотрудниками в дополнение к их существующим рабочим функциям.
- Путем передачи деятельности в области IR сторонним специалистам.

Из-за ограничений по размеру и бюджету многие организации полагаются на то, что штатные работники могут взять на себя несколько задач. Некоторые сотрудники будут выполнять обычные рабочие обязанности и реагировать на происшествия при необходимости. Для организаций с потребностями посложнее нужно создать укомплектованную специальную IR-команду. Чтобы была уверенность в том, что респонденты всегда готовы ответить, проходят соответствующую подготовку, имеют необходимый доступ к системе и достаточно времени для реагирования на широкий спектр инцидентов.

Независимо от выбранной модели вы можете применять следующие методы для создания успешных команд.

Определение членов команды и их ролей

При подготовке плана реагирования определите основную группу людей, которые будут реагировать на инциденты, и четко обозначьте их роли. Небольшие организации могут полагаться на участников из разных команд или даже на одну команду, реагирующую на все происшествия. Организации с большим количеством ресурсов могут определить специальную команду для каждой функциональной области. Например, команду реагирования на происшествия

безопасности, команду для работы с конфиденциальностью и оперативную команду для надежности публичных сервисов.

Вы можете передать некоторые функции сторонним специалистам, оставив часть для внутреннего реагирования. Если у вас недостаточно средств и нагрузки для полного штата своей криминалистической команды, вы можете передать эти задачи на аутсорсинг, сохраняя при этом команду реагирования на происшествия. Возможный недостаток аутсорсинга — внешние респонденты могут быть недоступны сразу в момент вызова. Учитывайте время отклика при определении того, какие функции следует оставить внутри, а какие передать сторонним специалистам и вызывать во время чрезвычайной ситуации.

Для реагирования на происшествия вам могут понадобиться некоторые или все из следующих ролей.

Руководитель по реагированию на происшествия

Лицо, управляющее реагированием на отдельное происшествие.

SR-инженеры (<https://oreil.ly/IeMvF>)

Люди, способные перенастроить уязвимую систему или внедрить код для исправления ошибки.

Служба связи с общественностью

Люди, отвечающие на публичные запросы или делающие заявления для СМИ. Эти сотрудники часто работают с руководителями по коммуникациям, создавая сообщения.

Служба поддержки клиентов

Люди, отвечающие на запросы клиентов или активно связывающие с пострадавшими клиентами.

Юристы

Адвокаты, консультирующие по правовым вопросам, таким как применимые законы, статуты, правила или контракты.

Специалисты по обеспечению конфиденциальности

Люди, решающие проблемы, связанные с вопросами технической конфиденциальности.

Специалисты по экспертизе

Люди, выполняющие реконструкцию и определение события, чтобы понять, что и как произошло.

Специалисты по безопасности

Люди, анализирующие влияние инцидента на безопасность, которые могут поработать с SR-инженерами или специалистами по обеспечению конфиденциальности для защиты системы.

Определяя роли для штатного персонала, внедрите модель замены сотрудников, при которой IR-команда работает посменно. Очень важно заменять сотрудников во время инцидента для снижения утомляемости и обеспечения постоянной поддержки. Вы можете принять эту модель для гибкости, так как основные должностные обязанности меняются и обновляются со временем. Помните, что это роли, а не отдельные лица. У одного человека может быть несколько ролей во время происшествия.

ИЗБЕГАЙТЕ ЕДИНИЧНЫХ ТОЧЕК ОТКАЗА

Инциденты не учитывают приглашения на встречи, стандартный рабочий день, планы поездок или график отпусков. Замена сотрудников улучшит вашу реакцию на инциденты, но старайтесь избегать единичных точек отказа. Например, руководителям нужно делегировать полномочия для утверждения исправления критического кода, изменения настроек и ответа на сообщения связи во время происшествий в период чьего-либо отпуска. Если ваша организация многонациональна, назначайте делегатов в соответствии с часовыми поясами.

После распределения ролей внутри организации составьте начальный список сотрудников для работы в отдельных IR-командах. Предварительная идентификация этих людей помогает уточнить роли, обязанности и ответственность во время реагирования. Это минимизирует хаос и время на подготовку. Полезно определить *руководителя* IR-команды — человека с достаточным стажем для выделения ресурсов и устранения препятствий. Руководитель поможет собрать команду и работать с главными специалистами в условиях пересечения приоритетов. Обратитесь к главе 21, чтобы узнать об этом подробнее.

Создание устава команды

Устав IR-команды должен начинаться с *миссии* — одного предложения, описывающего типы происшествий, на которые нужно будет реагировать. Миссия позволяет быстро понять обязанности команды.

Сфера действия устава должна описывать среду, в которой вы работаете, с упором на технологии, конечных пользователей, продукты и заинтересованные стороны. Здесь должны быть четко определены типы происшествий, которые будут обрабатываться командой, и какие инциденты должны обрабатываться внутренними сотрудниками, а какие передаются сторонним командам.

КОМАНДНЫЙ ДУХ

При составлении устава команды — будь то для выделенной команды реагирования или многофункциональной виртуальной группы — обязательно подумайте о соответствии объема задач и рабочей нагрузки ресурсам. Если люди перегружены работой, их производительность снизится. Они могут даже покинуть вашу организацию. Поддержание командного духа во время реагирования на инцидент подробнее описано в подразделе «Командный дух» в главе 17.

Для обеспечения того, чтобы IR-команда сосредоточилась на заданных инцидентах, важно согласование работ между руководством организации и руководителем IR. Хотя IR-команда могла бы отвечать на индивидуальные запросы клиентов о настройках брандмауэра системы и включении/проверке журналов, эти задачи лучше подойдут службе поддержки клиентов.

Наконец, важно определить, как выглядит *успех* команды. Другими словами, как вы узнаете, что работа IR-команды выполнена или может быть объявлена выполненной?

Создание моделей серьезности и приоритетов

Модели серьезности и приоритета могут помочь IR-команде определить и оценить серьезность происшествий и нужный для реагирования оперативный темп. Используйте обе модели одновременно, так как они связаны между собой.

Модель серьезности позволяет команде классифицировать инциденты по степени их влияния на организацию. Вы можете ранжировать инциденты по пятибалльной (0–4) шкале, где 0 — это наиболее серьезные инциденты, а 4 — наименее серьезные. Можете принять любую шкалу, соответствующую вашей организационной культуре (отображение с помощью цветов, животных и т. д.). Например, при использовании пятибалльной шкалы неавторизованный доступ к сети может квалифицироваться как инцидент с серьезностью 0, а временная недоступность журналов безопасности может быть серьезностью в 2 балла. При построении модели просмотрите все ранее выполненные анализы рисков для назначения категории инцидентов в соответствии с рейтингами серьезности. Это гарантирует, что не все происшествия получают критическую или среднюю степень серьезности. Точные оценки помогут руководителям расставить приоритеты, если сообщается о нескольких происшествиях одновременно.

Модель приоритетов определяет скорость реакции персонала на инциденты. Эта модель основана на вашем понимании серьезности инцидента и тоже может использовать пятибалльную (0–4) шкалу, где 0 — это высокий приоритет,

а 4 — низкий. Приоритет задает темп требуемой работы. На происшествие с рейтингом 0 нужно реагировать немедленно — в первую очередь. А происшествие с рейтингом 4 может быть обработано во время рутинной оперативной работы. Согласованная модель приоритетов помогает синхронизировать работу различных команд и руководителей. Представьте, что одна команда рассматривает приоритет инцидента как 0, а вторая команда за недостатком сведений рассматривает его как приоритет 2. Две команды будут работать в разном темпе, задерживая реагирование на происшествие.

Обычно серьезность происшествия после полного разбора неизменна на протяжении всего жизненного цикла происшествия. С другой стороны, приоритет может меняться в течение всего происшествия. На ранних этапах сортировки и реализации критического исправления приоритет может быть равен 0. После установки критического исправления вы можете снизить приоритет до 1 или 2, так как команды разработчиков будут выполнять работу по очистке системы.

Определите рабочие параметры для привлечения IR-команды

После установки моделей серьезности и приоритетов определите рабочие параметры для описания повседневной деятельности команды реагирования на происшествие. Это очень важно, если команды выполняют обычную работу в дополнение к реагированию на инциденты или если нужно поддерживать связь с удаленными командами. Рабочие параметры обеспечивают своевременное реагирование на инциденты серьезности 0 и приоритета 0.

Рабочие параметры могут включать следующее.

- Ожидаемое время для первоначального реагирования на объявленные происшествия — например, в течение 5 минут, 30 минут, часа или следующего рабочего дня.
- Ожидаемое время для начальной сортировки и разработки плана реагирования и графика работы.
- Цели уровня обслуживания (SLO) (https://oreil.ly/MtL_o), чтобы члены команды понимали, когда реагирование на инциденты должно быть приоритетнее повседневной работы.

Есть много способов организации замены сотрудников, чтобы гарантировать, что работа по реагированию на происшествия правильно сбалансирована нагрузкой в команде или сбалансирована в соответствии с запланированной текущей работой. Подробное обсуждение см. в главах 11 (<https://landing.google.com/sre/sre-book/chapters/being-on-call/>) и 14 (<https://landing.google.com/sre/sre-book/chapters/managing-incidents/>)

книги Site Reliability Engineering, главах 8 (<https://landing.google.com/sre/workbook/chapters/on-call/>) и 9 (<https://landing.google.com/sre/workbook/chapters/incident-response/>) книги Site Reliability Workbook и главе 14 книги Лимончелли, Чалуп и Хоган (2014)¹.

Разработка планов реагирования

Принятие решений во время серьезных происшествий — сложная задача, так как респондентам нужно быстро обрабатывать ограниченную информацию. Хорошо продуманные планы реагирования помогут респондентам сократить количество ненужных шагов и обеспечить общий подход к реагированию на разные инциденты. У организации может быть политика реагирования на инциденты в масштабах всей компании, тогда как IR-команде нужно создать набор планов реагирования, охватывающих такие темы.

Отчет об инцидентах

Способ получения сообщений о происшествии IR-командой.

Расстановка приоритетов

Список участников IR-команды, обязанных ответить на первоначальный отчет и начать оценку инцидента.

Цели уровня обслуживания

Ссылка на SLO о скорости действия команды реагирования.

Роли и обязанности

Четкие определения ролей и обязанностей участников IR-команды.

Оказание помощи

Способ связи с командами инженеров и участниками, которым может понадобиться помощь в реагировании на происшествия.

Коммуникации

Эффективное общение во время инцидента не происходит без предварительного планирования. Определите, как выполнять каждое из следующих действий.

- Информирование руководства по происшествию (например, по электронной почте, текстовым сообщением или по телефону) и информация, которую нужно добавить в эти сообщения.

¹ Limoncelli T. A., Chalup S. R., Hogan C. J. The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems. Boston, MA: Addison-Wesley, 2014.

- Связь внутри организации во время происшествия, внутри команд реагирования и между ними (постоянные чаты, видеоконференции, электронная почта, безопасные каналы IRC, инструменты отслеживания ошибок и т. д.).
- Общение с такими внешними заинтересованными сторонами, как регулирующие или правоохранительные органы, когда ситуация требует этого. Важно сотрудничать с юридическим и другими отделами организации для планирования и поддержания этого общения. Подумайте о сохранении контактных данных и методов связи для каждого внешнего участника. Если IR-команда становится достаточно большой, вы можете автоматизировать механизмы уведомления.
- Сообщение об инцидентах взаимодействующим с клиентами командам поддержки.
- Общение между командами реагирования и руководством без предупреждения противника, если система связи недоступна или взломана.

РИСК БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: ЧТО ДЕЛАТЬ, ЕСЛИ ВАШ ПОЧТОВЫЙ СЕРВИС ИЛИ СЕРВИС МГНОВЕННОЙ СВЯЗИ ВЗЛОМАН

Многие команды реагирования на происшествия очень полагаются на электронную почту или мессенджеры при управлении реагированием на инцидент. Члены команды, распределенные по удаленным сервисам, могут просматривать коммуникационные потоки, чтобы определить текущее состояние реагирования. К сожалению, IR-команда не всегда может полагаться на единую систему связи. К примеру:

- Злоумышленник, взломавший сервер электронной почты или обмена мгновенными сообщениями либо контроллер домена, может подключиться к группе рассылки для координации реагирования. После он может следить за ответными действиями и обходить будущие попытки обнаружения. Противник может узнать состояние мер по смягчению последствий и перейти к системам, которые IR-команда объявила «чистыми».
- Если система связи работает в автономном режиме, у IR-команды может не быть возможности связаться с заинтересованными сторонами или удаленными группами. Это сильно замедлит распределение усилий по реагированию и увеличит время, нужное для восстановления работы организации.

Чтобы такое событие не нарушило вашу способность поддерживать IR-команду, убедитесь, что раздел коммуникации в IR-плане охватывает резервные методы связи. Мы обсудим тему операционной безопасности (operational security, OpSec) подробнее в подразделе «Операционная безопасность» в главе 17.

В каждом плане реагирования должны быть процедуры высокого уровня, чтобы любой обученный респондент мог действовать согласно изложенным в нем пунктам. План должен содержать ссылки на подробные инструкции, которые

могут использоваться для конкретных действий. Хорошая идея — изложить общий подход к реагированию на конкретные классы инцидентов. Когда речь идет о проблемах с сетевым подключением, в плане реагирования должна быть обширная сводка областей для анализа и шаги по устранению неполадок, которые нужно выполнить. План должен ссылаться на сборник инструкций о том, как получить доступ к соответствующим сетевым устройствам. Например, способы подключения к маршрутизатору или брандмауэру. В планах реагирования могут быть указаны критерии реагирования на происшествия, чтобы определить, когда уведомлять старшее руководство об инцидентах и когда работать с локализованными командами разработчиков.

Создание подробных инструкций

Инструкции дополняют планы реагирования и содержат четкое описание шагов, позволяющих респонденту выполнить конкретные задачи от начала до конца. В них может быть описано, как предоставить респондентам временный административный доступ в чрезвычайных ситуациях к определенным системам, как выводить и анализировать определенные журналы или как обеспечить отказоустойчивость системы и когда настраивать постепенную деградацию функций¹. Инструкции носят процедурный характер и должны часто пересматриваться и обновляться. Обычно они относятся к конкретным командам. Это подразумевает, что реагирование на любое бедствие может объединять несколько команд, работающих по своим процедурным инструкциям.

Убедитесь в наличии механизмов доступа и обновления

Ваша команда должна определить место для хранения документации, чтобы материалы были доступны во время происшествия. Бедствие может повлиять на доступ к документации, например, если серверы компании отключены. Поэтому убедитесь в наличии копий в месте, доступном во время чрезвычайной ситуации, и в их своевременном обновлении.

Системы исправляются, обновляются и перенастраиваются. Угрозы тоже могут меняться. Вы можете найти новые уязвимости, или появятся новые способы взлома. Время от времени проверяйте и обновляйте свои планы реагирования для уверенности в том, что они точны и отражают любые недавние операционные изменения или обновления настроек.

¹ Эти темы описаны в главе 22 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/addressing-cascading-failures/>).

Хорошее управление инцидентами требует частого и надежного управления информацией. Команда должна определить подходящую систему для отслеживания и сохранения информации о происшествии. Для команд, занимающихся инцидентами безопасности и конфиденциальности, может потребоваться система, которая жестко контролируется доступом по принципу служебной необходимости. Командам реагирования, имеющим дело с перебоями в обслуживании или поставках, пригодится система, которая будет широко использоваться в компании.

Предварительная подготовка людей и систем

Вы проанализировали риски, создали IR-команду и задокументировали соответствующие процедуры. Теперь вам нужно определить действия, которые можно выполнить до того, как произойдет бедствие. Убедитесь, что вы рассматриваете предварительную подготовку, относящуюся к каждой фазе жизненного цикла реагирования на происшествие. Обычно предварительная подготовка содержит настройку систем с определенным сроком хранения журналов, автоматически ответами и четко определенными процедурами для сотрудников. Понимая каждый из этих элементов, команда реагирования может устранить пробелы в охвате между источниками данных и автоматизированными и человеческими ответами. Планы реагирования и инструкции (как обсуждалось в предыдущем разделе) описывают многое из того, что нужно человеческому взаимодействию, но респондентам необходим и доступ к соответствующим инструментам и инфраструктуре.

Чтобы облегчить быстрое реагирование на происшествия, IR-команда должна заранее определить соответствующие уровни доступа для реагирования на инциденты и составить процедуры эскалации, чтобы процесс получения экстренного доступа не был медленным и запутанным. У команды IR должен быть доступ для чтения журналов, чтобы анализировать и реконструировать события, и доступ к инструментам, чтобы анализировать данные, отправлять отчеты и проводить экспертные расследования.

Настройка систем

Вы можете внести изменения в системы перед бедствием или инцидентом, чтобы уменьшить первоначальное время ответа IR-команды. Например:

- Встройте отказоустойчивость в локальные системы и создайте автоматическое переключение при происшествии. Дополнительно об этом см. главы 8 и 9.

- Разверните такие экспертные агенты, как GRR (<https://github.com/google/grr>) или EnCase (<https://oreil.ly/7-gVj>) Remote Agents, в сети с включенными журналами. Это поможет как при реагировании, так и при последующем экспертном анализе. Имейте в виду, что для журналов безопасности понадобится длительный период хранения, как описано в главе 15 (среднее значение по отрасли для обнаружения вторжений приблизительно 200 дней, и удаленные до обнаружения инцидента журналы не могут использоваться для его расследования). Но в некоторых странах, например в странах Европейского союза, особые требования к продолжительности хранения журналов. При составлении плана хранения проконсультируйтесь с адвокатом вашей организации.
- Если ваша организация выполняет резервное копирование на кассеты или другие носители, сохраните такой же набор аппаратного и программного обеспечения, использованного для их создания, чтобы своевременно восстанавливать резервные копии, если основная система резервного копирования недоступна. Периодически выполняйте тренировки по восстановлению, чтобы убедиться в правильности работы вашего оборудования, ПО и процедур. IR-команды должны определить процедуры, которые они будут использовать для работы с отдельными командами (командами электронной почты или сетевого резервного копирования) для тестирования, проверки и восстановления данных во время происшествий.
- Создайте несколько запасных путей для доступа и восстановления в чрезвычайных ситуациях. Сбой, влияющий на вашу производственную сеть, может быть трудно исправить без безопасных альтернативных путей доступа к управлению сетью. Так же, если вы обнаружите нарушение и не будете уверены в величине радиуса взлома корпоративных рабочих станций, восстановление в разы упростится с известным безопасным и физически отдельным набором систем, заслуживающих доверия.

Обучение

Члены IR-команды должны знать принципы серьезности/приоритетности, рабочую модель IR-команды, требования к времени отклика, расположение планов реагирования и инструкций. Узнайте больше о подходе Google к реагированию на инциденты в главе 9 книги Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/incident-response/>).

Но требования к обучению для реагирования на инциденты не входят в область действий IR-команды. В чрезвычайных ситуациях некоторые инженеры могут

реагировать не задумываясь и не осознавая последствий своих действий. Для снижения такого риска обучите специалистов, которые будут помогать IR-команде, выполнять разные роли IR и их обязанности. Мы используем систему под названием «Управление инцидентами в Google» (Incident Management at Google, IMAG), основанную на системе управления инцидентами (<https://oreil.ly/LwmI6>). Фреймворк IMAG назначает критически важные роли, такие как руководитель по реагированию на происшествия, руководители операционных и коммуникационных отделов.

Обучите своих сотрудников распознавать инциденты и сообщать о них. Они могут быть обнаружены сотрудниками, клиентами/пользователями, автоматическими системами оповещения или администраторами. Для каждой стороны должны быть отдельные, четкие каналы для сообщения о происшествии, а сотрудники компании должны быть обучены тому, как и когда передавать инцидент IR-команде¹. Это обучение должно поддерживать IR-политику организации.

Установите период времени, за который инженер может попытаться разобраться с инцидентом самостоятельно до оповещения руководства. Это время зависит от уровня риска, который организация готова принять. Вы можете начать с 15-минутного окна и изменить его, если понадобится.

Задайте критерии принятия решения до наступления худшего момента происшествия, чтобы респонденты выбирали самый логичный порядок действий, а не принимали интуитивное решение на лету. Первым респондентам часто нужно немедленно принимать решения о том, переводить ли скомпрометированную систему в автономный режим или какой метод изоляции использовать. Дополнительную информацию по этой теме см. в главе 17.

Научите специалистов тому, что для реагирования на инциденты может понадобиться разрешение пересекающихся приоритетов, кажущихся взаимоисключающими. Например, при необходимости поддерживать максимальное время безотказной работы и доступности, сохраняя артефакты для экспертного расследования. Обучите сотрудников оставлять заметки об их действиях по реагированию, чтобы в дальнейшем они могли отличить эти действия от артефактов, оставленных злоумышленником.

¹ См. главу 14 книги SRE (<https://landing.google.com/sre/sre-book/chapters/managing-incidents/>) и рабочие примеры для производственной среды в главе 9 книги Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/incident-response/>).

Процессы и процедуры

Установление набора процессов и процедур, которые нужно выполнить до начала бедствия, значительно сократит время ответа и когнитивную нагрузку на респондентов. Мы рекомендуем следующее.

- Определите методы быстрой закупки оборудования и ПО. Вам может понадобиться дополнительное оборудование или такие ресурсы, как серверы, программное обеспечение или топливо для генераторов, во время чрезвычайных ситуаций.
- Установите процессы утверждения контрактов на услуги аутсорсинга. Для небольших организаций это может означать определение таких внешних возможностей, как услуги экспертизы.
- Создайте политики и процедуры для сохранения доказательств и журналов во время происшествий в сфере безопасности и предотвращения перезаписи журналов. Дополнительную информацию см. в главе 15.



Тестирование систем и планов реагирования

После создания всех материалов для подготовки к происшествию, как описано в предыдущих разделах, оцените их эффективность и исправьте любые обнаруженные недостатки. Мы советуем проводить тестирование с нескольких точек зрения.

- Оцените автоматизированные системы, чтобы убедиться в их правильной работе.
- Протестируйте процессы для устранения любых пробелов в процедурах и инструментах, используемых респондентами и командами разработчиков.
- Обучите реагирующих на инциденты сотрудников, чтобы убедиться в наличии у них нужных навыков для реагирования на кризис.

Запускайте эти тесты периодически (как минимум ежегодно), чтобы убедиться, что системы, процедуры и способы реагирования надежны и применимы в случае реальной чрезвычайной ситуации.

Каждый компонент жизненно важен для возвращения пострадавшей от аварии системы в рабочее состояние. Даже если ваша ИР-команда высоко квалифицирована, без процедур или автоматизированных систем ее способность реагировать на бедствия значительно снизится. Если технические процедуры задокументи-

рованы, но недоступны или неприменимы, они никогда не будут реализованы. Тестирование устойчивости каждого уровня плана реагирования на бедствия снижает эти риски.

Для многих систем важно задокументировать технические процедуры для устранения угроз, регулярно проверять средства управления (ежеквартально или ежегодно), чтобы убедиться, что они все еще применяются, и предоставить список исправлений инженерам для исправления любых найденных недостатков. Организации, только начинающие ИР-планирование, могут изучить сертификаты аварийного восстановления и планирования непрерывности ведения бизнеса.

Аудит автоматизированных систем

Проведите аудит всех критических и зависимых систем, включая системы резервного копирования, регистрации, средства обновления ПО, генераторы оповещений и системы связи, чтобы убедиться в правильности их работы. Полный аудит должен показать следующее.

Система резервного копирования работает правильно

Резервные копии должны создаваться правильно, храниться в безопасном месте в течение соответствующего времени и иметь правильные разрешения. Периодически проводите упражнения по восстановлению и проверке данных. Это поможет вам проверить возможность извлечения и использования данных из резервных копий. Больше информации о подходе Google к обеспечению целостности данных в главе 26 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/data-integrity/>).

Журналы событий (обсуждаемые в предыдущей главе) хранятся правильно

Благодаря этим журналам респонденты смогут в хронологическом порядке выстроить события во время экспертных исследований. Храните журналы событий в течение периода времени, равного уровню риска организации и другим соображениям.

Критические уязвимости исправляются своевременно

Проверяйте автоматические и ручные процессы исправлений для уменьшения необходимости вмешательства человека и вероятности человеческих ошибок.

Оповещения генерируются правильно

Системы генерируют оповещения — сообщения электронной почты, обновления панели, текстовые сообщения и т. д. — соблюдая определенные

требования. Проверьте каждый критерий оповещения, чтобы убедиться, что он срабатывает правильно. Не забудьте учесть зависимости. Как повлияет на оповещения отключение SMTP-сервера во время сбоя сети?

Такие инструменты коммуникации, как клиентские чаты, электронная почта, сервисы конференц-связи и безопасный интернет-чат, работают как положено

Функциональные каналы связи важны для команд реагирования. Проверьте возможность отработки отказа этих инструментов и убедитесь, что они сохраняют нужные сообщения даже после отказа.

Проведение ненавязчивых настольных упражнений

Настольные упражнения — очень ценные инструменты для тестирования документированных процедур и оценки эффективности команд реагирования. Они могут стать отправной точкой для оценки сквозных реакций на инциденты. Еще они могут быть полезны, когда тестирование в реальных условиях неосуществимо. Например, во время землетрясения. Масштаб моделирования может колебаться от малого до большого, и моделирование проходит ненавязчиво: оно не переводит систему в автономный режим, производственные среды не затрагиваются.

Как в упражнении «Колесо неудачи» из главы 15 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/postmortem-culture/>), можно выполнить настольное упражнение, представив участникам сценарий инцидента с разными вариантами изменения сюжетной линии. Попросите участников описать их реакцию на сценарий и какие процедуры и протоколы они будут соблюдать. Так сотрудники смогут совершенствовать навыки принятия решений и получать конструктивную обратную связь. Это упражнения с открытой структурой, поэтому в них могут участвовать многие работники, в том числе:

- инженеры, следующие подробной документации по восстановлению функциональности поврежденной системы;
- высшее руководство, принимающее решения на уровне бизнеса в отношении операций;
- профессионалы по связям с общественностью, координирующие юристов по внешним коммуникациям;
- консультирующие юристы, участвующие в публичных коммуникациях.

Важнейший аспект этих упражнений — обучение респондентов и предоставление всем заинтересованным сторонам возможности протестировать процедуры и процессы принятия решений до наступления бедствия.

Вот некоторые ключевые особенности, которые нужно учитывать, выполняя настольные упражнения.

Вероятность

Сценарии настольных упражнений должны быть правдоподобными. Привлекательный сценарий мотивирует участников продолжать испытания, а не искать подозрительные моменты. Например, упражнение может установить, что пользователь подвергается фишинговой атаке, благодаря чему злоумышленник может использовать уязвимость на рабочей станции пользователя. Вы можете основать опорные точки — шаги перемещения злоумышленника по сети — на реалистичных атаках, известных уязвимостях и слабых местах.

Детали

Разработчик сценария настольного упражнения должен заранее изучить его, а ведущий должен хорошо разбираться в деталях события и типичных реакциях на сценарий. Для большей правдоподобности создатель настольного упражнения может добавлять артефакты, с которыми участники столкнутся во время реального инцидента. Это могут быть файлы журналов, отчеты от клиентов или пользователей и оповещения.

Точки принятия решений

Как и в любых интерактивных историях, в настольном упражнении должны быть точки принятия решений, помогающие развернуть сюжет. В обычном часовом настольном упражнении примерно 10–20 основных сюжетных решений для вовлечения участников в процесс принятия решений, влияющий на результат упражнения. Если участники решили перевести скомпрометированный почтовый сервер в автономный режим, то они не смогут отправлять уведомления по электронной почте в остальной части сценария.

Участники и ведущие

Сделайте настольные упражнения интерактивнее. При прохождении упражнения ведущий может реагировать на выполненные респондентами действия и команды. Вместо простого обсуждения решений выберите действия. Если в IR-инструкции от респондента требуется передать управление атакой для вымогательства члену команды криминалистов для расследования и члену команды сетевой безопасности для блокирования трафика враждебного веб-сайта, позвольте респонденту выполнить эти процедуры во время настольного упражнения. «Реагирование» развивает у респондента мышечную память. Ведущий должен заранее ознакомиться со сценарием, чтобы

импровизировать и подталкивать респондентов в верном направлении, если это необходимо. Цель здесь — дать возможность участникам активно участвовать в сценарии.

Результаты

Участники не должны чувствовать себя проигравшими. Успешное настольное упражнение должно заканчиваться обратной связью, сообщающей, что сработало хорошо, а что — нет. У участников и ведущего должна быть возможность давать конкретные рекомендации по улучшению для команды реагирования на инциденты. Если это уместно, участники могут рекомендовать изменения в системах и политиках для исправления выявленных недочетов. Чтобы участники выполняли эти рекомендации, создайте список задач для конкретных владельцев.

Тестирование реагирования в производственных средах

Несмотря на всю полезность подобных упражнений, нужно протестировать некоторые сценарии, векторы атак и уязвимости в реальной производственной среде. Эти тесты работают на пересечении безопасности и надежности, позволяя ИР-командам понимать ограничения в использовании, практиковаться с реальными параметрами и наблюдать за влиянием их ответов на производственную среду и время безотказной работы.

Тестирование одной системы/выявление неисправностей

Вместо тестирования системы целиком вы можете разбить большие системы на отдельные программные и/или аппаратные части. Тесты могут быть разных форм и могут содержать единственный локальный компонент или компонент с охватом всей организации. Что происходит, когда злоумышленник подключает запоминающее устройство USB к рабочей станции и пытается загрузить конфиденциальный контент? Отслеживают ли локальные журналы активность USB-порта? Достаточно ли структурированы и расширены журналы для быстрого реагирования команды безопасности?

Очень важно проводить тестирование одной системы с помощью внедрения неисправностей. Это позволяет вам проводить целевые тесты, не нарушая работу всей системы. Еще важнее, что фреймворки внедрения ошибок позволяют отдельным командам тестировать системы без учета зависимостей. Рассмотрим HTTP-прокси Envoy с открытым исходным кодом, который часто используется для балансировки нагрузки. В дополнение к этому прокси поддерживает HTTP-фильтр внедрения ошибок (https://oreil.ly/rDsp_), который можно использовать для

возврата произвольных ошибок в процентах трафика или задержки запросов на время. Используя этот тип внедрения сбоев, можно проверить, правильно ли система обрабатывает тайм-ауты и что они не приводят к непредсказуемому поведению в производственной среде.

При обнаружении необычного поведения в производственной среде хорошо проработанная структура внедрения сбоев обеспечит более структурированное расследование, где вы можете воспроизвести производственные проблемы контролируемым образом. Представьте сценарий, когда пользователи компании пытаются получить доступ к определенным ресурсам, инфраструктура проверяет все запросы на аутентификацию с помощью одного источника. Далее компания переходит к сервису, требующему многократных проверок подлинности для разных источников, чтобы получить такую же информацию. В итоге клиенты превышают настроенное время ожидания для этих вызовов функций. Обработка ошибок в компоненте кэширования библиотеки аутентификации ошибочно рассматривает эти тайм-ауты как постоянные (а не временные) сбои, вызывая множество других мелких сбоев во всей инфраструктуре. С помощью установленной системы реагирования на инциденты с внедрением ошибки команда реагирования может легко воспроизвести поведение — добавив задержку в некоторых вызовах, подтвердить подозрения и разработать исправление.

Тестирование персонала

Многие касаются технической стороны системы, но они должны учитывать и ошибки персонала. Что происходит, когда сотрудник недоступен или не работает? Часто ИР-команды полагаются на людей с обширными знаниями организации, а не следуют установленным процессам. Если основные сотрудники или менеджеры недоступны во время реагирования, как будет действовать остальная ИР-команда?

Многокомпонентное тестирование

Работа с распределенными системами означает, что может произойти сбой любого количества зависимых систем или их компонентов. Планируйте многокомпонентные сбои и создавайте процедуры реагирования на инциденты. Рассмотрим систему, зависящую от нескольких компонентов, каждый из которых тестировался индивидуально. При выходе из строя двух или более компонентов с какими частями реагирования на происшествия нужно обращаться по-разному?

Мысленного упражнения может не хватить для проверки каждой зависимости. При рассмотрении сбоя сервиса в контексте безопасности учитывайте проблемы

безопасности в дополнение к сценариям сбоя. Учитывает ли система существующие ACL во время обработки отказа? Какие меры безопасности обеспечивают такое поведение? Закрывается ли зависимый сервис при тестировании сервиса авторизации? Подробнее об этой системе см. в главе 5.

Общесистемные сбои и отработки отказа

Рассмотрим, что происходит при сбое всей системы. Во многих организациях работают первичные и вторичные (или аварийные) центры обработки данных. До переключения на вторичный центр при отказе вы не можете быть уверены, что ваша стратегия восстановления после сбоя защитит ваш бизнес и безопасность. Google регулярно переключает электропитание во всех зданиях центра обработки данных, чтобы проверить, не вызывает ли сбой видимое для пользователя воздействие. Это гарантирует, что службы способны работать без привязки к определенному местоположению центра обработки данных и что выполняющие эту работу технические специалисты обладают практическим опытом в управлении процедурами включения/выключения питания.

Если ваш сервис работает в облачной инфраструктуре другого провайдера, подумайте, что произойдет с ним в случае сбоя во всей зоне доступности или регионе.

Тестирование красных команд

Помимо объявленного тестирования Google практикует упражнения по готовности к бедствиям, известные как упражнения *красных команд*. Это наступательное тестирование, проводимое организацией по обеспечению информационной безопасности. Подобно упражнениям DiRT (см. подраздел «Тестирование DiRT при экстренном доступе» далее в этой главе), они имитируют реальные атаки для проверки и улучшения возможностей обнаружения, реагирования и демонстрации влияния проблем безопасности на бизнес-логику.

Красная команда не уведомляет заранее отвечающих за инциденты сотрудников, а только высшее руководство. Красные команды знакомы с инфраструктурой Google, поэтому их тесты производительнее, чем стандартные тесты на проникновение в сеть. Из-за того, что эти упражнения выполняются внутри, они помогают найти баланс между полностью внешними (когда злоумышленник находится за пределами Google) и внутренними атаками (инсайдерский риск). Упражнения красных команд дополняют проверки безопасности, тестируя сквозную безопасность и поведение человека с помощью фишинга и социальной инженерии. Для более глубокого изучения действий красных команд см. подраздел «Специальные команды: синие и красные» в главе 20.

Оценка ответов

Реагируя на настоящие инциденты и на сценарии тестирования, создайте эффективный цикл обратной связи, чтобы не попадать каждый раз в одинаковые ситуации. Настоящие происшествия должны запускать постмортемы с конкретными элементами действий. Постмортемы и соответствующие элементы действий можно создавать и для тестирования. Хотя тестирование увлекательно и дает отличный опыт для обучения, для практики нужна определенная строгость. Отслеживайте выполнение теста, критически оценивайте его влияние и то, как люди в вашей организации реагируют на него. Выполнение упражнения без отработки извлеченных из него уроков — не более чем развлечение.

Оценивая реагирование организации на инциденты и тесты, примите во внимание следующие рекомендации.

- Измерьте ответы. Оценщики должны быть способны определить, что работает хорошо, а что нет. Измерьте количество времени, нужное для реализации каждого этапа ответа, чтобы внести коррективы.
- Пишите безобвинительные постмортемы и сосредоточьтесь на том, как можно улучшить системы, процедуры и процессы¹.
- Предусмотрите обратную связь для улучшения существующих или разработки новых планов по мере необходимости.
- Соберите артефакты и проведите их через системы обнаружения сигнала. Убедитесь, что вы устраняете любые найденные пробелы.
- Обязательно сохраняйте соответствующие журналы и другие материалы, особенно при проведении учений по безопасности. Чтобы вы могли проводить экспертный анализ и устранять пробелы.
- Оцените даже «неудачные» тесты. Что сработало, а что нужно улучшить?²
- Как описывается в подразделе «Специальные команды: синие и красные» в главе 20, создайте цветные команды, чтобы ваша организация действовала в соответствии с полученными уроками. Может пригодиться гибридная фиолетовая команда, чтобы убедиться, что синяя команда своевременно устраняет уязвимости, эксплуатируемые красной командой. Это не позволит злоумышленникам повторно использовать одни уязвимости. Вы можете представлять фиолетовые команды как регрессионное тестирование на уязвимости.

¹ См. главу 15 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/postmortem-culture/>).

² См. главу 13 этой книги (<https://landing.google.com/sre/sre-book/chapters/emergency-response/>).

Примеры из опыта Google

Для большей конкретики мы приведем несколько примеров из реальной жизни.

Тест с глобальным воздействием

В 2019 году Google провел тест реагирования на сильное землетрясение в районе залива Сан-Франциско. Сценарий содержал компоненты для моделирования воздействия на физический объект и его коммуникации, транспортную инфраструктуру, сетевые компоненты, коммунальные услуги и электроэнергию, связь, бизнес-операции и принятие управленческих решений. Нашей целью было проверить реакцию Google на крупный сбой и влияние на глобальные операции. Мы проверили следующее.

- Как Google может оперативно помочь людям, пострадавшим во время землетрясения и многочисленных повторных подземных толчков?
- Как Google предоставит помощь населению?
- Как сотрудники будут передавать информацию руководству Google?
- Как Google будет распространять информацию среди сотрудников в случае нарушения связи? Например, из-за неисправной сотовой сети или из-за нарушения LAN/MAN/WAN.
- Кто будет доступен для реагирования на месте, если сотрудники столкнутся с конфликтом интересов, например, если им потребуется заботиться о своих родственниках и жилищах?
- Как непроходимые второстепенные дороги повлияют на ситуацию в регионе? Как пробки повлияют на основные дороги?
- Как Google будет помогать сотрудникам, подрядчикам и посетителям, запертым в кампусах Google?
- Как Google может оценить ущерб, нанесенный своим зданиям, — сломанные трубы, проблемы с канализацией, разбитое стекло, потерю питания и разрыв сетевых подключений?
- Если локально затронутые команды не могут передать полномочия и ответственность, как SR-инженеры и команды разработчиков за пределами данной географической области могут взять под контроль системы?
- Как мы могли бы обеспечить контроль над ситуацией за пределами затронутой области для продолжения бизнес-операций и принятия деловых решений?

Тестирование DiRT при экстренном доступе

Иногда мы можем одновременно проверить отказоустойчивость операций надежности и безопасности. На одном из наших ежегодных учений по восстановлению после стихийного бедствия (DiRT)¹ SR-инженеры проверили процедуру и функциональность контрольных учетных данных². Могли ли они получить экстренный доступ к корпоративным и производственным сетям при недоступности стандартных служб ACL? Для добавления уровня тестирования безопасности команда DiRT подключилась к команде обнаружения сигналов. Когда SR-инженеры задействовали механизмы аварийного доступа, команда обнаружения смогла подтвердить, что сработало правильное предупреждение и запрос на доступ был проверен.

Отраслевые уязвимости

В 2018 году компания Google получила предварительное уведомление о двух уязвимостях в ядре Linux, лежащих в основе большей части нашей производственной инфраструктуры. Посылая специально созданные фрагменты IP и сегменты TCP, SegmentSmack (CVE-2018-5390 (<https://oreil.ly/MMhA7>)) или FragmentSmack (CVE-2018-5391 (<https://oreil.ly/cwl3J>)) могут заставить сервер выполнять дорогостоящие операции. Эта уязвимость использует большое количество процессорного и физического времени, тем самым позволяя злоумышленнику увеличить масштаб по сравнению с обычной атакой типа «отказ в обслуживании». Сервис, который обычно может справиться с атакой в 1 млн пакетов/с, может упасть при приблизительно 50 тыс. пакетов/с, что снижает устойчивость в 20 раз.

Планирование и подготовка к аварийным ситуациям позволили нам снизить этот риск в двух измерениях: технических аспектах и аспектах управления происшествиями. Что касается управления происшествиями, надвигающаяся катастрофа была настолько значимой, что Google назначил команду менеджеров по инцидентам для работы над проблемой на полный рабочий день. Команде нужно было определить затронутые системы, включая образы встроенного ПО поставщика, и принять комплексный план по снижению риска.

В техническом плане SRE уже внедрили меры защиты в глубину для ядра Linux. Патч времени выполнения, или *ksplICE*, которое использует таблицы перенаправления функций, чтобы сделать ненужной перезагрузку нового ядра, может

¹ См.: *Krishnan K.* Weathering the Unexpected (https://oreil.ly/cn_il) и на USENIX LISA15 10 Years of Crashing Google (<https://oreil.ly/ZRZAI>).

² Механизм аварийного доступа может обходить политики, что позволяет инженерам быстро устранять простои. См. подраздел «Механизм аварийного доступа» в главе 5.

решить многие проблемы безопасности. Google поддерживает и дисциплину развертывания ядра. Мы регулярно вводим новые ядра на весь парк машин с целевым сроком менее 30 дней. И у нас есть четко определенные механизмы для увеличения скорости развертывания этой стандартной рабочей процедуры при необходимости¹.

Если бы мы не смогли исправить уязвимость с помощью `ksplice`, можно было бы выполнить аварийное скоростное развертывание. Однако в этом случае нужно обратиться к двум затронутым функциям ядра — `tcp_collapse_ofo_queue` и `tcp_prune_ofo_queue` — с использованием функции сращивания ядра. SR-инженеры смогли применить `ksplice` к производственным системам без негативного влияния на производственную среду. Процедура развертывания уже была протестирована и одобрена, поэтому SR-инженеры быстро получили разрешение со стороны руководства на применение исправления во время замораживания кода.

Резюме

Когда вы размышляете о том, как развернуть тесты и планы аварийного восстановления с нуля, сам объем возможных подходов может показаться ошеломляющим. Однако вы можете применять концепции и рекомендации из этой главы даже в небольших масштабах.

Для начала определите важнейшую систему или часть критических данных, а затем то, как вы будете реагировать на разные бедствия, влияющие на нее. Определите, как долго вы можете работать без сервиса и на какое количество людей или других систем это влияет.

Сделав этот первый важный шаг, вы можете постепенно расширить охват, создав надежную стратегию готовности к бедствиям. Начните с выявления и предотвращения искр, разжигающих огонь, — и вы не заметите, как пройдете путь до реагирования на неизбежное пламя.

¹ В главе 9 обсуждаются дополнительные подходы к проектированию, которые готовят организацию к быстрому реагированию на инциденты.

Антикризисное управление

*Мэтт Линтон, Ник Сода
и Гари О’Коннор*

После запуска систем нужно поддерживать их в рабочем состоянии даже во время атак. Надежность систем — мера того, насколько хорошо ваша организация может противостоять кризису безопасности, а это напрямую влияет на удовлетворенность ваших пользователей.

В начале этой главы мы разъясняем, как распознать кризис, после чего следует подробный план того, как взять на себя командование и поддерживать контроль над инцидентом. Эта тема содержит глубокое погружение в операционную безопасность и форензику (компьютерную криминалистику). Коммуникации могут быть важной, но часто упускаемой из виду частью антикризисного управления. Мы проведем вас через некоторые ловушки, связанные с коммуникацией, и покажем, как их избежать с помощью наших примеров и шаблонов. В конце мы рассмотрим примерный сценарий кризиса, чтобы показать, как части реагирования на инциденты сочетаются друг с другом.

Реагирование на происшествия значимо для происшествий и в сфере надежности, и в сфере безопасности. Глава 14 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/managing-incidents/>) и глава 9 книги Site Reliability Engineering (<https://landing.google.com/sre/workbook/chapters/incident-response/>) исследуют реагирование на инциденты, связанные с перебоями в надежности. Мы используем ту же методологию — систему управления инцидентами в Google (IMAG) — для реагирования на инциденты безопасности.

Нельзя избежать происшествий в области безопасности. Распространенный принцип в отрасли гласит: «Есть только два типа компаний: те, которые знают, что они были скомпрометированы, и те, которые не знают». Последствия взлома зависят от уровня подготовки организации, а также от ее реакции. Для достижения зрелого уровня безопасности организации нужно внедрить и применять на практике возможности реагирования на инциденты (IR), как обсуждалось ранее.

Помимо привычного давления, связанного с проверками того, что неавторизованные стороны не могут получить доступ к системам и что данные хранятся там, где положено, ИР-команды сегодня сталкиваются с новыми сложными проблемами. Отрасль безопасности стремится к большей прозрачности¹ и нуждается в большей открытости для пользователей, поэтому такое ожидание — уникальная проблема для любой ИР-команды, привыкшей работать вдали от общественного внимания. Кроме того, такие нормативные акты, как Общий регламент по защите данных (GDPR, General Data Protection Regulation) (<https://gdpr.eu/>) и контракты на обслуживание с заботящимися о безопасности заказчиками, постоянно расширяют границы того, как быстро должны начинаться, продвигаться и завершаться расследования. Сегодня клиенты нередко просят уведомить о потенциальной проблеме безопасности в течение суток (или меньше) с момента первоначального обнаружения.

Уведомление об инцидентах стало основной особенностью области безопасности, наряду с такими технологическими достижениями, как простое и повсеместное использование облачных вычислений, широкое внедрение политики использования сотрудниками своих устройств (bring your own device, BYOD) на рабочем месте и интернета вещей (Internet of Things, IoT). Эти достижения создали новые проблемы для разработчиков и сотрудников службы безопасности. Например, ограниченный контроль над всеми активами организации и их видимость.

Это кризис или нет?

Не каждый инцидент можно назвать кризисом. Если ваша организация в хорошей форме, только немногие инциденты могут привести к кризису. Как только происходит эскалация, первым шагом ответчика в ее оценке является *сортировка*. Это использование имеющихся знаний и информации для обдуманных и обоснованных предположений о серьезности и возможных последствиях инцидента.

Сортировка — это хорошо зарекомендовавший себя навык в сообществе скорой медицинской помощи. Врач скорой медицинской помощи, прибывающий на место ДТП, сначала должен удостовериться в отсутствии прямого риска дальнейшего травмирования кого-либо на месте происшествия, а после проводит сортировку. Например, если автобус столкнулся с автомобилем, мы уже можем логически извлечь из этого некую информацию. Скорее всего, люди в машине будут серьезно травмированы, потому что столкновение с тяжелым автобусом

¹ См. отчет о прозрачности Google (<https://oreil.ly/vQNR3>).

может серьезно навредить. В автобусе помещается много пассажиров, поэтому у них могут быть многочисленные травмы. Вероятность присутствия опасных химических веществ мала, потому что их обычно не перевозят на автобусе. В течение первой минуты после прибытия врач скорой помощи знает, что им нужно вызвать еще несколько машин скорой помощи, возможно, подготовить отделение интенсивной терапии и вызвать спасателей, чтобы освободить всех пассажиров, оказавшихся в ловушке, из автомобиля меньшего размера. Специалисты по устранению опасных веществ не понадобятся.

Ваша команда реагирования на инциденты в сфере безопасности должна использовать эти же методы оценки для сортировки инцидентов по мере их поступления. Сначала они должны оценить потенциальную серьезность атаки.

Сортировка инцидента

Во время сортировки назначенный для расследования инженер должен собрать основные факты, чтобы определить, является ли проблема чем-то из следующего:

- ошибка (то есть ложное срабатывание);
- легко исправляемая проблема (возможно, случайный взлом);
- сложная и потенциально опасная проблема (например, намеренный взлом).

Специалисты должны уметь сортировать предсказуемые проблемы, ошибки и другие несложные неполадки при помощи заранее определенных процессов. Для более крупных и сложных проблем вроде целевых атак понадобится организованное управляемое реагирование.

У каждой команды должны быть заранее спланированные критерии для определения, что представляет собой инцидент. В идеале определите, какие виды рисков являются серьезными по сравнению с приемлемыми в их среде, еще до самого происшествия. Реакция будет зависеть от типа среды, в которой произошел инцидент, состояния предварительного контроля организации и сложности ее программы реагирования. Рассмотрим, как три организации могут отреагировать на одинаковую угрозу — атаку программ-вымогателей.

- В *организации 1* налажен грамотный процесс обеспечения безопасности и многоуровневая защита, включая ограничение, разрешающее выполнение только криптографически подписанного и утвержденного ПО. Вероятность того, что хорошо известная программа-вымогатель может заразить компьютер или распространиться по всей сети, очень мала. В противном случае система обнаружения выдает предупреждение, и кто-то начинает исследовать происшествие. Благодаря зрелым процессам и многоуровневой

защите один специалист может справиться с проблемой. Он может проверить, не было ли никаких подозрительных действий после попытки выполнения вредоносного ПО, и решить проблему с помощью стандартного процесса. Для такого сценария не нужно прилагать усилия для реагирования как на кризисные ситуации.

- В *организации 2* есть отдел продаж, который размещает демо заказчиков в облачной среде, где люди, желающие узнать больше о ПО организации, могут устанавливать свои тестовые экземпляры и управлять ими. Команда безопасности замечает, что эти пользователи часто совершают ошибки в настройках безопасности и это приводит к компрометациям системы. Чтобы эти уязвимости не требовали ручного вмешательства ответственного сотрудника, команда безопасности создает механизм для автоматического удаления и замены взломанных тестовых экземпляров в облаке. В таком случае червь-вымогатель тоже не потребует большого внимания криминалистов или команды реагирования на инциденты. Хотя организация 2 не предотвращает запуск программы-вымогателя (в отличие от организации 1), автоматизированные инструменты смягчения последствий могут сдержать риск.
- У *организации 3* меньше уровней защиты и ограниченная видимость того, скомпрометированы ли ее системы. Она подвергается гораздо большему риску распространения программы-вымогателя по своей сети и не способна быстро отреагировать. Это может коснуться критических для бизнеса систем, если вымогатель распространится по системе, и организация будет серьезно затронута. Исправление ситуации потребует значительных технических ресурсов для восстановления скомпрометированных сетей и систем. Этот червь очень опасен для организации 3.

Все три организации реагируют на одинаковый источник риска (атака с целью вымогательства), но получаемый урон зависит от различия в их защите и уровне зрелости процессов. Организации 1 потребуется просто инициировать реагирование, основанное на существующих инструкциях, а организация 3 может столкнуться с кризисом, требующим скоординированного управления инцидентами. Чем выше вероятность угрозы для организации, тем больше и вероятность того, что это потребует организованного реагирования и вовлечения большого количества участников.

Ваша команда может провести базовую оценку, чтобы определить, требует уязвимость стандартного подхода на основе инструкций или кризисного управления. Задайте себе следующие вопросы.

- Какие данные, которые могут быть доступны кому-либо в этой системе, вы храните? Какова ценность или критичность этих данных?

- Какие доверительные отношения у потенциально скомпрометированной системы с другими?
- Есть ли компенсирующие средства контроля, которые злоумышленник должен будет взломать (и которые кажутся нетронутыми), чтобы укрепиться в системе?
- Похожа ли атака на действия обычного вредоносного ПО (например, Adware) или она выглядит более продвинутой или целенаправленной (например, фишинговая кампания, созданная с учетом особенностей вашей организации)?

Продумайте все факторы для вашей организации и определите самый вероятный уровень организационного риска.

Компрометации и ошибки

IR-командам поручено реагировать на возможные вторжения и компрометации. Однако как насчет программных и аппаратных ошибок, также известных как уязвимости в безопасности? Считаете ли вы недавно обнаруженную уязвимость в системе взломом, который еще только предстоит обнаружить?

Ошибки в ПО неизбежны, и вы можете их планировать (как описано в главе 8). Хорошая защита устраняет или ограничивает возможные негативные последствия уязвимостей до их возникновения¹. Если вы тщательно планируете и внедряете защиту в глубину с дополнительными уровнями безопасности, вам не нужно обрабатывать исправления уязвимостей так же, как инциденты. Однако может оказаться целесообразным управлять сложными уязвимостями или уязвимостями с большим воздействием с помощью процессов реагирования на инциденты, которые могут помочь вам организовать и быстро реагировать.

В Google мы обычно рассматриваем уязвимости, связанные с большим риском, как инциденты. Даже если ошибка не используется активно, особенно серьезная ошибка все равно может представлять чрезвычайный риск. Если вы участвуете в устранении уязвимости до ее публичного раскрытия (эти действия часто называют *скоординированным раскрытием уязвимости* или CVD — от coordinated vulnerability disclosure), проблемы операционной безопасности и конфиденциальности могут требовать повышенного реагирования. В качестве альтернативы, если вы спешите исправить системы после публичного раскрытия, защита

¹ В одном из наших обзоров дизайна в Google инженер предположил, что «есть два типа разработчиков ПО: те, кто помещает Ghostscript в песочницу, и те, у кого Ghostscript должен находиться в песочнице».

систем, которые имеют сложные взаимозависимости, может потребовать срочных усилий, и развертывание исправлений может быть сложным и отнимать много времени.

Некоторые примеры особо опасных уязвимостей содержат Spectre и Meltdown (CVE-2017-5715 и 5753), glibc (CVE-2015-0235), Stagefright (CVE-2015-1538), Shellshock (CVE-2014-6271) и Heartbleed (CVE-2014-0160).

СКООРДИНИРОВАННОЕ РАСКРЫТИЕ УЯЗВИМОСТИ

Есть много толкований для CVD. Новый стандарт ISO 29147:2018 (<https://oreil.ly/GBGam>) дает некоторые рекомендации. В Google мы обычно определяем CVD как процесс, в котором команда должна поддерживать баланс между временем, нужным поставщику для выпуска исправлений безопасности, потребностями и пожеланиями человека, обнаружившего ошибку, и нуждами базы пользователей и заказчиков.

Управление инцидентом

Теперь, когда мы обсудили процесс сортировки и оценки риска, в следующих трех разделах предполагается, что мы столкнулись с «большим» риском. Вы определили или подозреваете целенаправленный взлом, и поэтому вам нужно выполнить полноценное реагирование на инцидент.

Первый шаг: не паникуйте!

У многих респондентов оповещения о серьезном происшествии вызывают растущее чувство паники и выброс адреналина. На базовых этапах обучения специалистов аварийно-спасательных служб в пожарной, спасательной и медицинской областях предупреждают о том, что не следует немедленно бежать на место происшествия. Риск не только усугубляет проблему, увеличивая возможность несчастного случая на месте происшествия, но и вызывает панику у респондента и общественности. Поэтому во время инцидента с безопасностью лишние несколько секунд, выигранные в спешке, быстро затмеваются последствиями провала планирования.

Хотя команды SRE и безопасности в Google выполняют управление инцидентами похожим образом, есть разница между кризисным реагированием на инцидент безопасности и на инцидент надежности, такой как выход из строя. Когда происходит сбой в работе, дежурный SR-инженер готовится приступить

к действию. Его цель — быстро найти ошибки и исправить их для восстановления работоспособности системы. Главное, что система не осознает свои действия и не будет противостоять исправлению.

При возможном взломе атакующий может внимательно следить за действиями, предпринимаемыми целевой организацией, и мешать респондентам во время их попыток исправления. Попытка исправить систему без предварительного завершения полного расследования может привести к катастрофическим последствиям. Типичная работа SR-инженера не сопряжена с таким риском, обычная его реакция — сначала исправить систему, а после задокументировать полученную из сбоя информацию. Например, если инженер отправляет список изменений, нарушающий работу производственной среды, SR-специалист может предпринять немедленные действия для отмены CL, что устранил проблему. Сразу после этого SR-инженер начнет расследование произошедшего. Вместо этого для обеспечения безопасности нужно, чтобы команда завершила полное расследование того, что произошло, до попыток что-то исправить.

Ваша первая задача как респондента — контролировать свои эмоции. Потратьте первые пять минут на то, чтобы несколько раз глубоко вдохнуть и выдохнуть. Позвольте панике пройти, напомните себе, что вам нужен план, и начните продумывать следующие шаги. Несмотря на сильное стремление немедленно реагировать, не спешите: на практике постмортемы редко сообщают о том, что меры безопасности были бы более эффективными, если бы сотрудники среагировали на пять минут раньше. Первоначальное дополнительное планирование будет ценнее.

Начинаем реагировать

В Google, если инженер решает, что проблема, с которой он сталкивается, является происшествием, мы следуем стандартному процессу. Этот процесс, который называется «Управление инцидентами в Google (IMAG)», полностью описан в главе 9 книги *Site Reliability Workbook* (<https://landing.google.com/sre/workbook/chapters/incident-response/>), вместе с примерами сбоев и событий, когда мы применили этот протокол. В этом разделе описывается, как вы можете использовать IMAG в качестве стандартной платформы для управления происшествиями.

Во-первых, чтобы быстро освежить память: как упоминалось в предыдущей главе, IMAG основан на формальном процессе, называемом системой управления инцидентами (ICS) (<https://oreil.ly/4cLxY>), который используется пожарными, спасательными и полицейскими органами по всему миру. Как и ICS, IMAG — гибкий фреймворк реагирования, простая программа для управления

небольшими инцидентами. Однако он может расширяться для охвата крупных и широкомасштабных проблем. Нашей программе IMAG поручено формализовать процессы для обеспечения максимального успеха в трех ключевых областях обработки инцидентов: командование, контроль и коммуникации.

Первый шаг в управлении инцидентом — принятие *командования*. В IMAG мы делаем это, публикуя объявление: «Наша команда объявляет об инциденте с участием X, и я являюсь командиром инцидента (incident commander, IC)». Прямое указание “это инцидент” может показаться простым и, возможно, ненужным. Однако такое явное указание во избежание недоразумений — основной принцип командования и связи. Начинать свой инцидент с заявления — первый шаг в согласовании ожиданий всех сотрудников. Инциденты сложны и необычны, связаны с большим напряжением и происходят на высокой скорости. Вовлеченные в них люди должны сосредоточиться. Исполнители должны быть уведомлены о том, что команды могут игнорировать или обходить обычные процессы до полного разрешения инцидента.

После того, как респондент принимает командование и становится IC, его задача — *сохранить контроль* над инцидентом. IC направляет ответные меры и обеспечивает постоянное продвижение людей к заданным целям, чтобы хаос и неопределенность вокруг кризиса не сбивали команды с пути. Для сохранения контроля IC и их руководители должны постоянно поддерживать отличную *связь* со всеми участниками.

Google использует IMAG как фреймворк общего назначения для реагирования на все виды происшествий. Все инженеры (в идеале) обучены одинаковому набору основ и тому, как их использовать для масштабирования и профессионального управления реагированием. Фокус SR-инженеров и команд безопасности может различаться. Однако благодаря одинаковой структуре реагирования обеим командам становится проще взаимодействовать в условиях стрессовой ситуации, когда работа с незнакомыми командами может быть наиболее сложной.

Создаем команды по реагированию на инцидент

В соответствии с моделью IMAG после объявления инцидента объявляющее лицо либо становится командиром инцидента, либо выбирает IC из других доступных сотрудников. Независимо от того, какой маршрут вы выберете, сделайте это назначение явным, во избежание недопонимания среди респондентов. Лицо, назначенное IC, должно явно подтвердить, что оно принимает назначение.

Далее IC оценит, какие действия нужно предпринять немедленно и кто может взять на себя определенные роли. Скорее всего, для начала расследования вам

понадобятся несколько квалифицированных инженеров. В крупных организациях есть специальная команда по безопасности; в очень крупных организациях может даже быть специальная команда реагирования на инциденты. В небольшой организации может быть один выделенный сотрудник службы безопасности или кто-то, кто занимается безопасностью неполный рабочий день наряду с другими оперативными обязанностями.

Независимо от размера или состава организации ИС должен найти сотрудников, хорошо знающих потенциально затронутые системы, и объединить этих людей в команду реагирования на происшествие. Если инцидент расширяется и требует больше персонала, можно назначить нескольких руководителей, чтобы возглавить определенные аспекты расследования. Почти в каждом инциденте нужен *оперативный руководитель* (operations lead, OL) — тактический партнер и соратник ИС. Пока ИС сосредоточен на определении стратегических целей для достижения прогресса в реагировании на инциденты, OL фокусируется на достижении этих целей и определении способов их достижения. Большая часть технического персонала, выполняющего исследование, исправления и обновления систем и т. д., должна отчитываться перед OL.

Рассмотрим другие ведущие роли, которые могут пригодиться.

Посредник с руководством

Вам может понадобиться человек для принятия трудных решений на месте. Кто из вашей организации может при необходимости отключить приносящий доход сервис? Кто может решить отправить сотрудников домой или отозвать полномочия других инженеров?

Юридический советник

Ваш инцидент может вызвать юридические вопросы, для разрешения которых вам понадобится помощь. Ваши сотрудники ожидают большей конфиденциальности? Например, если вы думаете, что кто-то из разработчиков загрузил вредоносное ПО через браузер, нужны ли вам дополнительные разрешения для просмотра истории его браузера? А если вы подозреваете, что он загрузил вредоносное ПО через свой личный профиль в браузере?

Менеджер по коммуникационным стратегиям

В зависимости от характера инцидента вам может потребоваться поддержание связи с вашими клиентами, регулирующими органами и т. д. Специалист, обладающий навыками общения, станет важным дополнением к вашей команде реагирования.



Операционная безопасность

В антикризисном управлении *операционная безопасность* (operational security, OpSec) относится к практике сохранения секретности вашей деятельности по реагированию. Независимо от того, работаете вы над предполагаемым взломом, расследуете злоупотребления со стороны инсайдера или изучаете опасную уязвимость, существование которой может привести к широкому использованию — у вас будет информация, которую вам нужно хранить в секрете, хотя бы в течение ограниченного времени. Мы советуем составить план OpSec сегодня — еще до происшествия, — чтобы не пришлось торопиться с этим планом в последнюю минуту. Утерянные секретные данные невозможно вернуть.

ИС ответственен за обеспечение того, чтобы правила конфиденциальности были установлены, сообщены и соблюдены. Каждый участник команды, которого попросили поработать над расследованием, должен быть проинформирован о том, чего стоит ожидать. У вас могут быть определенные правила для обработки данных или ожидания относительно того, какие каналы связи использовать. Например, если вы подозреваете, что ваш почтовый сервер в пределах нарушения, вы можете запретить сотрудникам отправлять друг другу электронные письма о нарушении, чтобы злоумышленник не перехватил эти разговоры.

ИС должен указать конкретные рекомендации для каждого члена команды реагирования на инциденты. Каждый должен рассмотреть и подтвердить эти рекомендации до начала работы над инцидентом. Если вы не будете четко сообщать правила конфиденциальности всем затронутым сторонам, это приведет к утечке информации или преждевременному разглашению.

В дополнение к защите ответных действий от злоумышленника хороший план OpSec описывает, как реагировать без дальнейшего раскрытия организации. Представьте себе злоумышленника, скомпрометировавшего одну из учетных записей вашего сотрудника, который пытается украсть другие пароли из памяти на сервере. Он будет в восторге, если системный администратор во время расследования зайдет на взломанное устройство с учетными данными администратора. Планируйте заранее получение доступа к данным и компьютерам, чтобы не дать злоумышленнику дополнительных возможностей. Один из способов добиться этого — заранее развернуть удаленные агенты форензики на всех ваших системах. Эти программные пакеты поддерживают путь доступа для авторизованных респондентов из вашей компании для получения артефактов форензики без риска своими учетными записями и без прямого входа в систему.

Если злоумышленник поймет, что вы обнаружили его атаку, последствия могут быть серьезными. Решительный злоумышленник, желающий остаться вне вашего расследования, может затаиться. Так вы не сможете оценить

степень взлома и пропустите точки опоры противника. Злоумышленник, который выполнил свою задачу и не хочет таиться, может отреагировать на ваше открытие, уничтожив как можно больше данных вашей организации на пути отступления!

Некоторые распространенные ошибки OpSec.

- Информирование или документирование инцидента на носителе (например, по электронной почте), позволяющем злоумышленнику отслеживать ответные действия.
- Вход на скомпрометированные серверы. Это предоставляет злоумышленнику полезные учетные данные для аутентификации.
- Подключение и взаимодействие с серверами «командования и контроля» злоумышленника. Например, не пытайтесь получить доступ к вредоносному ПО злоумышленника, загрузив его с компьютера, который вы используете для расследования. Ваши действия будут выделены в журналах злоумышленника как необычные и могут предупредить его о расследовании. Не сканируйте порты или не ищите домены компьютеров злоумышленника (распространенная ошибка новичков).
- Блокировка учетных записей или изменение паролей затронутых пользователей до завершения расследования.
- Отключение систем до определения полного масштаба атаки.
- Предоставление доступа к вашим рабочим станциям для анализа с теми же учетными данными, которые могли быть украдены злоумышленником.

Воспользуйтесь следующими рекомендациями для своего OpSec-реагирования.

- По возможности проводите встречи и обсуждения лично. Если нужно использовать чат или электронную почту, подключите новые устройства и инфраструктуру. Например, организация, столкнувшаяся с неизвестной степенью компрометации, может создать новую временную среду в облаке и развернуть компьютеры, которые отличаются от ее обычного парка (например, Chromebook), для респондентов. В идеале эта тактика обеспечивает чистую среду для чата, документирования и связи вне поля зрения злоумышленника.
- Убедитесь, что на устройствах настроены удаленные агенты или методы доступа на основе ключей. Это позволяет собирать доказательства без раскрытия секретных данных для входа в систему.
- Обращаясь к людям за помощью, говорите о конфиденциальности конкретно и недвусмысленно — они могут не знать, что определенная информация должна оставаться конфиденциальной, если вы об этом не скажете.

- На каждом этапе расследования учитывайте выводы, которые проницательный злоумышленник может сделать из ваших действий. К примеру, злоумышленник, скомпрометировавший сервер Windows, может заметить внезапное ужесточение групповой политики и понять, что он был обнаружен.

КОГДА ИНСТРУМЕНТЫ МОГУТ ПОМОЧЬ

Многие современные инструменты для общения и совместной работы (например, клиенты электронной почты и чата или редакторы документов для совместной работы) стараются быть полезными, автоматически обнаруживая шаблоны, выглядящие как интернет-контент, и создавая ссылки. Некоторые инструменты даже подключаются к этим удаленным ссылкам и кэшируют найденный контент, чтобы быстрее отображать его при запросе. Например, написание «example.com» в электронной таблице, электронной почте или в окне чата приведет к загрузке контента с этого сайта без дополнительного взаимодействия с вашей стороны.

Обычно такое поведение полезно, но при обеспечении операционной безопасности во время сбора данных о вашем противнике эти инструменты предадут вас, автоматически связываясь с инфраструктурой злоумышленника легко заметными способами.

Если несколько аналитиков обмениваются информацией в личном чате, автоматически получающем контент, в журналах злоумышленника может появиться что-то вроде такого:

```
10.20.38.156 - - [03/Aug/2019:11:30:40 -0700] "GET /malware_c2.gif
HTTP/1.1" 206 36277 "-" "Chatbot-LinkExpanding 1.0"
```

Если ваш противник следит за этим, то для него будет очевидно, что вы обнаружили его файл malware_c2.gif.

Привыкайте всегда записывать домены и другие сетевые индикаторы таким образом, чтобы ваши инструменты не искали совпадения и не использовали автозаполнение, даже если вы не расследуете происшествие. Запись example.com как example[dot]com уменьшит вероятность ошибки во время расследования.

Жертвуем хорошим OpSec для большего блага

Есть одно вопиющее исключение из общего совета о том, как сохранять реагирование на инцидент в секрете: если вы столкнулись с неизбежным и четко идентифицируемым риском. Если вы подозреваете, что взлом системы настолько критичен, что важные данные, системы или даже человеческие жизни могут оказаться под угрозой, крайние меры могут быть оправданы. Уязвимость или ошибка, рассматриваемая как инцидент, иногда может быть настолько легко использована и широко известна (как в случае с Shellshock¹), что завершение

¹ Shellshock (<https://oreil.ly/8eDCJ>) был настолько простым в развертывании, что за несколько дней после публикации миллионы серверов уже подвергались активной атаке.

работы или полное отключение системы может быть лучшим способом ее защиты. Конечно, это явно укажет злоумышленнику и другим людям, что что-то пошло не так.

КОМПРОМИСС БЕЗОПАСНОСТИ И НАДЕЖНОСТИ: НЕИЗБЕЖНЫЙ РИСК

Возможно, придется выдержать время простоя продукта и гнев пользователя для достижения долгосрочных целей безопасности и надежности. В начале 2019 года Apple отреагировала на публично раскрытую и легко используемую ошибку конфиденциальности в Facetime, отключив весь доступ к серверам Facetime до установки исправлений. Это однодневное отключение популярного сервиса было признано в индустрии безопасности как правильный курс действий. В этом случае Apple решила защитить своих пользователей от легко эксплуатируемой проблемы, путем отказа от гарантии доступа к сервису.

Подобные важные решения и организационные компромиссы не будут приниматься только командиром инцидента, за исключением очень рискованных ситуаций (например, при отключении системы управления энергосистемой для предотвращения катастрофы). Как правило, такие выборы делают руководители внутри организации. Однако мнение ИС, как признанного эксперта, учитывается при обсуждении этих решений. Его советы и опыт в области безопасности имеют решающее значение. В итоге большинство решений, принимаемых организацией в условиях кризиса, заключается не в том, чтобы сделать *правильный* выбор. Речь идет о принятии *лучшего возможного* варианта из ряда неоптимальных.



Процесс расследования

Исследование компрометации в области безопасности представляет собой попытку проследить каждую фазу атаки для восстановления шагов злоумышленника. В идеале ваша IR-команда (из любых назначенных на эту работу инженеров) будет пытаться поддерживать короткий цикл действий между несколькими задачами¹. Они направлены на выявление всех затронутых частей организации и максимально возможное изучение произошедшего.

Цифровая форензика относится к процессу выяснения всех действий, потенциально предпринятых злоумышленником на устройстве. Эксперт-аналитик (инженер, проводящий экспертизу — в идеале человек со специальной подготовкой

¹ У этого короткого цикла действий минимальная задержка между шагами — следующий шаг не может быть кем-либо выполнен до завершения предыдущего шага кого-то другого.

и опытом) анализирует все части системы, включая доступные журналы, чтобы попытаться определить, что произошло. Аналитик может выполнить некоторые или все из следующих действий.

Создание образов для форензики

Создание безопасной и доступной только для чтения копии (и контрольной суммы) любых устройств хранения данных, подключенных к скомпрометированной системе. Это сохраняет данные в известном состоянии на момент взятия копии без случайной перезаписи или повреждения исходного диска. Для судебных разбирательств часто нужны экспертные образы оригинальных дисков в качестве доказательств.

Создание образов памяти

Создание копии системной памяти (или, в некоторых случаях, памяти запущенного бинарного файла). Память может содержать много цифровых доказательств, полезных во время расследования. Например, деревья процессов, запущенные исполняемые файлы и даже пароли к файлам, зашифрованным злоумышленником.

Выделение файла

Извлечение содержимого диска, чтобы увидеть, можно ли восстановить определенные типы файлов, особенно те, которые, возможно, были удалены. Например, журналы, которые злоумышленник пытался стереть. Некоторые ОС не обнуляют содержимое файлов при их удалении. Они только отменяют связь с именем файла и помечают область диска как свободную для повторного использования. Таким образом вы сможете восстановить данные, от которых злоумышленник попытался избавиться.

Анализ журнала

Расследование связанных с системой событий, которые появляются в журналах в самой системе или других источниках. Сетевые журналы могут показать, кто и когда общался с системой. Журналы с других серверов и рабочих столов могут показывать другие действия.

Анализ вредоносных программ

Выполнение анализа инструментов, используемых злоумышленниками, для определения того, что эти инструменты делают, как они работают и с какими системами они могут взаимодействовать. Данные этого анализа обычно передаются командам, выполняющим работу по форензике и обнаружению, для лучшего понимания потенциальных признаков компрометации системы.

В цифровой экспертизе отношения между событиями важны, как и сами события.

Большая часть работы, выполняемой экспертом-аналитиком для получения артефактов, необходима для создания *криминалистической хронологии*¹. Собрав хронологически упорядоченный список событий, аналитик может определить взаимозависимость и причинно-следственную связь действий злоумышленника, доказав причину событий.

ПРИМЕР: ВЗЛОМ ПОЧТЫ

Рассмотрим вымышленный сценарий: неизвестный злоумышленник успешно скомпрометировал рабочую станцию инженера, отправив вредоносное вложение по электронной почте разработчику, случайно открывшему его. Это вложение установило вредоносное расширение браузера на рабочую станцию разработчика. При помощи этого расширения злоумышленник украл учетные данные разработчика и вошел на файловый сервер. Попав внутрь сервера, он приступил к сбору конфиденциальных файлов и копированию их на свой удаленный сервер. В итоге разработчик обнаружил компрометацию, просматривая установленные им расширения браузера, и сообщил о нарушении безопасности.

Первым делом вы можете решить заблокировать учетную запись разработчика. Однако не забывайте об упомянутых ранее проблемах операционной безопасности. Вы всегда должны начинать расследование с невмешательства, пока не узнаете достаточно об атаке, чтобы принять обоснованные решения о том, как реагировать. В начале расследования у вас очень мало информации. Вы знаете только, что на компьютере разработчика есть вредоносное расширение для браузера.

Ваш первый шаг — сохранять спокойствие и не паниковать. Затем объявите, что инцидент продолжается, и привлечите руководителя операции для проведения более глубокого расследования. Здесь OL должен создать команду, отвечающую на следующие вопросы.

- Как был установлен бэкдор?
- Каковы возможности бэкдора?
- В каких других браузерах организации существует этот бэкдор?
- Что сделал злоумышленник в системе разработчика?

Мы называем эти начальные вопросы *опорными точками*: по мере ответа на каждый из них возникают новые вопросы. Например, как только команда реагирования обнаруживает, что атака перешла на общий файловый ресурс, он становится объектом нового экспертного расследования. Команда исследователей должна в дальнейшем ответить на тот же набор вопросов для общего файлового ресурса и любых новых зацепок, последующих за этим. По мере того как команда идентифицирует каждый из инструментов и техник злоумышленника, они обрабатывают информацию, чтобы определить, на что следует нацелить любые дополнительные расследования.

¹ Криминалистическая хронология — это список всех произошедших в системе событий, в идеале сосредоточенный на событиях, связанных с расследованием. Такой список обычно упорядочивает события по времени их происшествия.

Фрагментация расследования

Если у вас достаточно сотрудников для одновременного выполнения нескольких действий (см. подраздел «Распараллеливание действий» далее в этой главе), рассмотрите возможность разделения ваших усилий на три направления. Каждое из них должно выполнять большую часть расследования. Например, OL может разделить свою временную команду на три команды.

- Команда *экспертов* — чтобы исследовать системы и определить, какие из них были затронуты атакующим.
- Команда *реверс-инжиниринга* для изучения подозрительных двоичных файлов, определения уникальных отпечатков пальцев, служащих индикаторами компрометации (indicator of compromise, IOC)¹.
- *Поисковая* команда для поиска этих отпечатков пальцев во всех системах. Она уведомляет команду *экспертов* каждый раз, когда они идентифицируют подозрительную систему.

Рисунок 17.1 показывает отношения между командами. OL отвечает за поддержание тесной обратной связи между ними.

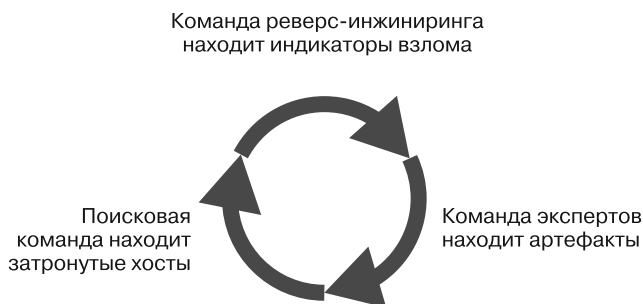


Рис. 17.1. Отношения между командами исследователей

В итоге скорость обнаружения новых зацепок замедляется. На этом этапе IC решает, что пришло время для восстановления. Вы узнали все, что можно узнать? Возможно, нет. Однако вы узнали все, что вам нужно знать для успешного избавления от злоумышленника и защиты нужных ему данных. Может быть сложно определить, где провести эту линию из-за всех неизвестных. Принятие этого решения во многом похоже на знание того, когда нужно прекратить разо-

¹ Обратная разработка вредоносного ПО — это специализированная работа, и не во всех организациях есть человек, опытный в этом вопросе.

гревание пакетика с попкорном в микроволновой печи: когда интервал между хлопками заметно возрастает, нужно двигаться вперед, пока не сторел весь пакет. Перед исправлением тщательно все спланируйте и скоординируйте. Глава 18 описывает это подробнее.

ЦИФРОВАЯ ЭКСПЕРТИЗА В МАСШТАБЕ

Описание криминалистического анализа в этой главе объясняет лишь основы, но тема проведения форензики в масштабе или сложных средах вроде облачного развертывания (где у вас может не быть привычных инструментов или ваши инструменты могут работать иначе) очень обширна. В ряде статей блога об экспертизе от Google (<https://oreil.ly/YGbtX>) эти темы рассматриваются более подробно.

Сохранение контроля над инцидентом

После объявления инцидента и назначения обязанностей членам команды задача ИС состоит в обеспечении бесперебойной работы. Она включает в себя прогнозирование потребностей команды реагирования и их удовлетворение до того, как они перерастут в проблемы. Ради эффективности ИС должен посвятить все свое время контролю и управлению происшествием. Если, как ИС, вы обнаруживаете, что подключаетесь к проверке журналов, выполняете задачи быстрой форензики или иначе вовлекаете себя в проводимые операции, пришло время сделать шаг назад и пересмотреть свои приоритеты. Если никто не стоит за штурвалом корабля, это может привести к отклонению от курса или даже крушению.

Распараллеливание действий

В идеале опытная команда ИР может *распараллелить* действия по устранению инцидента, разбив все части процесса реагирования и выполняя их одновременно, насколько это возможно. Если вам нужны результаты незапланированной задачи или часть информации, которая понадобится в течение жизненного цикла происшествия, назначьте кого-либо для выполнения задачи или подготовки к работе. Например, возможно, вы еще не готовы поделиться результатами экспертизы с правоохранительными органами или сторонней фирмой, которая поможет в расследовании. Однако если вы планируете в будущем делиться своими выводами, необработанные записи о расследовании для этого не подойдут. Поручите кому-нибудь подготовить отредактированный список индикаторов, которым можно поделиться по ходу вашего расследования.

Может показаться нелогичным начинать подготовку к очистке среды на ранних этапах расследования. Но если у вас есть свободные сотрудники, то самое время поручить им эту задачу. С IMAG можно создавать пользовательские роли в любое время, поэтому вы можете назначить кому-либо роль *руководителя по восстановлению* (remediation lead, RL) в любой момент во время происшествия. RL может начать изучение затронутых взломом областей по мере обнаружения их оперативной командой. Вооружившись этой информацией, RL может создать план по дальнейшей очистке и восстановлению. После завершения расследования у IC уже будет готовый план очистки. На этом этапе, вместо того чтобы принимать решение о последующих шагах на месте, они могут приступить к следующему шагу. Если у вас есть доступный персонал, никогда не рано назначать кого-то, чтобы начать подготавливать постмортем.

Используем для примера программную конструкцию — роль IC в крупном инциденте выглядит как набор шагов в цикле `while`:

(Пока инцидент продолжается):

1. Уточните с каждым из ваших руководителей:
 - Каков их статус?
 - Нашли ли они новую важную для других информацию?
 - Есть ли у них какие-нибудь препятствия?
 - Нужно ли им больше сотрудников или ресурсов?
 - Перегружены ли они?
 - Замечаете ли вы какие-то неполадки, которые могут проявиться позже или вызвать проблемы?
 - Каков уровень усталости каждого из руководителей? Каждой команды?
2. Обновите документы по текущему состоянию, информационные панели или другие источники информации.
3. Информированы ли соответствующие заинтересованные стороны (руководители, юристы, связи с общественностью и т. д.)?
4. Нужна ли вам помощь или дополнительные ресурсы?
5. Есть ли какие-либо задачи, которые можно было бы выполнить параллельно, находясь в ожидании?
6. Достаточно ли у вас информации для принятия решений по восстановлению и очистке?
7. Сколько времени нужно команде для предоставления следующего набора обновлений?
8. Завершился ли инцидент?

Возврат к началу цикла.

Цикл НОРД — это еще одна важная структура для принятия решений, связанных с инцидентами. Схема, в которой респондент должен *наблюдать, ориентироваться, решать* и *действовать*. Это полезная мнемоника для напоминания о том, что нужно внимательно рассмотреть новую информацию, подумать о том, как она соотносится с общей картиной вашего инцидента, принять осмысленное решение и только после этого действовать.

Передача обязанностей

Никто не может работать без остановки над одной проблемой долгое время и не испытывать при этом сложностей. Часто, чтобы справиться с кризисом, нужно время. Плавное распределение работы между респондентами — обязательное условие, помогающее создать культуру безопасности и надежности (см. главу 21).

Борьба с большим лесным пожаром требует усилий и может занять дни, недели или даже месяцы. Для тушения таких пожаров Калифорнийский департамент лесного хозяйства и противопожарной защиты разбивает лес на «зоны» и назначает каждой из них командира инцидента. IC устанавливает долгосрочную цель для своей области и краткосрочные цели для достижения общей цели. Он разбивает имеющиеся ресурсы на смены таким образом, чтобы, пока одна команда работает, другая команда отдыхала и была готова в срок подменить первую.

Когда команды, работающие на пожаре, исчерпывают все свои ресурсы, IC собирает обновления статуса от всех руководителей команд и назначает новых руководителей для их замены. Далее IC сообщает новому руководству об общей и конкретных целях для достижения, их назначении, доступных ресурсах, угрозах безопасности и другую важную информацию. После новая команда берет на себя управление, а ее сотрудники продолжают работу предыдущей смены. Каждый новый руководитель быстро связывается с предыдущим, чтобы не пропустить важной информации. Уставшая команда может отдыхать, пока тушение пожара продолжается.

Инциденты безопасности — это не пожар, требующий больших физических усилий, но ваши сотрудники могут испытывать такое же эмоциональное истощение и стресс, связанные с этим. Очень важно иметь план передачи своей работы другим при необходимости. Рано или поздно уставший респондент начнет ошибаться. Если ваша команда проработала усердно и достаточно долго, то она сделает больше ошибок, чем исправит. Это явление часто называют законом убывающей отдачи (https://oreil.ly/_1_iU). Слежение за усталостью персонала — это не просто забота о вашей команде. Усталость может навредить реагированию из-за ошибок и низкого морального духа. Во избежание переутомления ограничьте смены (включая смены IC) не более чем 12 часами непрерывной работы.

Если у вас большой штат сотрудников и вы хотите ускорить ответные действия, попробуйте разделить вашу команду реагирования на две поменьше. Они смогут работать 24 часа в сутки до полной ликвидации происшествия. Если у вас нет

большого персонала, придется реагировать медленнее, чтобы ваши сотрудники могли пойти домой и отдохнуть. Небольшие команды могут работать сверхурочно в «режиме героя», но он неустойчив и сильно снижает качество результатов. Не злоупотребляйте этим режимом.

Рассмотрим организацию команд в Северной и Южной Америке, Азиатско-Тихоокеанском и Европейском регионах. Такого рода организации могут осуществлять ротацию персонала по принципу «следуй за солнцем», чтобы новые респонденты постоянно переключались на инцидент по графику, как показано на рис. 17.2. У организации поменьше может быть похожий график, но с меньшим количеством мест и большим промежутком времени между сменами. Возможна и работа в рамках одного местоположения, но с половиной оперативного персонала, работающего в ночные смены для поддержания реагирования, пока остальные отдыхают.

ИС должен заранее подготовиться к передаче управления в случае происшествия. Она включает в себя обновление документации по отслеживанию, заметки о доказательствах, файлы и другие письменные записи, хранящиеся в течение смены. Организуйте логистику передачи, время и способ связи заранее. Встреча должна начинаться с краткого изложения текущего состояния инцидента и направления расследования. Вы должны включить официальную передачу данных от руководителя каждого подразделения (операционного, связного и т. д.) соответствующему заменяющему сотруднику.

Информация для сообщения команде зависит от происшествия. В Google мы обнаружили, что руководителю предыдущей команды полезно задавать себе вопрос: «Если бы я не передал вам это расследование, над чем бы я работал следующие 12 часов?» Новый ИС должен получить ответ на этот вопрос до конца собрания по передаче.

Повестка собрания по передаче управления может выглядеть примерно так.

1. [Предыдущий ИС] Выделить одного человека для ведения заметок, лучше из новой команды.
2. [Предыдущий ИС] Подведите итог текущему статусу работы.
3. [Предыдущий ИС] Описать задачи, которые ИС выполнял бы в течение следующих 12 часов, если бы он не передал управление происшествием.
4. [Все участники] Обсудить проблему.
5. [Новый ИС] Описать задачи, планируемые для выполнения в течение следующих 12 часов.
6. [Новый ИС] Установить время следующего собрания (-ий).

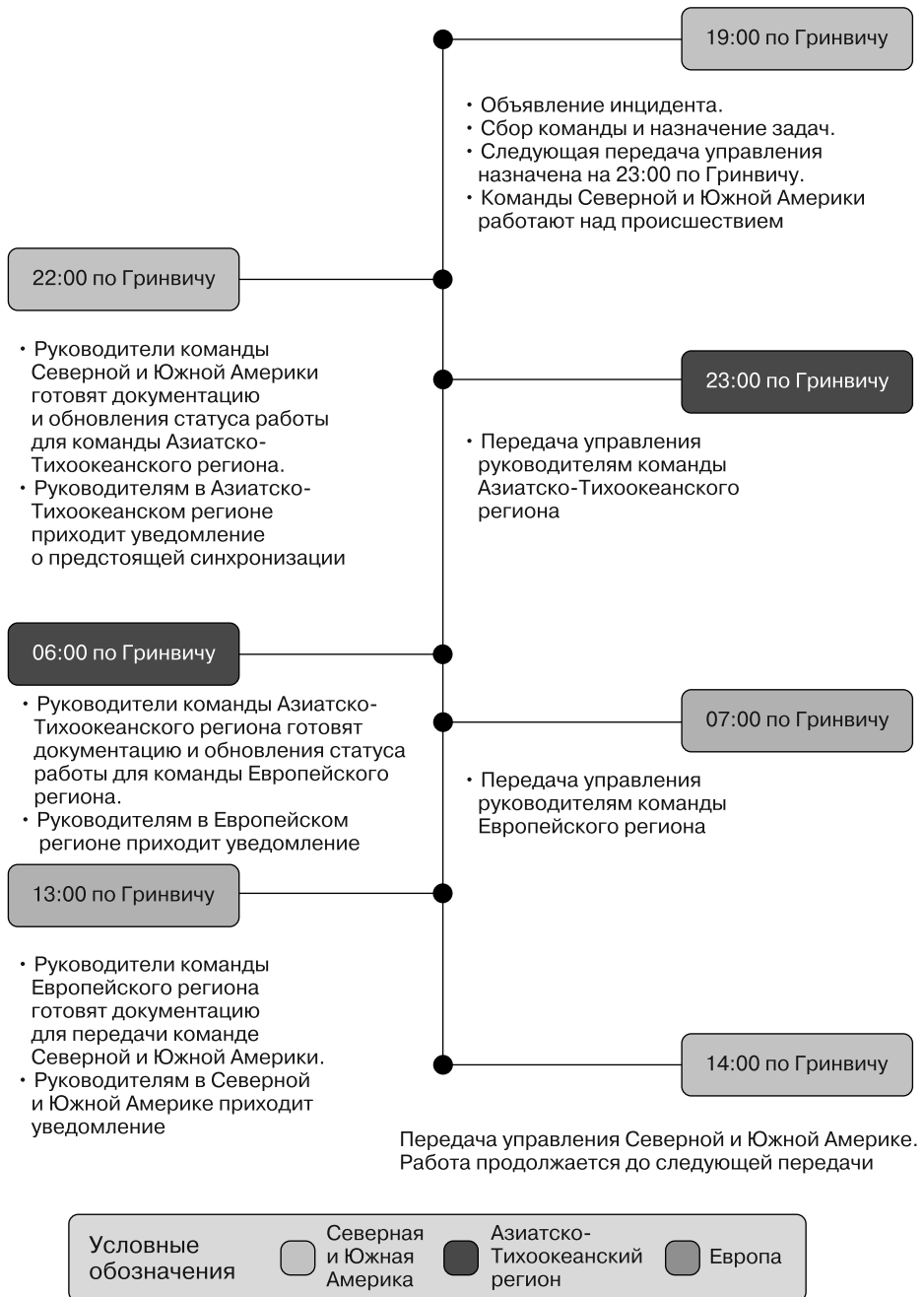


Рис. 17.2. Замена персонала по принципу «следуй за солнцем»

Командный дух

Во время любого серьезного происшествия ИС должен поддерживать командный моральный дух. Этот момент часто упускается из виду, но он тоже очень важен. Происшествия могут быть стрессовыми, и каждый инженер будет по-разному реагировать на эти ситуации высокого давления.

Некоторые принимают такие ситуации как вызов и участвуют в них с энтузиазмом, а другие находят сочетание напряженных усилий и двусмысленности глубоко разочаровывающим, и они не хотят ничего другого, кроме как покинуть место происшествия и вернуться домой.

Как ИС, не забывайте, что мотивация, поощрение и отслеживание общего эмоционального состояния вашей команды — основные факторы в достижении вашей цели. Вот несколько советов по поддержанию морального состояния в разгар кризиса.

Ешьте

Команда реагирования может быть готова работать, забыв о еде — но голод не станет ждать. Он может снизить эффективность и может вывести людей из себя. Заранее планируйте командные перерывы и перекусы. Это помогает сохранять необходимый позитивный настрой и целеустремленность.

Спите

Закон убывающей отдачи применим и к людям. Ваш персонал со временем станет менее эффективным по мере того, как наступит истощение и каждый сотрудник преодолеет свою собственную точку максимальной усталости. Это приведет к большему количеству ошибок, а не к прогрессу, что чревато замедлением реагирования на инциденты. Следите за утомляемостью ваших респондентов и убедитесь, что им можно отдыхать по мере необходимости. Иногда руководители должны вмешаться, чтобы люди, не осознающие свою потребность в отдыхе, тоже делали перерывы.

Расслабляйтесь

Если есть такая возможность, например, команда ожидает завершения восстановления файлового массива и не может выполнить параллельный прогресс, соберите всех вместе, чтобы избавиться от стресса. Несколько лет назад, во время особенно крупного и продолжительного происшествия в Google, команда реагирования сделала перерыв на час, чтобы уничтожить неисправный жесткий диск при помощи молотка и жидкого азота. Спустя годы команда, принимавшая участие в проекте, вспоминает это как о самом важном моменте реагирования.

Следите за выгоранием

Как IC, вы должны активно следить за признаками выгорания в команде (и самом себе). Не становятся ли специалисты более циничными и негативно настроенными? Выражают ли они опасения, что проблема нерешаемая? Может быть, они впервые принимают участие в устранении крупного происшествия, и им просто страшно. Поговорите откровенно и убедитесь, что они понимают ваши ожидания и могут им соответствовать. Если участнику команды нужен перерыв и у вас есть кто-то на замену — предложите сотруднику отдохнуть.

Подавайте пример

Реалистичный, но позитивный взгляд руководителя на ситуацию во многом определяет ожидания команды в отношении успеха. Позитивное видение стимулирует команду и заставляет сотрудников чувствовать, что они могут достичь своей цели. Кроме того, члены команды реагирования могут скептически относиться к тому, что действительно поощряется уделять время уходу за собой (например, для правильного питания и сна), пока они не увидят, что IC или OL открыто выполняют это как наилучшую практику.

Коммуникация

Из всех технических вопросов реагирования на происшествия коммуникация остается самым сложным. Даже при лучших обстоятельствах может быть сложно общаться с людьми за пределами вашей команды. Стресс, сжатые сроки и высокая угроза безопасности могут усилить эти трудности и быстро привести к задержке или упущенной возможности действовать. Вот несколько основных возможных проблем и советы по их решению.



Во многих книгах отлично раскрыта тема коммуникации. Чтобы углубиться в эту тему, мы рекомендуем книгу Ника Моргана *Can You Hear Me?* (Harvard Business Review Press, 2018) и Алана Алда *If I Understood You, Would I Have This Look on My Face?* (Random House, 2017).

Недопонимание

Недопонимание между респондентами может стать причиной проблем с общением. Люди, которые привыкли работать вместе, могут делать предположения о том, что не было сказано. Люди, не привыкшие работать вместе, могут использовать незнакомый жаргон, аббревиатуры, имеющие разное значение для разных команд, или принять общую систему координат, которая не является общей.

Например, рассмотрим фразу «Мы можем снова запустить сервис, когда атака будет устранена». Для команды разработчиков продукта это может означать, что как только инструменты злоумышленника будут удалены, система станет безопасной для использования. Для команды безопасности это может означать, что система небезопасна для использования, пока полное расследование не решит, что у злоумышленника больше нет доступа и он не может вернуться в скомпрометированную среду.

Когда вы обнаруживаете, что сами разбираетесь с инцидентом, как правило, полезно *быть откровенным и свержообщительным*. Объясните, что вы имеете в виду, когда просите что-то или ожидаете чего-либо, даже если думаете, что другой должен сразу вас понять. В примере из предыдущего абзаца было бы лучше сказать: «Мы можем снова запустить сервис, если убедимся, что все отходные пути были исправлены и злоумышленник больше не может получить доступ к нашим системам». Помните, что ответственность за передачу информации лежит на том, кто ее передает — только он может убедиться, что люди, с которыми он общается, получают предполагаемое сообщение.

Уклонение от ответа

Уклонение от ответа — еще одна популярная ошибка в общении. Когда от людей ожидают, что они могут дать советы в стрессовой ситуации, но из-за неуверенности они склонны добавлять уточнения к своим заявлениям. Избегать выражения уверенности в неопределенной ситуации, вроде «Мы уверены, что раскрыли все уязвимости», может показаться безопаснее. Однако такой подход часто запутывает ситуацию и ведет к неопределенности среди тех, кто принимает решения. Опять же, быть откровенным и чрезмерно коммуникабельным — лучшее средство здесь. Если ваш ИС спрашивает, нашли ли вы все инструменты злоумышленника, «Да, мы уверены» — плохой ответ. Скажите что-то вроде: «Мы уверены в серверах, NAS, электронной почте и общих файловых ресурсах, но не уверены в наших размещенных системах, потому что там у нас меньше видимости журналов».

Встречи

Для улучшения реагирования собирайте людей, чтобы скоординировать их усилия. Регулярная и быстрая синхронизация с основными участниками реагирования на происшествия эффективна для поддержания контроля и отслеживания всего происходящего. ИС, ОЛ, юрист компании и несколько ключевых руководителей могут встречаться каждые два-четыре часа, чтобы убедиться, что команда может быстро адаптироваться к любым новым работам.

Четко ограничьте список участников этих встреч. Если в комнате или на видео-конференции много людей и говорит лишь несколько, а остальные слушают или проверяют электронную почту — ваш список приглашений слишком велик. В идеале руководители инцидентов должны присутствовать на этих встречах, а остальные должны работать над выполнением задач. После встречи руководители могут передать обновления своим командам.

Встречи могут быть важной частью совместной работы, но их неправильная организация может помешать прогрессу. Мы рекомендуем, чтобы IC предоставлял заранее установленную повестку для каждой встречи. Это очевидно, но об этом легко забыть во время выброса адреналина и в стрессовой ситуации при длительном происшествии. Вот примерная повестка в Google для стартовой встречи по происшествиям безопасности.

1. [IC] Выделить одного человека для ведения заметок.
2. [IC] Все участники представляются, начиная с IC:
 - Имя
 - Команда
 - Роль
3. [IC] Правила взаимодействия:
 - Нужно ли учитывать конфиденциальность?
 - Есть ли конкретные проблемы операционной безопасности в работе?
 - Кому принадлежит право принимать решения? Кого нужно добавить в процессы принятия решений? Кого не следует добавлять?
 - Дайте знать своему руководителю, если вы переходите к новой задаче или закончили что-то делать.
4. [IC] Описать текущий статус: какую проблему мы решаем?
5. [Все] Обсудить проблему.
6. [IC] Подвести итог для действий и владельцев.
7. [IC] Задать вопросы команде:
 - Нужны ли нам еще какие-нибудь ресурсы?
 - Есть ли какие-то другие команды, которые мы должны привлечь?
 - Есть ли препятствия?
8. [IC] Определите время следующего собрания для синхронизации и ожидаемую посещаемость:
 - Чье присутствие требуется?
 - Кому необязательно присутствовать?

Вот пример повестки для совещания по синхронизации действий.

1. [ИС] Выделить одного человека для ведения заметок.
2. [ИС или руководитель ops] Подвести итог текущему состоянию.
3. [ИС] Получить обновления от каждого участника / потока деятельности.
4. [ИС] Обсудить следующие шаги.
5. [Руководитель ops] Назначить задания и попросить каждого повторить то, что ему нужно сделать.
6. [ИС] Установить время следующего собрания (-ий).
7. [Руководитель ops] Обновить систему отслеживания действий.

В обеих примерных повестках дня ИС назначает человека для ведения заметок. По своему опыту мы знаем, что ведущий заметки человек необходим для ведения расследования. Руководители заняты решением возникающих на собрании проблем, и у них просто нет времени делать хорошие заметки. Во время происшествия вы часто забываете вещи, которые, как вам *кажется*, вы помните. Эти заметки становятся бесценными и при написании постмортема. Помните, если у происшествия есть юридические последствия, лучше проконсультироваться с командой юристов по поводу управления этой информацией.

Информирование нужных людей с правильными уровнями детализации

Определение правильного уровня детализации для передачи информации — серьезная проблема взаимодействия в условиях происшествий. Несколько людей на разных уровнях должны будут знать *что-то*, связанное с происшествием, но для операционной безопасности они не могут или не должны знать *все* детали. Имейте в виду, что сотрудники, которые только смутно осведомлены о происшествии, скорее всего, займутся заполнением пробелов¹. Слухи среди сотрудников могут быстро внести путаницу и стать разрушительными, если их распространить за пределы организации.

Долгосрочное расследование содержит частые обновления с разными уровнями детализации для следующих людей.

Руководители и старшее руководство

Они должны получать краткие, сжатые обновления о прогрессе, препятствиях и возможных последствиях.

¹ Если у человека нет доступа к актуальной информации, он будет что-то выдумывать.

Команда IR

Этой команде нужна актуальная информация о расследовании.

Организационный персонал, не вовлеченный в происшествие

Решите, какая информация может быть доступна сотрудникам организации. Если вы сообщите всем сотрудникам о происшествии, то сможете попросить их о помощи. Однако эти люди могут распространять информацию дальше, чем вы намеревались. Если вы не сообщите персоналу организации ничего об инциденте, пойдут сплетни и вам придется с ними бороться.

Заказчики

Клиенты по закону имеют право на получение информации о происшествии в течение установленного времени. Однако ваша организация может добровольно уведомить их об этом. Если здесь имеет место отключение сервисов, к которым у клиентов есть доступ, у них могут возникнуть вопросы, требующие ответов.

Участники системы юстиции/правосудия

Если вы довели инцидент до правоохранительных органов, у них могут быть вопросы и они могут начать запрашивать информацию, которой вы изначально не планировали делиться.

Чтобы справиться со всеми этими требованиями, не отвлекая IC постоянно, назначьте *руководителя коммуникациями* (communications lead, CL). Его главная задача — оставаться в курсе происшествия по мере его распространения и готовить сообщения для заинтересованных сторон. Основные обязанности CL таковы.

- Работа с отделами продаж, поддержки и другими внутренними партнерами, чтобы ответить на любые возникающие вопросы.
- Подготовка кратких сводок для руководителей, юристов, сотрудников регуляторных органов и других лиц, выполняющих надзорные функции.
- Работа с прессой и СМИ для гарантии, что люди верно осведомлены, и чтобы делать точные и своевременные заявления об инциденте при необходимости. Убедитесь, что люди вне команды реагирования не делают противоречивых заявлений.
- Постоянное и тщательное наблюдение за распространением информации о происшествии и соблюдением принципа «служебной необходимости».

CL понадобится рассмотреть возможность обращения к экспертам по предметной области, внешним консультантам по кризисным коммуникациям или кому-либо еще, кому нужна помощь в управлении информацией, связанной с происшествием, с минимальной задержкой.

Объединяем все вместе

Этот раздел объединяет содержание этой главы, рассматривая предположительное реагирование на компрометацию, с которой может столкнуться организация любого размера. Рассмотрим сценарий, где инженер обнаруживает, что неизвестная учетная запись сервиса была добавлена в облачный проект, который он не видел ранее. В полдень он сообщает о своих опасениях в службу безопасности. После предварительного расследования команда безопасности определяет, что учетная запись инженера была скомпрометирована. При помощи данных ранее рекомендаций давайте разберемся, как можно отреагировать на такой взлом от начала до конца.

Сортировка инцидента

Первый шаг реагирования — это сортировка. Начните с предположения худшего: подозрения службы безопасности верны, учетная запись инженера взломана. Злоумышленник с привилегированным доступом для просмотра конфиденциальной внутренней информации и/или пользовательских данных станет серьезным нарушением безопасности, поэтому важно объявить о происшествии.

Объявление о происшествии

Как командир инцидента, вы уведомляете остальную команду безопасности о следующем:

- произошел инцидент;
- вы берете на себя роль IC;
- вам понадобится дополнительная поддержка от команды для расследования.

Коммуникация и операционная безопасность

Теперь, когда вы объявили о происшествии, другие люди в организации — руководители, юристы и т. д. — должны знать, что инцидент продолжается. Если злоумышленник скомпрометировал инфраструктуру вашей организации, отправка электронной почты или использование чата с этими людьми — серьезный риск. Следуйте рекомендациям по операционной безопасности. Допустим, ваш план действий в чрезвычайных ситуациях предполагает использование

кредитной карты организации для регистрации бизнес-аккаунта в стороннем облаке и создания учетных записей для каждого, кто участвует в реагировании. Для создания этой среды и подключения к ней вы используете заново перестроенные ноутбуки, не подключенные к инфраструктуре управления вашей организации.

Пользуясь новой средой, вы звоните руководителям и основным юристам, чтобы сообщить им, как получить защищенный ноутбук и облачную учетную запись, чтобы они могли использовать электронную почту и чаты. Все текущие респонденты в офисе, поэтому собрания для обсуждения подробностей будут в ближайшем конференц-зале.

Начало инцидента

Как IC, вам нужно назначить специалистов для расследования, поэтому вы попросите инженера из команды безопасности с опытом работы в области форензики стать операционным руководителем. Новый OL сразу же начинает расследование и, если нужно, нанимает других инженеров. Они начинают со сбора журналов из облака, ориентируясь на период времени, когда были добавлены данные учетной записи сервиса. После подтверждения того, что данные были добавлены учетной записью инженера, когда инженера точно не было в офисе, команда экспертов делает вывод, что учетная запись была взломана.

Теперь команда экспертов отталкивается от расследования подозрительной учетной записи и приступает к изучению остальных действий, связанных с добавлением этой учетной записи. Ее участники решают собрать все относящиеся к скомпрометированной учетной записи системные журналы, ноутбук и рабочую станцию этого инженера. Команда определяет, что расследование может занять некоторое время, поэтому принимает решение добавить больше сотрудников и распределить усилия.

В вашей организации нет большой команды безопасности, поэтому у вас не хватает экспертов-аналитиков для адекватного распределения задач по форензике. Однако у вас есть системные администраторы, которые хорошо разбираются в своих системах и могут помочь в анализе журналов. Вы решаете присоединить их к команде форензики. OL связывается с ними по электронной почте с просьбой «обсудить некоторые вещи» по телефону и полностью информирует их во время этого звонка. OL просит системных администраторов собрать все журналы, относящиеся к скомпрометированной учетной записи, из любой системы в организации, пока команда форензики анализирует ноутбук и настольный компьютер.

К 5 часам вечера становится ясно, что расследование продлится и по окончании рабочего дня. Ожидаемо, что ваша команда устанет и начнет ошибаться еще до ликвидации происшествия, поэтому нужно придумать план передачи обязанностей или обеспечения непрерывной работы. Вы уведомляете команду, что у них есть четыре часа для проведения как можно более обширного анализа записей журнала и хостов. В течение этого времени вы постоянно обновляете данные для руководителей и юристов и узнаете у OL, нужна ли их команде дополнительная помощь.

В 21:00 на встрече по синхронизации действий команд OL показывает, что их команда нашла начальную точку входа злоумышленника. Очень хорошо продуманное фишинговое письмо инженеру, обманным образом запускающее команду, которая загружает бэкдор в систему и устанавливает постоянное удаленное соединение.

Передача обязанностей

К моменту завершения встречи в 9 часов вечера многие устраняющие проблему инженеры работают уже 12 и более часов. Как прилежный IC, вы знаете, что продолжать работать в таком темпе рискованно и что на происшествие понадобится больше усилий. Вы решаете передать часть работы. Хотя организация не может полностью укомплектовать команду безопасности в главном офисе в Сан-Франциско, у вас есть отдельный офис в Лондоне с несколькими старшими сотрудниками.

Вы говорите своей команде потратить следующий час на завершение документирования результатов, связываясь в это время с лондонской командой. Следующим IC назначается старший инженер в лондонском офисе. Как предыдущий IC, вы информируете нового обо всем, что узнали. Получив права владения всей связанной с происшествием документацией и убедившись, что команда из Лондона понимает следующий шаг, новый IC подтверждает, что он отвечает за реагирование до 9 утра по тихоокеанскому времени. Команда Сан-Франциско вздохнула с облегчением и отправилась домой отдыхать. Ночью лондонская команда продолжает расследование, уделяя особое внимание анализу сценариев и действий злоумышленника.

Обратная передача обязанностей

В 9 утра следующего дня команды Сан-Франциско и Лондона проводят синхронизацию передачи. За ночь лондонская команда добилась большого прогресса. Она определила, что сценарий, запущенный на взломанной рабочей станции, установил простой бэкдор, позволяющий злоумышленнику войти

в систему с удаленного компьютера и начать осмотр системы. Заметив, что история оболочки ОС инженера включала входы в учетные записи облачных сервисов, злоумышленник воспользовался сохраненными учетными данными и добавил свой ключ учетной записи сервиса в список административных токенов.

После этого атакующий ничего не предпринимал на рабочей станции. Вместо этого журналы облачного сервиса показывают, что он напрямую взаимодействовал с API сервиса. Он загрузил новый образ машины и запустил десятки копий этой виртуальной машины в новом облачном проекте. Лондонская команда еще не проанализировала ни один из запущенных образов, но проверила все учетные данные во всех существующих проектах. Команда подтвердила, что учетная запись вредоносного сервиса и токены API, о которых они знают, являются единственными учетными данными, действующими нелегально.

С этим обновлением от лондонской команды вы подтверждаете получение новой информации и передачу управления как IC. Далее вы обрабатываете новую информацию и предоставляете краткое обновление руководителям и юристам. Вы информируете свою команду о новых результатах.

Вы знаете, что злоумышленник получил административный доступ к рабочим сервисам, но еще не знаете, подвергались ли пользовательские данные риску или они были затронуты. Перед командой экспертов стоит новая высокоприоритетная задача — проверить все другие действия, потенциально предпринимаемые злоумышленником в отношении существующих производственных машин.

Информирование пользователей и исправление

По мере продолжения расследования вы решаете, что пришло время распараллелить еще несколько компонентов реагирования на инцидент. Если злоумышленник получил доступ к пользовательским данным организации, возможно, нужно будет сообщить об этом пользователям. Подумайте и о смягчении атаки. Вы выбираете коллегу, опытного технического писателя, и назначаете его руководителем по коммуникациям (communications lead, CL). Один из системных администраторов, не вовлеченных в экспертизу, становится руководителем по восстановлению (remediation lead, RL).

В сотрудничестве с юристом организации CL готовит пост в блоге, где объясняет, что произошло и как это может повлиять на заказчиков. Несмотря на то что все еще много пробелов (таких как «данные <заполнить здесь> были <заполнить

здесь>»), подготовленная заранее структура и ее предварительное утверждение помогут быстрее передать ответ, когда вы будете знать все подробности.

Между тем RL составляет список всех ресурсов, затронутых злоумышленником, вместе с предложениями по очищению каждого ресурса. Даже если вы знаете, что пароль инженера не был указан в исходном фишинговом электронном письме, придется изменить его учетные данные. Ради безопасности вы решаете защититься от еще не найденных бэкдоров, которые могут появиться позже. Вы делаете копию важных данных инженера и удаляете домашний каталог, создавая новый на только что установленной рабочей станции.

По мере реагирования команда узнает, что злоумышленник не получил доступ к производственным данным — к огромному облегчению для всех! Дополнительные виртуальные машины, запущенные злоумышленником, — это множество серверов добычи криптовалюты, перенаправляющих полученные средства в виртуальный кошелек. Ваш RL отмечает, что вы можете удалить эти машины или сделать снимок и заархивировать их, чтобы позже сообщить об инциденте в правоохранительные органы. Можно и удалить проект, в котором были созданы машины.

Завершение реагирования

Примерно в середине дня у вашей команды закончились задачи. Они проверили каждый ресурс, к которому могли обращаться вредоносные ключи API, создали документ по смягчению последствий и подтвердили, что злоумышленник не затронул конфиденциальные данные. К счастью, это была лишь оппортунистическая попытка добычи криптовалюты, и злоумышленник не интересовался вашими данными. Ему просто нужно было много вычислительных возможностей за чей-то счет. Команда решает, что теперь нужно приступить к плану исправления. После окончательной встречи с юристами и руководящим составом для уверенности в том, что решение о закрытии происшествия получило их одобрение, вы сообщаете команде о начале действий.

Освободившись от задач по экспертизе, команда теперь выбирает задачи из плана по восстановлению и выполняет их как можно быстрее, полностью избавляясь от злоумышленника. Остаток дня уходит на запись наблюдений для постмортема. Наконец, вы (IC) проведете краткую встречу, на которой все участники могут обсудить, как прошел инцидент. Вы четко сообщаете о закрытии происшествия и что экстренное реагирование больше не требуется. Перед тем, как все отправятся домой отдыхать, нужно проинформировать лондонскую команду, чтобы они тоже знали о завершении происшествия.

Резюме

Управление инцидентами в масштабе становится настоящим искусством, которое отличается от общего управления проектами и менее масштабной работы по реагированию на инциденты. Сосредоточившись на процессах, инструментах и организационных структурах, можно собрать команду для эффективного реагирования на любой кризис с подходящей для современного рынка скоростью. Нет разницы, маленькая у вас организация, где специалисты из основной команды временно включаются в команду реагирования при необходимости, огромная компания с командами реагирования по всему миру или что-то среднее. Вы можете применять рекомендации из этой главы, чтобы эффективно действовать в случае происшествий в области безопасности. Распараллеливание реагирования на инциденты и профессиональное управление командой с использованием ICS/IMAG помогут вам масштабируемо и надежно реагировать на любые возникающие происшествия.

ГЛАВА 18

Восстановление и последствия

*Алекс Перри, Гэри О'Коннор
и Хизер Адкинс с Ником Сода*

Во избежание перебоев в обслуживании пользователей нужно иметь возможность быстро восстанавливаться после инцидентов, связанных с безопасностью и надежностью. Однако у восстановления после инцидента с безопасностью есть ключевое отличие: присутствие злоумышленника. Он может иметь постоянный доступ к вашей среде или повторно подключаться в любой момент — даже во время восстановления системы.

В этой главе мы рассмотрим информацию о восстановлении после атак, важную для проектирующих, внедряющих и обслуживающих системы сотрудников. Те, кто выполняет восстановление, часто не специалисты по безопасности — это люди, создающие системы и ежедневно их использующие. В уроках и примерах в этой главе рассказывается о том, как держать злоумышленника на расстоянии во время восстановления. Мы обсудим логистику, сроки, планирование и инициирование этапов восстановления. Мы коснемся основных компромиссов — например, когда нарушить деятельность злоумышленника, а когда оставить его в системе для получения дополнительной информации.

Знает ли ваша организация, как восстановиться после серьезного происшествия? Знают ли те, кто выполняет восстановление, какие решения принимать? В главе 17 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/testing-reliability/>) и главе 9 книги Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/incident-response/>) обсуждаются методы управления перебоем в обслуживании и их предотвращения. Многие из них относятся и к безопасности. Однако восстановление после атак безопасности имеет уникальные элементы — особенно когда в инциденте участвует активный злоумышленник (см. главу 2). И несмотря на то, что здесь дается общая схема работ по восстановлению, мы уделяем особое внимание тому, что инженеры по восстановлению должны знать об атаках на систему безопасности.

Как мы обсуждаем в главах 8 и 9, системы, построенные в соответствии с хорошими принципами проектирования, могут быть устойчивыми к атакам и их

легко восстановить. Это верно, независимо от того, является ли система единым вычислительным экземпляром, распределенной системой или сложным многоуровневым приложением. Для более простого восстановления хорошо выстроенная система должна быть интегрирована с тактикой антикризисного управления. Как мы обсуждали в предыдущей главе, благодаря эффективному антикризисному управлению вы сможете сохранять тонкий баланс между удержанием злоумышленника и восстановлением любых поврежденных активов до известного (улучшенного) хорошего состояния. В этой главе описываются нюансы правильного восстановления, которые включаются в хорошие чек-листы восстановления для достижения этих целей.

Инженеры по восстановлению — те, кто проектирует, внедряет и обслуживает эти системы ежедневно. Во время атаки может понадобиться помощь специалистов по безопасности для выполнения отдельных ролей, таких как проведение форензики, сортировка уязвимостей в системе безопасности или принятие взвешенных решений (см. главу 17). Однако для восстановления систем до заведомо исправного состояния нужен опыт, полученный благодаря ежедневной работе с системой. Благодаря партнерству и слаженным действиям специалисты по безопасности и инженеры по восстановлению могут обмениваться информацией для восстановления системы в двух направлениях.

Восстановление после атак безопасности часто подразумевает неоднозначную среду, охватывающую большую область, чем описано в заранее спланированных инструкциях¹. Злоумышленник может изменить поведение во время атаки, а инженеры по восстановлению могут ошибаться или обнаруживать непредвиденные характеристики и подробности о системах. В этой главе описан динамический подход к восстановлению, нацеленный на гибкость злоумышленника.

Восстановление может быть мощным инструментом для быстрого улучшения состояния безопасности системы. Оно принимает форму как краткосрочных тактических смягчений последствий, так и долгосрочных стратегических улучшений. В завершение этой главы мы предлагаем поразмышлять о неразрывной связи между инцидентом, восстановлением и периодом ожидания до следующего инцидента.

¹ Тема восстановления после инцидентов безопасности хорошо освещена во многих ресурсах, таких как NIST SP 800-184 (<https://oreil.ly/7N8mr>) — руководство по восстановлению после событий кибербезопасности. Здесь подчеркивается важность заблаговременного планирования путем создания проверенных чек-листов восстановления для всех типов ожидаемых событий задолго до наступления события. В этом руководстве есть инструкции по тестированию таких планов. Это хороший общий справочник, но выполнить советы из него во время кризиса может быть сложно.

Логистика восстановления

Как мы знаем из предыдущей главы, управляемый инцидент выигрывает от рас- параллеливания действий по реагированию. Особенно полезным оно будет во время восстановления. Работать над восстановлением и расследовать инцидент должны разные люди, по нескольким причинам.

- Этап расследования инцидента часто отнимает много времени. Он детализированный и требует сосредоточенности в течение длительных периодов времени. При долгосрочных происшествиях командам исследователей часто нужен перерыв к моменту начала восстановления.
- Этап восстановления после инцидента может начаться уже во время расследования. В итоге вам понадобятся отдельные команды, способные работать параллельно, передавая информацию друг другу.
- Навыки для проведения расследования могут отличаться от навыков для восстановления.

При подготовке к восстановлению и рассмотрении вариантов у вас должна быть формализованная структура команды. Количество человек в команде зависит от масштабов происшествия. Для более сложных инцидентов настройте механизмы координации — формальные группы, частые встречи, общие хранилища документации и экспертные оценки. Многие организации моделируют операции групп восстановления в существующих процессах разработки Agile при помощи спринтов, Scrum-команд и коротких циклов обратной связи.

О РОЛЯХ И ОТВЕТСТВЕННОСТИ

Темы из этой главы применимы к организациям всех размеров — от нескольких человек, работающих над проектом с открытым исходным кодом, до малого бизнеса и крупных компаний. Хотя мы часто говорим о командах и формализованных ролях, эти концепции можно адаптировать к любому масштабу.

Если говорить о формализованной структуре команды — ее можно рассматривать как внутреннюю команду сотрудников, каждый из которых согласен взять на себя одну или несколько ролей. Или же можно адаптировать модель, передав некоторые задачи восстановления третьей стороне — например, при помощи подрядчика для повторной сборки системы. Адаптивность особенно важна, ведь в восстановительных работах участвуют многие части организации, некоторые из которых обычно не справляются с кризисным реагированием.

Хорошо организованное восстановление после сложного происшествия может выглядеть как тщательно поставленный балетный спектакль, где действия разных людей взаимосвязаны. Важно, чтобы танцоры в восстановительном балете не наступали друг другу на ноги. Четко определите роли для подготовки, проверки и выполнения восстановления, убедившись, что все участники понимают операционные риски и часто общаются лицом к лицу.

По мере развития инцидента командир инцидента (IC) и руководитель операций (OL) должны назначить руководителя по восстановлению (RL), чтобы начать планирование восстановления, как описано в главе 17. RL должен тесно координировать свою работу по чек-листу восстановления с IC, чтобы убедиться, что усилия по восстановлению соответствуют остальной части расследования. RL отвечает за сбор команды людей с соответствующим опытом и составление чек-листа восстановления (обсуждается в подразделе «Чек-листы восстановления» далее в этой главе).

В Google команды, выполняющие восстановление, — это те, которые ежедневно создают и запускают системы. К ним относятся SR-инженеры, разработчики, системные администраторы, сотрудники службы поддержки и специалисты по безопасности, управляющие такими рутинными процессами, как аудит кода и проверка конфигурации.

Управление информацией и взаимодействие во время восстановления — важные составляющие успешного реагирования. Необработанные отчеты об инцидентах, черновики, чек-листы восстановления, новая рабочая документация и информация о самой атаке будут важными артефактами. Убедитесь, что у команды восстановления есть доступ к документации, но она недоступна злоумышленнику. Используйте для хранения что-то вроде отдельного компьютера с воздушным зазором (физической изоляцией). Например, вы можете комбинировать инструменты управления информацией: системы отслеживания ошибок, облачные инструменты совместной работы и настенные доски и стикеры. Убедитесь, что эти инструменты за пределами самой широкой возможной области взлома. Начните с рукописных заметок или добавления независимого поставщика услуг, как только вы убедитесь, что компьютеры членов команды восстановления не скомпрометированы.

Правильное управление информацией — еще один ключевой аспект безребойного восстановления. Используйте ресурсы, доступ к которым может получить любой разработчик, чтобы обновлять их в режиме реального времени при возникновении проблем или выполнении пунктов чек-листа. Если план

восстановления доступен только руководителю, это будет препятствовать быстрому выполнению.

Восстанавливая системы, важно вести надежные записи о том, что происходило во время восстановления. Если вы допустите ошибку на этом пути, журнал аудита поможет исправить любые проблемы. Назначение специалистов для ведения заметок или документации — хорошая идея. В Google мы используем труд технических писателей для оптимизации управления информацией во время усилий по восстановлению. Мы рекомендуем прочитать главу 21, в которой рассматриваются дальнейшие организационные аспекты.

Сроки восстановления

Время начала восстановления после инцидента сильно зависит от характера расследования. Если затронутая инфраструктура критически важна, вы можете запустить восстановление после атаки почти в момент обнаружения. Так часто бывает при восстановлении после атак типа «отказ в обслуживании». Другой вариант: если в инциденте участвует злоумышленник, полностью контролирующей инфраструктуру, вы можете начать планирование восстановления почти сразу, но приступить к выполнению, только когда полностью поймете, что сделал злоумышленник. Обсуждаемые в этой главе процессы восстановления применяются к любым срокам восстановления: во время расследования происшествия, после завершения этапа расследования или на обоих этапах.

Наличие достаточной информации об инциденте и понимание масштабов восстановления укажет на выбор пути. Обычно к моменту запуска операции восстановления команда исследователей уже приступила к написанию постмортема (возможно, в форме предварительных заметок), который будет обновляться по мере продвижения. Информация в этом документе станет основой для фазы планирования восстановления (см. следующий раздел «Планирование восстановления»), которая должна быть завершена до начала восстановления (см. раздел «Запуск восстановления» далее в главе).

Первоначальный план может измениться, поэтому этапы планирования и выполнения восстановления могут совпадать. Однако *не начинайте восстановление без какого-либо плана*. Мы советуем создавать чек-листы перед началом восстановления. К действиям после восстановления (см. раздел «Действия после восстановления» этой главы) нужно приступать сразу же после завершения восстановления. Если между этими двумя этапами пройдет слишком много времени, вы можете забыть подробности своих прошлых действий или отвлечься на работу над важными среднесрочными и долгосрочными исправлениями.

Планирование восстановления

Цель усилий по восстановлению в том, чтобы смягчить последствия атаки и вернуть системы в их обычное состояние, внедряя любые нужные улучшения. Сложные события безопасности часто требуют распараллеливания управления инцидентами и создания структурированных команд для работы над разными частями происшествия.

Планирование восстановления будет опираться на информацию, обнаруженную командой исследователей. Поэтому важно тщательно спланировать восстановление, прежде чем предпринимать какие-либо действия. К планированию нужно приступать, как только у вас будет достаточно исходной информации о действиях злоумышленника. В следующих разделах описываются рекомендации по подготовке и распространенные ошибки, которых нужно избегать.

Определение объема восстановления

Способ определения восстановления для вашего инцидента зависит от типа атаки, с которой вы столкнулись. Например, может быть просто восстановиться после незначительной проблемы, такой как программа-вымогатель, на одном компьютере. Достаточно просто переустановить систему. Но чтобы восстановиться после действий субъекта национального масштаба, который присутствует во всей вашей сети и проводит эксфильтрацию чувствительных данных, нужно сочетать несколько стратегий восстановления и наборов навыков по всей организации. Помните, что приложенные усилия могут не соответствовать тяжести проблемы. Отсутствие подготовки к простой атаке вымогателей может привести к взлому многих устройств, что потребует проведения ресурсоемкого восстановления.

Для начала восстановления после инцидента безопасности у команды восстановления должен быть полный список систем, сетей и данных, затронутых атакой. Дайте им достаточно информации о тактике, методах и процедурах злоумышленника (*tactics, techniques and procedures*, ТТР), чтобы определить любые связанные ресурсы, которые могут быть затронуты. Например, если команда восстановления обнаружит, что система распространения конфигурации была скомпрометирована, эта система будет в области восстановления. Любые системы, получившие настройки от этой системы, тоже могут учитываться в области действия. Поэтому команда по расследованию должна определить, изменил ли злоумышленник какие-то настройки и были ли они перенесены в другие системы.

Как упомянуто в главе 17, в идеале ИС назначает кого-то для ведения документации по смягчению последствий, что пригодится для формального постмортема (обсуждается в подразделе «Постмортемы» далее в этой главе) с самого начала расследования. В документе по смягчению последствий и постмортеме будут определены шаги по устранению первопричины компрометации. Вам понадобится достаточно информации, чтобы расставить приоритеты для списка задач и распределить их на краткосрочные (например, исправление известных уязвимостей) или стратегические долгосрочные изменения (например, изменение процессов сборки для предотвращения использования уязвимых библиотек).

Чтобы понять, как защитить эти активы в будущем, изучите все затронутые действиями злоумышленника активы. Например, если он смог использовать уязвимый программный стек на веб-сервере, для восстановления нужно понимать атаку, чтобы вы могли исправить дыру в других связанных системах. Таким же образом, если ваш злоумышленник получил доступ с помощью фишинга учетных данных пользователя, команда восстановления должна спланировать способ не дать другому злоумышленнику сделать то же самое завтра. Постарайтесь понять, что злоумышленник может использовать для будущей атаки. Сопоставьте список действий атакующего и возможных способов защиты от них, как мы это делали в главе 2 (см. табл. 2.3). Вы можете использовать этот список в качестве операционной документации для объяснения введения определенных новых мер защиты.

Для составления списка скомпрометированных активов и краткосрочных мер по смягчению угроз нужна тесная обратная связь с постмортемами, командами по расследованию и командиром инцидента. Команда по восстановлению должна как можно скорее узнавать о новых результатах расследования. Без эффективного обмена информацией между командами расследования и восстановления злоумышленники могут обойти меры по смягчению последствий. В плане восстановления также стоит описать вариант, когда все еще возможно присутствие злоумышленника в системе и наблюдение за вашими действиями.



Соображения по восстановлению

При разработке фазы восстановления после происшествия вы можете столкнуться с несколькими открытыми вопросами, на которые сложно ответить. В этом разделе рассматриваются частые ошибки и идеи балансировки компромиссов. Эти принципы будут уже знакомы специалистам по безопасности, часто работающим со сложными инцидентами. Однако такая информация полезна для всех, кто участвует в восстановлении. До принятия решения задайте себе следующие вопросы.

Как злоумышленник отреагирует на попытки восстановления?

Чек-лист действий по смягчению последствий и восстановлению (см. подраздел «Чек-листы восстановления» и раздел «Примеры» далее в этой главе) должен предусматривать разрыв любого соединения злоумышленника с вашими ресурсами и обеспечение невозможности повторного соединения. Выполнение этого шага — тонкое уравнивающее действие, требующее почти идеального знания атаки и четкого плана выполнения выбрасывания. Ошибка приведет к тому, что злоумышленник предпримет дополнительные действия, которые вы не сможете предвидеть или заметить.

Рассмотрим такой пример: во время инцидента команда исследователей обнаруживает, что злоумышленник взломал шесть систем, но команда не может определить, как началась атака. Непонятно даже, как злоумышленник получил доступ к любой из этих систем. Команда восстановления создает и реализует план восстановления. В этом сценарии команда действует без полного знания о начале атаки, как злоумышленник отреагирует и об активности злоумышленника в других системах. Активный злоумышленник сможет увидеть из своей позиции в другой скомпрометированной системе, что вы отключили найденные шесть систем, и продолжит уничтожать остальную доступную инфраструктуру.

Помимо взлома систем, злоумышленники могут просматривать электронную почту, системы отслеживания ошибок, изменения кода, календари и другие полезные для координации восстановления ресурсы. В зависимости от серьезности инцидента и типа компрометации вы можете провести расследование и восстановление с помощью систем, которые не видны злоумышленнику.

Рассмотрим команду восстановления, координирующую работу системы обмена мгновенными сообщениями, когда одна из учетных записей членов команды скомпрометирована. Злоумышленник, тоже вошедший в систему и просматривающий чат, может видеть все личные сообщения во время восстановления, включая любые известные элементы расследования. Атакующий может даже получить информацию, которую команда восстановления не знает. Он может использовать эти знания для взлома еще большего количества систем другим способом, минуя все видимые команде расследования области. В этом сценарии команда восстановления должна была установить новую систему обмена мгновенными сообщениями и развернуть новые компьютеры для связи с респондентами. Например, недорогие устройства Chromebook.

Эти примеры покажутся вам экстремальными, но они отражают очень простую мысль: по другую сторону атаки человек, реагирующий на ваши ответные действия. План восстановления должен учитывать его возможные действия после изучения ваших планов. Старайтесь полностью понять объем доступа злоумышленника и принять меры для снижения риска дальнейшего вреда.



Сейчас специалисты по реагированию на инциденты безопасности обычно соглашаются с тем, что нужно дождаться полного понимания атаки, прежде чем пытаться избавиться от злоумышленника. Это не дает ему наблюдать за вашими мерами и помогает защищаться.

Это хороший совет, но применяйте его осторожно. Если ваш злоумышленник уже совершает что-то опасное (например, извлекает конфиденциальные данные или разрушает системы), вы можете принять меры прежде, чем получите полную картину его действий. Если вы решите избавиться от злоумышленника до того, как получите полное представление о его намерениях и масштабах атаки, ожидайте игру в шахматы. Подготовьтесь соответственно и знайте, какие шаги нужно предпринять, чтобы объявить мат!

Если вы работаете со сложным инцидентом или если активный злоумышленник взаимодействует с вашими системами, план восстановления должен содержать тесную интеграцию с командой исследователей для гарантии того, что злоумышленник не восстановит доступ к системам или не предпримет обходных мер. Обязательно сообщите команде исследователей о планах восстановления. Они должны быть уверены, что ваши действия остановят атаку.

Скомпрометирована ли ваша инфраструктура восстановления или инструментарий?

На ранних этапах планирования восстановления определите, какая инфраструктура и инструменты вам нужны для реагирования, и уточните у команды расследования, считают ли они, что эти системы скомпрометированы. Их ответ определит, можно ли выполнить безопасное восстановление и какие дополнительные шаги по исправлению и подготовке более полных ответных действий необходимо подготовить.

Представьте, что злоумышленник скомпрометировал несколько ноутбуков в сети и сервер конфигурации, управляющий их настройкой. Здесь вам понадобится план исправления сервера конфигурации, до того, как вы сможете восстановить любые скомпрометированные ноутбуки. Аналогично, если злоумышленник ввел вредоносный код в пользовательский инструмент восстановления из резервной копии, вам необходимо найти внесенные им изменения и восстановить код в обычном режиме до восстановления любых данных.

Важнее то, что вы должны подумать, как восстанавливать активы — будь то системы, приложения, сети или данные — расположенные в инфраструктуре, находящейся под контролем злоумышленника. Восстановление актива, когда злоумышленник контролирует инфраструктуру, может привести к повторной компрометации со стороны того же злоумышленника. Обычный шаблон вос-

становления в таких ситуациях — установка «чистой» или «безопасной» версии ресурса — чистой сети или системы, изолированной от любых взломанных версий. Это может означать полное воспроизведение всей инфраструктуры или ее основных частей.

Возвращаясь к нашему примеру скомпрометированного сервера конфигурации: вы можете выбрать создание изолированной карантинной сети и пересобрать эту систему, установив новую ОС. Потом вы можете вручную настроить систему, чтобы можно было загружать с нее новые машины, не вводя никаких контролируемых злоумышленниками настроек.

Какие варианты атаки существуют?

Допустим, команда исследователей сообщает, что злоумышленник воспользовался уязвимостью переполнения буфера в инфраструктуре веб-обслуживания. Когда у него будет доступ только к одной системе, вы знаете, что на 20 других серверах работает такое же некорректное ПО. Планируя восстановление, учитывайте не только скомпрометированную систему, но и то, скомпрометированы ли другие 20 серверов, и то, как вы планируете смягчать последствия для всех этих устройств в будущем.

ПОВТОРЯЮЩИЕСЯ РИСКИ

Представьте, что большой корабль должен оставаться на одном месте в море. Корабль бросает якорь и цепляется за подводный трос, который недостаточно прочен и рвется. Риск того, что судно зацепит подводный кабель, небольшой, но последствия могут быть катастрофическими. Это сильно повлияет на пропускную способность межконтинентальной сети, и такой риск есть для всех подводных кабелей в мире, а не только для того, который был зацеплен заблудшим кораблем. Подобный сбой сети должен инициировать анализ более широкой категории риска. При следующем планировании нужно учитывать эту недавно обнаруженную категорию отключений, чтобы обеспечить достаточную резервную мощность.

Рассмотрите и то, подвержены ли ваши системы (в краткосрочной перспективе) видам атаки, с которой вы сталкиваетесь сейчас. В примере с переполнением буфера при планировании восстановления нужно искать любые связанные программные уязвимости в инфраструктуре — либо связанный класс уязвимости, либо ту же уязвимость в другой части ПО. Это особенно важно в пользовательском коде или в случаях, когда вы используете общие библиотеки. Мы рассмотрели несколько способов тестирования вариантов (например, нечеткое тестирование) в главе 13.

Если вы используете ПО с открытым исходным кодом или коммерческое ПО, а варианты тестирования вне вашего контроля, то надеемся, что люди, поддерживающие программное обеспечение, сами рассмотрели возможные варианты атаки и применили нужные меры защиты. Проверьте наличие доступных исправлений для других частей вашего программного стека и включите широкий ряд обновлений в процесс восстановления.

Может ли восстановление вводить векторы атаки повторно?

Многие методы восстановления направлены на возврат поврежденных ресурсов в заведомо исправное состояние. Эти усилия могут опираться на образы систем, исходный код из хранилищ или настройки. Основной момент для восстановления — это то, будут ли ваши действия по восстановлению вводить новые векторы атак, которые сделают систему уязвимой, или приведут к откату любых достигнутых успехов надежности или безопасности. Рассмотрим образ системы с уязвимым ПО, позволяющим злоумышленнику взломать систему. Если вы повторно используете этот системный образ во время восстановления, то снова введете уязвимое программное обеспечение.

Такое повторное введение уязвимости — частая ошибка во многих средах. В том числе современных облачных вычислениях и локальных средах, полагающихся на «золотые образы», которые обычно состоят из целых снимков системы. Важно обновить эти образы и удалить скомпрометированные снимки до нового подключения систем к сети либо из источника, либо из системы сразу после установки.

Если у злоумышленника получилось изменить части инфраструктуры восстановления (настройки, хранящиеся в репозитории с исходным кодом) и вы восстанавливаете систему с помощью этих скомпрометированных настроек, изменения злоумышленника повторно установятся в системе. Для восстановления системы до хорошего состояния понадобится использование очень старых версий во избежание такого поведения. Это означает, что нужно тщательно продумать временную шкалу атаки. Когда именно злоумышленник вносил изменения и как далеко нужно вернуться для их отмены? Если нельзя определить точное время введения изменения, вам может потребоваться параллельно пересобрать большую часть инфраструктуры с нуля.



Восстанавливая системы или данные из традиционных резервных копий (на магнитной ленте), подумайте о том, сохранила ли ваша система также резервные копии изменений, внесенных злоумышленником. Нужно либо уничтожить любые резервные копии или снимки данных, содержащие свидетельства действий злоумышленника, либо изолировать их для дальнейшего анализа.

Каковы варианты смягчения последствий?

Следуя рекомендациям по отказоустойчивому проектированию систем (см. главу 9), вы сможете быстро восстанавливаться после инцидентов безопасности. Если сервис — это распределенная система (в отличие от монолитного бинарного файла), вы можете быстро и легко применить исправления безопасности для отдельных модулей. Вы можете выполнить обновление “на месте” для неисправного модуля, не подвергая значительному риску окружающие модули. Аналогичным образом в средах облачных вычислений вы можете создать механизмы для закрытия скомпрометированных контейнеров или виртуальных машин и быстрой замены их на заведомо исправные версии.

В зависимости от скомпрометированных вашим злоумышленником активов (устройств, принтеров, камер, данных и учетных записей) вы можете обнаружить несколько неидеальных вариантов смягчения последствий. Возможно, вам придется решить, какой вариант является наименее плохим, и смириться с разной степенью технического долга ради краткосрочной выгоды от постоянного удаления злоумышленника из ваших систем. Например, чтобы заблокировать его доступ, вы можете вручную добавить запрещающее правило в текущую конфигурацию маршрутизатора. Чтобы злоумышленник не увидел вносимые изменения, обойдите обычные процедуры проверки и отслеживания таких изменений в системе контроля версий. В этой ситуации вам следует отключить автоматическую передачу правил до тех пор, пока вы не добавите новые правила брандмауэра в каноническую версию конфигурации. Установите и напоминание о том, чтобы снова включить эти автоматические изменения правил в будущем.

Решая, стоит ли принимать технический долг во время краткосрочных действий по смягчению последствий, чтобы избавиться от злоумышленника, задайте себе следующие вопросы.

- Как быстро (и когда) мы сможем устранить эти краткосрочные меры? (Другими словами, как долго будет существовать этот технический долг?)
- Стремится ли организация поддерживать меры по смягчению угрозы весь срок ее существования? Готовы ли команды, которым принадлежит новый технический долг, принять этот долг и погасить его позже, путем улучшений?
- Повлияет ли смягчение угрозы на работоспособность систем и не превысим ли мы бюджет ошибок?¹
- Как сотрудники смогут определить, что это смягчение краткосрочное? Подумайте о том, чтобы пометить эту часть смягчения как технический долг,

¹ Бюджеты ошибок описаны в главе 3 книги Site Reliability Engineering (<https://sre.google/sre-book/embracing-risk/>).

который будет удален позже, чтобы его статус был виден всем, кто работает в системе. Например, добавляйте комментарии к коду и коммиты с описанием или push-сообщения, чтобы любой, кто использует новую функциональность, знал, что он может измениться или исчезнуть.

- Как будущий инженер без опыта работы в предметной области в отношении инцидента может доказать, что смягчение больше не нужно и что его можно устранить, не создавая риск?
- Насколько эффективным будет краткосрочное смягчение, если оставить его на долгое время (случайно или из-за обстоятельств)? Представьте, что злоумышленник взломал одну из ваших БД. Вы решаете поддерживать ее в режиме онлайн, пока выполняете очистку и переносите данные в новую систему. Краткосрочное смягчение заключается в том, чтобы изолировать базу данных в отдельной сети. Спросите себя: что произойдет, если эта миграция займет шесть месяцев вместо запланированных двух недель? Могут ли люди в организации забыть, что БД была взломана, и случайно подключить ее к безопасным сетям?
- Выявил ли эксперт по предметной области лазейки в ваших ответах на предыдущие вопросы?

Чек-листы восстановления

После выяснения объема восстановления изложите возможные варианты (как описано в разделе «Запуск восстановления» далее) и тщательно обдумайте компромиссы. Эта информация — основа вашего чек-листа восстановления (или нескольких списков, в зависимости от сложности инцидента). Каждое усилие по восстановлению должно использовать привычные и проверенные методы. Тщательное документирование и совместное использование шагов по восстановлению облегчает сотрудникам, участвующим в реагировании на происшествие, совместную работу и обсуждение плана восстановления. А хорошо документированный чек-лист также позволяет вашей команде по восстановлению определить области деятельности, которые вы можете распараллелить, и помогает вам координировать работу.

Как показано в шаблоне чек-листа восстановления на рис. 18.1, каждый элемент в нем соответствует отдельной задаче и навыкам, нужным для ее выполнения¹. Люди в вашей команде восстановления могут претендовать на задачи в зависи-

¹ Ваша реализация инструктажа «руки прочь от клавиатуры» зависит от вас, но основная цель в упражнении в том, чтобы гарантировать полное внимание и участие аудитории. Во время происшествия респонденты могут захотеть постоянно следить за выполнением сценариев или информационными панелями состояния. Чтобы выполнить тонкое и сложное восстановление, важно привлекать всеобщее внимание во время брифингов, чтобы все были на одной волне.

мости от их навыков. Тогда командир инцидента может быть уверен в завершении всех отмеченных этапов восстановления.

INSTRUCTIONS:

- Highlight in **GREEN** when **COMPLETED**.
- Highlight in **ORANGE** TO-DO items.
- Highlight in **RED** if task cannot be completed; it must be replaced with an equivalent action.
- Highlight in **BLUE** if task will not be completed because it is operationally risky and we need to make adjustments.

Incident commander

- Give a "hands off keyboards" briefing.
- Provide an overview of tasks and constraints.
- Notify everyone involved of the exact time remediation will begin.

In order of operations, the remediation lead initiates execution of the following:

1. [Team name #1]: Description of task
 - a. Detailed commands or steps
2. [Team name #2]: Description of task
 - a. Detailed commands or steps
3. [Team name #3]: Description of task
 - a. Detailed commands or steps
4. [Team name #4]: Description of task
 - a. Detailed commands or steps

Рис. 18.1. Шаблон чек-листа

Чек-лист должен содержать такие данные, как конкретные инструменты и команды, которые нужно использовать для восстановления. Поэтому, когда вы начнете очистку, у всех членов команды будет четкое, согласованное представление о том, какие задачи должны быть выполнены и в каком порядке. В чек-листе нужно указать все этапы очистки или процедуры отката, которые понадобятся вам при сбое плана. Мы будем использовать шаблон чек-листа на рис. 18.1 в рабочих примерах в конце главы.

Запуск восстановления

После происшествия в сфере безопасности надежное восстановление системы зависит от эффективных процессов, таких как тщательно составленные чек-листы. В зависимости от типа инцидента, которым вы управляете, вам нужно будет рассмотреть эффективные технические варианты. Цель действий по смягчению последствий и восстановлению в том, чтобы избавиться от злоумышленника, убедиться, что он не может вернуться, и сделать систему безопаснее. Глава 9 раскрывает принципы заблаговременного проектирования вариантов

восстановления системы. В этом разделе рассматриваются практические примеры восстановления с учетом этих принципов, а также плюсы и минусы принятия определенных решений.

Изоляция активов (карантин)

Изоляция (также называемая *карантином*) — популярный метод смягчения последствий атаки. Простой пример — антивирусное ПО, которое перемещает вредоносный двоичный файл в папку карантина, где права доступа к файлам запрещают чему-либо еще в системе читать или выполнять двоичный файл. Карантин обычно используется для изоляции скомпрометированного хоста. Вы можете изолировать его на уровне сети (например, отключив порт коммутатора) или на самом хосте (например, отключив сеть). Можно поместить в карантин целые сети скомпрометированных компьютеров, используя сегментацию сети. Многие стратегии реагирования на DoS-атаки удаляют сервисы из уязвимых сетей.

BEYONDCORP

В Google концепция нулевого доверия нашей архитектуры BeyondCorp (<https://oreil.ly/jGrvI>) позволяет легко смягчать инциденты при помощи изоляции. По умолчанию архитектуры вроде BeyondCorp усиливают методы карантина и изоляции при смягчении последствий атак.

BeyondCorp требует, чтобы мы доверяли активам только после проверки их состояния безопасности и получения разрешения на связь с корпоративными сервисами. В итоге между активами устанавливается наименьшее доверие. Это создает естественную границу изоляции, предотвращающую такие распространенные методы атаки, как *боковое перемещение* (перемещение между двумя машинами в сети). Доступ к сервисам четко определен через прокси-инфраструктуру и надежные учетные данные, которые можно отозвать в любое время.

Изоляция активов полезна, если нужно оставить скомпрометированную инфраструктуру работающей. Рассмотрим сценарий, где одна из критических БД была скомпрометирована. Из-за ее важности вам нужно поддерживать базу данных в режиме онлайн во время смягчения последствий. Возможно, эта база критически важна и ее восстановление займет несколько недель, а ваша организация не хочет сворачивать весь свой бизнес надолго. Возможно, вы можете ограничить влияние злоумышленника, изолировав БД в отдельной сети и наложив ограничения на то, какой сетевой трафик она может отправлять и получать (в/из Интернета и к остальной части инфраструктуры).

Предупреждение: оставление скомпрометированных активов в сети — пагубная форма технического долга. Если вы не разберетесь с этим долгом своевременно и оставите его в сети дольше, чем предполагалось, этим скомпрометированным активам может быть нанесен существенный ущерб. Такое может произойти по нескольким причинам: если это единственная копия данных, помещенных на карантин (без резервных копий), если есть проблемы с заменой помещенного в карантин актива или если люди просто забудут о скомпрометированных активах во время суматохи происшествия. В худшем случае кто-то из новичков в вашей организации (или команде реагирования) может даже снять изоляцию со взломанного ресурса!

Рассмотрите способы пометки этих активов как скомпрометированные — использование хорошо видимых стикеров на устройствах или ведение обновленного списка MAC-адресов систем, находящихся на карантине, и отслеживание их появления в вашей сети. Стикеры предотвращают повторное использование, а список адресов обеспечивает быстрое удаление при необходимости. Убедитесь, что в вашем чек-листе восстановления и постмортеме указано, находятся ли какие-либо активы на безопасном и постоянном карантине.

Повторная сборка системы и обновление программного обеспечения

Допустим, вы обнаружили вредоносное ПО злоумышленника в трех системах и начинаете фазу восстановления после инцидента. Что вы делаете, чтобы избавиться от злоумышленника? Удаляете ли вредоносную программу и оставляете системы работающими или же переустанавливаете их? В зависимости от сложности и критичности систем вам, возможно, придется рассмотреть компромиссы между этими вариантами. С одной стороны, если затронутые системы очень важны и их трудно перестроить, у вас может возникнуть желание удалить вредоносное ПО и двигаться дальше. С другой — если злоумышленник установил несколько типов вредоносных программ и вы не знаете обо всех, вы можете упустить возможность комплексной очистки. Обычно лучшее решение — переустановка систем с нуля с использованием заведомо исправных образов и программного обеспечения¹.

Если вы управляете средой с использованием надежных и безопасных принципов проектирования, повторная сборка систем или обновление ПО должно

¹ В зависимости от изощренности злоумышленника и глубины атаки вам может понадобиться переустановить BIOS и форматировать жесткий диск или даже заменить физическое оборудование. К примеру, может быть непросто восстановить после руткитов UEFI, как описано в документации ESET Research на LoJax (<https://oreil.ly/ZxFQj>).

быть несложным. В главе 9 есть несколько советов о том, как узнать состояние вашей системы, включая управление хостом и встроенное ПО.

Например, при использовании системы с аппаратно поддерживаемой проверкой загрузки, которая следует криптографической цепочке доверия в ОС и приложении (хороший пример — Chromebook), восстановление системы — это просто циклическое отключение питания, возвращающее систему в известное исправное состояние. Автоматизированные системы выпуска, такие как Rapid (как описано в главе 8 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/release-engineering/>)), тоже могут обеспечить надежный и предсказуемый способ обновления ПО во время восстановления. В средах облачных вычислений вы можете полагаться на мгновенные контейнеры и выпуски программного обеспечения для стандартной замены любых взломанных систем на безопасный стандартный образ.

Если вы запускаете фазу восстановления после инцидента без систем контроля исходного кода или стандартных образов системы для управления настройками или систем с использованием известных исправных версий, попробуйте ввести части краткосрочного плана восстановления. Есть варианты с открытым исходным кодом для управления сборками (Bazel (<https://bazel.info/>)), настройками (Chef (<https://chef.io>)), интеграцией и развертыванием приложений (Helm (<https://helm.sh>)) для Kubernetes. Внедрение новых решений за короткий промежуток времени поначалу может показаться сложной задачей, и при настройке этих решений вам, возможно, потребуется сначала сделать приблизительный выбор правильной конфигурации. Если ее поиск требует времени и усилий за счет другой важной технической работы, вы можете перенести доработку конфигурации на потом. Убедитесь, что вы внимательно изучили технический долг, который вы накапливаете в обмен на краткосрочные выгоды в области безопасности, и разработали план улучшения настройки таких новых систем.

Очистка данных

В зависимости от масштаба происшествия вам может понадобиться подтверждение невмешательства злоумышленника в исходный код, двоичные файлы, образы и конфигурации или системы, которые вы используете для их создания, управления или выпуска. Один из распространенных методов очистки экосистемы — получение заведомо исправных копий из оригинальных источников (от поставщиков ПО с открытым исходным кодом или коммерческого ПО), резервных копий или систем контроля версий, не подвергшихся атаке. После получения заведомо исправной копии вы можете выполнить сравнение контрольных сумм версий, которые вы хотите использовать, с заведомо исправными состояниями и пакетами. Если ваши старые исправные копии хранятся

во взломанной инфраструктуре, убедитесь, что у вас есть высокая уверенность в том, что вы знаете, когда злоумышленник начал вмешиваться в ваши системы, и обязательно просмотрите источники данных.

Надежное бинарное происхождение кода (как описано в главе 14) упрощает восстановление. Представьте, что вы обнаружили, что злоумышленник ввел вредоносный код в библиотеку `glibc`, используемую в системе сборки. Нужно идентифицировать все двоичные файлы, созданные в течение периода «под угрозой», места развертывания этих двоичных файлов и любых их зависимостей. Выполняя исследование, четко помечайте известный скомпрометированный код, библиотеки и двоичные файлы. Добавьте тесты, не позволяющие повторно внедрить уязвимый или взломанный код. Эти меры гарантируют, что другие члены команды по восстановлению не смогут случайно использовать скомпрометированный код во время восстановления или внедрить уязвимые версии.

Проверьте, не подделал ли злоумышленник какие-то данные уровня приложения, например, записи в БД. Из главы 9 мы знаем, что обеспечение надежной криптографической целостности резервных копий повышает вашу уверенность в них. Благодаря этому вы можете убедиться в том, что любые сравнения, которые вам необходимо провести с потенциально скомпрометированными настоящими данными, будут точными. Согласование сделанных злоумышленником изменений может быть сложным, так как требует наличия специальных инструментов. К примеру, для использования частичного восстановления могут понадобиться специальные инструменты для склейки файлов или записей, полученных из резервных копий, в производственных системах, и одновременная проверка целостности. В идеале нужно создать и протестировать эти инструменты при разработке стратегии обеспечения надежности.

Данные для восстановления

Процессы восстановления данных часто основаны на инструментах, которые поддерживают ряд таких операций, как откат, восстановление, резервное копирование, герметичная повторная сборка и воспроизведение транзакций. В пункте «Постоянные данные» в главе 9 обсуждается безопасное хранение данных, используемых для этих операций.

У многих таких инструментов есть параметры, позволяющие устанавливать компромисс между скоростью действий и безопасностью данных. Лучше не изменять эти параметры по умолчанию, если инструменты регулярно не тестируются на реальных рабочих нагрузках производственного масштаба. Тестирование, стейджинг или (что еще хуже) моки не позволяют реалистично использовать инфраструктурные системы. Например, трудно реалистично смоделировать

задержку, нужную для заполнения кэшей памяти или для оценщиков балансировки нагрузки для стабилизации вне реалистичных производственных условий. Эти параметры разные от сервиса к сервису, и так как задержка обычно отображается в данных мониторинга, вам следует настроить эти параметры между инцидентами. Работа с инструментом, ведущим себя плохо при восстановлении от враждебной атаки на систему безопасности, так же сложна, как и сдерживание нового злоумышленника.

Возможно, вы уже пользуетесь мониторингом для обнаружения большой потери данных из-за ошибок ПО. Возможно и то, что ваш злоумышленник не допустил срабатывания этих предупреждений. Тем не менее всегда стоит просматривать эти журналы: данные могут определить точку перегиба, с которой началась атака. Если метрики показывают такую точку — вы получили независимую нижнюю границу количества резервных копий, которые можно пропустить.

Восстановление на месте из недостаточно старой резервной копии может привести к повторной активации любых компрометаций, которые были скопированы во время инцидента. Если для обнаружения вторжения понадобилось время, то, возможно, ваша самая старая «безопасная» резервная копия уже была перезаписана. Если это так, то восстановление данных может быть вашим единственным вариантом.

В резервной копии для восстановления могут быть как желательные исправления, так и вкрапления поврежденных данных. Повреждение может быть вызвано злонамеренным изменением (со стороны атакующего) или случайным событием (например, повреждением ленты или сбоем жесткого диска). Инструменты, восстанавливающие данные, обычно направлены на восстановление либо от случайного повреждения, либо от злонамеренного повреждения, но не от обоих сразу. Важно знать, какие функции предоставляют ваши инструменты восстановления и пересборки, и их ограничения. Иначе результаты восстановления данных могут вас огорчить. Здесь пригодятся привычные процедуры проверки целостности. Например, проверка восстановленных данных на соответствие известным надежным криптографическим сигнатурам. В итоге избыточность и постоянное тестирование — лучшая защита от случайных событий.

Учетные данные и замена секретов

Злоумышленники часто используют украденные учетные записи, применяемые в вашей инфраструктуре, чтобы действовать от имени законных пользователей или сервисов. Например, злоумышленники, производящие фишинговую атаку с использованием пароля, могут попытаться получить данные для учетной записи, которую они могут использовать для входа в системы. Таким же образом,

с помощью передачи хеша¹, злоумышленники могут получать и повторно использовать учетные данные, включая данные для учетных записей администратора. В главе 9 обсуждается другой сценарий: компрометация файла *authorized_keys* в SSH. Во время восстановления часто нужно заменить учетные данные системы, пользователя, сервиса и учетные записи приложений с помощью таких методов, как замена ключей (например, ключей, используемых при аутентификации SSH). В зависимости от результатов расследования вы можете дополнительно подключить сетевые устройства и внеполосные системы управления, а также облачные сервисы — приложения SaaS.

В вашей среде могут быть другие требующие внимания секретные данные — ключи для шифрования данных в состоянии покоя и криптографические ключи для SSL. Если инфраструктура веб-сервиса скомпрометирована или потенциально доступна злоумышленнику, вам может понадобиться заменить ключи SSL. Если вы ничего не сделаете после похищения ключей злоумышленником, он сможет использовать их для атаки «человек посередине». Аналогично, если ключ шифрования для записей в вашей БД на ее скомпрометированном сервере, самый безопасный путь — заменить ключи и повторно зашифровать данные.

Криптографические ключи часто используются и для связи на уровне приложений. Если у злоумышленника был доступ к системам, где хранятся такие ключи прикладного уровня, вы захотите их заменить. Тщательно продумайте место хранения ключей API, например, ключей для облачных сервисов. Хранение служебных ключей в исходном коде или локальных файлах конфигурации — распространенная уязвимость². Если у злоумышленника есть доступ к этим файлам, потом он может получить доступ к другим средам. В процессе восстановления определите, были ли эти файлы доступны злоумышленнику (хотя может быть невозможно доказать, что к ним был получен доступ). Хорошая практика — чаще менять такие служебные ключи.

В зависимости от сценария замена учетных данных может потребовать тщательного выполнения. В случае с одной украденной учетной записью можно просто

¹ Передача хеша (Pass-the-hash) (<https://oreil.ly/ZhNsa>) — это метод, позволяющий злоумышленнику повторно использовать локально украденные с компьютера учетные данные (хешы NTLM или LanMan) в сети для входа в другие системы без необходимости ввода пароля пользователя в виде открытого текста.

² Не рекомендуется хранить ключи облачного сервиса в локальных файлах и исходном коде. См. недавнее систематическое исследование открытых служебных ключей в GitHub: *Meli M., McNiece M. R., Reaves B. How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories* // Proceedings of the 26th Annual Networked and Distributed Systems Security Symposium, 2019. <https://oreil.ly/r65fM>.

попросить пользователя сменить пароль. Однако если у атакующего был доступ ко многим учетным записям, включая администраторов, или вы точно не знаете, какие учетные записи скомпрометированы, вам, возможно, придется заменить учетные данные всех пользователей. При создании чек-листов восстановления обязательно укажите порядок сброса учетных записей, расставьте приоритеты для учетных данных администратора, известных скомпрометированных учетных записей и учетных записей, дающих доступ к конфиденциальным ресурсам. Если у вас в системе много пользователей, подумайте о замене учетных данных для всех пользователей сразу.

ВОСПОЛЬЗУЙТЕСЬ ВОЗМОЖНОСТЬЮ

Единовременная замена учетных данных в крупных организациях часто приводит к серьезным сбоям в работе. Представьте, что вы просите тысячи людей действовать одновременно. Службы единого входа (Single sign-on, SSO) и централизованные системы аутентификации могут ослабить влияние замены учетных данных, упрощая действия, которые необходимо выполнить пользователям на этапе восстановления. Однако внедрение таких сложных систем, как SSO, во время восстановления после происшествия может быть трудоемким и дорогостоящим. Поэтому оно может быть нецелесообразным для краткосрочного смягчения последствий.

Современные решения для идентификации и доступа, использующие облако, все популярнее. Благодаря им можно относительно быстро добавлять двухфакторную аутентификацию. Ее использование в вашей среде, особенно для таких менее защищенных точек входа, как доступ к электронной почте, дает быстрое преимущество. Двухфакторная проверка подлинности тоже дает дополнительное преимущество: вы будете знать, пытается ли еще злоумышленник войти в учетные записи. Эти попытки входа в систему будут выглядеть как сбои при входе в учетную запись.

Для улучшения вашего состояния безопасности ваш план долгосрочного смягчения последствий может использовать облачный сервис двухфакторной аутентификации во время краткосрочного смягчения и содержать более систематические изменения, такие как внедрение ключей безопасности на основе FIDO (о чем мы говорим в главе 7).

Есть еще одна сложность, связанная с заменой учетных данных. Она возникает, если в вашей организации применяются слабые методы обработки. Атака и дальнейшее смягчение угрозы могут ухудшить ситуацию. Допустим, что ваша компания использует централизованную систему, такую как база данных LDAP или Windows Active Directory, для управления учетными записями сотрудников. Некоторые из этих систем хранят историю паролей (обычно хешированных и соленых). Обычно системы сохраняют историю паролей, чтобы можно было сравнивать новые с более старыми и предотвращать повторное использование паролей пользователями с течением времени. Если у злоумышленника есть до-

ступ ко всем хешированным паролям и способ их взломать¹, он может определить шаблоны, по которым пользователи обновляют свои пароли. Например, если у пользователя в каждом пароле есть год (*password2019*, *password2020* и т. д.), злоумышленник может быть в состоянии предсказать следующий в последовательности. Это опасно, если смена пароля — часть стратегии восстановления.

Если вы можете связаться со специалистами по безопасности, посоветуйтесь с ними при создании плана восстановления. Эти специалисты могут выполнить моделирование угроз и дать советы об улучшении методов обработки учетных данных. Они могут порекомендовать устранить необходимость в сложных схемах паролей при помощи двухфакторной аутентификации.

Действия после восстановления

Следующий шаг после избавления от злоумышленника и восстановления — выход из инцидента и рассмотрение долгосрочных последствий. Если вы столкнулись с таким незначительным инцидентом, как кража одной учетной записи одного сотрудника или одной системы, фаза восстановления и последствия могут быть относительно простыми. Если вы столкнулись с происшествием посерьезнее с более сильным воздействием, объем работ по восстановлению и последствия могут быть длительными.

После операции «Аврора» в 2009 году Google планировал вносить систематические и стратегические изменения в среду. Некоторые из них мы внесли почти сразу, а такие, как BeyondCorp и наша работа с FIDO по организации широкого применения двухфакторной аутентификации с использованием ключей безопасности², потребовали больше времени. Ваш постмортем должен различать возможные достижения в краткосрочной и долгосрочной перспективе.

¹ Самый безопасный способ хранения паролей — использование функции формирования ключа с солью (https://en.wikipedia.org/wiki/Key_derivation_function), специально разработанной для хеширования паролей, такой как scrypt (<https://oreil.ly/m5mge>) или Argon2 (https://oreil.ly/_yS2Q). В отличие от все еще распространенных подходов к хешированию паролей, таких как SHA-1 с солью, эти функции защищают от распространенных атак при восстановлении паролей, от предварительно заполненных таблиц поиска и до недорогого специального оборудования. Даже при использовании надежного хеша пароля проверяйте, чтобы сами хеши были зашифрованы в состоянии покоя с использованием хорошо защищенного ключа. Это добавляет еще одно препятствие для потенциального злоумышленника.

² Подробнее о работе Google по внедрению ключей безопасности для себя и пользователей см.: Lang J. et al. Security Keys: Practical Cryptographic Second Factors for the Modern Web // Proceedings of the 20th International Conference on Financial Cryptography and Data Security, 2016. P. 422–440. <https://oreil.ly/bL9Fm>.

Постмортемы

Хорошая практика для работающих над инцидентом команд — ведение записей о работе, которые позже можно будет добавить в официальный постморт. Каждый постморт должен содержать список действий, направленных на устранение основных проблем, обнаруженных во время инцидента. Качественный постморт охватывает технологические проблемы, использованные злоумышленником, и распознает возможности для улучшения обработки инцидентов. Кроме того, вы должны задокументировать временные рамки и усилия, связанные с этими действиями, и решить, какие относятся к краткосрочным, а какие — к долгосрочным дорожным картам. Мы подробно рассказываем о безупречных постмортемах в главе 15 книги *Site Reliability Engineering* (<https://landing.google.com/sre/sre-book/chapters/postmortem-culture/>). Здесь мы приведем дополнительные вопросы безопасности.

- Каковы были основные факторы, способствовавшие инциденту? Есть ли похожие случаи и такие же проблемы в других местах в среде, которые можно решить?
- Какие процессы тестирования или аудита должны были обнаружить эти факторы раньше? Если их еще нет, можно ли создать такие процессы для отлавливания похожих факторов в будущем?
- Был ли этот инцидент обнаружен ожидаемым технологическим контролем (например, системой обнаружения вторжений)? Если нет, как можно улучшить системы обнаружения?
- Как быстро инцидент был обнаружен и на него отреагировали? Находится ли это в приемлемом диапазоне времени? Если нет, как можно улучшить время отклика?
- Достаточно ли были защищены важные данные от злоумышленника? Какие новые средства контроля нужно добавить?
- Могла ли команда восстановления эффективно использовать инструменты, включая управление версиями исходного кода, развертывание, тестирование, резервное копирование и службы восстановления?
- Какие обычные процедуры — процессы тестирования, развертывания или выпуска — команде пришлось обойти во время восстановления? Какое смягчение последствий вы можете применить прямо сейчас?
- Были ли какие-то изменения в инфраструктуре или инструментах временными мерами смягчения последствий, которые теперь нужно реорганизовать?
- Какие ошибки вы нашли и зафиксировали на этапах инцидента и восстановления, которые теперь нужно устранить?
- Какие передовые практики есть в отрасли и среди коллегиальных групп, которые могли бы помочь вам на любом этапе предотвращения, обнаружения или реагирования на атаку?

Мы рекомендуем составить четкий набор пунктов действий с четкими владельцами, отсортированных в порядке краткосрочных и долгосрочных инициатив. Краткосрочные инициативы, как правило, просты, не требуют много времени для реализации и решают проблемы относительно небольших масштабов. Мы часто называем эти действия «низко висящими фруктами», потому что их можно легко распознать и устранить. Примеры: добавление двухфакторной аутентификации, сокращение времени на применение исправлений или настройка программы обнаружения уязвимостей.

Долгосрочные инициативы войдут в более масштабную программную стратегию расширенной программы по улучшению состояния безопасности организации. Эти действия обычно имеют более фундаментальное значение для работы систем и процессов. Например, создание специальной команды безопасности, развертывания магистрального шифрования или изменение вариантов выбора операционной системы.

В идеальном мире полный жизненный цикл происшествия — от момента взлома до улучшения безопасности — завершится до следующего происшествия. Однако помните, что период обычного устойчивого состояния после последнего инцидента также будет и обычным состоянием, предшествующим следующему. Это затишье — ваша возможность учиться, подстраиваться, выявлять новые угрозы и готовиться к следующему происшествию. Последняя часть этой книги посвящена улучшению и поддержанию состояния безопасности во время подобных штилей.

Примеры

В следующих рабочих примерах показана взаимосвязь между постмортемами, составлением чек-листов восстановления и восстановлением. Мы рассмотрим и то, как долгосрочные меры по снижению риска включаются в более масштабный план программы безопасности. Эти примеры не охватывают все аспекты реагирования на инциденты. Вместо этого мы сосредоточены на нахождении компромисса между избавлением от злоумышленника, смягчением его сиюминутных действий и внесением долгосрочных изменений.

Скомпрометированные облачные экземпляры

Сценарий: веб-пакет программного обеспечения, используемый вашей организацией для обслуживания пользовательского трафика с виртуальных машин (ВМ) в инфраструктуре облачного провайдера, имеет общую уязвимость ПО. Оппортунистический злоумышленник, использующий инструменты сканирования уязвимостей, обнаруживает ваши уязвимые веб-серверы и использует их. После захвата виртуальных машин злоумышленник применяет их для запуска новых атак.

В этом случае руководитель по восстановлению использует заметки команды расследования, чтобы определить, что команде нужно исправить ПО, повторно развернуть виртуальные машины и закрыть скомпрометированные экземпляры. RL определяет, что краткосрочное смягчение должно обнаружить и устранить связанные уязвимости. На рис. 18.2 вы видите возможный чек-лист восстановления для этого инцидента. Для краткости мы исключили конкретные команды и этапы выполнения — элементы, которые должны быть в ваших настоящих чек-листах восстановления.

INSTRUCTIONS:

- Highlight in **GREEN** when **COMPLETED**.
- Highlight in **ORANGE** TO-DO items.
- Highlight in **RED** if task cannot be completed; it must be replaced with an equivalent action.
- Highlight in **BLUE** if task will not be completed because it is operationally risky and adjustments need to be made.

Incident commander:

- Give a "hands off keyboards" briefing.
- Provide an overview of tasks and constraints.
- Notify all people involved of the exact time remediation will begin.

The remediation lead initiates execution of the following, in order:

1. Security specialists (or security-focused SREs or developers): Set up vulnerability scanning.
 - a. *This section should include specific details of setting up vulnerability scanning of other software used on the cloud instances to see if you need to apply additional fixes.*
2. Web developer team: Patch vulnerable software.
 - a. *This section should include specific details of where vulnerable software is sourced from, which versions should be deployed, and the steps (or commands) to do so.*
3. SRE team: Deploy new cloud instances (in parallel with compromised instances).
 - a. *This section should include specific instructions for how to deploy clean cloud instances with the upgraded software installed by default. Once tested and validated, they should be configured to serve user traffic again.*
4. SRE team: Disable compromised cloud instances.
 - a. *This section should include specific steps of how to shut down compromised cloud instances, and either archive them permanently or delete them so they can't be reused.*

Рис. 18.2. Чек-лист для восстановления:
скомпрометированные облачные экземпляры

После завершения восстановления разработанный всеми участниками реагирования постмортем инцидента указывает на необходимость в формальном средстве управления уязвимостями для своевременного упреждающего обнаружения и исправления известных проблем. Эта рекомендация включена в долгосрочную стратегию улучшения состояния безопасности организации.

Крупномасштабная фишинговая атака

Сценарий: в течение недели злоумышленник запускает кампанию фишинга паролей в вашей организации, не использующей двухфакторную аутентификацию. 70 % ваших сотрудников становятся жертвами этой атаки. Злоумышленник использует пароли для чтения электронной почты организации и передачи конфиденциальной информации прессе.

Как ИС, в этом сценарии вы столкнулись с рядом сложностей.

- Команда исследователей определяет, что злоумышленник при помощи паролей еще не пытался получить доступ к какой-либо системе, кроме электронной почты.
- VPN вашей организации и связанные с ней ИТ сервисы, включая управление облачными системами, используют независимые от сервиса электронной почты системы аутентификации. Однако многие сотрудники используют одинаковые пароли для этих систем.
- Злоумышленник заявил, что в ближайшие дни он снова атакует вашу организацию.

Вместе с руководителем по восстановлению вам приходится выбирать между быстрым избавлением от злоумышленника (удалив его доступ к электронной почте) и обеспечением того, чтобы злоумышленник не смог в это время получить доступ к другим системам. Это восстановление требует точного выполнения, для которого на рис. 18.3 представлен гипотетический чек-лист. Мы опускаем точные команды и процедуры для краткости, но ваш настоящий чек-лист должен их учитывать.

После завершения восстановления официальный постмортем выявит необходимость в следующем:

- более широкое использование двухфакторной аутентификации в критически важных системах связи и инфраструктуры;
- решение для единого входа;
- обучение сотрудников противодействию фишинговым атакам.

Ваша организация также отмечает необходимость в специалисте по информационной безопасности, который консультировал бы вас по стандартным передовым практикам. Она включена в долгосрочную стратегию улучшения состояния безопасности организации.

INSTRUCTIONS:

- Highlight in **GREEN** when **COMPLETED**.
- Highlight in **ORANGE** TO-DO items.
- Highlight in **RED** if task cannot be completed; it must be replaced with an equivalent action.
- Highlight in **BLUE** if task will not be completed because it is operationally risky and adjustments need to be made.

Incident commander

- Give a "hands off keyboards" briefing.
- Provide an overview of tasks and constraints.
- Notify all people involved of the exact time remediation will begin.

The remediation lead initiates execution of the following, in order:

1. System administrators/SREs: Implement infrastructure for two-factor authentication for email.
 - a. *This section should include specific details of how to enroll all employees in two-factor authentication for email.*
2. Email SREs: Lock email accounts.
 - a. *This section should include specific details of how to lock the email accounts of employees until they are reset and use two-factor authentication. Due to the ongoing activity of the attacker, you may choose to lock all the accounts at once rather than providing employees with a rolling window.*
3. IT staff (such as Helpdesk): Enact adoption of two-factor authentication and password resets.
 - a. *This section should include specific details of how IT staff should help users change their email, VPN, and cloud-based passwords, and begin using two-factor authentication for email. Depending on the size of the organization, this step might involve an all-hands staff meeting.*
4. System administrators/SREs: Adopt two-factor authentication for VPN and cloud systems.
 - a. *This section should include specific details for implementing two-factor authentication for email, VPN, and cloud services.*

Рис. 18.3. Чек-лист для восстановления: крупномасштабная фишинговая атака

Целенаправленная атака, требующая сложного восстановления

Сценарий: злоумышленник без вашего ведома имеет доступ к вашим системам более месяца. Он украл ключи SSL, используемые для защиты связи с клиентами через Интернет, внедрил бэкдор в исходный код для получения 2 центов от каждой транзакции и отслеживает электронные письма руководителей организации.

Как IC, вы столкнулись с рядом сложностей.

- Команда исследователей не знает, как началась атака и какой способ доступа используется злоумышленником.
- Злоумышленник может контролировать действия руководителей и команды реагирования на инциденты.

- Злоумышленник может изменить исходный код и развернуть его в производственных системах.
- У злоумышленника есть доступ к инфраструктуре, где хранятся ключи SSL.
- Зашифрованные веб-коммуникации с клиентами потенциально скомпрометированы.
- Злоумышленник украл деньги.

Мы не будем подробно обзирать примеры чек-листов восстановления для этой сложной атаки. Вместо этого сосредоточимся на одном очень важном аспекте инцидента: необходимости распараллеливания восстановления. Совместно с RL создайте чек-лист восстановления для каждой из этих проблем для лучшей координации шагов восстановления. В течение этого времени вам нужно также оставаться на связи с командой исследователей, которая будет получать больше сведений о предыдущих и текущих действиях злоумышленника.

Например, вам понадобится подключить некоторые или все команды восстановления из следующего списка (вы можете подумать и о других командах).

Команда электронной почты

Она отключит доступ злоумышленника к системе электронной почты и будет гарантировать, что он не сможет повторно войти в систему. Для этого может потребоваться сбросить пароли, внедрить двухфакторную аутентификацию и т. д.

Команда разработчиков исходного кода

Эта команда определит, как прервать доступ к исходному коду, очистить затронутые файлы и повторно развернуть очищенный код в производственных системах. Для выполнения этого команде нужно знать, когда злоумышленник изменил исходный код в рабочей среде и есть ли безопасные версии. Эта команда должна убедиться, что злоумышленник не внес изменения в постоянную историю управления версиями и не может внести дополнительные изменения в исходный код.

Команда ключей SSL

Эта команда определит, как безопасно разворачивать новые ключи SSL в инфраструктуре веб-обслуживания. Для этого они должны знать, как злоумышленник получил доступ к ключам и как предотвратить несанкционированный доступ в будущем.

Команда обработки данных клиентов

Они определяют, к какой информации о клиенте злоумышленник потенциально имел доступ и нужно ли проводить исправление, например, изменять пароли клиента или сбрасывать сеанс. Эта команда по исправлению может быть тесно связана с любыми дополнительными проблемами, поднятыми службой поддержки, юристами и другими связанными сотрудниками.

Команда синхронизации

Эта команда будет оценивать последствия извлечения 2 центов из каждой транзакции. Нужно ли добавить записи в БД для учета долгосрочного ведения финансового журнала? Эта команда может быть тесно связана с любыми дополнительными проблемами, поднятыми сотрудниками отдела финансов и юристами.

В сложных случаях, подобных этому, часто нужно выбирать между несколькими неидеальными вариантами во время восстановления. Рассмотрим команду, которой необходимо отключить доступ к электронной почте вашей компании. У них может быть несколько вариантов: отключение системы электронной почты, блокировка всех учетных записей или параллельное создание новой системы электронной почты. Каждый из них будет как-то влиять на организацию, будет предупреждать злоумышленника, при этом необязательно учитывая, как он получил доступ. Команды по восстановлению и расследованию должны тесно сотрудничать, чтобы найти путь продвижения вперед, соответствующий ситуации.

Резюме

Восстановление — особый и важный шаг при реагировании на инциденты, для которого нужна команда специалистов, хорошо знающих свою предметную область. Планируя восстановление, учитывайте много аспектов: как отреагирует противник? Кто и когда выполняет определенные действия? Что нужно сообщить пользователям? Процесс восстановления должен быть распараллелен, тщательно спланирован и синхронизирован с командиром инцидента на происшествии и командой исследователей. Ваши усилия всегда должны быть направлены на определенную цель: избавиться от злоумышленника, исправить последствия взлома и повысить общую безопасность и надежность систем. Каждая атака должна стать уроком, позволяющим вам улучшить состояние безопасности вашей организации в будущем.

ЧАСТЬ V

Организация и культура

Хотя методы проектирования из этой книги помогут вашей организации создать безопасные и надежные системы, ваши усилия будут эффективны, только если все сотрудники организации вносят свой вклад в поддержание культуры безопасности и надежности. Культура — мощный и уникальный определяющий компонент каждой организации, и вам не следует недооценивать ее роль в вашей способности инициировать изменения.

Часть V этой книги посвящена культурным аспектам реализации представленных ранее подходов. Chrome был одним из первых продуктов Google, которому была назначена специальная команда, активно продвигающая культуру безопасности. Мы начнем с ее тематического исследования, сосредоточив внимание на ее роли в популярности и успехе Chrome. В главе 20 мы попробуем убедить вас в том, что все сотрудники организации ответственны за безопасность и надежность. Специалисты по безопасности должны внедрять технологии безопасности, требующие специальных знаний, обучать лучшим практикам и политикам и разрабатывать их. Глава 21 завершает книгу обсуждением стратегий формирования здоровой культуры безопасности и надежности.

Пример из практики: команда безопасности Chrome

*Париса Тебриз с Сюзанной Ландерс
и Полом Бланкиншипом*

В первые годы существования Google работа по обеспечению безопасности, включая безопасность, ориентированную на продукт, была полностью централизованной. Chrome был одним из первых браузеров, созданных с ориентацией в первую очередь на безопасность. В этой главе описывается эволюция команды безопасности Chrome, основные принципы, разработанные нами, и некоторые идеи о масштабировании безопасности на всю организацию.

Возникновение и развитие команды

В 2006 году в Google была сформирована команда, целью которой было создание браузера для Windows с открытым исходным кодом менее чем за два года. Этот браузер должен был быть безопаснее, быстрее и стабильнее, чем имеющиеся на рынке альтернативы. Цель была амбициозной и приводила к уникальным проблемам безопасности.

- Современные браузеры по своей сложности аналогичны операционной системе, и большая часть их функциональных возможностей считается критически важной для безопасности.
- ПО на стороне клиента и Windows отличалось от большинства существующих в то время продуктов и системных предложений от Google, поэтому центральный отдел безопасности Google обладал лишь ограниченным количеством передаваемых знаний в области безопасности.

- Проект должен был использовать в основном открытый исходный код, поэтому у него были уникальные требования к разработке и использованию и он не мог полагаться на методы или решения корпоративной безопасности Google.

Этот проект браузера был запущен в 2008 году под названием Google Chrome. С тех пор Chrome был признан проектом, переопределившим стандарт безопасности в Интернете (<https://oreil.ly/Rb6TJ>), и стал одним из самых популярных браузеров в мире.

За последнее десятилетие организация Chrome, ориентированная на безопасность, прошла четыре основных этапа эволюции:

Команда v0.1

Chrome формально не создавал команду безопасности вплоть до официального запуска в 2008 году, но опирался на опыт, распространенный в команде разработчиков, наряду с консультациями центральной службы безопасности Google и сторонних поставщиков безопасности. Первоначальный запуск не обошелся без недостатков безопасности — в течение первых двух недель после выхода в свет было обнаружено несколько критических переполнений буфера (<https://oreil.ly/GxjAV>)! Многие ошибки при первоначальном запуске соответствуют шаблону недостатков, возникающих в результате того, что разработчики в условиях нехватки времени пытаются выпустить код на C++, оптимизированный по производительности. Были ошибки и при реализации браузерного приложения и веб-платформы. Обнаружение ошибок, их исправление, написание тестов для предотвращения регрессий и разработка — часть обычного процесса развития команды.

Команда v1.0

Через год после выпуска публичной бета-версии, когда браузер стал популярнее у пользователей, была создана специальная команда безопасности Chrome. Изначально она состояла из инженеров центральной команды безопасности Google и новых сотрудников. В работе они использовали рекомендации и нормы, установленные в Google, и добавили новые перспективы и опыт извне.

Команда v2.0

В 2010 году Chrome запустил программу вознаграждения за найденные уязвимости (Vulnerability Reward Program, VRP) (<https://oreil.ly/cMY8z>) в знак признания вклада более широкого сообщества исследователей безопасности. Множество реакций на объявление VRP привнесло ценный вклад в решение вопросов безопасности уже в первые дни после запуска. Изначально Chrome был основан на WebKit — движке для отображения HTML-страниц

с открытым исходным кодом. Ранее он не был тщательно проанализирован с точки зрения безопасности, поэтому одной из первых задач команды было реагирование на огромный поток внешних сообщений об ошибках. В то время команда разработчиков Chrome была небольшой и еще не была знакома со всей кодовой базой WebKit. Поэтому команда безопасности обнаружила, что наиболее целесообразным подходом к устранению уязвимости часто было просто погрузиться в работу, накопить опыт в кодовой базе и исправить многие ошибки самостоятельно!

Эти ранние решения сильно повлияли на дальнейшую культуру команды. Команда безопасности стала не просто кучкой изолированных консультантов или аналитиков, а гибридной инженерной командой экспертов по безопасности. Одно из самых сильных преимуществ этого подхода — уникальное и практическое понимание того, как внедрить безопасную разработку в повседневные процессы каждого инженера, работающего над Chrome.

Команда v3.0

К 2012 году использование Chrome снова возросло, как и амбиции команды — и внимание со стороны злоумышленников. Для упрощения масштабирования безопасности в растущем проекте Chrome основная команда безопасности создала, отредактировала и опубликовала набор базовых принципов безопасности (https://oreil.ly/aeU6_).

В 2013 году после привлечения менеджера по инжинирингу и найма множества специалистов по безопасности команда провела выездное совещание, чтобы обсудить проделанную работу, определить свою миссию и провести мозговой штурм насчет более крупных проблем безопасности, которые нужно было решить. В результате мы получили заявление, в котором была сформулирована общая цель команды: предоставить пользователям Chrome максимально безопасную платформу для навигации и в целом повысить безопасность в Интернете.

На этой выездной сессии 2013 года в процессе коллективного мозгового штурма все разработчики записывали свои идеи на стикерах. В результате было определено несколько приоритетных направлений работы.

Обзоры безопасности

Команда безопасности регулярно консультируется с другими командами, чтобы помочь в проектировании и оценке безопасности новых проектов и проверить чувствительные к безопасности изменения в кодовой базе. Проверки безопасности — общая ответственность команды, они способствуют передаче знаний. Команда масштабирует эту работу, создавая

документацию, проводя обучение по безопасности и выступая в качестве владельцев (<https://oreil.ly/t4EoT>) критически важных для безопасности частей кода Chrome.

Поиск и исправление ошибок

Благодаря миллионам строк чувствительного к безопасности кода и сотням разработчиков по всему миру, постоянно вносящих изменения, команда вкладывается в разные подходы для более быстрого нахождения и исправления ошибок.

Архитектура и смягчение последствий эксплойтов

Признавая, что невозможно предотвратить все ошибки безопасности, команда инвестирует в проекты безопасного проектирования и архитектуры для снижения влияния любой отдельной ошибки. Chrome доступен в популярных настольных и мобильных ОС (например, Microsoft Windows, macOS, Linux, Android и iOS), которые постоянно развиваются, поэтому нужны постоянные инвестиции и стратегии для конкретных ОС.

Удобная безопасность

Несмотря на уверенность команды в неуязвимости ПО Chrome и систем, используемых для его создания, все равно важно учитывать, как и когда пользователи (которые совершают ошибки) сами принимают решения, чувствительные к безопасности. Учитывая широкий спектр цифровой грамотности среди пользователей браузеров, команда инвестирует в то, чтобы помочь им принимать безопасные решения при просмотре веб-страниц, делая безопасность удобнее.

Безопасность веб-платформы

Помимо Chrome, команда работает над улучшением безопасности для разработчиков, создающих веб-приложения, чтобы любому было проще создавать безопасные приложения в Интернете.

Выявление ответственных руководителей для каждой области деятельности и специально назначенных менеджеров помогло создать более масштабируемую организацию команды. Важно, что это выделение руководителей улучшает гибкость, способствует обмену информацией в команде и сотрудничеству на проектах. Благодаря такому подходу ни один человек или область фокуса не изолируется от других.

Поиск и удержание хороших людей — сотрудников, заботящихся о миссии команды и ее основных принципах и хорошо сотрудничающих с другими, — имеет

решающее значение. Каждый член команды вносит свой вклад в наем новых сотрудников, проведение интервью и дает постоянную мотивирующую обратную связь для товарищей по команде. Наличие правильных людей важнее любых организационных деталей.

Что касается фактического поиска кандидатов, команда активно использует свои личные связи и постоянно работает над их развитием и расширением. Мы перевели множество стажеров в ряды штатных сотрудников. Иногда мы обращаемся к людям, выступающим на конференциях или публикующим работы, показывающие, что они заботятся о сети и создании продуктов для масштабирования. Одно из преимуществ открытой работы в том, что мы можем поделиться с возможными кандидатами подробностями работы нашей команды и недавними достижениями при помощи вики-страницы разработчиков Chromium (<https://www.chromium.org/>), чтобы те могли быстрее вникнуть в суть работы, проблем и культуры команды.

Важно, что мы рассматривали и людей, *заинтересованных* в безопасности, чей опыт или достижения относились к другим сферам. Например, мы наняли одного инженера с опытом работы в сфере SRE — он полностью разделял нашу миссию по обеспечению безопасности людей и был заинтересован в изучении этой темы. Такое разнообразие опыта и перспектив — ключевой фактор успеха команды.

В следующих разделах мы расскажем о том, как основные принципы безопасности Chrome применялись на практике. Эти принципы остаются такими же актуальными для Chrome сегодня, как и сразу после их определения в 2012 году.

Безопасность — командная ответственность

Одна из основных причин того, почему Chrome уделяет такое пристальное внимание безопасности, в том, что мы принимаем безопасность как основной принцип продукта и создаем культуру, где безопасность считается командной ответственностью.

Хотя у команды безопасности Chrome есть привилегия почти полностью сосредоточиться на безопасности, члены команды признают, что они никогда не смогут отвечать за безопасность всего Chrome. Вместо этого они усиленно внедряют знания о безопасности и лучшие практики в повседневные действия и процессы всех, кто работает над продуктом. Соглашения о проектировании системы направлены на создание легкого, быстрого, хорошо освещенного и безопасного пути. Для такого результата часто требовались дополнительные

усилия, но это привело к более эффективному сотрудничеству в долгосрочной перспективе.

Один из практических примеров — подход команды к ошибкам безопасности. Все инженеры, включая членов команды безопасности, исправляют ошибки и пишут код. Если команды безопасности заняты только поиском ошибок и сообщением о них, они могут перестать осознавать, как трудно писать код без ошибок или исправлять их. Это также помогает смягчить иногда возникающее разделение членов команды на «наших» и «ваших», если инженеры по безопасности не участвуют в традиционных задачах.

По мере выхода команды за рамки первоначальной борьбы с уязвимостями мы стали разрабатывать более активный подход к безопасности. Это означало вложение сил в создание и поддержание инфраструктуры нечеткого тестирования и инструментария для разработчиков, позволяющих быстрее и проще выявлять изменения, приводящие к ошибкам, и откатывать или исправлять их. Чем быстрее разработчик сможет выявить новую ошибку, тем легче ее исправить и тем меньше она повлияет на конечных пользователей.

Помимо создания полезного инструментария для разработчиков, команда вводит положительные стимулы для проведения нечеткого тестирования инженерами команды. Ежегодно проводятся соревнования по нечеткому тестированию с призами и выпускаются руководства по нему (<https://oreil.ly/alX7Q>), помогающие любому инженеру научиться тестировать. Организация мероприятий и возможность внесения своего вклада помогает людям понять, что для повышения безопасности им не нужно быть «экспертами по безопасности». Инфраструктура нечеткого тестирования в Chrome начиналась с малого — одного компьютера под рабочим столом инженера. Начиная с 2019 года инфраструктура поддерживает фаззинг в Google и по всему миру¹. В дополнение к нечеткому тестированию команда безопасности создает и поддерживает защищенные базовые библиотеки (библиотеку безопасных чисел), так что по умолчанию любой может вносить изменения безопасным способом.

Члены команды безопасности часто отправляют бонусы (<https://oreil.ly/ahPxg>) или положительные отзывы отдельным людям или их менеджерам, если замечают внедрение строгих мер безопасности. Работа по обеспечению безопасности иногда остается незамеченной или невидимой для конечных пользователей, поэтому дополнительные усилия по ее распространению напрямую или в соответствии с карьерными целями помогают создать положительные стимулы для повышения безопасности.

¹ В начале 2019 года Google открыл исходный код ClusterFuzz (<https://oreil.ly/3PrQI>) сервера нечеткого тестирования для своего сервиса OSS-Fuzz (<https://oreil.ly/mIjbw>).

Независимо от выбранной тактики, если организации еще не рассматривают безопасность как основную ценность и общую ответственность, сначала нужно предоставить более серьезные размышления и обсуждения для доказательства важности безопасности и надежности, чтобы достичь их основных целей.

Помогаем пользователям безопасно перемещаться в Интернете

Безопасность не должна зависеть от опыта конечного пользователя. В любом продукте с обширной клиентской базой нужно сбалансировать удобство использования, возможности и другие бизнес-ограничения. В основном Chrome стремится сделать безопасность почти незаметной для пользователя. Мы осуществляем прозрачные обновления, стремимся к безопасным значениям по умолчанию и постоянно пытаемся сделать безопасные решения простыми, чтобы помочь пользователям избежать небезопасных.

На этапе команды v3.0 мы коллективно признали наличие ряда проблем с практической безопасностью — проблем, возникающих из-за взаимодействия людей с ПО. Например, мы знали, что пользователи становятся жертвами социальной инженерии и фишинговых атак, и были обеспокоены эффективностью предупреждений безопасности в Chrome. Мы хотели решить эти проблемы, но у нас было недостаточно опыта в области программного обеспечения, ориентированного на человека. Мы решили, что нужно стратегически привлекать опытных специалистов извне для более эффективной экспертизы безопасности. По счастливой случайности мы нашли внутреннего кандидата, заинтересовавшегося новой ролью.

В то время этот кандидат относился к числу научных сотрудников, а у Chrome не было опыта приема на работу таких людей. Мы убедили руководство нанять кандидата, несмотря на предварительные оговорки, подчеркнув, что академические знания кандидата будут преимуществом для команды и необходимы для расширения существующего набора навыков. Тесное сотрудничество с UX-командой, с которой время от времени возникали противоречия по вопросам безопасности, и новое дополнение к команде позволили продолжить создание новой области деятельности Chrome — практическую безопасность. В итоге мы наняли дополнительных дизайнеров и UX-исследователей, которые помогли нам более глубоко понять потребности пользователей в безопасности и конфиденциальности. Мы узнали, что эксперты по безопасности, несмотря на глубокое понимание работы компьютерных систем и сети, часто не видят многих проблем, с которыми сталкиваются пользователи.

Скорость имеет значение

Безопасность пользователя зависит от быстрого обнаружения ее недостатков и предоставления исправлений до того, как этим смогут воспользоваться злоумышленники. Одна из важнейших функций безопасности Chrome — быстрые автоматические обновления. С первых дней команда безопасности тесно сотрудничала с менеджерами технических программ (technical program manager, ТРМ), которые определили процесс выпуска Chrome (<https://oreil.ly/J1Vao>) и управляли качеством и надежностью каждой новой версии. Отвечающие за выпуски ТРМ измеряют частоту сбоев, гарантируют своевременное исправление высокоприоритетных ошибок, аккуратно и поэтапно продвигают выпуски вперед, оказывают давление на инженеров, когда процесс слишком ускоряется, и стремятся создавать выпуски, повышающие надежность или безопасность, для пользователей так быстро, насколько это допустимо.

Ранее мы использовали соревнования по взлому Pwn2Own (<https://oreil.ly/5PwMU>) и более поздний Pwnium (<https://oreil.ly/bQthO>) в качестве форсирующих функций, чтобы проверить наши возможности по выпуску и развертыванию критических исправлений безопасности менее чем за 24 часа. Это стало возможным благодаря надежному сотрудничеству, командной взаимопомощи и поддержке со стороны команд ТРМ выпуска. Хотя мы продемонстрировали эту возможность, нам редко приходилось ее использовать благодаря вложениям Chrome в защиту в глубину.

Проектирование для защиты в глубину

Независимо от скорости команды при обнаружении и исправлении любой отдельной ошибки безопасности в Chrome они неизбежно возникают, особенно если учесть недостатки безопасности C++ и сложность браузера. Злоумышленники постоянно совершенствуются, поэтому Chrome регулярно инвестирует в разработку методов предотвращения эксплойтов и архитектуры, помогающей избежать единичных точек отказа. Команда создала цветовую диаграмму компонентов с разным уровнем риска (<https://oreil.ly/ssfOj>), чтобы каждый мог иметь представление об архитектуре безопасности Chrome и разных уровнях защиты для обоснования своей работы.

Один из лучших примеров защиты в глубину на практике — постоянные инвестиции в возможности песочницы. Изначально Chrome был запущен с многопроцессорной архитектурой и процессами рендеринга в песочнице. Благодаря этому вредоносный веб-сайт не позволял захватить компьютер пользователя полностью. В то время это было большим достижением в архитектуре браузера. В 2008 году самой большой интернет-угрозой была вредоносная веб-страница,

использующая взломанный браузер для установки вредоносного ПО на компьютер пользователя. Архитектура Chrome успешно взяла эту проблему под контроль.

Но возможности использования компьютеров расширялись, и с растущей популярностью облачных вычислений и веб-сервисов все больше конфиденциальных данных переводилось в онлайн. Это привело к тому, что кража данных между веб-сайтами может стать такой же важной целью, как и взлом локальной машины. Было неясно, когда фокус атак сместится со «взлома рендеринга, устанавливающего вредоносное ПО» на «взлом рендеринга, крадущего межсайтовые данные», но команда знала, что этот шаг неизбежен. Поэтому в 2012 году мы приступили к реализации Site Isolation (https://oreil.ly/_mfzd) — проекта по улучшению состояния песочницы для изоляции отдельных сайтов.

Изначально команда предсказывала, что на реализацию проекта Site Isolation уйдет год, но мы ошиблись более чем в пять раз! Подобные ошибки оценивания обычно вызывают пристальное внимание к проекту со стороны высшего руководства — на это есть веские причины. Команда регулярно рассказывала руководству и заинтересованным сторонам о значимости проекта, его ходе и причинах того, почему работы оказалось больше, чем предполагалось. Члены команды демонстрировали положительное влияние на общее состояние кода Chrome. Все это позволяло команде доносить ценность проекта до заинтересованных сторон на протяжении многих лет, вплоть до его возможного публичного запуска (кстати, Site Isolation частично смягчает уязвимости упреждающего выполнения (<https://oreil.ly/EBJb9>), обнаруженные в 2018 году).

Менее вероятно, что защита в глубину приведет к видимым для пользователя изменениям (если все сделано правильно), для руководства еще более важно активно управлять этими проектами, распознавать их и инвестировать в них.

Будьте открыты и вовлекайте сообщество

Прозрачность изначально была основной ценностью для команды Chrome. Мы не преуменьшаем влияние на безопасность и не скрываем уязвимости с помощью незаметных исправлений, потому что это плохо влияет на обслуживание пользователей. Вместо этого мы даем пользователям и администраторам информацию, нужную для точной оценки риска. Команда безопасности публикует информацию о том, как она решает проблемы безопасности (<https://oreil.ly/pA9ba>), раскрывая все уязвимости, исправленные в Chrome и его зависимостях — неза-

висимо от того, обнаружены они внутренне или внешне — и, по возможности, перечисляет все исправленные проблемы безопасности в примечаниях к выпуску (<https://oreil.ly/C-Qm>).

Еще мы публикуем ежеквартальные сводки о том, что делаем публично (<https://oreil.ly/teoUI>), и взаимодействуем с пользователями через электронную рассылку (security-dev@chromium.org), с помощью которой каждый может отправлять нам идеи, вопросы или участвовать в дискуссиях. Мы активно поощряем людей, которые делятся своей работой на конференциях, собраниях по безопасности или в социальных сетях. Мы сотрудничаем с более широким сообществом безопасности через программу вознаграждения за найденные уязвимости в Chrome и спонсируем конференции по безопасности. Chrome стал безопаснее благодаря вкладу многих не входящих в команду людей, и мы делаем все возможное, чтобы высоко оценить и поощрить их, обеспечив надлежащее признание и выплату денежного вознаграждения.

В Google мы организовали ежегодное выездное собрание для связи команды безопасности Chrome с центральной командой безопасности и другими внутренними командами (например, командой безопасности Android). Мы также призываем энтузиастов безопасности в Google выполнять до 20 % своей работы в Chrome (и наоборот) или находить возможности сотрудничества с научными исследователями в проектах Chromium.

Благодаря работе в открытой среде команда может делиться наработками, достижениями и идеями и получать обратную связь или поддерживать сотрудничество за пределами Google. Все это делается для продвижения общего понимания безопасности браузера и сети. Многолетние усилия команды по расширению внедрения HTTPS (см. главу 7) — один из примеров того, как информирование и взаимодействие с более обширным сообществом может привести к изменению экосистемы.

Резюме

Команда, работающая над Chrome, определила безопасность как основной принцип на ранних этапах проекта и расширила инвестиции и стратегию по мере роста команды и пользовательской базы. Команда безопасности Chrome была официально создана всего через год после запуска браузера, и со временем ее роли и обязанности определились более четко. Команда разработчиков сформулировала миссию и набор основных принципов безопасности и определила главные направления своей работы.

Определение безопасности как коллективной ответственности, прозрачность и взаимодействие с сообществами вне Chrome помогли создать и развить ориентированную на безопасность культуру. Стремление к быстрым инновациям привело к созданию динамичного продукта и способности гибко реагировать на меняющийся ландшафт. Тщательное проектирование для защиты в глубину помогло оградить пользователей от разовых ошибок и новых атак. Учитывая человеческие аспекты безопасности, от опыта конечного пользователя до процесса найма сотрудников, команда расширила понимание безопасности и решила более сложные задачи. Готовность к трудностям и способность учиться на ошибках позволила нам работать над созданием пути, безопасного по умолчанию.

Понимание ролей и обязанностей

*Хизер Адкинс, Сайрус Везуна,
Хантер Кинг, Феликс Греберт
и Дэвид Чаллонер с Сюзанной Ландерс,
Стивеном Роддисом, Сергеем Симаковым,
Шилайей Нукала, Джанет Вонг,
Дугласом Колишем, Бетси Бейер
и Полом Бланкиншипом*

В этой главе мы ответим на вопрос, кто должен работать над безопасностью. Мы бросаем вызов распространенному мифу о том, что безопасность — это тема только для экспертов. Вместо этого мы утверждаем, что за безопасность отвечает каждый, хотя иногда вам может понадобиться отдельный специалист. Мы также рассматриваем роль экспертов по безопасности в мире, где она тесно интегрирована в жизненный цикл систем и обрабатывается другими специалистами. В заключение мы рассмотрим несколько специализированных вариантов поддержки организации, особенно с учетом ее роста с течением времени.

Как многократно подчеркивается в книге, создание систем — это *процесс*, а процессы повышения безопасности и надежности зависят от людей. Это значит, что для построения безопасных и надежных систем нужно решить два вопроса.

- Кто отвечает за безопасность и надежность в организации?
- Как усилия по обеспечению безопасности и надежности интегрированы в организацию?

Ответы на эти вопросы по большей части зависят от целей и культуры вашей организации (тема следующей главы). В следующих разделах вы увидите некоторые общие рекомендации о том, как размышлять об этих вопросах, и узнаете, как в Google со временем обратились к ним.

О НАДЕЖНОСТИ

Связанные с надежностью роли и обязанности рассматриваются в других источниках, поэтому большая часть этой главы содержит сопоставимые сведения о безопасности. Для получения дополнительной информации о разных способах, которыми SR-инженеры могут обеспечивать надежность продуктов компании, и о том, как эта модель может развиваться с течением времени, см. главу 32 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/evolving-sre-engagement-model/>) и главы 18 (<https://landing.google.com/sre/workbook/chapters/engagement-model/>), 19 (<https://landing.google.com/sre/workbook/chapters/reaching-beyond/>) и 20 (<https://landing.google.com/sre/workbook/chapters/team-lifecycles/>) книги Site Reliability Workbook.

Кто отвечает за безопасность и надежность

Кто работает над безопасностью и надежностью в организации? Мы считаем, что они должны быть интегрированы в жизненный цикл систем. Поэтому они становятся ответственностью каждого сотрудника. Мы хотели бы опровергнуть миф о том, что в организациях за это должны отвечать только эксперты.

Если надежность и безопасность передаются изолированной группе людей, которые не могут поручить другим группам вносить изменения, связанные с безопасностью, одинаковые сбои будут повторяться неоднократно. Этот труд может стать сизифовым — повторяющимся и непродуктивным.

Мы призываем организации сделать надежность и безопасность ответственностью *каждого*: разработчиков, SR-инженеров, инженеров по безопасности, инженеров-тестировщиков, технических руководителей, продакт-менеджеров, технических писателей, руководителей высшего звена и т. д. Таким образом, нефункциональные требования из главы 4 становятся фокусом для всей организации на протяжении всего жизненного цикла системы.

АНАЛОГИЯ ДЛЯ СИСТЕМ БЕЗОПАСНОСТИ И НАДЕЖНОСТИ

На примере современных автомобилей можно провести хорошую аналогию того, как безопасность и надежность обеспечиваются еще на этапе проектирования и поставки продукта. Каждая деталь автомобиля должна быть как безопасной, так и надежной. Сиденья спроектированы так, чтобы выдержать удар, фары расположены под углом, чтобы не ослеплять встречные автомобили. Ремни безопасности должны выдерживать тысячи пристегиваний. Лобовое стекло должно защищать от любых погодных условий, а фары — всегда должны включаться при необходимости. Цифровые системы автомобиля должны быть защищены аналогичным образом. И за это должен отвечать каждый, кто принимает участие в создании автомобиля, а не только специалисты по безопасности и надежности.

Роли специалистов

Если за безопасность и надежность отвечает каждый, то вы можете спросить: какова роль специалиста по безопасности или эксперта по надежности? Согласно одной из точек зрения, инженеры, создающие систему, должны в первую очередь сосредоточиться на ее основной функциональности. Например, разработчики могут сфокусироваться на создании набора критических пользовательских путей для мобильного приложения. В дополнение к работе команды разработчиков инженер по вопросам безопасности рассмотрит приложение с точки зрения злоумышленника, стремящегося подорвать его безопасность. Специалист, ориентированный на надежность, поможет понять цепочки зависимостей и на их основе определить, какие показатели следует измерять, чтобы получить довольных клиентов и систему, соответствующую SLA. Такое разделение труда принято во многих средах разработки, но важно, чтобы эти люди работали вместе, а не изолированно.

Для развития этой идеи, в зависимости от сложности системы, организации могут понадобиться люди со специальным опытом для принятия решений в особо сложных ситуациях. Невозможно создать абсолютно надежную или абсолютно безопасную систему, устойчивую к любым атакам. Поэтому рекомендации экспертов помогут в управлении командами разработчиков. В идеале оно должно быть интегрировано в жизненный цикл разработки. Такая интеграция может принимать различные формы, и специалисты по безопасности и надежности должны работать напрямую с разработчиками или другими сотрудниками, консультируясь с ними на каждом этапе жизненного цикла для улучшения систем¹. Например, консультация по безопасности может проходить в несколько этапов.

1. *Обзор проектирования безопасности* в начале проекта, чтобы определить, как интегрирована безопасность.
2. *Постоянные аудиты безопасности*, чтобы убедиться, что продукт создан правильно, в соответствии со спецификациями безопасности.
3. *Тестирование*, чтобы увидеть, какие уязвимости может обнаружить независимый человек.

Эксперты по безопасности должны быть ответственны за внедрение специфических технологий безопасности, требующих специальных знаний. Криптография — базовый пример: «не создавай собственную криптографию» — распространенная в отрасли поговорка, призванная отговорить предприимчивых разработчиков от реализации своих решений. Реализация криптографии

¹ В главе 18 книги Site Reliability Workbook (<https://landing.google.com/sre/workbook/chapters/engagement-model/>) описано, какую ценность опытный SR-инженер представляет на протяжении всего жизненного цикла продукта.

в библиотеках или на оборудовании должна осуществляться на усмотрение экспертов. Если вашей организации нужно предоставлять безопасные сервисы (например, веб-сервис через HTTPS), используйте принятые в отрасли и проверенные решения, а не пытайтесь написать свой алгоритм шифрования. Специалисты по безопасности могут понадобиться для реализации таких типов очень сложной инфраструктуры безопасности, как пользовательские системы аутентификации, авторизации и аудита (authentication, authorization, and auditing, AAA), или новых безопасных фреймворков для предотвращения распространенных уязвимостей безопасности.

ОЦЕНКА РИСКА БЕЗОПАСНОСТИ И НАДЕЖНОСТИ

Обоснованные рекомендации по безопасности и надежности полезны при принятии решений о рисках. Допустим, разработчики, работающие над проектом, не успели встроить желаемую защиту или не подумали об изящной деградации, но должны скоро выпустить продукт или систему. Инженер по безопасности и SR-инженер помогут понять, что может произойти, если система будет запущена в текущем состоянии. Смогут ли противники атаковать уязвимую систему? Выйдет ли из строя глобальная система, если пользовательский трафик для определенной страны будет выше ожидаемого? Какова вероятность любого из этих событий? Располагает ли организация временными средствами для устранения потенциальных проблем, которые могут возникнуть? Опытные инженеры по безопасности и SR-инженеры могут дать ценные советы в таких ситуациях.

У инженеров по надежности (в том числе SR-инженеров) больше возможностей для разработки централизованной инфраструктуры и автоматизации всей организации. В главе 7 книги *Site Reliability Engineering* (<https://sre.google/sre-book/automation-at-google/>) обсуждается ценность и эволюция горизонтальных решений и показано, как критическое ПО, позволяющее разрабатывать и запускать продукты, может развиваться до уровня платформы.

Специалисты по безопасности и надежности могут разработать лучшие практики, политики и тренинги с учетом рабочих процессов вашей организации. Благодаря этим инструментам разработчики смогут применять рекомендации и внедрять эффективные методы обеспечения безопасности и надежности. Все участники должны стремиться к созданию «коллективного разума» для организации, постоянно отслеживая новые события в отрасли и повышая осведомленность (см. подраздел «Культура осведомленности» в главе 21). Работая в этом направлении, специалист может помочь организации постепенно становиться безопаснее и надежнее. Например, у Google есть образовательные программы, ориентированные на SRE и безопасность, дающие базовый уровень знаний для всех новых сотрудников в конкретных ролях. Материалы курса доступны для всей компании, а еще мы предлагаем сотрудникам множество курсов для самостоятельного обучения по этим темам.

Что такое экспертиза в безопасности

Любой, кто пытался нанять специалистов по безопасности в компанию, знает, что это сложная задача. На что ориентироваться при найме, если вы не специалист по безопасности? Проведите аналогию с медработниками: все они имеют общее представление о здоровье человека, но многие специализируются на определенной области. Семейные врачи или врачи-терапевты обычно отвечают за первичную медицинскую помощь. В более серьезных случаях не обойтись без специалиста по неврологии, кардиологии или другой области. Аналогичным образом все специалисты по безопасности, как правило, владеют общей информацией, но они также склонны специализироваться в нескольких конкретных областях.

До найма специалиста по безопасности определите, какие навыки нужны вашей организации. Если фирма небольшая — например, вы представляете стартап или проект с открытым исходным кодом — специалист-универсал удовлетворит большинство ваших потребностей. По мере роста и развития организации ее проблемы с безопасностью могут становиться сложнее и требовать отдельной специализации. В табл. 20.1 есть некоторые ключевые этапы в ранней истории Google, требовавшие соответствующей экспертизы безопасности.

Таблица 20.1. Необходимые знания о безопасности на основных этапах в истории Google

Вехи в истории компании	Необходимые экспертные знания	Проблемы безопасности
Поисковая система Google (1998). <i>Поисковая система Google дает пользователям возможность находить общедоступную информацию</i>	Основные	Защита данных журнала поисковых запросов. Защита от атак типа отказа в обслуживании. Безопасность сети и систем
Google AdWords (2000) <i>Google AdWords (Google Ads) позволяет рекламодателям показывать объявления в поисковой системе Google и других продуктах</i>	Основные. Безопасность данных. Безопасность сети. Безопасность систем. Безопасность приложений. Соблюдение требований и аудит. Защита от мошенничества. Конфиденциальность. Отказ в обслуживании Инсайдерский риск	Защита финансовых данных. Соответствие нормативным требованиям. Сложные веб-приложения. Идентичность. Злоупотребление учетными записями. Мошенничество и злоупотребление инсайдеров

Продолжение ➤

Таблица 20.1 (продолжение)

Вехи в истории компании	Необходимые экспертные знания	Проблемы безопасности
Blogger (2003) <i>Blogger — это платформа, благодаря которой пользователи могут публиковать свои веб-страницы</i>	Основные. Безопасность данных. Безопасность сети. Безопасность систем. Безопасность приложений. Злоупотребление контентом. Отказ в обслуживании	Отказ в обслуживании. Злоупотребление платформой. Сложные веб-приложения
Google Mail (Gmail) (2004) <i>Gmail — это бесплатный почтовый веб-сервис Google, расширенные функции которого доступны после покупки учетной записи GSuite</i>	Основные. Приватность. Безопасность данных. Безопасность сети. Безопасность систем. Безопасность приложений. Криптография. Антиспам. Антизлоупотребление. Реагирование на инциденты. Инсайдерский риск. Корпоративная безопасность	Защита высокочувствительного пользовательского контента в состоянии покоя и при транспортировке. Модели угроз с участием высококвалифицированных внешних злоумышленников. Сложные веб-приложения. Системы идентификации. Злоупотребление аккаунтом. Спам по электронной почте и злоупотребления. Отказ в обслуживании. Злоупотребление инсайдерскими полномочиями. Корпоративные потребности

Сертификаты и образование

Некоторые эксперты по безопасности хотят получить сертификаты в своей предметной области. Отраслевые сертификаты, ориентированные на безопасность, предлагаются учреждениями по всему миру и могут быть хорошими показателями заинтересованности кого-либо в развитии соответствующих навыков для своей карьеры и способности изучать ключевые концепции. Для получения такого сертификата обычно нужно пройти стандартизированный тест. Некоторые требуют минимального обучения, участия в конференциях или опыта работы. Почти все истекают через определенное время или подразумевают для сертифицированных лиц обновления минимальных требований.

Но результаты такого стандартизированного тестирования не обязательно говорят о чьей-то склонности к успеху как специалиста по безопасности в вашей организации. Поэтому мы рекомендуем придерживаться сбалансированного подхода к оценке специалистов, учитывая все их квалификации в целом: практический опыт, сертификаты и личный интерес. Хотя сертификаты могут говорить о чьей-либо способности сдавать экзамены, мы встречали квалифицированных специалистов, которым было трудно применять свои знания на практике для решения проблем. Начинающие же кандидаты или переходящие на работу с других должностей могут использовать сертификаты для быстрого повышения уровня своих знаний.

Если у таких кандидатов есть большой интерес к этой области или практический опыт работы с проектами с открытым исходным кодом, они на ранних этапах карьеры могут быстро повысить свою ценность.

Эксперты по безопасности становятся все востребованнее, многие отрасли и университеты разрабатывают и развивают ориентированные на безопасность академические программы. Некоторые учреждения предлагают обучение в области безопасности, которое охватывает многие вопросы безопасности. Одни программы сосредоточены на определенной области безопасности, а другие предлагают смешанную учебную программу, которая фокусируется на частичном пересечении вопросов кибербезопасности и таких областей, как государственная политика, законодательство и приватность. Как и в случае с сертификациями, мы советуем рассматривать академические достижения кандидата вместе с его практическим опытом и потребностями вашей организации.

Возможно, вы захотите сначала нанять опытного специалиста по безопасности, а после подключить талантливых молодых сотрудников, как только команда будет сформирована и сможет предложить наставничество. Если ваша организация работает над нишевой технической проблемой (например, обеспечение безопасности автомобилей с автоматическим управлением), на эту роль может хорошо подойти новый аспирант, обладающий глубокими знаниями в этой конкретной области исследований, но небольшим опытом работы.

Интеграция безопасности в организацию

Понимание того, когда начинать работать над безопасностью, — больше искусство, чем наука. Есть много разных мнений на эту тему. Однако можно с уверенностью сказать, что чем раньше вы начнете думать о безопасности, тем лучше для вас. Говоря более конкретно, в течение многих лет мы сталкивались

с условиями, способными стать для многих организаций (в том числе и нашей) предпосылками для начала построения программы безопасности.

- Организация начинает обрабатывать личные данные — журналы конфиденциальной активности пользователя, финансовую информацию, медицинские записи или электронную почту.
- Когда организации необходимо создать высокозащищенную среду или пользовательские технологии, такие как пользовательские функции безопасности в браузере.
- Когда нормативные акты требуют соблюдения стандарта (закон Сарбейнса — Оксли, PCI DSS или GDPR) или соответствующего аудита¹.
- У организации есть договоренности с клиентами, особенно в отношении уведомления о нарушении или минимальных стандартов безопасности.
- Произошла компрометация или утечка данных (в идеале, до нее).
- Как реакция на компрометацию со стороны партнера, работающего в той же отрасли.

В общем, лучше начать работу над безопасностью задолго до выполнения какого-либо из этих условий — особенно до утечки данных! Гораздо проще реализовать безопасность до, чем после такого события. Например, если ваша компания планирует запустить новый продукт, принимающий онлайн-платежи, вы можете обратиться к специализированному поставщику для реализации этой функциональности. Проверка поставщика и его методов обработки данных потребует времени.

Представьте, что вы начинаете работу с поставщиком, не обеспечивающим надежную интеграцию системы онлайн-платежей. Нарушение данных повлечет за собой штрафные санкции со стороны регулирующих органов, потерю доверия клиентов и снижение производительности, так как вашим инженерам придется потратить время на повторное правильное внедрение системы. Многие организации просто перестают существовать после таких инцидентов².

¹ Закон Сарбейнса — Оксли от 2002 г. (закон о реформе бухгалтерского учета и защите инвесторов) устанавливает стандарты для публичных компаний США в отношении их практики бухгалтерского учета и включает темы информационной безопасности. Стандарт безопасности данных индустрии платежных карт устанавливает минимальные рекомендации по защите информации о кредитных картах. Его должны соблюдать все, кто занимается такой обработкой платежей. Общий регламент по защите данных — правило ЕС, касающееся обработки персональных данных.

² Несколько известных примеров, когда организации прекратили свою деятельность или объявили о банкротстве после обнаружения нарушений, — случаи с компаниями Code Spaces (<https://oreil.ly/Oj9ng>) и American Medical Collection Agency (<https://oreil.ly/BcR9C>).

Теперь представьте, что ваша компания подписывает новый контракт с партнером, у которого есть дополнительные требования к обработке данных. Предположительно, ваша команда юристов может посоветовать вам выполнить эти требования до подписания контракта. Что произойдет, если вы отложите дополнительные меры защиты и это приведет к нарушению?

Связанные с этим вопросы часто возникают при рассмотрении стоимости программы безопасности и ресурсов, которые ваша компания может вложить в нее: насколько дорого это внедрение безопасности и может ли компания это себе позволить? Мы не будем здесь углубляться в эту сложную тему, но выделим два основных момента.

Во-первых, для эффективности безопасности ее нужно сбалансировать с другими требованиями вашей организации. Чтобы представить эту рекомендацию в перспективе, мы можем сделать продукты Google почти на 100 % защищенными от действий злоумышленников, отключив центры обработки данных, сети, вычислительные устройства и т. д. Это позволит достичь высокого уровня безопасности, но у Google больше не будет клиентов, и она затеряется в длинном списке обанкротившихся компаний. Доступность — основной принцип безопасности! Для разработки разумной стратегии безопасности поймите, что нужно бизнесу для работы (и в случае большинства компаний — что нужно для получения прибыли). Найдите правильный баланс между бизнес-требованиями и адекватным контролем безопасности.

Во-вторых, безопасность — это ответственность каждого. Вы можете снизить стоимость некоторых процессов безопасности, распределяя их среди команд. Например, рассмотрим компанию, у которой есть шесть продуктов. Каждый из них обслуживается отдельной командой разработчиков и защищен 20 брандмауэрами. Здесь один общий подход состоит в том, чтобы центральная команда безопасности поддерживала конфигурацию всех 120 брандмауэров. Для этого она должна владеть обширными знаниями о шести разных продуктах. Это приведет к возможным проблемам с надежностью или задержкам в системных изменениях, способным увеличить стоимость вашей программы безопасности. Другой подход: возложить на команду безопасности ответственность за эксплуатацию автоматизированной системы конфигурации, которая принимает, проверяет, утверждает и продвигает изменения брандмауэра, предложенные шестью командами, отвечающими за продукты. Так, каждая команда разработчиков может эффективно предложить незначительные изменения для проверки и масштабирования процесса конфигурации. Такие оптимизации могут сэкономить время и даже повысить надежность системы за счет раннего обнаружения ошибок без участия человека.

Безопасность — важная часть жизненного цикла организации, поэтому представители нетехнических отделов тоже должны учитывать ее на ранних этапах.

Например, советы директоров часто изучают методы обеспечения безопасности и конфиденциальности контролируемых ими организаций. Судебные иски, следующие за утечкой данных, такие как иск акционера против Yahoo! в 2017 году, особенно подогревают интерес к теме¹. При подготовке плана по обеспечению безопасности учитывайте всех возможных участников вашего процесса.

Еще важно создать процессы для поддержания постоянного понимания текущих проблем, которые необходимо решить, и их приоритетов. Если рассматривать безопасность как непрерывный процесс, она требует постоянной оценки рисков, с которыми сталкивается бизнес. Для того, чтобы со временем повторять управление защиты в глубину, вам необходимо включить оценку рисков в жизненный цикл разработки программного обеспечения и методы обеспечения безопасности. В следующем разделе обсуждаются эффективные стратегии.

Внедрение специалистов и команд по безопасности

Многие годы мы видели, как компании экспериментировали с внедрением специалистов и команд по безопасности в организационные структуры. Подходы были разные: от полного внедрения специалистов по безопасности в продуктовые команды (см. главу 19) до использования полностью централизованных команд безопасности. Центральная команда безопасности Google — гибрид обоих вариантов.

У многих компаний есть и разные механизмы подотчетности для принятия решений. Мы видели, как директора по информационной безопасности (Chief Information Security Officers, CISO) и другие отвечающие за безопасность руководители отчитывались почти перед каждым руководителем высшего звена (уровня C): генеральным (CEO) и финансовым директором (CFO), директором по информационным технологиям (CIO) или исполнительным директором (главный операционный директор, Chief operating officer, COO), главным юрисконсультom компании, техническим директором и даже начальником службы безопасности (CSO, Chief Security Officer, обычно ответственный также за физическую безопасность). Нет правильного или неправильного подхода, и выбор вашей организации будет сильно зависеть от того, что окажется эффективнее в вашей конкретной ситуации.

¹ В США руководители и советы директоров все чаще привлекаются к ответственности за безопасность в своих организациях. Юридическая школа Конкорда в Университете Пердью выпустила хорошую статью (<https://oreil.ly/w17Yn>) об этой тенденции.

В оставшейся части этого раздела описываются некоторые подробности о подходах, которые мы успешно использовали многие годы. Хотя мы представляем крупную технологическую компанию, многие из этих подходов хороши и для небольших или средних организаций. Однако мы полагаем, что предложенные подходы могут не сработать для финансовой компании или коммунального предприятия, где ответственность за безопасность могут нести разные заинтересованные стороны. Ваш опыт может отличаться от нашего.

Пример: внедрение безопасности в Google

В Google мы сначала создали центральную организацию по вопросам безопасности, работающую как равноправный партнер по разработке продуктов. Руководитель этой организации — старший руководитель в области инжиниринга (вице-президент). Это позволило создать структуру отчетности, при которой служба безопасности помогает команде разработчиков, но в то же время получает достаточную независимость для постановки вопросов и разрешения споров без конфликта интересов, который может возникнуть у других руководителей. Это похоже на то, как организация SRE в Google поддерживает отдельные цепочки отчетности от команд разработчиков продуктов¹. Таким образом мы создаем открытую и прозрачную модель взаимодействия, сосредоточенную на улучшениях. Иначе вы рискуете создать команду со следующими характеристиками.

- Невозможно поднимать серьезные проблемы, так как запуски получают высший приоритет.
- Команда рассматривается как источник запретов, которые необходимо организационно обойти с помощью бесшумных запусков.
- Работа замедляется из-за недостаточного доступа к документации или коду.

Центральная команда безопасности Google полагается на такие стандартные процессы, как системы трекинга ошибок, для взаимодействия с остальной частью организации, когда командам необходимо запросить проверку проектирования, изменение контроля доступа и т. д. Чтобы понять работу этого процесса, см. врезку «Интеллектуальная система приема данных Google» далее.

По мере роста Google стало полезно встраивать «чемпиона безопасности» в отдельные группы специалистов по разработке продуктов. Он должен быть посредником для облегчения сотрудничества между центральной командой безопасности и командой разработки продукта. Вначале эта роль идеально подходит для старших

¹ См. главу 31 в книге Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/communication-and-collaboration/>).

инженеров с хорошим статусом в организации и интересом или опытом в области безопасности. Эти инженеры также становятся техническими руководителями инициатив по безопасности продуктов. По мере усложнения продуктовых команд эта роль передается руководителю высшего звена — директору или вице-президенту. Он может принимать сложные решения (например, чередовать запуски и исправления безопасности), получать ресурсы и разрешать конфликты.

В модели с чемпионом безопасности важно установить и согласовать обязанности и процесс взаимодействия. Например, центральная команда может продолжать выполнять проверки и аудиты проектирования, устанавливать политики и стандарты безопасности в масштабах всей организации, создавать безопасные фреймворки приложений (см. главу 13) и разрабатывать общую инфраструктуру, такую как методы с наименьшими привилегиями (см. главу 5). Распределенные чемпионы безопасности — ключевые заинтересованные стороны в этих задачах. Они должны помочь решить, как эти средства контроля будут работать в их продуктовых командах. Чемпионы безопасности также управляют внедрением политик, фреймворков, инфраструктуры и методов в рамках своих продуктовых команд. Такая организационная структура требует тесного взаимодействия с помощью уставов команд, совещаний, списков рассылки, каналов чата и т. д.

Из-за большого размера Google и Alphabet, в дополнение к центральной команде безопасности и распределенным чемпионам безопасности, у нас есть специальные децентрализованные команды безопасности для более сложных продуктов. Например, команда безопасности Android входит в состав отдела проектирования. У Chrome похожая модель (см. главу 19). Это означает, что команды безопасности Android и Chrome несут ответственность за комплексную безопасность соответствующих продуктов, включая принятие решений о стандартах, фреймворках, инфраструктуре и методах, специфичных для конкретного продукта. Эти специализированные команды безопасности курируют процесс проверки безопасности продукта и имеют специальные программы для повышения надежности продуктов. Например, команда безопасности Android поработала над усилением медиастека (<https://oreil.ly/rME44>) и получила выгоду от комплексного подхода к обеспечению безопасности и инжинирингу.

Во всех этих моделях важно, чтобы команда безопасности была открытой и доступной. В дополнение к процессу проверки безопасности, в ходе которого разработчики могут получить помощь от экспертов в предметной области, инженерам необходима своевременная и последовательная обратная связь по вопросам, связанным с безопасностью, на протяжении всего жизненного цикла проекта. Культурные проблемы, связанные с этим взаимодействием, мы рассматриваем в главе 21.

ИНТЕЛЛЕКТУАЛЬНАЯ СИСТЕМА ПРИЕМА ДАННЫХ GOOGLE

Во многих организациях очередь тикетов — единственный канал связи с командой безопасности. Для экономии времени наших команд мы создали умную систему в виде интерфейса очереди тикетов, чтобы максимально автоматизировать процесс консультирования.

Вместо того чтобы предоставлять простую форму или формы ввода для нашей очереди, мы создали систему с динамическим опросником. Когда пользователь описывает свой запрос, система автоматически задает общие вопросы, связанные с безопасностью, и возвращает предупреждения и рекомендации, информируя пользователей о рискованных решениях. С помощью этих вопросов пользователи могут определить, используют ли они безопасный для памяти язык, применяют ли они проверенную систему шаблонов/фреймворк, обрабатывают ли конфиденциальные данные или модифицируют критически важную систему.

После заполнения формы система может обнаружить явную проблему и автоматически перенаправить заявку нужному пользователю или команде. Далее инженер по безопасности может быстро проанализировать внутренне структурированную информацию и соответствующие данные для помощи пользователю.

Работа по обеспечению безопасности постоянно развивается, и опрос не охватывает все варианты использования, поэтому форма приема заявки дает возможность пользователям обходить предложенные разделы и выбирать вариант «прочее», если их запрос не соотносится с определенным рабочим процессом. Чтобы опцию «прочее» не использовали по умолчанию, мы явно указываем, что она предназначена для одноразовых запросов с ограничением времени.

Одна из основных особенностей экспертной системы — ее способность развиваться и расти вместе с организацией. Если множество пользователей пропустит нашу анкету, мы узнаем, что наша экспертная система нуждается в доработке. Инженеры по безопасности периодически проверяют разделы, которые пользователи чаще всего обходят, и либо добавляют новые виды вопросов, либо изменяют наиболее сложные разделы. Цель в том, чтобы побудить пользователей сосредоточиться на основных вопросах безопасности, которые им нужно принять во внимание.

Создание этой системы помогло нам добиться следующего.

- Позволить пользователю выполнять самообслуживание и самообучение в процессе.
- Сэкономить время для конструктивного и продуктивного сотрудничества при предоставлении рекомендаций.
- Значительно повысить общую скорость понимания проблемы и предоставления рекомендаций.
- Значительно увеличить общее количество успешно закрытых тикетов за неделю.
- Улучшить качество информации в тикетах.

Специальные команды: синие и красные

Команды безопасности часто помечают цветами для обозначения их роли в обеспечении безопасности организации¹. Все команды, имеющие цветовую маркировку, работают над достижением общей цели — улучшением состояния безопасности компании.

Синие команды ответственны за оценку и укрепление ПО и инфраструктуры. Они также несут ответственность за обнаружение, сдерживание и восстановление в случае успешной атаки. Членами синих команд могут быть все сотрудники организации, которые занимаются ее защитой, включая людей, которые создают безопасные и надежные системы.

Красные команды проводят наступательные учения по безопасности: сквозные атаки, имитирующие действия реалистичных противников. Эти упражнения выявляют слабые стороны защиты организации и проверяют ее способность обнаруживать атаки и защищаться от них.

Обычно красные команды сосредоточены на следующем.

Конкретная цель

Например, красная команда может попытаться использовать данные учетной записи клиента (или, более конкретно, найти и выполнить эксфильтрацию в безопасное место данных учетной записи клиента, доступных в вашей среде). Такие упражнения очень похожи на действия противников.

Наблюдение

Его цель — определить, позволяют ли ваши методы наблюдения обнаружить разведку противника. На основе наблюдений также можно спланировать будущие действия.

Целевые атаки

Цель в том, чтобы показать возможность использования проблемных мест с недостаточной защитой, которые предположительно очень маловероятны для использования. В итоге можно определить, для каких мест следует усилить безопасность.

Перед запуском программы красных команд не забудьте получить согласие сотрудников отделов, на которые эти упражнения могут повлиять, включая юристов и руководителей. Нелишним будет и определить границы — например, красные

¹ Эта цветовая схема заимствована у военных США (<https://oreil.ly/nuNjO>).

команды не должны получать доступ к данным клиентов или нарушать работу производственных сервисов. Они должны использовать искусственные или неиспользуемые данные для имитации их кражи и перебоев в обслуживании (например, путем взлома только данных тестовых учетных записей). Эти границы должны обеспечивать баланс между проведением реалистичных упражнений и установлением сроков и масштабов, удобных для ваших команд-партнеров. Ваши противники не будут соблюдать эти границы, поэтому красные команды должны уделять дополнительное внимание областям, не имеющим достаточной защиты.

Некоторые красные команды делятся своими планами атак с синими и очень тесно сотрудничают с ними, чтобы быстро и в полной мере понимать ситуацию. Такие отношения можно даже формализовать путем создания фиолетовой объединяющей команды¹. Это полезно, если вы выполняете много упражнений и хотите быстро продвигаться или если нужно возложить работу красных команд на инженеров продукта. Такая ситуация может вдохновить красную команду на поиск мест, которые в противном случае она могла бы и не рассматривать. Инженеры, которые проектируют, внедряют и обслуживают системы, лучше всего знают систему и ее недостатки.

ОБНАРУЖЕНИЕ КРАСНЫХ КОМАНД

Если ваши красная и синяя команды решили не делиться информацией друг с другом, убедитесь, что вы установили протокол того, что делать, если синяя команда обнаруживает красную. Представьте себе такой сценарий: красная команда успешно взламывает вашу БД клиентов. Синяя видит это и запускает процедуру экстренного реагирования. В итоге о происшествии узнают руководители, юристы и регулирующие органы! Хороший протокол для деэскалации после обнаружения предотвратит такую путаницу.

Красные команды — не команды по поиску уязвимостей или тестированию на проникновение. *Команды сканирования уязвимостей* ищут предсказуемые и известные слабые места в ПО и конфигурациях, подходящих для автоматизированного сканирования. *Команды тестирования на проникновение* больше внимания уделяют поиску уязвимостей и тех, кто пытается их использовать. Сфера их действия уже. Они ориентированы на конкретный продукт, компонент инфраструктуры или процесс. Эти команды в основном тестируют средства предотвращения и некоторые инструменты обнаружения защиты безопасности организации, поэтому обычно их взаимодействие длится несколько дней.

¹ Для получения дополнительной информации о фиолетовых командах см.: *Brotherston L., Berlin A. Defensive Security Handbook: Best Practices for Securing Infrastructure*. Sebastopol, CA: O'Reilly Media, 2017.

В отличие от этого, работа красных команд направлена на достижение цели и обычно длится неделями. Их сфера деятельности — конкретные цели, такие как извлечение интеллектуальной собственности или данных клиентов. У них широкая область применения, и они используют любые средства для достижения целей (в пределах безопасности), изучая продукт, инфраструктуру и внутренние/внешние границы.

При наличии времени хорошие красные команды могут достичь своих целей без обнаружения. Вместо того чтобы рассматривать успешную атаку красной команды как оценку плохого или неэффективного бизнес-подразделения, используйте эту информацию, чтобы лучше понять некоторые из ваших более сложных систем¹. Используйте упражнения красных команд, чтобы лучше узнать, как эти системы взаимосвязаны и как они совместно используют границы доверия. Красные команды нужны для того, чтобы помогать укреплять модели угроз и создания защиты.



Атаки красной команды не отражают в точности поведение внешних злоумышленников, поэтому они — не идеальная проверка ваших возможностей обнаружения и реагирования. Это особенно верно, если в красной команде работают внутренние инженеры, знающие о системах, в которые они пытаются проникнуть.

Вы также не можете достаточно часто проводить атаки красной команды, чтобы давать своим командам обнаружения и реагирования представление о вашей уязвимости к атакам в режиме реального времени или открывать статистически значимые показатели. Красные команды созданы для обнаружения редких крайних случаев, которые нельзя выявить при обычном тестировании. Несмотря на все предостережения, регулярное проведение упражнений красной команды — это хороший способ от начала до конца понять свой уровень в области безопасности.

Красную команду можно использовать и для обучения людей, проектирующих, внедряющих и обслуживающих системы, мышлению противника. Добавьте их непосредственно в такую команду — например, для небольшого проекта. Это даст им четкое представление из первых рук о том, как злоумышленники изучают систему в поисках уязвимостей и обходят средства защиты. Позже они могут использовать эти знания в процессе разработки.

Взаимодействие с красной командой поможет вам лучше понять состояние безопасности организации и разработать план по снижению рисков. Понимая особенности вашей текущей отказоустойчивости, определите, нужны ли какие-то изменения.

¹ Сделайте это с опорой на культуру безобвинительных постмортемов, как описано в главе 15 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/postmortem-culture/>).

Внешние исследователи

Еще один способ проверить и улучшить вашу безопасность — тесно сотрудничать с внешними исследователями и энтузиастами, находящими уязвимости в системах. Как мы упоминали в главе 2, так можно получить полезные отзывы о ваших системах.

Многие компании работают с внешними исследователями, создавая *Программы вознаграждений за нахождение уязвимостей (Vulnerability Reward Programs, VRP)*, проще говоря, ведя *охоту за багами (bug bounty programs)*. В таких программах можно получить вознаграждение за нахождение уязвимостей в вашей системе — оно может быть в том числе материальным¹. В первой VRP от Google, запущенной в 2006 году, в качестве награды шла футболка, а также словесная благодарность на нашей веб-странице.

С помощью программ вознаграждений вы можете расширить поиск ошибок за пределы вашей организации и привлечь больше исследователей безопасности.

Перед запуском VRP рекомендуется сначала изучить основы поиска и устранения обычных проблем безопасности, которые находят при тщательных проверках и базовом сканировании. Иначе вы заплатите другим людям за ошибки, которые ваши команды сами в состоянии легко обнаружить. Но ведь не в этом главная цель VRP. Еще один недостаток — об одной проблеме могут сообщить сразу несколько исследователей (<https://oreil.ly/6qBkN>).

До запуска программы вознаграждения за ошибки выполните следующие основные шаги.

1. Определите готовность вашей компании к такой программе.
 - Определите целевые области системы. Например, нельзя сделать корпоративные системы целевыми.
 - Определите уровни выплат и выделите средства для них².
2. Подумайте, нужно ли запускать свою программу вознаграждения или достаточно нанять организацию, которая специализируется на этих программах.
3. Если вы сами запускаете программу, настройте процесс принятия ошибок, сортировки, исследования, валидации, контроля и исправления ошибок. Это занимает около 40 часов для каждой серьезной проблемы, исключая исправления.

¹ См. философские рассуждения в блоге исследователя sirdarckcat (<https://oreil.ly/Lgtv3>) о наградах за нахождение уязвимостей в Google.

² См. пост sirdarckcat о размере вознаграждений за найденные уязвимости (<https://oreil.ly/11Q2P>).

4. Определите процесс выполнения платежей. Помните, что респонденты могут быть по всему миру, а не только в вашей стране. Проконсультируйтесь с юристами и финансистами, чтобы понять ограничения, возможные в вашей организации.
5. Запустите, изучите и повторяйте.

Каждая VRP может столкнуться с некоторыми проблемами.

Необходимость точного регулирования потока сообщений о проблемах

В зависимости от вашей репутации в отрасли, поверхности атаки, суммы выплат и простоты поиска ошибок вы можете столкнуться с ошеломляющим количеством отчетов. Заранее определите, какой уровень реагирования потребуется от вашей организации.

Плохое качество отчетов

Программа вознаграждения за ошибки может стать обременительной, если большинство инженеров будут выявлять простые проблемы или то, что не имеет особого значения. Мы обнаружили, что это особенно верно для веб-сервисов. Многие пользователи из-за неправильной настройки браузеров «находят» ошибки, которые на самом деле ошибками не являются. Менее вероятно, что исследователи безопасности будут делать точно так же, но иногда бывает трудно сразу определить квалификацию человека, сообщившего об ошибке.

Языковые барьеры

Исследователь уязвимости не обязательно сообщит об ошибке на вашем родном языке. Здесь может пригодиться программа для перевода, или в вашей организации может быть человек, понимающий язык респондента.

Рекомендации по раскрытию уязвимостей

Правила раскрытия уязвимостей в целом не согласованы. Должен ли исследователь раскрыть то, что он знает, и если да, то когда? Сколько времени исследователь должен дать вашей организации на исправление ошибки? Какие результаты будут вознаграждены, а какие нет? Есть много разных мнений о «правильных» методах в такой ситуации — вот несколько советов.

- Команда безопасности Google написала пост в блоге (https://oreil.ly/t_FP9) об ответственном раскрытии¹.

¹ sirdarckcat также написал пост о раскрытии уязвимостей (<https://oreil.ly/iQn3z>).

- Project Zero, внутренняя команда по исследованию уязвимостей в Google, тоже разместила пост в блоге (<https://oreil.ly/oDjT->) об обновлениях политики раскрытия информации на основе данных.
- Команда безопасного программирования Университета Оулу делится полезной коллекцией публикаций о раскрытии уязвимостей (<https://oreil.ly/qILyu>).
- Международная организация по стандартизации (ISO) выпустила рекомендации (<https://oreil.ly/kRc5Z>) по раскрытию уязвимостей.

Будьте готовы своевременно решать проблемы, о которых сообщают исследователи. Также имейте в виду, что они могут обнаружить проблемы, которые активно использовались злоумышленником в течение некоторого периода времени, и в этом случае вам также может потребоваться устранить нарушение безопасности.

Резюме

Безопасность и надежность зависят от наличия и качества процессов и методов. Самые важные движущие силы этих процессов и методов — люди. Эффективные сотрудники способны взаимодействовать независимо от ролей, отделов и культурных границ. Мы живем в мире, где будущее неизвестно, а противники непредсказуемы. Ответственность всех сотрудников вашей организации за безопасность и надежность — лучшая защита!

Формирование культуры безопасности и надежности

*Хизер Адкинс, Питер Вальчев,
Феликс Гроберт, Ана Опра,
Сергей Симаков, Дуглас Колиш
и Бетси Бейер*

Представьте нового сотрудника в вашей организации. Осознает ли он важность безопасности и надежности и то, какой приоритет они занимают среди других целей организации? Знают ли сотрудники свою роль в обеспечении того, чтобы системы могли противостоять ошибкам, вызванным вредным изменением или действиями злонамеренных противников?

В этой главе описаны аспекты здоровой культуры безопасности и надежности. Мы поговорим о способах влияния на организационную культуру с опорой на лучшие практики при внесении изменений. Наконец, мы даем представление о том, как можно повлиять на руководство и команды по всей организации для их заинтересованности в обеспечении безопасности и надежности.

Здесь мы делимся некоторыми практиками, которые сочли полезными в Google и не только. Однако имейте в виду, что нет двух абсолютно одинаковых организаций — вам нужно подстроить эти методы к культуре вашей компании. Вы обнаружите, что, скорее всего, невозможно применить все описанные стратегии. Эта глава — руководство и справочный материал для постоянного рассмотрения. Нужно отметить, что в Google мы не практикуем описанные здесь рекомендации ежедневно, а только стремимся улучшить все подходы, воспринимая это как часть создания общей культуры.

Безопасность и надежность на высоком уровне, если организации осознают их важность и создают соответствующую культуру, основанную на этих фундаментальных принципах. Организации, четко разрабатывающие, внедряющие и поддерживающие культуру, которую они стремятся воплотить, добиваются успеха, превращая внедрение культуры в командную работу. Такой подход — ответственность каждого, от генерального директора и высшего руководства до

технических руководителей, менеджеров и людей, непосредственно проектирующих, внедряющих и поддерживающих системы.

Представьте такой сценарий: только на прошлой неделе генеральный директор сказал всем сотрудникам, что принятие следующей Большой Сделки может стать решающим для будущего компании. Сегодня днем вы обнаружили доказательства присутствия злоумышленника в системе и знаете, что эти системы придется перевести в автономный режим. Клиенты будут злиться, и Большая Сделка окажется под угрозой. Вы знаете и то, что именно вашу команду могут обвинить в том, что она не применяла исправления безопасности в прошлом месяце, но многие были в отпуске, и все пытаются уложиться в сжатые сроки для принятия Большой Сделки. Какие решения сотрудников в этой ситуации соответствует культуре вашей компании? Работоспособная организация с сильной культурой безопасности будет поощрять сотрудников немедленно сообщать об инцидентах, несмотря на риск откладывания Большой Сделки.

Допустим, что во время исследования действий атакующего команда разработчиков фронтенда случайно вносит серьезные изменения, предназначенные для промежуточного использования, в систему реального производства. Эта ошибка приводит к финансовым потерям компании — система более часа находится в автономном режиме, а клиенты обрывают горячую линию службы поддержки. Доверие вашей клиентской базы быстро снижается. Наличие грамотной культуры надежности побудило бы сотрудников перестроить процесс, который случайно изменил фронтенд, чтобы команды могли управлять потребностями клиентов, пусть и с риском пропустить или отложить Большую Сделку.

В этих случаях культурные нормы должны поощрять создание безобвинительных постмортемов, чтобы выявить закономерности неудач, которые можно исправить, тем самым избегая опасных ситуаций в будущем¹. Компании со здоровой культурой знают, что один взлом — это очень неприятно, а два — еще хуже. Они знают и то, что достижение показателя в 100 % никогда не будет правильной целью надежности. Использование таких инструментов, как бюджеты ошибок² и средства контроля за отправкой безопасного кода, позволит улучшить работу пользователей, если установить правильный баланс между надежностью и скоростью. Наконец, компании со здоровой культурой безопасности и надежности знают, что в долгосрочной перспективе клиенты ценят прозрачность, ведь инциденты неизбежно происходят, а их сокрытие может подорвать доверие пользователей.

¹ См. главу 15 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/postmortem-culture/>).

² Это обсуждается в главе 3 той же книги (<https://landing.google.com/sre/sre-book/chapters/embracing-risk/>).

В этой главе описываются паттерны и антипаттерны построения культуры безопасности и надежности. Мы надеемся, что эта информация будет полезна для организаций всех форм и размеров. Однако культура — особая часть организации, созданная в контексте ее задач и особенностей. У разных организаций не может быть одинаковой культуры, и не все рекомендации отсюда могут быть применимы ко всем. В этой главе содержится ряд идей по теме культуры, но не каждая организация сможет принять все обсуждаемые здесь практики. Описанные далее предложения идеализированы, они не полностью применимы в реальности. В Google мы не всегда правильно понимаем культуру и постоянно стремимся улучшить существующее положение вещей. Надеемся, что из всего разнообразия вариантов вы найдете что-то, подходящее именно вам.

Определение здоровой культуры безопасности и надежности

Подобно здоровым (то есть работоспособным) системам, здоровую командную культуру можно явно разработать, внедрить и поддерживать. В книге мы фокусировались на технических и технологических компонентах построения работоспособных систем. Существуют такие же принципы проектирования для создания здоровой культуры. На самом деле культура — основная составляющая проектирования, внедрения и поддержки безопасных и надежных систем.

Культура безопасности и надежности по умолчанию

Как мы обсуждали в главе 4, часто есть соблазн отложить рассмотрение вопросов безопасности и надежности до более поздних стадий жизненного цикла проекта. Эта отсрочка позволяет ускорить процесс выпуска, но это происходит за счет потенциального увеличения стоимости модернизации. Со временем эти изменения увеличат технический долг или разработчики введут их в неправильном порядке, что приведет к сбою. Представьте, что при покупке автомобиля вам пришлось отдельно искать продавца ремня безопасности, тестировщика безопасности лобового стекла и инспектора, проверяющего подушки безопасности. Решение проблем безопасности и надежности только *после* изготовления автомобиля ляжет тяжелым бременем на потребителя, который не всегда может правильно оценить достаточность реализованных решений.

В этом примере отражена необходимость обеспечения *безопасности и надежности систем по умолчанию*. Когда выбор в пользу безопасности и надежности делается на протяжении всего жизненного цикла проекта, легче поддерживать согласованность. Кроме того, когда эти свойства интегрированы в систему, они могут стать незаметными для потребителя. Возвращаясь к аналогии с автомобилями: потребителям не нужно задумываться о ремнях безопасности, лобовых стеклах или камерах заднего вида, чтобы быть уверенными в том, что они поступают правильно.

Организации со здоровой культурой безопасности и надежности по умолчанию призывают сотрудников обсуждать такие темы на ранних этапах жизненного цикла проекта. Например, на этапе проектирования и на протяжении каждой итерации внедрения. По мере роста продуктов их безопасность и надежность будут и в дальнейшем развиваться органически. Это заложено в жизненном цикле разработки ПО.

Благодаря такой культуре люди, проектирующие, обслуживающие и внедряющие системы, могут автоматически и прозрачно внедрять безопасность и надежность. Например, вы можете внедрить автоматизацию для непрерывных сборок, санитайзеров, обнаружения уязвимостей и тестирования. Фреймворки приложений и общие библиотеки помогут разработчикам избежать таких частых уязвимостей, как XSS и SQL-инъекция. Рекомендации по выбору подходящих языков программирования или функций его языка помогут избежать ошибок повреждения памяти. Этот тип автоматической защиты направлен на снижение помех (таких как медленный аудит кода) и ошибок (уязвимостей, не обнаруженных во время проверки) и должен быть относительно прозрачным для разработчиков. По мере повышения безопасности и надежности систем сотрудники будут все больше доверять реализациям.

Культура рецензирования

При наличии сильной культуры рецензирования всем разработчикам предлагается заранее подумать об их ролях в утверждении изменений. Экспертные проверки позволяют гарантировать, что система будет безопасна и надежна при различных сценариях изменений.

- Многосторонние проверки авторизации для доступа или изменения для соблюдения принципа наименьших привилегий (см. главу 5).
- Экспертные оценки для обеспечения уместности и высокого качества изменений в коде, включая соображения безопасности и надежности (см. главу 12).

- Экспертные оценки изменений конфигурации перед их внедрением в производственные системы (см. главу 14).

Для создания такой культуры нужно широкое понимание ценности проверок и способа их выполнения во всей организации.

Методы проверки изменений должны быть задокументированы, чтобы вы точно знали, что произойдет во время проверки. Например, для экспертных обзоров кода вы можете задокументировать методы, связанные с рецензированием в вашей организации, и поделиться ими со всеми новыми разработчиками¹.

Когда требуется экспертная оценка для схем авторизации с несколькими участниками (как описано в главе 5), документируйте условия предоставления доступа и отказа. Это позволит создать общий набор ожиданий в рамках всей организации, так что только валидные запросы на утверждение будут успешными. Аналогичным образом вы должны ожидать, что, если утверждающий отклонит запрос, причины будут понятны на основе задокументированной политики, чтобы можно было избежать обид между людьми.

С помощью документации можно обучать рецензентов, чтобы они понимали базовые ожидания от этой работы. Такое обучение лучше вводить на ранней стадии, во время адаптации к организации или проекту. Подумайте о привлечении новых рецензентов через схему обучения, при которой более опытный или квалифицированный рецензент просматривает подтвержденные учеником изменения.

Для культуры рецензирования важно участие в процессах рецензирования всех сотрудников. В то время как владельцы несут ответственность за обеспечение общего направления и стандартов в своих областях, они также должны нести индивидуальную ответственность за изменения, которые инициируют. Никто не должен отказываться от рецензирования просто потому, что он руководит или не хочет участвовать. Ответственный за кодовое дерево не освобождается от проверки своего кода и изменений конфигурации. Аналогичным образом владельцы систем не освобождаются от многосторонней авторизации для входа в систему.

¹ Методы Google по рецензированию кода задокументированы в руководстве разработчика по обзору кода (<https://oreil.ly/mnPdJ>). Дополнительную информацию о процессе и культуре рецензирования кода в Google см. в статье: *Sadowski C. et al. Modern Code Review: A Case Study at Google*. 2018. <https://oreil.ly/IffJ1>.

ОБЛЕГЧЕНИЕ РАБОТЫ

Подумайте о создании облегченных вариантов рецензирования, позволяющих ускорить внесение небольших изменений. Как мы заявляем в главе 14, проверки эффективны, только если они обязательны. Однако обзоры не должны быть обременительными. Для небольшого изменения конфигурации или запроса многосторонней авторизации для временного доступа к нечувствительному к безопасности ресурсу не нужно следовать сложным процессам. Убедитесь, что у вас есть четко установленные инструкции о том, что должно быть проверено через «тяжелый» процесс, а что через «легкий», чтобы у разработчиков и рецензентов были одинаковые ожидания.

Выбор правильного инструментария рабочего процесса также облегчит жизнь разработчикам — инструменты, позволяющие легко инициировать изменения, предоставляют разный функционал для рецензентов и имеют поддержку автоматизации для небольших изменений.

Убедитесь, что у рецензентов есть контекст, необходимый для принятия решений, и возможность отклонить или перенаправить рецензию без наличия достаточного контекста для точной оценки того, безопасно ли изменение. Это особенно важно при рассмотрении свойств безопасности и надежности запроса на доступ, таких как многосторонняя авторизация или изменения фрагментов кода с последствиями для безопасности. Если рецензент не знаком со сложностями, на которые нужно обратить внимание, то рецензия не будет служить достаточным средством контроля. Контекст может быть создан при помощи автоматических проверок. Например, в главе 13 мы обсуждали, как Google использует Tricorder (<https://oreil.ly/rdYsN>) для автоматического оповещения разработчиков и рецензентов о проблемах безопасности на этапе предварительной проверки изменений кода.

Культура осведомленности

Если сотрудники знают о своих обязанностях по обеспечению безопасности и надежности и способе их исполнения, они быстро добьются хороших результатов. Например, инженеру может потребоваться принять дополнительные меры для защиты учетной записи, так как она получает доступ к конфиденциальным системам. Кто-то, чья работа связана с частым общением с клиентами, может получать больше фишинговых писем. Руководитель может подвергаться большему риску при путешествиях в определенные страны мира. Организации со здоровой культурой повышают осведомленность о подобных вещах, например, через образовательные программы.

Стратегии информирования и обучения — ключ к формированию сильной культуры безопасности. Сделайте процесс проще и забавнее, чтобы обучающиеся были заинтересованы в содержании. Люди запоминают разные типы информации с разной скоростью, в зависимости от способа передачи этой информации, от их предварительного ознакомления с материалом и даже от личных факторов (возраст и предыдущий опыт).

По нашему опыту, многие обучающиеся лучше запоминают информацию при использовании интерактивных методов обучения, таких как практические занятия, чем при использовании пассивных, таких как просмотр видео. Планируя программу обучения, оптимизируйте процесс, тщательно продумывая, какую информацию люди должны запомнить и как ее лучше донести.

У Google несколько подходов к обучению сотрудников безопасности и надежности. В широком масштабе все наши сотрудники проходят обязательное ежегодное обучение. Далее мы подкрепляем полученные ими знания специальными программами для определенных ролей.

Вот несколько полезных тактик, которые мы вывели за годы внедрения этих программ в Google.

Интерактивные беседы

На выступлениях, где поощряется активное участие аудитории, проще донести до людей сложную информацию. Например, в Google совместный разбор основных причин и мер по устранению серьезных происшествий безопасности и надежности помог сотрудникам лучше понять, почему мы уделяем этим темам особое внимание. Мы поняли, что подобные дискуссии побуждают людей делиться теми проблемами, которые они обнаружили, от подозрительной активности на рабочих станциях до кода с ошибками, который может вывести системы из строя. Это помогает людям почувствовать себя частью команды, что делает организацию надежнее и безопаснее.

Игры

Геймификация безопасности и надежности — еще один способ повысить осведомленность. Эти методы легче масштабировать для более крупных организаций, которые, возможно, смогут лучше предоставлять игрокам гибкость при прохождении обучения и возможность пройти его повторно, если те захотят. Наша XSS-игра (<https://xss-game.appspot.com>) (рис. 21.1) привела к большим успехам в обучении разработчиков этой распространенной уязвимости веб-приложений.

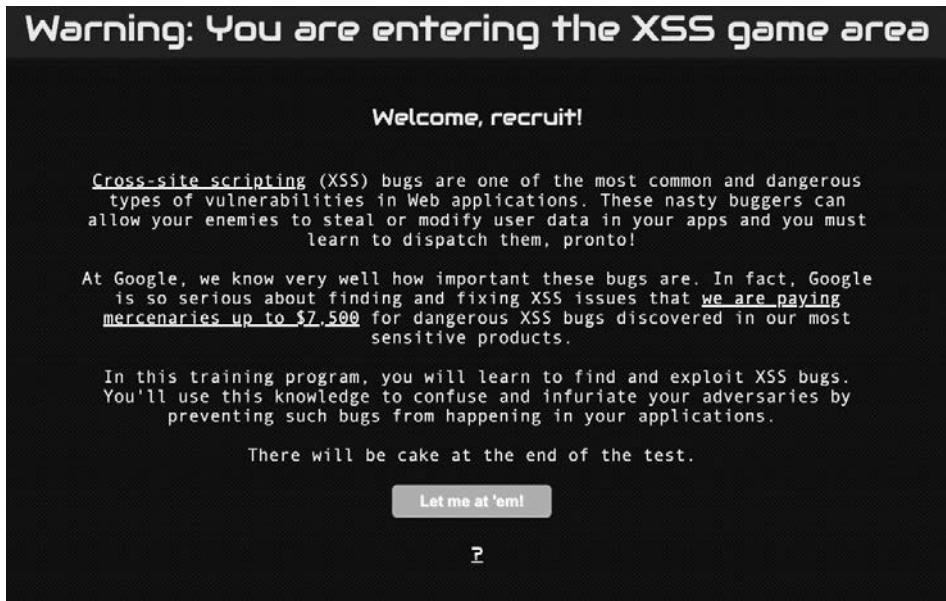


Рис. 21.1. Учебная игра по безопасности

Справочная документация

Бесспорно, знания лучше усваиваются при практической работе, а не при чтении документации. Однако мы поняли, что очень важно предоставить разработчикам надежную документацию, на которую те смогут по необходимости ссылаться. Как мы отмечаем в главе 12, справочная документация важна, потому что трудно одновременно держать в уме все нюансы безопасности и надежности. Для руководства по распространенным проблемам безопасности у Google есть рекомендации в отношении внутренней безопасности, где инженеры могут искать решения проблем при их появлении¹. Каждый документ должен поддерживаться в актуальном состоянии, для этого за каждым должен быть закреплен свой автор. Неактуальные документы нужно помечать как устаревшие.

Кампании по повышению осведомленности

Может быть трудно уведомить всех разработчиков о недавних проблемах и разработках безопасности и надежности. Для решения этой задачи Google еженедельно публикует технические рекомендации, которые помещаются на

¹ Google поддерживает набор рекомендаций для межсайтового скриптинга (<https://oreil.ly/qYpj8>).

одну страницу (пример показан на рис. 21.2). Эти выпуски «Тестирования в туалете» (https://oreil.ly/cO_P8) расклеивают в туалетах во всех офисах Google. Изначально программа была нацелена на улучшение тестирования, но она иногда затрагивает вопросы безопасности и надежности. На флаер, размещенный в заметном месте, обязательно обратят внимание, а значит, прочитают и текст¹.



Testing on the Toilet Presents... Security in the Stall

October 10, 2017

Avoid Inline Event Handlers in HTML

October is Security & Privacy Month. It includes a Security Fixit, which focuses on protecting our code from common web vulnerabilities, including XSS (cross-site scripting). Learn more about getting involved at the Security Fixit site.

A common anti-pattern in web applications is the use of inline event handlers, such as `onclick` or `onload`. You can easily introduce them in your markup, like this:

```
<body onload="prepareBunnyRace()">
  <b onclick="runBunniesRun()">Make the bunnies run!</b>
</body>
```

Why are inline event handlers bad?

- **They can lead to XSS bugs** because their contents require tricky escaping; getting it wrong can allow an attacker to inject malicious scripts into your page.
- **Their contents are not compiled** or covered by checkers such as [JS Conformance](#), making it easier to write bad or insecure code; this pattern violates the [JavaScript style guide](#).
- **They prevent your application from adopting Content Security Policy**, a new web standard to provide defense-in-depth against script injection, which requires all scripts to come from trusted sources.
- **They mix document structure and behavior**, which is strongly discouraged by the [HTML style guide](#) because it increases the cost of maintaining your markup.

Luckily, most instances of inline event handlers in your HTML are easy to refactor into safer and cleaner alternatives. The best way is to **add event handlers in JavaScript**: move the handlers to the JS file that contains the rest of your client-side code:

<pre><body> <b id="bunnyRunner"> Make the bunnies run! </body></pre>	<pre>goog.events.listen(document, 'DOMContentLoaded', () => { prepareBunnyRace(); goog.events.listen(document.getElementById('bunnyRunner'), 'click', () => { runBunniesRun(); }); });</pre>
---	--

Рис. 21.2. Эпизод «Тестирования в туалете»

¹ Недавнее исследование программы «Тестирование в туалете» в Google показало, что она повышает осведомленность разработчиков. См.: *Murphy-Hill E. et al. Do Developers Learn New Tools on the Toilet? // Proceedings of the 41st International Conference on Software Engineering, 2019. <https://oreil.ly/ZN18B>.*

Уведомления «точно в срок»

Во время важных этапов работы (таких как проверка кода или обновление ПО) вы можете напомнить людям о безопасности и надежности. Своевременные уведомления в таких ситуациях помогут принимать более взвешенные решения о рисках. В Google мы экспериментировали с показом всплывающих окон и подсказок в текущий момент — например, когда сотрудники обновляют программное обеспечение из ненадежного хранилища или пытаются загрузить конфиденциальные данные в неутвержденные облачные системы хранения. Если пользователи видят оповещения в режиме реального времени, они лишний раз задумываются и принимают более правильные решения, что позволяет избежать ошибок, когда это нужно¹. В связи с этим, как мы обсудили в главе 13, предоставление разработчикам предварительных рекомендаций по безопасности и надежности помогает улучшить их выбор при разработке кода.

Культура согласия

Со временем организации могут выработать консервативную культуру рисков, особенно если нарушения безопасности или проблемы с надежностью привели к потере дохода или другим неблагоприятным последствиям. В экстремальной форме это приведет к *культуре отказа*: склонности избегать рисков и негативных последствий, влекомых за ними. Поддерживаемая во имя безопасности или надежности культура отказа может привести организацию к застою и даже неспособности внедрять инновации. Мы обнаружили, что есть способ решить проблему. Он заключается в том, чтобы сказать «да», когда использование возможности требует некоторого сознательного риска. Чтобы принять риск, вам нужно уметь его оценивать и измерять.

В главе 8 мы описываем подход, используемый для защиты Google App Engine, платформы, предлагающей запуск стороннего непроверенного кода. В этой ситуации служба безопасности Google могла бы посчитать запуск рискованным. Ведь запуск произвольного ненадежного кода — известная угроза безопасности. Вы не знаете, например, может ли третья сторона, управляющая кодом, быть вредоносной и пытаться выйти из среды выполнения платформы

¹ Эта тема тесно связана с *подталкиванием* — методом изменения поведения путем ненавязчивого побуждения людей поступать правильно. Теория подталкивания была разработана Ричардом Талером и Кассом Санстейном, которые были удостоены Нобелевской премии по экономике за вклад в поведенческую экономику. Для получения дополнительной информации см.: Талер Р. Х., Санстейн К. Р. Nudge. Архитектура выбора. 2018.

и скомпрометировать вашу инфраструктуру. Для устранения этого риска мы решились на амбициозное сотрудничество между командами продуктов и безопасности. Это помогло создать многоуровневую защищенную систему, позволившую нам запустить продукт, который иначе рассматривался бы как слишком опасный. Такое сотрудничество со временем облегчило встраивание дополнительной безопасности в платформу, так как между командами было налажено доверие.

БАЛАНС МЕЖДУ ПОДОТЧЕТНОСТЬЮ И ПРИНЯТИЕМ РИСКОВ

Склонность говорить «нет» рискованным изменениям также может вытекать из страха человека сделать неправильный выбор. В контексте обзоров проектных решений и аудита кода это может быть особенно сложно, ведь люди, выполняющие ручные проверки, не могут обнаружить каждую проблему безопасности или надежности. Если люди думают, что являются единственной линией защиты от рискованных изменений, они могут быть менее склонны к риску.

Организации, принимающие на себя риск, защищаются от этого давления, применяя такие стратегии проектирования, как использование наименьших привилегий, повышение отказоустойчивости и тестирование. Эти подходы облегчают людям принятие решений. Если кто-то делает ошибку во время проверки, другие проверки и варианты могут предотвратить катастрофу.

Другой подход к принятию риска — использование бюджетов ошибок (<https://oreil.ly/gd3MY>), допускающих сбой до определенного предела. Как только организация достигает предопределенной максимальной границы, команды должны начать сотрудничать для снижения риска до нормального уровня. В бюджете ошибок сохраняется акцент на безопасность и надежность на протяжении всего жизненного цикла продукта, поэтому люди могут вносить определенное количество рискованных изменений.

Культура неизбежности

Идеальных систем нет, и любая может в итоге выйти из строя. В какой-то момент ваша организация может столкнуться с перебоями в обслуживании или инцидентом безопасности. Принятие этой неизбежности поможет командам настроиться на подходящее мышление для создания безопасных и надежных систем и реагирования на сбой¹. В Google мы предполагаем, что сбой может

¹ Дейв Ренсин, руководитель службы обеспечения надежности клиентов в Google, подробнее рассматривает эту тему в своем выступлении *Less Risk Through Greater Humanity* (<https://oreil.ly/ZrWkS>).

произойти в любое время. Не потому, что мы сделали что-то не так, а потому что мы знаем, что реальные системы никогда не могут быть на 100 % безопасными и надежными.

В главе 16 обсуждается необходимость подготовки к неизбежному. Команды, которые придерживаются культуры неизбежности, готовятся к чрезвычайным происшествиям, чтобы в случае чего эффективно реагировать. Они открыто говорят о возможном сбое и выделяют время для имитации аварийных сценариев. Навыки реагирования на инциденты эффективны только при регулярном их использовании. Поэтому как можно чаще выполняйте специальные упражнения, атаки красных команд, практические тесты восстановления и играйте в ролевые игры (https://oreil.ly/hyDi_) для проверки и усовершенствования процессов в организации. Организации, принимающие неизбежное, тоже изучают любые происходящие сбои, в том числе в организациях, подобных им. Они используют безобвинительные постмортемы (обсуждаются в главе 18 этой книги и в главе 15 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/postmortem-culture/>)), чтобы уменьшить страх неудачи и укрепить уверенность в том, что повторение событий маловероятно. Они также изучают опубликованные другими организациями отчеты о последствиях, как в своей отрасли, так и за ее пределами. Эти отчеты дают более широкое понимание сценариев отказов, которые могут иметь отношение к организации.

ИСПОЛЬЗОВАНИЕ СВЯЗАННЫХ ТЕМАТИЧЕСКИХ ИССЛЕДОВАНИЙ

В 2003 году НАСА организовало Совет по расследованию стихийных бедствий в Колумбии. Его целью было выяснение основных причин инцидента с космическим шаттлом «Колумбия»¹. Итоговый отчет — полезный пример того, как организация может изучить причины неспособности применения серьезных изменений в целях формирования культуры безопасности и надежности. В частности, в главе 7 отчета отмечается, что организационная структура и нормы НАСА препятствовали культуре безопасности.

Этот общедоступный реальный пример показывает грамотное использование постсобытийных отчетов, которые мы советуем применять как безобвинительные постмортемы. Такие отчеты могут дать руководству четкое представление о том, как проблемы культуры могут привести к проблемам надежности и безопасности.

¹ Окончательный отчет Колумбийского совета по расследованию стихийных бедствий хранится на веб-сайте НАСА (<https://oreil.ly/ew-BA>). Глава 7 посвящена культуре безопасности в НАСА и ее влиянию на катастрофу. Мы обнаружили, что результаты отчета можно перенести на организационную культуру и других технических организаций.

Культура рационального использования

Для обеспечения свойств надежности и безопасности системы в долгосрочной перспективе ваша организация должна постоянно предпринимать усилия по их совершенствованию и выделять достаточно ресурсов (персонал и время) для выполнения этой задачи. Рациональное использование подразумевает создание средств для обработки перебоев, инцидентов безопасности и других чрезвычайных ситуаций на постоянной основе с использованием четко определенных процессов.

Для поддержания этих усилий команды должны быть способны сбалансировать время, затрачиваемое на оперативную работу, и проактивные инвестиции, которые окупятся в долгосрочной перспективе. Если вспомнить пример Калифорнийского департамента лесного хозяйства и противопожарной защиты из главы 17, эффективные команды разделяют тяжелую работу между многими людьми, чтобы ни на одного человека не легла чрезмерная ответственность.

Организации с *культурой рационального использования* измеряют рабочие нагрузки, необходимые для выполнения оперативной работы (например, реагирования на инциденты¹), а также инвестиции, необходимые для внесения улучшений с течением времени². Они учитывают стресс, выгорание и командный дух при планировании, добавляя достаточно ресурсов для поддержания долгосрочных усилий или откладывая работу при необходимости, путем расстановки приоритетов. В таких организациях нет нужды в героизме, так как прописаны повторяющиеся и предсказуемые действия реагирования на чрезвычайные ситуации и проводится ротация персонала, который регулярно занимается оперативной работой. Они стараются поддерживать командный дух, выявляя проблемы отдельных людей и постоянно мотивируя участников.

КУЛЬТУРА УСТОЙЧИВОЙ НАДЕЖНОСТИ И БЕЗОПАСНОСТИ В GOOGLE

В Google много часто меняющихся элементов системы и сложный ландшафт для злоумышленника. Это требует наличия больших команд преданных своему делу специалистов по SRE и оперативной безопасности. Команды работают посменно по принципу «следуй за солнцем» во избежание стресса и выгорания и укомплектовываются соответствующим образом. Они проводят только часть своего времени, выполняя оперативную работу, а после вносят улучшения в инфраструктуру. Мы выделяем специальные ресурсы на развитие долгосрочных инициатив. Все эти инициативы направлены на автоматизацию сложной

¹ См. главу 29 книги Site Reliability Engineering (<https://landing.google.com/sre/sre-book/chapters/dealing-with-interrupts/>).

² См. главу 32 той же книги (<https://landing.google.com/sre/sre-book/chapters/evolving-sre-engagement-model/>).

оперативной работы, повышение командного духа и разгрузку людей, чтобы они могли больше времени уделять решению сложных задач.

Основа усилий Google в области рационального использования — то, как мы управляем работоспособностью наших людей. Менеджеров обучают сосредотачиваться на создании и поддержании работоспособных команд. Это включает устранение негативных последствий работы. Такие принципы закодированы в нашей культуре как часть проекта Oxygen (<https://oreil.ly/9cZK1>).

Наличие культуры рационального использования — это знание того, что иногда исключительные обстоятельства могут вызывать временные отклонения от ожидаемой рабочей нагрузки, и наличие хороших процессов для обработки этих отклонений. Например, если несколько критически важных для бизнеса систем долгое время не выполняли свои SLO или серьезное нарушение безопасности приводит к чрезвычайным усилиям по реагированию, вам может понадобиться авральное усилие, чтобы вернуть ситуацию в нужное русло. В этот период команды могут полностью посвятить время оперативной работе и повышению безопасности или надежности. Хотя вам, возможно, придется отложить всю другую организационную работу, а также отклониться от лучших практик.

В работоспособных организациях такие чрезвычайные нарушения нормальной бизнес-деятельности должны быть редкими. Чтобы сохранить культуру рационального использования, придерживайтесь следующих рекомендаций.

- Работая вне привычных операций, уточните, что ситуация временная. Например, если вы требуете, чтобы все разработчики вручную контролировали вносимые изменения (вместо использования только автоматизированных систем), это может привести к излишним трудовым затратам и проблемам в долгосрочной перспективе. Дайте понять, что вы ожидаете скорого исправления ситуации, и сделайте предположение о том, когда это произойдет.
- Создайте специальную команду в режиме ожидания, осознающую риски и имеющую полномочия для быстрого принятия решений. Например, эта команда может предоставлять исключения из стандартных процедур безопасности и надежности. Это уменьшит сложности при исполнении и даст организации уверенность в том, что механизмы безопасности все еще действуют. Помечайте периоды, когда приходилось обходить или отменять лучшие практики, и обязательно учитывайте эти моменты позже.
- По завершении реагирования, когда будете составлять постмортем, обязательно проанализируйте систему поощрения, которая могла привести к чрезвычайной ситуации. Иногда такие проблемы культуры, как приоритизация запуска функций над функциями надежности или безопасности, могут увеличить технический долг. Активное решение таких проблем поможет организации поддерживать стабильную работу.

Изменение культуры с помощью хорошей практики

Может быть непросто повлиять на культуру организации, особенно если команда или проект, над которым вы работаете, уже хорошо зарекомендовали себя. Бывает, что организация хочет повысить безопасность и надежность, но обнаруживает культурные препятствия на своем пути. Контрпродуктивные культуры есть по многим причинам: подходы руководства, нехватка ресурсов и многое другое.

Очень часто необходимым для повышения безопасности и надежности изменениям сопротивляются по причине страха. Изменения могут казаться причиной хаоса, сложностей, потери производительности и контроля, а также повышенного риска. При внедрении новых средств контроля надежности и безопасности нередко переживают о сложности. Новые проверки доступа, процессы и процедуры можно посчитать вмешательством в производительность разработчика или ОС. Когда организации сталкиваются с жесткими сроками и высокими ожиданиями, будь то по собственной инициативе или под давлением руководства, страх перед этими новыми мерами контроля может усилить опасения. Однако, по нашему мнению, нельзя связывать повышение безопасности и надежности с усложнением системы. Если вы реализуете изменения с учетом определенных культурных соображений, то они действительно улучшат опыт каждого.

В этом разделе обсуждаются технические стратегии для внесения изменений, полезных даже в самых сложных культурах. Вы можете не быть генеральным директором или руководителем организации, но даже каждый разработчик, SR-инженер и специалист по безопасности может вносить изменения в своей сфере влияния. Делая осознанный выбор в отношении проектирования, внедрения и обслуживания систем, можно положительно повлиять на культуру вашей организации. При выборе определенных стратегий вы можете обнаружить, что со временем ситуация изменилась, возросло доверие и царит доброжелательность.

Советы из этого раздела основаны именно на такой цели, но культура развивается долгое время, и она сильно зависит от людей и ситуаций. Испытывая изложенные здесь стратегии в вашей организации, вы можете обнаружить, что одни завершаются с переменным успехом, а другие вообще не работают. Некоторые культуры статичны и устойчивы к изменениям. Однако наличие здоровой культуры, ориентированной на безопасность и надежность, так же важно, как и разработка, внедрение и обслуживание систем. Поэтому тот факт, что ваши усилия могут не всегда работать, не должен помешать вам попробовать внести изменения.

Согласуйте цели проекта и поощрения участников

Для установления доверия нужно тяжело работать, а потерять его можно в один миг. Чтобы люди, разрабатывающие, внедряющие и поддерживающие системы, могли работать вместе с другими профильными специалистами, им нужна общая система вознаграждений.

На техническом уровне надежность и безопасность проекта могут регулярно оцениваться с помощью таких наблюдаемых показателей, как SLO (<https://oreil.ly/m9rU1>), и в процессе моделирования угроз (см. главы 2 и 14). На уровне процессов и персонала убедитесь, что возможности продвижения по службе поощряют безопасность и надежность. В идеале отдельные сотрудники должны оцениваться в соответствии с задокументированными ожиданиями высокого уровня. Это должен быть не просто набор поставленных флажков — нужно выделять темы и цели, к изучению которых специалисты должны стремиться. Например, в требованиях к разработчикам ПО начального уровня в Google указано, что инженеры должны овладеть по крайней мере одним общим навыком, не связанным непосредственно с программированием, например, уметь добавлять мониторинг в свои сервисы или писать тесты безопасности.

Согласование целей проекта со стратегией вашей организации без согласования стимулов участников может привести к формированию недружественной культуры, в результате которой людей, ответственных за безопасность и надежность ваших продуктов, не окажется в списке тех, кто обычно получает повышение. Постарайтесь сделать так, чтобы сотрудники, которые вносят свой вклад в удовлетворение потребностей пользователей, были справедливо вознаграждены за свою работу.

Уменьшайте опасения с помощью механизмов снижения риска

Вы когда-нибудь ловили себя на том, что хотите внести существенные изменения, такие как внедрение новой функции или нового элемента управления, только для того, чтобы обнаружить, от чего организация отказывается из-за предполагаемого риска? Вы можете убедить своих коллег, сделав правильный выбор при развертывании. Многое из этого мы обсуждаем в главе 7. Однако здесь нужно особо отметить влияние культуры. Вот некоторые стратегии, которые можно попробовать.

Канареечное тестирование и поэтапное развертывание

Вы можете снизить опасения, медленно внедряя серьезные изменения через небольшие группы пользователей или системы. Таким образом радиус действия «плохого» изменения будет небольшим, если что-то пойдет не так. Подумайте и о том, чтобы пойти еще дальше и реализовать все изменения с помощью поэтапного развертывания и канареечного тестирования (см. главу 16 книги *Site Reliability Workbook* (<https://landing.google.com/sre/workbook/chapters/canarying-releases/>)). На практике у этого подхода много преимуществ. В главе 19 мы обсуждаем, как поэтапный цикл выпуска для Chrome уравнивает конкурирующие потребности в быстрых обновлениях и надежности. Со временем поэтапные выпуски Chrome укрепили его репутацию безопасного браузера. Мы также обнаружили, что, когда поэтапное внедрение становится нормой, сотрудники компании привыкают, что все изменения вносятся осторожно и старательно, а это только укрепляет уверенность в переменах и уменьшает страх.

Испытания «на собственной шкуре»

Показав пользователям, что вы не боитесь своих изменений, вы можете внушать уверенность в стабильности и производительности любого из них. *Испытания «на собственной шкуре»* — самостоятельное принятие изменения до того, как оно повлияет на других. Это очень важно, если вы затрагиваете системы и процессы, влияющие на повседневную жизнь людей. Например, если вы внедряете новый механизм с наименьшими привилегиями, такой как многофакторная авторизация, проверьте его в собственной команде, прежде чем распространять на всех сотрудников. В Google перед выпуском финальной версии нового ПО для безопасности конечных точек мы сначала тестируем его на самых требовательных пользователях (команде безопасности).

Доверенные тестировщики

Приглашайте сотрудников вашей организации для помощи в тестировании изменений на ранних этапах проекта — так вы снизите страх перед будущими изменениями. Такой подход позволит заинтересованным сторонам увидеть изменения до того, как они станут окончательными. Тогда они смогут заранее поделиться своими соображениями и опасениями. Предоставьте им прямой канал обратной связи, если что-то пойдет не так. Демонстрация готовности собирать отзывы на этапах тестирования может сократить разногласия между отделами вашей организации. Дайте тестировщикам понять, что вы доверяете их обратной связи. Используйте их отзывы, чтобы они знали, что их услышали. Не всегда можно учесть все полученные отзывы — не все из них будут толковыми или полезными — но, объясняя свои решения команде тестирования, вы дадите им понять, что их мнение важно для вас.

Временное выставление изменений как необязательных

Следствием стратегии тестирования «на собственной шкуре» и привлечения доверенных тестировщиков является то, что новый элемент управления на время становится необязательным. Это дает командам возможность вносить изменения в удобные для них сроки. Сложные изменения, такие как новые элементы управления авторизацией или платформы тестирования, имеют свои издержки; организации может потребоваться время, чтобы полностью внедрить такие изменения, и часто вам необходимо сбалансировать эти изменения с другими приоритетами. Если команды знают, что у них есть время для внесения изменений в своем темпе, они могут меньше сопротивляться им.

Постепенное увеличение строгости

Если нужно принять строгую новую политику для надежности или безопасности, подумайте, можно ли усилить строгость постепенно. Возможно, стоит сначала ввести средство контроля более низкого уровня, оказывающее меньшее влияние, до того как команды будут готовы принять более строгие элементы с более тяжелыми последствиями. Представим, что нужно добавить средства контроля с наименьшими привилегиями, требующие от сотрудников обоснования доступа к определенным данным. Пользователи, которые не обосновывают доступ таким образом, будут заблокированы. Здесь можно было начать с добавления в систему фреймворка обоснования (такого как библиотеки), но оставим обоснования конечного пользователя необязательными. Когда вы посчитаете систему производительной и безопасной, можно перейти к обоснованиям для доступа к данным без блокировки пользователей, не соответствующих установленным критериям. Вместо этого система может предоставлять подробные сообщения об ошибках, если пользователь дает неточное обоснование. Это обеспечивает цикл обратной связи для обучения пользователей и улучшения использования системы. Через некоторое время, когда метрики покажут высокий уровень успеха для правильных обоснований, вы можете сделать обязательным строгий контроль, блокирующий пользователей.

Обеспечьте подстраховку

Для повышения надежности и безопасности часто приходится удалять долгосрочные ресурсы, не отвечающие введенным новым стандартам безопасности. Представьте, что нужно изменить то, как люди в вашей организации используют привилегии root Unix (или похожие высокопривилегированные права доступа), возможно, путем внедрения новых прокси-систем (см. главу 3). Страх перед внесением таких изменений — это естественно. Что будет, если команда

внезапно потеряет доступ к критически важному ресурсу? Что, если изменение приведет к простоям?

Вы можете уменьшить страх перед изменениями, если для подстраховки добавите процедуры «аварийного доступа» (обсуждаемые в главе 5), позволяющие пользователям обходить новый строгий контроль. Однако эти аварийные процедуры лучше использовать с осторожностью и подвергать тщательному аудиту. Рассматривайте их как последнее средство, а не как удобную альтернативу. Процедуры аварийного доступа при правильном внедрении могут дать опасующимся командам уверенность в том, что они смогут принять изменение или отреагировать на инцидент без полной потери контроля или производительности. Допустим, есть поэтапная процедура развертывания, требующая длительного канареечного тестирования. Она была реализована как механизм безопасности для предотвращения проблем с надежностью. Вы можете предусмотреть пути ее обхода с помощью механизма аварийного доступа, чтобы изменение кода происходило немедленно, если это абсолютно необходимо. Мы обсуждали такие ситуации в главе 14.

Повышайте производительность и удобство использования

Многие люди опасаются организационных изменений, касающихся безопасности и надежности, так как считают, что они только усложнят проект. Если люди считают новые средства контроля контрпродуктивными, они могут подумать, что их принятие негативно повлияет на организацию. Поэтому важно тщательно продумать стратегию принятия новых инициатив. Подумайте, сколько времени нужно для внесения изменений, может ли изменение замедлить производительность и превышает ли выгода затраты на внесение изменений. Мы обнаружили, что следующие методы помогают упростить работу.

Создайте прозрачную функциональность

В главах 6 и 12 мы обсуждаем освобождение разработчиков от ответственности за безопасность и надежность с помощью API, фреймворков и библиотек. Благодаря безопасному варианту по умолчанию разработчик может делать правильный выбор, не принимая на себя слишком высокую ответственность. Такой подход позволяет постепенно уменьшить сложность, потому что разработчики не только видят преимущества наличия безопасных и надежных систем, но и понимают ваше намерение упростить и облегчить инициативы. Мы обнаружили, что это может со временем укрепить доверие между командами.

Ориентируйтесь на удобство использования

Ориентация на удобство использования положительно влияет на культуру безопасности и надежности¹. Если новое средство контроля легче использовать, чем предыдущее, это может создать позитивные стимулы для изменений.

В главе 7 обсуждалось, как мы сфокусировались на удобстве использования при развертывании ключей безопасности для двухфакторной аутентификации. Пользователи обнаружили, что применять ключ безопасности для аутентификации гораздо проще, чем вводить одноразовые пароли, генерируемые аппаратными токенами.

Дополнительный бонус — повышенная безопасность этих ключей позволила нам реже менять пароли². Мы провели анализ рисков по этой теме с учетом компромиссов в удобстве использования, безопасности и возможности аудита. Мы обнаружили, что ключи безопасности делают удаленную кражу паролей неэффективной. В сочетании с мониторингом для обнаружения подозрительной компрометации³ и принудительной заменой паролей мы смогли сбалансировать безопасность и удобство использования.

Есть и другие возможности, при которых функции безопасности и надежности могут исключать устаревшие или нежелательные процессы и делать использование удобнее. Внедрение таких возможностей повысит доверие пользователей к решениям безопасности и надежности.

Введите самостоятельную регистрацию и самостоятельное разрешение

С помощью порталов саморегистрации и саморазрешения разработчики и конечные пользователи могут напрямую решать проблемы безопасности и надежности. Им не нужно прибегать к помощи центральной команды,

¹ Используемые решения для безопасности и конфиденциальности давно признаны ключом к успешному внедрению технологических средств контроля. Если вам интересны дискуссии об этом, вы можете изучить материалы конференции SOUPS (<https://oreil.ly/8bTuI>), посвященной безопасности и конфиденциальности в использовании.

² Исследования показали, что пользователи делают неправильный выбор и это угрожает их паролям. Больше информации о неблагоприятных последствиях выбора пароля пользователем см.: Zhang Y., Monroe F., Reiter M. K. The Security of Modern Password Expiration: An Algorithmic Framework and Empirical Analysis // Proceedings of the 17th ACM Conference on Computer and Communications Security, 2010. P. 176–186. <https://oreil.ly/NbfFj>. Стандарты и режимы соответствия тоже учитывают эти последствия. К примеру, NIST 800-63 (<https://oreil.ly/q2Bgw>) был обновлен таким образом, чтобы он требовал смены пароля только при подозрении на то, что он был скомпрометирован.

³ Password Alert — это расширение браузера Chrome, которое предупреждает, когда пользователь вводит свой пароль Google или GSuite на вредоносном веб-сайте.

которая может быть перегружена работой. Google использует списки разрешений и запретов, чтобы контролировать, какие приложения могут работать в системах, используемых сотрудниками. Эта технология эффективна для предотвращения запуска вредоносных программ (например, вирусов).

Недостаток в том, что, если сотрудник хочет запустить ПО, которого еще нет в списке разрешений, это разрешение нужно будет получить. Для уменьшения сложностей при запросах исключений мы разработали портал самопомощи под названием Upvote (<https://github.com/google/upvote>), благодаря которому пользователи могут получить одобрение для приемлемого ПО. Иногда мы можем автоматически определить безопасное программное обеспечение и утвердить его. Если мы не можем этого сделать, то даем пользователю возможность дожидаться одобрения от определенного числа сотрудников.

Поощряйте общение и прозрачность

При пропаганде изменений средства коммуникации могут влиять на результаты. Как мы обсуждаем в главах 7 и 19, хорошее общение — ключ к созданию заинтересованности и уверенности в успехе. Предоставление людям информации и четкого понимания процесса изменений уменьшит страх и укрепит доверие. Мы выделили следующие успешные стратегии.

Документирование решений

При внесении изменений четко документируйте, почему они происходят, как выглядит успешная работа, как откатится изменение при ухудшении условий работы и к кому нужно обратиться при возникновении проблем. Убедитесь, что вы четко даете понять причину внесения изменений, особенно если они напрямую влияют на сотрудников. Например, для каждой SLO качества в Google нужно документированное обоснование. Поскольку работа службы SRE измеряется этими SLO, важно, чтобы SR-инженеры понимали их смысл¹.

Создание каналов обратной связи

Сделайте общение двунаправленным с каналами обратной связи, с помощью которых люди могут высказывать опасения. Это может быть форма обратной связи, ссылка на систему отслеживания ошибок или даже просто адрес элек-

¹ Старшие руководители SRE периодически проводят обзоры качества в производстве, оценивая их по ряду стандартных мер и обеспечивая обратную связь и поддержку. SLO в 99,95 % может сопровождаться следующим объяснением: «Раньше мы хотели достичь успеха с вероятностью 99,99 %, но на практике это оказалось нереально. Мы не выявили негативного влияния на производительность разработчиков при SLO на уровне 99,95 %».

тронной почты. Как мы упоминаем в дискуссии о доверенных тестировщиках (см. подраздел «Избавление от страха с помощью механизмов снижения риска» выше в этой главе), прямое участие партнеров и заинтересованных сторон в изменениях может уменьшить их опасения.

Использование информационных панелей

Если вы вносите сложные изменения, затрагивающие несколько команд или частей инфраструктуры, используйте информационные панели, с помощью которых четко покажите, чего вы ожидаете от людей, и предоставьте обратную связь о том, насколько хорошо они справляются. Информационные панели также полезны для отображения общей картины внедрения и поддержания сотрудников организации в курсе прогресса.

Частое обновление статуса

Если изменение очень длительное (некоторые изменения в Google занимали несколько лет), назначьте кого-то для документирования регулярных (например, ежемесячных) обновлений состояния и описания прогресса для заинтересованных сторон. Это укрепит уверенность, особенно среди руководства, в том, что работа над проектом движется и что кто-то внимательно следит за состоянием программы.

Развивайте эмпатию

Вы не сможете понять человека, пока
не пройдете несколько миль в его ботинках.

Неизвестный автор

Люди начинают понимать проблемы, с которыми сталкивается кто-то другой, когда понимают все тонкости его роли. Эмпатия между командами очень важна, когда речь заходит о свойствах надежности и безопасности системы, так как (как описано в главе 20) эти обязанности должны быть распределены по всей организации. Формирование эмпатии и взаимопонимания поможет уменьшить страх перед необходимыми изменениями.

В главе 19 мы упомянули о нескольких методах создания эмпатии между командами. В частности, о том, как команды могут распределять обязанности по написанию, отладке и исправлению кода. Точно так же команда безопасности Chrome выполняет исправления не только для повышения безопасности продукта, но и для сплочения коллектива. В идеале команды последовательно разделяют обязанности с самого начала.

Слежение за работой или обмен ею — это еще один подход к формированию эмпатии, не требующий постоянных нисходящих организационных изменений. Длительность мероприятий может меняться от нескольких часов (обычно менее формальные) до нескольких месяцев (могут потребовать привлечения руководства). Приглашая других поработать в вашей команде, вы даете понять, что готовы идти вразрез с организационными структурами и выработать общее понимание. Программа обмена безопасности SRE от Google позволяет SR-инженерам в течение недели следить за работой другого SR-инженера или инженера по безопасности. В конце обмена SR-инженер пишет отчет с рекомендациями по улучшению как для своей, так и для принимающей команды. Если программа проводится в одном офисе, то она требует очень низких затрат, но это дает множество преимуществ в плане обмена знаниями во всей организации.

Программа Google Mission Control (<https://oreil.ly/MSIrf>) призывает людей присоединяться к организации SRE на полгода, в течение которых они учатся думать как SR-инженер и реагировать на чрезвычайные ситуации. При этом они видят результаты влияния изменений ПО, инициированных в партнерских организациях. Параллельная программа, известная как Hacker Camp, призывает на шесть месяцев присоединиться к команде безопасности, чтобы поработать над проверками безопасности и мерами реагирования.

Подобные программы могут начинаться с небольших экспериментов с одним или двумя инженерами и в случае успеха постепенно развиваться. Мы поняли, что благодаря таким перестановкам можно повысить эмпатию, а также нередко генерировать новые захватывающие идеи по решению проблем.

Убедите руководство

Если вы работаете в крупной организации, может быть непросто получить поддержку для внесения важных изменений в надежность и безопасность. Поскольку многие организации заинтересованы в том, чтобы тратить свои ограниченные ресурсы на усилия, приносящие доход, или на продвижение миссии, может быть сложно получить поддержку для улучшений, которые, как считается, происходят за кулисами.

В этом разделе рассматриваются стратегии, которые мы использовали в Google или видели в другом месте, для получения поддержки руководства в отношении изменений безопасности и надежности. Как и с другими советами в этой главе, ваш опыт может отличаться. Одни из этих стратегий будут эффективными, а другие — нет. Уникальна не только культура каждой организации, но и каждый

лидер и руководящая команда. Повторим наш совет: то, что вы считаете, что одна из этих стратегий не сработает, не обязательно означает, что вам не следует ее попробовать. Результаты могут вас удивить.

Поймите, как принимаются решения

Допустим, вы хотите внести крупные изменения в пользовательскую инфраструктуру веб-сервиса организации, включив защиту от DDoS; например, ссылаясь на преимущества из главы 10. Вы знаете, что это сильно повысит надежность и безопасность системы, но для этого нужно подключить несколько команд, чтобы добавить новые библиотеки или реструктурировать код. На интеграцию и тестирование этого изменения могут уйти месяцы. С учетом высокой стоимости, но положительного влияния кто в вашей организации может решить двигаться вперед и как он примет это решение? Знание ответов на эти вопросы — ключ к пониманию того, как повлиять на руководство.

Здесь под словом *«руководство»* в широком смысле мы понимаем принимающих решения людей, независимо от того, относятся ли эти решения к заданию направления, распределению ресурсов или разрешению конфликтов. Это авторитетные и ответственные люди. Те, на кого вам нужно повлиять, а значит, вам нужно выяснить, кто в организации подходит под эти критерии. Если вы работаете в крупной компании, это могут быть вице-президенты или другие высокопоставленные лица в руководстве. В небольших фирмах, вроде стартапов и некоммерческих организаций, главным лицом, принимающим решения часто считают генерального директора (CEO). В проекте с открытым исходным кодом это может быть основатель проекта или топовый участник.

Ответ на вопрос «Кто принимает решение об этих изменениях?» бывает сложно найти. Полномочия принятия решений действительно могут быть у кого-то на вершине иерархии управления или у того, кто играет очевидную роль защитника, например юриста или специалиста по рискам. Однако в зависимости от типа предлагаемых изменений решение может принимать технический лидер или вы сами. Подтверждение может понадобиться не от одного человека. Это может быть группа заинтересованных людей из разных отделов организации, таких как юридический, инженерный, отдел по связям с общественностью и отдел развития продукта.

Иногда решение может принимать группа людей или все сообщество. Например, в главе 7 мы описываем, как команда Chrome участвовала в расширении использования HTTPS в Интернете. В этой ситуации решение о смене направления

принималось внутри сообщества и потребовало достижения консенсуса по всей отрасли.

Определение того, кто несет ответственность за изменения, может потребовать некоторого анализа, особенно если вы новичок в организации или в ней нет процессов, указывающих, как это сделать. Однако вы не можете пропустить этот шаг. Как только вы выясните, чье мнение решающее, попытайтесь понять, с каким давлением и требованиями они сталкиваются. Давление может быть связано с их руководством, советами директоров или акционерами и может быть в форме внешних воздействий, таких как ожидания клиентов. Важно понимать это, чтобы вы могли определить, где вписываются предлагаемые вами изменения. Вернемся к нашему более раннему примеру добавления защиты от DDoS центральной инфраструктуры веб-сервиса — как эти изменения будут соотноситься с приоритетами руководства?

Обоснуйте причину для изменения

Как мы уже упоминали в этой главе, сопротивление переменам может быть вызвано страхом или беспокойством по поводу увеличения сложности. Однако часто оно может быть вызвано непониманием причины изменений. В условиях множества приоритетов перед лицами, принимающими решения, и заинтересованными сторонами стоит трудная задача выбора между разными целями, которых они хотели бы достичь. Как они узнают о ценности ваших изменений? Обосновывая изменения, важно понимать, с какими трудностями сталкиваются лица, принимающие решения. Вот некоторые шаги для успешного обоснования причины¹.

Соберите данные

Вы знаете, что нужно внести изменения. Как вы пришли к такому выводу? Важно иметь данные, подтверждающие необходимость предлагаемых вами изменений. Например, если вы знаете, что встраивание автоматизированных тестовых фреймворков в процесс сборки экономит время разработчиков, можете ли вы показать, сколько именно времени вы выиграете? Если вы выступаете за непрерывные сборки, так как это стимулирует разработчиков исправлять ошибки, можете ли вы показать, как они экономят время в процессе выпуска? Проведите исследования и изучите пользователей, создавая развернутые отчеты с графиками, диаграммами и собирая анекдотические истории на основе их опыта. Далее обобщите это таким образом,

¹ Практический пример того, как мы успешно обосновали причину для изменения, см. в пункте «Пример: увеличение использования HTTPS» в главе 7.

чтобы лица, принимающие решения, могли все обдумать. Например, если вы хотите сократить время вашей команды на исправление уязвимостей в системе безопасности или решение проблем с настройкой надежности, рассмотрите возможность создания панели мониторинга, отслеживающей прогресс для каждой команды разработчиков. Демонстрация этих панелей руководителям соответствующих подразделений может мотивировать отдельные команды на достижение целей. Помните об усилиях, которые вам необходимо будет предпринять, чтобы собрать высококачественные, релевантные данные.

Обучайте других

Проблемы безопасности и надежности может быть трудно понять, если вы не сталкиваетесь с ними каждый день. По возможности проводите беседы и информационные сессии. В Google мы используем постмортемы красных команд (см. главу 20) для информирования руководителей высшего звена о видах рисков, с которыми мы сталкиваемся. Хотя красные команды не создавались ради образовательных целей, их работу можно использовать, чтобы повысить осведомленность на всех уровнях компании.

Объедините мотивацию

Используя собранные данные и свои знания о давлении, с которым сталкиваются руководители, вы сможете решить другие проблемы в сфере их влияния. В нашем предыдущем примере с DDoS внесение предлагаемых изменений в структуру обеспечило бы преимущества безопасности, но более надежный веб-сайт также мог бы увеличить продажи. Это может быть веским аргументом для руководства компании. В качестве примера из реальной жизни в главе 19 обсуждается, как быстрые выпуски Chrome быстрее доставляют исправления пользователям, имея при этом дополнительное преимущество — быстрое развертывание для решения проблем надежности и новых функций. Это отлично подходит для пользователей и заинтересованных сторон команды разработки продукта. Не забудьте обсудить, как вы уменьшаете сложности, сопровождающие изменения. Как упоминалось в этой главе, развертывание ключей безопасности Google позволило исключить непопулярные политики смены паролей и уменьшить сложности для конечного пользователя при двухфакторной аутентификации — весомые аргументы в пользу изменения.

Найдите союзников

Скорее всего, не только вы осведомлены о полезности предлагаемых изменений. Найдите союзников и попросите о поддержке для большей

весомости ваших аргументов. Это будет особенно полезно, если эти люди организационно близки к принимающим решения лицам. Такие союзники могут проверить ваши предположения об изменениях. Возможно, они знают о других аргументах, основанных на данных, или у них другой взгляд на организацию.

Следите за тенденциями в отрасли

Если вы принимаете изменения, которые уже были приняты другими организациями, используйте их опыт, чтобы убедить свое руководство. Проведите исследование — статьи, книги, публичные выступления на конференциях и другие материалы могут показать, как и почему организация приняла изменения. Там могут указываться дополнительные сведения, которые вы можете использовать при обосновании причины для изменения. Для обсуждения с руководством конкретных тем и тенденций в отрасли вы можете даже привлечь опытных докладчиков.

Измените дух времени

Если вы сможете со временем изменить отношение людей к вашей проблеме, потом вам будет легче убедить лиц, принимающих решения. Это особенно актуально, когда для изменений нужна поддержка широкой аудитории. Мы кратко обсудили эту динамику в тематическом исследовании HTTPS в главе 7. Там команда Chrome и другие представители отрасли долгое время меняли поведение разработчиков, пока HTTPS не стал протоколом по умолчанию.

Расставляйте приоритеты

Если ваша организация часто сталкивается с проблемами надежности и безопасности, люди могут воспротивиться дополнительным изменениям. Не распыляйтесь по пустякам: расставляйте приоритеты в инициативах, у которых есть шанс на успех, и знайте, когда следует прекратить отстаивать безнадежные дела. Это покажет лидерам и лицам, принимающим решения, что вы решаете самые важные проблемы.

Проигранные дела — предложения, которые нужно отложить — тоже ценны. Даже когда вы не можете успешно выступить за перемены, наличие данных, подтверждающих ваши идеи, и союзников, поддерживающих ваш план, а также информирование людей об этой проблеме может оказаться ценным. В какой-то момент ваша организация окажется готова решить ранее изученную вами проблему. Если у вас уже будет план, ожидающий своего часа, команды смогут продвигаться быстрее.

Эскалация и решение проблем

Несмотря на все усилия, в какой-то момент вопрос принятия изменений в области безопасности или надежности может встать ребром. Возможно, серьезный сбой или нарушение безопасности покажет, что вам нужно быстро найти дополнительные ресурсы и персонал. Или у двух команд разные мнения насчет способа решения проблемы, и привычный метод принятия решений не подходит. Эскалация — то, что происходит, когда команда не может справиться с проблемой (например, устранить инцидент) самостоятельно и должна привлечь других, профильных специалистов. При решении проблем эскалации мы рекомендуем следующее.

- Сформируйте группу из коллег, наставников, технических руководителей или менеджеров для предоставления информации о ситуации с обеих сторон. Лучше, чтобы ситуацию оценил кто-то с непредвзятым взглядом до принятия решения об эскалации.
- Пусть члены группы подытожат ситуацию и предложат варианты решения для руководства. Сделайте это резюме как можно короче. Строго придерживайтесь фактов и добавляйте ссылки на любые подтверждающие проблему данные, разговоры, ошибки, проектные решения и т. д.
- Передайте резюме руководству вашей команды для дальнейшего согласования возможных решений. Возможно, вы захотите объединить эскалации или подчеркнуть другие аспекты сложившейся ситуации.
- Запланируйте рабочее совещание, чтобы обрисовать ситуацию представителям всех затронутых цепочек управления, и назначьте сотрудников, принимающих решения в каждой цепочке. Руководство должно принять официальное решение или встретиться отдельно для обсуждения проблемы.

Иногда проблемы с безопасностью в Google нужно эскалировать, когда между командой разработчиков продукта и специалистом по безопасности возникает неразрешимое разногласие по поводу дальнейших действий. В таком случае эскалация инициируется командой безопасности. На этом этапе два руководителя высшего звена в организации договариваются о компромиссе или решают реализовать один из вариантов, предложенных командой безопасности или командой разработчиков продукта.

Резюме

Подобно тому, как вы проектируете системы и управляете ими, вы можете постепенно разрабатывать, внедрять и поддерживать культуру организации для обеспечения безопасности и надежности. Усилия по обеспечению надежности

и безопасности должны рассматриваться так же тщательно, как и инженерные цели.

Повышение безопасности и надежности может вызвать страх или беспокойство по поводу увеличения сложности. Есть стратегии для преодоления этих опасений и оказания помощи людям, на которых повлияют изменения. Убедитесь в соответствии целей видению заинтересованных сторон и руководства. Сосредоточьтесь на удобстве использования и помощи пользователям. Это может мотивировать сотрудников на изменения.

Как мы говорили в начале этой главы, нет одинаковых культур, и вам нужно будет адаптировать намеченные нами стратегии для вашей организации. Поступая таким образом, вы также обнаружите, что, скорее всего, не сможете реализовать все эти стратегии. Возможно, стоит выбрать области, в которых ваша организация больше всего нуждается, и со временем улучшать их. Именно таким образом Google стремится к постоянному улучшению в долгосрочной перспективе.

Заключение

Мы написали эту книгу, так как считаем, что технологии и методы, обеспечивающие защиту систем и поддержание их надежности, существенно пересекаются и организации должны интегрировать концепции безопасности и надежности в проектирование, внедрение и обслуживание систем на протяжении всего процесса проектирования, внедрения и обслуживания систем. Хотя традиционно они рассматривались как отдельные дисциплины, мы считаем, что безопасность и надежность являются неотъемлемыми свойствами системы. И за них несут ответственность все участники жизненного цикла проекта. Многие технологические изменения идут полным ходом, и мы уверены, что это вдохновит организации на принятие новых точек зрения.

Эта технологическая революция — некоторые считают ее четвертой промышленной — меняет привычный нам мир. Сдвиг ощущается не только потребителями на примере более сложных продуктов, но и разработчиками, производящими их. Организации все больше полагаются на технологии, даже если это не является основой их бизнеса. Например, мы видим системы, позволяющие хирургам проводить операции пациентам, находящимся на противоположной стороне земного шара. Ученые используют автономные летательные аппараты для обследования археологических объектов, изучения последствий эрозии почвы и защиты исчезающих видов. Появляются роботы для выполнения опасных работ в космосе и на местах ядерных катастроф.

Расширение взаимосвязанности технологий означает, что мы все больше зависим от надежности этих решений. Мы предоставляем наши данные третьим лицам, поэтому должны быть уверены в том, что это безопасно. Доверие людей к нам основано на надежности и безопасности технологий, на основе которых построена наша инфраструктура. Это справедливо для организаций любого размера: от проекта с открытым исходным кодом из трех человек, на который полагаются тысячи других проектов, до крупной многонациональной корпорации, продающей продукт миллионам пользователей.

Сложность современных систем и скорость их разработки означают, что для максимальной эффективности безопасность и надежность должны быть интегрированы с самого начала. Рассматривать безопасность и надежность как неотъемлемые свойства системы не только естественно, но и очень важно в современном автоматизированном, взаимосвязанном и сложном технологическом пространстве.

Нет ничего удивительного в том, что в более широком сообществе DevOps и DevSecOps являются инициаторами разговоров об устойчивости систем. Однако понятию интегрированной модели безопасности и надежности нужно время, чтобы дорасти и стать естественной частью экосистемы.

При написании этой книги мы собрали команды со всего Google — от разработчиков до SR-инженеров и инженеров по безопасности. В Google мы делаем безопасность и надежность частью процесса разработки продукта и побуждаем людей с разным опытом и навыками работать вместе и прислушиваться к мнению других. Люди важны так же, как и системы, поэтому мы рекомендуем вдумчиво инвестировать в то, как вы создаете свои команды и структурируете их обязанности. Люди должны согласовывать общие требования до обсуждения технических решений, во время которого может быть трудно прийти к согласию. Не недооценивайте инвестиции в укрепление доверия и обеспечение того, чтобы все говорили на одном языке.

Увлеченным безопасностью и надежностью мы дадим следующий совет: ваша способность работать в разных областях знаний и внедрять экспертные знания в нужных местах — ключ к успеху организации. Безопасность и надежность должны быть неотъемлемой частью всей вычислительной среды. Все части должны работать вместе в гармонии для решения проблем. Никакой чек-лист или «серебряная пуля», которую мы могли бы предложить, не компенсируют вашу способность помочь организации развиваться и расти по мере изменения проблем безопасности и надежности, с которыми она сталкивается.

Мы подошли к концу этой книги, но ожидаем, что она станет началом многих важных дискуссий. Надеемся, что вы тоже присоединитесь к этой дискуссии, участвуя в сообществах с другими профессионалами и делясь своими историями. В этом диалоге мы призываем вас уважать разные точки зрения, собравшие людей разных ролей за одним столом, поддерживать поиск решений и делиться своими находками. Мы уверены, что это обсуждение поможет всем нам в общих усилиях по созданию безопасных и надежных систем.

Матрица оценки риска бедствий

Для тщательного анализа рисков бедствий мы советуем ранжировать риски, с которыми сталкивается организация, с помощью стандартизированной матрицы, учитывающей вероятность возникновения каждого риска и его возможное влияние на организацию. Таблица П.1 — образец матрицы оценки рисков, которую как крупные, так и небольшие организации могут применить к своим системам.

Чтобы использовать матрицу, оцените значения, подходящие для каждого из столбцов вероятности и воздействия. Как мы подчеркивали в главе 16, эти значения зависят от деятельности вашей организации, ее инфраструктуры и месторасположения. Для организации, работающей в Лос-Анджелесе, штат Калифорния, США, более высока вероятность возникновения землетрясения, чем для организации из Гамбурга в Германии. Если у вашей компании есть офисы во многих точках мира, вы можете провести оценку рисков для каждого из них.

После вычисления значений вероятности и воздействия перемножьте их, чтобы определить рейтинг каждого риска. Полученные значения можно использовать для упорядочения рисков от высшего к низшему. Это служит ориентиром для определения приоритетов и подготовки. Для риска с рейтингом 0,8 понадобится более быстрое реагирование, чем для риска со значением 0,5 или 0,3. Обязательно разработайте планы ответных действий для наиболее критических рисков, с которыми сталкивается ваша организация.

Таблица П.1.1. Пример матрицы оценки риска бедствий

Предмет беспокой- ства	Риск	Вероятность возникнове- ния в течение года	Влияние на организацию в случае возникновения события	Ранжирование	Названия подвержен- ных риску систем
Окружающая среда		Почти никогда: 0,0 Очень маловероятно: 0,2 Маловероятно: 0,4 Вероятно: 0,6 Очень вероятно: 0,8 Неизбежно: 1,0	Незначительное: 0,0 Минимальное: 0,2 Умеренное: 0,5 Сильное: 0,8 Критическое: 1,0	Вероятность × влияние	
	Землетрясение				
	Наводнение				
	Пожар				
	Ураган				
Надежность инфраструк- туры	Отключение электроэнергии				
	Потеря подключения к Интернету				
	Отключение системы аутентифи- кации				
	Высокая задержка системы/ замедление инфраструктуры				

Предмет беспокой- ства	Риск	Вероятность возникнове- ния в течение года	Влияние на организацию в случае возникновения события	Ранжирование	Названия подвержен- ных риску систем
Безопасность	Компрометация системы				
	Инсайдерская кража интеллекту- альной собственности				
	DDoS/DoS-атака				
	Злоупотребление системными ресурсами, например майнинг криптовалюты				
	Умышленная порча веб-сайтов				
	Фишинговая атака				
	Ошибка безопасности ПО				
	Ошибка безопасности оборудования				
	Возникающая серьезная уязвимость. Например, Meltdown/Spectre, Heartbleed				

Об авторах

Хизер Адкинс — ветеран Google с 18-летним стажем и один из основателей команды безопасности Google. Будучи старшим директором по информационной безопасности, она создала глобальную команду, отвечающую за обеспечение безопасности и защиту сетей, систем и приложений Google. У нее большой опыт в системном и сетевом администрировании с акцентом на практическую безопасность. Хизер работала над созданием и обеспечением безопасности некоторых крупнейших в мире инфраструктур. Сейчас она основное внимание уделяет защите компьютерной инфраструктуры Google и работе с индустрией для решения некоторых из самых серьезных проблем безопасности.

Бетси Бейер — технический писатель Google в Нью-Йорке, специализируется на обеспечении надежности информационных систем. Она соавтор книг «Site Reliability Engineering. Надежность и безотказность как в Google» и «Site Reliability Workbook. Практическое применение». На пути к своей нынешней карьере Бетси изучала международные отношения и английскую литературу, а также получила степень в Стэнфордском и Тулейнском университетах.

Пол Бланкиншип руководит командой технических авторов в отделе безопасности и конфиденциальности Google. Помогал разрабатывать внутреннюю политику безопасности и конфиденциальности Google. Он не только технический писатель, но и музыкант.

Петр Левандовски является старшим штатным инженером по надежности сайтов. Последние девять лет он занимался улучшением безопасности инфраструктуры Google. Как технический руководитель по безопасности производственной среды, Петр отвечает за гармоничное сотрудничество команд, ответственных за надежность и безопасность. На своей предыдущей должности он возглавлял команду, ответственную за надежность критически важной инфраструктуры безопасности Google. До прихода в Google Петр создал стартап, работал в CERT Polska и получил степень в области компьютерных наук в Варшавском политехническом университете.

Ана Опреа специализируется на безопасности, SRE, планировании и стратегии для технической инфраструктуры Google. До этого она занимала должности

разработчика программного обеспечения, технического консультанта и сетевого администратора.

Адам Стабблфилд — выдающийся инженер и региональный технический руководитель по безопасности в Google. За последние восемь лет он помог создать большую часть базовой инфраструктуры безопасности Google. Адам получил степень Ph. D. в области компьютерных наук в Университете Джона Хопкинса.

Иллюстрация на обложке

Животное на обложке — водяная агама (*Physignathus cocincinus*), разновидность агамовых ящериц, обитающих в некоторых частях Юго-Восточной Азии, включая Камбоджу, Лаос, Вьетнам, Таиланд и Китай. Эти ящерицы обычно обитают в теплом влажном климате вблизи постоянных источников стоячей воды — озер, болот — и в тропических лесах.

Водяные агамы вырастают до 90 сантиметров в длину, они могут быть от ярко-зеленого до темно-зеленого цвета. Основная окраска изумрудно-зеленая, более яркая у самцов. Нижняя челюсть с розовым оттенком. Под горлом у самцов есть участок оранжевого или желтого цвета. Хвост, длина которого составляет почти 70 % от длины тела, уплощен с боков, с зелеными и бурыми поперечными полосами в виде колец. Агамы пользуются своим длинным хвостом для сохранения равновесия при лазании по ветвям и как веслом при плавании, а также как хлыстом, когда отбиваются от хищников. Эти ящерицы питаются насекомыми, рыбой, птицами и мелкими грызунами, а также растительной пищей или яйцами.

Как и у многих рептилий, у водяных агам есть небольшое блестящее пятно на макушке, называемое *теменным*, или *шишковидным*, *глазом* (в народе известным как «третий глаз»). Считается, что оно помогает рептилиям регулировать температуру, реагируя на изменения в освещении. Водяные агамы еще и отличные пловцы, несмотря на то что проводят большую часть своего времени на деревьях — в испуге они падают с веток и могут оставаться под водой до 25 минут. А еще они могут бегать на двух лапах!

Международный союз охраны природы относит водяную агаму к категории уязвимых видов. Многие животные на обложках O'Reilly под угрозой исчезновения. Все они важны для мира.

Иллюстрация на обложке выполнена Карен Монтгомери на основе черно-белой гравюры из *Histoire Naturelle*.

*Хизер Адкинс, Бетси Бейер, Пол Бланкиншип,
Петр Левандовски, Ана Опра, Адам Стабблфилд*

**Безопасные и надежные системы:
лучшие практики проектирования, внедрения
и обслуживания как в Google**

*Перевел с английского С. Черников
Научные редакторы А. Белых, В. Лобанов*

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературные редакторы	<i>Т. Сажина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>Е. Павлович, Н. Терех</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».

Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,

Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 08.2024. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014,

58.11.12 — Книги печатные профессиональные, технические и научные.

Импортёр в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 26.07.24. Формат 70×100/16. Бумага офсетная. Усл. п. л. 47,730. Тираж 700. Заказ 0000.

КРОК

СОЗДАЕМ НАСТОЯЩЕЕ,
ИНТЕГРИРУЕМ БУДУЩЕЕ



croc.ru

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.

